HTML & CSS Handbook

Semantic HTML · accessibility · forms · CSS cascade · box model · flexbox · grid · responsive design · animation · modern CSS

14 parts · one place to study: DesignGurus courses + classic books + the web, combined, beginner to intermediate

what it is · problem · how it works · trade-offs · real world · interview · self-check · TL;DR

Personal study notes. Course diagrams belong to DesignGurus. Not for redistribution.

Contents

1. **Chapter 1 — How the Web Works**
   - 1.1 The web in one picture
   - 1.2 URLs — the address of everything
   - 1.3 DNS — turning names into addresses
   - 1.4 Connections: TCP, TLS and the HTTP versions
   - 1.5 HTTP basics — requests and responses
   - 1.6 "What happens when you type a URL and press Enter?"
   - 1.7 The browser rendering pipeline
   - 1.8 Browser DevTools
   - 1.9 Your first page and file structure
   - 1.10 Networking fundamentals
   - Chapter 1 TL;DR
2. **Chapter 2 — HTML Structure and Text**
   - 2.1 Elements, tags and attributes
   - 2.2 Document structure and the doctype
   - 2.3 The `<head>`: metadata that matters
   - 2.4 Headings
   - 2.5 Paragraphs, whitespace and text blocks
   - 2.6 Text-level semantics
   - 2.7 Lists
   - 2.8 Links
   - 2.9 Character references and comments
   - 2.10 Validation
   - Chapter 2 TL;DR
3. **Chapter 3 — Semantic HTML and Accessibility**
   - 3.1 Why semantics and accessibility
   - 3.2 Landmarks
   - 3.3 Headings and the outline
   - 3.4 The accessibility tree
   - 3.5 Alt text
   - 3.6 ARIA
   - 3.7 Keyboard and focus
   - 3.8 Accessible forms
   - 3.9 WCAG 2.2
   - 3.10 Testing tools
   - Chapter 3 TL;DR
4. **Chapter 4 — Images, Media and Embeds**
   - 4.1 The `<img>` element
   - 4.2 Image formats — JPEG, PNG, GIF, WebP, AVIF, SVG
   - 4.3 Responsive images — `srcset`, `sizes` and `<picture>`
   - 4.4 Loading hints — `loading`, `decoding`, `fetchpriority`
   - 4.5 `<figure>` and `<figcaption>`
   - 4.6 Audio and video — `<audio>`, `<video>`, `<source>`, `<track>`
   - 4.7 Embedding other pages — `<iframe>`
   - 4.8 SVG basics
   - 4.9 `<canvas>` — an introduction
   - 4.10 Choosing the right element
   - Chapter interview round-up
   - Chapter TL;DR
5. **Chapter 5 — Tables and Forms**
   - 5.1 Table semantics
   - 5.2 How a form works — `action`, `method`, `name`
   - 5.3 Input types
   - 5.4 Choosing and grouping — `select`, `datalist`, `textarea`, `fieldset`, `output`, `progress`, `meter`
   - 5.5 Built-in validation and the Constraint Validation API
   - 5.6 `autocomplete` — help the browser fill the form
   - 5.7 Accessible error messages
   - Chapter interview round-up
   - Chapter TL;DR
6. **Chapter 6 — CSS Fundamentals**
   - 6.1 Syntax and adding CSS to a page
   - 6.2 Selectors
   - 6.3 Pseudo-classes and pseudo-elements
   - 6.4 Specificity
   - 6.5 The cascade and origins
   - 6.6 Inheritance
   - 6.7 Units
   - 6.8 Colours
   - 6.9 Custom properties (CSS variables)
   - 6.10 How it all fits together
   - Chapter interview round-up
   - Chapter TL;DR
7. **Chapter 7 — The Box Model and Layout Basics**
   - 7.1 The box model
   - 7.2 `box-sizing`
   - 7.3 Margin collapsing
   - 7.4 `display`
   - 7.5 Normal flow and formatting contexts
   - 7.6 Positioning
   - 7.7 Overflow
   - 7.8 `z-index` and stacking contexts
   - 7.9 Floats (legacy)
   - Chapter 7 TL;DR
8. **Chapter 8 — Flexbox**
   - 8.1 Flex container and flex items
   - 8.2 The two axes, `flex-direction` and `order`
   - 8.3 `flex-grow`, `flex-shrink` and `flex-basis`
   - 8.4 The `flex` shorthand and minimum sizes
   - 8.5 Alignment
   - 8.6 Wrapping and `gap`
   - 8.7 Common patterns
   - 8.8 Pitfalls, debugging, and interview prep
   - Chapter 8 TL;DR
9. **Chapter 9 — CSS Grid**

- 9.1 Grid vocabulary
- 9.2 Defining tracks
- 9.3 `repeat()`, `auto-fill` and `auto-fit`
- 9.4 Placing items
- 9.5 The implicit grid and auto-placement
- 9.6 Alignment
- 9.7 Subgrid
- 9.8 Grid vs Flexbox
- 9.9 Patterns
- 9.10 Debugging, pitfalls and interview prep
- Chapter 9 TL;DR

10. **Chapter 10 — Responsive Design**
    - 10.1 What responsive design is
    - 10.2 The viewport meta tag and viewport units
    - 10.3 Media queries
    - 10.4 Mobile-first in practice
    - 10.5 User-preference media queries
    - 10.6 Container queries
    - 10.7 Fluid typography with `clamp()`
    - 10.8 Responsive images recap (and `image-set()`)
    - 10.9 Intrinsic layouts — responsive without breakpoints
    - 10.10 Testing responsive design
    - Chapter 10 TL;DR
11. **Chapter 11 — Typography, Colour and Visual Effects**
    - 11.1 Fonts
    - 11.2 Loading fonts well: `font-display`, preload, subsetting
    - 11.3 Text properties
    - 11.4 Colour and contrast
    - 11.5 Backgrounds and gradients
    - 11.6 Borders, radius and outlines
    - 11.7 Shadows
    - 11.8 Filters, blend modes, clipping and masks
    - 11.9 `object-fit` and `object-position`
    - Chapter 11 TL;DR
12. **Chapter 12 — Transitions, Transforms and Animation**
    - 12.1 Transitions
    - 12.2 Timing functions (easing)
    - 12.3 2D transforms
    - 12.4 3D transforms
    - 12.5 Keyframe animations
    - 12.6 The Web Animations API (WAAPI)
    - 12.7 Performance: animate `transform` and `opacity`
    - 12.8 Enter and exit animations: `@starting-style` and `allow-discrete`
    - 12.9 Reduced motion and accessibility
    - 12.10 View transitions
    - 12.11 Scroll-driven animations
    - Chapter 12 TL;DR
13. **Chapter 13 — Modern CSS and Architecture**
    - 13.1 Native CSS nesting
    - 13.2 `:is()` and `:where()`
    - 13.3 `:has()` — the relational ("parent") selector
    - 13.4 Cascade layers (`@layer`)
    - 13.5 `@scope`
    - 13.6 Logical properties
    - 13.7 Why CSS needs an architecture — and BEM
    - 13.8 Utility-first CSS and Tailwind CSS v4
    - 13.9 CSS Modules
    - 13.10 CSS-in-JS
    - 13.11 Choosing an approach
    - 13.12 Preprocessors (Sass) and PostCSS
    - 13.13 Resets and normalize
    - 13.14 Theming and dark mode with custom properties
    - Chapter 13 TL;DR
14. **Chapter 14 — Performance, SEO, Interview Prep and Cheat Sheets**
    - 14.1 The critical rendering path
    - 14.2 Render-blocking resources
    - 14.3 Core Web Vitals: LCP, INP, CLS
    - 14.4 Fixing each metric (and the other basics)
    - 14.5 SEO basics
    - 14.6 Interview questions
    - 14.7 Cheat sheets
    - 14.8 Glossary
    - Chapter 14 TL;DR

Chapter 1 — How the Web Works

Before you write a single tag, it helps to know what actually happens between "I type an address" and "pixels appear". This chapter follows one page from a server's disk to your screen: the URL that names it, the DNS lookup that finds the server, the HTTP conversation that delivers it, and the browser pipeline that turns text into a picture. It ends with your first real page and a sensible folder layout for a small site.

What you'll learn

#	Topic	You will be able to…
1.1	The web in one picture	explain client, server, browser, and the jobs of HTML, CSS and JavaScript
1.2	URLs	take a URL apart and predict how a relative link resolves
1.3	DNS	describe how a name like `example.com` becomes an IP address
1.4	Connections: TCP, TLS, HTTP versions	say what HTTPS protects and how HTTP/1.1, /2 and /3 differ
1.5	HTTP basics	read a raw request and response: methods, status codes, headers, caching
1.6	"What happens when you type a URL?"	give the classic interview answer end to end
1.7	The rendering pipeline	walk through parse → DOM/CSSOM → render tree → layout → paint → composite
1.8	Browser DevTools	use the Elements, Console, Network and Performance panels
1.9	Your first page and file structure	build, open and serve a small multi-page site
1.10	Networking fundamentals	explain OSI vs TCP/IP layers, TCP vs UDP vs QUIC, ports, IPv4/IPv6, NAT, more DNS records, firewalls, proxies and VPNs

Every example lives in `code/`. The `.mjs` files (JavaScript files written as ES modules) run with Node 24 (`node code/01_url_parts.mjs`) — **Node.js** is a program that runs JavaScript outside the browser, from the command line. The ones that need a DOM (the browser's tree of page objects, explained in §1.7) use **jsdom**, a library that imitates a browser's document in Node, and two drive real **headless Chrome** (Chrome running without a visible window, controlled by a script). Outputs below are copied from real runs.

1.1 The web in one picture

What it is

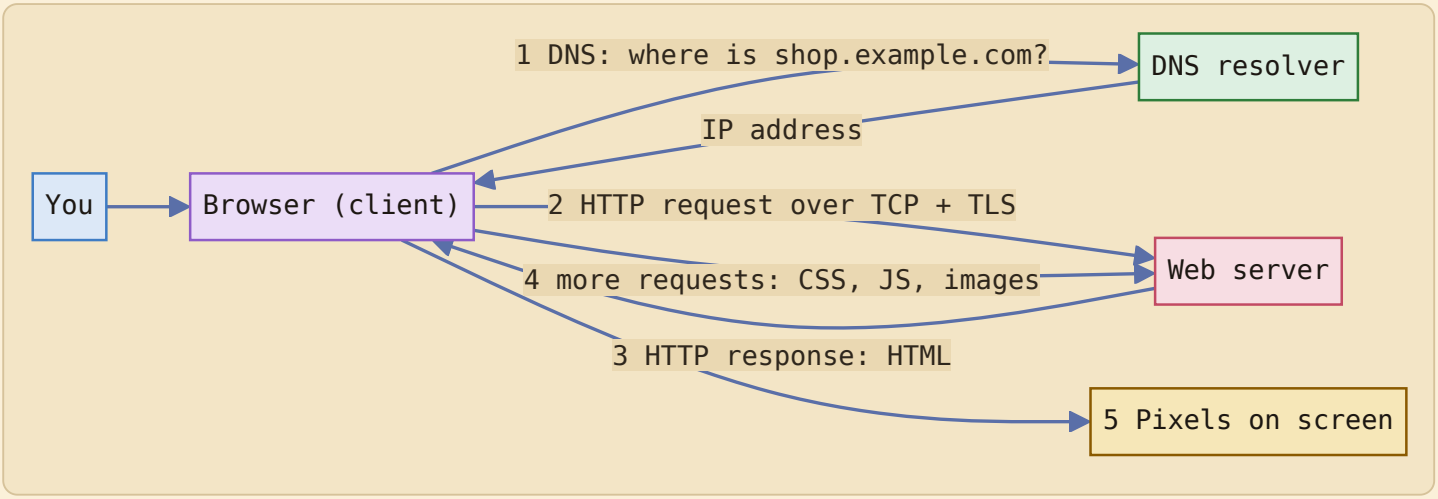
The **Internet** is the global network of connected computers and the low-level rules (IP, TCP, UDP) that move data between them. A **protocol** is simply an agreed set of rules for how two programs talk; **IP** (Internet Protocol) gives every machine an address and moves small chunks of data, called **packets**, towards it, while TCP and UDP (§1.4, §1.10) carry data between programs on those machines. The **World Wide Web** is one service running on top of it: documents and apps identified by URLs, fetched with HTTP, and written in HTML. Email, online games and SSH (Secure Shell, a way to log in to a remote computer's command line) also use the Internet but are not "the web".

Tim Berners-Lee proposed the web at CERN in 1989 and the first website went live in 1991. Three ideas from that design still hold everything up: **URLs** to name things, **HTTP** to fetch them, and **HTML** to describe them and link them together.

Analogy: the Internet is the road network; the web is one delivery company using those roads. The URL is the delivery address, HTTP is the order form, and HTML is the parcel.

The roles

Term	Meaning	Example
Client	The program that asks for something	Your browser, a phone app, `curl` (a command-line download tool)
Server	A program (and the machine running it) that answers requests	nginx (a popular web server), a Node.js app, a CDN edge node (a nearby copy server, §1.3)
User agent	The spec's (official specification's) name for the program acting for the user — usually the browser	Chrome, Firefox, Safari, a screen reader (software that reads the page aloud) + browser
Resource	Anything a URL points at	An HTML page, an image, a JSON response
Browser engine	The part of a browser that parses (reads and understands) and draws pages	Blink (Chrome, Edge, Opera), Gecko (Firefox), WebKit (Safari)
JavaScript engine	The part that runs JS	V8 (Chrome, Node.js), SpiderMonkey (Firefox), JavaScriptCore (Safari)



The three languages of the front end

Language	Its job	If it were a house
HTML (HyperText Markup Language)	Structure and meaning: "this is a heading, this is a list, this is a button"	The walls, rooms and doors
CSS (Cascading Style Sheets)	Presentation: colour, spacing, layout, animation	Paint, furniture, lighting
JavaScript	Behaviour: react to clicks, fetch data, change the page	Electricity and plumbing

Keeping them separate is not just tidiness. A screen reader, a search engine and a "reader mode" (the browser's simplified, text-only view) all use the HTML alone; a well-structured page stays usable even if the CSS fails to load or JavaScript is switched off. This idea is called **progressive enhancement**: start with working HTML, then layer CSS and JS on top.

Who defines the rules

- **HTML** and the DOM are maintained as a "living standard" (a document that is edited continuously instead of being released as numbered versions) by the **WHATWG** (Web Hypertext Application Technology Working Group) at html.spec.whatwg.org. There is no "HTML6" coming; the standard is updated continuously and "HTML5" is just a historical label.
- **CSS** is specified by the **W3C** CSS Working Group as separate modules (Flexbox, Grid, Colour…).
- **HTTP** is defined by the **IETF** in RFCs (Requests for Comments).
- **JavaScript** is standardised as **ECMAScript** by Ecma TC39 (see the JavaScript book).
- **Accessibility** (making pages usable by people with disabilities) guidelines (WCAG, the Web Content Accessibility Guidelines) come from the W3C's Web Accessibility Initiative (see Chapter 3).

Baseline is a label (shown on MDN — Mozilla's free web reference site — and on Google's web.dev) for web features that work in all major browsers: *newly available* once the last of the core browsers — Chrome, Edge, Firefox and Safari, on desktop and mobile — ships it, and *widely available* 30 months after that. This book uses it whenever browser support matters.

Self-check

1. Is email part of the web? → No. It uses the Internet but not HTTP/HTML/URLs as its core protocol.
2. Which language would you change to move a sidebar from the left to the right? → CSS.
3. What is a user agent? → The software acting on the user's behalf, normally the browser.

1.2 URLs — the address of everything

What it is

A **URL** (Uniform Resource Locator) is a string that says *how* to reach a resource and *where* it is. Every link, image, stylesheet and API call you write in HTML is a URL.

How it works — the parts

`https://user@shop.example.com:8443/books/html.html?lang=en&page=2#reviews`

Part	In the example	Meaning
Scheme	`https:`	Which protocol to use (`http`, `https`, `mailto`, `tel`, `file`, `data`…)
Userinfo	`user@`	Rare, mostly legacy; never put passwords here
Host	`shop.example.com`	Domain name (a human-readable server name) or IP address of the server
Port	`:8443`	Which "door" on the server; omitted = the scheme's default (80 for http, 443 for https)
Path	`/books/html.html`	Which resource on that server
Query	`?lang=en&page=2`	Extra `name=value` parameters, separated by `&`
Fragment	`#reviews`	A spot *inside* the page (usually an element `id`); never sent to the server

The **origin** is scheme + host + port (`https://shop.example.com:8443`). It matters a lot for security: browsers isolate pages by origin (the *same-origin policy*: a script from one origin may not read data belonging to another), which you will meet again with cookies (small pieces of data a site stores in your browser), `fetch` (JavaScript's function for making HTTP requests) and CORS (Cross-Origin Resource Sharing, the rules by which a server can allow other origins to read its responses) in the JavaScript book.

Node implements the same WHATWG URL Standard that browsers follow, so we can explore URLs in Node:

```
// code/01_url_parts.mjs
const u = new URL('https://user@shop.example.com:8443/books/html.html?lang=en&page=2#reviews');
for (const k of ['href', 'protocol', 'username', 'hostname', 'port', 'origin', 'pathname', 'search', 'hash'])
  console.log(k.padEnd(9), u[k]);
console.log('lang    =', u.searchParams.get('lang'), '| page =', u.searchParams.get('page'));

// Relative URLs are resolved against the page's own URL (the "base").
const base = 'https://shop.example.com/books/html.html';
for (const rel of ['css.html', './img/cover.png', '../about.html', '/cart', '?page=3', '#top', '//cdn.example.net/a.js'])
  console.log(rel.padEnd(24), '->', new URL(rel, base).href);

// Special characters are percent-encoded; default ports disappear.
console.log(new URL('https://example.com:443/my file.html?q=café').href);
```

Output:

```
href      https://user@shop.example.com:8443/books/html.html?lang=en&page=2#reviews
protocol  https:
username  user
hostname  shop.example.com
port      8443
origin    https://shop.example.com:8443
pathname  /books/html.html
search    ?lang=en&page=2
hash      #reviews
lang    = en | page = 2
css.html                -> https://shop.example.com/books/css.html
./img/cover.png         -> https://shop.example.com/books/img/cover.png
../about.html           -> https://shop.example.com/about.html
/cart                   -> https://shop.example.com/cart
?page=3                 -> https://shop.example.com/books/html.html?page=3
#top                    -> https://shop.example.com/books/html.html#top
//cdn.example.net/a.js  -> https://cdn.example.net/a.js
https://example.com/my%20file.html?q=caf%C3%A9
```

Absolute vs relative URLs

Kind	Looks like	Resolves to	Use for
Absolute	`https://other.site/page`	Exactly that	Links to other sites
Document-relative	`css.html`, `./img/a.png`, `../about.html`	Relative to the current page's folder	Files next to the page
Root-relative	`/cart`	Same origin, from the site root	Links inside a site served from a domain root
Scheme-relative	`//cdn.example.net/a.js`	Same scheme as the page	Legacy; just write `https://` today
Query / fragment only	`?page=3`, `#top`	Same page with a new query or jump target	Pagination, in-page links

`.` means "this folder", `..` means "the parent folder". Relative links are what make a site movable: the same files work on your laptop, a staging server (a private copy of the site used for final testing) and production (the live site real users visit).

Common mistakes

- Using Windows-style backslashes (`img\cat.png`) — URLs always use `/`.
- Root-relative links (`/css/style.css`) when opening files directly from disk: `/` then means the root of your *disk*, not your project. Use a local server (§1.9) or document-relative paths.
- Putting secrets or personal data in the query string — URLs end up in logs, history and `Referer` headers (the header in which a browser tells a server which page the visitor came from).
- Forgetting that the fragment is client-side only: the server never sees `#reviews`.

Self-check

1. What is the origin of `http://example.com/a/b?c#d`? → `http://example.com` (port 80 implied).
2. On page `https://site.com/docs/intro/start.html`, where does `../api.html` go? → `https://site.com/docs/api.html`.
3. Does changing only the fragment trigger a new HTTP request? → No, the browser just scrolls to the target.

TL;DR

A URL = scheme + host + port + path + query + fragment. Origin = scheme + host + port. Relative URLs resolve against the current page's URL.

1.3 DNS — turning names into addresses

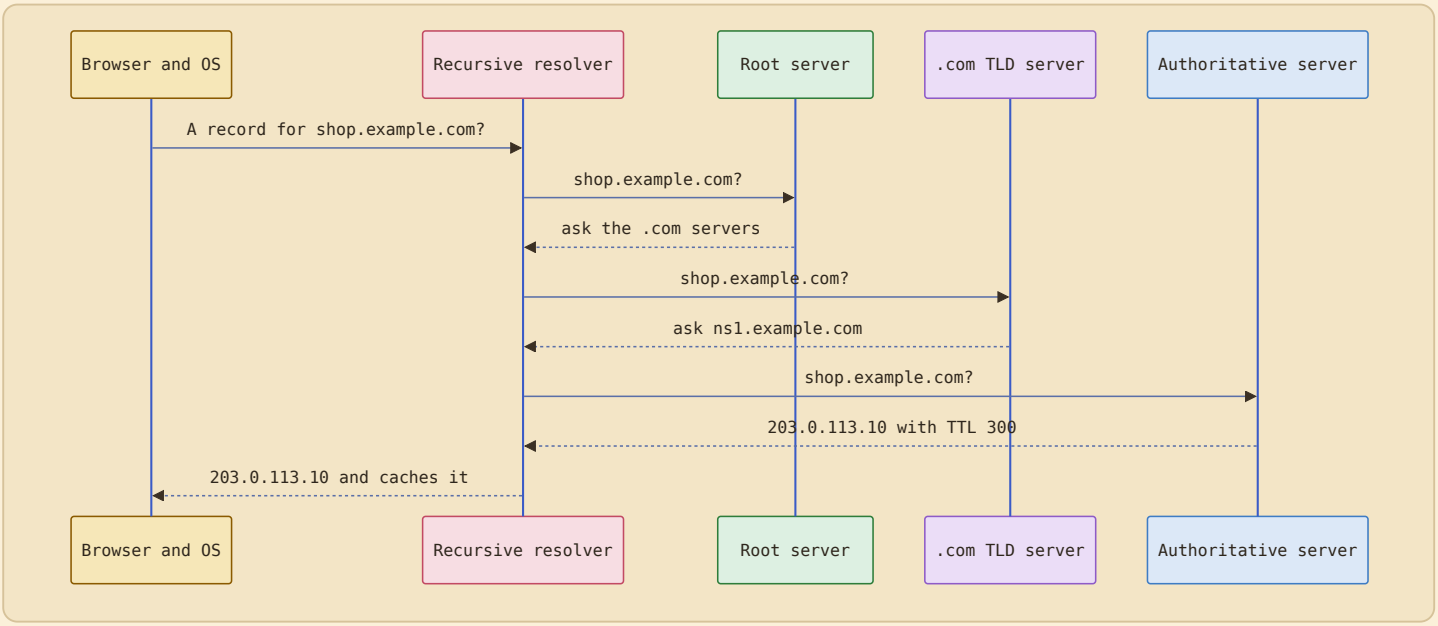
What it is

Computers route traffic by **IP address** (a numeric address that identifies a machine on a network — for example `203.0.113.10` in IPv4 or `2001:db8::10` in IPv6, the old and new address formats compared in §1.10 — both from

ranges reserved for documentation). People prefer names. The **Domain Name System (DNS)** is the Internet's distributed phone book that maps names to addresses. ("Distributed" means no single computer holds the whole book; the job is split across many servers. For the bigger picture see System Design Fundamentals.)

How it works

1. The browser checks its own **cache** (a store of recent answers kept so they need not be fetched again), then asks the operating system, which checks its cache and the local `hosts` file (`/etc/hosts` on macOS/Linux).
2. If still unknown, the OS asks a **recursive resolver** — usually run by your ISP (Internet service provider — the company that sells you your connection), company, or a public service such as 1.1.1.1 or 8.8.8.8.
3. The resolver walks the hierarchy from the right of the name: a **root server** says who runs `.com`; the **TLD (top-level domain) server** for `.com` says who is authoritative for `example.com`; the **authoritative name server** returns the actual record.
4. Every answer carries a **TTL** (time to live, in seconds). The resolver, OS and browser cache it for that long, so most lookups finish in a few milliseconds without leaving your machine or network.



Record types you will meet

Record	Maps	Example use
`A`	name → IPv4 address	`shop.example.com → 203.0.113.10`
`AAAA`	name → IPv6 address	same, for IPv6
`CNAME`	name → another name (an alias)	`www.example.com → example.netlify.app`
`MX`	domain → mail servers	where email for `@example.com` goes
`TXT`	arbitrary text	domain ownership checks, SPF/DKIM email settings (anti-spoofing rules for email)
`NS`	domain → its authoritative name servers	delegation (handing a sub-part of the name tree to other servers)

Classic DNS travels over UDP port 53, unencrypted. Newer variants, **DNS over HTTPS (DoH)** and **DNS over TLS (DoT)**, encrypt lookups; most browsers can use DoH.

Real-world examples

- When you "point a domain" at GitHub Pages, Netlify or Vercel, you add an `A` or `CNAME` record at your registrar (the company you bought the domain name from).
- A **CDN** (content delivery network) such as Cloudflare or Akamai often answers DNS with the IP of the edge server (one of many copy servers spread around the world, at the "edge" of the network close to users) nearest to you, so the same name leads to different machines in different countries (see CDN in the system-design notes).
- Changing DNS "takes time to propagate" because old answers stay cached until their TTL expires.

Self-check

1. Where does a lookup go after the browser and OS caches miss? → To the recursive resolver.
2. Which record makes `www` an alias for another hostname? → `CNAME`.
3. Why lower the TTL a day before moving a site to a new server? → So caches forget the old IP quickly after the switch.

1.4 Connections: TCP, TLS and the HTTP versions

What it is

Once the browser has an IP address it needs a reliable, private channel to talk over.

- **TCP** (Transmission Control Protocol) gives an ordered, reliable byte stream. Opening it costs one **round trip** (a message going to the server and a reply coming back) — the *three-way handshake*: SYN ("let's talk"), SYN-ACK ("OK, let's talk"), ACK ("got it").
- **TLS** (Transport Layer Security) encrypts that stream and proves the server's identity with a **certificate** (a digital ID card stating which domain a public key belongs to) signed by a certificate authority (a trusted organisation, such as Let's Encrypt, that checks you control a domain before vouching for it). TLS 1.3 (2018) needs one extra round trip for a new connection. `https://` is simply HTTP carried inside TLS.
- **HTTP** is the request/response language spoken inside the channel.

Why HTTPS matters

HTTPS gives three guarantees: **confidentiality** (others on the Wi-Fi cannot read your data), **integrity** (nobody can inject ads or malware into the page on the way) and **authentication** (you are really talking to `bank.com`). Browsers mark plain-HTTP pages "Not secure", many modern features (service workers — background scripts that can intercept requests and make a site work offline — geolocation, camera, clipboard) only work in **secure contexts** (HTTPS or `localhost`), and free certificates from Let's Encrypt removed the cost excuse.

HTTP versions compared

Version	Year (current RFC)	Transport	Key idea	Practical effect
HTTP/1.1	1997 (RFC 9112, 2022)	TCP	Text messages, persistent connections	One request at a time per connection; browsers open ~6 connections per host
HTTP/2	2015 (RFC 9113, 2022)	TCP	Binary framing, **multiplexing** (interleaving) many requests on one connection, header compression	Many small files are cheap; one lost packet still stalls every stream (TCP head-of-line blocking: everything queued behind a lost packet waits)
HTTP/3	2022 (RFC 9114)	**QUIC** over UDP	Streams handled independently; TLS 1.3 built in	Faster set-up, better on lossy mobile networks, connections survive network changes

The *meaning* of HTTP — methods, status codes, headers — is shared by all versions (RFC 9110, "HTTP Semantics"). You write the same HTML regardless; the server and browser negotiate the version.

Self-check

1. What does TLS add on top of TCP? → Encryption, integrity and server authentication.
2. Name one thing HTTP/2 fixed from HTTP/1.1. → Many requests share one connection (multiplexing).
3. What does HTTP/3 run on? → QUIC, over UDP.

1.5 HTTP basics — requests and responses

What it is

HTTP (HyperText Transfer Protocol) is a **request/response** protocol: the client sends a request, the server sends back exactly one response. It is **stateless** — each request stands alone, so anything the server must remember between requests (who is logged in, what is in your cart) is carried by cookies, tokens (signed strings that prove who you are) or URLs.

How it works — anatomy

A **request** has: a *method* + *path* + *version* on the first line, then *headers* (`Name: value`), a blank line, and an optional *body*. A **response** has: *version* + *status code* + *reason*, headers, a blank line, and the body.

The script below starts a tiny server on your own machine and talks to it with a raw TCP **socket** (one end of a network connection that a program can read from and write to), so you see the exact bytes (HTTP/1.1 is plain text; HTTP/2 and /3 carry the same information in binary frames).

```
// code/02_http_exchange.mjs (key parts)
function send(res, status, headers, body = '') {
  res.sendDate = false;                                  // no Date header -> identical output on every run
  res.writeHead(status, { ...headers, 'Content-Length': Buffer.byteLength(body) });
  res.end(body);
}
const server = http.createServer((req, res) => {
  if (req.url === '/') send(res, 200, { 'Content-Type': 'text/html; charset=utf-8', 'Cache-Control': 'max-age=60' },
                            '<!DOCTYPE html><title>Hi</title><h1>Hello</h1>');
  else if (req.url === '/old') send(res, 301, { Location: '/' });
  else send(res, 404, { 'Content-Type': 'text/plain' }, 'Not found');
});
// ... raw(path) opens a socket and writes:
//   `GET ${path} HTTP/1.1\r\nHost: localhost\r\nAccept: text/html\r\nConnection: close\r\n\r\n`
await raw('/'); await raw('/old'); await raw('/missing');
```

Output:

```
>>> REQUEST
GET / HTTP/1.1
Host: localhost
Accept: text/html
Connection: close
<<< RESPONSE
HTTP/1.1 200 OK
Content-Type: text/html; charset=utf-8
Cache-Control: max-age=60
Content-Length: 46
Connection: close

<!DOCTYPE html><title>Hi</title><h1>Hello</h1>

>>> REQUEST
GET /old HTTP/1.1
Host: localhost
Accept: text/html
Connection: close
<<< RESPONSE
HTTP/1.1 301 Moved Permanently
Location: /
Content-Length: 0
Connection: close

>>> REQUEST
GET /missing HTTP/1.1
Host: localhost
Accept: text/html
Connection: close
<<< RESPONSE
HTTP/1.1 404 Not Found
Content-Type: text/plain
Content-Length: 9
Connection: close

Not found
```

What each part shows:

- `Host` is mandatory in HTTP/1.1 — one IP address can serve many websites, and this header says which one you want.
- `Content-Type: text/html; charset=utf-8` is how the browser knows the bytes are HTML and how to decode them. The browser trusts this header, not the `.html` file extension.
- `301` plus `Location` is a redirect: a browser would immediately request `/` instead.
- The fragment (`#...`) never appears in a request line.

Methods

Method	Purpose	Safe? (no side effects)	Idempotent? (repeat = same result)	Sent by plain HTML?
`GET`	Read a resource	Yes	Yes	Links, images, forms with `method="get"`
`HEAD`	Like GET, headers only	Yes	Yes	No
`POST`	Create / submit / trigger an action	No	No	Forms with `method="post"`
`PUT`	Replace a resource	No	Yes	No (use `fetch`)
`PATCH`	Partially update	No	Not necessarily	No
`DELETE`	Remove	No	Yes	No
`OPTIONS`	Ask what is allowed (used in CORS preflight, a permission check sent before the real request)	Yes	Yes	No

HTML itself can only send `GET` and `POST` (links and forms, Chapter 5); JavaScript's `fetch` can send any method.

Status codes

Class	Meaning	Codes you should know
1xx	Informational	`103 Early Hints`
2xx	Success	`200 OK`, `201 Created`, `204 No Content`
3xx	Redirection	`301 Moved Permanently`, `302 Found`, `304 Not Modified`, `307` / `308` (redirects that keep the method)
4xx	Client error	`400 Bad Request`, `401 Unauthorized` (not logged in), `403 Forbidden` (logged in, not allowed), `404 Not Found`, `429 Too Many Requests`
5xx	Server error	`500 Internal Server Error`, `502 Bad Gateway`, `503 Service Unavailable`

Headers worth recognising

Header	Direction	What it does
`Host`	request	Which site on this server
`User-Agent`	request	Identifies the browser
`Accept`, `Accept-Language`, `Accept-Encoding`	request	Preferred formats, languages, compression (gzip, br, zstd — ways to shrink the bytes)
`Cookie` / `Set-Cookie`	request / response	Carry small pieces of state; the server sets, the browser sends back
`Content-Type`, `Content-Length`, `Content-Encoding`	both	What the body is, how big, how compressed
`Location`	response	Where a redirect points
`Cache-Control`, `ETag`, `Last-Modified`	response	Caching rules and validators (version stamps used to check a copy is still fresh)
`Content-Security-Policy`, `Strict-Transport-Security`	response	Security policies (which scripts may run; always use HTTPS)

Caching in one paragraph

Downloading the same stylesheet on every page would be slow, so responses can be cached. `Cache-Control: max-age=60` lets the browser reuse the copy for 60 seconds without asking. After that it can *revalidate*: send `If-None-Match: <etag>` and get a tiny `304 Not Modified` if nothing changed. Sites commonly put a content hash (a short fingerprint computed from the file's bytes, which changes whenever the file changes) in file names (`app.3f9a2c.css`) and cache those for a year, while the HTML itself is cached briefly or revalidated every time. More in Chapter 14 and the caching module.

Common mistakes

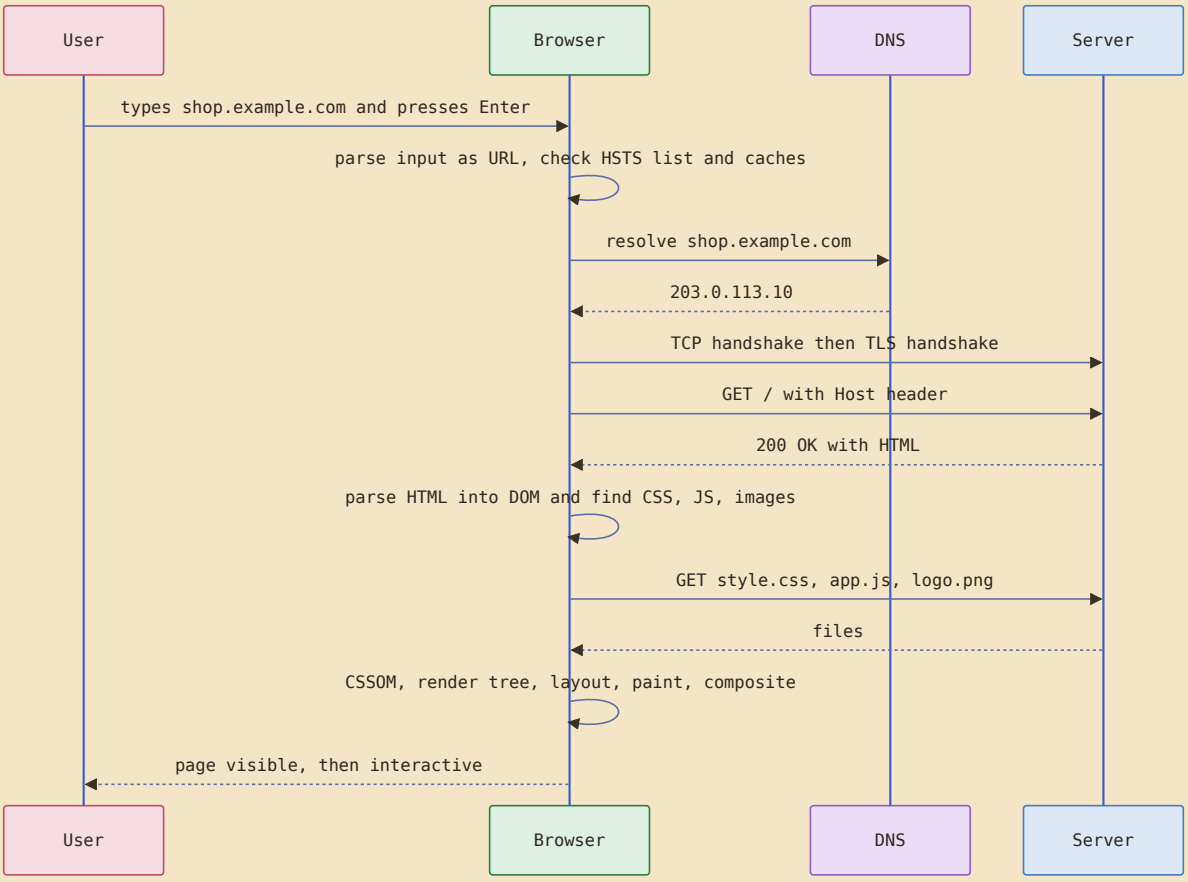
- Changing data with a `GET` link such as `<a href="/delete?id=5">`: crawlers (search-engine robots that follow every link) and link-prefetchers (features that load linked pages in advance) follow GETs.
- Serving HTML with the wrong `Content-Type` (for example `text/plain`) — the browser will show the source code.
- Confusing `401` (who are you?) with `403` (I know who you are, and no).

Self-check

1. Which header lets one IP host many sites? → `Host`.
2. What does `304` mean and when do you see it? → Not Modified; when revalidating a cached copy that is still current.
3. Why is HTTP called stateless? → The server treats every request independently; state must travel in cookies, tokens or URLs.
4. Which methods can a plain HTML form send? → `GET` and `POST`.

1.6 "What happens when you type a URL and press Enter?"

This is one of the most common web interview questions because it checks the whole chapter at once.



In the interview

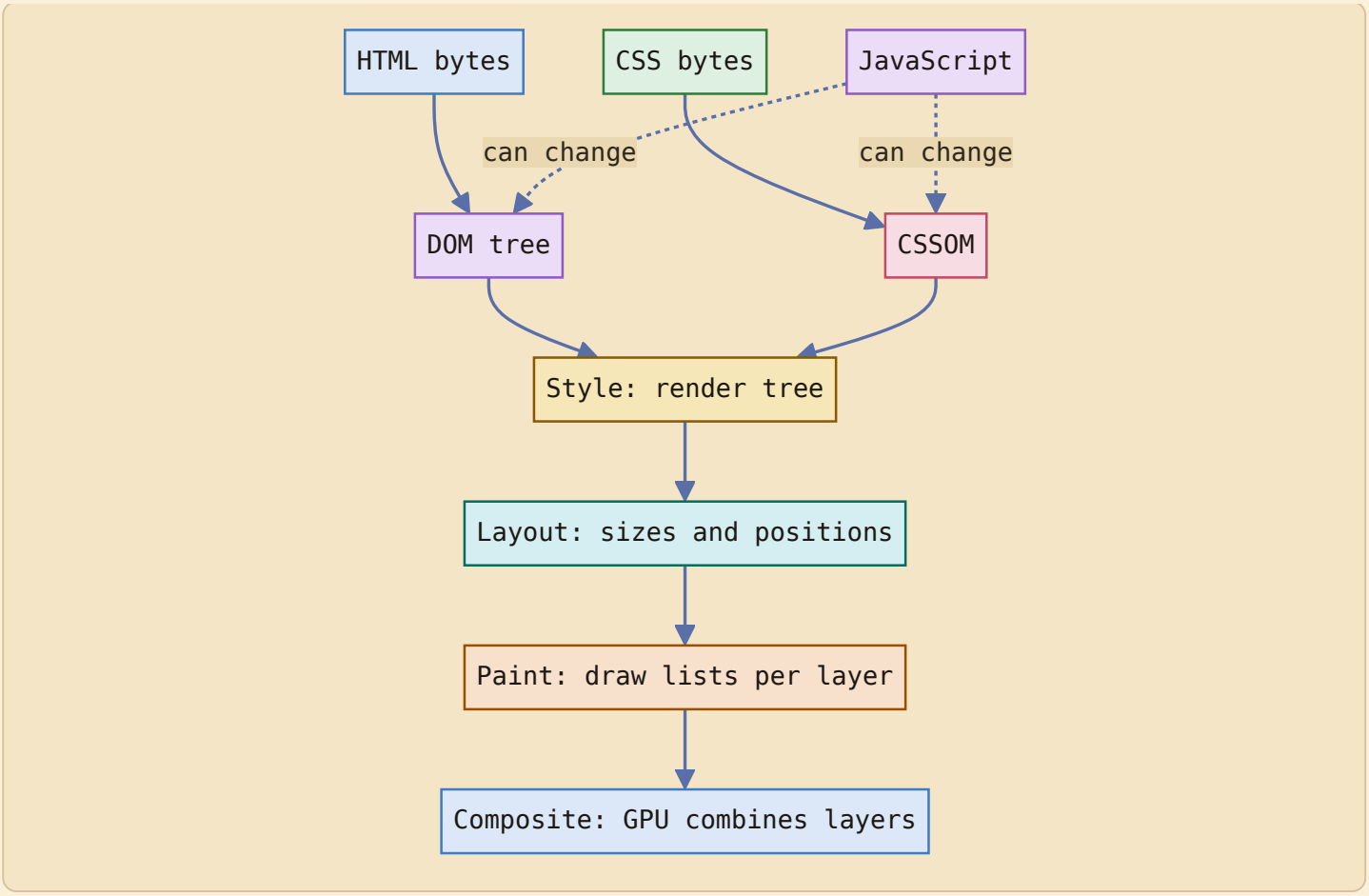
30-second answer: "The browser decides whether I typed a URL or a search, then resolves the host name to an IP via its caches and DNS. It opens a TCP connection and does a TLS handshake for HTTPS (or uses QUIC for HTTP/3), sends a GET request with headers such as Host and cookies, and gets back a response — maybe a redirect, ideally 200 with HTML. It parses the HTML into the DOM while fetching the CSS, JavaScript and images it discovers, builds the CSSOM, combines them into a render tree, computes layout, paints and composites layers to the screen. Scripts may then run and make the page interactive."

- *How could you make that faster?* → Fewer round trips (HTTP/2 or /3, CDN near users, keep-alive — reusing one open connection for many requests — and `preconnect`, a hint to open a connection early), smaller and cacheable assets, no render-blocking CSS/JS beyond what is needed (Chapter 14).
- *What is HSTS?* → `Strict-Transport-Security` tells the browser to always use HTTPS for that site, so it upgrades `http://` itself without ever sending an insecure request.
- *Where can caching short-circuit this?* → DNS caches, the browser's HTTP cache (no request at all under `max-age`), a service worker, a CDN edge cache.

1.7 The browser rendering pipeline

What it is

The **rendering pipeline** (also called the *critical rendering path* when we talk about the first view) is the sequence of steps a browser engine uses to turn HTML and CSS bytes into pixels:



Understanding it explains many rules in later chapters: why CSS goes in `<head>`, why scripts get `defer` (an attribute explained below), why animating `transform` is smooth but animating `width` is not.

Step 1 — Parsing HTML into the DOM

The bytes are decoded to characters (using the `charset`), split into **tokens** (the smallest meaningful pieces: start tags, end tags, text, comments) and fed to a **tree builder**. The result is the **DOM** (Document Object Model): a tree of objects — *nodes* — that JavaScript can read and change. HTML parsing is defined precisely by the WHATWG spec, including what to do with broken markup, so every modern browser builds the same tree from the same input. It never "fails": it recovers.

> **Key term — tree:** In programming, a *tree* is a structure where each item (a **node**) has one parent and any number of children, like a family tree or a folder hierarchy. In the DOM, `<html>` is the root; `<head>` and `<body>` are its children; a paragraph's text is a child of the `<p>` node. "Parent", "child", "sibling" (same parent) and "descendant" (anywhere below) are used throughout this book.

jsdom implements the same algorithm, so we can watch it fix a sloppy document:

```
// code/03_parse_dom.mjs (key parts)
const body = `<title>Parsing demo</title>
<p>First paragraph
<p>Second <b>bold <i>both</b> italic?</i>
<table><tr><td>cell</td></tr></table>`;
const std = new JSDOM('<!DOCTYPE html>\n' + body).window.document;
console.log('Mode:', std.compatMode, '(CSS1Compat = standards mode)');
show(std.documentElement);                       // prints the tree, one node per line
console.log('\nSerialized back:\n' + std.body.innerHTML);
const quirks = new JSDOM(body).window.document;   // same markup, no doctype
console.log('Is the <table> inside the 2nd <p>?  standards:', !!std.querySelector('p > table'),
            '| quirks:', !!quirks.querySelector('p > table'));
```

Output:

```
Mode: CSS1Compat (CSS1Compat = standards mode)
Source had no <html>, <head>, <body>, </p> or <tbody> — the parser added them:

html
  head
    title
      "Parsing demo"
  body
    p
      "First paragraph"
    p
      "Second"
      b
        "bold"
        i
          "both"
      i
        "italic?"
    table
      tbody
        tr
          td
            "cell"

Serialized back:
<p>First paragraph
</p><p>Second <b>bold <i>both</i></b><i> italic?</i>
</p><table><tbody><tr><td>cell</td></tr></tbody></table>

Without <!DOCTYPE html> -> mode: BackCompat (BackCompat = quirks mode)
Is the <table> inside the 2nd <p>?  standards: false | quirks: true
```

What happened:

1. `<html>`, `<head>` and `<body>` were created automatically; `<title>` went into the head.
2. A new `<p>` closed the previous one — paragraphs cannot nest.
3. The mis-nested `<b><i></b></i>` was repaired by closing `<i>` and reopening a fresh `<i>` after `</b>` (the spec's "adoption agency algorithm").
4. A `<tbody>` was inserted into the table. (This is why CSS like `table > tr` never matches.)
5. Without `<!DOCTYPE html>` the page is in **quirks mode**, which imitates 1990s browser bugs — here, a table does *not* close the paragraph. Always start with the doctype (Chapter 2).

Relying on error recovery is still bad practice: the fixed-up tree may not be what you meant. Validate your HTML (check it with a tool that reports rule-breaking markup, §2.10).

Scripts and parsing. A plain `<script src>` is *parser-blocking*: the parser stops, downloads and runs the script (which might call `document.write`, an old function that inserts markup into the page being parsed), then continues. Two attributes change that:

Script tag	Downloads	Runs	Order kept?	Use for
`<script src>`	blocks parsing	immediately	yes	Rarely needed today
`<script src defer>`	in parallel	after parsing finishes, before `DOMContentLoaded` (the "HTML is parsed" event)	yes	Most page scripts
`<script src async>`	in parallel	as soon as downloaded (pauses parser briefly)	no	Independent scripts, e.g. analytics (visitor-tracking code)
`<script type="module">`	in parallel	deferred by default	yes	Modern ES-module code (JS files that `import`/`export` each other)

Browsers also run a **preload scanner** that reads ahead in the raw HTML and starts downloading CSS, scripts and images early, even while the main parser is blocked.

Step 2 — Parsing CSS into the CSSOM

CSS from `<link rel="stylesheet">`, `<style>` blocks and `style=""` attributes is parsed into the **CSSOM** (CSS Object Model): the rules, their selectors (the part that says which elements a rule targets) and declarations (`property: value` pairs). CSS is **render-blocking**: the browser will not paint the page until it has the stylesheets in `<head>`, because painting unstyled content and then restyling it would cause a "flash of unstyled content" (FOUC). That is why stylesheets belong in the head and should be small.

Step 3 — Style and the render tree

The browser matches every DOM element against the CSS rules and computes its final value for every property (**computed style**; the cascade and specificity that decide this are covered in Chapter 6). Elements that produce boxes form the **render tree** (Chrome calls it the *layout tree*). What gets left out:

- elements with `display: none`, and everything inside them;
- things that never render: `<head>`, `<script>`, `<meta>`, `<link>`.

`visibility: hidden` and `opacity: 0` elements *stay* in the tree: they are invisible but still take up space.

```
// code/04_render_tree.mjs (key parts)
const html = `... <style>
  .hidden { display: none; }  .ghost { visibility: hidden; }  .faded { opacity: 0; }  p { color: #333; }
</style> ... <h1>Title</h1> <p>Normal paragraph</p> <p class="hidden">...</p>
  <p class="ghost">...</p> <p class="faded">...</p> <script>/* ... */</script> ...`;
for (const rule of document.styleSheets[0].cssRules) console.log('  ' + rule.cssText);   // the CSSOM
for (const el of document.body.children) {
  const cs = window.getComputedStyle(el);
  const inTree = cs.display !== 'none';
  const visible = inTree && cs.visibility !== 'hidden' && cs.opacity !== '0';
  // ... print one row per element
}
```

Output:

```
CSSOM rules:
  .hidden { display: none; }
  .ghost { visibility: hidden; }
  .faded { opacity: 0; }
  p { color: rgb(51, 51, 51); }

Element           display  visibility  in render tree?  painted visibly?
<h1>               block    visible     true             true
<p>                block    visible     true             true
<p .hidden>        none     visible     false            false
<p .ghost>         block    hidden      true             false
<p .faded>         block    visible     true             false
<script>           none     visible     false            false
```

Notice the CSSOM stores `#333` as `rgb(51, 51, 51)` — it holds parsed values, not your text. Here is the same idea in real Chrome (`code/render-tree.html`):

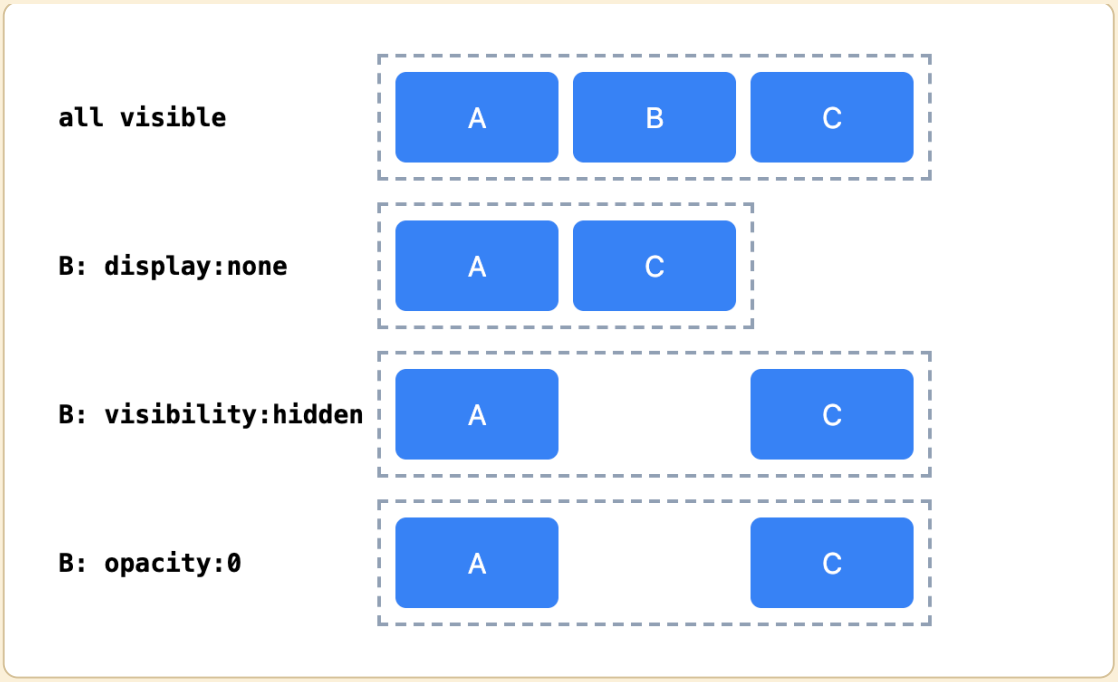


Figure 1.1 — `display: none` removes B's box entirely; `visibility: hidden` and `opacity: 0` keep its space.

Technique	Takes space?	Clickable?	Read by screen readers?
`display: none`	No	No	No
`visibility: hidden`	Yes	No	No
`opacity: 0`	Yes	**Yes**	**Yes**
`hidden` attribute	No (it applies `display: none`)	No	No

Step 4 — Layout

Layout (Firefox says *reflow*) computes the exact position and size of every box in pixels, starting from the **viewport** width (the visible area of the browser window): percentages, `em`s, flex and grid rules all become concrete numbers. Changing anything geometric later — width, font size, adding content — can force layout again for part or all of the page.

Step 5 — Paint

Paint turns each box into drawing instructions: backgrounds, borders, text, shadows, images, in the correct stacking order. The page may be split into several **layers** (for example for a fixed header, a video, or an element with `will-change: transform`), and each is painted separately. The instructions are then *rasterised* (converted from shapes and text into a grid of coloured pixels), often on the GPU (graphics processing unit — the chip specialised for drawing).

Step 6 — Composite

Compositing stacks the layers on the screen in the right order. Changes to `transform` and `opacity` can often be handled here alone — no new layout or paint — which is why they animate smoothly at 60 frames per second or more (Chapter 12).

What a change costs

You change	Layout	Paint	Composite	Example
`width`, `height`, `margin`, `top`, font size, content	yes	yes	yes	Expanding an accordion
`color`, `background-color`, `box-shadow`	no	yes	yes	Hover colour
`transform`, `opacity`	no	no	yes	Sliding a drawer

Events you can observe

- `DOMContentLoaded` — the HTML is fully parsed and deferred scripts have run (images may still be loading).

- `load` — everything, including images and iframes, has loaded.
- First Contentful Paint (when the first text or image appears), Largest Contentful Paint (when the biggest visible element appears) — performance milestones measured in DevTools and Lighthouse (Chapter 14).

Common mistakes

- Putting `<script>` tags without `defer` in `<head>` — nothing renders until they download and run.
- Hiding content with `opacity: 0` and forgetting that it is still clickable and read aloud.
- Reading a layout value (`offsetHeight`) right after changing styles in a loop — forcing layout again and again ("layout thrashing" — the browser recalculates geometry many times when once would do).

In the interview

> **30-second answer:** "The browser parses HTML into the DOM and CSS into the CSSOM, combines them into a render tree of visible boxes, runs layout to get geometry, paints each layer, and composites the layers on the GPU. CSS is render-blocking and classic scripts are parser-blocking, so I keep critical CSS small and load scripts with `defer` or as modules. For animation I prefer `transform` and `opacity` because they skip layout and paint."

- *Difference between `display:none` and `visibility:hidden`?* → The first removes the box from the render tree and frees its space; the second keeps the space but draws nothing.
- *`defer` vs `async`?* → Both download in parallel; `defer` runs in document order after parsing, `async` runs whenever it arrives, in any order.
- *What is a reflow?* → Recomputing layout because geometry changed; it is expensive on large pages.

Self-check

1. Name the six stages in order. → Parse HTML → DOM; parse CSS → CSSOM; style/render tree; layout; paint; composite.
2. Is `<head>` in the render tree? → No.
3. Why is CSS render-blocking? → To avoid painting unstyled content and immediately repainting it.
4. What does the parser do with a `<p>` inside a `<p>`? → It closes the first paragraph.

TL;DR

HTML → DOM, CSS → CSSOM, both → render tree → layout → paint → composite. CSS blocks rendering; classic scripts block parsing; `defer`/modules fix that. `transform` and `opacity` changes are the cheapest.

1.8 Browser DevTools

What it is

Every desktop browser ships **developer tools** — panels that show the DOM, styles, network traffic and performance of the page you are on. Open them with **F12**, **Ctrl+Shift+I** (Windows/Linux) or **Cmd+Option+I** (macOS), or right-click an element → *Inspect*.

The panels you will use (Chrome names; Firefox and Safari are similar)

Panel	What it shows	Use it to…
Elements (Firefox: Inspector)	The live DOM (not your source file) + the CSS rules applied to the selected element, with crossed-out overridden declarations, the box model diagram and computed values	Find why a style is not applied, try CSS changes live, see flex/grid overlays
Console	Errors, warnings, `console.log` output; a JavaScript prompt	Spot failing scripts, test expressions (`$0` = selected element)
Sources (Firefox: Debugger)	All loaded files; breakpoints (places where execution pauses so you can inspect it)	Step through JavaScript
Network	Every request: status, type, size, timing waterfall (a bar chart of when each request started and ended), headers	Find 404s, slow files, check caching and `Content-Type`
Performance	A recorded timeline of parsing, scripting, style, layout, paint	Find what makes the page slow or janky (stuttering)
Lighthouse	Automated audits for performance, accessibility, SEO (search engine optimisation — helping search engines find and rank the page), best practices	A quick report card (Chapter 14)
Application (Firefox: Storage)	Cookies, local storage (key–value data a site keeps in the browser), service workers, cache	Debug state and offline behaviour
Accessibility pane / tree	The accessibility tree (the simplified tree assistive tools read), roles, names, contrast	Chapter 3

Also useful: the **device toolbar** (Ctrl/Cmd+Shift+M) to simulate phone screens and slow networks, and "Rendering" options such as paint flashing and emulating `prefers-color-scheme: dark` (pretending the user chose dark mode, see Chapter 10).

Seeing what the Network panel sees

The next script serves the first-page site (§1.9) over HTTP, opens it in real headless Chrome (using Puppeteer, a Node library that remote-controls Chrome) and prints each response, like the Network panel's list. It then checks computed style and clicks the button, as you would in the Elements panel and on the page:

```
// code/05_what_the_browser_fetched.mjs (key parts)
const server = http.createServer(/* serves files from site/ with the right Content-Type, 404 otherwise */);
const browser = await puppeteer.launch({ executablePath: '/Applications/Google Chrome.app/Contents/MacOS/Google Chrome',
headless: 'new' });
const page = await browser.newPage();
page.on('response', r => lines.push(`${r.status()}  ${r.request().resourceType().padEnd(10)} ${r.url().replace(origin,
'').padEnd(15)} ${r.headers()['content-type'] ?? ''}`));
await page.goto(origin + '/index.html', { waitUntil: 'networkidle0' });
// ... print lines, then:
console.log('\nAfter load:', await page.evaluate(() => ({
  title: document.title,
  elements: document.querySelectorAll('*').length,
  h1Color: getComputedStyle(document.querySelector('h1')).color,   // proves the CSS was applied
})));
await page.click('#hi');                                            // proves the JavaScript runs
console.log('After clicking "Say hi":', await page.$eval('#msg', el => el.textContent));
```

Output:

```
status type       path           content-type
200  document   /index.html     text/html; charset=utf-8
200  stylesheet /css/style.css  text/css
200  script     /js/main.js     text/javascript
404  other      /favicon.ico

After load: { title: 'My First Page', elements: 31, h1Color: 'rgb(29, 78, 216)' }
After clicking "Say hi": Hi! JavaScript ran.
```

One HTML request led to two more that the parser discovered. The `404` for `/favicon.ico` is the browser automatically asking for a tab icon; add `<link rel="icon" href="...">` (Chapter 2) to control it.

Common mistakes

- Editing styles in DevTools and forgetting they vanish on reload — copy the change back to your CSS file.
- Reading the Elements panel as if it were your source: it shows the DOM *after* parsing fixes and JavaScript changes. Use "View Page Source" (Ctrl/Cmd+U) for the original.
- Debugging caching with "Disable cache" still ticked (it only applies while DevTools is open).

Self-check

1. Where do you see which CSS rule was overridden? → Elements → Styles (overridden declarations are struck through).
2. How do you check a file's status code and `Content-Type`? → Network panel, click the request → Headers.
3. Why might the Elements panel differ from View Source? → It shows the live DOM after parsing fixes and script changes.

1.9 Your first page and file structure

How it works

1. Create a folder for the site and a file called `index.html`. Web servers return `index.html` when a URL names a folder (`https://site.com/` → `/index.html`), so it is the conventional home page.
2. Type the page in any plain-text editor (VS Code, Microsoft's free code editor, is the common choice; a word processor is not — it adds formatting).
3. Open it: double-click it (the browser shows a `file://` URL), or better, run a tiny local server so URLs behave as they will in production, e.g. `npx serve .` (npx runs a Node package without installing it permanently) or `python3 -m http.server 8000`, then visit `http://localhost:8000/` (`localhost` is the name your computer uses for itself).
4. Edit, save, reload. Use DevTools to see what the browser made of it.

Here is the complete first page; every line is explained in Chapter 2.

```
<!-- code/site/index.html -->
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="utf-8">
  <meta name="viewport" content="width=device-width, initial-scale=1">
  <title>My First Page</title>
  <link rel="stylesheet" href="css/style.css">
</head>
<body>
  <header>
    <h1>Hello, web!</h1>
    <nav><a href="index.html">Home</a> · <a href="about.html">About</a></nav>
  </header>
  <main>
    <p>This site is four plain text files: <code>index.html</code>, <code>about.html</code>, <code>css/style.css</code>
and <code>js/main.js</code>.</p>
    <p>The browser read the HTML, fetched the stylesheet it links to, and drew the result.</p>
    <ul>
      <li>HTML says <strong>what</strong> things are.</li>
      <li>CSS says how they <strong>look</strong>.</li>
      <li>JavaScript says how they <strong>behave</strong>.</li>
    </ul>
    <button type="button" id="hi">Say hi</button>
    <output id="msg"></output>
  </main>
  <footer><small>Made with a text editor.</small></footer>
  <script src="js/main.js"></script>
</body>
</html>
```

```css
/* code/site/css/style.css (excerpt) */
body   { font-family: system-ui, sans-serif; max-width: 40rem; margin: 2rem auto; padding: 0 1rem; line-height: 1.5;
color: #1f2933; }
header { border-bottom: 2px solid #3b82f6; margin-bottom: 1rem; }
h1     { color: #1d4ed8; margin: 0 0 .25rem; }
button { font: inherit; padding: .4rem .9rem; border-radius: 6px; border: 1px solid #1e3a8a; background: #1d4ed8; color:
white; }
```

```js
// code/site/js/main.js
document.getElementById('hi').addEventListener('click', () => {
  document.getElementById('msg').textContent = 'Hi! JavaScript ran.';
});
```

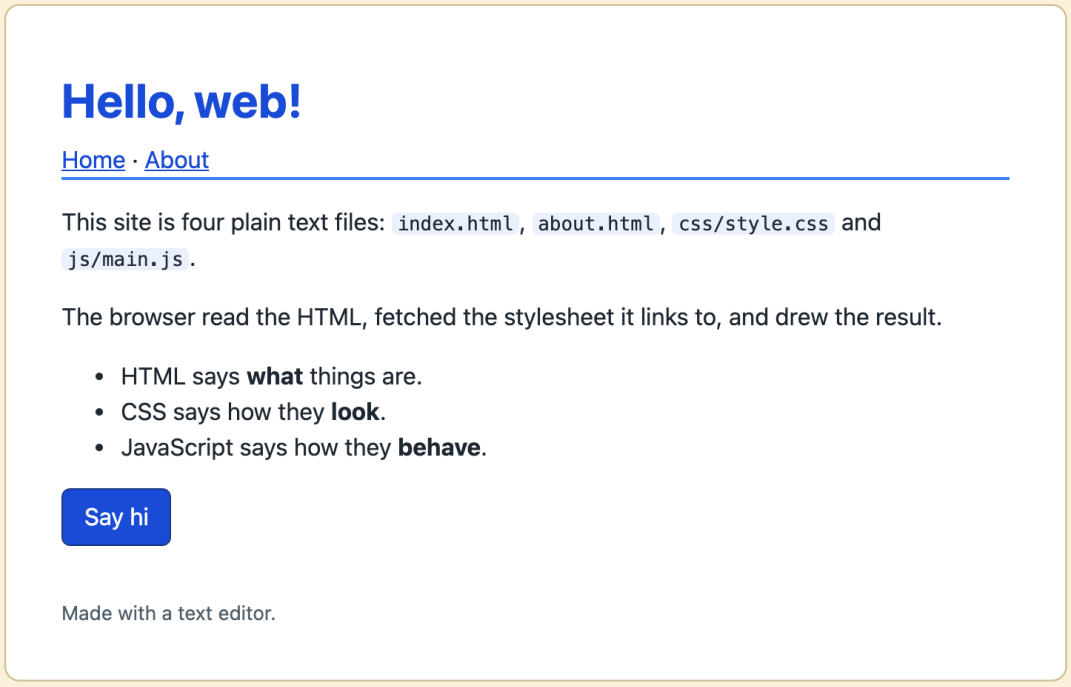


Figure 1.2 — `code/site/index.html` in Chrome.

The script tag sits at the end of `<body>`, so the button exists when it runs. (An equally good choice is `<script src="js/main.js" defer>` in the head.)

A sensible file structure

```
my-site/
├── index.html          home page  →  /
├── about.html          →  /about.html
├── blog/
│   └── index.html      →  /blog/
├── css/
│   └── style.css
├── js/
│   └── main.js
├── img/
│   ├── logo.svg
│   └── hero.avif
└── favicon.ico
```

Rule	Why
Lower-case names, words separated by hyphens (`contact-us.html`)	URLs are case-sensitive on most servers; spaces become `%20`; hyphens read well and are what search engines expect
One folder per asset type (`css/`, `js/`, `img/`) — or per feature in larger projects	Easy to find, easy to set cache rules
`index.html` in every folder that should be a "page"	Clean URLs such as `/blog/`
Document-relative links between pages	The site works from any folder or domain
Keep source and generated files apart once you add build tools (programs that transform and bundle your files for production) (`src/` → `dist/`)	You never edit generated output by hand

Real-world examples

Static sites (sites made of ready-made files, with no code running on the server per request) like this are **deployed** (put online) by uploading the folder to GitHub Pages, Netlify, Cloudflare Pages or an S3 bucket (a folder in Amazon's file-storage service) behind a CDN. Frameworks (ready-made libraries and conventions for building apps, such as React, Next.js, Astro) generate the same kind of HTML, CSS and JS files; the browser still receives exactly what this chapter describes.

Common mistakes

- Saving as `index.html.txt` (hidden extensions on Windows/macOS).
- Linking `href="CSS/Style.css"` when the file is `css/style.css` — works on a case-insensitive laptop (one that treats `A` and `a` in file names as the same), breaks on a Linux server.
- Missing `<!DOCTYPE html>` or `<meta charset="utf-8">`: quirks mode and garbled accents.

Self-check

1. Why is the home page called `index.html`? → Servers return it by default for a folder URL.
2. Why prefer a local server over double-clicking? → Root-relative URLs, `fetch`, modules and some APIs behave differently or fail on `file://`.
3. Why is the script at the end of `<body>` (or `defer`)? → So parsing is not blocked and the elements it uses already exist.

1.10 Networking fundamentals

What it is

Sections 1.3–1.5 followed one request from name to page. This section steps down one level and names the machinery underneath: the **layers** that split the job of moving data, the two transport protocols (**TCP** and **UDP**) and their modern mix (**QUIC**), how addresses work (**IPv4, IPv6, NAT**), and the boxes that sit in the path (**firewalls, proxies, VPNs**). You do not need this to write HTML, but interviewers ask it, and it explains everyday puzzles such as "why does my laptop have the address `192.168.1.20` when the website sees a different one?".

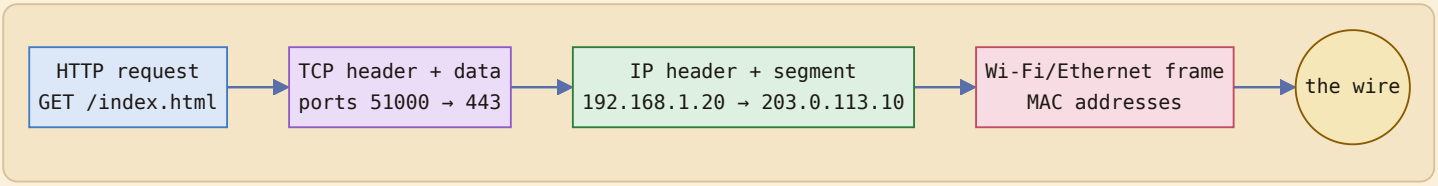
Analogy: sending a parcel. You write a letter (the application data), put it in an envelope with the flat number (the **port**), the post office adds the street address (the **IP address**), and a van moves it along one road at a time (the **link**). Each layer only cares about its own label.

How it works — the layer models

A **protocol** is an agreed set of rules for a conversation. Networking stacks protocols in **layers**: each layer uses the one below and serves the one above, and each wraps the data in its own **header** (a block of labels and control information in front of the data) — this wrapping is called **encapsulation**. There are two famous layer models:

OSI model (7 layers, a teaching model, ISO 7498)	TCP/IP model (4 layers, what the Internet actually runs, RFC 1122)	Examples	Unit of data
7 Application · 6 Presentation · 5 Session	**Application**	HTTP, DNS, TLS*, SSH, SMTP (email sending)	message
4 Transport	**Transport**	TCP, UDP, QUIC	segment / datagram
3 Network	**Internet**	IP (v4, v6), ICMP (`ping`; error and test messages)	packet
2 Data link · 1 Physical	**Link**	Ethernet, Wi-Fi, fibre, radio	frame / bits

*TLS sits awkwardly between transport and application; people call it "layer 4.5" or put it in OSI layer 6.



A **load balancer** is a server that spreads incoming requests across several identical back-end servers so no single one is overloaded (System Design Fundamentals). When engineers say "a layer-4 load balancer" they mean one that looks only at IP addresses and ports; "layer 7" means it reads HTTP (URLs, headers, cookies). See Load Balancing and Reverse Proxy in the system-design notes.

How it works — ports and sockets

An IP address finds the **machine**; a **port** (a number 0–65535) finds the **program** on it. A **socket** is one end of a conversation: IP + port + protocol. Servers listen on **well-known ports**; clients get a random **ephemeral port** (a short-lived port used for just one connection) from the OS (the demo below shows `61804`).

Port	Service	Port	Service
22	SSH	443	HTTPS (TCP), HTTP/3 (UDP)
25 / 587	SMTP (mail between servers / from clients)	3306 / 5432	MySQL / PostgreSQL (databases)
53	DNS (UDP, TCP for large answers)	6379	Redis (in-memory data store)
80	HTTP	8080 / 3000 / 5173	common dev-server choices

How it works — TCP vs UDP vs QUIC

	TCP (RFC 9293)	**UDP** (RFC 768)	**QUIC** (RFC 9000)
Connection set-up	3-way handshake (1 round trip), then TLS	none — just send	1 round trip including TLS 1.3; 0-RTT on resume
Delivery	reliable, ordered, retransmits lost data	best effort: may lose, duplicate or reorder	reliable and ordered **per stream**
Shape of data	one **byte stream**, no message boundaries	separate **datagrams**, boundaries kept	many independent streams in one connection
Flow / congestion control (slowing down so the receiver / network is not overwhelmed)	yes	no (the app must be polite)	yes
Head-of-line blocking	one lost packet stalls everything behind it	none	only the stream that lost the packet waits
Used by	HTTP/1.1, HTTP/2, SSH, databases, email	DNS, video calls, games, VoIP (internet phone calls), DHCP (hands out IP addresses on a network)	HTTP/3

Why do games and video calls use UDP? A voice sample that arrives late is useless — better to skip it than to stall the call waiting for a retransmission (sending lost data again). QUIC runs **on top of UDP** for a practical reason: routers (devices that forward packets between networks) and firewalls (filters, see below) everywhere already pass UDP, while a brand-new transport protocol would be blocked. QUIC then re-implements reliability and encryption itself, and its connection IDs let a download survive moving from Wi-Fi to 4G.

How it works — addresses: IPv4, IPv6, NAT

- **IPv4** (RFC 791): 32-bit addresses (a **bit** is a single 0 or 1; 32 bits allow about 4.3 billion different values) written as four numbers, `203.0.113.10` — about 4.3 billion in total, which ran out (the last big regional pools were exhausted between 2011 and 2019).
- **IPv6** (RFC 8200): 128-bit addresses in hex (base-16, digits 0–9 and a–f) groups, `2001:db8::10` (`::` means "a run of zeros") — enough that every device can have its own public address. IPv6 also drops header checksums (small numbers used to detect corrupted data) and in-network fragmentation (routers splitting packets that are too big), and makes auto-configuration standard. Both run side by side today (**dual stack**); a browser that has both tries IPv6 and IPv4 almost in parallel ("Happy Eyeballs", RFC 8305) and uses whichever connects first.
- **CIDR notation** `10.0.0.0/8` means "the first 8 bits are fixed" — a range of 2^24 addresses. It is how subnets (a block of addresses that belong to one network) and firewall rules are written.
- **Private ranges** (RFC 1918): `10.0.0.0/8`, `172.16.0.0/12`, `192.168.0.0/16` are never routed on the public Internet; any home or office can reuse them. `127.0.0.1` / `::1` is **loopback** — this machine.
- **NAT** (network address translation) is how a house full of private addresses shares one public IPv4 address. The router rewrites the source of each outgoing packet to *its* public IP and a fresh port, remembers the mapping, and rewrites replies back. Side effect: nobody outside can start a connection to a device inside unless a mapping exists — which is why peer-to-peer (device-to-device, no server in the middle) video calls need tricks (STUN/TURN — helper servers that discover your public address or relay traffic — used by WebRTC, the browser's real-time audio/video API).
- **Unicast** = to one host; **broadcast** = to every host on the local network (IPv4 only); **multicast** = to a subscribed group; **anycast** = one address announced from many places, and routing delivers you to the nearest — how public DNS resolvers and CDNs answer from a nearby city.
- **LAN** = the local network in one building; **WAN** = networks joined across distance (the Internet is the biggest WAN).

How it works — DNS records beyond the basics

§1.3 covered `A`, `AAAA`, `CNAME`, `MX`, `TXT` and `NS`. Four more show up in real set-ups: `SOA` (start of authority: the zone's primary server and timers), `PTR` (reverse lookup: IP → name, used by mail servers to

check senders), `SRV` (service → host + port, e.g. for SIP — internet telephony — or some databases), and `CAA` (which certificate authorities may issue certificates for the domain). A `CNAME` cannot sit at the bare domain (`example.com`) alongside other records, so DNS hosts offer non-standard "ALIAS/ANAME" flattening (looking up the alias themselves and publishing the resulting IP) for that case.

How it works — firewalls, proxies and VPNs

- A **firewall** filters traffic by rules. A *packet filter* matches addresses, ports and protocols ("allow TCP 443 from anywhere, allow 22 only from 10.0.0.0/8, deny the rest"). A *stateful* firewall also tracks connections, so replies to traffic you started are let back in automatically. Cloud **security groups** are stateful firewalls. A **WAF** (web application firewall) works at layer 7 and inspects HTTP for attacks such as SQL injection (sneaking database commands into form fields or URLs).
- A **forward proxy** acts for **clients**: a company proxy that all employee traffic goes through (filtering, caching, logging). A **reverse proxy** acts for **servers**: nginx, a load balancer or a CDN in front of your app, which ends TLS (decrypts HTTPS so the app behind it does not have to — "TLS termination"), caches and spreads load. From the browser's side a reverse proxy *is* "the server".
- A **VPN** (virtual private network) wraps your packets in an encrypted tunnel to another network so you behave as if you were plugged in there. **Remote-access VPNs** connect a laptop to the office; **site-to-site VPNs** join two offices or an office and a cloud network. Common protocols: **IPsec**, **WireGuard** (small, modern), and TLS-based VPNs such as OpenVPN. A VPN hides your traffic from the local Wi-Fi, not from the VPN provider.

Seeing it: `code/06_network_basics.mjs`

The script uses only the loopback interface: it resolves `localhost`, sends three writes over TCP and three datagrams (self-contained UDP messages) over UDP, checks addresses against a firewall-style allow-list (`net.BlockList`), and simulates a NAT table.

```
localhost -> ::1 (IPv6), 127.0.0.1 (IPv4)
isIPv4("203.0.113.10") = true | isIPv6("2001:db8::10") = true
TCP server listening on 127.0.0.1:61803
TCP server sees client 127.0.0.1:61804 (ephemeral port picked by the OS)
TCP bytes received      : "one|two|three|" (in order, as one stream)
UDP datagrams received  : ["one","two","three"] (boundaries kept; on a real network some could be lost or reordered)
firewall: 192.168.1.20  private=true  -> ALLOW (admin port 22)
firewall: 10.4.5.6      private=true  -> ALLOW (admin port 22)
firewall: 172.20.0.1    private=true  -> ALLOW (admin port 22)
firewall: 203.0.113.10  private=false -> DENY
firewall: 8.8.8.8       private=false -> DENY
NAT out: 192.168.1.10:51000 -> 203.0.113.5:40001
NAT out: 192.168.1.11:51000 -> 203.0.113.5:40002
NAT in : reply to :40001     -> 192.168.1.10:51000
NAT in : packet to :40999    -> DROP (no mapping: unsolicited inbound traffic)
all assertions passed
```

Notice that TCP handed the server `one|two|three|` as a single stream — the `|` separators are there because TCP itself keeps no message boundaries, so every protocol on top (HTTP included) must mark where messages end (`Content-Length`, chunked encoding — sending the body in pieces, each prefixed with its size — and frames). UDP delivered three separate messages. Port numbers change on every run.

Common mistakes

- "UDP is unreliable, so never use it" → it is the right choice when late data is worthless (live media, games) or when the app adds its own reliability (QUIC, DNS retries).
- Treating TCP reads as messages → one `send` can arrive split across reads, or several can arrive together; always frame your messages.
- Thinking your laptop's `192.168.x.x` address is what servers see → they see the router's public address after NAT.
- Opening a database port (5432, 6379) to `0.0.0.0/0` in a cloud security group → allow only the app servers' subnet.
- Assuming a VPN makes you anonymous → the VPN operator sees everything the Wi-Fi provider used to.

In the interview

30-second answer — TCP vs UDP: "TCP is connection-based: a handshake, then a reliable, ordered byte stream with flow and congestion control — used by HTTP/1.1 and /2, databases and SSH. UDP just sends

> independent datagrams with no guarantees and no set-up cost — used by DNS, games and video calls where speed beats completeness. QUIC builds reliable, encrypted, multiplexed streams on top of UDP and powers HTTP/3."

- *OSI vs TCP/IP?* → OSI is a 7-layer teaching model; TCP/IP is the 4-layer model the Internet runs (application, transport, internet, link).
- *Why IPv6?* → IPv4's 32-bit space ran out; NAT stretched it at the cost of end-to-end reachability. IPv6's 128 bits give every device a public address.
- *Forward vs reverse proxy?* → Forward acts for clients (egress filtering — controlling outgoing traffic); reverse acts for servers (TLS termination, load balancing, caching).

Self-check

1. Which layer does a port belong to? → Transport (TCP/UDP).
2. Why does HTTP/3 run over UDP rather than a new protocol? → Middleboxes (routers, firewalls and other devices in the path) already pass UDP; QUIC adds reliability and TLS itself.
3. What does a home router's NAT table map? → Public IP:port ↔ private IP:port of each outgoing connection.
4. Which CIDR ranges are private IPv4? → `10/8`, `172.16/12`, `192.168/16`.
5. Stateful vs packet-filter firewall? → Stateful tracks connections and lets replies back in automatically.

TL;DR

Data travels down the layers (application → transport → IP → link), each adding a header. TCP = reliable ordered stream after a handshake; UDP = fast independent datagrams; QUIC = TCP-like reliability plus TLS over UDP (HTTP/3). IPv4 is 32-bit and stretched by NAT and private ranges; IPv6 is 128-bit. Firewalls filter by address/port/state, reverse proxies front servers, VPNs tunnel you into another network.

Sources: RFC 1122 (Internet host layers), RFC 9293 (TCP), RFC 768 (UDP), RFC 9000 (QUIC), RFC 791 / RFC 8200 (IPv4 / IPv6), RFC 1918 (private addresses), RFC 8305 (Happy Eyeballs), RFC 1035 (DNS), Node.js docs for `net` and `dgram`.

Chapter 1 TL;DR

- The **web** runs on the Internet: **URLs** name resources, **HTTP** fetches them, **HTML** describes them; CSS styles, JavaScript adds behaviour.
- **URL** = scheme://host:port/path?query#fragment; **origin** = scheme + host + port; relative URLs resolve against the page URL.
- **DNS** maps names to IPs through resolver → root → TLD → authoritative servers, with TTL-based caching.
- Connections use **TCP + TLS** (HTTPS) or **QUIC** (HTTP/3). HTTPS gives confidentiality, integrity and authentication.
- **HTTP** is stateless request/response: method + path + headers → status + headers + body. Know GET/POST, 2xx/3xx/4xx/5xx, `Content-Type`, `Cache-Control`.
- **Rendering**: HTML → DOM, CSS → CSSOM → render tree → layout → paint → composite. The parser always recovers from bad markup; the doctype keeps you out of quirks mode.
- CSS is render-blocking; classic scripts are parser-blocking; use `defer` or modules.
- **DevTools** show the live DOM, applied styles, network traffic and performance — use them constantly.
- A site is a folder of text files; `index.html` is the default page; lower-case, hyphenated names and relative links keep it portable.
- **Networking**: layers (application → transport → IP → link); **TCP** = reliable ordered stream, **UDP** = independent datagrams, **QUIC** = reliable encrypted streams over UDP (HTTP/3); IPv4 (32-bit, stretched by **NAT** and private ranges) vs IPv6 (128-bit); firewalls filter, reverse proxies front servers, VPNs tunnel.

Next: Chapter 2 — HTML Structure and Text.

Chapter 2 — HTML Structure and Text

HTML is the skeleton of every page. This chapter teaches the grammar of the language (elements, attributes, nesting), the fixed structure every document needs, the `<head>` **metadata** (data *about* the page — its title, language, character set — rather than the content itself) that browsers, search engines and social networks read, and the elements you use for text: headings, paragraphs, inline meaning, lists and links. It closes with character references, comments and validation — how to check that what you wrote is what you meant.

What you'll learn

#	Topic	You will be able to…
2.1	Elements, tags and attributes	read and write any HTML element correctly, including void and boolean ones
2.2	Document structure and the doctype	write a valid page skeleton from memory
2.3	The `<head>`: charset, viewport, title, description, links	choose the right metadata for every page
2.4	Headings	build a heading hierarchy that works for people, screen readers (software that reads pages aloud for blind users) and search
2.5	Paragraphs and text blocks	use `p`, `br`, `hr`, `pre`, `blockquote`, `address` and understand whitespace
2.6	Text-level semantics	pick between `strong` / `b`, `em` / `i`, `mark`, `code`, `time` …
2.7	Lists	use `ul`, `ol` (with `start`, `reversed`, `type`), nested lists and `dl`
2.8	Links	write absolute, relative, fragment, `mailto:` / `tel:` and download links; use `target` and `rel` safely
2.9	Character references and comments	escape `<`, `&` and quotes; comment your markup
2.10	Validation	find mistakes the browser silently repairs

Examples are in `code/`. Each `.html` file is shown as a screenshot from headless Chrome (Chrome run by a script, with no window); the `.mjs` scripts parse those same files with jsdom, a Node.js library that builds a browser-like DOM without a browser (`node code/01_head.mjs`).

2.1 Elements, tags and attributes

What it is

HTML (HyperText Markup Language) **marks up** plain text (adds labels to it that describe its structure — the labels are the *markup*): you wrap pieces of content in **tags** to say what each piece *is*.

```
<a href="https://example.com" title="Opens the example site">Visit Example</a>
```

Piece	In the example
Start tag	`<a href="https://example.com" title="Opens the example site">`
Element name	`a`
Attributes (name = value)	`href` = `https://example.com`, `title` = `Opens the example site`
Content	`Visit Example`
End tag	`</a>`

- An **element** = start tag + content + end tag (`<p>Hello</p>`). Elements nest to form a tree (the DOM, Chapter 1).
- An **attribute** adds information to an element: `name="value"`. Quote values (double quotes are the convention); names are case-insensitive, but write them in lower case.

- **Void elements** have no content and no end tag: `area`, `base`, `br`, `col`, `embed`, `hr`, `img`, `input`, `link`, `meta`, `source`, `track`, `wbr`. A trailing slash (`<br />`) is allowed and ignored — it is a habit from XHTML (a stricter, XML-based variant of HTML popular in the 2000s), not required.
- **Boolean attributes** are true when present, false when absent: `<input disabled>`, `<ol reversed>`. `disabled="false"` still means *disabled* — the value is irrelevant.
- **Nesting** must be proper: close inner elements before outer ones (`<b><i>…</i></b>`, not `<b><i>…</b></i>`).

Analogy: tags are labels on moving boxes ("kitchen — fragile"). The contents are the same; the labels tell everyone how to treat them.

Global attributes (usable on any element)

Attribute	Purpose
`id`	Unique identifier in the page: link target (`#id`), label/ARIA references (ARIA = extra accessibility attributes, Chapter 3), JS hook (a handle scripts use to find the element)
`class`	Space-separated names for CSS and JS; many elements can share one
`style`	Inline CSS — styles written directly on one element (avoid except for quick tests or dynamic values)
`title`	Advisory tooltip text (a small label that pops up on mouse hover); not reliably shown on touch or read by screen readers — never put essential info only here
`lang`	Language of the content (`en`, `fr-CA`) — affects pronunciation, hyphenation (splitting words across lines with a hyphen), fonts, translation
`dir`	Text direction: `ltr` (left-to-right), `rtl` (right-to-left, e.g. Arabic), `auto`
`hidden`	Not rendered (acts like `display: none`)
`tabindex`	Focus order control — *focus* is which element currently receives keyboard input (Chapter 3)
`data-*`	Your own data for scripts: `data-product-id="42"` → `el.dataset.productId`
`inert`	Makes a subtree (an element and everything inside it) unfocusable and unclickable (e.g. content behind a modal — a dialog that blocks the rest of the page)
`translate`, `spellcheck`, `contenteditable`	Translation hints, spell-check, in-place editing

Block-level and inline — how elements flow

By default some elements start on a new line and take the full width (**block**: `p`, `h1`, `ul`, `div`, `section`), while others sit inside a line of text (**inline**: `a`, `strong`, `em`, `span`, `code`). This is purely a *default CSS* `display` *value* (the CSS property that decides how an element's box takes part in layout; Chapter 7); the spec itself groups elements into **content categories** — *flow content* (almost everything in `body`) and *phrasing content* (text and inline-level markup). Each element has a **content model** saying what it may contain: a `<p>` may only contain phrasing content, so a `<div>` or `<ul>` inside a `<p>` is invalid (the parser actually closes the `<p>` first). An `<a>` is *transparent* (its allowed content is whatever its parent allows): it may wrap blocks (a whole card) as long as it contains no other interactive element (something users can click or type into, such as a link, button or input).

`<div>` (block) and `<span>` (inline) are the generic containers: they mean nothing and exist only as hooks for CSS and JS. Use them when no meaningful element fits.

Common mistakes

- Unquoted attribute values with spaces: `class=big red` gives class `big` and a stray attribute `red`.
- Repeating an `id` — ids must be unique; use `class` for groups.
- Writing `<br></br>` or `<img></img>` — void elements have no end tag.
- Choosing elements for their default look ("I need bold, so `<h3>`"). Choose for meaning; CSS handles looks.

Self-check

1. Is `<input required="no">` required? → Yes — boolean attributes are on whenever present.

2. What is the difference between an element and a tag? → A tag is the `<p>` / `</p>` text; the element is the whole thing including content (and the DOM node it becomes).
3. Can a `<p>` contain a `<ul>`? → No; the parser would close the paragraph before the list.

2.2 Document structure and the doctype

What it is

Every HTML document has the same outer shape:

```
<!DOCTYPE html>
<html lang="en">
  <head>  <!-- information ABOUT the page: not displayed -->
  </head>
  <body>  <!-- the page itself: everything the user sees -->
  </body>
</html>
```

Part	Role
`<!DOCTYPE html>`	Must be first. Switches the browser to **standards mode**; without it you get **quirks mode**, which emulates old browser bugs (Chapter 1 showed a table ending up *inside* a paragraph). It is case-insensitive and is not an element.
`<html lang="en">`	The **root element**. `lang` tells screen readers which voice/pronunciation to use, lets browsers offer translation and hyphenate correctly. It is also a WCAG requirement (WCAG = Web Content Accessibility Guidelines, the international accessibility standard; success criterion 3.1.1 Language of Page).
`<head>`	Metadata: character set (how bytes map to letters), title, viewport (the visible screen area), links to CSS, scripts, icons, social previews (the card shown when a link is shared).
`<body>`	Content.

The `html`, `head` and `body` tags are technically optional (the parser creates the elements anyway), but always write them — explicit structure is easier to read and tools expect it.

The whole skeleton, with a realistic head, is `code/skeleton.html`:

```
<!DOCTYPE html>
<!-- The doctype above switches the browser to standards mode. This is an HTML comment. -->
<html lang="en">
<head>
  <meta charset="utf-8">
  <meta name="viewport" content="width=device-width, initial-scale=1">
  <title>Trail Running Basics — Mountain Club</title>
  <meta name="description" content="A beginner's guide to trail running: shoes, pacing, safety and the five trails our club runs every week.">
  <link rel="icon" href="data:image/svg+xml,<svg ...><circle ... fill='%23f97316'/></svg>">
  <link rel="canonical" href="https://club.example.org/guides/trail-running">
  <meta name="theme-color" content="#f97316">
  <meta property="og:title" content="Trail Running Basics">
  <meta property="og:image" content="https://club.example.org/img/trail-card.jpg">
  <style> body { font-family: system-ui, sans-serif; max-width: 36rem; margin: 1.5rem auto; padding: 0 1rem; line-height: 1.5; } </style>
</head>
<body>
  <h1>Trail Running Basics</h1>
  <p>Everything in <code>&lt;head&gt;</code> is information <em>about</em> the page. Everything in <code>&lt;body&gt;</code> is the page.</p>
  <p>The title, description and icon you cannot see here show up in the browser tab, in search results and in link previews.</p>
</body>
</html>
```

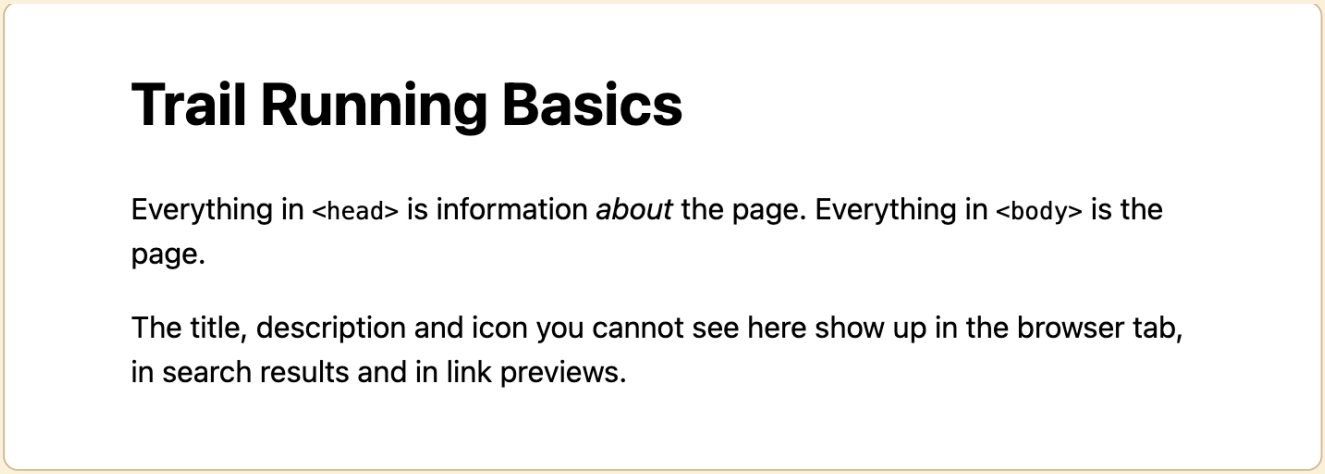


Figure 2.1 — Only the `<body>` is drawn. The head's work shows up elsewhere: the tab title and icon, search snippets, link previews.

Older doctypes such as `<!DOCTYPE HTML PUBLIC "-//W3C//DTD HTML 4.01//EN" ...>` (DTD = Document Type Definition, the old formal grammar of a markup language) mostly still trigger standards (or "almost standards") mode — though a few legacy ones, such as HTML 4.01 Transitional without its URL, switch on quirks mode — so there is no reason to write them today.

Self-check

1. What happens without a doctype? → Quirks mode: legacy layout behaviour you do not want.
2. Why set `lang` on `<html>`? → Correct screen-reader pronunciation, translation, hyphenation; WCAG 3.1.1.
3. Where does the `<title>` appear? → Tab/window title, bookmarks, history, search result headline — not in the page body.

2.3 The `<head>`: metadata that matters

How it works

The browser reads the head top to bottom before rendering. Order matters for two items: `charset` must come first, and render-blocking CSS (CSS the browser must have before it draws anything, Chapter 1) should be early so it downloads early.

The script below parses `skeleton.html` and prints what a browser, a search engine or a social network would learn from it:

```
// code/01_head.mjs
const source = fs.readFileSync('skeleton.html');
const { window } = new JSDOM(source, { url: 'https://club.example.org/guides/trail-running' });
const doc = window.document;
console.log('doctype   :', `<!DOCTYPE ${doc.doctype?.name}>`, '->', doc.compatMode);
console.log('lang      :', doc.documentElement.lang);
console.log('charset   :', doc.characterSet, `(declared at byte ${source.indexOf('charset')} — must be within the first
1024)`);
console.log('title     :', doc.title);
for (const m of doc.head.querySelectorAll('meta[name], meta[property]'))
  console.log('meta      :', (m.name || m.getAttribute('property')).padEnd(12), '=', m.content.slice(0, 60));
for (const l of doc.head.querySelectorAll('link'))
  console.log('link      :', l.rel.padEnd(12), '->', l.href.slice(0, 60));
// ... comments and head children
```

Output:

```
doctype   : <!DOCTYPE html> -> CSS1Compat
lang      : en
charset   : UTF-8 (declared at byte 140 — must be within the first 1024)
title     : Trail Running Basics — Mountain Club
meta      : viewport     = width=device-width, initial-scale=1
meta      : description  = A beginner's guide to trail running: shoes, pacing, safety a
meta      : theme-color  = #f97316
meta      : og:title     = Trail Running Basics
meta      : og:image     = https://club.example.org/img/trail-card.jpg
link      : icon         -> data:image/svg+xml,<svg xmlns='http://www.w3.org/2000/svg' v
link      : canonical    -> https://club.example.org/guides/trail-running
comments  : [
  'The doctype above switches the browser to standards mode. This is an HTML comment.'
]
head children: meta, meta, title, meta, link, link, meta, meta, meta, style
```

The essential head elements

Element	Example	Why
`<meta charset="utf-8">`	—	Tells the browser how to turn bytes into characters. Must appear within the **first 1024 bytes**. UTF-8 (the universal encoding that covers every language and emoji) is the only encoding you should use; without it accented letters and emoji can turn into garbage ("cafÃ©"). An HTTP `Content-Type: …; charset=` header overrides it.
`<meta name="viewport">`	`content="width=device-width, initial-scale=1"`	Without it, phones pretend the screen is ~980 px wide and shrink the page. This line makes CSS pixels (the browser's logical pixel unit, which may span several physical screen dots) match the device width — the foundation of responsive design (layouts that adapt to any screen size, Chapter 10). Do **not** add `maximum-scale=1` or `user-scalable=no`: blocking zoom fails WCAG 1.4.4 (Resize Text).
`<title>`	`Trail Running Basics — Mountain Club`	Required. Unique and descriptive per page, most specific part first. It is the first thing a screen reader announces and the headline in search results.
`<meta name="description">`	one or two sentences	Often used as the search-result snippet (the short summary under the link; search engines may choose other text). Snippets are usually cut at about 150–160 characters. Not a ranking trick (it does not move you up the results); it is a sales pitch for the click.
`<link rel="stylesheet">`	`href="css/style.css"`	Loads CSS (render-blocking).
`<script src defer>` / `type="module"`	`src="js/app.js"`	Loads JS without blocking the parser (Chapter 1).
`<link rel="icon">`	`.ico`, `.png` or `.svg` (SVG = a vector image format, Chapter 4)	The tab icon (favicon). Add `rel="apple-touch-icon"` for iOS home screens.
`<link rel="canonical">`	full URL	Tells search engines the preferred ("canonical" = official) URL when the same content is reachable at several addresses.
Open Graph (a metadata format started by Facebook) / Twitter Card metas	`property="og:title"`, `og:description`, `og:image`	Control the preview card when the link is shared on social networks and chat apps.
`<meta name="theme-color">`	`#f97316` (a hex colour code)	Tints the browser UI (user interface — its toolbars and address bar) on some mobile browsers.
`<meta name="robots">`	`content="noindex"`	Keeps a page out of search results.
`<base href>`	`href="/docs/"`	Changes the base for *all* relative URLs on the page. Rarely a good idea — it surprises people.
`<link rel="preload">`, `rel="preconnect"`	fonts, hero image (the large banner image at the top), CDN origin	Performance hints ("fetch this early", "connect to this server early") (Chapter 14).

A useful habit is a **template** you copy for every new page: doctype, `lang`, charset, viewport, title, description, stylesheet, deferred script.

Common mistakes

- `<meta charset>` placed after a long comment or `<title>` with non-ASCII characters (anything outside basic English letters, digits and punctuation, such as é or emoji) — it may fall outside the first 1024 bytes.
- The same `<title>` ("Home") on every page — useless in tabs, bookmarks and search results.
- Putting visible content in the head: the parser moves it into the body anyway.
- Disabling zoom in the viewport tag.

Self-check

1. Which head element must come first, and why? → `<meta charset>`, because the browser needs the encoding before interpreting the rest (within 1024 bytes).
2. What breaks on a phone without the viewport meta? → The page lays out at ~980 px and is shrunk, so text is tiny.
3. Is the meta description visible on the page? → No; it may appear in search snippets.

2.4 Headings

What it is

`<h1>`…`<h6>` are section headings, `h1` the most important. Together they form the page's **outline** — its table of contents.

Why it matters

- **Screen-reader users** navigate by headings: in WebAIM's (a US non-profit accessibility organisation) screen-reader user surveys, a large majority say that jumping through the headings is the first thing they do to find information on a long page. A clean hierarchy is the fastest way through your content (Chapter 3).
- **Everyone** skims headings before reading.
- **Search engines** use headings to understand what each part of a page is about.

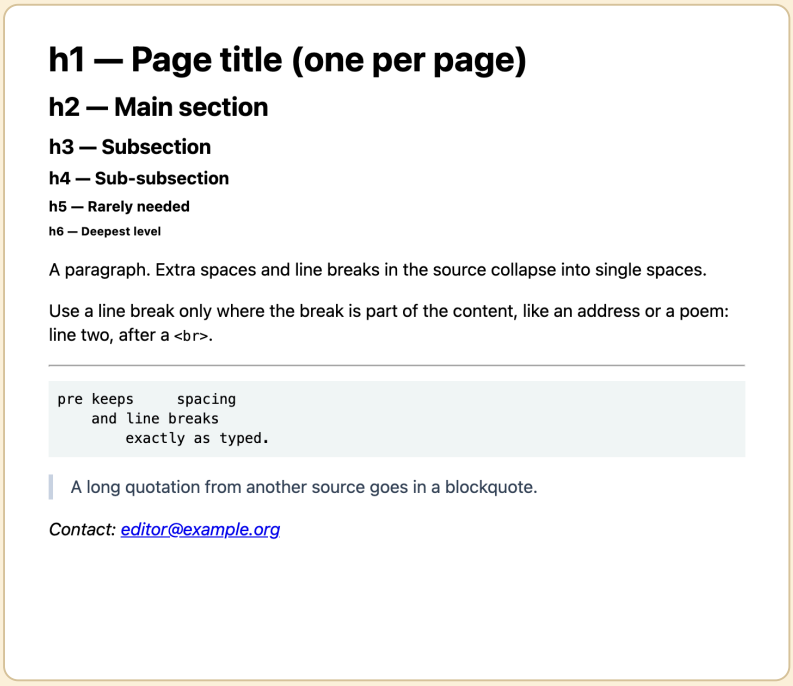


Figure 2.2 — `code/text-blocks.html`: the six heading levels at their default sizes, then the block text elements from §2.5.

Rules that work

1. **One `<h1>` per page** describing the page (usually close to the `<title>`). The spec allows more, but a single `h1` is the clear best practice.
2. **Don't skip levels going down**: after `h2`, the next subheading is `h3`, not `h4`. Going back up any number of levels is fine (`h4` → `h2`).
3. **Choose the level by structure, never by size.** Want a small `h2`? Style it with CSS.
4. **Don't use headings for non-headings** (a big bold slogan is a `<p>` with a class).
5. A heading plus a subtitle/tagline: wrap them in `<hgroup>` — `<hgroup><h1>Title</h1><p>Tagline</p></hgroup>` — so the tagline is not a fake heading.

There is no "document outline algorithm". Older HTML5 drafts described one in which every `<section>` restarted heading levels, so each section could use its own `h1`. No browser or screen reader ever implemented it, and the WHATWG removed it from the spec in 2022. Heading levels are simply what the numbers say.

Common mistakes

- Using `<h4>` because it "looks right" — a screen reader announces "heading level 4" and users assume there are missing sections.
- Several `h1`s in a template (logo *and* page title).
- Headings made from `<div class="title">` — they look like headings but do not appear in any heading list.

Self-check

1. After an `h2`, may the next heading be an `h4`? → Not for a subsection (skip). Use `h3`.
2. After an `h4`, may the next heading be an `h2`? → Yes — it starts a new major section.
3. Does wrapping content in `<section>` change heading levels? → No; there is no outline algorithm.

2.5 Paragraphs, whitespace and text blocks

What it is

Element	Meaning	Notes
`<p>`	A paragraph	The workhorse. Don't use empty `<p>`s for spacing — use CSS `margin`.
` `	A line break that is part of the content	Addresses, poems, song lyrics. Never for layout spacing.
`<hr>`	A thematic break between paragraphs (topic change, scene change)	Drawn as a line; screen readers may announce a separator. For a purely decorative line, use a CSS border.
`<pre>`	Preformatted text: whitespace and line breaks kept, monospace font (every character the same width)	Code blocks (`<pre><code>…</code></pre>`), ASCII art (pictures drawn with keyboard characters)
`<blockquote>`	A long quotation from another source	Optional `cite="URL"` attribute (not shown); put the visible attribution outside or in a `<figcaption>` (a caption element, Chapter 4)
`<address>`	Contact information for the nearest `<article>` or the whole page	Not for any postal address in text
`<div>`	Generic block container, no meaning	Use when nothing else fits

Whitespace collapsing

In normal text, HTML **collapses whitespace**: any run of spaces, tabs and newlines becomes a single space, and leading/trailing whitespace in a line is dropped. That is why the paragraph in Figure 2.2 shows single spaces even though the source has many. You can indent your HTML freely. To keep whitespace, use `<pre>` or the CSS `white-space` property; to force a space that never breaks a line, use ` ` (§2.9).

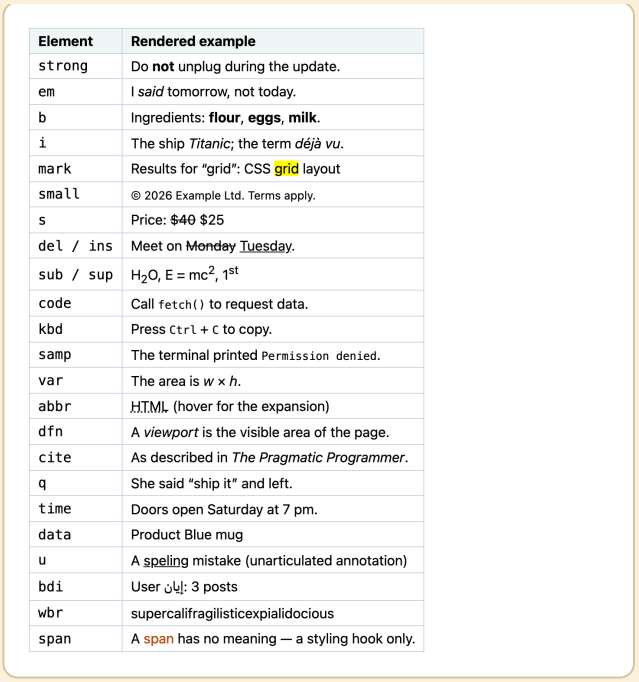
Self-check

1. How do you add space between two paragraphs? → CSS `margin`, not `<br>` or empty paragraphs.
2. When is `<br>` correct? → When the line break is part of the content itself (poem, address).
3. Why do five spaces in your source show as one? → Whitespace collapsing.

2.6 Text-level semantics

What it is

Inline elements that say what a *phrase* is. Browsers give most of them a default look (bold, italic, monospace), but the look is secondary: choose by meaning, then style with CSS.



Element	Rendered example
`strong`	Do **not** unplug during the update.
`em`	I *said* tomorrow, not today.
`b`	Ingredients: **flour**, **eggs**, **milk**.
`i`	The ship *Titanic*; the term *déjà vu*.
`mark`	Results for “grid”: CSS grid layout
`small`	© 2026 Example Ltd. Terms apply.
`s`	Price: ~~$40~~ $25
`del / ins`	Meet on ~~Monday~~ Tuesday.
`sub / sup`	H_2O, E = mc^2, 1^{st}
`code`	Call `fetch()` to request data.
`kbd`	Press `Ctrl` + `C` to copy.
`samp`	The terminal printed `Permission denied`.
`var`	The area is *w* × *h*.
`abbr`	HTML (hover for the expansion)
`dfn`	A *viewport* is the visible area of the page.
`cite`	As described in *The Pragmatic Programmer*.
`q`	She said “ship it” and left.
`time`	Doors open Saturday at 7 pm.
`data`	Product Blue mug
`u`	A speling mistake (unarticulated annotation)
`bdi`	User إيان: 3 posts
`wbr`	supercalifragilisticexpialidocious
`span`	A span has no meaning — a styling hook only.

Figure 2.3 — `code/inline-semantics.html`: every common text-level element with its default rendering.

Element	Means	Use for	Not for
`<strong>`	Strong importance, seriousness or urgency	Warnings, key facts	Headings, general "make it bold"
`<em>`	Stress emphasis that changes the meaning of the sentence	"I *said* tomorrow"	Titles, foreign words
`<b>`	Draw attention without extra importance	Keywords, product names in a review, lead sentence	Importance (use `strong`)
`<i>`	Different voice or mood	Technical terms, foreign phrases (`<i lang="fr">`), ship names, thoughts	Emphasis (use `em`), icons
`<mark>`	Highlighted because of relevance to the current context	Search matches	General highlighting for style
`<small>`	Side comments, fine print	Copyright, legal notes	Making text small for design
`<s>`	No longer accurate or relevant	Old price	Document edits (use `del`)
`<del>` / `<ins>`	Removed / added text in an edit	Change tracking; optional `datetime`, `cite`	—
`<sub>` / `<sup>`	Subscript / superscript required by typography	H_2O, footnote markers, 1st	Pure visual offsets
`<code>`	A fragment of computer code	Inline code; blocks in `<pre><code>`	—
`<kbd>`	User input (keys, commands to type)	Ctrl+C	—
`<samp>`	Sample output from a program	Error messages	—
`<var>`	A variable in maths or programming	x, $w \times h$	—
`<abbr>`	Abbreviation or acronym; `title` gives the expansion	First use of "HTML" — but also write the expansion in the text once, since `title` is not reliably exposed	—
`<dfn>`	The defining instance of a term	Where a term is first defined	Every later mention
`<cite>`	Title of a creative work	Books, films, papers	People's names
`<q>`	Short inline quotation; browser adds quotation marks	"She said *ship it*"	Long quotes (`blockquote`)
`<time>`	A date/time with a machine-readable `datetime`	`<time datetime="2026-10-03T19:00">Saturday at 7 pm</time>`	—
`<data>`	Content with a machine-readable `value`	Product IDs, numbers	Dates (use `time`)
`<u>`	Unarticulated, non-textual annotation	Spelling-error squiggles, Chinese proper-name marks	Links look alike — avoid in normal text
`<bdi>`	Isolate text whose direction is unknown	User names that may be Arabic/Hebrew in an LTR (left-to-right) page	—
`<wbr>`	A place where a long word *may* break	Long URLs, compound words	—
`<span>`	Nothing — a styling/scripting hook	When no element above fits	—

Why it matters

Semantics feed machines: `<time datetime>` lets calendars and search engines read dates; `<code>` gets monospace in reader modes; `<abbr>`, `<dfn>` and `lang` help translation and screen readers. Note that most screen readers

do not announce `<strong>` or `<em>` by default, so never rely on them alone to convey critical information — say it in words too ("Warning: …").

Common mistakes

- Using `<i>` for icons (`<i class="icon-star">`): it is a legacy habit from icon fonts (fonts whose "letters" are pictures); use `<span aria-hidden="true">` (hidden from screen readers) or an SVG.
- Wrapping whole paragraphs in `<strong>`: if everything is important, nothing is.
- `<cite>` around a person's name.

Self-check

1. `<b>` or `<strong>` for "Warning: this deletes your account"? → `<strong>` (importance).
2. How do you mark a date so machines can read it? → `<time datetime="2026-10-03">…</time>`.
3. Which element shows keyboard keys? → `<kbd>`.

2.7 Lists

What it is

Element	Use when	Items
`<ul>`	Order doesn't matter (shopping list, nav links, features)	`<li>`
`<ol>`	Order matters (steps, rankings, legal clauses)	`<li>`
`<dl>`	Name–value groups (glossary, metadata, FAQ)	`<dt>` term(s) + `<dd>` description(s); groups may be wrapped in a `<div>` for styling
`<menu>`	A toolbar of commands	Semantically the same as `<ul>`

`<ol>` attributes:

Attribute	Effect
`start="4"`	First number is 4
`reversed`	Counts down
`type="1	a
`<li value="10">`	Sets this item's number; later items continue from it

```
<!-- code/lists.html (excerpt) -->
<ul>
  <li>Milk</li>
  <li>Bread
    <ul><li>Sourdough</li><li>Rye</li></ul>   <!-- nested list goes INSIDE the <li> -->
  </li>
  <li>Apples</li>
</ul>
<ol start="4" type="a"> <li>fourth item is "d"</li> <li>then "e"</li> </ol>
<ol reversed> <li>Bronze</li> <li>Silver</li> <li>Gold</li> </ol>
<ol type="I"> <li>Introduction</li> <li value="10">Jumps to X</li> <li>Then XI</li> </ol>
<dl>
  <dt>HTML</dt><dd>Structure and meaning</dd>
  <dt>CSS</dt><dd>Presentation</dd>
  <dt>JS</dt><dt>JavaScript</dt><dd>Behaviour (two terms, one description)</dd>
</dl>
```

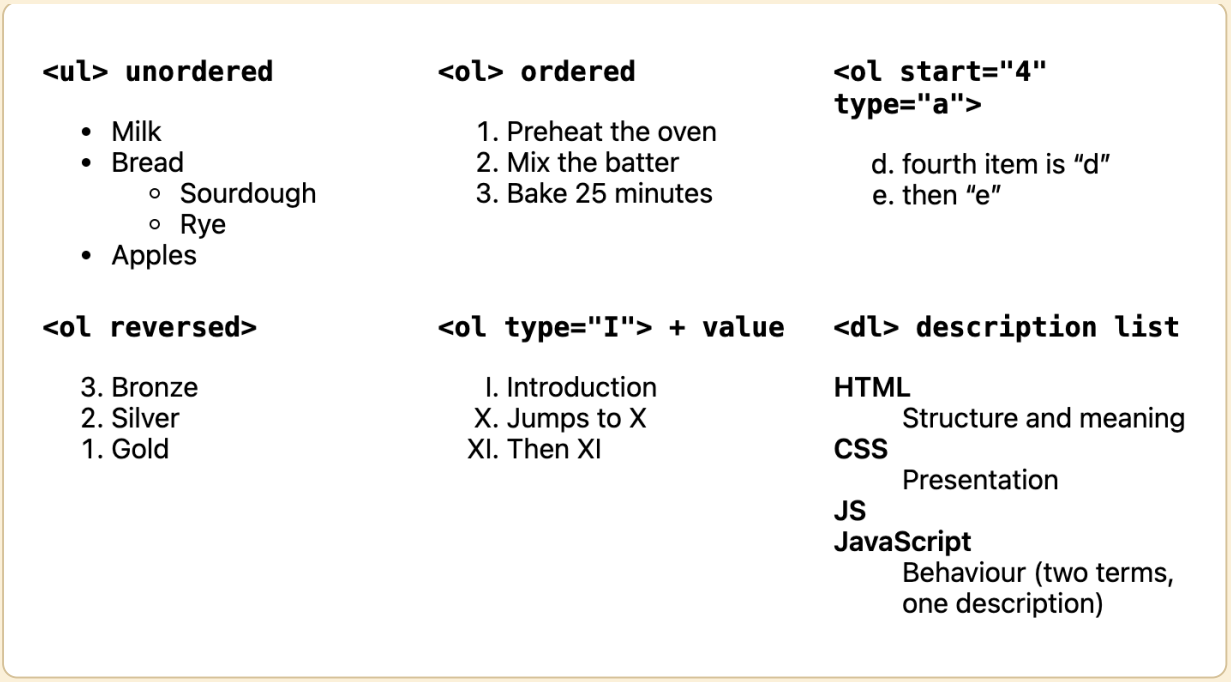


Figure 2.4 — `code/lists.html` in Chrome.

Why it matters

Screen readers announce "list, 3 items" and let users jump between lists and items, so a real list tells users how much is coming. Navigation menus are usually `<nav><ul>…</ul></nav>`.

Common mistakes

- Putting anything other than `<li>` directly inside `<ul>`/`<ol>` (a `<p>` or `<div>`) — invalid, and the list semantics break.
- Nesting a sub-list as a sibling of `<li>` instead of inside one.
- Faking lists with `<br>` and "•" characters.
- Removing bullets with `list-style: none` — fine visually, but Safari/VoiceOver (Apple's built-in screen reader) then stops announcing it as a list; add `role="list"` (an ARIA attribute that states what the element is, Chapter 3) to the `<ul>` if the count matters.

Self-check

1. Recipe steps: `ul` or `ol`? → `ol`.
2. Where does a nested list go? → Inside the parent `<li>`.
3. What pairs make up a `<dl>`? → `<dt>` terms and `<dd>` descriptions.

2.8 Links

What it is

`<a href="…">` — the *anchor* element — is the "hyper" in hypertext. Its content (text, image, even a whole card) becomes clickable and keyboard-focusable, and activating it navigates to the `href` URL.

```
<!-- code/links.html (excerpt) -->
<a href="https://developer.mozilla.org/en-US/docs/Web/HTML">MDN HTML docs</a>        <!-- absolute -->
<a href="text-blocks.html">Headings and text</a>                                     <!-- document-relative -->
<a href="../index.html">All guides</a>                                              <!-- up one folder -->
<a href="#faq">Jump to the FAQ</a>                                                  <!-- fragment: element with id="faq"
-->
<a href="mailto:help@example.org?subject=Question">Email us</a>
<a href="tel:+15555550123">Call +1 555 555 0123</a>
<a href="skeleton.html" download="page-template.html">Download the template</a>
<a href="https://example.com/" target="_blank" rel="noopener noreferrer">Example.com (opens in new tab)</a>
<a href="https://example.com/sponsor" rel="sponsored">A paid link</a>
<a>Placeholder link (no href)</a>
...
<h2 id="faq">FAQ</h2>
```

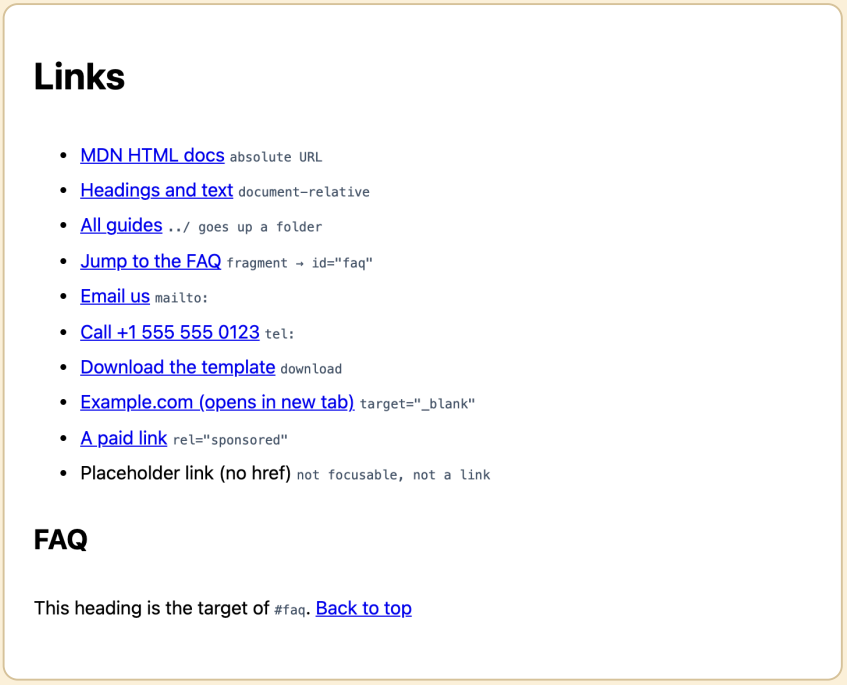


Figure 2.5 — `code/links.html`. The `<a>` without `href` is not styled or focusable as a link.

The next script opens the same file as if it lived at `https://club.example.org/guides/html/links.html` and shows how each `href` resolves:

```
// code/02_links.mjs (key parts)
const { window } = new JSDOM(fs.readFileSync('links.html'), { url: 'https://club.example.org/guides/html/links.html' });
const doc = window.document;
console.log('attribute vs property:', doc.querySelector('a[href="text-blocks.html"]').getAttribute('href'), '->',
            doc.querySelector('a[href="text-blocks.html"]').href, '\n');
for (const a of doc.querySelectorAll('a')) {
  // prints text, the resolved a.href, protocol, target, rel, download, and same-page jumps
}
```

Output:

```
attribute vs property: text-blocks.html -> https://club.example.org/guides/html/text-blocks.html

MDN HTML docs              https://developer.mozilla.org/en-US/docs/Web/HTML       https:
Headings and text          https://club.example.org/guides/html/text-blocks.html   https:
All guides                 https://club.example.org/guides/index.html              https:
Jump to the FAQ            https://club.example.org/guides/html/links.html#faq     https: same-page jump to #faq
Email us                   mailto:help@example.org?subject=Question                mailto:
Call +1 555 555 0123       tel:+15555550123                                        tel:
Download the template      https://club.example.org/guides/html/skeleton.html      https: download=page-template.html
Example.com (opens in new  https://example.com/                                    https: target=_blank rel=noopener
noreferrer
A paid link                https://example.com/sponsor                             https: rel=sponsored
Placeholder link (no href) (no href: not in document.links = false)
Back to top                https://club.example.org/guides/html/links.html#top     https: same-page jump to #top
```

The `href` *attribute* holds what you typed; the `.href` *property* is the fully resolved URL (see §1.2 for the resolution rules).

Link attributes

Attribute	What it does	Notes
`href`	Destination URL	Without it, `<a>` is a placeholder: not a link, not focusable
`target="_blank"`	Open in a new tab/window	Other values: `_self` (default), `_parent`, `_top`, or a named window
`rel`	Relationship of the target to this page	Space-separated list, see below
`download`	Download instead of navigate; value = suggested file name	Only for same-origin (same scheme, host and port), `blob:` (in-memory file made by JS) or `data:` (file contents inside the URL) URLs
`hreflang`, `type`	Hints about the target's language / MIME type (a label such as `text/html` or `image/png` naming a file's format)	Advisory
`referrerpolicy`	How much of this page's URL to send in `Referer`	e.g. `no-referrer`
`ping`	URLs to notify when the link is followed	Tracking; not supported in every browser

`rel` value	Meaning
`noopener`	The new page gets no `window.opener` reference (a JS handle back to the page that opened it), so it cannot redirect your tab ("reverse tabnabbing" — swapping your original tab for a fake page). Modern browsers apply this automatically to `target="_blank"`; writing it explicitly is harmless.
`noreferrer`	Also don't send the `Referer` header (implies `noopener`)
`nofollow`	"I don't vouch for this link" — search engines treat it as a hint
`sponsored`	Paid or affiliate link (one that earns you commission; search engines ask you to mark these)
`ugc`	User-generated content link (comments, forum posts)
`external`, `author`, `license`, `next`, `prev`	Descriptive relations

Opening new tabs — should you?

Mostly no. Users can choose to open a new tab themselves; forcing one breaks the Back button and can disorient screen-reader and mobile users. Reasonable exceptions: links that would interrupt a form or a playing video, and help pages beside a task. When you do it, say so in the link text ("(opens in new tab)") or with an icon that has a text alternative.

Links vs buttons

- A **link** goes somewhere (a URL). Keyboard: Enter.
- A **button** does something on this page (submit, open a menu, toggle). Keyboard: Enter or Space.

`<a href="#" onclick="…">` pretends to be a button: it adds `#` to the URL, scrolls to the top without JS, and screen readers announce "link". Use `<button type="button">` (Chapter 3).

Writing good link text

Link text must make sense out of context — screen-reader users often pull up a list of all links. "Click here", "read more" and bare URLs are meaningless in that list. Write "Download the 2026 price list (PDF, 2 MB)" instead of "Click here to download". This is WCAG 2.4.4 (Link Purpose (In Context)).

Real-world examples

- "Skip to main content" links at the top of big sites are fragment links (`href="#main"`) — Chapter 3.
- GitHub and MDN give every heading an `id` so you can link straight to a section.

- Shops use `tel:` links so mobile users can call with one tap.

Common mistakes

- Links with no text (only an icon) — they have no accessible name (the label a screen reader announces for an element).
- `href="www.example.com"` without `https://` — treated as a *relative* path (`/current/www.example.com`).
- Two links next to each other going to the same place (image + title of a card) — noisy for keyboard and screen-reader users; wrap both in one link.

In the interview

> **30-second answer:** "An anchor with an `href` navigates to a URL — absolute, relative, fragment or another scheme like `mailto:`. I keep link text meaningful out of context, avoid `href="#"` for actions because those should be buttons, and use `target="_blank"` sparingly. `rel` describes the relationship: `noopener` / `noreferrer` for security and privacy, `nofollow` / `sponsored` / `ugc` for search engines."

- *Why was `rel="noopener"` needed with `target="_blank"`?* → The opened page could use `window.opener.location` to swap your tab for a phishing page (a fake page that tricks you into typing passwords). Browsers now imply `noopener` for `_blank`.
- *Link or button for "Add to cart"?* → Button: it performs an action on the current page.
- *What does `download` do on a cross-origin URL (one on a different origin from the page)?* → It is ignored; the browser navigates normally.

Self-check

1. What does `href="#faq"` do? → Scrolls to the element with `id="faq"` on the same page (no request).
2. Why is "click here" bad link text? → It is meaningless out of context, e.g. in a screen reader's link list (WCAG 2.4.4).
3. Is `<a>` without `href` keyboard-focusable? → No.

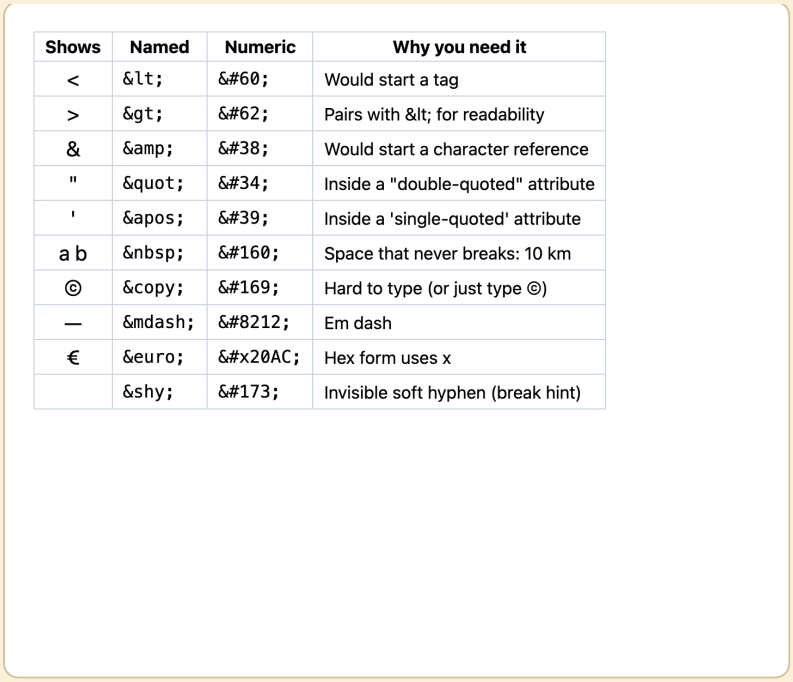
2.9 Character references and comments

What it is

Some characters have special meaning in HTML: `<` starts a tag and `&` starts a **character reference**. To show them as text you write a reference instead:

- **Named**: `<` `>` `&` `"` ` ` `©`
- **Decimal**: `<` (the Unicode code point in decimal — Unicode is the worldwide catalogue that gives every character a number, its *code point*)
- **Hexadecimal** (base 16, digits 0–9 and A–F): `<`

People often call them "entities" (strictly, *named character references*).



Shows	Named	Numeric	Why you need it
<	<	<	Would start a tag
>	>	>	Pairs with < for readability
&	&	&	Would start a character reference
"	"	"	Inside a "double-quoted" attribute
'	'	'	Inside a 'single-quoted' attribute
a b			Space that never breaks: 10 km
©	©	©	Hard to type (or just type ©)
—	—	—	Em dash
€	€	€	Hex form uses x
			Invisible soft hyphen (break hint)

Figure 2.6 — `code/entities.html`: what each reference renders as.

With `<meta charset="utf-8">` you can type almost any character directly (©, —, €, é, emoji). You only *need* references for:

Character	Where	Reference
`<`	In text	`<`
`&`	In text or attributes, when followed by something that looks like a reference	`&` (always escaping it is simplest)
`"` / `'`	Inside an attribute quoted with the same character	`"` / `'`
Invisible characters	Non-breaking space, soft hyphen, zero-width characters	` `, ``, `` — references make them visible in your source

```
// code/03_entities.mjs
import '../../../../tools_dom.mjs';
const p = document.createElement('p');
p.innerHTML = 'Fish &amp; chips &lt;b&gt; &copy; &#169; &#xA9; 10 km &mdash; café';
console.log('textContent :', JSON.stringify(p.textContent));
console.log('nbsp code   :', p.textContent.codePointAt(p.textContent.indexOf('km') - 1).toString(16));
console.log('innerHTML   :', p.innerHTML);
const q = document.createElement('p');
q.textContent = '<img src=x onerror=alert(1)> & "quotes"';     // text, not HTML
console.log('\nescaped     :', q.innerHTML);
console.log('elements    :', q.children.length, '(the tag became text, not an element)');
const r = document.createElement('p');
r.innerHTML = 'AT&T, R&D, a&b=1';
console.log('\nbare &      :', r.textContent);
const s = document.createElement('div');
s.innerHTML = '<p>price&copy=1</p><a href="?a=1&copy=2">x</a>';
console.log('in text     :', s.querySelector('p').textContent, '| in href:', s.querySelector('a').getAttribute('href'));
```

Output:

```
textContent : "Fish & chips <b> © © © 10 km — café"
nbsp code   : a0
innerHTML   : Fish &amp; chips &lt;b&gt; © © © 10 km — café

escaped     : &lt;img src=x onerror=alert(1)&gt; &amp; "quotes"
elements    : 0 (the tag became text, not an element)

bare &      : AT&T, R&D, a&b=1
in text     : price©=1 | in href: ?a=1&copy=2
```

What it shows:

1. The parser **decodes** references into ordinary characters; `©`, `©` and `©` are the same `©`.
2. ` ` is the real character U+00A0 (code `a0`), not a space; use it to keep "10 km" or "Mr. Smith" on one line.
3. When serialising (turning the DOM back into HTML text, e.g. reading `innerHTML`), the browser escapes `&`, `<`, `>` and non-breaking spaces again.
4. Setting `textContent` is safe: markup becomes text. Building HTML strings from user input and assigning them to `innerHTML` is how **cross-site scripting (XSS)** happens — an attacker gets their own script to run inside your page — covered in the JavaScript book.
5. A bare `&` usually survives, but it is fragile: about a hundred legacy names such as `&copy` are decoded even *without* a semicolon, so the text `price&copy=1` shows as `price©=1`. (Inside attributes the parser has a special rule that leaves `&copy=` alone, which is why most query strings survive.) Writing `&` removes all doubt.

Comments

```
<!-- This is a comment. It can span
     several lines. It is not displayed. -->
```

Comments stay in the DOM as comment nodes (the `01_head.mjs` output above found one) and are **visible to anyone** with View Source — never put passwords, internal URLs or rude notes in them. They cannot be nested, and `--` inside a comment is best avoided. Build tools (programs that process your source files before publishing) usually strip them from production HTML (the version served to real users).

Self-check

1. How do you display the text `<br>` on a page? → `<br>` (or set it with `textContent`).
2. Is `©` the same as `©`? → Yes, both are ©.
3. Can users see HTML comments? → Yes, in the page source.

2.10 Validation

The problem it solves

Browsers never show HTML errors. They silently repair them (Chapter 1) — so a broken page often *looks* fine to you while behaving differently for screen readers, search engines, other browsers, or your CSS and JavaScript.

Here is a page with a dozen problems (`code/broken.html`):

```
<html>
<head>
<title>Broken page</title>
</head>
<body>
<h1>Welcome</h1>
<h3>Our services</h3>
<p>Great deals <b>today <i>only</b></i>
<p id="promo">Sale!</p>
<p id="promo">Another sale!</p>
<img src="data:image/svg+xml,...">
<a href="#" onclick="go()">Click here</a>
<ul><p>Not a list item</p><li>ok</li></ul>
<h1>Also welcome</h1>
</body>
</html>
```

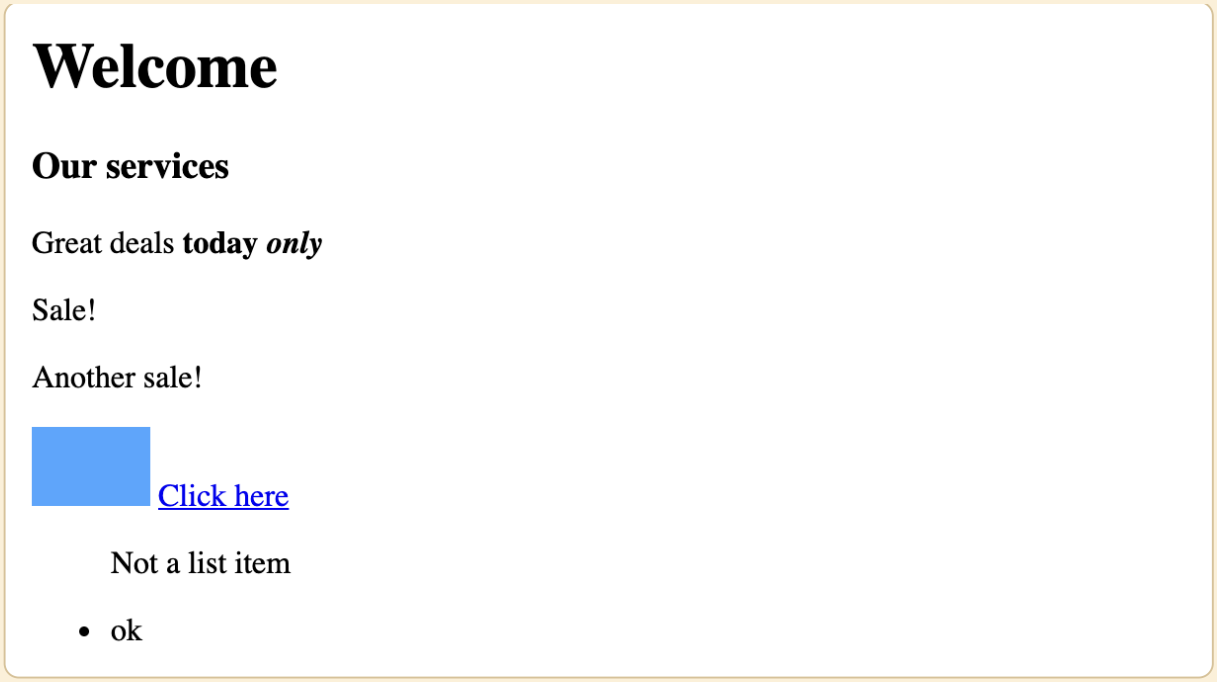


Figure 2.7 — It renders without complaint. Nothing on screen tells you about the 12 problems below.

How it works — checking

1. **The W3C Nu HTML Checker** (validator.w3.org/nu) is the reference validator: paste code, upload a file or give a URL. It reports **parse errors** (bad nesting, stray end tags) and **content-model errors** (a `<p>` inside a `<ul>`, duplicate ids, missing required attributes such as `alt`).
2. **Editor and build tools**: VS Code extensions, `html-validate` or `vnu` (the Nu checker's command-line version) in CI (continuous integration — automated checks that run every time code is pushed).
3. **Accessibility checkers** (axe, Lighthouse, WAVE — tools that scan a page for barriers to disabled users; Chapter 3) catch semantic issues validators don't: empty links, poor contrast.

To make the idea concrete, `code/04_validate.mjs` is a tiny checker that parses a file with jsdom and looks for a dozen of the most common problems:

```
// code/04_validate.mjs (key parts)
if (!/^\s*<!doctype html>/i.test(src)) problems.push('missing <!DOCTYPE html> -> quirks mode');
if (!doc.documentElement.hasAttribute('lang')) problems.push('<html> has no lang attribute');
const charset = src.search(/<meta[^>]+charset/i);
if (charset < 0 || charset > 1024) problems.push('no <meta charset> in the first 1024 bytes');
// ... viewport, title, number of h1s, heading-level jumps, duplicate ids, img without alt,
// ... href="#" / javascript: links, vague link text, non-<li> children of lists, <b><i></b> mis-nesting
lint('skeleton.html');
lint('broken.html');
```

Output:

```
skeleton.html: OK
broken.html: 12 problem(s)
  - missing <!DOCTYPE html> -> quirks mode
  - <html> has no lang attribute
  - no <meta charset> in the first 1024 bytes
  - no viewport meta (page will not scale on phones)
  - 2 <h1> elements (aim for exactly one)
  - heading jumps from h1 to h3: "Our services"
  - duplicate id="promo"
  - <img> without alt
  - fake link "Click here" (use <button>)
  - vague link text "Click here"
  - <p> directly inside <ul>
  - mis-nested tags <b><i>...</b> (parser repaired it)
```

Some of these are validity errors (duplicate id, `<p>` in `<ul>`, missing `alt`, missing doctype/title), others are best-practice or accessibility warnings (heading jumps, two `h1`s, vague link text). A real validator plus an accessibility checker covers both.

Trade-offs

Validation is not the goal in itself — a page can be valid and still inaccessible or confusing. But valid HTML is predictable across browsers and tools, and most validator errors point at real bugs. Treat errors as bugs and warnings as review items.

In the interview

> **30-second answer:** "Browsers are forgiving: the HTML parser repairs almost any markup, so errors are invisible. That is exactly why I validate — the repaired DOM may not be what I meant, and assistive technology (screen readers, switch devices and other tools disabled people use), crawlers (search-engine robots) and my CSS selectors all see the repaired tree. I use the Nu HTML Checker or html-validate in CI, plus axe or Lighthouse for accessibility."

- *Name three common validation errors.* → Duplicate ids, block elements inside `<p>`, missing `alt` on `<img>` (also: bad nesting, non-`li` children in lists).
- *Does invalid HTML affect SEO (search engine optimisation — how well a page is found and ranked)?* → Not directly as a ranking factor, but broken structure can hide content, headings or links from crawlers.

Self-check

1. Why won't the browser tell you about invalid HTML? → The spec requires it to recover silently.
2. Which tool is the official validator? → The W3C Nu HTML Checker.
3. Does a valid page guarantee accessibility? → No — add accessibility testing.

Chapter 2 TL;DR

- **Elements** = tags + content; attributes add information; void elements have no end tag; boolean attributes are on when present. Nest properly.
- Every page: `<!DOCTYPE html>` → `<html lang>` → `<head>` (charset first, viewport, unique `<title>`, description, CSS, deferred JS, icon, canonical, social tags) → `<body>`.
- **Headings** build the outline: one `h1`, don't skip levels, choose by structure not size. There is no outline algorithm for `<section>`.
- **Text blocks**: `p` for paragraphs, `br` only for meaningful breaks, `pre` keeps whitespace, `blockquote` for long quotes; whitespace otherwise collapses.
- **Inline semantics**: `strong` (importance) vs `b` (attention), `em` (stress) vs `i` (other voice), plus `mark`, `code`, `kbd`, `time`, `abbr`, `cite`, `q`… `span`/`div` have no meaning.
- **Lists**: `ul` unordered, `ol` ordered (`start`, `reversed`, `type`, `value`), `dl` for name–value pairs; nested lists go inside `li`.
- **Links**: resolve like URLs; meaningful text; `href="#"` actions should be buttons; `target="_blank"` sparingly (implies `noopener`); `rel` for security, privacy and SEO hints.
- **References**: escape `<` and `&` (`<`, `&`), ` ` for non-breaking space; `textContent` is safe, raw `innerHTML` is an XSS risk. Comments are public.
- **Validate**: browsers silently repair errors; use the Nu checker/html-validate plus accessibility tools.

Next: Chapter 3 — Semantic HTML and Accessibility.

Chapter 3 — Semantic HTML and Accessibility

A page can look perfect and still be unusable for someone who navigates with a keyboard, listens through a screen reader (software that reads the page aloud and lets blind users move around it with keys), zooms to 400 %, or cannot tell red from green. **Accessibility** (often shortened to *a11y*: "a", 11 letters, "y") means building pages that work for all of them — and the single most effective tool for it is **semantic HTML**: using the element whose meaning matches the content. This chapter shows how browsers turn your markup into an *accessibility tree*, how landmarks, headings, alt text (the text alternative for an image), ARIA (extra attributes that describe elements to assistive tools), focus management (controlling which element receives keyboard input) and form labels feed that tree, what WCAG 2.2 (the Web Content Accessibility Guidelines, the standard most laws point to) requires, and how to test.

What you'll learn

#	Topic	You will be able to…
3.1	Why semantics and accessibility	explain who benefits, what goes wrong most often, and the legal picture
3.2	Landmarks	structure a page with `header`, `nav`, `main`, `article`, `section`, `aside`, `footer`, `search`
3.3	Headings and outline	produce a heading outline screen-reader users can navigate
3.4	The accessibility tree	describe role, name, state and value, and how the name is computed
3.5	Alt text	write the right `alt` for informative, decorative, functional and complex images
3.6	ARIA	apply the first rule of ARIA and use labels, descriptions, states and live regions correctly
3.7	Keyboard and focus	keep everything reachable, visible and in order; build skip links; manage focus
3.8	Accessible forms	label, group, describe and report errors for form fields
3.9	WCAG 2.2	use POUR, the A/AA/AAA levels, the new 2.2 criteria and the contrast formula
3.10	Testing tools	combine automated checkers, a keyboard test and a screen reader

Code is in `code/`. The `.html` files are shown as screenshots. Most scripts open those files in **real headless Chrome** (Chrome without a window, driven by a script via `puppeteer-core`, a Node.js library for remote-controlling Chrome; helper `code/a11y_lib.mjs`) and print Chrome's own accessibility tree — the same data a screen reader receives. `06_mini_audit.mjs` uses jsdom (a browser-like DOM that runs in Node.js) and `dom-accessibility-api`.

3.1 Why semantics and accessibility

What it is

Semantic HTML means choosing elements for what content *is* — a `<button>` for a button, `<nav>` for navigation, `<h2>` for a section heading — rather than for how it looks. **Accessibility** means people with disabilities can perceive, understand, navigate and interact with the page. The two are linked because browsers expose the semantics of your HTML to **assistive technologies** (AT): screen readers, screen magnifiers (enlarge part of the screen), voice control, switch devices (one or two big buttons that replace a keyboard and mouse), braille displays (a strip of raised pins that shows text as braille).

Analogy: semantic HTML is the signage in a building. Sighted visitors may find the exit by looking around; a blind visitor depends on the signs being real (braille, announced floors). A `<div>` styled to look like a button is a painted-on door.

Who benefits

Group	Examples	What they rely on
Blind and low-vision users	Screen readers (JAWS, NVDA, VoiceOver, TalkBack), magnifiers, high zoom	Roles and names, headings, alt text, contrast (difference in brightness between text and background), reflow (content rewrapping to fit a narrow, zoomed view)
Motor impairments	Keyboard only, switch access, voice control ("click Submit")	Focusable controls, visible focus, big targets, visible labels that match names
Deaf and hard of hearing	Captions (on-screen text of what is said), transcripts (the full text of an audio/video)	Media alternatives (Chapter 4)
Cognitive and learning disabilities	Clear language, consistent layout, no time pressure	Predictable structure, clear errors, no memory tests
Everyone, sometimes ("situational")	Broken arm, bright sunlight, noisy train, slow network, old age	All of the above

The World Health Organization estimates that about 1.3 billion people — roughly one in six — live with a significant disability.

Why it matters

- **It is mostly not happening.** The WebAIM Million survey of the top one million home pages (February 2026) found detectable WCAG failures on 95.9 % of them, about 56 errors per page. Six problem types made up 96 % of errors: low-contrast text (83.9 % of pages), missing image alt text, missing form labels, empty links, empty buttons, and missing page language. Every one of those is fixed with the basics in this chapter; §3.10 builds a small checker for exactly those six.
- **It is often the law.** The US ADA (Americans with Disabilities Act) and Section 508 (the US rule for government websites), the European Accessibility Act (applying since 28 June 2025) and similar laws elsewhere require accessible digital services, usually by reference to WCAG level AA.
- **It is better engineering.** Semantic HTML gives you keyboard support, focus handling and screen-reader output *for free*; it also improves SEO (search engine optimisation — being found in search results), reader modes, voice assistants, automated testing (Testing Library, a popular testing tool, finds elements by role — see the React book), and code readability.

Self-check

1. What is assistive technology? → Software/hardware that helps people with disabilities use computers: screen readers, magnifiers, switch devices, voice control.
2. What is the most common accessibility failure on the web? → Low-contrast text.
3. Why does semantic HTML help accessibility? → Browsers expose its built-in roles, names, states and keyboard behaviour to assistive technology automatically.

3.2 Landmarks

What it is

Landmarks are the big regions of a page — banner, navigation, main content, sidebar, footer. Screen-reader users can list them and jump straight to one (e.g. NVDA's `D` key, VoiceOver's rotor — a menu that lists headings, links or landmarks to jump to), the way a sighted user glances at a layout. HTML's **sectioning elements** (the elements that mark out large page regions) create landmarks automatically. The **role** in the table is the *kind* of thing a screen reader announces it as:

Element	Landmark role (ARIA)	Rules
`<header>`	`banner` — only when it is *not* inside `article`, `aside`, `main`, `nav` or `section`	Site header: logo, site nav, search. Inside an article it is just that article's header.
`<nav>`	`navigation`	Major navigation blocks only (not every group of links). With several, label each: `aria-label="Main"`, `"Breadcrumb"`
`<search>`	`search`	Wraps a search/filter form. Baseline 2023, widely available since April 2026. (Before it existed: `<form role="search">`.)
`<main>`	`main`	The primary content, **one visible per page**; target of the skip link
`<aside>`	`complementary`	Related but separable content: sidebar, related links, pull quote (a short quote repeated in large type as decoration)
`<footer>`	`contentinfo` — only when not inside `article`, `aside`, `main`, `nav` or `section`	Site footer: copyright, contact, legal links
`<section>`	`region` — **only if it has an accessible name** (the label a screen reader reads; here via `aria-labelledby` pointing at its heading)	A thematic group with its own heading
`<form>`	`form` — only if it has an accessible name	
`<article>`	`article` (a document structure role, not a landmark)	A self-contained piece that would make sense on its own: blog post, product card, comment, news story

`<section>` vs `<article>` vs `<div>`:

- **article** — could be syndicated (republished on other sites or in a feed) or reused elsewhere by itself (a post, a comment, a card).
- **section** — a chunk of something bigger that has its own heading (a chapter, a tab panel, "Reviews" on a product page).
- **div** — no meaning; just a wrapper for layout or styling. When unsure between `section` and `div`, ask "would this appear in a table of contents?" If not, use `div`.

Example

`code/landmarks.html` is a small news page. The yellow tags are visual labels for this book (marked `aria-hidden="true"` so they don't pollute the names):

```
<a class="skip" href="#main">Skip to main content</a>
<header>
  <a class="logo" href="/">Mountain Club</a>
  <nav aria-label="Main">
    <ul>
      <li><a href="/">Home</a></li>
      <li><a href="/news" aria-current="page">News</a></li>
      <li><a href="/events">Events</a></li>
    </ul>
  </nav>
  <search>
    <form action="/search"><label for="q">Search</label> <input id="q" name="q" type="search"></form>
  </search>
</header>
<div class="wrap">
  <main id="main">
    <h1>Trail news</h1>
    <article>
      <header>
        <h2>Ridge trail reopens</h2>
        <p>By Sam Lee · <time datetime="2026-09-20">20 September 2026</time></p>
      </header>
      <p>After three months of repairs, the ridge trail is open again.</p>
      <section aria-labelledby="h-what">
        <h3 id="h-what">What changed</h3>
        <p>New steps on the steep part and two rest benches.</p>
      </section>
      <footer><p>Filed under <a href="/tags/trails">trails</a></p></footer>
    </article>
  </main>
  <aside aria-labelledby="h-side">
    <h2 id="h-side">Upcoming</h2>
    <ul><li>Night run, Fri</li><li>Hill repeats, Sun</li></ul>
  </aside>
</div>
<footer><small>© 2026 Mountain Club</small></footer>
```

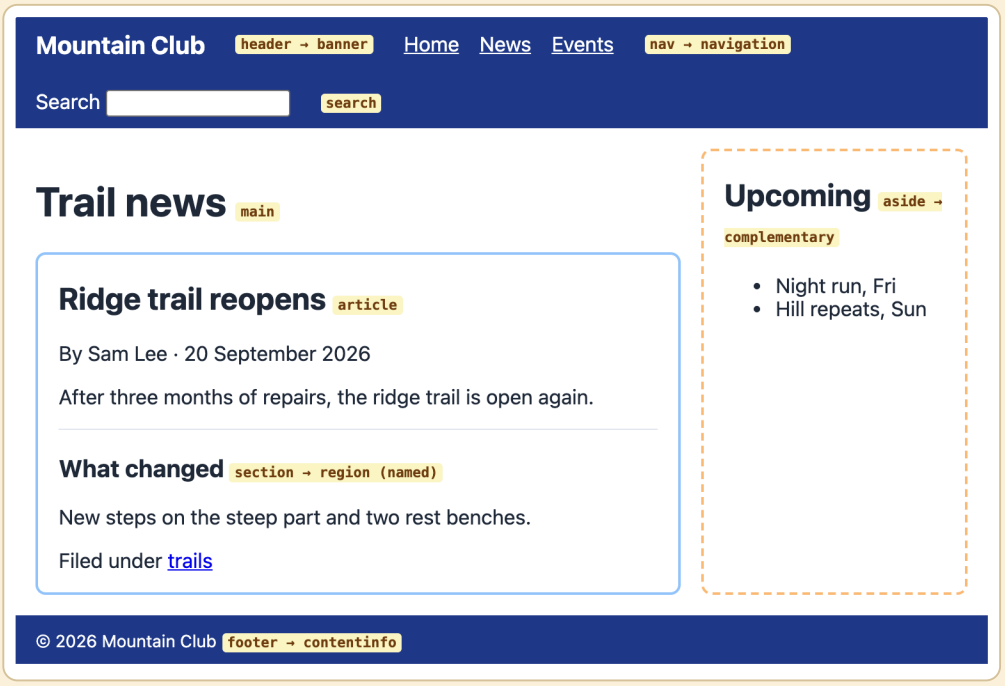


Figure 3.1 — `code/landmarks.html`. The layout is CSS; the landmarks come from the elements.

Now the same page as Chrome's accessibility tree (`code/01_landmarks_tree.mjs`):

```
import { openPage, printTree, fullTree } from './a11y_lib.mjs';
const { browser, page } = await openPage('landmarks.html');
printTree(await fullTree(page), { skipRoles: ['StaticText', 'generic', 'none', 'InlineTextBox', 'LineBreak', 'ListMarker',
'LabelText', 'listitem', 'time'] });
const outline = await page.$$eval('h1,h2,h3,h4,h5,h6', hs => hs.map(h => '  '.repeat(+h.tagName[1] - 1) +
h.tagName.toLowerCase() + ' ' + h.firstChild.textContent.trim()));
console.log('\nHeading outline:\n' + outline.join('\n'));
```

Output:

```
RootWebArea "Trail news — Mountain Club"
  link "Skip to main content"
  banner
    link "Mountain Club"
    navigation "Main"
      list
        link "Home"
        link "News"
        link "Events"
    search
      form
        searchbox "Search"
  main
    heading "Trail news"  level=1
    article
      sectionheader
        heading "Ridge trail reopens"  level=2
      region "What changed"
        heading "What changed"  level=3
      sectionfooter
        link "trails"
  complementary "Upcoming"
    heading "Upcoming"  level=2
    list
  contentinfo

Heading outline:
h1 Trail news
  h2 Ridge trail reopens
    h3 What changed
  h2 Upcoming
```

Notice:

1. The top-level `<header>` became `banner` and the top-level `<footer>` became `contentinfo`; the article's own header and footer did *not* (Chrome calls them `sectionheader`/`sectionfooter`, which are not landmarks).
2. `<section>` became a `region` landmark because `aria-labelledby` gave it a name; `<aside>` got its name "Upcoming" the same way.
3. The `<nav>` is announced as "Main navigation" thanks to `aria-label`; `aria-current="page"` tells users which link is the current page. (`Breadcrumb`, used below, is the trail of links such as Home › Shop › Boots that shows where a page sits in a site.)
4. Everything except the skip link sits inside a landmark — a good rule to aim for, so nothing is "orphaned" outside the regions users jump between.

Common mistakes

- "Div soup": a page built only from `<div class="header">`, `<div class="nav">` — zero landmarks.
- Several `<main>` elements, or none.
- Wrapping every block in `<section>` "because HTML5" — sections without headings add noise, not structure.
- Several unlabelled `<nav>`s: users hear "navigation, navigation, navigation".

Real-world examples

GOV.UK, the BBC and GitHub use exactly this skeleton: skip link → `header` (banner) with `nav` → `main` → `footer` (contentinfo). Try it: open a site, turn on VoiceOver (Cmd+F5 on macOS) and open the rotor (Ctrl+Option+U) → Landmarks.

Self-check

1. Which element makes the `main` landmark, and how many per page? → `<main>`, one visible.
2. When is `<section>` a landmark? → Only when it has an accessible name (e.g. `aria-labelledby` its heading).
3. Is a `<header>` inside an `<article>` a banner? → No.
4. `<article>` or `<section>` for a product card in a grid? → `<article>` — it stands on its own.

3.3 Headings and the outline

Headings are the other main navigation tool: screen-reader users press `H` (NVDA/JAWS) or use the rotor to jump heading to heading, or pull up a list of all headings as a table of contents. The outline printed above is exactly that list. The rules from §2.4 apply: one `h1`, don't skip levels going down, choose levels by structure, and remember that `<section>` does not reset levels (the old "outline algorithm" was never implemented and has been removed from the spec).

Two accessibility-specific points:

- Each landmark region usually starts with a heading, which is why naming a `section` or `aside` with `aria-labelledby` pointing at its heading is so common.
- If a region has no visible heading (a site's main nav, for example), give it a label with `aria-label` instead of adding a fake heading.

Self-check

1. How do screen-reader users skim a page? → By landmarks and by headings (and by links/form fields lists).
2. Does starting a new `<article>` reset heading levels? → No; levels are what the numbers say.

3.4 The accessibility tree

What it is

Alongside the DOM, browsers build an **accessibility tree**: a simplified tree containing only what matters to assistive technology, exposed through the operating system's accessibility API (API = application programming interface, a defined way for one program to ask another for data; here UI Automation on Windows, NSAccessibility on macOS, AT-SPI on Linux). Screen readers never read your HTML; they read this tree.

Each node has:

Property	Question it answers	From
Role	What is it? (button, link, heading, textbox, checkbox, navigation…)	The element's implicit role (the one it has automatically, e.g. `<button>` → button), or an ARIA `role`
Name	What is it called? ("Save", "Search", "Close")	Content, `<label>`, `alt`, `aria-label`, `aria-labelledby`…
Description	Extra help text	`aria-describedby`, sometimes `title`
State / properties	What condition is it in? (checked, expanded, disabled, invalid, required, level 2, current page)	Native attributes (`checked`, `disabled`, `required`) or ARIA states
Value	Current value of a field, slider or progress bar	The control's value

A screen reader turns that into speech such as: *"Email, edit text, required, invalid entry, We only use it for your receipt."*

How the accessible name is computed

The W3C *Accessible Name and Description Computation* spec defines an order of precedence. Simplified, the first of these that yields text wins:

1. `aria-labelledby` — text of the referenced element(s), in order (can combine several ids).
2. `aria-label` — a string on the element.
3. The native labelling mechanism: `<label>` for form fields, `alt` for images, `<caption>` for tables, `<legend>` for fieldsets, `<figcaption>` for figures.
4. The element's content, for roles that allow "name from content" (buttons, links, headings, cells…).
5. `title` (and, for inputs, `placeholder`) as a last resort.

Hidden content (`display: none`, `hidden`, `aria-hidden="true"`) is left out — unless you reference it directly with `aria-labelledby`.

The examples in `code/aria.html` (discussed in §3.6) show each rule; here is what Chrome computed (`code/03_names_and_states.mjs`):

```
--- aria.html
RootWebArea "ARIA patterns"
  StaticText "1. Icon-only button:"
  button "Close"
    StaticText "×"
  StaticText "2."
  StaticText "Shopping cart"
  StaticText ":"
  button "Shopping cart 3 items"
  StaticText "3."
  StaticText "Password"
  textbox "Password"  description="At least 12 characters."
  StaticText "At least 12 characters."
  StaticText "4."
  button "Shipping details"  expanded=true
  StaticText "Ships in 2 business days."
  StaticText "5."
  navigation "Breadcrumb"
    link "Home"
    StaticText "›"
    link "Shop"
    StaticText "›"
    link "Boots"
  StaticText "6."
  button "Add to cart"
  status
  StaticText "7."
  StaticText "Rated 4 out of 5"
  StaticText "(stars hidden, text for screen readers)"

after clicking "Shipping details" and "Add to cart": {
  expanded: 'false',
  panelHidden: true,
  status: 'Added. Cart has 4 items.'
}
```

- The "×" button's name is **"Close"** (`aria-label` beat the content).
- The cart button's name combines two elements via `aria-labelledby`: **"Shopping cart 3 items"**.
- The password field's name comes from its `<label>`; its **description** comes from `aria-describedby`.
- The stars were hidden with `aria-hidden="true"`, so only the visually hidden text "Rated 4 out of 5" is in the tree.

Inspecting it yourself

Chrome DevTools → Elements → **Accessibility** pane (or enable the full-page accessibility tree view); Firefox DevTools → **Accessibility** panel; Safari → Elements → Node → Accessibility. These show role, name, and states for any element.

In the interview

30-second answer: "Browsers derive an accessibility tree from the DOM — each node has a role, an accessible name, maybe a description, states and a value — and expose it to screen readers through OS accessibility APIs. Native elements fill this in correctly by default; ARIA can override or add to it. The name comes, in order, from `aria-labelledby`, `aria-label`, native labels like `<label>` or `alt`, the content, and finally `title` or placeholder."

- *Does CSS affect the accessibility tree?* → Yes: `display: none` and `visibility: hidden` remove nodes; generated content (text added by CSS with `::before`/`::after`, Chapter 6) can become part of names; `opacity: 0` does not hide anything.
- *Can a screen reader see a `<div>` with a click handler (a JavaScript function run when the element is clicked) as a button?* → No; it has no role, no name-as-button, and is not focusable (demonstrated in §3.7).

Self-check

1. Name the four main things a node in the accessibility tree has. → Role, name, state(s)/properties, value (plus description).
2. A button has text "×" and `aria-label="Close"`. What is its name? → "Close".
3. Where do screen readers get their information from? → The accessibility tree via the OS accessibility API, not the raw HTML.

3.5 Alt text

What it is

The `alt` attribute gives an image its **text alternative**: what a screen reader speaks, what search engines index, and what shows if the image fails to load. It is WCAG success criterion (a single testable rule, §3.9) 1.1.1 (Non-text Content, level A) and one of the six most common failures.

How to decide — ask "what is this image *for*?"

Ask in order	If yes →
1. Is it purely decorative, or does nearby text already say the same thing?	`alt=""` (empty — the image is ignored)
2. Is it the content of a link or button?	Describe the **destination or action** ("Mountain Club home", "Search")
3. Is it complex (chart, diagram, map)?	A short `alt` naming it, plus the full information in nearby text or a `<figcaption>`
4. Is it an image of text?	`alt` repeats the text — better still, use real text styled with CSS
5. Otherwise (a photo or illustration that carries meaning)	Describe what matters about it **in this context**, briefly

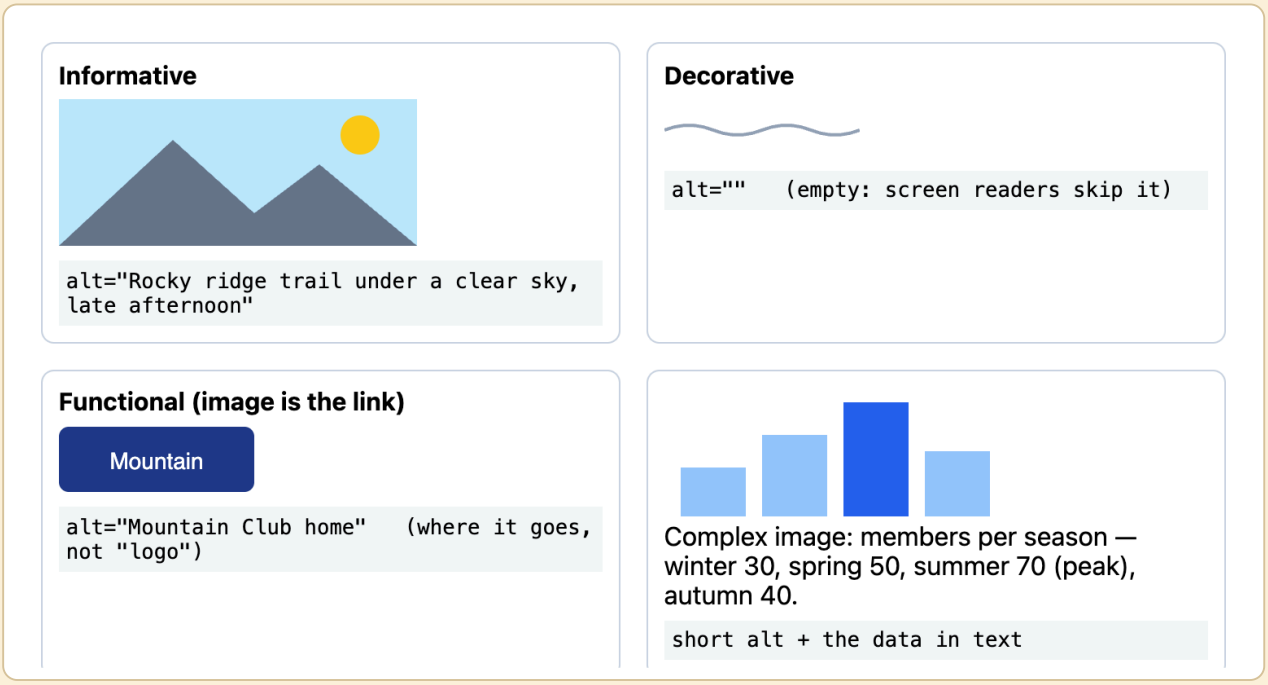


Figure 3.2 — `code/alt-text.html`: the four most common cases.

```
<img src="ridge.jpg" alt="Rocky ridge trail under a clear sky, late afternoon">       <!-- informative -->
<img src="squiggle.svg" alt="">                                                        <!-- decorative -->
<a href="/"><img src="logo.svg" alt="Mountain Club home"></a>                          <!-- functional -->
<figure>                                                                                 <!-- complex -->
  <img src="members.svg" alt="Bar chart of club members per season">
  <figcaption>Complex image: members per season — winter 30, spring 50, summer 70 (peak), autumn 40.</figcaption>
</figure>
```

Chrome's view of those images (from `03_names_and_states.mjs`):

```
--- alt-text.html (images only)
image "Rocky ridge trail under a clear sky, late afternoon"
image "Mountain Club home"  (inside link "Mountain Club home")
image "Bar chart of club members per season"
decorative <img alt=""> in tree? false
```

The empty-alt image is gone from the tree entirely — exactly what you want for decoration. The linked image gives its name to the link.

Rules of thumb

- **Missing** `alt` is different from **empty** `alt`. Missing: screen readers may read the file name ("IMG underscore 2041 dot jpg"). Empty: the image is skipped.
- Don't start with "Image of…" or "Picture of…" — the role already says "image". (Do say "Photo", "Illustration", "Screenshot" when the *type* matters.)
- Keep it short (a sentence or so); describe what matters **in this context**. The same photo of a person might be "Dr. Ana Ruiz" on a staff page and "Ana Ruiz holding the award trophy" in a news story.
- Don't repeat the caption or adjacent text word for word; if the text already says it, use `alt=""`.
- Icons next to visible text ("🗑 Delete") are decorative; icon-only buttons need a name on the button (`aria-label`), not just on the icon.
- CSS background images have no alt: never put meaningful content in them.
- Inline `<svg>` needs `role="img"` plus `aria-label` (or a `<title>` child) when it is informative, `aria-hidden="true"` when decorative (Chapter 4).

Self-check

1. A decorative divider image: what `alt`? → `alt=""`.
2. A magnifying-glass image that is the only content of a search button? → `alt="Search"` (the action).
3. What is the difference between no `alt` and `alt=""`? → No alt = screen readers may announce the file name; empty = image ignored.

3.6 ARIA

What it is

WAI-ARIA (Web Accessibility Initiative — Accessible Rich Internet Applications; usually just "ARIA") is a set of HTML attributes that change or add to what the accessibility tree says about an element:

- **Roles** — `role="button"`, `role="tablist"`, `role="dialog"`, `role="alert"`…
- **Properties** — mostly stable facts: `aria-label`, `aria-labelledby`, `aria-describedby`, `aria-controls`, `aria-required`, `aria-haspopup`…
- **States** — things that change with interaction: `aria-expanded`, `aria-checked`, `aria-pressed`, `aria-selected`, `aria-invalid`, `aria-disabled`, `aria-current`, `aria-hidden`…

The crucial fact: **ARIA only changes what is announced. It adds no behaviour** — no focusability, no keyboard handling, no styling. `role="button"` on a `<div>` makes a screen reader *say* "button"; you still have to make it focusable and respond to Enter and Space yourself.

The first rule of ARIA (and the other four)

The W3C "Using ARIA" note sets five rules. In short:

1. **Use native HTML first.** If an element or attribute with the meaning and behaviour you need exists (`<button>`, `<a href>`, `<input type="checkbox">`, `<details>`, `<dialog>`, `required`, `disabled`), use it instead of

re-creating it with ARIA.

2. **Don't change native semantics** unless you really must (`<h2 role="tab">` is wrong; put a `<button role="tab">` inside the `<h2>` or restructure).
3. **Every interactive ARIA control (a *widget* — a custom interactive component such as a tab set or menu) must work with the keyboard.**
4. **Never put `role="presentation"`/`"none"` or `aria-hidden="true"` on a focusable element** — users would tab to something that "doesn't exist".
5. **Every interactive element must have an accessible name.**

A popular summary: *no ARIA is better than bad ARIA*. WebAIM's surveys consistently find that home pages *with* ARIA have more detected errors on average than pages without it — usually because it is used to patch div-based widgets instead of using native elements (the built-in HTML elements such as `<button>`).

Useful ARIA — the patterns in `aria.html`

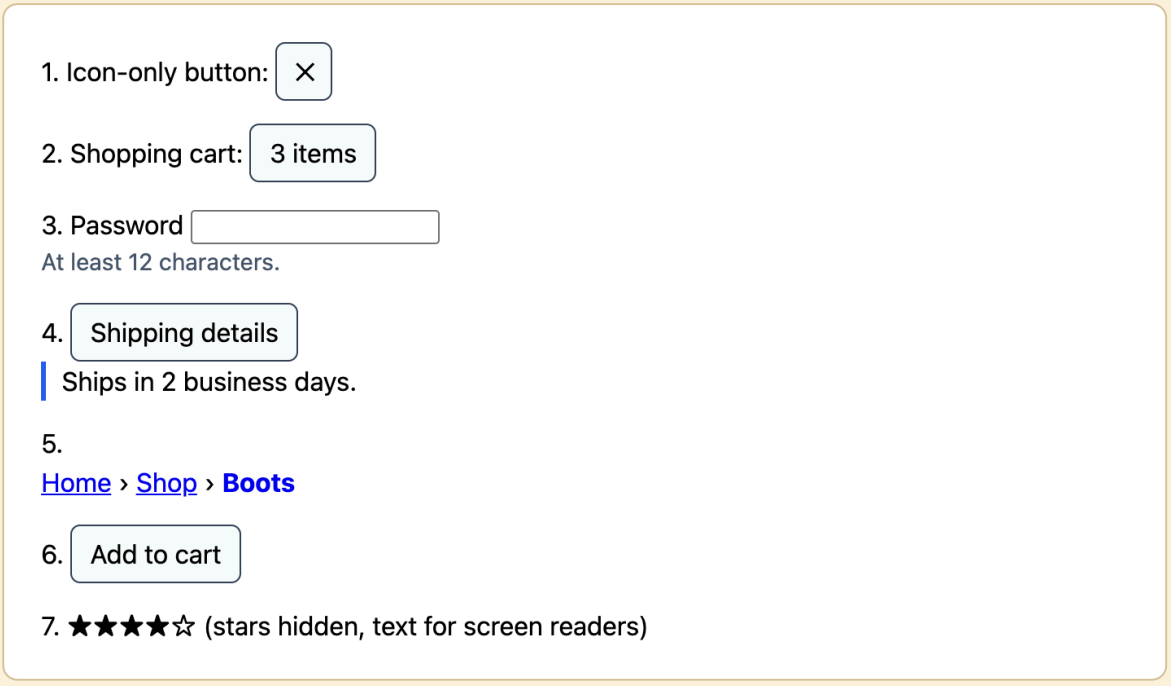


Figure 3.3 — `code/aria.html`. Visually ordinary — the differences are all in the accessibility tree shown in §3.4.

```
<!-- 1. Name an icon-only control -->
<button type="button" aria-label="Close">x</button>

<!-- 2. Build a name from visible text elsewhere (ids, in order) -->
<span id="cart-h">Shopping cart</span>:
<button type="button" aria-labelledby="cart-h cart-n"><span id="cart-n">3 items</span></button>

<!-- 3. Add help text as a description -->
<label for="pw">Password</label> <input id="pw" type="password" aria-describedby="pw-hint">
<div id="pw-hint">At least 12 characters.</div>

<!-- 4. Disclosure: state that JavaScript keeps in sync -->
<button type="button" aria-expanded="true" aria-controls="panel" id="toggle">Shipping details</button>
<div id="panel">Ships in 2 business days.</div>

<!-- 5. Mark the current item in a set of links -->
<nav aria-label="Breadcrumb"> … <a href="/shop/boots" aria-current="page">Boots</a></nav>

<!-- 6. Live region: announce changes without moving focus -->
<button type="button" id="add">Add to cart</button> <span role="status" id="live"></span>

<!-- 7. Hide decoration, provide a text equivalent -->
<span aria-hidden="true">★★★★☆</span><span class="visually-hidden">Rated 4 out of 5</span>
```

```
// the only JavaScript: keep aria-expanded true to the UI, and write into the live region
t.addEventListener('click', () => { const open = t.getAttribute('aria-expanded') === 'true';
  t.setAttribute('aria-expanded', String(!open)); p.hidden = open; });
document.getElementById('add').addEventListener('click', () => { live.textContent = 'Added. Cart has 4 items.'; });
```

Attribute	Use when	Watch out
`aria-label`	No visible text exists (icon buttons, several `<nav>`s)	Not translated by some translation tools; voice-control users can't see it — prefer visible text when possible
`aria-labelledby`	Visible text elsewhere already names it	Takes ids; overrides everything else
`aria-describedby`	Hints, format rules, error messages	Read after the name, sometimes after a pause
`aria-expanded`	Buttons that show/hide something (menus, accordions — stacks of headings that expand to reveal content)	Must be updated by your script
`aria-current`	Current page/step/date in a set	Values: `page`, `step`, `location`, `date`, `time`, `true`
`aria-hidden="true"`	Decorative or duplicated content	Hides the whole subtree; never on focusable things
`aria-live` / `role="status"` / `role="alert"`	Content that changes without a page load (toasts — small pop-up notices that vanish on their own — counters, errors)	The region must **exist before** the text changes; `status` = polite (waits), `alert` = assertive (interrupts) — use `alert` sparingly
`aria-invalid`, `aria-required`	Form errors / required fields when native attributes don't fit	Native `required` already implies `aria-required`

The **visually-hidden** pattern (often a `.visually-hidden` or `.sr-only` class) hides text on screen but keeps it in the accessibility tree:

```
.visually-hidden { position: absolute; width: 1px; height: 1px; overflow: hidden;
                   clip-path: inset(50%); white-space: nowrap; }
```

When not to use it

- To re-create a native control (`<div role="checkbox">` instead of `<input type="checkbox">`).
- To "fix" a heading structure (`<div role="heading" aria-level="2">`) when you can edit the HTML.
- `aria-label` on a `<div>` or `<span>` with no role: many screen readers ignore names on generic elements.
- `role="application"` — it switches off the screen reader's normal reading keys; almost never appropriate.

> **Key term — live region:** A *live region* is an element whose text changes are read out automatically by screen readers, even though focus stays where it is. You mark it with `aria-live`, `role="status"` or `role="alert"`, and later change its text with JavaScript. It is how a blind user learns that "Item added" or "3 results found" without being moved around the page.

In the interview

> **30-second answer:** "ARIA lets me set roles, states and properties in the accessibility tree when HTML alone can't express something — like `aria-expanded` on a disclosure button (a button that shows or hides a panel), `aria-live` regions, or naming an icon button. But the first rule of ARIA is to use native HTML when it exists, because ARIA only changes semantics: it adds no focus, keyboard or behaviour. A `<button>` gives me all of that for free."

- *`aria-label` vs `aria-labelledby`?* → `aria-label` is a string on the element; `aria-labelledby` points at other elements' visible text (and wins if both exist).
- *`role="status"` vs `role="alert"`?* → Both are live regions; status is polite (announced when the user is idle), alert is assertive (interrupts) — for urgent errors only.
- *What does `aria-hidden="true"` do?* → Removes the element and its descendants from the accessibility tree; it doesn't hide it visually or stop focus.

Self-check

1. Does `role="button"` make a `<div>` keyboard accessible? → No — it only changes the announced role.
2. How do you announce "Saved" after an AJAX save (saving with a background request, without reloading the page) without moving focus? → Update the text of an existing `role="status"` (polite live region).
3. What is the first rule of ARIA? → Use a native HTML element/attribute with the needed semantics and behaviour instead of ARIA whenever possible.

3.7 Keyboard and focus

What it is

Many people never use a mouse: keyboard-only users, screen-reader users (screen readers are keyboard-driven), switch and voice users whose tools emulate keys. **Focus** is the element that receives keyboard input; **Tab** moves focus forward, **Shift+Tab** backwards, **Enter** activates links and buttons, **Space** activates buttons and toggles checkboxes, **arrow keys** move within composite widgets (controls made of several parts, such as radio groups, menus, tabs, selects), **Escape** closes dialogs and popups.

WCAG requires that everything work from a keyboard (2.1.1), with no keyboard traps (2.1.2 — places where Tab cannot get you out again), in a logical order (2.4.3), with a visible focus indicator (2.4.7) that is not hidden behind sticky headers (2.4.11, new in 2.2 — headers that stay fixed at the top while you scroll).

The div-button problem, measured

`code/div-vs-button.html` has three controls that look identical:

```
<div class="btn" onclick="save()">Save (div)</div>
<button class="btn" type="button" onclick="save()">Save (button)</button>
<div class="btn" role="button" tabindex="0" onclick="save()">Save (div + role + tabindex)</div>
<p id="out" role="status"></p>
```

Figure 3.4 — Pixel-identical. The screenshot cannot tell you which one works.

`code/02_div_vs_button.mjs` asks Chrome, then presses keys like a keyboard user:

```
printTree(await fullTree(page));
for (let i = 1; i <= 4; i++) { await page.keyboard.press('Tab'); /* print document.activeElement */ }
// focus the <button>, press Enter, then Space; focus the role=button div, press Enter; print the status text
```

Output:

```
Accessibility tree:
RootWebArea "div vs button"
  StaticText "All three look identical. Only the middle one is a real button."
  StaticText "Save (div)"
  button "Save (button)"
  button "Save (div + role + tabindex)"
  status

Pressing Tab 4 times:
  Tab 1 -> Save (button)
  Tab 2 -> Save (div + role + tabindex)
  Tab 3 -> (body — focus left the page)
  Tab 4 -> Save (button)

button     + Enter: status = "Saved!"

button     + Space: status = "Saved!"

div + role + Enter: status = ""
```

What this proves:

1. The plain `<div>` is **just text** to a screen reader and can **never be reached** with Tab.
2. The `<button>` is announced as a button, is focusable, and works with **both Enter and Space** — all for free.
3. The `div` with `role` and `tabindex` is announced and focusable, but **pressing Enter does nothing**: `onclick` on a div only fires for pointer clicks (mouse, touch, pen), never for keys. To finish it you'd need a `keydown` handler (code that runs when a key is pressed) for Enter *and* Space, disabled-state handling, and form-submission behaviour. Or you could just use `<button>`.

`tabindex`

Value	Effect	Use
(none)	Natively interactive elements (`a[href]`, `button`, `input`, `select`, `textarea`, `summary`…) are focusable in DOM order	Default — keep DOM order = visual order
`tabindex="0"`	Adds a non-interactive element to the Tab order at its DOM position	Custom widgets (with a role and keyboard handlers), scrollable regions
`tabindex="-1"`	Focusable by script (`el.focus()`) or link target, but not by Tab	Skip-link targets, headings you move focus to, items inside roving-focus widgets (only one item is tabbable; arrows move between the rest), dialog containers
`tabindex="1"` or more	Jumps ahead of everything else in the Tab order	**Never** — it breaks the natural order and is nearly impossible to maintain

Also keep the **DOM order** (the order elements appear in the HTML source) **equal to the visual order**: CSS `order`, `flex-direction: row-reverse`, grid placement and absolute positioning move boxes visually but not in the Tab sequence (Chapters 8–9).

Visible focus

Browsers draw a focus ring by default. The most common way to break keyboard access is `outline: none` in a reset stylesheet (a starter CSS file that removes browsers' default styles). Instead, style it with **`:focus-visible`**, which matches when the browser decides a focus indicator is useful (keyboard navigation) but not for mouse clicks on buttons:

```
:focus-visible { outline: 3px solid #c2410c; outline-offset: 2px; }
.bad:focus { outline: none; }     /* never do this without a replacement */
```

For the ring itself, aim for at least 3:1 contrast (a ratio of brightness, explained in §3.9) against the background (non-text contrast, 1.4.11) — the dark orange here is about 5:1 on white, whereas a light amber would be around 2:1.

Skip links

Keyboard users would otherwise have to Tab through the whole header on every page. A **skip link** is the first focusable element and jumps to `<main>`:

```
<a class="skip" href="#main">Skip to main content</a>
...
<main id="main" tabindex="-1">…</main>
```

```
.skip { position: absolute; left: 1rem; top: -3rem; }   /* off-screen until focused */
.skip:focus { top: 0; }
```

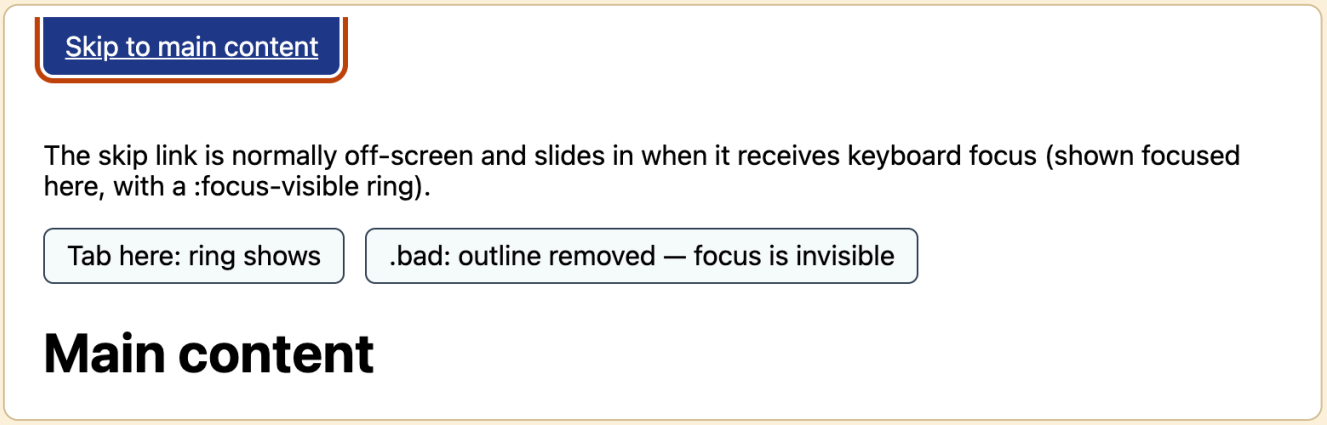


Figure 3.5 — `code/focus.html` with the skip link focused (the page focuses it on load for this screenshot). Without focus it is off-screen.

The skip link satisfies WCAG 2.4.1 (Bypass Blocks); landmarks and headings help too.

Managing focus in dynamic UIs

- **Opening a dialog**: move focus into it; keep Tab inside it while open; Escape closes; return focus to the button that opened it. The native `<dialog>` element opened with `showModal()` makes the rest of the page inert and handles Escape for you.
- `inert` **attribute**: makes a whole subtree unfocusable and hidden from AT — for content behind a custom modal (a dialog that blocks the page behind it) or an off-canvas menu (a menu that slides in from the screen edge).
- **Single-page app navigation** (in a single-page app, JavaScript swaps the page content instead of loading a new HTML page): after a route change (moving to a new "page" URL inside the app), move focus to the new page's `<h1>` (with `tabindex="-1"`) or announce the change; otherwise screen-reader users don't know anything happened (see the React book).
- **Deleting the focused item**: move focus to a sensible neighbour, not the top of the page.
- **Target size**: WCAG 2.2 (2.5.8, AA) asks for pointer targets (things you click or tap with a mouse, finger or pen) of at least **24×24 CSS pixels** (or enough spacing around smaller ones).

Common mistakes

- `outline: none` with no replacement.
- Clickable `div`s and `span`s; links used as buttons and vice versa.
- Positive `tabindex`.
- Custom dropdowns that can't be opened with the keyboard or trap focus.
- Hover-only menus that never open on focus.

In the interview

30-second answer: "Everything interactive must be reachable and operable by keyboard, in a logical order, with a visible focus indicator. Native elements give me that; I only use `tabindex=0` for genuine custom widgets and `-1` for programmatic focus targets, never positive values. I style focus with `:focus-visible` instead of removing outlines, add a skip link, and manage focus explicitly for dialogs and route changes."

- *Why not* `tabindex="5"`*?* → It pulls the element ahead of all `tabindex=0`/native elements, creating an order that no longer matches the page.
- `:focus` *vs* `:focus-visible`*?* → `:focus` matches any focus; `:focus-visible` matches when the browser heuristics (built-in rules of thumb) say a visible indicator is needed, typically keyboard focus.
- *How do you test keyboard access?* → Unplug the mouse: Tab through the whole page, operate every control, open and close every popup, and watch where focus goes.

Self-check

1. Which keys activate a `<button>`? A link? → Button: Enter and Space. Link: Enter.
2. What does `tabindex="-1"` do? → Makes an element focusable by script or fragment navigation but not by Tab.
3. Why is `onclick` on a `<div>` not enough, even with `role="button"` and `tabindex="0"`? → Keyboard activation (Enter/Space) doesn't fire `click` on a div; you must add key handling.

3.8 Accessible forms

Forms are where accessibility failures cost the most — a user who can't identify a field cannot sign up, pay or apply. Full form coverage is in Chapter 5; these are the accessibility essentials.

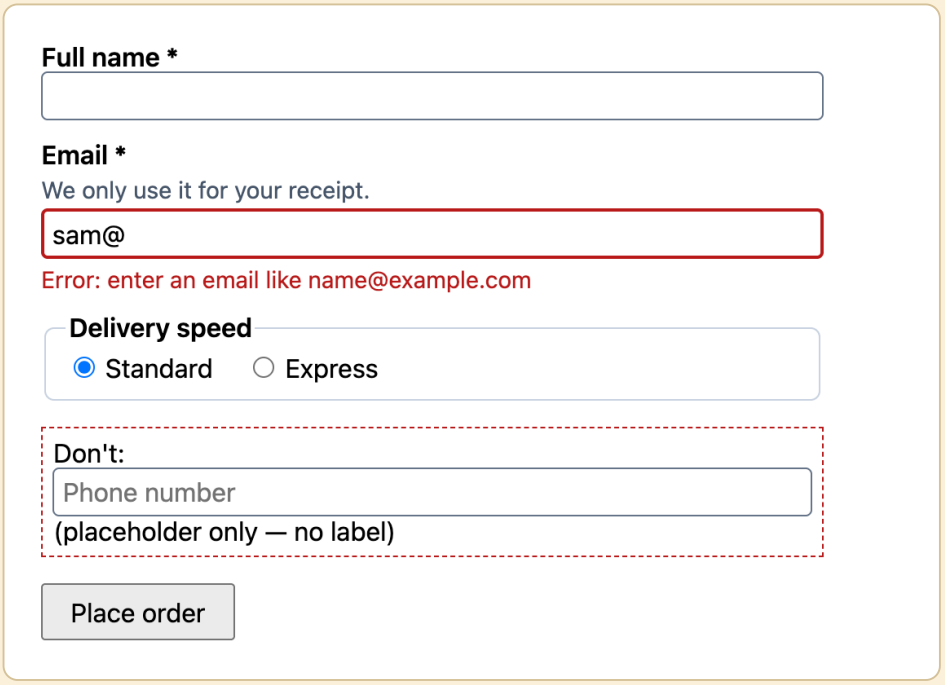


Figure 3.6 — `code/form-a11y.html`.

```
<label for="name">Full name <span aria-hidden="true">*</span></label>
<input id="name" type="text" name="name" autocomplete="name" required>

<label for="email">Email <span aria-hidden="true">*</span></label>
<p class="hint" id="email-hint">We only use it for your receipt.</p>
<input id="email" type="email" name="email" autocomplete="email" required
       aria-invalid="true" aria-describedby="email-hint email-err" value="sam@">
<p class="error" id="email-err">Error: enter an email like name@example.com</p>

<fieldset>
  <legend>Delivery speed</legend>
  <input type="radio" id="std" name="speed" checked><label for="std">Standard</label>
  <input type="radio" id="exp" name="speed"><label for="exp">Express</label>
</fieldset>

Don't: <input type="text" placeholder="Phone number">   <!-- placeholder only, no label -->
```

What Chrome exposes (`code/04_form_tree.mjs`):

```
RootWebArea "Accessible form"
  form
    textbox "Full name"  required=true
    textbox "Email"  required=true invalid="true" value="sam@" description="We only use it for your receipt. Error: enter
an email like name@example.com"
    group "Delivery speed"
      Legend
      radio "Standard"  checked=true
      radio "Express"  checked=false
    textbox "Phone number"
    button "Place order"

After typing into the placeholder-only field, the visible hint is gone: { value: '555', placeholderShown: false }
```

The rules behind that output:

1. **Every field has a** `<label>` tied by `for`/`id` (or by wrapping the input). The label becomes the name *and* enlarges the click target — clicking "Standard" checks the radio.
2. **Group related controls** (radio buttons, checkboxes, address parts) in `<fieldset>` with a `<legend>`; the legend names the group ("Delivery speed, grouping").
3. **Hints and errors** are linked with `aria-describedby` so they are read with the field; `aria-invalid="true"` marks the error state. Error text should say what is wrong and how to fix it, and should not rely on colour alone (the word "Error:" and the thicker border help — WCAG 1.4.1 Use of Color, 3.3.1 Error Identification, 3.3.3 Error Suggestion).
4. **Required**: use the native `required` attribute (exposed as `required=true`); a visual `*` is decorative for AT (`aria-hidden`) because the state is already announced — explain the asterisk once at the top of the form.
5. **Placeholders are not labels.** Chrome falls back to the placeholder as a name, but the hint vanishes as soon as the user types (see the output), placeholder text is usually low-contrast, and some AT ignores it.
6. `autocomplete` tokens (`name`, `email`, `street-address`, `one-time-code`…) let browsers and password managers (apps that store and fill in your passwords) fill fields — WCAG 1.3.5 (Identify Input Purpose), and a big help for users with motor or memory impairments. WCAG 2.2 adds 3.3.7 Redundant Entry (don't make people type the same information twice) and 3.3.8 Accessible Authentication (don't block password managers or paste; no memory/puzzle tests without an alternative).
7. On submit with errors, move focus to an error summary or the first invalid field.

Self-check

1. Two ways to associate a label with an input? → `<label for="id">` + `id` on the input, or wrap the input inside the `<label>`.
2. How do you name a group of radio buttons? → `<fieldset>` with a `<legend>`.
3. Why isn't a placeholder a label? → It disappears on input, is often low contrast, and isn't reliably announced.

3.9 WCAG 2.2

What it is

The **Web Content Accessibility Guidelines** (WCAG) from the W3C are the international standard that laws and contracts point to. **WCAG 2.2** became a W3C Recommendation (the W3C's final, official standard status) on 5 October 2023. It is backwards compatible (nothing that passed before now fails): a page that meets 2.2 also meets 2.1 and 2.0.

Structure:

- **4 principles — POUR**: content must be **Perceivable**, **Operable**, **Understandable**, **Robust**.
- **13 guidelines** under those principles (e.g. 1.1 Text Alternatives, 2.1 Keyboard Accessible, 3.3 Input Assistance).
- **Success criteria** — testable statements, numbered like `1.4.3`, each at a **level**:
  - **A** — the minimum; without it some users are completely blocked.
  - **AA** — the usual target for laws and company policies. "Conform to WCAG 2.2 AA" means meeting every A and AA criterion.

- **AAA** — enhanced; not recommended as a blanket requirement for whole sites because some criteria cannot be met for all content.

Criteria you will meet most often

Principle	SC	Level	In plain words
Perceivable	1.1.1 Non-text Content	A	Images, icons and controls have text alternatives (§3.5)
	1.3.1 Info and Relationships	A	Structure shown visually is in the markup too: headings, lists, labels, tables
	1.3.5 Identify Input Purpose	AA	Personal-data fields use `autocomplete`
	1.4.1 Use of Color	A	Colour is not the only way information is shown
	1.4.3 Contrast (Minimum)	AA	Text contrast ≥ 4.5:1 (large text ≥ 3:1)
	1.4.4 Resize Text	AA	Works at 200 % text size
	1.4.10 Reflow	AA	Usable at 320 CSS px (browser pixels) wide (400 % zoom) without horizontal scrolling
	1.4.11 Non-text Contrast	AA	UI parts and meaningful graphics (borders of inputs, focus rings, icons) ≥ 3:1
Operable	2.1.1 Keyboard	A	Everything works from a keyboard
	2.1.2 No Keyboard Trap	A	Focus can always move away
	2.4.1 Bypass Blocks	A	Skip links / landmarks
	2.4.2 Page Titled	A	Descriptive `<title>`
	2.4.3 Focus Order	A	Logical Tab order
	2.4.4 Link Purpose (In Context)	A	Link text makes sense
	2.4.6 Headings and Labels	AA	Headings and labels describe their topic or purpose
	2.4.7 Focus Visible	AA	A visible focus indicator
Understandable	3.1.1 Language of Page	A	`<html lang>`
	3.3.1 Error Identification	A	Errors are described in text
	3.3.2 Labels or Instructions	A	Inputs have labels/instructions
Robust	4.1.2 Name, Role, Value	A	Every UI component exposes a correct name, role and state (§3.4)
	4.1.3 Status Messages	AA	Status messages are announced without moving focus (live regions)

What's new in WCAG 2.2

Nine new success criteria:

SC	Level	Requirement
2.4.11 Focus Not Obscured (Minimum)	AA	A focused element is not entirely hidden by sticky headers, cookie banners, etc.
2.4.12 Focus Not Obscured (Enhanced)	AAA	…not hidden at all
2.4.13 Focus Appearance	AAA	The focus indicator is big enough and has enough contrast
2.5.7 Dragging Movements	AA	Anything done by dragging can also be done with single clicks/taps
2.5.8 Target Size (Minimum)	AA	Pointer targets at least 24×24 CSS px (or enough spacing)
3.2.6 Consistent Help	A	Help links/contact info appear in the same place across pages
3.3.7 Redundant Entry	A	Don't ask for the same information twice in a process
3.3.8 Accessible Authentication (Minimum)	AA	No cognitive tests (remembering, transcribing, puzzles) to log in (authentication = proving who you are) unless an alternative or aid exists — allow paste and password managers
3.3.9 Accessible Authentication (Enhanced)	AAA	Stricter version (no object/image recognition exceptions)

And one removal: **4.1.1 Parsing** is obsolete — browsers and AT now handle markup errors consistently (the parsing algorithm from Chapter 1), so it no longer needs a criterion. Validation remains good practice (§2.10).

Colour contrast — the formula

Contrast ratio compares the **relative luminance** (perceived brightness, from 0 = black to 1 = white) of text and background:

1. Convert each sRGB channel (sRGB is the standard colour space of screens; a channel is the red, green or blue part) from 0–255 to 0–1 and undo the gamma curve (screens store brightness on a curve, not a straight line, so this step turns it back into real light intensity): `c ≤ 0.04045 ? c/12.92 : ((c+0.055)/1.055)^2.4`.
2. Luminance `L = 0.2126 R + 0.7152 G + 0.0722 B` — green counts most because our eyes are most sensitive to it.
3. Ratio = `(L_lighter + 0.05) / (L_darker + 0.05)`, from 1:1 (same colour) to 21:1 (black on white).

Level	Normal text	Large text (≥ 24 px, or ≥ 18.66 px bold)	UI components & graphics
AA	4.5:1	3:1	3:1 (1.4.11)
AAA	7:1	4.5:1	—

```mjs
// code/contrast_lib.mjs
export function luminance(hex) {
  const [r, g, b] = hex.replace('#', '').match(/../g).map(h => parseInt(h, 16) / 255)
    .map(c => (c <= 0.04045 ? c / 12.92 : ((c + 0.055) / 1.055) ** 2.4));   // undo sRGB gamma
  return 0.2126 * r + 0.7152 * g + 0.0722 * b;                              // green looks brightest to the eye
}
export function contrast(fg, bg) {
  const [a, b] = [luminance(fg), luminance(bg)].sort((x, y) => y - x);
  return (a + 0.05) / (b + 0.05);
}
```

`code/05_contrast.mjs` output:

```
pair                    ratio   normal text   large text / UI
black on white          21.00:1 pass AA       pass AA | AAA ok
mid grey on white       4.54:1  pass AA       pass AA
one step lighter        4.48:1  FAIL          pass AA
light grey placeholder  2.85:1  FAIL          FAIL
white on blue-500       3.68:1  FAIL          pass AA
white on blue-700       6.70:1  pass AA       pass AA
error red on white      6.47:1  pass AA       pass AA
white on amber          2.15:1  FAIL          FAIL
```

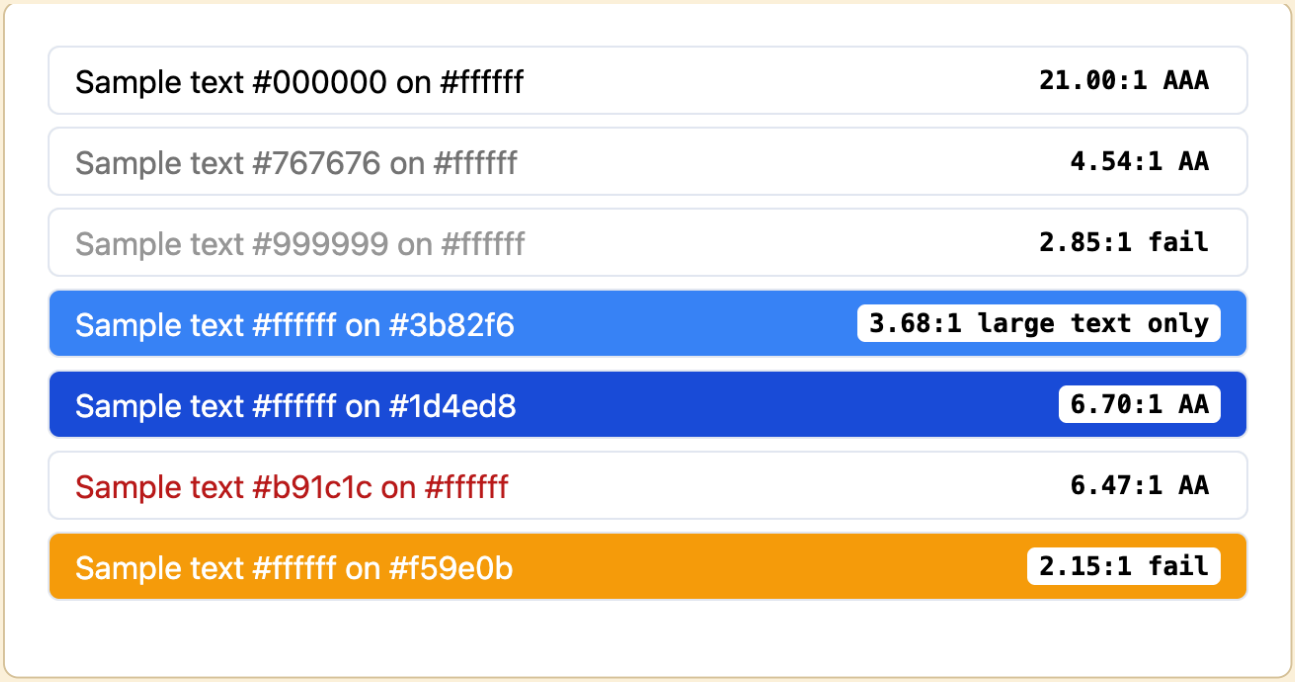


Figure 3.7 — `code/contrast.html` computes each ratio in the browser with the same formula.

Lessons from the numbers: `#767676` is the lightest grey that passes AA on white (4.54:1); one step lighter fails. White text on a mid-blue (`#3b82f6`, a very common button colour) fails for normal text — this book's own Chapter 1 page originally used it and was switched to the darker `#1d4ed8`. White on amber/yellow almost always fails. The ratio thresholds are strict: 4.49:1 is a fail. DevTools' colour picker shows the ratio and AA/AAA lines as you adjust a colour.

Self-check

1. What does POUR stand for? → Perceivable, Operable, Understandable, Robust.
2. Which level do most laws require? → AA.
3. Minimum contrast for 16 px body text at AA? → 4.5:1.
4. Name two criteria new in 2.2. → Target Size (Minimum) 2.5.8, Focus Not Obscured (Minimum) 2.4.11 (also Dragging Movements, Consistent Help, Redundant Entry, Accessible Authentication).
5. Which criterion was removed in 2.2? → 4.1.1 Parsing.

3.10 Testing tools

How it works — three layers

1. **Automated checkers** find the mechanical problems quickly: missing alt, missing labels, contrast, empty links/buttons, invalid ARIA, missing `lang`. They cannot judge whether alt text is *good*, whether focus order makes sense, or whether a custom widget is usable — only part of the problems can be detected automatically (estimates range from roughly a third of WCAG success criteria to just over half of real-world issues by volume).
2. **Manual keyboard testing** — no tools needed.
3. **Screen-reader testing** — hear what users hear.

Tool	Type	Notes
axe DevTools / **axe-core** (Deque)	Browser extension; JS library for CI (continuous integration — automatic checks on every code change), e.g. with Playwright, Cypress, Jest (`jest-axe`) testing tools	The engine behind many other tools; designed for no false positives (no false alarms)
Lighthouse	Built into Chrome DevTools	Accessibility score uses a subset of axe rules; also performance/SEO (Chapter 14)
WAVE (WebAIM)	Extension / website	Shows icons on the page itself — good for learning
DevTools Accessibility pane	Chrome/Firefox/Safari	Inspect role, name, states; Firefox can simulate colour-vision deficiencies (colour blindness) and show Tab order
W3C Nu checker / html-validate	Validators	Structural errors (§2.10)
eslint-plugin-jsx-a11y (a lint rule set — automatic code-style checks), **Testing Library** `getByRole`	Dev-time	Catch issues in React code; tests that query by role double as accessibility tests

Screen reader	Platform	Cost	Start / key tips
NVDA	Windows	Free (open source — its code is public)	`Insert` or `CapsLock` is the NVDA key; `H` next heading, `D` next landmark, `Tab` next focusable
JAWS	Windows	Commercial	Similar single-key navigation; widely used in workplaces
VoiceOver	macOS, iOS	Built in	macOS: Cmd+F5 to toggle; VO = Ctrl+Option; VO+U opens the rotor (headings, landmarks, links)
TalkBack	Android	Built in	Swipe right/left to move; double-tap to activate
Narrator	Windows	Built in	Basic testing

Test with the common pairings: NVDA or JAWS with Chrome/Firefox/Edge, VoiceOver with Safari.

A 5-minute keyboard test

1. Click the address bar, then press Tab repeatedly. Is the first stop a skip link? Does it work?
2. Can you **see** focus at every step? Is it ever hidden behind a sticky header?
3. Does the order follow the visual layout?
4. Can you operate every control (Enter/Space/arrows), open and close every menu and dialog (Escape), and never get stuck?
5. Zoom to 200 % and 400 %: does anything overlap or require horizontal scrolling?

A mini automated audit

To see how automated tools work, `code/06_mini_audit.mjs` implements a handful of axe-style rules — including checks for all six WebAIM Million top failures — using jsdom and `dom-accessibility-api` (a JavaScript implementation of the accessible-name spec, the same library Testing Library uses). It runs against a deliberately bad sign-up form:

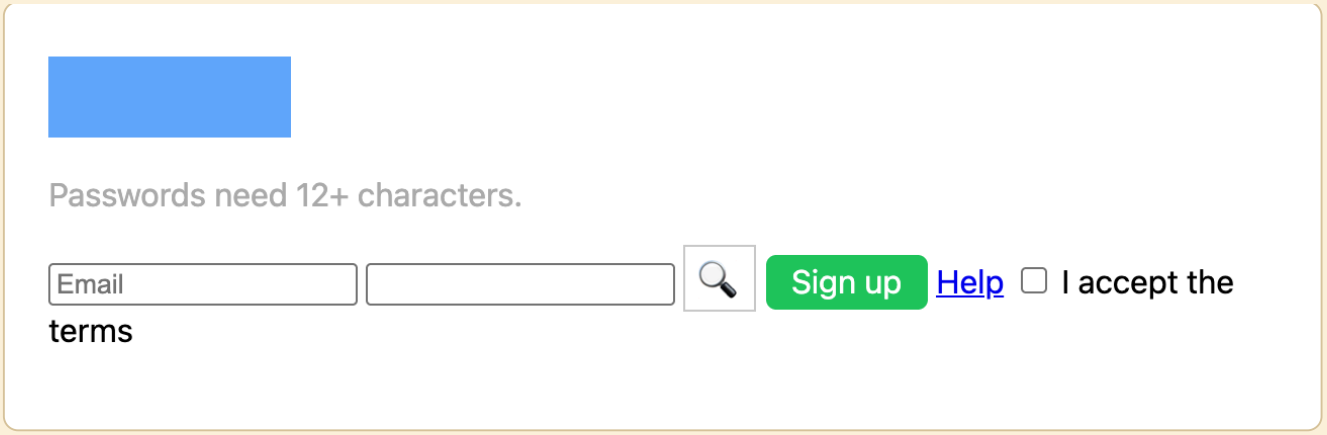


Figure 3.8 — `code/bad-a11y.html`. Looks usable — with a mouse and good eyesight.

```
// code/06_mini_audit.mjs (key parts)
import { computeAccessibleName, getRole } from 'dom-accessibility-api';
import { contrast } from './contrast_lib.mjs';
// ...
for (const el of doc.querySelectorAll(interactive)) {
  const name = computeAccessibleName(el);
  const role = getRole(el) ?? `${el.type} field`;          // <input type=password> has no ARIA role
  if (el.closest('[aria-hidden="true"]')) { add('aria-hidden-focus', '4.1.2', el, 'focusable element inside aria-hidden');
continue; }
  if (!name.trim() && el.placeholder) add('label', '3.3.2', el, `only a placeholder ("${el.placeholder}"), no label`);
  else if (!name.trim()) add('accessible-name', '4.1.2', el, `${role} has no accessible name`);
  if (el.tabIndex > 0) add('tabindex', '2.4.3', el, `tabindex="${el.tabIndex}" breaks the natural focus order`);
  if (el.matches('[role=button]:not([tabindex])')) add('focusable-role', '2.1.1', el, 'role="button" but not keyboard-
focusable');
}
// ... plus: html lang, empty headings, img alt, broken aria-describedby/labelledby ids, orphan labels, inline-colour
contrast
```

Output:

```
bad-a11y.html: 12 issue(s)
rule                 WCAG    element / problem
html-has-lang        3.1.1   <html> no lang attribute
empty-heading        BP      <h1> heading has no text
image-alt            1.1.1   <img> missing alt (use alt="" if decorative)
label                3.3.2   <input> only a placeholder ("Email"), no label
accessible-name      4.1.2   <input> password field has no accessible name
tabindex             2.4.3   <button> tabindex="3" breaks the natural focus order
focusable-role       2.1.1   <div> role="button" but not keyboard-focusable
aria-hidden-focus    4.1.2   <a> focusable element inside aria-hidden
accessible-name      4.1.2   <input> checkbox has no accessible name
aria-valid-ref       4.1.2   <input> references missing id "tos-text"
orphan-label         1.3.1   <label> "I accept the terms" is not tied to any control
color-contrast       1.4.3   <p> #aaaaaa on #ffffff is 2.32:1 (needs 4.5:1)

form-a11y.html: 1 issue(s)
rule                 WCAG    element / problem
label                3.3.2   <input> only a placeholder ("Phone number"), no label
```

(`BP` = best practice rather than a specific success criterion.) The good form's only finding is the "Don't" example we put in on purpose. Note what the tool **cannot** know: whether "🔍" is a good name for the search button (a screen reader may say "magnifying glass tilted left"), whether the alt texts are meaningful, or whether the Tab order matches the layout. That is what the keyboard and screen-reader passes are for. In real projects, use axe-core rather than hand-written rules.

In the interview

30-second answer: "I layer it: automated checks with axe or Lighthouse in development and CI catch things like missing labels, alt text and contrast; then a manual keyboard pass for focus order, visibility and traps; then a screen-reader pass — VoiceOver with Safari, NVDA with Chrome or Firefox — on key flows. Automation only finds part of the issues, so the manual steps are not optional."

- *Can Lighthouse give 100 and the page still be inaccessible?* → Yes; it checks a subset of rules and nothing about meaning, order or usability.
- *How do you keep regressions (things that used to work breaking again) out?* → axe-core in component/end-to-end tests (tests that drive the real app like a user would), lint rules, role-based queries in tests, and accessibility in the definition of done (the checklist a task must meet before it counts as finished).

Self-check

1. Name one thing automated tools can't check. → Quality of alt text; logical focus order; whether a custom widget is actually usable.
2. Which free screen reader runs on Windows? → NVDA.
3. How do you open VoiceOver's list of headings and landmarks? → VO+U (Ctrl+Option+U), the rotor.

Chapter 3 TL;DR

- **Semantic HTML is the foundation of accessibility**: native elements come with roles, names, states, focus and keyboard behaviour that assistive technology understands.
- **Landmarks**: top-level `header` → banner, `nav` → navigation (label multiples), `search`, one `main`, `aside` → complementary, top-level `footer` → contentinfo; `section` is a region only when named; `article` = self-contained.
- **Headings** form the navigable outline; no outline algorithm exists.
- **Accessibility tree** = role + name + description + states + value; names come from `aria-labelledby` → `aria-label` → native label/`alt` → content → `title`/placeholder.
- **Alt text**: describe purpose in context; `alt=""` for decoration; describe the destination for linked images; give complex images a text equivalent.
- **ARIA**: first rule — use native HTML. ARIA changes semantics only, never behaviour. Use it for names (`aria-label`/`labelledby`), descriptions, states (`aria-expanded`, `aria-current`, `aria-invalid`), live regions (`role="status"`), and hiding decoration (`aria-hidden`).
- **Keyboard**: everything reachable and operable; DOM order = visual order; `tabindex` 0 / -1 only; `:focus-visible` rings with ≥ 3:1 contrast; skip link; manage focus for dialogs and route changes; targets ≥ 24×24 px.
- **Forms**: real `<label>`s, `fieldset`/`legend` for groups, `aria-describedby` for hints and errors, `aria-invalid`, native `required`, `autocomplete`; placeholders are not labels.
- **WCAG 2.2 AA** is the usual target: POUR, 13 guidelines, A/AA/AAA; contrast 4.5:1 text, 3:1 large text and UI; nine new criteria, 4.1.1 removed.
- **Test in layers**: axe/Lighthouse/WAVE → keyboard pass → screen reader (NVDA, JAWS, VoiceOver, TalkBack).

Next: Chapter 4 — Images, Media and Embeds.

Chapter 4 — Images, Media and Embeds

Text alone rarely makes a website. Product photos, logos, charts, podcasts, tutorial videos, maps and payment widgets (small self-contained interactive components, often supplied by another company) are all "media": content that is not plain text and usually lives in its own file. HTML has a small set of elements for them — `<img>`, `<picture>`, `<figure>`, `<audio>`, `<video>`, `<track>`, `<iframe>` (a window showing another web page), `<svg>` (vector drawings) and `<canvas>` (a drawing surface for JavaScript) — and each comes with rules about accessibility, performance and security that beginners often miss.

Media is also where most of a page's weight (total download size) is. Images alone are typically the largest share of the bytes on a page, and the biggest image above the fold (visible without scrolling) is very often the element that decides how fast the page *feels* (the **Largest Contentful Paint**, LCP — covered in Chapter 14). So this chapter is as much about "which file does the browser download, and when" as about markup.

What you'll learn

1. The `<img>` element: `src`, `alt`, `width`/`height`, and how to write alt text.
2. Image formats — JPEG, PNG, GIF, WebP, AVIF, SVG — and how to choose (with real file sizes).
3. Responsive images: `srcset` with `x` and `w` descriptors, `sizes`, and `<picture>` for art direction and format switching.
4. Loading hints: `loading="lazy"`, `decoding`, `fetchpriority`.
5. `<figure>` and `<figcaption>`.
6. `<audio>`, `<video>`, `<source>` and `<track>` (captions with WebVTT).
7. `<iframe>`: embedding other pages safely with `sandbox`, `allow`, `title`, `srcdoc`.
8. SVG basics: coordinates, `viewBox`, shapes, paths, icons with `currentColor`.
9. `<canvas>`: drawing with JavaScript, and when to prefer SVG.

How the examples work. Every example is a real file in `code/`. No example touches the network: all pictures, the audio beep and the video clip were generated locally by `code/make_assets.py` (Python + Pillow, a Python image-processing library) and `code/make_video.mjs` (headless Chrome records an animated canvas). Screenshots in `img/` were taken with headless Chrome (Chrome run by a script, without a window) at a 900 CSS-pixel-wide window and a device pixel ratio of 2 (a "Retina"-style screen — Apple's name for high-density displays with two or more physical dots per CSS pixel; both terms are defined properly in §4.3). The two Node scripts (`format_sizes.mjs`, `srcset_pick.mjs`) print the outputs shown below.

4.1 The `<img>` element

What it is

`<img>` embeds a picture into the page. It is a **void element** (no closing tag, no content) and a **replaced element**: the browser replaces the element's box with an external resource, the image file, whose content CSS cannot style from the inside.

Analogy: `<img>` is a picture frame nailed into the text. The HTML decides where the frame goes and what the caption card says (`alt`); the file decides what is inside.

Why it matters

- Images carry meaning (a chart, a product, a diagram). People who cannot see them — screen-reader users (blind users whose software reads the page aloud), people on slow connections where images fail, search engines — only get the `alt` text.
- Images without known dimensions make the page jump while it loads (**layout shift**), which annoys users and hurts the Core Web Vital CLS (Core Web Vitals are Google's three headline page-experience measurements; CLS, Cumulative Layout Shift, scores how much content jumps around).

How it works

The essential attributes:

Attribute	Required?	What it does
`src`	yes	URL of the image file (relative like `media/a.jpg` or absolute like `https://…`).
`alt`	yes, in practice	Text alternative. Shown when the image fails, read by screen readers, indexed by search engines. `alt=""` marks the image as decorative.
`width`, `height`	strongly recommended	The image's intrinsic size (its natural size) in CSS pixels, **without units** (`width="800"`). Lets the browser reserve space before the file arrives.
`srcset`, `sizes`	optional	Candidate files for different screens (4.3).
`loading`, `decoding`, `fetchpriority`	optional	Loading hints (4.4).

Steps the browser follows:

1. The HTML parser meets `<img>` and (unless it is lazy) starts downloading `src` right away — even before CSS is ready. A separate "preload scanner" (a fast look-ahead pass over the raw HTML) finds images early.
2. From `width`/`height` the browser computes an **aspect ratio** (width divided by height — the image's shape; its built-in stylesheet effectively applies `aspect-ratio: auto <width> / <height>`), so it can reserve a box of the right shape even if CSS makes the image fluid (resizing with its container).
3. When the bytes arrive, the image is decoded (unpacked from its compressed file format into raw pixels) and painted into the reserved box.
4. If the file fails, the browser shows a broken-image indicator plus the `alt` text.

The standard CSS to make images shrink with their container but never stretch:

```
img { max-width: 100%; height: auto; }   /* fluid width, height follows the aspect ratio */
```

`height: auto` matters: without it, the HTML `height="533"` would keep the image 533 px tall while the width shrinks, squashing it.

```
<!-- code/01_img_basics.html (excerpt) -->
<figure>
  <img src="media/hills-800.jpg" alt="Two green hills under a blue sky with a yellow sun"
       width="800" height="533">
  <figcaption>Figure 1 — JPEG, 800x533. width/height reserve the space before it loads.</figcaption>
</figure>
<img src="media/logo.png" alt="HC" width="80" height="80">
<img src="media/logo.svg" alt="" width="80" height="80">   <!-- decorative duplicate -->

<!-- three images whose file is missing -->
<img src="media/does-not-exist.jpg" alt="Sales chart: revenue up 12% in Q3" width="200" height="120">
<img src="media/does-not-exist.jpg" alt="image123.jpg" width="200" height="120">
<img src="media/does-not-exist.jpg" width="200" height="120">
```

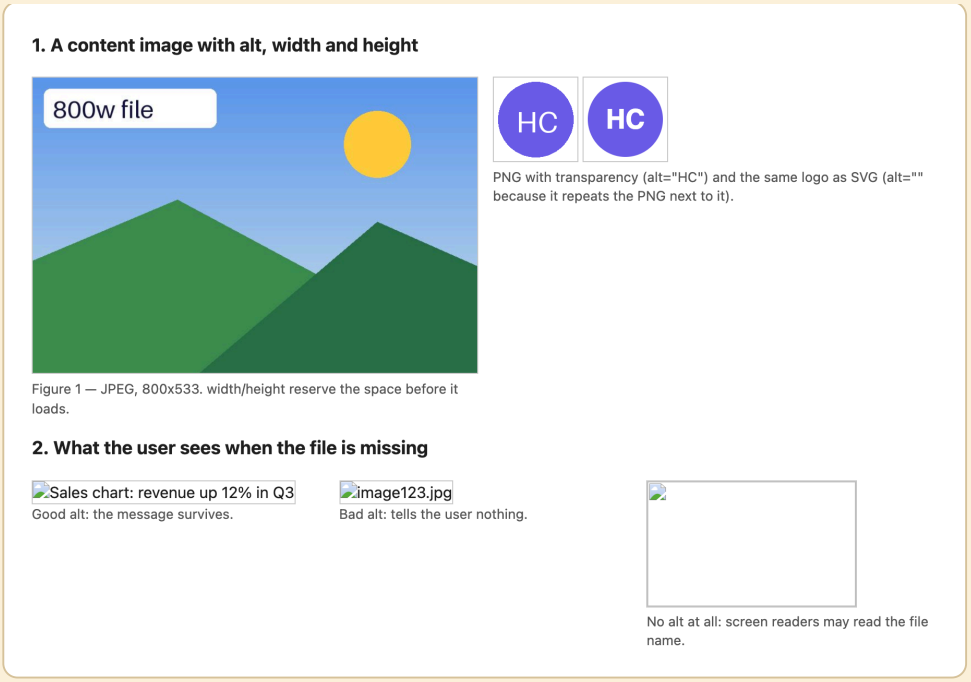


Figure 4.1 — The bottom row shows what alt text is for: when the file is missing, only the first image still communicates anything.

Writing alt text — a decision list

1. **Does the image carry information?** Describe the *information*, not the pixels. For a chart: `alt="Revenue rose 12% in Q3"`, not `alt="bar chart"`.
2. **Is it a link or button with no other text?** Describe the *action/destination*: a magnifier icon inside a search button → `alt="Search"`.
3. **Is it purely decorative** (a flourish, a background texture) **or does it repeat nearby text?** Use `alt=""` (empty, not missing). Screen readers then skip it. Even better, put pure decoration in CSS (`background-image`).
4. **Is it complex** (a detailed diagram)? Give a short `alt` and put the full description in the surrounding text or a `<figcaption>`.
5. Never start with "image of" / "picture of" — screen readers already announce "image".

Missing `alt` is different from empty `alt`. With no `alt`, many screen readers fall back to reading the file name ("image one two three dot jpg"). Accessibility rules for images are covered in more depth in Chapter 3.

Common mistakes

- Omitting `width`/`height` → content jumps down when the image loads.
- Putting units in the attributes (`width="800px"`). The attributes take plain integers; units belong in CSS.
- Using CSS to set `width` but forgetting `height: auto` → distorted images.
- Keyword-stuffed or file-name alt text.
- Serving a 4000-px-wide camera photo into a 300-px slot (fixed by 4.3).

In the interview

30-second answer: "`<img>` is a void, replaced element. `src` and `alt` are required in practice; `alt` should convey the image's purpose, and be empty for decorative images. Always set `width` and `height` so the browser knows the aspect ratio and reserves space, preventing layout shift, and use `max-width: 100%; height: auto` in CSS for fluid images."

- *Q: What's the difference between `alt=""` and no alt?* A: Empty alt says "decorative, skip it"; missing alt is an error, and assistive tech (screen readers and similar tools) may read the URL instead.
- *Q: Do width/height attributes stop an image being responsive?* A: No. CSS `max-width:100%; height:auto` overrides the displayed size; the attributes still supply the aspect ratio.
- *Q: CSS background image or `<img>`?* A: `<img>` for content (it has alt, is indexable, can be responsive with srcset); CSS backgrounds for decoration.

Self-check

1. Which two attributes prevent layout shift? → `width` and `height`.
2. A company logo that links to the home page — alt text? → The destination/name, e.g. `alt="Acme home"`.
3. Why is `height: auto` needed with `max-width: 100%`? → So the height scales with the width instead of staying at the HTML height.

TL;DR

`<img src alt width height>` every time; meaningful alt for content, `alt=""` for decoration; fluid with `max-width:100%; height:auto`.

4.2 Image formats — JPEG, PNG, GIF, WebP, AVIF, SVG

What it is

An **image format** is the way pixels (or shapes) are stored in a file. **Raster** formats store a grid of pixels (JPEG, PNG, GIF, WebP, AVIF). **Vector** formats store drawing instructions — "circle here, radius 40" — so they scale to any size without blur (SVG).

Two more terms:

- **Lossy compression** throws away detail the eye is unlikely to notice to make files much smaller (JPEG, lossy WebP, AVIF).
- **Lossless compression** keeps every pixel exactly (PNG, GIF, lossless WebP/AVIF).

Comparison table

Format	Type	Compression	Transparency	Animation	Best for	Browser support
JPEG (`.jpg`)	raster	lossy	no	no	photos, when you need one universal format	everywhere
PNG	raster	lossless	yes (alpha channel: per-pixel see-through level)	no (APNG, animated PNG, exists)	screenshots, UI, logos with transparency, sharp text	everywhere
GIF	raster	lossless, 256 colours max	1-bit (on/off)	yes	legacy animations — prefer video or WebP/AVIF	everywhere
WebP	raster	lossy or lossless	yes	yes	general replacement for JPEG/PNG	all modern browsers (Safari since 2020)
AVIF	raster	lossy or lossless	yes	yes	smallest files for photos; HDR (high dynamic range: brighter highlights)/wide colour	Baseline 2024 (widely available since July 2026)
SVG	vector	text (XML, a tag-based data format like HTML)	yes	yes (CSS/SMIL, SVG's own animation tags)	icons, logos, illustrations, simple charts	everywhere

"Baseline" is the web.dev/MDN label for features that work in the current versions of Chrome, Edge, Firefox and Safari.

Real numbers

The same 800×533 picture was saved in four formats by `make_assets.py` (JPEG quality 80, WebP quality 80, AVIF quality 60, PNG optimised) — once as a flat graphic and once with photo-like grain added.

```
// code/format_sizes.mjs
import { statSync } from 'fs';
for (const stem of ['hills-800', 'photo-800']) {
  const png = statSync(`media/${stem}.png`).size;
  console.log(`${stem}  (${stem.startsWith('hills') ? 'flat graphic' : 'photo-like grain'})`);
  for (const ext of ['png', 'jpg', 'webp', 'avif']) {
    const kb = statSync(`media/${stem}.${ext}`).size / 1024;
    const vsPng = ((kb * 1024) / png * 100).toFixed(0);
    console.log(`  ${ext.padEnd(5)} ${kb.toFixed(1).padStart(7)} KB   ${vsPng.padStart(4)}% of PNG`);
  }
}
```

Output:

```
hills-800  (flat graphic)
  png       8.8 KB    100% of PNG
  jpg      19.1 KB    216% of PNG
  webp      7.4 KB     84% of PNG
  avif      4.3 KB     48% of PNG
photo-800  (photo-like grain)
  png     508.2 KB    100% of PNG
  jpg     106.8 KB     21% of PNG
  webp    106.3 KB     21% of PNG
  avif     73.7 KB     14% of PNG
```

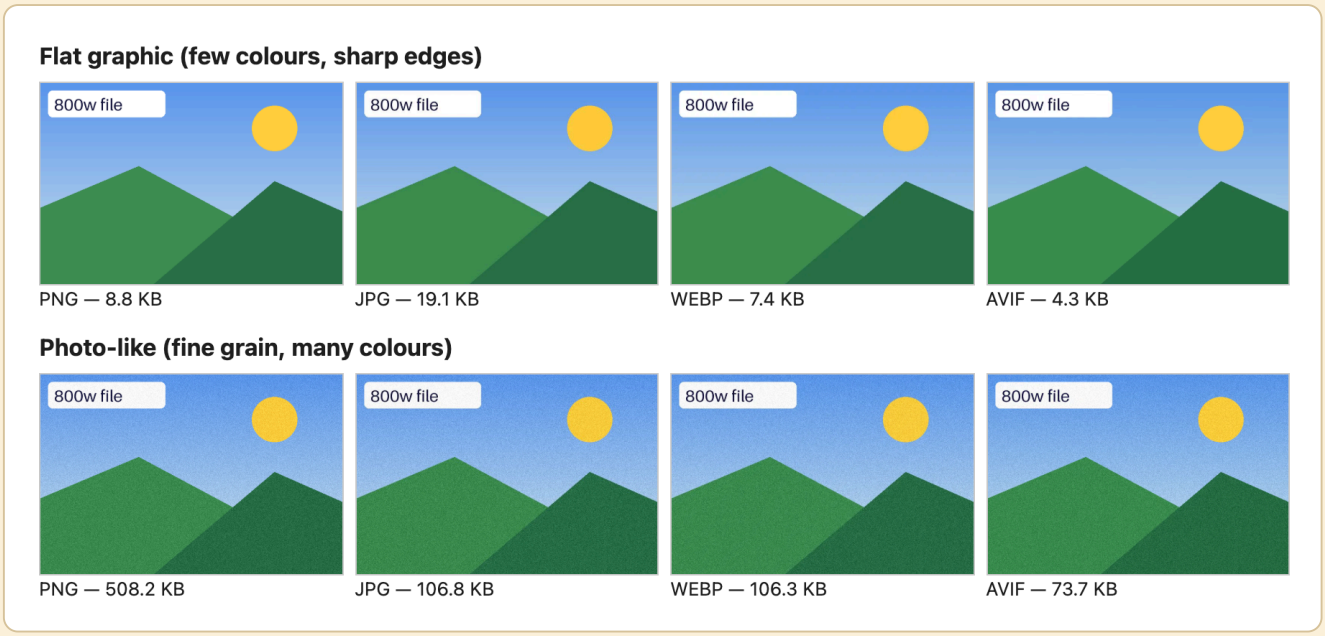


Figure 4.2 — Visually identical at this size; the byte counts are not.

What the numbers teach:

1. **The content decides the winner.** For a flat graphic, lossless PNG (8.8 KB) beats lossy JPEG (19.1 KB) — JPEG wastes bytes on its block artefacts (smudgy blocky noise — JPEG compresses in 8×8-pixel blocks) around sharp edges. For a photo, PNG is 5× bigger than JPEG.
2. **AVIF was smallest in both cases**, WebP was roughly equal to or better than the older format. Exact savings vary per image and per quality setting, so measure your own images.
3. For vector-like artwork, SVG is often smaller still and perfectly sharp at any size (4.8).

How to choose (quick rules)

1. Icon, logo, illustration with flat colours → **SVG**.
2. Photo → **AVIF** (or WebP) with a **JPEG** fallback (a second choice used when the first is not supported) through `<picture>` (4.3).
3. Screenshot or graphic needing transparency / exact pixels → **PNG** or lossless WebP.
4. Short animation → a muted, looping `<video>` (MP4/WebM, the common video file types) is usually far smaller than a GIF.

Common mistakes

- Saving a photo as PNG "for quality" — huge files with no visible benefit.
- Saving a logo as JPEG — fuzzy edges and no transparency.
- Using only AVIF with no fallback when you must support old devices.
- Re-compressing an already-compressed JPEG many times (quality degrades each time).

Real-world examples

Image CDNs (content delivery networks — services that store copies of your files on servers around the world and can convert images on the fly; e.g. Cloudinary, imgix, Cloudflare Images, Next.js `<Image>`; see System Design Fundamentals) inspect the browser's `Accept` request header (where the browser lists the file types it understands) and serve AVIF or WebP automatically when supported, falling back to JPEG.

Self-check

1. Which formats support transparency? → PNG, GIF (1-bit), WebP, AVIF, SVG — not JPEG.
2. Why was PNG smaller than JPEG for the flat graphic? → Lossless compression is very efficient on large areas of identical colour; JPEG is designed for smooth photographic detail.
3. What replaces an animated GIF today? → A muted, looping, autoplaying `<video>` (or animated WebP/AVIF).

TL;DR

SVG for vectors, AVIF/WebP with JPEG fallback for photos, PNG for pixel-exact graphics; measure — content type changes which format wins.

4.3 Responsive images — `srcset`, `sizes` and `<picture>`

The problem it solves

A phone at 360 CSS px wide does not need the 1600-px hero image (the big banner image at the top of a page) a large desktop monitor needs. But a phone with a device pixel ratio (DPR) of 3 — three physical pixels per CSS pixel — *does* need more pixels than its CSS width suggests to look sharp. One `src` can't be right for everyone. Two separate problems:

1. **Resolution switching** — same picture, different sizes. The browser should download the smallest file that still looks sharp. (`srcset` + `sizes` on `<img>`.)
2. **Art direction** — a *different crop or picture* for different layouts (e.g. a wide landscape on desktop, a tight square crop on phones). You decide, not the browser. (`<picture>` + `<source media>`.)

A third, related need is **format switching**: serve AVIF/WebP to browsers that support them, JPEG to the rest (`<picture>` + `<source type>`).

Terms:

- **CSS pixel** — the unit CSS lengths use; roughly the size of a pixel on a traditional 96-dpi screen.
- **Device pixel ratio (DPR)** — physical pixels per CSS pixel (`window.devicePixelRatio`): 1 on older monitors, 2 on Retina laptops, often 3 on phones.

How it works — density descriptors (`x`)

For an image with a **fixed display size**, list files by pixel density:

```
<img srcset="media/hills-400.jpg 1x, media/hills-800.jpg 2x"
     src="media/hills-400.jpg" alt="Hills at sunrise" width="200" height="133">
```

On a DPR-1 screen the browser takes the `1x` file; on DPR 2 or more, the `2x`. `src` remains the fallback for very old browsers.

How it works — width descriptors (`w`) + `sizes`

For an image whose **display size changes with the layout**, describe each file's real pixel width with `w`, and tell the browser how wide the image will be displayed with `sizes`:

```
<img srcset="hills-400.jpg 400w, hills-800.jpg 800w, hills-1600.jpg 1600w"
     sizes="(max-width: 600px) 100vw, (max-width: 1000px) 50vw, 400px"
     src="hills-800.jpg" alt="Hills at sunrise" width="800" height="533">
```

The browser's algorithm, step by step:

1. **Evaluate** `sizes` left to right. Each entry is `(media condition) length` — a media condition is a test about the screen, such as `(max-width: 600px)` = "the viewport is at most 600 px wide"; the first matching condition wins; the last entry (no condition) is the default. Result: the **slot width** in CSS px. (`vw` = 1% of the viewport width; the **viewport** is the visible area of the browser window.)
2. **Why does it need `sizes` at all?** The browser picks the image while it is still downloading the HTML, *before* CSS is loaded, so it cannot know the layout. `sizes` is your promise about the layout. Without `sizes`, a `w`-descriptor `srcset` assumes `100vw`.
3. **Compute each candidate's density**: file width ÷ slot width. `800w` in a 400-px slot = 2x.
4. **Pick** a candidate whose density is at least the screen's DPR — typically the smallest one that is sharp enough. The spec deliberately lets browsers deviate (for example, reusing a larger file already in cache — the browser's local store of previously downloaded files — or saving data on a slow connection).

A small Node program that models this rule for three viewports and three DPRs:

```
// code/srcset_pick.mjs (key part)
const srcset = 'hills-400.jpg 400w, hills-800.jpg 800w, hills-1600.jpg 1600w';
const sizes = '(max-width: 600px) 100vw, (max-width: 1000px) 50vw, 400px';

function slotWidth(sizesAttr, viewport) {       // first matching media condition wins
  for (const part of sizesAttr.split(',').map(s => s.trim())) {
    const m = part.match(/^\((max|min)-width:\s*(\d+)px\)\s+(.+)$/);
    const length = m ? m[3] : part;
    const ok = !m || (m[1] === 'max' ? viewport <= +m[2] : viewport >= +m[2]);
    if (ok) return length.endsWith('vw') ? (parseFloat(length) / 100) * viewport : parseFloat(length);
  }
}
function pick(viewport, dpr) {
  const slot = slotWidth(sizes, viewport);
  const withDensity = candidates.map(c => ({ ...c, density: c.w / slot })).sort((a, b) => a.w - b.w);
  const chosen = withDensity.find(c => c.density >= dpr) ?? withDensity.at(-1);
  return { slot, needed: slot * dpr, chosen };
}
```

Output:

```
sizes = "(max-width: 600px) 100vw, (max-width: 1000px) 50vw, 400px"
viewport  dpr  slot(CSS px)  needs(px)  chosen
     360  1x           360        360  hills-400.jpg
     360  2x           360        720  hills-800.jpg
     360  3x           360       1080  hills-1600.jpg
     768  1x           384        384  hills-400.jpg
     768  2x           384        768  hills-800.jpg
     768  3x           384       1152  hills-1600.jpg
    1280  1x           400        400  hills-400.jpg
    1280  2x           400        800  hills-800.jpg
    1280  3x           400       1200  hills-1600.jpg
```

Notice the biggest file goes to the *phone* with DPR 3, not to the desktop — the number of physical pixels, not the screen size, decides.

How it works — `<picture>`

`<picture>` wraps one `<img>` and any number of `<source>` elements. The browser walks the sources **in order** and uses the **first** one whose `media` matches and whose `type` it supports; if none match, it uses the `<img>`. The `<img>` is always required: it is the element that is actually displayed, and it carries `alt`, `width`, `height`, `loading` for all sources. (When an art-directed crop has a different shape, you can also put `width`/`height` on that `<source>` so the browser reserves the right aspect ratio.)

```html
<!-- code/03_responsive_images.html (excerpt) -->
<picture>
  <!-- art direction: narrow screens get a square crop -->
  <source media="(max-width: 600px)" srcset="media/hills-square-600.jpg">
  <!-- format switching: best format the browser can decode -->
  <source type="image/avif" srcset="media/hills-800.avif">
  <source type="image/webp" srcset="media/hills-800.webp">
  <img src="media/hills-800.jpg" alt="Hills at sunrise" width="800" height="533">
</picture>
```

The demo page prints `img.currentSrc` — the file the browser actually chose — under each image:

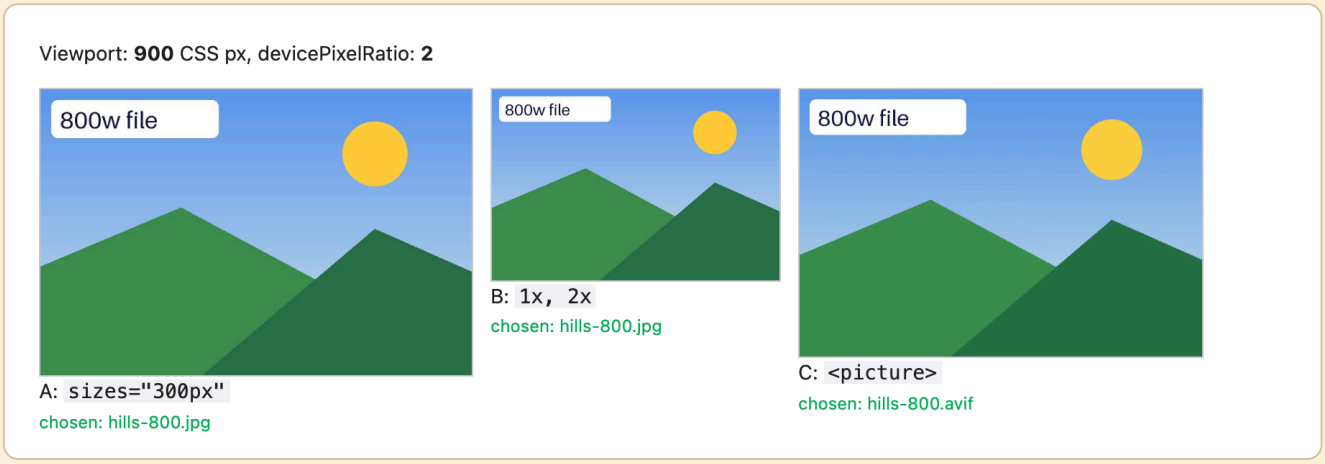


Figure 4.3 — At 900 px wide and DPR 2. A: slot 300 px × 2 = 600 px needed → the 800w file. B: DPR 2 → the `2x` file. C: no media match, AVIF supported → the AVIF.

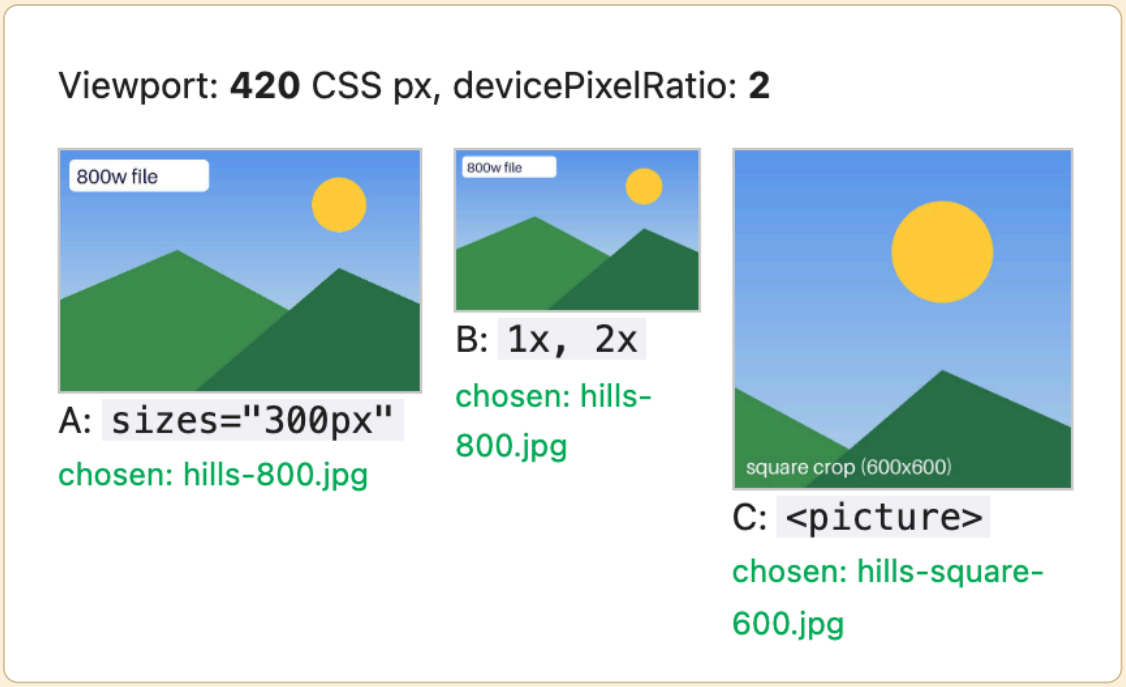


Figure 4.4 — The same page at 420 px wide. Only `<picture>` (C) changed its crop, because `(max-width: 600px)` now matches; A and B still pick by pixel density.

Trade-offs

Need	Use	Who decides?
Fixed-size image, sharp on high-DPR screens	`srcset` with `1x, 2x`	browser
Fluid image, many screen sizes	`srcset` with `w` + `sizes`	browser
Different crop/composition per layout	`<picture>` + `<source media>`	you
Modern format with fallback	`<picture>` + `<source type>`	browser (by support)

Common mistakes

- `w` descriptors without `sizes` — the browser assumes the image is 100vw and downloads files far too big for small slots.
- `sizes` that lie about the layout (e.g. `sizes="100vw"` for a sidebar thumbnail).
- Putting `alt` on `<source>` — it belongs on the `<img>` only.
- Using `<picture>` just to switch resolution — plain `srcset` is enough and lets the browser choose smarter.
- Mixing `x` and `w` descriptors in one `srcset` (invalid).

When not to use it

Small icons and SVGs don't need `srcset` — vectors are sharp at any size. For background images use CSS `image-set()` instead (see Chapter 10).

In the interview

> **30-second answer:** "`srcset` lists candidate files; with `x` descriptors for fixed-size images, with `w` descriptors plus `sizes` for fluid ones. `sizes` tells the browser the display width before CSS loads, the browser divides each file's width by that to get its density, then picks one dense enough for the device pixel ratio. `<picture>` is for when *I* must control the choice — art direction with `media`, or format fallbacks with `type`; its `<img>` child is mandatory and carries alt and dimensions."

- *Q: Why can't the browser just measure the layout?* A: Image downloads start during HTML parsing, before CSS and layout exist; waiting would delay every image.
- *Q: With `<picture>`, which source is used if two match?* A: The first one in source order.
- *Q: How do you check which file was chosen?* A: `img.currentSrc` in the console, or the Network panel in DevTools.

Self-check

1. Viewport 768 px, DPR 2, `sizes="(max-width: 1000px) 50vw, 400px"` — slot and needed pixels? → 384 CSS px; 768 device px → pick the 800w file.
2. Where do `alt` and `width`/`height` go in a `<picture>`? → On the `<img>`.
3. What happens without `sizes` when `srcset` uses `w`? → The slot is assumed to be `100vw`.

TL;DR

`x` for fixed size, `w` + honest `sizes` for fluid size, `<picture>` only when you need art direction or format fallback; `currentSrc` shows the winner.

4.4 Loading hints — `loading`, `decoding`, `fetchpriority`

What they are

Three attributes that change *when* and *how urgently* an image is fetched or decoded. They are hints: the page works without them, but loads faster with them.

Attribute	Values	Effect
`loading`	`eager` (default), `lazy`	`lazy` defers fetching until the image is near the viewport. Also works on `<iframe>`.
`fetchpriority`	`high`, `low`, `auto` (default)	Raises or lowers the download priority relative to other images. Baseline since Oct 2024.
`decoding`	`sync`, `async`, `auto` (default)	Whether decoding the image may be done off the main rendering step (in the background, not holding up drawing the rest of the page). Rarely needed.

Native lazy loading for images and iframes has been Baseline since December 2023 (Firefox 121 was the last engine to add it for iframes; images worked everywhere earlier), so JavaScript lazy-loading libraries are no longer needed for the basic case.

How it works

1. The browser checks each `loading="lazy"` element's distance to the viewport. Browsers use a generous margin (on the order of a screen or more), so images normally arrive before the user scrolls to them.
2. Lazy images do **not** delay the window `load` event, because they may never be fetched.
3. Lazy loading only happens when scripting (JavaScript) is enabled (a privacy measure: otherwise a site could see how far you scrolled from which images you request, even with JavaScript switched off).
4. `fetchpriority="high"` moves the image up the download queue (the waiting list of files to fetch) — useful for the one image that is the page's LCP element.

```
<!-- code/04_lazy_loading.html (excerpt) -->
<img id="hero" src="media/hills-400.jpg" alt="Hero: hills at sunrise" width="400" height="267"
     fetchpriority="high">                                  <!-- above the fold -->
<img id="lazy1" class="far" src="media/photo-800.jpg" alt="Grainy hills"
     width="400" height="267" loading="lazy">               <!-- 3000 px lower -->
<img id="lazy2" src="media/hills-1600.jpg" alt="Large hills"
     width="400" height="267" loading="lazy">
<script>
  addEventListener('load', () => {
    report.textContent = 'after the load event:\n' + [...document.images]
      .map(i => `${i.id.padEnd(6)} ${(i.getAttribute('loading') || '(none)').padEnd(6)} ${i.naturalWidth ? 'LOADED' : 'not loaded yet'}`)
      .join('\n');
  });
</script>
```

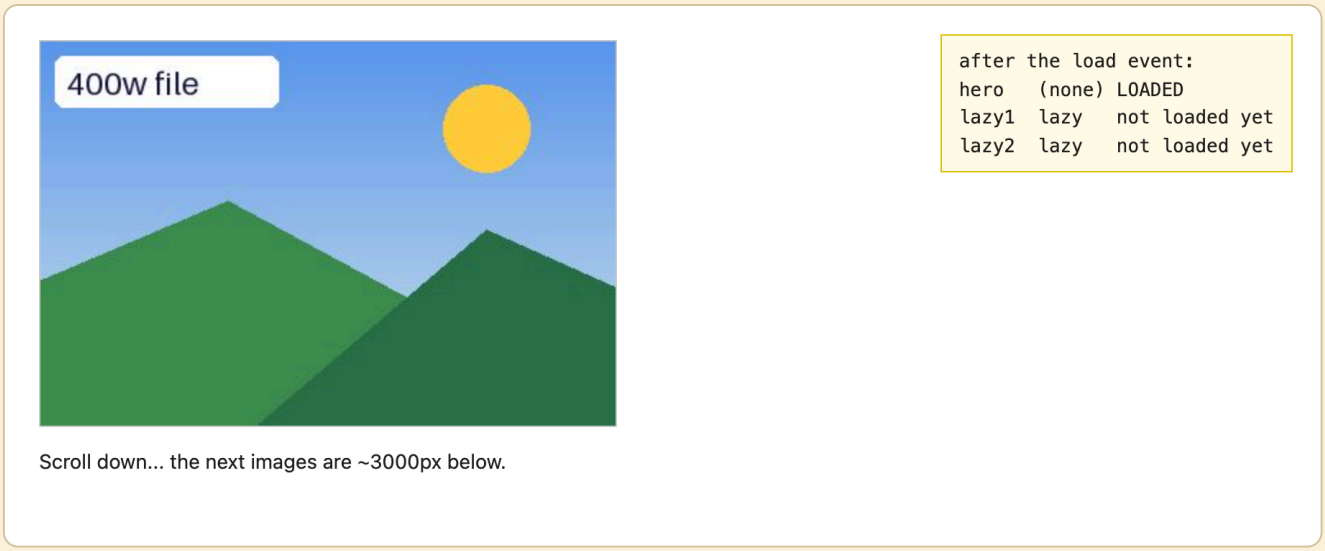


Figure 4.5 — When the page finished loading, only the hero had been downloaded; the two lazy images (~660 KB together) were never fetched because nobody scrolled to them.

Common mistakes

- **Lazy-loading the hero / LCP image.** It then waits until layout is known, delaying the most important paint. Never put `loading="lazy"` on above-the-fold images.
- Lazy images without `width`/`height` — they start as 0×0, so the page jumps when they arrive, and the browser can misjudge which ones are near the viewport.
- Putting `fetchpriority="high"` on many images: if everything is high priority, nothing is.

Self-check

1. Which image on a product page should get `fetchpriority="high"`? → The main product photo above the fold (usually the LCP element).
2. Does a lazy image block the `load` event? → No.
3. What else can take `loading="lazy"`? → `<iframe>`.

TL;DR

`loading="lazy"` for below-the-fold images and iframes, never for the hero; `fetchpriority="high"` on the single most important image; always give dimensions.

4.5 `<figure>` and `<figcaption>`

What it is

`<figure>` groups **self-contained content that is referenced from the main text** — an image, a diagram, a code listing, a quotation, a table — and `<figcaption>` (first or last child) gives it a caption. The caption becomes the figure's accessible name (the label screen readers announce for it).

Test for "self-contained": could you move it to an appendix or the side of the page, with the text saying "see Figure 3", without breaking the flow? Then it's a figure.

```
<figure>
  <img src="media/hills-800.jpg" alt="Two green hills under a blue sky with a yellow sun"
       width="800" height="533">
  <figcaption>Figure 1 — JPEG, 800x533.</figcaption>
</figure>
```

(Rendered in Figure 4.1 above.)

Caption vs alt

They do different jobs. The **alt** *replaces* the image for people who can't see it; the **caption** *adds* context for everyone. Don't copy one into the other — a screen reader would read the same sentence twice. If the caption already fully describes the image, a short alt is fine.

Common mistakes

- Wrapping *every* image in `<figure>` (decorative images and inline icons are not figures).
- Using a `<p>` or `<div class="caption">` instead of `<figcaption>` — the association is lost for assistive technology.

TL;DR

`<figure>` = referenced, movable content; `<figcaption>` = its visible caption; alt and caption complement, not duplicate.

4.6 Audio and video — `<audio>`, `<video>`, `<source>`, `<track>`

What it is

`<audio>` and `<video>` play media natively in the browser — no plugins (separately installed add-ons such as the old Flash player). Both are **media elements** with the same JavaScript API (the set of functions and properties scripts use to control them: `play()`, `pause()`, `currentTime`, `volume`, events like `play`, `ended`, `timeupdate`).

How it works

1. The browser reads the `<source>` children **in order** and picks the first whose `type` (MIME type — a label such as `video/webm` naming the file format — optionally with codecs, the compression methods used inside the file) it can play. (Safari 17.4+ may pick the best supported one instead of the first.) Or use a single `src` attribute on the element.
2. Content inside the element other than `<source>`/`<track>` (for example a download link) is **fallback** for browsers that don't support the element at all — not for unsupported formats.
3. `<track>` elements attach timed text files in **WebVTT** format (`.vtt`) — captions, subtitles, descriptions, chapters.
4. Without `controls`, the element shows no UI (user interface — buttons, sliders); you would have to build your own buttons with the JS API.

Key attributes:

Attribute	Element	Meaning
`controls`	both	Show the browser's built-in play/pause/volume/seek (jump to a time) UI.
`autoplay`	both	Try to start immediately. Browsers block autoplay **with sound**; muted autoplay is generally allowed.
`muted`	both	Start muted (needed for autoplay to be allowed).
`loop`	both	Restart at the end.
`preload`	both	Hint: `none` (fetch nothing until play), `metadata` (data about the file: duration/dimensions only), `auto` (may download all).
`poster`	video	Image shown before playback starts.
`playsinline`	video	On iPhone, play inside the page instead of going full-screen.
`width` / `height`	video	Reserve space (like `<img>`).

`<track>` attributes: `kind` (`subtitles` default, `captions`, `descriptions`, `chapters`, `metadata`), `src`, `srclang`, `label` (shown in the captions menu), `default` (turn on by default).

Captions vs subtitles. *Subtitles* translate dialogue for viewers who can hear but don't know the language. *Captions* are for viewers who can't hear the audio: they include dialogue **and** meaningful sounds ("[door slams]").

A WebVTT file is plain text:

```
WEBVTT

00:00:00.000 --> 00:00:01.400
The sun rises on the left.

00:00:01.400 --> 00:00:03.000
[soft wind] It drifts to the right.
```

```
<!-- code/05_audio_video.html (excerpt) -->
<audio controls preload="metadata">
  <source src="media/tone.wav" type="audio/wav">
  <p>Your browser cannot play this audio. <a href="media/tone.wav">Download the WAV</a>.</p>
</audio>

<video controls preload="none" poster="media/poster.jpg" width="640" height="360">
  <source src="media/clip.webm" type="video/webm">
  <track kind="captions" src="media/captions.vtt" srclang="en" label="English" default>
  <p>Your browser cannot play this video. <a href="media/clip.webm">Download it</a>.</p>
</video>
```

The second video in the demo is seeked to 2.0 s by script and shows the matching cue. One practical gotcha surfaced here: browsers refuse to load `<track>` files when the page is opened from `file://` (text tracks must be same-origin or served with CORS — a set of HTTP headers by which a server allows other origins to read its files), so the demo adds the same cues (individual timed caption lines) with the TextTrack API — `video.addTextTrack('captions', 'English', 'en').addCue(new VTTCue(1.4, 3.0, '…'))`. Served over `http://`, the `<track>` element works as written.

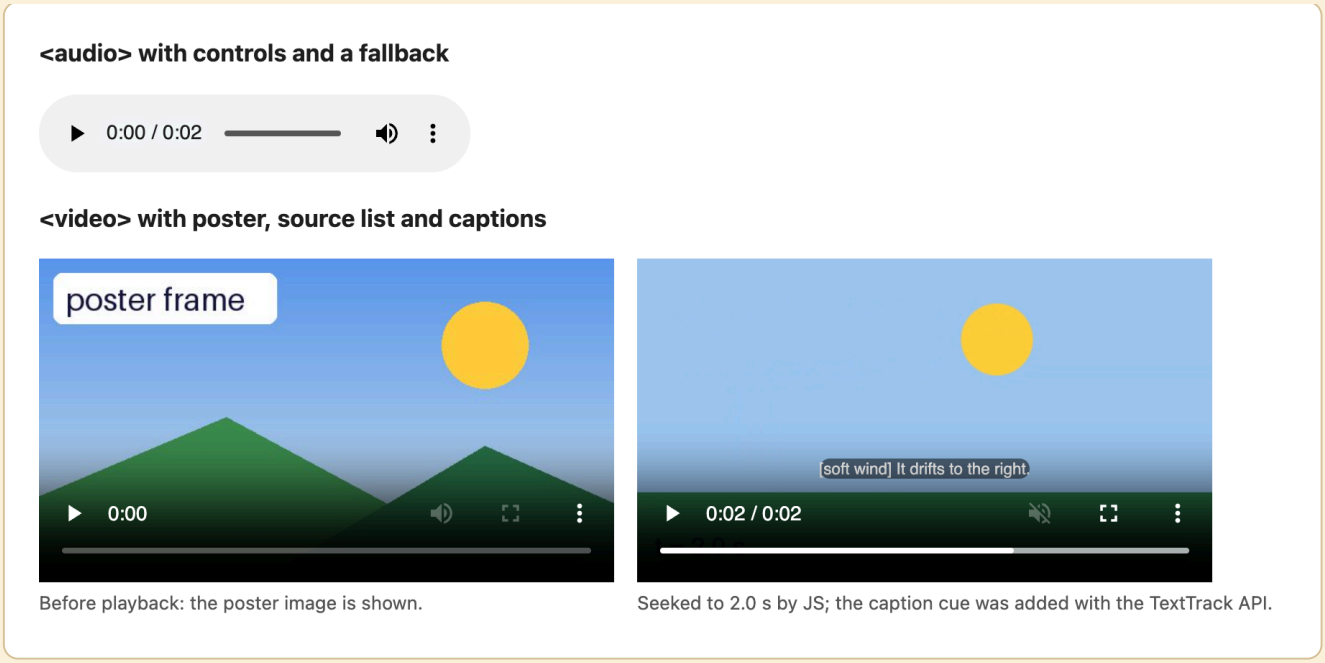


Figure 4.6 — Native controls, a poster before playback, and a caption cue at t = 2 s.

Formats in practice

A **container** is the file wrapper (MP4, WebM) that holds the separately compressed video and audio streams, each squeezed with its own codec (short for coder–decoder).

Container	Typical codecs	Support
MP4	H.264 video + AAC audio	every browser — the safest single choice
WebM	VP9 or AV1 video + Opus audio	Chrome, Edge, Firefox, Safari (14.1+ on macOS, 17.4+ on iOS)
MP4	AV1 or HEVC (H.265)	varies by browser and hardware — always provide a fallback
MP3 / AAC / Opus / WAV	audio	MP3 and AAC everywhere; WAV is uncompressed (raw, not shrunk — large)

List the smallest well-supported format first and MP4/H.264 last as the universal fallback. Declare `type` so the browser can skip formats without downloading them.

Accessibility and UX checklist

- Captions for every video with speech (WCAG 2.2 success criterion 1.2.2); a transcript for audio-only content (1.2.1).
- Don't autoplay with sound; if something moves for more than 5 seconds, users must be able to pause it (2.2.2).
- Give `<video>` a `poster` and dimensions so the page doesn't jump.
- For big sites, video is usually served by a streaming service using **adaptive bitrate** (HLS or DASH, which switch quality with bandwidth — how much data per second your connection can carry) via Media Source Extensions (a browser API that lets JavaScript feed video data into `<video>` piece by piece) — `<video src="file.mp4">` downloads one fixed-quality file.

Common mistakes

- `autoplay` without `muted` and wondering why it doesn't play.
- Thinking the fallback paragraph appears when the *format* is unsupported (it doesn't — it's for browsers without `<video>`).
- Serving a 50 MB unoptimised video file as a "background video".
- `preload="auto"` on a page with many videos — wastes bandwidth.

Real-world examples

Social feeds replace GIFs with `<video autoplay muted loop playsinline>` because the MP4 is typically a fraction of the GIF size. YouTube and Netflix use `<video>` with Media Source Extensions for adaptive streaming.

In the interview

> **30-second answer:** "`<video>` / `<audio>` with `controls`, multiple `<source>` elements with `type` so the browser picks a format it can play, and `<track kind="captions">` with a WebVTT file for accessibility. Autoplay only works muted; add `playsinline` for iOS. For large video, use adaptive streaming (HLS/DASH) instead of one big file."

- *Q: Captions vs subtitles?* A: Captions include non-speech sounds for deaf/hard-of-hearing users; subtitles translate dialogue.
- *Q: Why does my `autoplay` video not start?* A: Browser autoplay policies (built-in rules about when media may start on its own) block audible autoplay; add `muted` (and `playsinline` on iOS).

Self-check

1. Which attribute shows an image before playback? → `poster`.
2. What file format do `<track>` files use? → WebVTT (`.vtt`, starting with the line `WEBVTT`).
3. What does `preload="none"` do? → Nothing is fetched until the user presses play.

TL;DR

Native media elements + `controls` + ordered `<source type>` list + `<track>` captions; muted for autoplay; stream big video adaptively.

4.7 Embedding other pages — `<iframe>`

What it is

An `<iframe>` (inline frame) displays **another complete HTML document** inside a rectangle of your page. The embedded page has its own DOM, its own CSS and JavaScript, and — if it comes from another origin — the browser's **same-origin policy** (the core browser security rule that pages from different origins cannot read each other's data) keeps it and your page from reading each other. (An **origin** is scheme + host + port, e.g. `https://example.com:443`.)

Typical uses: YouTube/Vimeo players, maps, payment fields (Stripe, Adyen host card inputs in iframes so card numbers never touch your page), social posts, ads, code playgrounds (online editors that run code you type).

Why it matters

You are putting someone else's code inside your page. Iframes are also the tool attackers use for **clickjacking** — loading *your* site invisibly inside theirs and tricking users into clicking. Both directions need security controls.

How it works — key attributes

Attribute	Purpose
`src`	URL of the embedded document.
`srcdoc`	Inline HTML to use instead of `src` (handy for previews and sandboxed user content).
`title`	**Required for accessibility**: screen readers announce it so users know what the frame contains.
`width`, `height`	Size (default 300×150).
`loading="lazy"`	Defer loading off-screen frames — a big win for heavy embeds like videos and maps.
`sandbox`	Apply extra restrictions (below).
`allow`	Permissions policy (a list of browser features a page is allowed to use): which powerful features the frame may use, e.g. `allow="fullscreen; camera; geolocation"`.
`referrerpolicy`	How much of your URL to send as the `Referer` header (the request header that tells a server which page the visitor came from), e.g. `no-referrer`.

`sandbox`. An empty `sandbox` attribute turns *everything* off: no scripts, no forms, no pop-ups, no top-level navigation (changing the address of the whole browser tab), and the content is treated as a unique **opaque origin** (so it can't read cookies/storage of its real origin — cookies are small pieces of data a site keeps in your browser). You then add back only what the embed needs, with tokens such as `allow-scripts`, `allow-forms`, `allow-popups`, `allow-same-origin`, `allow-modals`, `allow-downloads`, `allow-top-navigation-by-user-activation`.

> Warning: `allow-scripts` together with `allow-same-origin` on a document from **your own origin** lets its script reach into the parent and remove the sandbox attribute — the sandbox becomes meaningless. Use that combination only for content from a different origin.

The demo loads the same small document (via `srcdoc`) into three frames. Its script writes "allowed" if it ran and then tries to read the parent page's title:

```
<!-- code/06_iframe.html (excerpt) -->
<iframe class="widget" title="Widget without sandbox"></iframe>
<iframe class="widget" sandbox title="Widget, fully sandboxed"></iframe>
<iframe class="widget" sandbox="allow-scripts" title="Widget, scripts only"></iframe>

<!-- inside each frame -->
<script>
  document.querySelector('#js span').textContent = 'allowed (set by JS)';
  let parentTitle;
  try { parentTitle = window.parent.document.title; }
  catch (e) { parentTitle = 'blocked (' + e.name + ')'; }
</script>
```

`no sandbox`

I am a separate document inside an iframe.

Scripts: allowed (set by JS)

Read parent title: Iframe parent page

`sandbox` (everything off)

I am a separate document inside an iframe.

Scripts: blocked (no JS ran)

`sandbox="allow-scripts"`

I am a separate document inside an iframe.

Scripts: allowed (set by JS)

Read parent title: blocked (SecurityError)

A real embed from another site looks like this (not loaded here — this page makes no network requests):

```
<iframe src="https://www.openstreetmap.org/export/embed.html?bbox=..."
        title="Map of the office" width="600" height="400"
        loading="lazy" referrerpolicy="no-referrer"
        sandbox="allow-scripts allow-same-origin allow-popups"></iframe>
```

Figure 4.7 — `srcdoc` content shares the parent's origin, so without a sandbox it can read the parent. `sandbox` blocks scripts entirely; `allow-scripts` lets them run but, without `allow-same-origin`, in an opaque origin — so touching the parent throws `SecurityError`.

The other direction: stopping others from framing *you*

Your server decides whether other sites may embed your pages, using HTTP response headers (labelled extra information sent along with a page):

- `Content-Security-Policy: frame-ancestors 'self'` (modern; can list allowed sites), or
- the older `X-Frame-Options: DENY` / `SAMEORIGIN`.

Banks and login pages send these to prevent clickjacking. That is also why embedding some sites in an iframe simply shows an error.

`<embed>` and `<object>`

Older elements for plugin content (Flash, Java applets — both gone). Today they are mostly used for PDFs (fixed-layout document files), and even then an `<iframe>` or a plain link is usually better. Don't use them for new work.

Common mistakes

- No `title` on the iframe.
- Heavy third-party (provided by another company) embeds (video, maps, chat widgets) loaded eagerly — each can pull in megabytes of scripts. Use `loading="lazy"` or a "click to load" placeholder ("facade").
- `sandbox="allow-scripts allow-same-origin"` on same-origin, untrusted content.
- Using iframes for your own page layout (a 1990s pattern) — breaks URLs, history, accessibility.

In the interview

> **30-second answer:** "An iframe embeds another document with its own browsing context (its own separate window environment: document, history, scripts); cross-origin frames are isolated by the same-origin policy. I always add a `title`, lazy-load off-screen embeds, and restrict untrusted content with `sandbox`, adding only the tokens needed, and `allow` for permissions. To stop my site being framed — clickjacking — I send `Content-Security-Policy: frame-ancestors`."

- *Q: What does an empty `sandbox` do?* A: Blocks scripts, forms, pop-ups, top navigation, plugins and gives an opaque origin.
- *Q: Why do payment providers use iframes?* A: The card field lives on their origin, so the merchant's page (and any malicious script on it) cannot read the card number, which also reduces the merchant's PCI compliance scope (PCI DSS is the card industry's security standard; the less card data you touch, the fewer of its rules apply to you).

Self-check

1. Which attribute is needed for screen readers? → `title`.
2. Which header stops other sites framing your page? → `Content-Security-Policy: frame-ancestors` (or `X-Frame-Options`).
3. Which two sandbox tokens should not be combined for same-origin untrusted content? → `allow-scripts` + `allow-same-origin`.

TL;DR

`<iframe title loading="lazy">`, `sandbox` with the fewest tokens, `allow` for permissions; protect your own pages with `frame-ancestors`.

4.8 SVG basics

What it is

SVG (Scalable Vector Graphics) is an XML language (XML is a general tag-based text format; HTML's stricter cousin) for 2-D vector drawings. Because it describes shapes, not pixels, it stays razor-sharp at any zoom and

is usually tiny for icons and logos. SVG can be written **inline** in HTML (then it is part of the DOM — CSS and JS can style and animate each shape), referenced as a file with `<img src="x.svg">`, or used in CSS (`background-image: url(x.svg)`).

How it works

1. **Coordinate system.** `viewBox="minX minY width height"` defines the drawing's internal coordinates. `viewBox="0 0 200 120"` means x runs 0→200 and y runs 0→120, **y pointing down**. The `width`/`height` attributes (or CSS) set the displayed size; the viewBox is scaled to fit (preserving aspect ratio by default).
2. **Shapes.** `<rect x y width height rx>`, `<circle cx cy r>`, `<ellipse>`, `<line x1 y1 x2 y2>`, `<polyline>`/`<polygon points="x,y x,y …">`, `<text>`.
3. **Paths.** `<path d="…">` is the general shape: `M x y` move to, `L` line to, `H`/`V` horizontal/vertical line, `C` cubic Bézier curve (a smooth curve shaped by two invisible "pull" points), `Q` quadratic curve (the same idea with one pull point), `T` smooth continuation of a quadratic, `A` arc, `Z` close. Upper-case = absolute coordinates, lower-case = relative.
4. **Painting.** `fill` (inside colour), `stroke` (outline colour), `stroke-width`. These are also CSS properties, so they can live in a stylesheet.
5. **Reuse.** Define an icon once with `<symbol id="…" viewBox="…">`, then place copies with `<use href="#id"/>`.
6. **`currentColor`.** `fill="currentColor"` or `stroke="currentColor"` makes the icon take the text colour of its context, so one icon works on a green button and a red one.

```
<!-- code/07_svg.html (excerpt) -->
<svg width="200" height="120" viewBox="0 0 200 120" role="img" aria-labelledby="t1">
  <title id="t1">Basic SVG shapes</title>
  <rect x="10" y="10" width="60" height="40" rx="6" fill="#6c5ce7"/>
  <circle cx="110" cy="30" r="20" fill="#fdcb6e" stroke="#e17055" stroke-width="4"/>
  <line x1="150" y1="10" x2="190" y2="50" stroke="#2d3436" stroke-width="3"/>
  <polygon points="10,110 40,70 70,110" fill="#00b894"/>
  <path d="M90 80 Q 120 40 150 80 T 190 80" fill="none" stroke="#0984e3" stroke-width="3"/>
</svg>

<!-- same viewBox, half the display size: still sharp -->
<svg width="100" height="60" viewBox="0 0 200 120" aria-hidden="true">…</svg>

<!-- icon sprite + currentColor -->
<svg width="0" height="0" style="position:absolute">
  <symbol id="check" viewBox="0 0 24 24">
    <path d="M4 12l5 5L20 6" fill="none" stroke="currentColor" stroke-width="3"/></symbol>
</svg>
<button><svg class="icon" aria-hidden="true"><use href="#check"/></svg>Save</button>

<img src="media/logo.svg" alt="HC logo" width="60" height="60">
```

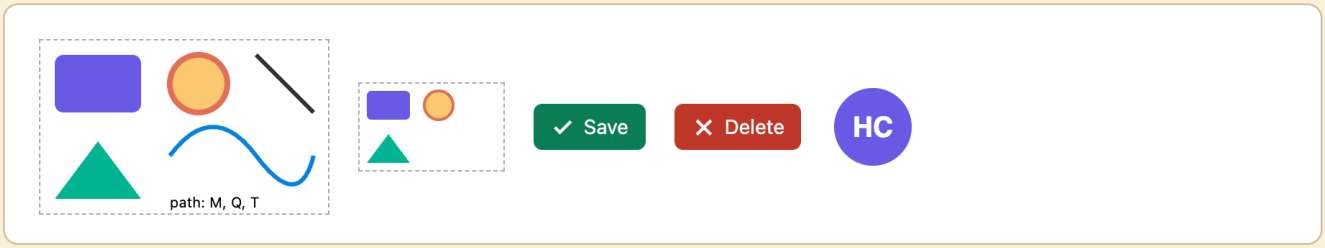


Figure 4.8 — Shapes and a path; the same drawing scaled down; one icon definition coloured by each button's `color` via `currentColor`; an SVG file used as an `<img>`.

Inline SVG vs `<img src="x.svg">`

	Inline `<svg>`	`<img src="x.svg">` / CSS background
Style parts with page CSS / animate with JS	yes	no (it's a closed image)
`currentColor` follows the text colour	yes	no
Cached separately by the browser	no (part of the HTML)	yes
Scripts inside the SVG run	yes — so never inline untrusted SVG	no (scripts and external resources are disabled)
Accessibility	`role="img"` + `<title>`/`aria-label`, or `aria-hidden="true"` if decorative	`alt` like any image

Common mistakes

- Forgetting `viewBox` → the drawing doesn't scale; it gets cropped.
- Icons inside buttons without hiding them (`aria-hidden="true"`) *and* without text → buttons with no accessible name.
- Inlining user-uploaded SVG — it can contain `<script>` (an XSS — cross-site scripting — risk: an attacker's code running in your page). Serve user SVGs through `<img>` or sanitise them (strip out anything dangerous with a cleaning library).
- Exported SVGs from design tools full of editor metadata (extra information the tool stores for itself); run them through an optimiser such as SVGO (a tool that removes unneeded data from SVG files).

Self-check

1. What does `viewBox="0 0 24 24"` mean? → The drawing uses a 24×24 coordinate space, scaled to whatever display size is set.
2. How does one icon change colour per button? → Use `fill`/`stroke="currentColor"` and set CSS `color` on the button.
3. Which way does the y-axis point in SVG? → Down.

TL;DR

SVG = sharp, small, styleable vectors; always set `viewBox`; `symbol` + `use` + `currentColor` for icon systems; never inline untrusted SVG.

4.9 `<canvas>` — an introduction

What it is

`<canvas>` is a blank bitmap (a grid of pixels) that JavaScript paints into, pixel by pixel, through a drawing API — the 2D context (`getContext('2d')`) or WebGL/WebGPU (APIs for GPU-accelerated graphics) for 3-D. Once drawn, the canvas remembers only **pixels**, not shapes: there's no DOM element for "bar 3", so to change something you redraw.

Analogy: SVG is a stack of cut-out paper shapes you can move individually; canvas is a painting — to move the sun you paint over the whole thing again.

How it works

1. The `width`/`height` **attributes** set the size of the drawing buffer (the block of memory holding the canvas's pixels) in pixels (default 300×150). CSS width/height only stretch that buffer on screen, which blurs it.
2. On high-DPR screens, make the buffer `devicePixelRatio` times bigger, keep the CSS size, and `ctx.scale(dpr, dpr)` so you can keep drawing in CSS-pixel coordinates.
3. Draw with methods like `fillRect`, `strokeRect`, `beginPath/moveTo/lineTo/arc/fill/stroke`, `fillText`, `drawImage`, after setting properties like `fillStyle`, `strokeStyle`, `font`, `textAlign`.

4. For animation, clear and redraw inside `requestAnimationFrame` (a browser function that calls your code once just before each screen refresh) (that's how `make_video.mjs` drew the clip for 4.6).

```
<!-- code/08_canvas.html (excerpt) -->
<canvas id="chart" width="420" height="220" role="img"
        aria-label="Bar chart of visits: Mon 12, Tue 19, Wed 7, Thu 15, Fri 22">
  Visits: Mon 12, Tue 19, Wed 7, Thu 15, Fri 22.
</canvas>
<script>
  const data = { Mon: 12, Tue: 19, Wed: 7, Thu: 15, Fri: 22 };
  const canvas = document.getElementById('chart');
  const dpr = window.devicePixelRatio || 1;                // sharp on high-DPI screens
  canvas.style.width = canvas.width + 'px'; canvas.style.height = canvas.height + 'px';
  canvas.width *= dpr; canvas.height *= dpr;
  const ctx = canvas.getContext('2d');
  ctx.scale(dpr, dpr);

  const max = Math.max(...Object.values(data)), barW = 50, gap = 25, base = 190;
  ctx.font = '13px system-ui'; ctx.textAlign = 'center';
  Object.entries(data).forEach(([day, v], i) => {
    const x = 30 + i * (barW + gap), h = (v / max) * 150;
    ctx.fillStyle = '#6c5ce7'; ctx.fillRect(x, base - h, barW, h);
    ctx.fillStyle = '#222';
    ctx.fillText(v, x + barW / 2, base - h - 6);
    ctx.fillText(day, x + barW / 2, base + 18);
  });
  ctx.beginPath(); ctx.moveTo(20, base); ctx.lineTo(410, base); ctx.stroke();
</script>
```

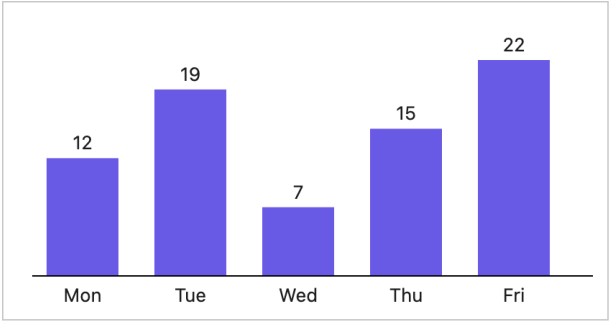


Figure 4.9 — A bar chart painted with the 2D context. The canvas's text content and `aria-label` give the same data to assistive technology, which cannot "see" the pixels.

Canvas vs SVG

	SVG	Canvas
Model	retained (the browser keeps every shape): each shape is a DOM node	immediate (draw and forget): only pixels remain
Scales without blur	yes	no — redraw at the new size
Events per shape (click a bar)	yes, native	you do hit-testing (working out which shape is under the pointer) yourself
Accessibility	text and titles are real content	none built in — add fallback text / `aria-label` / a data table
Performance sweet spot	up to a few thousand elements	many thousands of objects, pixel effects, games, real-time charts
Examples	icons, logos, charts in D3 (a JavaScript charting library)	games, image editors, Figma-style editors (Figma is a browser-based design tool), big-data plots

Common mistakes

- Sizing the canvas with CSS only → blurry, stretched drawing.
- Forgetting fallback content / accessible text for meaningful drawings.

- Drawing text-heavy UI on canvas that would be simpler and more accessible as HTML.

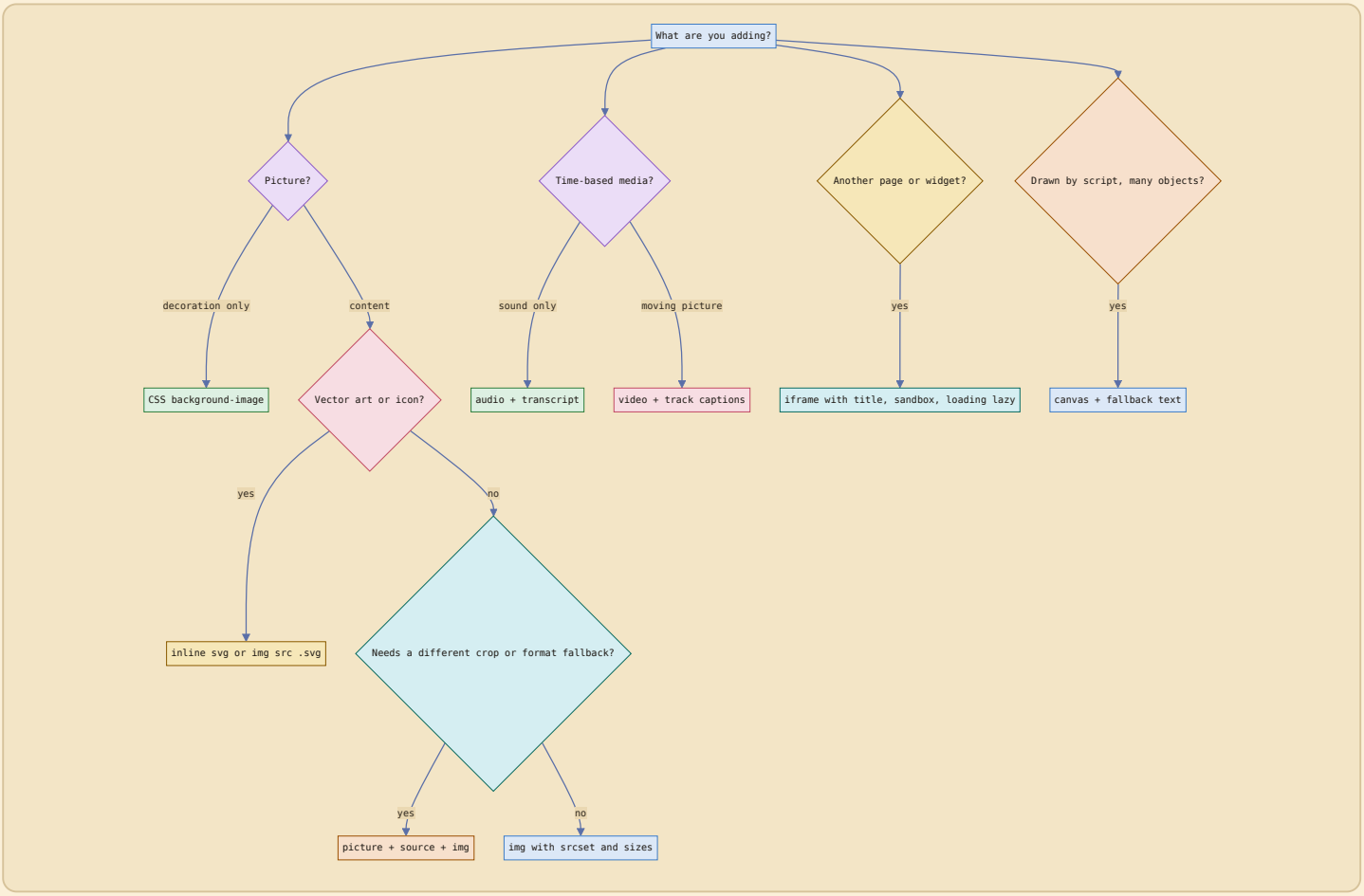
Self-check

1. Default canvas size? → 300×150.
2. Why multiply the buffer by `devicePixelRatio`? → So one canvas pixel maps to one physical pixel on high-DPR screens.
3. Canvas or SVG for a clickable map of 50 regions? → SVG: each region is an element with its own events and accessible name.

TL;DR

Canvas is a JS-painted bitmap: fast for lots of pixels and animation, but no DOM, no built-in accessibility; set buffer size via attributes × DPR.

4.10 Choosing the right element



Chapter interview round-up

1. **How do you make images responsive?** CSS `max-width:100%; height:auto`; `srcset` (`x` or `w` + `sizes`) to serve the right file; `<picture>` for art direction/format fallback; `width`/`height` to avoid layout shift.
2. **How do you improve image loading performance?** Right format (AVIF/WebP), right size (srcset), compress, lazy-load below the fold, `fetchpriority="high"` for the LCP image, dimensions to avoid CLS, a CDN.
3. **When is `alt=""` correct?** For decorative images and images whose information is already in adjacent text.
4. **How would you embed an untrusted HTML snippet?** In an iframe with `sandbox` (no `allow-same-origin`), possibly via `srcdoc`, or sanitise it.
5. **SVG vs PNG for a logo?** SVG: sharp at any size, usually smaller, styleable; PNG only if the artwork is too complex for vectors.

6. **Canvas vs SVG for a chart?** SVG for typical dashboards (pages of charts and key numbers) (accessible, interactive per element); canvas for tens of thousands of points or real-time updates.

Chapter TL;DR

- `<img src alt width height>`: meaningful alt, empty alt for decoration, dimensions to prevent layout shift, fluid via `max-width:100%; height:auto`.
- Formats: SVG for vectors; AVIF/WebP (JPEG fallback) for photos; PNG for pixel-exact graphics; video instead of GIF. Measure — content type changes the winner.
- `srcset` + `sizes` lets the browser choose by device pixels; `<picture>` lets *you* choose by layout (`media`) or format (`type`).
- `loading="lazy"` below the fold, never on the hero; `fetchpriority="high"` for the LCP image.
- `<figure>`/`<figcaption>` for referenced, self-contained content.
- `<video>`/`<audio>` with `controls`, ordered `<source type>`s, WebVTT `<track>` captions; autoplay only when muted.
- `<iframe>`: `title`, `loading="lazy"`, minimal `sandbox` tokens, `allow`; defend your own pages with `frame-ancestors`.
- SVG: `viewBox`, shapes and paths, `symbol`/`use`, `currentColor`; never inline untrusted SVG.
- `<canvas>`: script-painted pixels, scale by `devicePixelRatio`, provide accessible fallback.

Next: Chapter 5 — Tables and Forms.

Chapter 5 — Tables and Forms

Tables and forms are the two parts of HTML where *structure* matters most. A table is not a grid of boxes — it is a set of relationships ("this 135 is North's revenue in Q2") that screen readers (software that reads the page aloud for blind users) announce cell by cell. A form is not a set of text boxes — it is a contract with a server (the remote program that receives the data) about which named values get sent, how, and when, plus a built-in validation engine (the browser's own checker that rejects bad input before it is sent) that most developers under-use.

Get these right and you get keyboard support, autofill (the browser filling in saved details such as your address), mobile keyboards, validation messages and accessibility almost for free. Get them wrong (a `<div>` "button", an input without a label, an error shown only in red) and no amount of JavaScript fully repairs it.

What you'll learn

1. Table semantics: `<caption>`, `<thead>`/`<tbody>`/`<tfoot>`, `<th scope>`, `colspan`/`rowspan`, `headers`, `<colgroup>`.
2. How a form works end to end: `action`, `method`, `name`, encoding, what gets sent (and what doesn't).
3. Labels, and every `<input>` type with its mobile keyboard.
4. `<select>`, `<datalist>`, `<textarea>`, `<fieldset>`/`<legend>`, `<button>`, `<output>`, `<progress>`, `<meter>`.
5. Built-in validation attributes, the `:user-invalid` pseudo-class (a CSS selector that matches an element in a particular state) and the Constraint Validation API (the JavaScript functions and properties for checking fields).
6. `autocomplete` and `inputmode`.
7. Accessible error messages: the pattern used by government and banking sites.

Every example is a file in `code/`. HTML files were screenshotted with headless Chrome (Chrome driven by a script, with no window; 900 CSS px wide); the Node scripts (`table_headers.mjs`, `form_data.mjs`, `constraint_api.mjs`) run under jsdom — a JavaScript implementation of the DOM — and print the outputs shown.

5.1 Table semantics

What it is

A **data table** is a grid where each **data cell** (`<td>`) is described by one or more **header cells** (`<th>`) — a column header, a row header, or both. The HTML elements exist so that software can compute that relationship.

Analogy: reading a timetable, your eyes go up to the column heading and left to the row heading to understand a number. A screen reader has no eyes; `<th>` + `scope` are the lines it follows.

Why it matters

- A screen reader user moving into the cell "135" hears "Q2, North, 135". Without header markup they just hear "135".
- Proper structure lets the browser repeat `<thead>` on every printed page and lets CSS style header rows and columns with plain selectors (the part of a CSS rule that picks which elements it applies to, Chapter 6).

How it works — the elements

Element	Role
`<table>`	The table.
`<caption>`	The table's title — its accessible name (the label assistive technology announces). First child of `<table>`.
`<colgroup>` / `<col>`	Describe columns; only a few CSS properties apply to them (`background`, `border`, `width`, `visibility: collapse`).
`<thead>`	Header row(s).
`<tbody>`	Body rows. You may have several to group rows. Browsers insert one automatically if you omit it.
`<tfoot>`	Footer/summary rows (totals).
`<tr>`	A row.
`<th>`	Header cell. `scope="col"`, `"row"`, `"colgroup"` or `"rowgroup"` says which cells it describes.
`<td>`	Data cell.
`colspan` / `rowspan`	Make a cell span several columns/rows.
`headers="id1 id2"`	On a `<td>`: explicitly list the ids of its header cells (for irregular tables where `scope` isn't enough).

Steps to build a correct table:

1. Start with `<caption>` stating what the table shows (include units).
2. Put the column headings in `<thead>` as `<th scope="col">`.
3. Make the first cell of each body row a `<th scope="row">` if it names the row.
4. Put totals in `<tfoot>`.
5. For two-level headings, use `colspan`/`rowspan` with `scope="colgroup"`/`"rowgroup"`. `colgroup` refers to a column group, so declare the groups with `<colgroup>` elements; `rowgroup` refers to the enclosing `<thead>`/`<tbody>`/`<tfoot>`. For truly irregular tables, use `id` + `headers`.
6. Style with CSS: `border-collapse: collapse`, right-aligned numbers, `font-variant-numeric: tabular-nums` (all digits the same width, so columns line up).

```html
<!-- code/01_table.html (first table) -->
<table>
  <caption>Quarterly revenue by region (USD thousands)</caption>
  <colgroup><col><col><col class="highlight"><col></colgroup>
  <thead>
    <tr><th scope="col">Region</th><th scope="col">Q1</th><th scope="col">Q2</th><th scope="col">Q3</th></tr>
  </thead>
  <tbody>
    <tr><th scope="row">North</th><td class="num">120</td><td class="num">135</td><td class="num">150</td></tr>
    <tr><th scope="row">South</th><td class="num">98</td><td class="num">102</td><td class="num">97</td></tr>
    <tr><th scope="row">West</th><td class="num">143</td><td class="num">160</td><td class="num">171</td></tr>
  </tbody>
  <tfoot>
    <tr><th scope="row">Total</th><td>361</td><td>397</td><td>418</td></tr>
  </tfoot>
</table>
```

```css
table { border-collapse: collapse; }
caption { caption-side: top; text-align: left; font-weight: 700; }
th, td { border: 1px solid #c8c8d8; padding: 6px 10px; }
tbody tr:nth-child(even) td { background: #f8f8fc; }        /* zebra stripes */
td.num, tfoot td { text-align: right; font-variant-numeric: tabular-nums; }
col.highlight { background: #fff6d6; }                     /* whole Q2 column */
```

The second table in the same file has two header levels:

```
<colgroup span="2"></colgroup>   <!-- Term + Course -->
<colgroup span="2"></colgroup>   <!-- the column group that "Students" spans -->
<thead>
  <tr><th rowspan="2" scope="col">Term</th><th rowspan="2" scope="col">Course</th>
      <th colspan="2" scope="colgroup">Students</th></tr>
  <tr><th scope="col">Enrolled</th><th scope="col">Completed</th></tr>
</thead>
<tbody>
  <tr><th rowspan="2" scope="rowgroup">Spring</th><th scope="row">HTML</th><td>40</td><td>35</td></tr>
  <tr><th scope="row">CSS</th><td>32</td><td>30</td></tr>
</tbody>
<tbody> … Autumn … </tbody>
```

Quarterly revenue by region (USD thousands)

Region	Q1	Q2	Q3
North	120	135	150
South	98	102	97
West	143	160	171
Total	**361**	**397**	**418**

Course enrolment (two-level headers)

Term	Course	Students	
		Enrolled	Completed
Spring	HTML	40	35
	CSS	32	30
Autumn	HTML	52	41
	CSS	45	44

Figure 5.1 — Two data tables. The highlighted column comes from a single `<col class="highlight">`.

What assistive technology computes

`table_headers.mjs` rebuilds each table's grid (accounting for `rowspan`/`colspan`) and, for every data cell, collects the `<th>` cells whose `scope` points at it — a simplified form of the HTML standard's "assigning header cells" algorithm (a precise step-by-step procedure):

```
// code/table_headers.mjs (key part)
function headersFor(g, r, c) {          // g = grid of cells, r/c = slot of a data cell
  const found = [];
  const add = h => { if (!found.includes(h)) found.push(h); };
  for (let rr = 0; rr < r; rr++) {      // look up the column
    const h = g[rr][c];
    if (h.tagName === 'TH' && ['col', 'colgroup'].includes(h.scope)) add(h);
  }
  for (let cc = 0; cc < c; cc++) {      // look left along the row
    const h = g[r][cc];
    if (h.tagName === 'TH' && ['row', 'rowgroup'].includes(h.scope)) add(h);
  }
  return found.map(h => h.textContent.trim());
}
```

Output (abridged — the first table prints all nine cells the same way):

```
Table: Quarterly revenue by region (USD thousands)
   120  <- Q1 / North
   135  <- Q2 / North
   150  <- Q3 / North
    98  <- Q1 / South
   ...
Table: Course enrolment (two-level headers)
    40  <- Students / Enrolled / Spring / HTML
    35  <- Students / Completed / Spring / HTML
    32  <- Students / Enrolled / Spring / CSS
    30  <- Students / Completed / Spring / CSS
    52  <- Students / Enrolled / Autumn / HTML
    41  <- Students / Completed / Autumn / HTML
    45  <- Students / Enrolled / Autumn / CSS
    44  <- Students / Completed / Autumn / CSS
```

That is roughly the context a screen reader speaks when you move into each cell.

Responsive tables

Wide tables overflow small screens. The safe fix keeps the table a table and scrolls it inside a labelled, focusable region (so keyboard users can scroll it):

```
<div role="region" aria-labelledby="cap1" tabindex="0" style="overflow-x: auto">
  <table><caption id="cap1">…</caption>…</table>
</div>
```

Changing `display` on the table elements themselves (for example `display: block` on `<table>` to stack cells) has historically made some browsers drop the table semantics — test with a screen reader if you do it.

When not to use it

Never for page layout. Layout tables were common before CSS; today use CSS Grid or Flexbox (the two modern CSS layout systems) (Chapter 8, Chapter 9). If you inherit one, `role="presentation"` on the `<table>` tells assistive technology (screen readers and similar tools) to ignore its table semantics (its meaning as a table).

Common mistakes

- Using `<td><b>…</b></td>` for headers instead of `<th>`.
- No `<caption>` (or a heading outside the table that isn't connected to it).
- The obsolete `summary` attribute — put explanations in the caption or nearby text.
- Empty cells used for spacing; merged cells everywhere "because it looks nicer" — each merge makes the structure harder to navigate.
- Using `<table>` for a list of cards or a form layout.

In the interview

30-second answer: "Tables are for tabular data only. I give them a `<caption>`, use `<thead>/<tbody>/<tfoot>`, mark headers with `<th>` and `scope` (`col`/`row`, or `colgroup`/`rowgroup` for spanning headers), and fall back to `id`/`headers` for irregular tables. That lets screen readers announce the right headers per cell. For small screens I wrap the table in a scrollable, focusable region rather than changing its display."

- *Q: What does `scope` do?* A: Tells assistive technology which cells a header applies to — the rest of its column (`col`), row (`row`), or the group it spans.
- *Q: When do you need `headers`?* A: When cells have headers that aren't in the same row/column in a regular way, e.g. multi-level irregular tables.
- *Q: `border-collapse: collapse` vs `separate`?* A: Collapse merges adjacent cell borders into one line; separate (the default) draws each cell's border with `border-spacing` between them.

Self-check

1. Where does `<caption>` go? → First child of `<table>`.
2. A header spanning two columns — which `scope`? → `colgroup`.
3. How do you right-align numbers so digits line up? → `text-align: right` + `font-variant-numeric: tabular-nums`.
4. How do you tell a screen reader a legacy layout table is not data? → `role="presentation"`.

TL;DR

`caption` + `thead` / `tbody` / `tfoot` + `th scope`; `headers` for irregular tables; never for layout; scroll wide tables in a focusable region.

5.2 How a form works — `action`, `method`, `name`

What it is

A `<form>` collects user input and **submits** it: it builds a list of name/value pairs from its controls (the interactive fields and buttons) and sends it as an HTTP request (a message from browser to server; see Chapter 1). Controls are `<input>`, `<select>`, `<textarea>`, `<button>` (and `<output>`, which is never submitted).

How it works — submission step by step

1. The user presses a submit button, or presses **Enter** in a text field (called *implicit submission*).
2. The browser runs **constraint validation** (5.5). If any control is invalid, it stops, shows a message and focuses the first invalid control.
3. A `submit` event (a notification that scripts can listen for) fires on the form. JavaScript can call `event.preventDefault()` (cancel the browser's normal action) to handle it itself (e.g. with `fetch`, JavaScript's function for sending HTTP requests).
4. The browser builds the **form data set** from *successful controls*:
   - the control has a `name` (no name → not sent);
   - it is not **disabled** (disabled → not sent; `readonly` → sent);
   - checkboxes and radios only if **checked** (value defaults to `"on"` if you omit `value`);
   - `<select multiple>` sends one pair per selected option;
   - only the submit button actually used is included (if it has a name).
5. It encodes the data (converts it into a text format safe to send — e.g. a space becomes `+` and `@` becomes `%40`) using `enctype` and sends it to `action` using `method`:
   - `method="get"` (default): data goes in the URL query string: `/signup?name=Ada+Lovelace&email=ada%40example.com`. Bookmarkable and cacheable (the browser or a server in between may store and reuse the response) — right for searches and filters, wrong for passwords or anything that changes data.
   - `method="post"`: data goes in the request body (the part of the HTTP message after the headers, not shown in the URL). Use for anything that creates or changes something, or is private.
   - `method="dialog"`: inside a `<dialog>`, closes it instead of sending anything.
6. `enctype` (POST only): `application/x-www-form-urlencoded` (default, like a query string), `multipart/form-data` (**required for file uploads**), or `text/plain` (debugging only).
7. The browser navigates to the server's response (unless JavaScript prevented it).

Other attributes: `action` defaults to the current URL; `target` works like a link's target; `novalidate` turns off step 2's built-in UI; a button can override the form with `formaction`, `formmethod`, `formenctype`, `formnovalidate`, `formtarget`. A control *outside* the form can still belong to it with `form="formId"`.

Labels

Every control needs a **label** — the text that names it. A `<label>` is connected in one of two ways:

```
<label for="name">Full name</label>          <!-- explicit: for = the input's id -->
<input id="name" name="name">

<label>Email <input type="email" name="email"></label>   <!-- implicit: wrap the input -->
```

What a real `<label>` gives you: the control's accessible name (screen readers read it on focus — when the control becomes the one receiving keyboard input), a bigger click target (clicking the text focuses the input or toggles the checkbox), and voice-control users can say "click Email". A `placeholder` is **not** a label: it disappears when typing starts, usually has poor contrast (too little difference between text and background colour), and isn't reliably announced.

Buttons

`<button>` has three types, and **the default inside a form is `submit`**:

type	Does
`submit` (default)	Submits the form.
`reset`	Resets every control to its initial value (rarely wanted — users click it by accident).
`button`	Nothing by itself — for JavaScript actions.

Forgetting `type="button"` on a "Show password" or "Add row" button inside a form is a classic bug: clicking it submits the form.

```html
<!-- code/02_form_basics.html (excerpt) -->
<form action="02_form_basics.html" method="get">
  <label for="name">Full name</label>
  <input id="name" name="name" autocomplete="name" value="Ada Lovelace">
  <label>Email <input type="email" name="email" autocomplete="email" value="ada@example.com"></label>
  <label><input type="checkbox" name="newsletter" value="yes" checked> Send me the newsletter</label>
  <input type="hidden" name="source" value="chapter5">
  <button type="submit">Sign up</button>
  <button type="reset">Reset</button>
  <button type="button" id="preview">Preview data</button>
</form>
<script>
  const form = document.querySelector('form');
  const show = () => result.textContent = '?' + new URLSearchParams(new FormData(form));
  show();
  preview.addEventListener('click', show);
</script>
```

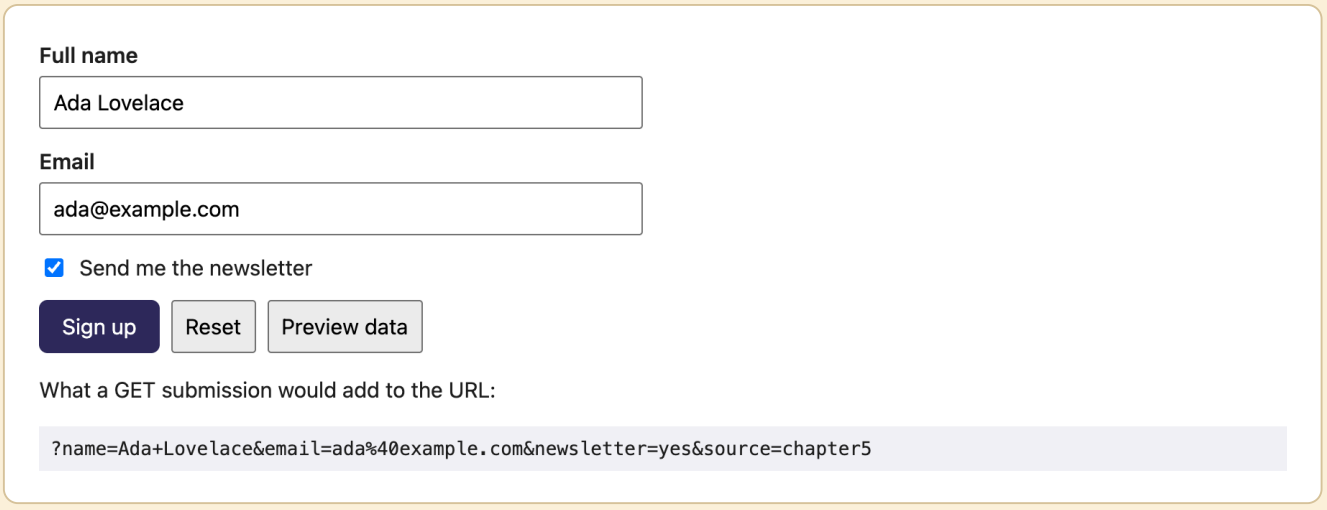


Figure 5.2 — `FormData` + `URLSearchParams` produce exactly the query string a GET submission would send. Note the space encoded as `+` and `@` as `%40`.

Exactly what gets sent

`form_data.mjs` puts every tricky case in one form:

```
// code/form_data.mjs (form markup, abridged)
<input name="name" value="Ada Lovelace">
<input name="email" type="email" value="ada@example.com">
<input name="nick">                                  <!-- empty text: still sent, as "" -->
<input type="checkbox" name="newsletter" value="yes" checked>
<input type="checkbox" name="terms" value="yes">       <!-- unchecked: NOT sent -->
<input type="radio" name="plan" value="free">
<input type="radio" name="plan" value="pro" checked>
<select name="skills" multiple> <option selected>HTML</option><option selected>CSS</option><option>JS</option> </select>
<input name="coupon" value="SAVE10" disabled>          <!-- disabled: NOT sent -->
<input name="country" value="IN" readonly>             <!-- readonly: sent -->
<input value="no name attribute">                      <!-- no name: NOT sent -->
<input type="hidden" name="source" value="ch5">
<textarea name="bio">Hi &amp; bye</textarea>

const data = new FormData(form);
for (const [k, v] of data) console.log(`  ${k.padEnd(10)} = ${JSON.stringify(v)}`);
console.log('\ndata.getAll("skills") =', data.getAll('skills'));
console.log('\nGET URL:\n  ' + form.action + '?' + new URLSearchParams(data));
```

Output:

```
FormData entries:
  name       = "Ada Lovelace"
  email      = "ada@example.com"
  nick       = ""
  newsletter = "yes"
  plan       = "pro"
  skills     = "HTML"
  skills     = "CSS"
  country    = "IN"
  source     = "ch5"
  bio        = "Hi & bye"

data.getAll("skills") = [ 'HTML', 'CSS' ]

GET URL:
  http://localhost/signup?
name=Ada+Lovelace&email=ada%40example.com&nick=&newsletter=yes&plan=pro&skills=HTML&skills=CSS&country=IN&source=ch5&bio=Hi+%26+bye
```

(`http://localhost` is jsdom's pretend page address.) The unchecked checkbox, disabled field and unnamed input are simply absent. The server-side consequence: **"checkbox missing" means "unchecked"** — code that expects `terms=no` will never see it.

Security notes

- Client-side (running in the user's browser) anything (validation, `readonly`, hidden fields, `disabled`) is **not security** — anyone can edit the request. Server-side code runs on your own machine, which users cannot tamper with. The server must validate everything again.
- Never send passwords or personal data with GET — the URL lands in history, logs (servers' records of every request) and `Referer` headers (which tell the next site what page you came from).
- POST forms that change data need CSRF protection on the server (CSRF, cross-site request forgery, is when another site secretly makes your browser submit a form to a site you are logged in to; defences are secret per-form tokens or `SameSite` cookies, which the browser will not send on requests started by other sites; see the JavaScript book).

In the interview

30-second answer: "A form submits its *successful controls* — named, enabled, and for checkboxes/radios, checked — to `action` using `method`. GET puts data in the URL (searches, filters); POST puts it in the body (changes, private data); file uploads need `multipart/form-data`. Every control needs a `<label>`, and buttons default to `type=submit`, so non-submit buttons need `type=button`. Server-side validation is mandatory."

- *Q: Disabled vs readonly?* A: Disabled: not focusable, not submitted, not validated. Readonly: focusable, submitted, not editable, not validated.
- *Q: How do you submit via JS without reloading?* A: Listen for `submit`, `preventDefault()`, send `new FormData(form)` with `fetch`. Use `form.requestSubmit()` (not `form.submit()`) to trigger submission from code — it

runs validation and fires the `submit` event; `submit()` skips both.

Self-check

1. An `<input>` without `name` — is it sent? → No.
2. Which `enctype` for uploads? → `multipart/form-data` (with `method="post"`).
3. Why is a `<button>` inside a form submitting unexpectedly? → Its default type is `submit`.
4. Placeholder as label — why not? → It disappears on typing, often low contrast, and isn't a reliable accessible name.

TL;DR

Named + enabled + checked → sent. GET for reading, POST for changing, multipart for files. Real `<label>`s, `type="button"` for script buttons, validate on the server.

5.3 Input types

What it is

`<input type="…">` changes the control's UI, its validation and — on phones — the on-screen keyboard. Unknown types fall back to `text`, so using a specific type is never worse than `text`.

Figure 5.3 — How Chrome on macOS draws each type (`code/03_input_types.html`). Other browsers draw them differently.

Type	Use for	Built-in validation	Mobile keyboard / UI
`text`	any single-line text	—	normal
`email`	email address	`typeMismatch` if not `x@y` shaped; `multiple` allows a comma list	`@` key visible
`password`	secrets	—	masked (shown as dots); password managers (apps that store and fill passwords) engage
`tel`	phone numbers	**none** (formats vary worldwide)	phone keypad
`url`	web addresses	`typeMismatch` without a scheme like `https:`	`/` and `.com` keys
`search`	search boxes	—	"search" enter key; may show a clear button
`number`	real quantities (count, age)	`min`, `max`, `step`, `badInput`	numeric keypad, spin buttons (small up/down arrows)
`range`	imprecise values (volume)	always has a value within range	slider
`color`	colour pick	value always `#rrggbb`	colour picker
`date`, `time`, `datetime-local`	dates/times	`min`, `max`, `step`	native pickers; value format `YYYY-MM-DD`, `HH:MM`, `YYYY-MM-DDTHH:MM`
`month`, `week`	months/weeks	same	**not supported in desktop Firefox and Safari** — they show a text box
`checkbox`	independent yes/no	`required` = must be checked	—
`radio`	one choice from a group (same `name`)	`required` on any one = must pick one	arrow keys move within the group
`file`	uploads	`accept` filters the picker (not enforced); `multiple`	camera/photos on phones (`capture`)
`hidden`	data the user doesn't see	—	not rendered (and trivially editable — not secret)
`submit`, `reset`, `button`, `image`	buttons (prefer `<button>`, which can contain HTML)	—	—

Rules of thumb:

- `type="number"` **only for numbers you'd do maths on.** A credit-card number, postcode or ID is a *string of digits*: use `type="text" inputmode="numeric"` (and maybe `pattern="[0-9]*"`). `number` drops leading zeros, accepts `e`, and scroll-wheels can change the value.
- The **display** format of a date follows the user's locale (their language-and-region setting — `09/28/2026` for a US locale), but the **value** is always ISO `2026-09-28` (the international standard date format, year-month-day).

`inputmode` and `enterkeyhint`

`inputmode` chooses the virtual keyboard (the on-screen keyboard of a phone or tablet) **without** changing validation: `text`, `numeric`, `decimal`, `tel`, `email`, `url`, `search`, `none`. `enterkeyhint` labels the Enter key: `enter`, `done`, `go`, `next`, `previous`, `search`, `send`.

Other common input attributes

`placeholder` (a hint, not a label), `value` (initial value), `required`, `readonly`, `disabled`, `autofocus` (use sparingly — it can disorient screen-reader users), `size`, `list` (points at a `<datalist>`), `multiple`, `accept`, `spellcheck`, `autocapitalize`.

Self-check

1. Best markup for a 6-digit one-time code? → `type="text" inputmode="numeric" autocomplete="one-time-code"`.
2. What is the value of a date input showing 09/28/2026? → `"2026-09-28"`.
3. Why is `type="tel"` not validated? → Phone formats differ too much by country; add a `pattern` if you need one.

TL;DR

Pick the most specific type for keyboards and validation; `number` only for real numbers; `inputmode` for keyboards without validation side effects.

5.4 Choosing and grouping — `select`, `datalist`, `textarea`, `fieldset`, `output`, `progress`, `meter`

```
<!-- code/04_select_textarea_fieldset.html (excerpt) -->
<label>Country
  <select name="country" autocomplete="country">
    <option value="">-- choose --</option>
    <optgroup label="Europe"><option value="DE">Germany</option><option value="FR">France</option></optgroup>
    <optgroup label="Asia"><option value="IN" selected>India</option><option value="JP">Japan</option></optgroup>
  </select>
</label>

<select name="skills" multiple size="4"> … </select>

<input name="browser" list="browsers" value="Fire">
<datalist id="browsers"><option value="Chrome"><option value="Firefox"><option value="Safari"><option value="Edge">
</datalist>

<textarea name="message" rows="4" maxlength="200">Hello!
Line two keeps its line break.</textarea>

<fieldset>
  <legend>Delivery speed</legend>
  <label><input type="radio" name="speed" value="standard" checked> Standard (free)</label>
  <label><input type="radio" name="speed" value="express"> Express (+$5)</label>
</fieldset>
<fieldset disabled> <legend>Gift options (disabled fieldset)</legend> … </fieldset>

<progress max="100" value="70">70%</progress>
<meter min="0" max="100" low="60" high="85" optimum="20" value="91">91%</meter>
<input type="number" id="qty" name="qty" value="3"> x $12 = <output name="total" for="qty">$36</output>
```

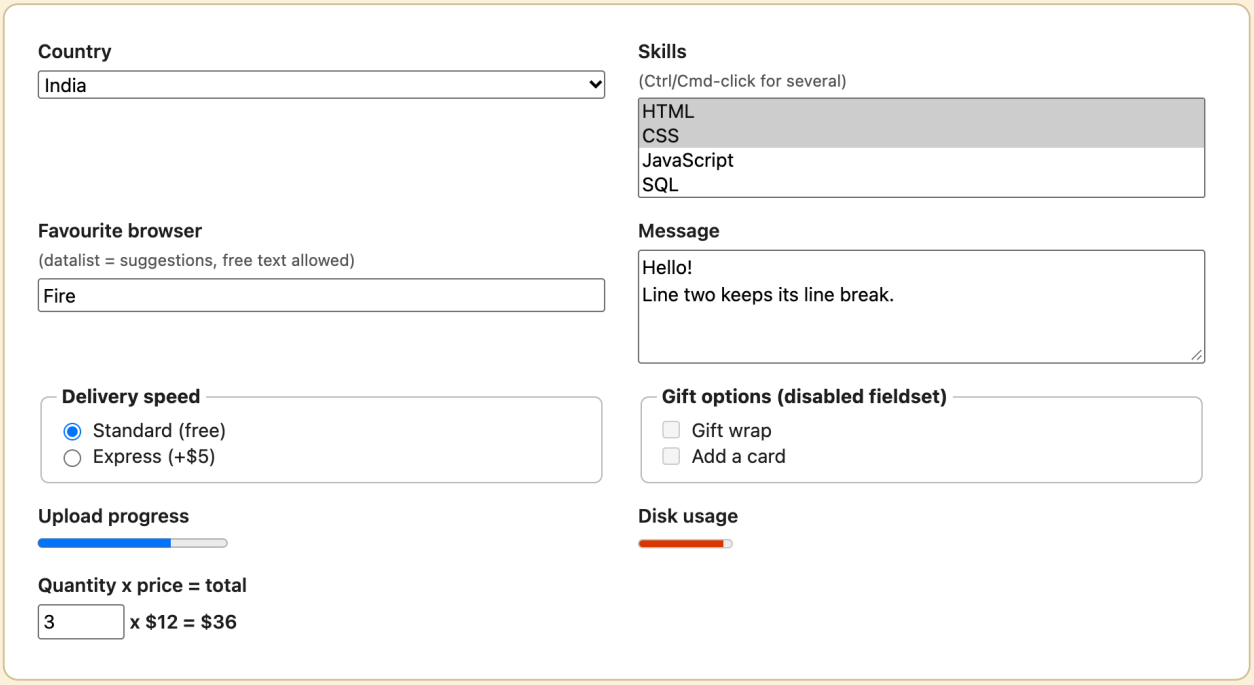


Figure 5.4 — The disabled fieldset greys out and disables every control inside it; the meter turns red because 91 is above `high` and far from `optimum`.

Element	What it is	Key details
`<select>`	dropdown (a list that opens when clicked) of fixed options	Each `<option>` has a `value` (sent) and text (shown). A first option with `value=""` + `required` forces a real choice. `<optgroup label>` groups options.
`<select multiple>`	list box, several choices	Hard to use (Ctrl/Cmd-click); for more than a few options, a group of checkboxes is usually friendlier.
`<datalist>`	**suggestions** for an `<input list>`	The user may still type anything; it's autocomplete, not a restriction.
`<textarea>`	multi-line text	Initial value is its **content** (not a `value` attribute); `rows` / `cols`; `maxlength`; line breaks are preserved. CSS `resize` controls the drag handle.
`<fieldset>` + `<legend>`	groups related controls	Screen readers announce the legend with each control ("Delivery speed, Standard, radio button"). Use it for every radio group and checkbox group (the ARIA alternative is `role="radiogroup"` / `"group"` with `aria-labelledby`). `disabled` on the fieldset disables everything inside.
`<output>`	result of a calculation	`for` lists the ids of inputs it depends on; has an implicit live region (screen readers read changes aloud automatically), so updates can be announced. Not submitted.
`<progress>`	completion of a task	`value` / `max`; omit `value` for an indeterminate ("busy", no known percentage) bar.
`<meter>`	a measurement within a known range	`min`, `max`, `low`, `high`, `optimum` decide the colour (good/average/bad). For disk usage, ratings, password strength. Not for task progress.

Common mistakes

- Radio buttons without a `<fieldset>` / `<legend>` → the question ("Delivery speed") is never announced.
- Radios in one group with different `name`s → they don't exclude each other.
- `<select>` for 2–3 options (radios show all choices at once and need one click fewer) or for 200 countries without type-ahead support (jumping to an option by typing its first letters; a combobox — a text box combined with a filtered suggestion list — may be better).
- Using `<progress>` for a static measurement (use `<meter>`).

Self-check

1. How do you make all controls in a section read-only-looking and unsubmittable at once? → `<fieldset disabled>`.
2. Datalist vs select? → Datalist suggests but allows free text; select restricts to its options.
3. Where does a textarea's initial value go? → Between its tags.

TL;DR

`select` for fixed choices, `datalist` for suggestions, `textarea` for long text, `fieldset` + `legend` to group (always for radio groups), `output` / `progress` / `meter` for results and levels.

5.5 Built-in validation and the Constraint Validation API

What it is

Browsers can validate forms without any JavaScript. You declare **constraints** with attributes; the browser checks them on submit, blocks submission if any fail, shows a message next to the first failing control, and exposes the result to CSS and JavaScript.

The constraint attributes → the validity flag they set

Attribute	Applies to	Fails when	validity flag
`required`	most inputs, select, textarea	empty / unchecked / nothing selected	`valueMissing`
`type="email"`, `type="url"`	input	value not shaped like an email/URL	`typeMismatch`
`pattern="regex"`	text-like inputs	value doesn't match the **whole** regex (regular expression — a mini-language for text patterns; it's implicitly anchored with `^…$`, i.e. must match from start to end)	`patternMismatch`
`minlength` / `maxlength`	text-like, textarea	shorter / longer — **only checked after the user edits the value**	`tooShort` / `tooLong`
`min` / `max`	number, range, date/time	below / above	`rangeUnderflow` / `rangeOverflow`
`step`	number, range, date/time	not a multiple of `step` from the base (`min`, else the initial `value`, else 0)	`stepMismatch`
(the browser)	number, date…	the user typed something unparsable	`badInput`
`setCustomValidity("msg")`	any	message is non-empty	`customError`

Empty optional fields skip every check except `required` — an empty `pattern` field is valid.

Fields that are `disabled`, `readonly`, `type="hidden"`, or inside a `<datalist>` are **barred from constraint validation** (`willValidate === false`).

Styling states: `:invalid` vs `:user-invalid`

`:invalid`/`:valid` match *immediately*, so a required field is red before the user has typed anything — a hostile experience. `:user-invalid`/`:user-valid` only match **after the user has interacted with the field or tried to submit**. They are Baseline 2023 (all major engines since November 2023; widely available since May 2026).

```
input:user-invalid { border-color: #c0392b; background: #fdeeee; }
input:user-valid   { border-color: #1e8449; }
```

The JavaScript API

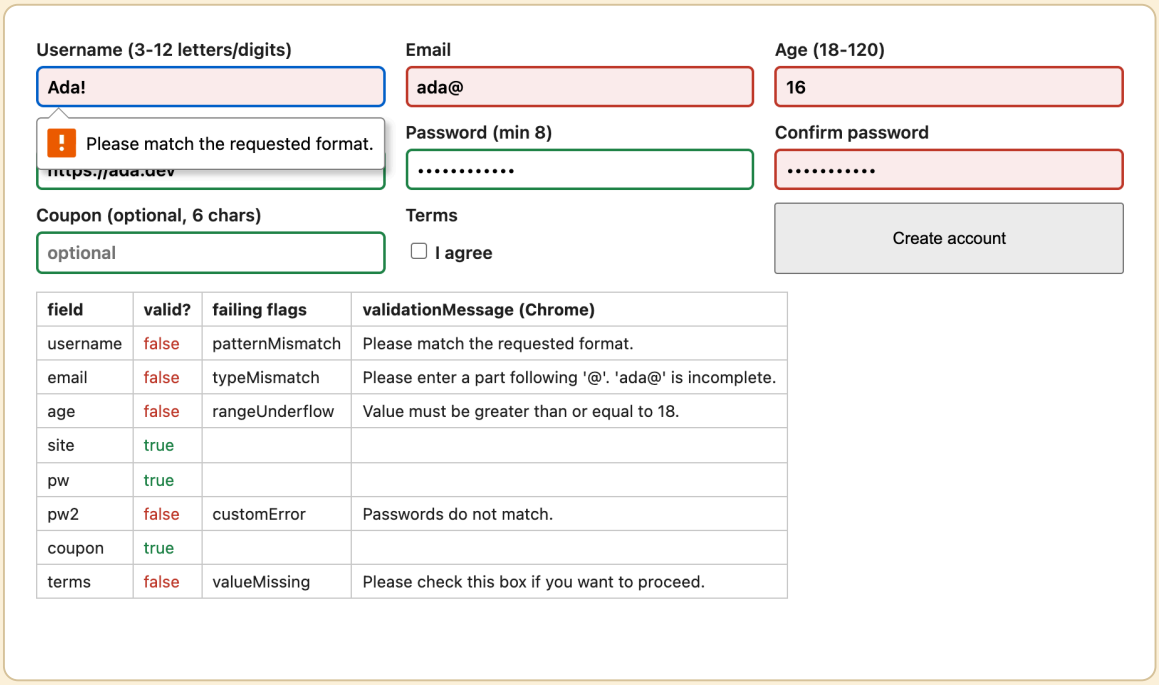
Each control has:

- `validity` — a `ValidityState` object with the flags above and `valid`.
- `validationMessage` — the browser's localised (translated into the user's language) message (empty when valid).
- `willValidate` — whether it takes part in validation at all.
- `checkValidity()` — returns true/false (a **boolean**); fires an `invalid` event on failure.
- `reportValidity()` — same, plus shows the browser's message bubble.
- `setCustomValidity(msg)` — add your own rule; **you must clear it with `''`** when the value becomes valid, or the field stays invalid forever.

Forms have `checkValidity()`, `reportValidity()` and `requestSubmit()`.

Demo in Chrome: a sign-up form with deliberately bad values; the script adds one custom rule (passwords must match), calls `form.requestSubmit()` (exactly like clicking the button), then prints each field's validity:

```
<!-- code/05_validation.html (excerpt) -->
<input name="username" required pattern="[a-z0-9]{3,12}" value="Ada!">
<input name="email" type="email" required value="ada@">
<input name="age" type="number" min="18" max="120" value="16">
<input name="site" type="url" value="https://ada.dev">
<input name="pw" type="password" required minlength="8" value="correcthorse">
<input name="pw2" type="password" required value="correcthors">
<input name="coupon" pattern=".{6}" placeholder="optional">
<input name="terms" type="checkbox" required>
<script>
  function checkMatch() {
    form.pw2.setCustomValidity(form.pw2.value === form.pw.value ? '' : 'Passwords do not match.');
  }
  form.pw.addEventListener('input', checkMatch);
  form.pw2.addEventListener('input', checkMatch);
  checkMatch();
  form.requestSubmit();
</script>
```



field	valid?	failing flags	validationMessage (Chrome)
username	false	patternMismatch	Please match the requested format.
email	false	typeMismatch	Please enter a part following '@'. 'ada@' is incomplete.
age	false	rangeUnderflow	Value must be greater than or equal to 18.
site	true		
pw	true		
pw2	false	customError	Passwords do not match.
coupon	true		
terms	false	valueMissing	Please check this box if you want to proceed.

Figure 5.5 — Chrome blocked the submission, focused the first invalid field and showed its bubble; after the attempted submit, `:user-invalid` (red) and `:user-valid` (green) apply. The empty optional coupon is valid.

The same API from Node, using jsdom:

```
// code/constraint_api.mjs (excerpt)
<input id="a" required>
<input id="b" type="email" value="not-an-email">
<input id="c" type="number" min="1" max="5" value="9">
<input id="d" pattern="[A-Z]{3}" value="abc">
<input id="e" pattern="[A-Z]{3}" value="">
<input id="f" required disabled>
<input id="g" minlength="5" value="ab">
…
d.setCustomValidity('That code is already taken.');
…
form.addEventListener('invalid', () => invalidEvents++, true);   // capture: invalid doesn't bubble
console.log('\nform.checkValidity() =', form.checkValidity(), '| invalid events fired:', invalidEvents);
```

Output:

```
a: willValidate=true valid=false valueMissing
b: willValidate=true valid=false typeMismatch
c: willValidate=true valid=false rangeOverflow
d: willValidate=true valid=false patternMismatch
e: willValidate=true valid=true -
f: willValidate=false valid=true -
g: willValidate=true valid=true -

after d.value = "ABC":           d: willValidate=true valid=true -
after setCustomValidity("..."):  d: willValidate=true valid=false customError | That code is already taken.
after setCustomValidity(""):     d: willValidate=true valid=true -

form.checkValidity() = false | invalid events fired: 3
```

Three details worth remembering from that output:

1. `e` — an empty optional field with a `pattern` is **valid**.
2. `f` — the disabled required field is barred (`willValidate=false`), so it never blocks submission. Chrome gives the same answer.
3. `g` — `minlength` is not enforced on a value set in HTML or by script; `tooShort` only appears after the user types.

Trade-offs: built-in bubbles vs custom messages

	Built-in UI	`novalidate` + your own messages (5.7)
Effort	zero	moderate
Message wording	generic, browser-controlled, localised to the browser language	your words, specific ("Postcode must be 5 digits")
Styling	bubble can't be styled	full control
Shows all errors at once	no — one at a time	yes (error summary)
Accessibility	acceptable; bubbles vanish quickly	best, if done as in 5.7

Most production design systems (a company's shared library of UI components, styles and rules) use `novalidate` and custom messages, **but still use the constraint attributes and API underneath** (`checkValidity()` still works with `novalidate`).

Common mistakes

- Forgetting to reset `setCustomValidity('')` → the field can never become valid.
- `pattern` on `type="email"` duplicating what the type already checks, or patterns too strict for real names ("O'Brien", "José", single-letter surnames).
- Relying on client validation for security.
- Styling `:invalid` so forms load already red.

In the interview

30-second answer: "HTML has declarative constraints — `required`, `type`, `pattern`, `min/max/step`, `minlength/maxlength` — which the browser checks on submit and exposes through `element.validity` flags, `validationMessage`, `checkValidity()` / `reportValidity()` and `setCustomValidity()` for custom rules. Style with `:user-invalid` so errors only show after interaction. Client-side validation is UX (user experience — it makes the form nicer to use); the server must re-validate."

- *Q: `checkValidity` vs `reportValidity`?* A: Both return a boolean and fire `invalid`; `reportValidity` also displays the message to the user.
- *Q: How do you validate "confirm password"?* A: On input, `setCustomValidity(match ? '' : 'Passwords do not match.')`.

Self-check

1. Which flag does an out-of-range number set? → `rangeUnderflow` or `rangeOverflow`.

2. Why might a required field inside `<fieldset disabled>` not block submit? → Disabled controls are barred from validation.
3. `:invalid` vs `:user-invalid`? → The latter waits for interaction or a submit attempt.

TL;DR

Declare constraints in HTML, read them with `validity`, add rules with `setCustomValidity` (and clear it), style with `:user-invalid`, and always validate again on the server.

5.6 `autocomplete` — help the browser fill the form

What it is

The `autocomplete` attribute tells the browser and password managers **what a field means**, using standard tokens, so they can fill it correctly. It's also a WCAG 2.2 (Web Content Accessibility Guidelines, Chapter 3) requirement: success criterion 1.3.5 *Identify Input Purpose* (level AA) asks for it on fields collecting information about the user.

Field	Token
Full name / given / family	`name`, `given-name`, `family-name`
Email, phone	`email`, `tel`
Username	`username`
Login password	`current-password`
Sign-up / change password	`new-password` (triggers strong-password suggestions)
SMS (text message) one-time code	`one-time-code`
Address	`street-address`, `address-line1`, `address-level2` (city), `postal-code`, `country` / `country-name`
Card	`cc-name`, `cc-number`, `cc-exp`, `cc-csc` (expiry date and the 3–4 digit security code)
Birthday	`bday`
Turn off (for things like a search of other people's names)	`off`

Tokens can be prefixed with a section (`shipping street-address`, `billing postal-code`).

Why it matters

Autofill turns a 2-minute checkout into seconds, reduces typos, and helps people with motor or memory disabilities. Password managers rely on `current-password` / `new-password` to tell sign-in from sign-up. WCAG 2.2 also added 3.3.7 *Redundant Entry* (don't make people re-type information they already gave in the same process) — autofill and "same as shipping" checkboxes help meet it.

Common mistakes

- `autocomplete="off"` on login fields "for security" — it pushes users to weaker, reused passwords; browsers often ignore it for passwords anyway.
- Splitting phone numbers or dates into many boxes, which autofill can't fill.
- Blocking paste in password fields (hurts password managers; also runs against WCAG 2.2's 3.3.8 *Accessible Authentication*).

TL;DR

Give every personal-data field its `autocomplete` token; `new-password` vs `current-password`; don't disable autofill or paste.

5.7 Accessible error messages

The problem it solves

Built-in bubbles appear one at a time and disappear; colour alone ("the red one is wrong") fails colour-blind users (people who cannot tell certain colours, often red and green, apart); an error message far from its field is not announced when the field is focused. WCAG requires errors to be **identified in text** (3.3.1), fields to have **labels or instructions** (3.3.2), and fixes to be **suggested** when known (3.3.3).

How it works — the pattern

Popularised by design systems such as the UK's GOV.UK Design System (the UK government's shared web components and patterns):

1. Add `novalidate` to the form so you control the messages; still use constraint attributes and `checkValidity()`.
2. On submit, check every field. For each failing one:
   - set `aria-invalid="true"` (screen readers say "invalid entry");
   - write a specific message into an element next to the field;
   - connect it with `aria-describedby="help-id error-id"` so it's read after the label when the field is focused.
3. Show an **error summary** at the top: a heading with the count and a list of links to each field (`href="#fieldId"`).
4. **Move focus to the summary** (it has `tabindex="-1"` so it can be focused by script) so screen readers announce it immediately.
5. Don't rely on colour: add an icon or the word "Error", and keep the red text at sufficient contrast.
6. Remove `aria-invalid` and the message as soon as the field is fixed.

```
<!-- code/06_accessible_errors.html (excerpt) -->
<form id="f" novalidate>
  <div id="summary" tabindex="-1" hidden>
    <h2>There are 2 problems with your details</h2>
    <ul></ul>
  </div>
  <div class="field">
    <label for="email">Email address</label>
    <span class="help" id="email-help">We'll send the receipt here.</span>
    <input id="email" name="email" type="email" required autocomplete="email"
           aria-describedby="email-help email-err" value="ada@">
    <span class="error" id="email-err"></span>
  </div>
  <div class="field">
    <label for="zip">Postcode <span class="help">(required)</span></label>
    <input id="zip" name="zip" required autocomplete="postal-code" inputmode="numeric"
           pattern="[0-9]{5}" aria-describedby="zip-err" value="">
    <span class="error" id="zip-err"></span>
  </div>
  …
</form>
<script>
  const messages = {
    email: v => v.valueMissing ? 'Enter your email address.' : 'Enter an email address like name@example.com.',
    zip:   v => v.valueMissing ? 'Enter your postcode.' : 'Postcode must be 5 digits.',
    name:  v => 'Enter the name shown on your card.',
  };
  form.addEventListener('submit', e => {
    e.preventDefault();
    const problems = [];
    for (const input of form.querySelectorAll('input')) {
      const err = document.getElementById(input.id + '-err');
      if (input.checkValidity()) { input.removeAttribute('aria-invalid'); err.textContent = ''; }
      else {
        const msg = messages[input.name](input.validity);
        input.setAttribute('aria-invalid', 'true');
        err.textContent = msg;
        problems.push(`<li><a href="#${input.id}">${msg}</a></li>`);
      }
    }
    summary.hidden = problems.length === 0;
    summary.querySelector('ul').innerHTML = problems.join('');
    if (problems.length) summary.focus();
  });
</script>
```

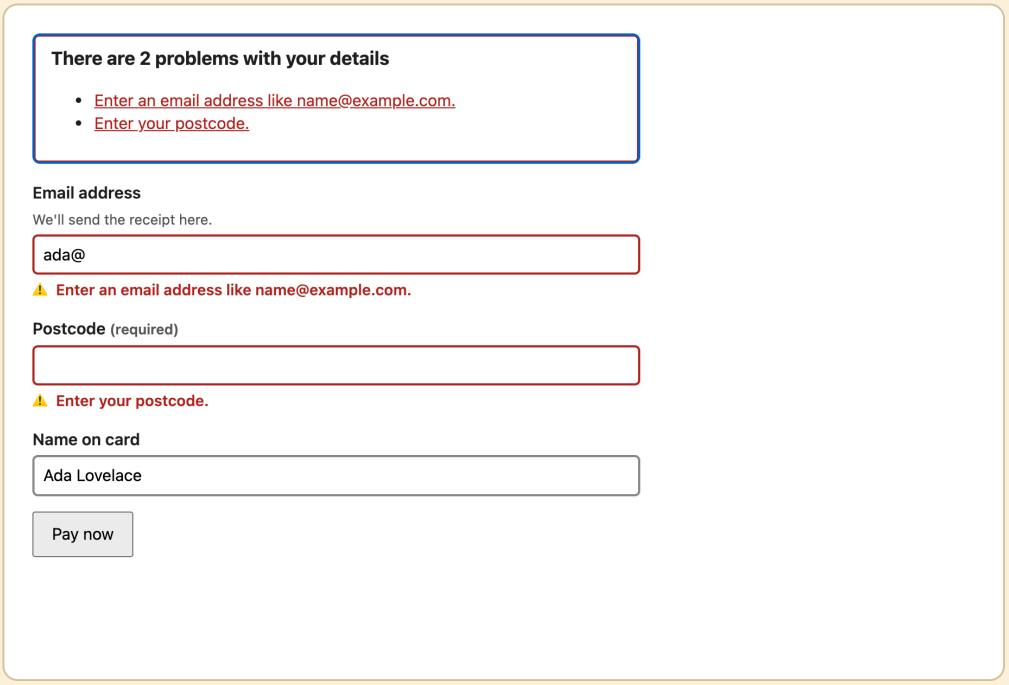


Figure 5.6 — After one failed submit: summary focused (blue focus ring), each error has an icon + text next to its field, and messages say how to fix the problem.

Good error messages

- Say what's wrong **and how to fix it**: "Postcode must be 5 digits", not "Invalid input".
- Use the field's label words; no codes, no blame ("You entered an invalid…").
- Validate on submit (and optionally on `blur` — the event fired when a field loses focus — or, after the first submit, on `input`, fired on every change), not on every keystroke while the user is still typing.

Common mistakes

- Error text not programmatically associated (linked in the markup so software knows they belong together; no `aria-describedby`).
- Only a red border.
- Clearing the whole form on error.
- Disabling the submit button until the form is valid — users can't discover *why* it's disabled. Let them submit and explain.

In the interview

> **30-second answer:** "I use `novalidate` with the constraint API, then on submit mark each bad field with `aria-invalid`, show a specific text message next to it linked via `aria-describedby`, render an error summary with links at the top and move focus to it. Never colour alone, and keep the user's input."

Self-check

1. Which attribute links a field to its error text? → `aria-describedby`.
2. Why `tabindex="-1"` on the summary? → So script can focus it without adding it to the Tab order.
3. What does `aria-invalid="true"` add? → Screen readers announce the field as invalid.

TL;DR

Specific text messages next to fields (`aria-describedby`), `aria-invalid`, a focused error summary with links, never colour alone.

Chapter interview round-up

1. **How do you make a data table accessible?** `<caption>`, `<th>` with `scope`, `thead/tbody/tfoot`, `headers` for irregular tables, scroll container for small screens.
2. **GET vs POST for forms?** GET = idempotent (repeating it has the same effect as doing it once — see System Design Fundamentals) reads, data in URL; POST = changes/private data in body; files need `multipart/form-data`.
3. **What isn't submitted?** Unnamed, disabled, unchecked controls, unselected options, buttons other than the one used, `<output>`.
4. **Why a `<label>` rather than a placeholder?** Accessible name, larger click area, persists while typing.
5. **How does built-in validation work and how do you customise it?** Constraint attributes → `validity` flags; `setCustomValidity`; `:user-invalid`; `novalidate` + custom UI for full control.
6. **What does `autocomplete="new-password"` do?** Tells password managers it's a new password (offer to generate/save), not a login.
7. **`<button>` inside a form refreshes the page — why?** Default `type="submit"`.

Chapter TL;DR

- Tables are for data: `caption`, `thead`/`tbody`/`tfoot`, `th scope`, `headers` for complex cases; never for layout.
- A form sends **named, enabled, checked** controls to `action` via `method`; GET for reads, POST for changes, multipart for files.
- Every control needs a real `<label>`; group radios/checkboxes with `fieldset` + `legend`; buttons default to submit.
- Choose the most specific `type`; `inputmode` for keyboards; `number` only for numeric quantities.

- Built-in validation: `required`, `type`, `pattern`, `min/max/step`, `minlength/maxlength` → `validity`; `setCustomValidity` for extra rules; `:user-invalid` for styling.
- `autocomplete` tokens on personal-data fields; don't block paste or autofill.
- Accessible errors: `aria-invalid`, text messages via `aria-describedby`, focused error summary, never colour alone — and always validate on the server.

Next: Chapter 6 — CSS Fundamentals.

Chapter 6 — CSS Fundamentals

HTML says *what* things are; **CSS (Cascading Style Sheets)** says how they look: colours, fonts, spacing, layout, animation. This chapter covers the machinery every later CSS chapter relies on — how rules are written and attached, how selectors (patterns that say which elements a rule applies to) pick elements, and above all how the browser decides which of several competing rules wins. Most "CSS is broken" moments are really misunderstandings of three ideas: **specificity**, **the cascade** and **inheritance**. After this chapter you should be able to predict the winner of any rule conflict on paper — and prove it with the scripts here.

What you'll learn

1. CSS syntax and the three ways to add CSS to a page.
2. Every kind of selector: type, class, ID, universal, attribute (all 7 forms), combinators, selector lists.
3. Pseudo-classes (structural, state, logical `:is()` / `:where()` / `:not()` / `:has()`) and pseudo-elements (`::before`, `::after`, `::first-letter`, `::marker`, …).
4. Specificity — scored by hand in a worked table, and computed by a small Node program.
5. The cascade: origins, `!important`, inline styles, cascade layers, source order — with a program that models it.
6. Inheritance and the keywords `inherit`, `initial`, `unset`, `revert`.
7. Units: `px`, `em`, `rem`, `%`, `vw` / `vh` (and the new `dvh` family), `ch`.
8. Colours: named, hex, `rgb()`, `hsl()`, `oklch()`, alpha, `currentColor`, `color-mix()`.
9. Custom properties (CSS variables).

All examples are in `code/`. The HTML pages were screenshotted in headless Chrome (Chrome run by a script, with no window; 900 CSS px wide); several print the values the browser actually computed (via `getComputedStyle`, the JavaScript function that returns an element's final style values) right on the page. `specificity.mjs` and `cascade.mjs` are plain Node programs (JavaScript run outside the browser with Node.js).

6.1 Syntax and adding CSS to a page

What it is

A stylesheet is a list of **rules**. Each rule has a **selector** (which elements) and a **declaration block** (what to do to them). Each **declaration** is a **property**, a colon, a **value**, and a semicolon.

```
/*  selector      declaration block                    */
   h1          {  color: navy;   font-size: 2rem;  }
/*                property value  property  value      */
```

Other syntax pieces:

- **Comments** are `/* … */` (there are no `//` comments in CSS).
- **At-rules** start with `@`: `@media` (conditions), `@import` (load another sheet), `@font-face` (load a font file), `@keyframes` (define an animation), `@layer`, `@supports` ("only if the browser supports this feature"), `@container` (conditions on a parent's size).
- CSS is forgiving: an **invalid declaration is ignored** (not the whole rule), and an unknown selector in a list invalidates the whole rule — the basis of progressive enhancement (building a page that works everywhere and adding extras that only newer browsers use; older browsers skip what they don't understand).
- Property names and keywords are case-insensitive; class names, IDs and attribute values follow the document's case rules (case-sensitive in HTML).

Three ways to add CSS

Method	How	Use for	Drawbacks
External	`<link rel="stylesheet" href="styles.css">` in `<head>`	real sites	one extra request (but cached — kept by the browser for reuse — and shared by every page)
Internal	`<style>…</style>` in `<head>`	single pages, demos, critical above-the-fold CSS (styles for what is visible before scrolling)	not cached separately, repeated per page
Inline	`style="color: red"` on an element	one-off values set by JavaScript, emails	beats selectors (hard to override), no media queries (screen-size conditions, Chapter 10) or pseudo-classes (§6.3), clutters HTML

`@import url(other.css);` inside a stylesheet also works, but it makes the browser discover the second file only after downloading the first — a slow chain. Prefer several `<link>`s or a build tool (a program that processes your source files before publishing) that bundles files (joins them into one).

Stylesheets in `<head>` are **render-blocking**: the browser won't paint (draw the page) until it has them, which prevents a flash of unstyled content (the page briefly showing as plain unstyled HTML) but means large CSS files delay the first paint (Chapter 14).

```
<!-- code/01_adding_css.html (excerpt) -->
<head>
  <link rel="stylesheet" href="01_styles.css">          <!-- 1. external -->
  <style>
    .internal { background: #eafbea; border-color: #2e9d49; }   /* 2. internal */
  </style>
</head>
<body>
  <div class="box external">1. Styled by an external file</div>
  <div class="box internal">2. Styled by an internal &lt;style&gt; element</div>
  <div class="box" style="background: #fff3e0; border-color: #e67e22;">3. Inline style</div>
  <div class="box external" style="border-style: dashed;">External class + inline style</div>
</body>
```

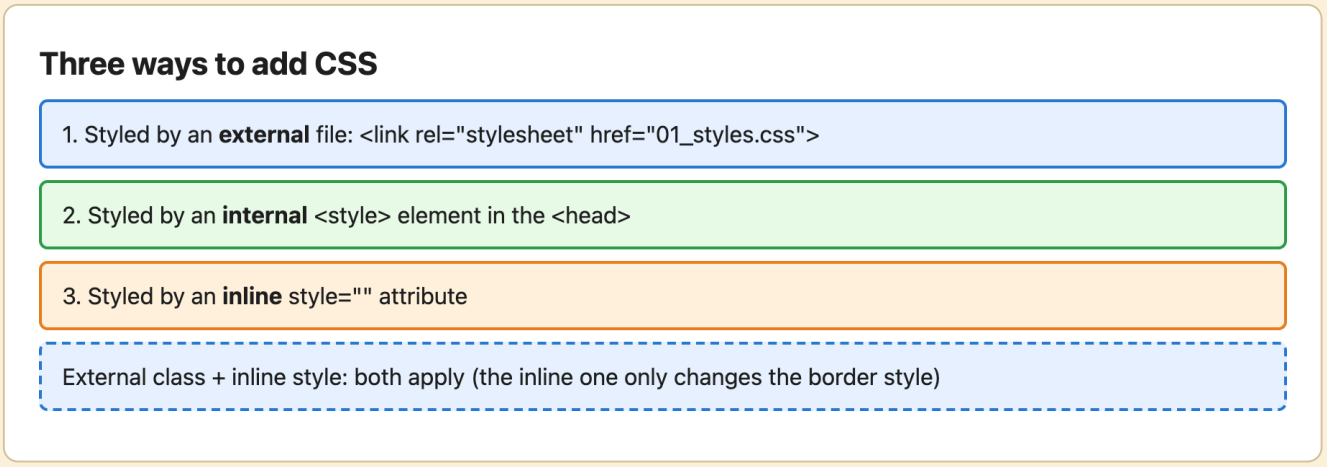


Figure 6.1 — Styles from all sources combine; the last box gets its colours from the external file and only its border style from the inline attribute.

> **TL;DR**

Rule = selector + `{ property: value; }`. Use external stylesheets; inline styles only when you must. Invalid declarations are skipped, not fatal.

6.2 Selectors

What it is

A **selector** is a pattern that matches elements in the DOM (the browser's tree of page elements, Chapter 1). The browser tests each rule's selector against elements to decide where declarations apply.

Basic selectors

Selector	Example	Matches
Type (element)	`p`	every `<p>`
Class	`.card`	every element whose `class` list contains `card`
ID	`#main`	the element with `id="main"` (IDs should be unique per page)
Universal	`*`	every element
Compound (no space)	`button.primary`	a `<button>` that also has class `primary`
Selector list	`h1, h2, h3`	any of them (one invalid selector in the list drops the whole rule, except inside `:is()` / `:where()`, which are "forgiving")

Attribute selectors — all seven forms

Selector	Matches when the attribute…	Example use
`[attr]`	exists (any value)	`a[title]`, `input[required]`
`[attr="v"]`	equals `v` exactly	`input[type="email"]`
`[attr~="v"]`	is a space-separated list containing the word `v`	`[data-tags~="new"]`
`[attr|="v"]`	is exactly `v` or starts with `v-`	`[lang|="en"]` matches `en`, `en-GB`
`[attr^="v"]`	starts with `v`	`a[href^="https"]`
`[attr$="v"]`	ends with `v`	`a[href$=".pdf"]`
`[attr*="v"]`	contains `v` anywhere	`a[href*="example"]`

Add `i` before `]` to compare case-insensitively: `a[href$=".pdf" i]` also matches `CV.PDF`.

Combinators — relationships between elements

Combinator	Example	Matches
Descendant (space)	`nav a`	an `a` anywhere inside a `nav`, at any depth
Child `>`	`ul > li`	an `li` that is a **direct** child of a `ul`
Next sibling `+`	`h2 + p`	the `p` **immediately** after an `h2` (same parent)
Subsequent siblings `~`	`h2 ~ p`	**every** `p` after an `h2` (same parent)

Selectors are matched **right to left** by browsers: for `nav a`, the engine finds every `a` and then checks for a `nav` ancestor (an element anywhere above it in the tree: parent, grandparent…). That's why the rightmost part (the **key selector**) matters most for performance — though for normal pages selector speed is rarely a real problem.

The demo page applies one yellow highlight to whatever each selector matches:

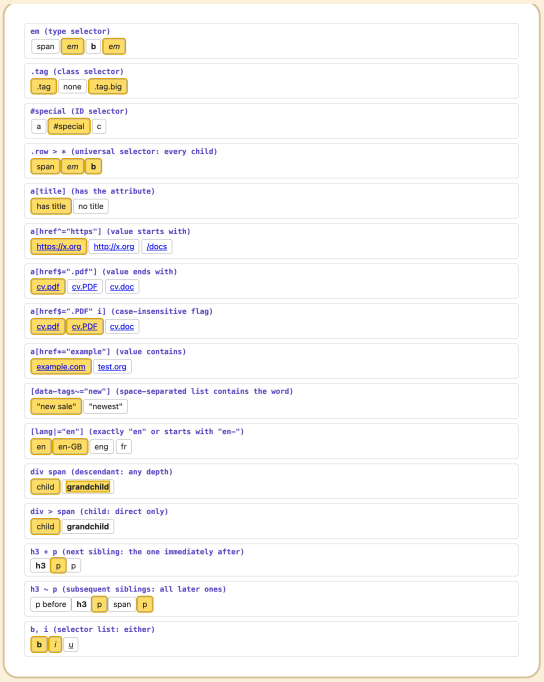


Figure 6.2 — `code/02_selectors.html`. Compare "div span" (child and grandchild both match) with "div > span" (only the direct child), and "h3 + p" (one paragraph) with "h3 ~ p" (every later paragraph, skipping the span, never the paragraph before).

Common mistakes

- Writing `.nav .active` (descendant) when you meant `.nav.active` (the same element has both classes).
- Styling by ID everywhere — IDs are extremely specific (6.4), making later overrides painful.
- Long chains like `body div.container ul li a` — fragile (breaks when HTML changes) and highly specific. Prefer one meaningful class.
- Forgetting that `+` and `~` only look at *later* siblings with the *same parent*.

Self-check

1. Select PDF links regardless of case? → `a[href$=".pdf" i]`.
2. `.menu li` vs `.menu > li` for a nested menu list? → The first matches items at every nesting level; the second only the top-level items.
3. Style the paragraph immediately after each heading? → `h2 + p`.

TL;DR

Types, classes, IDs, `*`, 7 attribute forms, 4 combinators, comma lists. Prefer classes; keep selectors short.

6.3 Pseudo-classes and pseudo-elements

What they are

- A **pseudo-class** (one colon, `:hover`) selects elements in a particular **state or position** that isn't expressed by a simple attribute — being hovered, being the first child, being checked.
- A **pseudo-element** (two colons, `::before`) selects a **part** of an element, or creates a box that isn't in the HTML — the first letter, the list bullet, generated content.

The old single-colon forms `:before`, `:after`, `:first-line`, `:first-letter` still work for backwards compatibility; new code uses `::`.

Pseudo-classes by family

Family	Pseudo-classes	Notes
User action	`:hover`, `:active`, `:focus`, `:focus-visible`, `:focus-within`	`:focus-visible` matches only when the browser thinks a focus ring (the outline showing which element has keyboard focus) is needed (keyboard use), so you can remove rings for mouse clicks without hurting keyboard users. Link order rule: `:link`, `:visited`, `:hover`, `:active` ("LVHA").
Links	`:link`, `:visited`, `:any-link`, `:target`	`:visited` is restricted to a few colour properties for privacy. `:target` = the element whose id is in the URL `#fragment`.
Structural	`:first-child`, `:last-child`, `:only-child`, `:nth-child(An+B)`, `:nth-last-child()`, `:first-of-type`, `:nth-of-type()`, `:only-of-type`, `:empty`, `:root`	`An+B`: `odd` = `2n+1`, `even` = `2n`, `3n` = every third, `-n+3` = first three. `:nth-child(2 of .item)` counts only elements matching `.item`.
Form state	`:checked`, `:disabled`, `:enabled`, `:required`, `:optional`, `:valid`, `:invalid`, `:user-valid`, `:user-invalid`, `:in-range`, `:out-of-range`, `:placeholder-shown`, `:read-only`, `:default`, `:indeterminate`	See Chapter 5.
Logical	`:not(list)`, `:is(list)`, `:where(list)`, `:has(relative list)`	`:is()` and `:where()` shorten long lists (`:is(h1,h2,h3) a`); they differ only in specificity (6.4). `:has()` matches an element **based on what it contains or what follows it** — the long-awaited "parent selector" (Baseline — supported in all major browsers — since December 2023, widely available since June 2026).
Other	`:lang(en)`, `:dir(rtl)`, `:defined`, `:fullscreen`, `:modal`, `:popover-open`	

`:nth-child` vs `:nth-of-type`: `li:nth-child(2)` means "an `li` that is the 2nd child of its parent" (fails if the 2nd child is not an `li`), while `span:nth-of-type(2)` means "the 2nd `span` among its siblings", ignoring other element types.

Pseudo-elements

Pseudo-element	Selects / creates
`::before`, `::after`	a generated child box at the start/end of the element's content. **Only exists if `content` is set** (`content: ""` for a purely decorative box). Replaced elements (elements whose content comes from outside the CSS, such as an image file) such as `<img>` and most `<input>` types don't render them.
`::first-letter`	the first letter of a block (drop caps — a large first letter spanning several lines)
`::first-line`	the first **rendered** line — changes as the width changes
`::marker`	a list item's bullet or number
`::placeholder`	an input's placeholder text
`::selection`	text the user has highlighted
`::file-selector-button`	the button of `<input type="file">`
`::backdrop`	the layer behind a modal (page-blocking) `<dialog>` or full-screen element

Content added with `::before`/`::after` (called *generated content*) is visual decoration: screen readers (software that reads pages aloud) may or may not read it and users can't select it, so never put essential information there.

```css
/* code/03_pseudo.html (excerpt) */
#p-odd li:nth-child(odd)            { background: #ffe066; }
#p-of  li:nth-child(2 of .item)     { background: #ffe066; }
#p-not li:not(.skip)                { background: #ffe066; }
#p-has .card:has(img)               { background: #ffe066; }
#p-checked input:checked + label    { background: #ffe066; }

.quote::before { content: "\201C"; }            /* opening curly quote */
.quote::after  { content: "\201D"; }
.required::after { content: " *"; color: #c0392b; }
a.ext[href^="http"]::after { content: " \2197"; }  /* arrow after external links */
.dropcap::first-letter { font-size: 2.6em; float: left; line-height: 1; }
.firstline::first-line { font-weight: 700; }
ol.fancy li::marker { color: #e67e22; }
input.ph::placeholder { color: #9b59b6; font-style: italic; }
.sel::selection { background: #2d2a5a; color: #fff; }
```

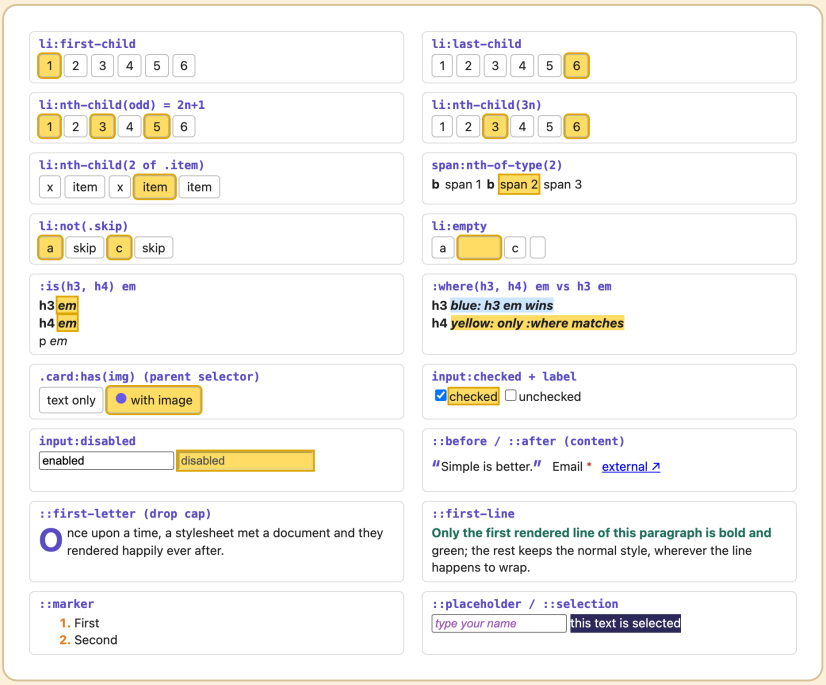


*Figure 6.3 — `code/03_pseudo.html`. Note the `:empty` row: the `<li>` containing only a space is **not** matched — whitespace counts as content in current browsers.*

Common mistakes

- `::before` with no `content` → nothing appears.
- `li:first-child` when the first child is a `<div>` → nothing matches (use `:first-of-type` or restructure).
- Removing focus outlines with `:focus { outline: none }` → keyboard users are lost. Style `:focus-visible` instead.
- Putting real text (prices, labels) in `content:`.

In the interview

30-second answer: "Pseudo-classes select elements by state or position — `:hover`, `:focus-visible`, `:nth-child`, `:checked`, `:has()`. Pseudo-elements style a part of an element or generate content — `::before` / `::after` need `content`, plus `::first-letter`, `::marker`, `::placeholder`, `::selection`. `:has()` lets a parent be styled by its contents, and `:is()` / `:where()` group selectors, differing only in specificity."

- *Q: `:nth-child(2)` vs `:nth-of-type(2)`?* A: The first counts all siblings, the second only siblings of the same element type.
- *Q: How would you style a card differently when it contains an image, without JavaScript?* A: `.card:has(img) { … }`.

Self-check

1. Select every third row starting from the first? → `tr:nth-child(3n+1)`.

2. Why doesn't `img::after` work? → `<img>` is a replaced element; its content comes from the image file, so there is nowhere to insert generated boxes.
3. Highlight a form group that contains an invalid input? → `.group:has(:user-invalid)`.

TL;DR

`:` = state/position, `::` = part/generated box. Structural (`nth-child`), state (`:checked`), logical (`:is/:where/:not/:has`); `::before/::after` need `content`.

6.4 Specificity

What it is

When several rules set the **same property on the same element**, and they come from the same origin and layer (both explained in §6.5), the browser compares their selectors' **specificity** — a measure of how *targeted* each selector is. The more specific selector wins, regardless of source order. Only when specificity ties does the later rule win.

Analogy: specificity is like a three-digit score read left to right, but with **no carrying**: a thousand points in the second digit can never add up to one point in the first.

How it works — the score (A, B, C)

Count the parts of a selector into three columns:

Column	Counts	Examples
A	ID selectors	`#main`
B	class selectors, attribute selectors, pseudo-classes	`.btn`, `[type="text"]`, `:hover`, `:nth-child(2)`
C	type selectors and pseudo-elements	`p`, `li`, `::before`
(nothing)	universal `*`, combinators (, `>`, `+`, `~`), `:where(…)`	

Special rules:

1. `:is(…)`, `:not(…)` and `:has(…)` add the specificity of their **most specific argument** — even if a less specific argument is the one that matched.
2. `:where(…)` always adds **zero** — perfect for low-priority defaults in libraries (reusable code written by others) and resets (stylesheets that wipe out browsers' default styles).
3. `:nth-child(An+B of S)` counts as one pseudo-class **plus** the most specific selector in `S`.
4. Compare A first; only if equal compare B; only if equal compare C. **No carrying:** `(0,11,0)` loses to `(1,0,0)`.
5. Inline `style=""` isn't scored with selectors — it beats any selector (6.5). `!important` beats both (6.5).
6. Repeating a class (`.btn.btn`) really does add specificity — sometimes used as a deliberate override hack.

Worked scoring table

Work each selector left to right, putting every simple selector into its column:

Selector	A (IDs)	B (classes, attrs, pseudo-classes)	C (types, pseudo-elements)	Score
`*`	–	–	–	(0,0,0)
`li`	–	–	`li`	(0,0,1)
`ul > li + li`	–	–	`ul` `li` `li`	(0,0,3)
`.nav`	–	`.nav`	–	(0,1,0)
`a:hover`	–	`:hover`	`a`	(0,1,1)
`input[type="email"]`	–	`[type="email"]`	`input`	(0,1,1)
`p::first-line`	–	–	`p` `::first-line`	(0,0,2)
`.nav a.active`	–	`.nav` `.active`	`a`	(0,2,1)
`#main .card p`	`#main`	`.card`	`p`	(1,1,1)
`#main #sidebar`	`#main` `#sidebar`	–	–	(2,0,0)
`:is(#a, .b, p) span`	`#a` (max arg)	–	`span`	(1,0,1)
`:where(#a, .b, p) span`	–	–	`span`	(0,0,1)
`button:not(.primary, [disabled])`	–	`.primary` (max arg — tie, either counts 1)	`button`	(0,1,1)
`.card:has(> img)`	–	`.card`	`img` (from the argument)	(0,1,1)
`li:nth-child(2n+1 of .item)`	–	`:nth-child` + `.item`	`li`	(0,2,1)
`html body div#app main.content article.post > p:first-child::first-letter`	`#app`	`.content` `.post` `:first-child`	`html` `body` `div` `main` `article` `p` `::first-letter`	(1,3,7)

The calculator

`specificity_lib.mjs` implements these rules in about 70 lines (a scanner that walks the selector character by character, recursing into — calling itself again on the inside of — `:is()` / `:not()` / `:has()`), and `specificity.mjs` prints the scores and **checks each against the hand-computed value from the table above**:

```
// code/specificity_lib.mjs (core loop, abridged)
export function specificity(selector) {
  let s = [0, 0, 0];
  // ... walk the string:
  //   '#name'  -> s += [1,0,0]      '.name' or '[...]' -> s += [0,1,0]
  //   '::name' or legacy ':before' etc.              -> s += [0,0,1]
  //   ':where(...)'                                  -> nothing
  //   ':is(...)', ':not(...)', ':has(...)'           -> s += max(specificity of each argument)
  //   ':nth-child(An+B of S)'                        -> s += [0,1,0] + max(specificity of S)
  //   any other ':pseudo-class'                      -> s += [0,1,0]
  //   'name' (type selector)                         -> s += [0,0,1]
  //   '*', spaces, '>', '+', '~'                     -> nothing
  return s;
}
export const compare = (x, y) => x[0] - y[0] || x[1] - y[1] || x[2] - y[2];
```

Output of `node specificity.mjs` (no `MISMATCH` markers means every computed score equals the hand-computed one):

```
selector                                                                (A,B,C)
*                                                                       (0,0,0)
li                                                                      (0,0,1)
ul li                                                                   (0,0,2)
ul > li + li                                                            (0,0,3)
.nav                                                                    (0,1,0)
a:hover                                                                 (0,1,1)
input[type="email"]                                                     (0,1,1)
p::first-line                                                           (0,0,2)
p:first-line                                                            (0,0,2)
.nav a.active                                                           (0,2,1)
#main                                                                   (1,0,0)
#main .card p                                                           (1,1,1)
#main #sidebar                                                          (2,0,0)
:is(#a, .b, p) span                                                     (1,0,1)
:where(#a, .b, p) span                                                  (0,0,1)
button:not(.primary, [disabled])                                        (0,1,1)
.card:has(> img)                                                        (0,1,1)
li:nth-child(2n+1 of .item)                                             (0,2,1)
html body div#app main.content article.post > p:first-child::first-letter    (1,3,7)

Winner order for <nav id="top" class="nav"><a class="active">…</a></nav> (highest first):
  1. #top a                   (1,0,1)
  2. a:is(.active, #x)        (1,0,1)
  3. nav a.active             (0,1,2)
  4. .nav a                   (0,1,1)
  5. :where(#top) a.active    (0,1,1)
  6. a                        (0,0,1)
```

In the last list, the two `(1,0,1)` selectors tie, so whichever appears **later in the stylesheet** would win; `a:is(.active, #x)` scores an ID even though the link matched through `.active`.

Seeing it in the browser

Each box in `04_specificity.html` has competing rules; a script prints the colour Chrome actually computed:

```css
/* code/04_specificity.html (excerpt) */
#c1 p      { color: gray; }    /* (1,0,1) */
#c1 .lead  { color: green; }   /* (1,1,0) */
#c1 #intro { color: blue; }    /* (2,0,0)  <- wins */

#c3 .a.b.c.d.e.f.g.h.i.j.k { color: green; }  /* (1,11,0) */
#c3 #x                     { color: blue; }   /* (2,0,0)  <- wins: A is compared first */

#c6 :is(#nope, .z) { color: blue; }  /* (2,0,0) -- counts #nope although .z matched */
#c6 .z.z.z         { color: green; } /* (1,3,0) */
```

Figure 6.4 — Every prediction confirmed by `getComputedStyle`. Each rule is prefixed with its case id (`#c1 …`) so cases don't interfere; that prefix adds `(1,0,0)` to every selector in a case without changing who wins.

Trade-offs — keeping specificity low

High specificity forces ever-higher specificity to override it: a "specificity war" ending in `!important` everywhere. Strategies:

- Style with **single classes** (`.card-title`), as in BEM (a class-naming convention: `block__element--modifier`) or utility CSS (many tiny one-purpose classes such as `.mt-4`; Chapter 13).
- Avoid IDs in selectors (use them for anchors — link targets — and JS hooks — handles scripts use to find an element).
- Wrap defaults in `:where()` so anything can override them.
- Use **cascade layers** (`@layer`) to order whole groups of rules regardless of specificity (6.5).

In the interview

> **30-second answer:** "Specificity is a three-part score — IDs, then classes/attributes/pseudo-classes, then types/pseudo-elements — compared column by column with no carrying, so one ID beats any number of classes. `:is`, `:not` and `:has` take their most specific argument; `:where` is zero; `*` and combinators are zero. Equal scores fall back to source order; inline styles and `!important` sit above specificity in the cascade."

- *Q: Does `.a.a` beat `.a`?* A: Yes, (0,2,0) vs (0,1,0).
- *Q: Score of `#nav ul li a:hover`?* A: (1,1,3).
- *Q: How do you write a reset that anything can override?* A: `:where(ul, ol) { margin: 0 }` → specificity (0,0,0); or put it in an early `@layer`.

Self-check

1. Score `ul#menu > li.item:not(.disabled) a[href]`. → A: `#menu` = 1; B: `.item`, `.disabled` (from `:not`), `[href]` = 3; C: `ul`, `li`, `a` = 3 → (1,3,3).
2. Which wins: `div div div div p` or `.text`? → `.text` (0,1,0) beats (0,0,5).
3. Specificity of `:where(.x) .y`? → (0,1,0).

TL;DR

(IDs, classes/attrs/pseudo-classes, types/pseudo-elements), compared left to right, no carrying; `:is/:not/:has` = max argument, `:where` = 0; keep it low with single classes.

6.5 The cascade and origins

What it is

The **cascade** is the algorithm (a fixed step-by-step procedure) that picks one value when several declarations try to set the same property on the same element. Specificity is only one of its steps. Following CSS Cascading and Inheritance Level 5, declarations are compared by these criteria **in order**, stopping at the first difference:

1. **Origin and importance** — where the declaration came from, and whether it's `!important`.
2. **Context** — shadow DOM encapsulation (a way for a component to keep a private DOM and styles sealed off from the page; beyond this chapter).
3. **Element-attached styles** — a `style=""` attribute beats rules from stylesheets.
4. **Cascade layers** — which `@layer` the rule is in.
5. **Specificity** (6.4).
6. **Order of appearance** — the later declaration wins.

Origins

- **User-agent** styles — the browser's built-in stylesheet (why `<h1>` is big and bold and `<a>` is blue).
- **User** styles — set by the person using the browser (accessibility settings, extensions — add-ons installed in the browser).

- **Author** styles — yours.

Precedence, strongest first:

Rank	Declarations
1	CSS transitions (smooth changes between two values, Chapter 12) (while running)
2	`!important` user-agent
3	`!important` user
4	`!important` author
5	CSS animations (while running)
6	normal author
7	normal user
8	normal user-agent

The reversal for `!important` is deliberate: normally your CSS beats the user's, but a user who marks something `!important` (for example, large text for low vision) beats you.

Cascade layers (`@layer`)

Layers let you order **groups** of rules explicitly, so a low-priority group can never win by accident of specificity:

```
@layer reset, base, components;          /* order declared up front: later = stronger */
@layer reset      { #l1 p   { color: gray; } }
@layer components { .card p { color: teal; } }
p.note { color: navy; }                  /* unlayered: stronger than every layer */
```

For **normal** declarations: later layers beat earlier ones, and **unlayered styles beat all layers**. For `!important` declarations it flips: earlier layers win, and layered `!important` beats unlayered `!important`. `@layer` has been Baseline since 2022; architecture patterns with layers are in Chapter 13.

A model of the cascade

`cascade.mjs` turns criteria 1, 3, 4, 5 and 6 into a sort key and sorts the competing declarations for `<p id="intro" class="lead" style="color: purple">`:

```
// code/cascade.mjs (key part)
const ORIGIN_RANK = {
  'normal ua': 1, 'normal user': 2, 'normal author': 3,
  'important author': 5, 'important user': 6, 'important ua': 7,
};
function layerRank(decl, layerOrder) {
  const n = layerOrder.length;
  const pos = decl.layer ? layerOrder.indexOf(decl.layer) : n;     // unlayered = "last layer"
  return decl.important ? n - pos : pos;                           // !important flips the order
}
function key(decl, layerOrder) {
  const origin = ORIGIN_RANK[`${decl.important ? 'important' : 'normal'} ${decl.origin}`];
  const attached = decl.inline ? 1 : 0;
  const spec = decl.inline ? [0, 0, 0] : specificity(decl.selector);
  return [origin, attached, layerRank(decl, layerOrder), ...spec, decl.order];
}
```

Output (the key is `origin.attached.layer.A.B.C.order`; bigger wins, compared left to right):

```
A. Specificity decides (same origin, no layers):
  WIN color: blue     #intro                [author]  key=3.0.0.1.0.0.1
      color: green    .lead                 [author]  key=3.0.0.0.1.0.2
      color: black    p                     [author]  key=3.0.0.0.0.1.0

B. Equal specificity -> the later rule wins:
  WIN color: orange   .lead                 [author]  key=3.0.0.0.1.0.1
      color: green    .lead                 [author]  key=3.0.0.0.1.0.0

C. Inline style beats any selector; !important beats inline:
  WIN color: red      p                     [author, !important]  key=5.0.0.0.0.1.2
      color: purple   style=""              [author]  key=3.1.0.0.0.0.1
      color: blue     #intro                [author]  key=3.0.0.1.0.0.0

D. Origins: the browser default loses to author CSS, but user !important beats everything else:
  WIN color: yellow   p                     [user, !important]  key=6.0.0.0.0.1.2
      color: blue     #intro                [author, !important]  key=5.0.0.1.0.0.1
      color: canvastext p                     [ua]  key=1.0.0.0.0.1.0

E. Layers (@layer reset, base, components): unlayered wins even with lower specificity:
  WIN color: navy     p                     [author]  key=3.0.3.0.0.1.2
      color: teal     .lead                 [author, @layer components]  key=3.0.2.0.1.0.1
      color: gray     #intro                [author, @layer reset]  key=3.0.0.1.0.0.0

F. ...but with !important the layer order flips: the earliest layer wins:
  WIN color: gray     p                     [author, !important, @layer reset]  key=5.0.3.0.0.1.0
      color: teal     #intro                [author, !important, @layer components]  key=5.0.1.1.0.0.1
      color: navy     #intro                [author, !important]  key=5.0.0.1.0.0.2
```

Case E is confirmed in the browser in Figure 6.5 below (first box: navy).

`!important` — when (not) to use it

`!important` jumps a declaration into a higher band of the cascade. Legitimate uses: utility classes that must always apply (`.hidden { display: none !important }`), overriding third-party (written by another company) inline styles you can't edit, user stylesheets. Everywhere else it's a sign of a specificity problem, and the only way to beat it is *another* `!important`.

Common mistakes

- Believing "the last rule always wins" — only among equal origin, layer and specificity.
- Fighting a library's styles with ever-longer selectors instead of putting the library in a lower `@layer`.
- Forgetting that the browser's own stylesheet exists (margins on `body`, `h1`, `ul`…) — hence resets/normalize (normalize.css: a stylesheet that makes browser defaults consistent instead of removing them; Chapter 13).

In the interview

30-second answer: "The cascade picks a winner by comparing origin and importance first — user-agent, user, author, with `!important` reversing that order — then inline style attributes, then cascade layers (later layers win, unlayered beats layered, reversed for `!important`), then specificity, then source order. So specificity only matters between rules in the same origin and layer."

- *Q: Why can't my `#id` rule beat a Bootstrap (a popular ready-made CSS library) class?* A: It can unless the class uses `!important`; a common real cause is load order or a layer.
- *Q: Where would you put a third-party CSS library?* A: `@import url(lib.css) layer(vendor);` or wrap it in an early layer so your unlayered or later-layer rules win without specificity tricks.

TL;DR

Origin & importance → inline → layer → specificity → order. `!important` reverses origins and layers. Use layers, not `!important`, to control priority.

6.6 Inheritance

What it is

Some properties pass their value from parent to child automatically — **inherited properties**. Most **text** properties inherit (`color`, `font-*`, `line-height`, `letter-spacing`, `text-align`, `visibility`, `cursor`, `list-style`); most **box** properties don't (`margin`, `padding`, `border`, `background`, `width`, `display`, `position`). Each property's definition in the spec (and on MDN, Mozilla's web reference site) says "Inherited: yes/no".

Inheritance only applies when **no declaration** for that property matched the element — any matching rule, however weak, beats an inherited value. That's why `a { color: blue }` from the browser's own stylesheet beats `body { color: red }` for links.

Analogy: inherited properties are the family's language — children speak it unless taught otherwise; borders are the parents' coats — not handed down.

The global keywords (valid for every property)

Keyword	Meaning
`inherit`	Use the parent's computed value (the final value after the cascade, with units resolved) — even for a normally non-inherited property.
`initial`	Use the property's **initial value from the spec** (e.g. `display: initial` is `inline`, `color: initial` is `canvastext` — usually black).
`unset`	Acts as `inherit` for inherited properties and `initial` for the rest — "as if no rule matched".
`revert`	Roll back to what the **previous origin** would give — for author styles, usually the browser's default (`display: revert` on a `<div>` gives `block`, not `inline`).
`revert-layer`	Roll back to the value from the previous cascade layer.

`all: unset` (or `all: revert`) applies a keyword to every property at once — handy to strip a `<button>`'s default look before restyling it.

```
/* code/05_cascade_inheritance.html (excerpt) */
.parent { color: #b03a2e; font-style: italic; border: 2px solid #b03a2e; padding: 6px; }

.kw .inherit { border: inherit; }            /* copies the parent's border */
.kw .initial { color: initial; }             /* spec default: black */
.kw .unset   { color: unset; border: unset; }/* color inherits, border resets */
.kw .block        { display: block; }
.kw .block.revert { display: revert; }       /* back to the browser default for <b>: inline */

.fonts .fixed { font: inherit; }             /* form controls don't inherit font by default */
```

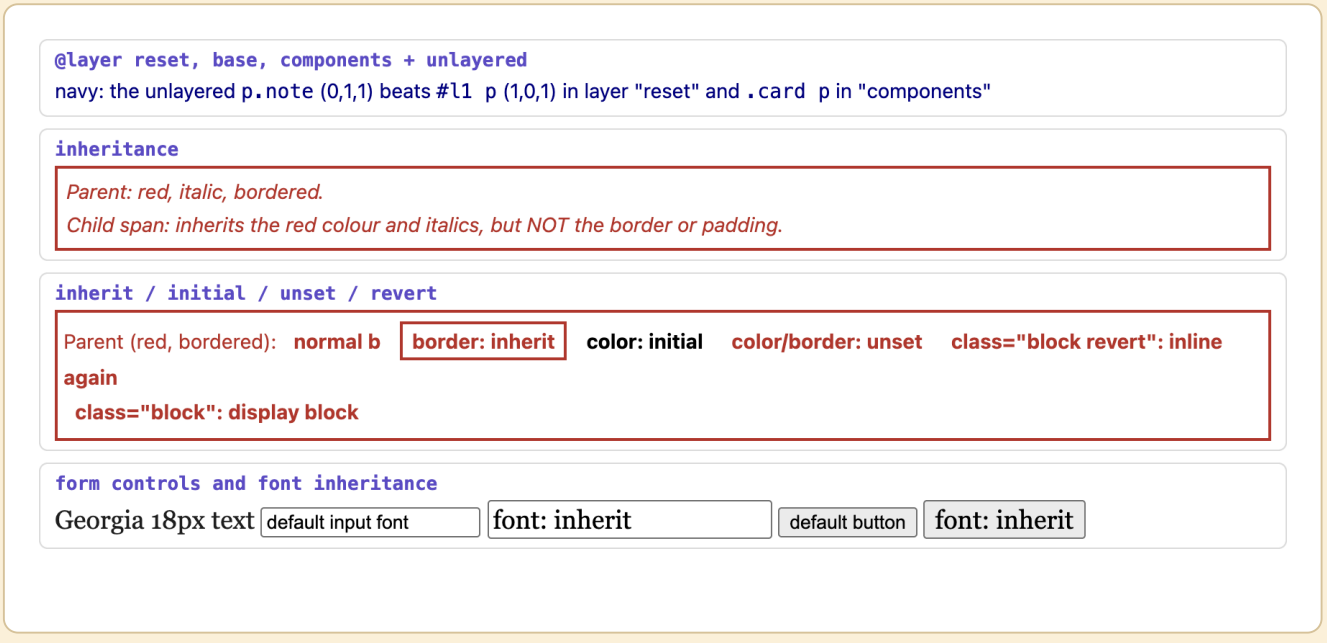


Figure 6.5 — Inheritance and the global keywords. Last row: the browser's stylesheet gives form controls their own font, so they ignore the page font until you add `font: inherit` — one of the first lines in most CSS resets.

Self-check

1. Does `padding` inherit? → No.
2. `color: unset` on a child of a red parent? → Red (color is inherited, so `unset` = `inherit`).
3. Why doesn't `body { color: red }` make links red? → The UA stylesheet has a rule matching `a`; a matching rule beats inheritance.

TL;DR

Text properties inherit, box properties don't; any matching rule beats inheritance; `inherit` / `initial` / `unset` / `revert` control it explicitly.

6.7 Units

What it is

CSS lengths are **absolute** (fixed size) or **relative** (depend on something else: the font, the parent, the viewport). Relative units are what make layouts adapt to user settings and screen sizes.

Unit	Relative to	Typical use
`px`	the CSS pixel (1/96 inch at arm's length; not a device pixel — one physical dot on the screen)	borders, shadows, small fixed details
`em`	the element's own `font-size` — **except in `font-size` itself, where it's the parent's**	padding/margins that scale with the text, e.g. buttons
`rem`	the root (`<html>`) font-size — 16px by default	font sizes and spacing across the site
`%`	depends on the property: `width`, `padding`, `margin` (**including top/bottom**) → containing block's (usually the parent's content box, Chapter 7) **width**; `height` → containing block's height; `font-size` → parent font-size	fluid widths
`vw` / `vh`	1% of the viewport (the visible browser area) width / height	full-screen sections, fluid type (text that grows with the screen)
`svh` / `lvh` / `dvh`	small / large / dynamic viewport height (accounts for mobile browser toolbars; Baseline 2022)	`min-height: 100dvh` hero sections (the big banner at the top of a page)
`vmin` / `vmax`	the smaller / larger of `vw`, `vh`	square elements that fit any orientation
`ch`	width of the "0" glyph (the drawn shape of a character) of the current font	text measure (line length), e.g. `max-width: 65ch`; input widths
`ex`, `lh`, `rlh`, `cqw`…	x-height (height of a lowercase "x"), line height, container query width (Chapter 10)…	specialised
`cm`, `mm`, `in`, `pt`, `pc`	physical units (1in = 96px)	print stylesheets (CSS used when the page is printed)

```css
/* code/06_units.html (excerpt) */
html { font-size: 16px; }                /* 1rem = 16px */
body { font-size: 14px; }
.em  li { font-size: 1.2em; }            /* 1.2 x parent -> compounds when nested */
.rem li { font-size: 1.2rem; }           /* always 1.2 x 16 = 19.2px */
.btn { font-size: 20px; padding: 0.5em 1em; }   /* em = own font-size */
.outer { width: 400px; }
.pct   { width: 50%; padding-top: 10%; }  /* both relative to the parent's WIDTH */
.vw { width: 50vw; }
.measure { max-width: 40ch; }
```

Figure 6.6 — Values computed by Chrome and printed by the page's script.

Worked arithmetic from the figure:

1. **em compounding:** body 14px → level 1 = 14 × 1.2 = **16.8px** → level 2 = 16.8 × 1.2 = **20.16px** → level 3 = 20.16 × 1.2 = **24.192px**. With `rem` every level is 16 × 1.2 = **19.2px**.
2. **em in padding:** the button's own font-size is 20px → 0.5em = **10px**, 1em = **20px**.
3. **%:** 50% of a 400px parent = **200px** wide; `padding-top: 10%` is 10% of the parent's *width* = **40px** (not of its height).
4. **vw:** 50vw of a 900px viewport = **450px**.
5. **ch:** 40 × the width of "0" in 14px system-ui ≈ **352.7px**.

Why `rem` for font sizes

If a user sets their browser's default font size to 20px for readability, everything in `rem` (and `em`) scales with it; sizes in `px` ignore the setting. This only works if you **don't set the root font-size in `px`** (the demo does, just to make the arithmetic exact) — leave it alone or use a percentage. The common practice: `rem` for type and spacing, `em` for component-internal spacing that should track its own text, `px` for hairlines (very thin lines, e.g. 1px borders), `%`/`fr` (a grid fraction unit, Chapter 9)/`vw` for layout.

Common mistakes

- Nested `em` font sizes snowballing (or shrinking) unexpectedly.
- `height: 100vh` on mobile hiding content behind the browser toolbar — use `100dvh` (or `svh`).
- `width: 100vw` causing a horizontal scrollbar on desktops, because `vw` includes the vertical scrollbar's width — use `width: 100%`.
- Expecting `padding-top: 50%` to be half the parent's height (it's half the width — historically used for aspect-ratio boxes, now replaced by `aspect-ratio`).
- Setting `html { font-size: 62.5% }` tricks and then forgetting to reset body text — harmless but confusing; plain `rem` math (1rem = 16px) is fine.

In the interview

> **30-second answer:** "`px` is fixed; `em` is relative to the element's font-size (the parent's when used in font-size, so it compounds); `rem` is relative to the root font-size, so it respects user settings without compounding; `%` depends on the property — widths and even vertical padding are relative to the containing block's width; `vw` / `vh` are viewport percentages, with `dvh` / `svh` fixing mobile toolbars; `ch` is handy for line length."

Self-check

1. Parent 20px, child `font-size: 1.5em; padding: 1em` — child padding? → Child font-size 30px, so padding 30px.
2. `margin-top: 10%` inside a 600×300 box? → 60px (10% of the width).
3. A readable line length? → Around 45–75 characters, e.g. `max-width: 65ch`.

TL;DR

`rem` for type/spacing, `em` for component-relative spacing, `%` for fluid widths (vertical padding uses width!), `dvh` for mobile full height, `ch` for measure, `px` for hairlines.

6.8 Colours

What it is

A **colour value** in CSS can be written in several notations; most of them describe the same **sRGB** colours (the standard colour range of ordinary screens), and newer ones (`oklch()`, `oklab()`, `color()`) reach wider-gamut colours (a *gamut* is the set of colours a screen can show; wide-gamut screens show more vivid ones) on modern displays.

Notation	Example	Notes
Named	`rebeccapurple`, `tomato`	~150 names; plus `transparent` and `currentColor`
Hex	`#663399`, short `#639`, with alpha `#66339980`, `#6398`	two hex (base-16) digits per channel (red, green, blue: 00–ff); 4th pair = alpha (opacity)
`rgb()`	`rgb(102 51 153)`, `rgb(102 51 153 / 50%)`	modern space-separated syntax with `/ alpha`; legacy `rgba(102, 51, 153, 0.5)` still works (`rgba` is now an alias of `rgb`)
`hsl()`	`hsl(270 50% 40%)`	hue angle (position on the colour wheel: 0 red, 120 green, 240 blue), saturation (how vivid vs grey), lightness — easy to tweak by hand
`oklch()`	`oklch(44% 0.16 303)`	**L**ightness (0–100%), **C**hroma (colourfulness, ~0–0.4), **H**ue angle. Perceptually uniform: equal L *looks* equally light across hues. Baseline since 2023.
`color-mix()`	`color-mix(in oklch, #663399 50%, white)`	mixes two colours in a chosen colour space (the mathematical model used to describe colours, which changes how the blend looks). Baseline since 2023.
`currentColor`	`border: 2px solid currentColor`	the element's current `color` value — keeps borders, SVG icons and text in sync

```
<!-- code/07_colors.html (excerpt) -->
<div class="sw" style="background: rebeccapurple">rebeccapurple</div>
<div class="sw" style="background: #663399">#663399</div>
<div class="sw" style="background: rgb(102 51 153)">rgb(102 51 153)</div>
<div class="sw" style="background: hsl(270 50% 40%)">hsl(270 50% 40%)</div>
<div class="sw" style="background: oklch(44.03% 0.1603 303.37)">oklch(…)</div>
…
<div class="sw" style="background: hsl(60 100% 50%)">hsl yellow</div>   <!-- … and 4 more hues -->
<div class="sw" style="background: oklch(70% 0.15 100)">oklch yellow</div>
…
<div class="sw" style="background: color-mix(in srgb, red, blue)">srgb red+blue</div>
<div class="sw" style="background: color-mix(in oklch, red, blue)">oklch red+blue</div>
<button class="btn">currentColor border</button>
```

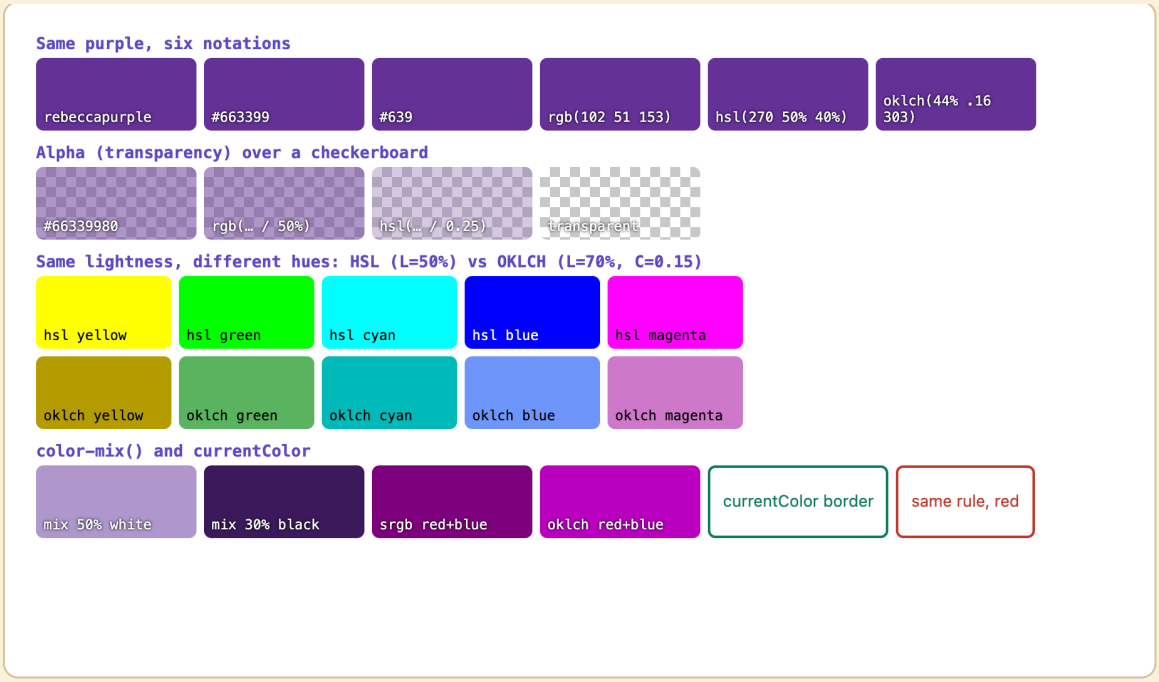


Figure 6.7 — The HSL row claims one lightness (50%) but yellow looks far brighter than blue; the OKLCH row really is evenly light. Mixing red and blue in sRGB gives a darker purple than mixing in OKLCH.

Why OKLCH matters

Design systems (a team's shared set of colours, components and rules) generate palettes ("brand-100 … brand-900") by changing lightness. In HSL, the same lightness step gives visibly uneven results across hues, so text contrast becomes unpredictable. In OKLCH, lightness is perceptual, so a palette built by varying L behaves consistently — and adjusting one number to lighten or darken a colour is easy to reason about. Contrast requirements (WCAG, the Web Content Accessibility Guidelines) are covered in Chapter 11.

Common mistakes

- Mixing old and new syntax: `rgb(102, 51, 153 / 50%)` is invalid (commas *or* spaces with `/`, not both).
- Using opacity (`opacity: 0.5`) when you meant a translucent (partly see-through) background — `opacity` fades the element's text and children too; use an alpha colour instead.
- Hard-coding the same colour in dozens of places instead of a custom property (6.9).

Self-check

1. Hex for 50%-transparent black? → `#00000080` (0x80 = 128 ≈ 50%).
2. What does `currentColor` resolve to? → The element's computed `color`.
3. Why is `oklch` better than `hsl` for palettes? → Its lightness is perceptually uniform across hues.

TL;DR

Hex/`rgb()`/`hsl()` for everyday sRGB; `oklch()` for predictable palettes; `/ alpha` for transparency; `currentColor` and `color-mix()` to derive colours.

6.9 Custom properties (CSS variables)

What it is

A **custom property** is a property you name yourself, starting with two dashes (`--brand: #5b4bc4;`), whose value you read with `var(--brand)`. Unlike preprocessor variables (a preprocessor such as Sass is a tool that compiles an extended CSS-like language into plain CSS before publishing; Sass writes variables as `$brand`), custom properties are **live**: they follow the cascade and inheritance, can be changed per element, per media query or by JavaScript at runtime (while the page is running in the browser), and every use updates instantly.

How it works

1. **Declare** on a selector — commonly `:root` (the `<html>` element) so the value is inherited everywhere.

2. **Use** with `var(--name)` or `var(--name, fallback)`; the fallback applies if the property isn't defined.
3. Custom properties **inherit** by default, so redefining one on an element changes it for that element and all its descendants — the basis of theming (switching a whole set of colours, e.g. light/dark) and component variants.
4. They're case-sensitive (`--Brand` ≠ `--brand`) and can hold any token sequence (any run of CSS words, numbers and symbols); the value is only checked when substituted (pasted into the property that uses it).
5. If the substituted value is invalid for the property (e.g. `color: var(--space)` where `--space: 12px`), the declaration becomes **invalid at computed-value time**: the property behaves as `unset` — it does **not** fall back to an earlier `color` declaration.
6. JavaScript reads them with `getComputedStyle(el).getPropertyValue('--x')` and writes them with `el.style.setProperty('--x', value)`.
7. Combine with `calc()`: `padding: calc(var(--space) * 2)`.

```css
/* code/08_custom_properties.html (excerpt) */
:root { --brand: #5b4bc4; --radius: 8px; --space: 12px; --surface: #fff; --text: #222; }
[data-theme="dark"] { --surface: #1e1b3a; --text: #eceaff; --brand: #a99cff; }

.panel { background: var(--surface); color: var(--text); padding: var(--space); }
.card  { border: 2px solid var(--brand); border-radius: var(--radius); padding: var(--space); }
.card h3 { color: var(--brand); }
.card.danger   { --brand: #c0392b; }                       /* local override */
.card.fallback { border-color: var(--not-defined, orange); }
.card.big      { padding: calc(var(--space) * 2); }
.card.bad      { color: var(--space); }                    /* invalid -> inherits */
```

```js
card.style.setProperty('--brand', '#0a7d55');                    // write
getComputedStyle(document.documentElement).getPropertyValue('--radius'); // read -> "8px"
```

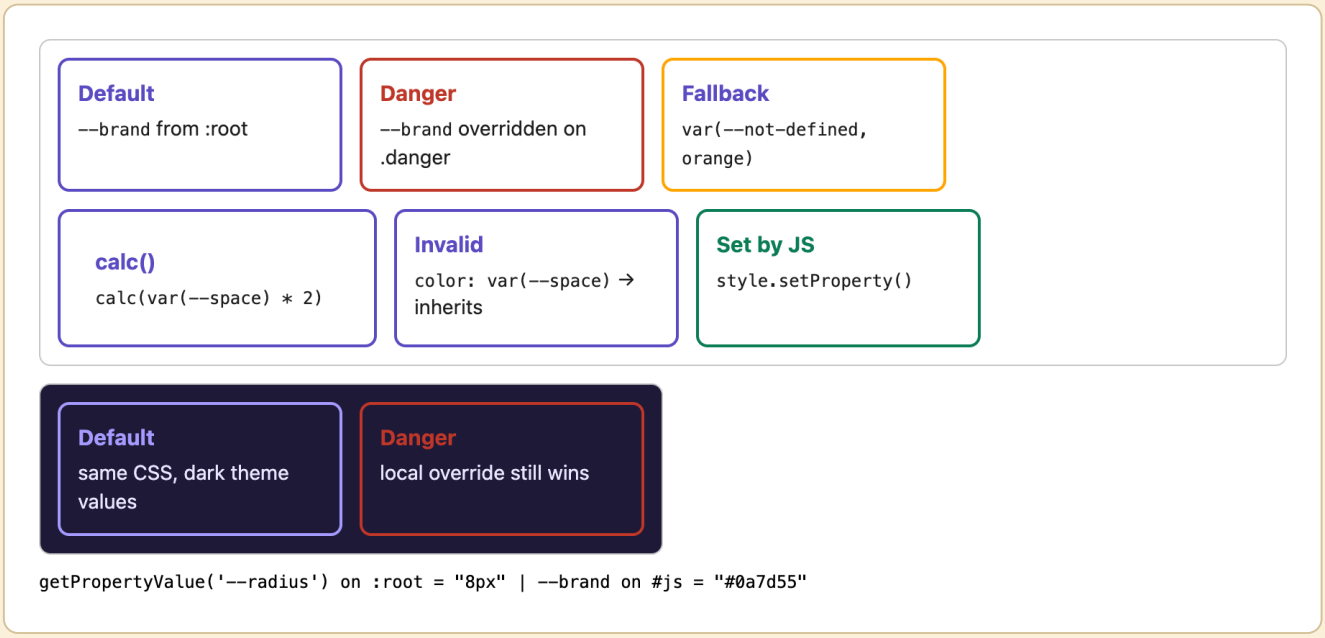


Figure 6.8 — One set of rules, many looks: the dark panel only redefines the variables, and the danger card's local `--brand` wins in both themes.

Trade-offs

	Custom properties	Preprocessor variables (Sass)
When resolved	at runtime in the browser	at build time (when the preprocessor runs, before publishing) — gone from the output CSS
Cascade/inheritance, per-element values	yes	no
Change from JS / media queries	yes	no
Use in selectors or media query conditions	no	yes
Type checking (verifying a value is the right kind, e.g. a colour)	none by default; `@property` can register a type and initial value (Baseline 2024), which also makes them animatable	compile errors (errors reported by the build tool)

Common mistakes

- Forgetting the `--` or the `var()`: `color: --brand` does nothing.
- Expecting a fallback to trigger when the variable *is* defined but has an unsuitable value (fallbacks only cover *undefined*).
- Defining theme variables on a component instead of `:root` and wondering why other components don't see them (inheritance flows down only).
- Using a variable inside a media query condition (`@media (min-width: var(--bp))`) — not allowed.

In the interview

30-second answer: "Custom properties are author-defined properties like `--brand` used with `var(--brand, fallback)`. They cascade and inherit, so I define tokens (named design values such as brand colour or spacing) on `:root`, override them on a theme attribute or component class, and change them from JS with `setProperty`. Unlike Sass variables they exist at runtime. An invalid substitution makes the declaration behave like `unset`."

- *Q: How do you implement dark mode with them?* A: Define colour tokens on `:root`, redefine them under `@media (prefers-color-scheme: dark)` (a media query that matches when the user's system is set to dark mode) or `[data-theme="dark"]`, and use only the tokens in components (see Chapter 13).

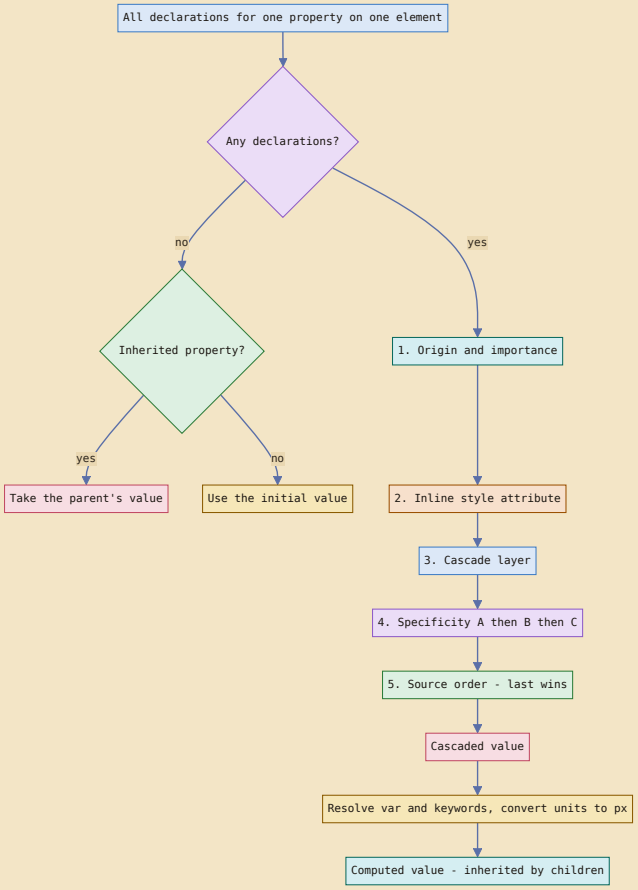
Self-check

1. `var(--gap, 1rem)` when `--gap` is defined as `2rem`? → `2rem`.
2. Where do you declare global tokens? → `:root`.
3. Can a custom property be used in a selector? → No.

TL;DR

`--name: value` + `var(--name, fallback)`; they cascade and inherit, so tokens on `:root` + overrides = themes and variants; editable from JS.

6.10 How it all fits together



Chapter interview round-up

1. **Explain the cascade.** Origin & importance → inline → layers → specificity → order; `!important` reverses origins and layers.
2. **Calculate specificity of `.nav > li:first-child a:hover`.** (0,3,2).
3. **`:is()` vs `:where()`?** Same matching; `:is` takes its most specific argument's specificity, `:where` is zero.
4. **`em` vs `rem`?** `em` follows the element's (or, for font-size, the parent's) font size and compounds; `rem` follows the root.
5. **Which properties inherit?** Mostly text properties (`color`, `font`, `line-height`); box properties don't.
6. **`initial` vs `unset` vs `revert`?** Spec default / inherit-or-initial / browser default (previous origin).
7. **Why custom properties over Sass variables?** Runtime, cascading, per-element, JS-accessible — enables theming.
8. **Pseudo-class vs pseudo-element?** State/position (`:hover`) vs part/generated content (`::before`).

Chapter TL;DR

- Rule = selector + declarations; external stylesheets; inline only when necessary.
- Selectors: type, class, ID, `*`, 7 attribute forms (with `i` flag), combinators (space, `>`, `+`, `~`), lists.
- Pseudo-classes select states/positions (incl. `:is`, `:where`, `:not`, `:has`); pseudo-elements select parts (`::before` needs `content`).
- Specificity = (IDs, classes/attrs/pseudo-classes, types/pseudo-elements), compared without carrying; `:where` = 0, `:is/:not/:has` = max argument.
- Cascade order: origin & importance → inline → layer → specificity → source order.
- Text properties inherit; `inherit`, `initial`, `unset`, `revert` override the default behaviour.

- Units: `rem` for type, `em` for local scaling, `%` of the containing block (vertical padding uses width), `dvh` for mobile, `ch` for measure.
- Colours: hex/`rgb`/`hsl`, `oklch` for uniform palettes, `/ alpha`, `currentColor`, `color-mix()`.
- Custom properties cascade and inherit — the foundation of themes.

Next: Chapter 7 — The Box Model and Layout Basics.

Chapter 7 — The Box Model and Layout Basics

Chapter 6 taught you how the browser decides *which* declarations apply to an element. This chapter is about *where the element ends up*. Everything on a web page is a rectangle — a **box** — and layout is the job of sizing those boxes and placing them. Before Flexbox (Chapter 8, for laying items out in a row or column) and Grid (Chapter 9, for rows *and* columns at once) make sense, you need the rules they build on: how a box's size is calculated, how boxes flow down the page by default, how you pull a box out of that flow with `position`, what happens when content does not fit, and who is painted on top of whom.

What you'll learn

#	Topic	You will be able to…
7.1	The box model	name the four areas of a box and predict its rendered size
7.2	`box-sizing`	explain why almost every project starts with `border-box`
7.3	Margin collapsing	predict the gap between two blocks and stop margins leaking out of a parent
7.4	`display`	choose between block, inline, inline-block, none, flow-root and contents
7.5	Normal flow and formatting contexts	describe how boxes are laid out when you do nothing, and what a BFC (block formatting context) is
7.6	Positioning	use relative, absolute, fixed and sticky and know each one's reference box
7.7	Overflow	pick between visible, hidden, clip, scroll and auto; truncate text with an ellipsis (the "…" that shows text was cut off)
7.8	`z-index` and stacking contexts	explain why `z-index: 9999` (the property that decides which overlapping box is in front) sometimes "doesn't work"
7.9	Floats (legacy — old, kept for compatibility)	wrap text around an image, contain floats, and read old float layouts

Every example is a small page in `code/` with a screenshot in `img/`. The outlines and labels you see are drawn by the example CSS so the boxes are visible. The `.mjs` scripts open the same pages in real headless Chrome (Chrome without a window, driven by a script via `puppeteer-core`, a Node.js library for controlling Chrome; the same pattern as Chapter 1's network script) and print the sizes the browser actually computed with `getBoundingClientRect()` (a DOM method that returns an element's on-screen position and size) — so every number in this chapter is measured, not guessed. Run one with `node code/02_margin_collapse.mjs`.

> **Tip — outlines are your layout debugger.** An `outline` is drawn *around* a box but takes up no space, so adding `outline: 1px solid red` never changes the layout you are inspecting (a `border` would). Many developers keep a snippet (a short reusable piece of code) like `* { outline: 1px solid rgb(255 0 0 / .3) }` handy for exactly this.

7.1 The box model

What it is

The **CSS box model** says every element generates a rectangular box made of four nested areas, from the inside out:

Area	What it is	Set with	Takes the element's background?
Content	Where text, images and child boxes go	`width`, `height` (and `min-`/`max-` versions)	Yes
Padding	Breathing room between the content and the border	`padding`, `padding-top` …	Yes
Border	A line (or image) around the padding	`border`, `border-width`, `border-style`, `border-color`	Drawn on top of the background
Margin	Empty space *outside* the border that pushes other boxes away	`margin`, `margin-left` …	No — always transparent

The outer edge of each area has a name used all over the specs: **content box**, **padding box**, **border box**, **margin box**.

Analogy: a framed photo on a wall. The photo is the content, the white mat around it is padding, the frame is the border, and the gap you leave to the next frame is the margin.

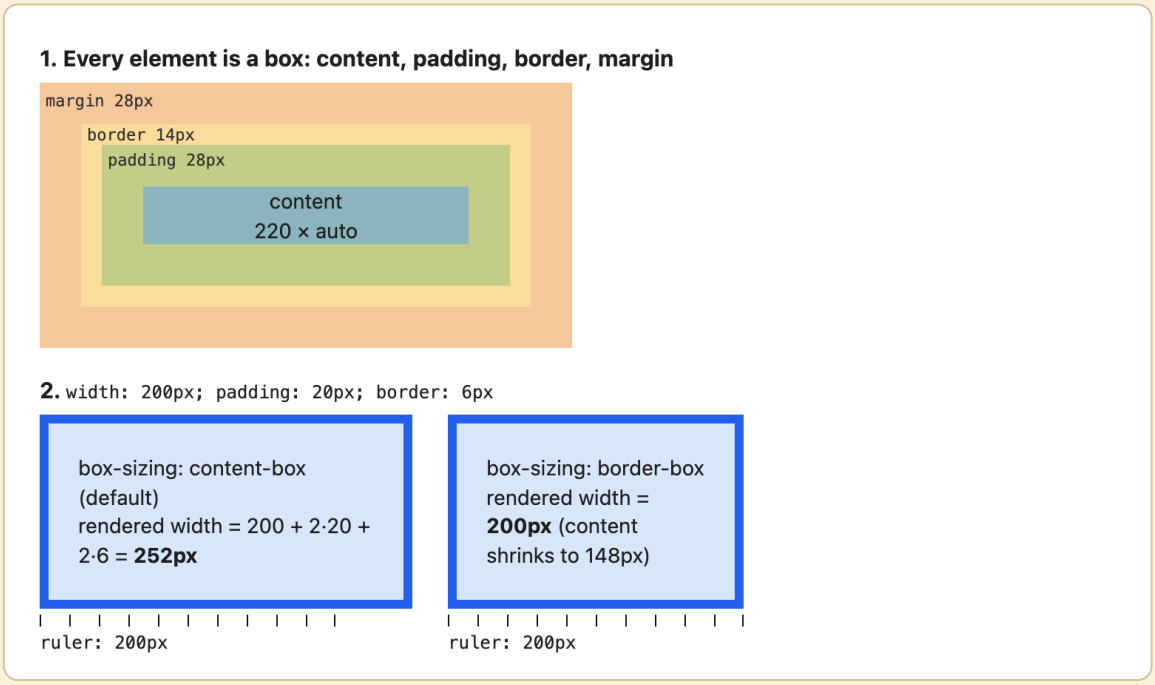


Figure 7.1 — `code/01_box_model.html`. The top diagram uses the same colours as the Chrome/Edge DevTools (the browser's built-in developer tools, F12) box-model panel (Elements → Computed). The blue content and green padding come from two background layers clipped to `content-box` and `padding-box`.

How the size is calculated

With the default `box-sizing: content-box`, `width` and `height` set the **content box only**. The space the element occupies is:

```
rendered width  = width  + padding-left + padding-right + border-left + border-right
space taken     = rendered width + margin-left + margin-right
```

So `width: 200px; padding: 20px; border: 6px solid` gives 200 + 2×20 + 2×6 = **252px** on screen. That surprise is exactly what `box-sizing` (7.2) fixes.

Shorthands — the clock rule

`margin`, `padding`, `border-width` and friends take 1–4 values that go **clockwise from the top**:

Written	Top	Right	Bottom	Left
`margin: 10px`	10	10	10	10
`margin: 10px 20px`	10	20	10	20
`margin: 10px 20px 30px`	10	20	30	20
`margin: 10px 20px 30px 40px`	10	20	30	40

`border: 2px solid #333` sets width, style and colour of all four sides; the style is required — `border: 2px #333` draws nothing because the default style is `none`. Modern code often uses **logical properties** such as `margin-block` (top + bottom in horizontal text) and `padding-inline` (left + right), which follow the writing direction (left-to-right, right-to-left or vertical, depending on the language); they are covered in Chapter 13.

Sizing rules worth knowing

- **`width: auto`** (the default) on a block box means "fill the containing block's width, minus my margins, borders and padding". The **containing block** is the rectangle a box is sized and positioned against — for ordinary boxes, the parent's content box (§7.6 gives the full rules). It is *not* the same as `width: 100%`, which ignores margins and can overflow.
- **`height: auto`** means "as tall as my content". A percentage height (`height: 50%`) only works if the parent has a definite height; otherwise it behaves like `auto`.
- **Percentages for padding and margin — even top and bottom — refer to the containing block's *width*.** That is why the old `padding-top: 56.25%` trick made 16:9 boxes (today, use `aspect-ratio: 16 / 9`).
- **`min-width` / `max-width`** clamp the box: `max-width: 65ch` keeps text readable; `min-height: 100vh` makes a full-screen section that can still grow. If they conflict, `min-*` wins.
- **Intrinsic sizes**: `width: min-content` (as narrow as the longest word allows), `max-content` (as wide as the content wants with no wrapping), and `fit-content` (max-content, but never wider than the available space). You will meet these again as flex and grid track sizes (a *track* is one row or column of a grid).
- **Margins can be negative** (pull a box or its neighbours closer); **padding cannot**.
- **`margin: auto`** on a block with a set width splits the leftover horizontal space equally — the classic `margin: 0 auto` horizontal centring.

Border vs outline vs box-shadow

	Takes space?	Per side?	Follows `border-radius`?	Typical use
`border`	Yes	Yes	Yes	Visible frames, dividers
`outline`	No (drawn outside the border box)	No	Yes in current browsers	Focus rings (the outline showing which element has keyboard focus, Chapter 3), debugging
`box-shadow`	No	No (but can be offset)	Yes	Elevation (making a card look raised off the page), fake borders that don't shift layout

Common mistakes

- Setting `width: 100%` *and* padding on a content-box element → it overflows its parent by the padding + border.
- Expecting `height: 100%` to fill the screen when the parent's height is `auto`.
- Writing `border: 1px red` and wondering why nothing shows (no style).

Self-check

1. A content-box element has `width: 300px; padding: 10px 20px; border: 2px solid`. How wide is it on screen? → 300 + 40 + 4 = **344px**.
2. `padding-top: 10%` inside a 600px-wide, 200px-tall parent — how tall is that padding? → **60px** (percentages use the width).
3. Which of border/outline changes the layout? → Only `border`.

7.2 `box-sizing`

What it is

`box-sizing` decides *which box* `width` and `height` describe:

Value	`width` means…	Padding and border…
`content-box` (default)	the content box	are **added** on top
`border-box`	the border box	are **taken out of** the width; the content shrinks

The problem it solves

Designers think in outer sizes: "the sidebar is 300px wide". With content-box, adding 20px padding secretly makes it 340px and a carefully measured two-column layout breaks. With border-box, 300px stays 300px whatever padding you add later. The measured result for the two boxes in Figure 7.1:

```
// code/01_box_sizing.mjs (key part) — both boxes: width: 200px; padding: 20px; border: 6px
const rows = await withPage('01_box_model.html', page => page.$$eval('.demo', els => els.map(el => {
  const cs = getComputedStyle(el);
  const content = el.clientWidth - 2 * parseFloat(cs.paddingLeft);   // clientWidth = content + padding
  return `${cs.boxSizing}  width property ${cs.width}  content ${content}px  ` +
         `on screen (border box) ${el.getBoundingClientRect().width}px`;
})));
```

Output:

```
content-box  width property 200px  content 200px  on screen (border box) 252px
border-box   width property 200px  content 148px  on screen (border box) 200px
```

`withPage` is a ten-line helper in `code/measure.mjs`: launch headless Chrome, open the file, run a function in the page, close. The three DOM measurements you will use most:

API	Returns
`el.getBoundingClientRect()`	The border box's position and size on screen (includes transforms — CSS moves, rotations and scaling — and fractional pixels such as 12.5)
`el.offsetWidth` / `offsetHeight`	Border box size, rounded to an integer, ignores transforms
`el.clientWidth` / `clientHeight`	Padding box size minus any scrollbar (the bar you drag to scroll)

The universal reset

Almost every modern stylesheet (and every CSS reset — a starter stylesheet that removes or evens out browser defaults, Chapter 13) starts with:

```
*, *::before, *::after { box-sizing: border-box; }
```

`box-sizing` is not inherited, so the rule targets every element and pseudo-element directly. A variant sets `html { box-sizing: border-box }` and `*, *::before, *::after { box-sizing: inherit }` so a third-party widget (a ready-made component from another vendor) can opt back into content-box for its whole subtree (the element and everything inside it).

Browsers' default styles already give many form controls border-box sizing (in Chrome, `<input>`, `<select>`, `<button>` and `<textarea>` all compute to `border-box`), and browsers differ on some of them — one more reason to set it for everything yourself, so a `div` and an `input` with the same `width` line up.

When not to use it

There is essentially no downside to border-box. The one thing to remember: if padding + border are larger than the width (say `width: 20px; padding: 20px`), the content box cannot go below zero, so the box grows beyond `width` anyway.

In the interview

> **30-second answer:** "Every element is a box of content, padding, border and margin. By default `width` sets only the content, so padding and border are added on top — 200 wide plus 20 padding plus 6 border is 252. With `box-sizing: border-box`, `width` includes padding and border, so the box stays 200 and the content shrinks. Most projects apply border-box to everything in the reset because it matches how designs are measured. Margin is never included in either model."

- *Is margin part of the element's width under border-box?* → No. Margin is always outside.
- *Is `box-sizing` inherited?* → No, hence the `*` selector (or the `inherit` pattern).

Self-check

1. Border-box, `width: 250px; padding: 25px; border: 5px`. Content width? → 250 − 50 − 10 = **190px**.
2. Why include `*::before, *::after` in the reset? → Pseudo-elements (CSS-generated boxes, Chapter 6) are boxes too and `*` alone does not match them.

7.3 Margin collapsing

What it is

In normal block layout, when two **vertical** margins touch with nothing between them, they **collapse** into a single margin whose size is the *larger* of the two instead of their sum. Horizontal margins never collapse.

Analogy: two people who each want "at least 30 cm of personal space" stand 30 cm apart, not 60.

Why it matters

It is the rule behind the most common "why is there a gap there?" and "why did my margin disappear?" questions. It was designed for text: a paragraph with `margin: 1em 0` followed by another should be 1em apart, not 2em.

How it works — the three situations

1. **Adjacent siblings** (elements with the same parent, one right after the other): the bottom margin of one block and the top margin of the next collapse.
2. **Parent and first/last child**: if a parent has no border, padding, inline content or clearance (extra space added by `clear` below a float, §7.9) separating it from its first child, the child's top margin collapses *through* the parent — it appears **outside** the parent. Same for the last child's bottom margin (when the parent's height is `auto`).
3. **Empty blocks**: a block with no content, padding, border or height has its own top and bottom margins collapse together.

The arithmetic

Margins meeting	Result
All positive (30, 20)	The largest: **30**
Positive and negative (30, −10)	Largest positive + most negative: 30 + (−10) = **20**
All negative (−10, −25)	The most negative: **−25**

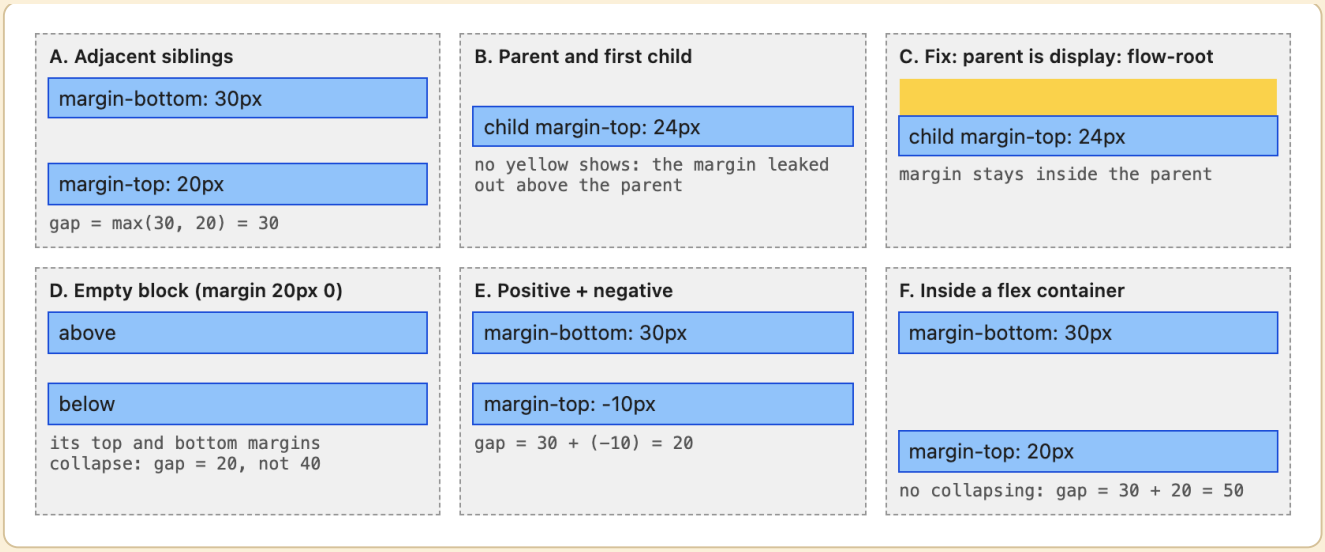


Figure 7.2 — `code/02_margin_collapse.html`. In case B no yellow is visible at all: the 24px margin sits above the parent, and the child fills the parent exactly.

The measured gaps:

```
// code/02_margin_collapse.mjs (key part)
const gap   = (a, b) => r(b).top - r(a).bottom;                 // space between two boxes
const inset = id => { const p = document.getElementById(id);    // child top minus parent top
  return p.firstElementChild.getBoundingClientRect().top - p.getBoundingClientRect().top; };
```

Output:

```
A siblings 30 / 20           -> 30px
B child top inside parent    -> 0px
C same with flow-root parent -> 24px
D across empty block 20/20   -> 20px
E 30 / -10                   -> 20px
F flex column 30 / 20        -> 50px
```

What stops margins collapsing

- Anything **between** them: padding or border on the parent (even `padding-top: 1px`), inline content, or a `clear`ed float.
- The parent starting a new **block formatting context** (7.5): `display: flow-root`, `overflow` other than `visible`/`clip`, `display: inline-block`, floats, absolute positioning.
- **Flex and grid layouts**: margins of flex and grid items (the direct children of a flex or grid container) never collapse (case F). This is one reason many teams lay out vertical stacks with `display: flex; flex-direction: column; gap: …` or `display: grid; gap: …` — `gap` is predictable and never collapses.
- Floated and absolutely positioned boxes never collapse with anything.

Common mistakes

- Giving a card's first heading `margin-top` and seeing the *whole card* move down (parent-child collapse). Fix: `display: flow-root` or padding on the card, or remove the child's top margin.
- Setting `margin-bottom: 20px` on every list item *and* `margin-top: 20px` on the next and expecting 40px.
- Trying to "fix" collapsing with `overflow: hidden` — it works (new BFC) but also clips shadows and focus rings. Prefer `flow-root`.

Real-world examples

Many design systems (a team's shared library of components and style rules — and the "Every Layout" book's *Stack* idea) avoid the whole topic by setting margins in **one direction only** — for example `.stack > * + * { margin-block-start: 1rem }` — or by using `gap`.

In the interview

30-second answer: "Vertical margins between blocks in normal flow collapse to the larger one — 30 and 20 give 30, not 50. It happens between siblings, between a parent and its first or last child when nothing

> separates them, and on empty blocks. Negative margins are added to the largest positive one. It doesn't happen in flex or grid, for floats or absolutely positioned boxes, or when padding, a border or a new block formatting context like `display: flow-root` separates the margins."

- *Why did my `h2`'s margin push the whole section down?* → Parent-child collapse; add padding or `flow-root` to the section.
- *Do horizontal margins collapse?* → Never.

Self-check

1. Margins 40px and 25px between siblings → gap? → **40px**.
2. −20px and −5px? → **−20px**.
3. The same two items inside a `display: grid` container with no gap, margins 40 and 25? → **65px**, no collapsing in grid.
4. One-line fix to keep a child's margin inside its parent? → `display: flow-root` on the parent.

7.4 `display`

What it is

`display` answers two questions about a box:

- **Outer display type** — how the box itself takes part in its parent's layout: as a **block** (own line) or **inline** (flows within a line of text).
- **Inner display type** — how it lays out its *children*: normal **flow**, **flex**, **grid**, **table**…

Every familiar keyword is a combination. Since 2023 all major browsers (Baseline) also accept the explicit **two-value syntax**:

Keyword	Two-value form	Outer	Children laid out as
`block`	`block flow`	block	normal flow
`inline`	`inline flow`	inline	normal flow (inline)
`inline-block`	`inline flow-root`	inline	flow, in its own formatting context
`flow-root`	`block flow-root`	block	flow, in its own formatting context
`flex`	`block flex`	block	flex (Ch. 8)
`inline-flex`	`inline flex`	inline	flex
`grid` / `inline-grid`	`block grid` / `inline grid`	block / inline	grid (Ch. 9)
`list-item`	`block flow list-item`	block	flow, plus a `::marker`
`table`, `table-row`, `table-cell`	–	–	table layout (what `<table>` uses)

Two keywords generate **no box of their own**:

- `display: none` — the element and its whole subtree produce no boxes: no space, not painted, not in the accessibility tree (the version of the page screen readers read), not focusable. (Compare `visibility: hidden` and `opacity: 0`, which keep the space — measured in Chapter 1, §1.7.)
- `display: contents` — the element's own box disappears but its **children** are laid out as if they were children of the grandparent. Handy for making a wrapper `<div>` or `<ul>`'s items take part in a parent's flex or grid layout. Caution: browsers historically dropped the element's semantics (its meaning, such as "this is a list") from the accessibility tree (a `<ul>` with `display: contents` stopped being announced as a list); most engines have fixed the common cases, but test with a screen reader and avoid it on buttons and other interactive elements.

Block vs inline vs inline-block

Behaviour	block	inline	inline-block
Starts on a new line	Yes	No	No
Default width	Fills the container	Its content	Its content (shrink-to-fit: as narrow as its content allows)
`width` / `height`	Respected	**Ignored**	Respected
Horizontal padding/border/margin	Push neighbours	Push neighbours	Push neighbours
Vertical padding/border	Push neighbours	**Painted but don't move lines** (overlap)	Push neighbours
Vertical margin	Applies (may collapse)	**Ignored**	Applies (never collapses)
Default for	`div`, `p`, `h1`–`h6`, `ul`, `section`, `form`	`span`, `a`, `strong`, `em`, `code`	nothing by default; `button`, `input`, `img` behave similarly

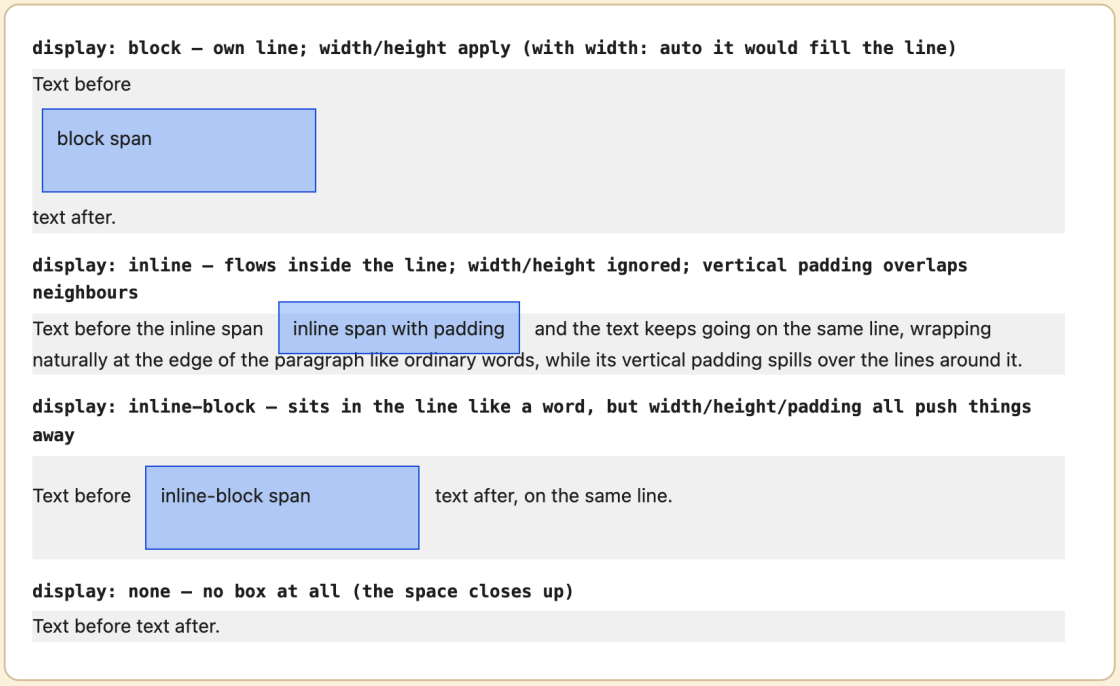


Figure 7.3 — `code/03_display.html`. Every blue span has exactly the same `width: 180px; height: 40px; padding: 10px; margin: 8px`; only `display` differs.

```
block         rendered 200 x 60  height of its <p>: 121
inline        rendered 176 x 37  height of its <p>: 45
inline-block  rendered 200 x 60  height of its <p>: 76
none          rendered   0 x  0  height of its <p>: 22
```

Output of `code/03_display.mjs`. The inline span ignores `width: 180px` (its 176px is just text + padding) and its paragraph is only two ordinary lines tall (45px) despite the 10px vertical padding. The inline-block keeps its 200×60 border box and the line grows to fit it.

Which elements are block or inline by default is only a **user-agent stylesheet default** (the browser's own built-in CSS) — HTML itself defines content models (rules for what may go inside what), not layout (see Chapter 2). A `<span>` with `display: block` is still phrasing content (text-level content) for validity purposes.

Replaced elements

`<img>`, `<video>`, `<iframe>`, `<input>` are **replaced elements**: their content comes from outside CSS, they have an **intrinsic size**, and they are inline-level by default yet accept `width` and `height`. The classic surprise: an `<img>` inside a `div` leaves a few pixels of gap underneath, because as an inline box it sits on the text **baseline** (the invisible line letters sit on) and leaves room for letter descenders (the tails of g, p, y that hang below it). Fix with `img { display: block }` (common in resets) or `vertical-align: middle`.

Common mistakes

- Setting `width` on an `<a>` or `<span>` and seeing no change → make it `inline-block` or `block`.
- Adding `margin-top` to an inline link to push it down → vertical margins on inline boxes are ignored.
- Using `inline-block` for rows of cards and fighting the whitespace gap between them (the HTML spaces between tags render as a space character). Use flex or grid with `gap`.
- Hiding content that should stay available to screen readers with `display: none`; use a *visually-hidden* class instead (Chapter 3).

Self-check

1. Which `display` value would you give an `<a>` that must be 44px tall but stay in a line of text? → `inline-block` (or `inline-flex`).
2. What does `display: inline flex` mean? → The box is inline on the outside, lays its children out with flexbox inside — same as `inline-flex`.
3. Why does an image in a div have a small gap below it? → It is inline and aligned to the text baseline; make it `display: block`.

7.5 Normal flow and formatting contexts

What it is

Normal flow is the layout you get when you write no layout CSS at all. Its rules are few:

1. **Block-level boxes** stack one after another in the **block direction** (top to bottom in English), each starting on a new line and, if `width: auto`, filling the container's width.
2. **Inline-level boxes** (text, `span`, `a`, images) sit side by side in the **inline direction** (left to right in English) and wrap into **line boxes** (one invisible box per line of text) when a line is full.
3. Vertical margins between blocks collapse (7.3).
4. A block's height grows to fit its content.

"Block direction" and "inline direction" are used instead of "vertical" and "horizontal" because they rotate with the writing mode (the CSS setting for text direction, horizontal or vertical): in vertical Japanese text, blocks stack right-to-left and lines run top to bottom. This is why modern properties are named `margin-block`, `inline-size` and so on.

Formatting contexts

A **formatting context** is a region in which one set of layout rules applies to the children. The ones you will meet:

Context	Created by	Children laid out by
Block formatting context (BFC)	root `<html>`, `display: flow-root`, `inline-block`, floats, `position: absolute/fixed`, `overflow` other than `visible`/`clip`, table cells, flex/grid items, `contain: layout/paint` (a performance hint isolating an element's layout/painting), multi-column containers (text flowing into newspaper-style columns)	block and inline flow
Inline formatting context (IFC)	a block that contains only inline-level content	line boxes
Flex formatting context	`display: flex / inline-flex`	flexbox (Ch. 8)
Grid formatting context	`display: grid / inline-grid`	grid (Ch. 9)

A new **BFC** is a sealed room. Inside it:

- child margins do not collapse through its edges (7.3),
- it **contains its floats** — its height includes them (7.9),
- it does not overlap floats outside it; it sits beside them instead.

`display: flow-root` exists *only* to create a BFC with no side effects, which is why it replaced the old `overflow: hidden` and "clearfix" hacks (workarounds, shown in §7.9). It is supported in every current browser.

Centring in normal flow

Goal	Normal-flow technique
Centre inline content (text, inline images) horizontally	`text-align: center` on the parent
Centre a block with a known or max width	`max-width: 40rem; margin-inline: auto`
Centre a block's children vertically	`align-content: center` on the block itself (works on plain blocks since 2024 — Baseline newly available)
Centre anything both ways	Flexbox or grid: `display: grid; place-items: center` (Chapter 9)

Self-check

1. In a vertical writing mode, which direction do block boxes stack? → Horizontally (the block axis is now horizontal).
2. Name two things a new BFC does. → Stops margin collapse through its edges; contains floats; doesn't overlap outside floats.
3. What is the side-effect-free way to create a BFC? → `display: flow-root`.

7.6 Positioning

What it is

The `position` property lets you move a box away from where normal flow would put it, using the **inset properties** `top`, `right`, `bottom`, `left` (or the shorthand `inset: top right bottom left`, same clock rule as `margin`).

Value	Stays in flow (keeps its space)?	Offsets measured from	Scrolls with the page?	Typical use
`static` (default)	Yes	— insets ignored	Yes	Everything
`relative`	**Yes** — the original space is kept	Its own normal position	Yes	Small nudges; anchor for absolute children
`absolute`	**No**	Nearest **positioned** ancestor's (an element above it in the tree whose `position` is not `static`) padding box (else the initial containing block)	Yes, with that ancestor	Badges, dropdowns, overlays (boxes laid on top of other content) inside a card
`fixed`	No	The **viewport** (the visible browser area)	**No** — stays put	Sticky headers of old, cookie bars, chat buttons
`sticky`	Yes	Its nearest **scrolling ancestor** (scrollport — the visible window of a scrollable box)	Scrolls until the threshold, then sticks	Table headers, section headings, sidebars

A **positioned element** is any element whose `position` is not `static`. The rectangle an element's offsets and percentages refer to is its **containing block**.

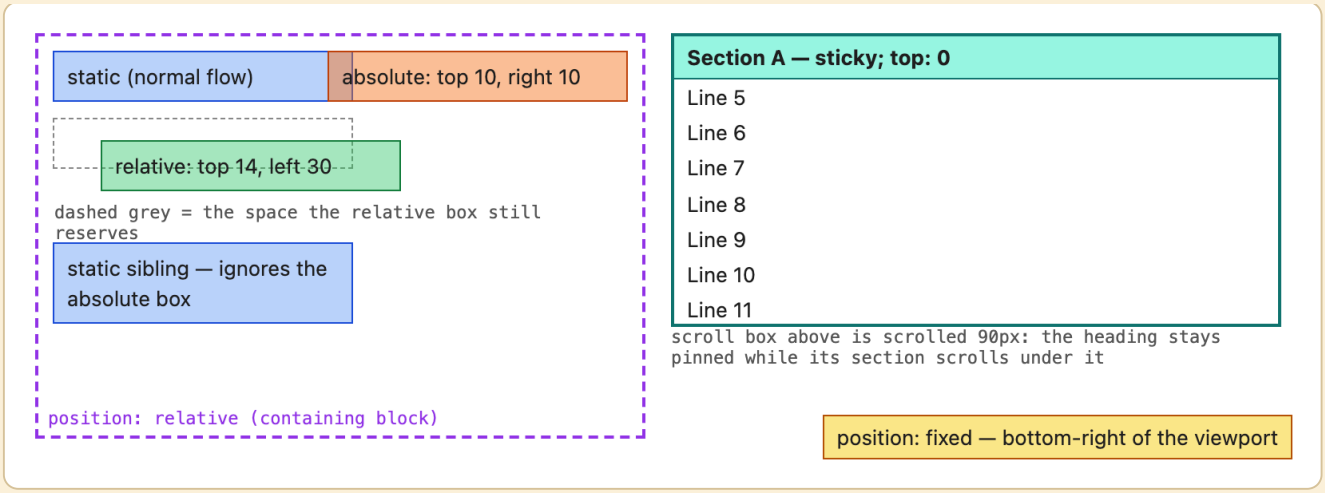


Figure 7.4 — `code/04_positioning.html`. The dashed grey rectangle is the space the relative box still reserves; the absolute box overlaps the static box because it no longer takes part in the flow.

The key CSS:

```
.container { position: relative; }                    /* becomes the containing block for .abs */
.rel  { position: relative; top: 14px; left: 30px; }    /* nudged; space kept */
.abs  { position: absolute; top: 10px; right: 10px; }   /* pinned to the container's corner */
.scroller { height: 180px; overflow: auto; }
.scroller h4 { position: sticky; top: 0; }              /* sticks to the scroll box's top */
.fixed { position: fixed; right: 12px; bottom: 12px; }  /* glued to the viewport */
```

Measured by `code/04_positioning.mjs` — including scrolling the page 500px to prove the fixed box does not move:

```
relative : moved 30px right, 14px down from its normal spot
absolute : 10px from container's right padding edge, 10px from its top
sticky   : heading top - scroller top = 0px after scrolling 90px (stays pinned)
fixed    : before scroll right/bottom gap = 12/12px, after scrolling 500px = 12/12px (glued to the viewport)
```

How each one works

`relative`

1. The box is laid out in normal flow.
2. It is then shifted visually by the insets. Neighbours do not move — they still see the original position.
3. Its most important job today is making itself the **containing block** for absolute children, usually with no offsets at all.

`absolute`

1. The box is removed from flow — siblings close up as if it weren't there.
2. The browser walks up the ancestors to the first one with `position` other than `static` (or with a `transform`, `filter`, `perspective`, `contain: layout/paint`, or `will-change` of those). Its **padding box** is the containing block. If there is none, the initial containing block (the size of the viewport, anchored at the top of the document) is used.
3. Insets position the box's margin edges relative to that rectangle. With `width: auto` the box **shrinks to fit** its content.
4. Setting both `left` and `right` (or `inset: 0`) stretches it; `inset: 0; margin: auto` plus a width and height centres it.

`fixed` Like absolute, but the containing block is the **viewport**, so it stays in place while the page scrolls. The trap: if any ancestor has a `transform`, `filter`, `perspective`, `backdrop-filter`, `contain: paint` or `will-change: transform`, that ancestor becomes the containing block instead, and your "fixed" modal scrolls away with it. That is one reason modals (dialogs that block the page behind them) are often rendered at the end of `<body>` (React "portals", which render a component somewhere else in the DOM) — or, better, with the native `<dialog>` element, which uses the **top layer** (a special layer the browser paints above the whole page) and escapes all of this.

`sticky`

1. The box behaves as `relative` (in flow) until its scroll container scrolls it to the threshold you gave, e.g. `top: 0`.

2. Then it sticks at that offset — but **never leaves its parent**: when the parent's bottom edge scrolls up, the sticky box is pushed away with it. That is how "each section heading sticks until the next section" works.
3. It needs at least one inset (`top`, `bottom`…). Without one it never sticks.
4. It sticks to the nearest ancestor that is a **scroll container** (a box that can scroll its content: `overflow: auto/scroll/hidden`). If some wrapper has `overflow: hidden` without being the thing that actually scrolls, the sticky element sticks to that wrapper and appears not to work. (`overflow: clip` does not create a scroll container, so it is safe.)

Trade-offs

Approach	Good for	Beware
Absolute positioning for layout	Overlays, badges, decorative bits	Content can overlap and text can be cut off when it grows or when users zoom; not for main page layout
Fixed headers/footers	Always-available controls	Covers content (use `scroll-padding-top` so anchor links — links to an `#id` on the same page — don't hide under a fixed header); eats space on small screens
Sticky	Headers that should stay visible while scrolling	Silent failures from ancestor `overflow` or a parent that is exactly as tall as the sticky element

Common mistakes

- `position: absolute` with no positioned ancestor — the box jumps to the page's corner. Add `position: relative` to the intended parent.
- Using `top: 50%; left: 50%` to centre and forgetting `translate: -50% -50%` (the offsets move the *top-left corner* to the centre). Today prefer `inset: 0; margin: auto` or grid.
- Sticky inside a parent that is only as tall as the sticky element → there is no room to stick.
- Expecting `z-index` to work on a static element (7.8).

In the interview

> **30-second answer:** "`static` is normal flow. `relative` keeps the element's space and shifts it visually; mostly it's used as the reference for absolute children. `absolute` removes it from flow and positions it against the nearest positioned ancestor's padding box. `fixed` positions against the viewport so it doesn't scroll — unless an ancestor has a transform or filter, which becomes the containing block. `sticky` is in flow until it hits a threshold in its scroll container, then it sticks, but only within its parent."

- *Why does my sticky header not stick?* → No `top`, an ancestor with `overflow` that isn't the scroller, or the parent is too short.
- *Why is my `position: fixed` modal scrolling with the page?* → An ancestor has `transform`/`filter`/`will-change`; move it out, or use `<dialog>`.
- *What's the containing block of an absolute element?* → The padding box of the nearest ancestor with non-static position (or a transform etc.); otherwise the initial containing block.

Self-check

1. Does a `position: relative; top: 20px` element move its next sibling down? → No; the sibling sees the original position.
2. An absolute child has `right: 0` in a `position: relative` parent with `padding: 20px; border: 5px`. Where is it? → Flush with the inner edge of the border (the padding box's right edge), not 20px in.
3. Which `overflow` value can you set on an ancestor without breaking `position: sticky`? → `visible` or `clip`.

7.7 Overflow

What it is

Overflow is content that does not fit its box — typically because you fixed a `height`, or a long word or wide image exceeds a `width`. The `overflow` property (or `overflow-x` / `overflow-y`) decides what happens:

Value	Overflowing content	Scrollbars	Scrollable by script?	Creates a BFC / scroll container?
`visible` (default)	Spills out and paints over what follows	No	No	No
`hidden`	Clipped (cut off, not drawn) at the padding box	No	**Yes** (`el.scrollTop = …`, focus, anchor links)	Yes / yes
`clip`	Clipped	No	**No**	No / no
`scroll`	Clipped	**Always**, even if not needed	Yes	Yes / yes
`auto`	Clipped	Only when needed	Yes	Yes / yes

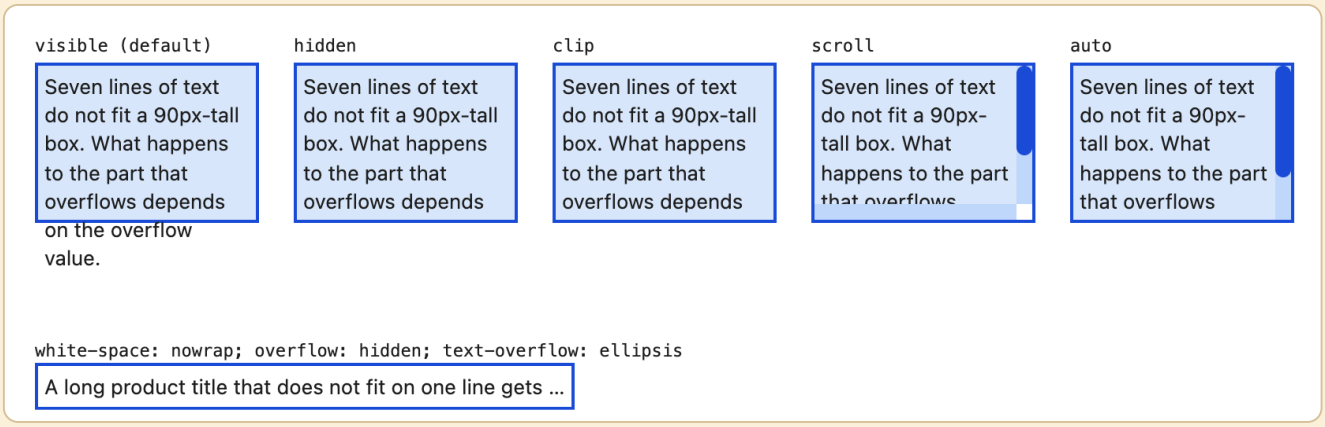


Figure 7.5 — `code/05_overflow.html`. The scrollbars are coloured with `::-webkit-scrollbar` so they show up in the screenshot; macOS normally uses overlay scrollbars (thin bars drawn over the content) that only appear while scrolling.

`code/05_overflow.mjs` sets `scrollTop = 20` on each box and reads it back:

```
visible  clientHeight 98  scrollHeight 131  scrollTop after setting 20 -> 0
hidden   clientHeight 98  scrollHeight 135  scrollTop after setting 20 -> 20
clip     clientHeight 98  scrollHeight 131  scrollTop after setting 20 -> 0
scroll   clientHeight 88  scrollHeight 135  scrollTop after setting 20 -> 20
auto     clientHeight 98  scrollHeight 135  scrollTop after setting 20 -> 20
```

Three things to read from this: `hidden` boxes *can* still be scrolled programmatically (by code) (a keyboard user tabbing to a link inside will scroll it too), `clip` cannot; `overflow: scroll` spent 10px of height on a horizontal scrollbar it didn't need (clientHeight 88); and the scroll containers count their bottom padding in the scrollable area (135 vs 131). `overflow: clip` reached Baseline in 2022.

Useful companions

- **Single-line truncation** — all three are required:

  ```
  .title { white-space: nowrap; overflow: hidden; text-overflow: ellipsis; }
  ```

- **Multi-line truncation**: `display: -webkit-box; -webkit-box-orient: vertical; -webkit-line-clamp: 3; overflow: hidden;` — prefixed (the `-webkit-` vendor prefix marks a property that started as one browser's own extension), but works in all current browsers.
- **Long words and URLs** breaking out of narrow columns: `overflow-wrap: anywhere` (or `break-word`).
- **Mixing axes**: if one axis is `visible` and the other isn't, `visible` computes to `auto` — you cannot clip only vertically while letting content spill horizontally (use `clip` for that).
- `scrollbar-gutter: stable` reserves space for a scrollbar so content doesn't jump when one appears.
- `overscroll-behavior: contain` stops scrolling inside a modal or chat panel from "chaining" to the page behind it (carrying on and scrolling the page once the panel reaches its end).

Common mistakes

- Using `overflow: hidden` to fix a float or margin problem and later finding focus outlines and box-shadows clipped.

- Putting `overflow: hidden` on `body`/`html` to lock scrolling and breaking `position: sticky` below it.
- Truncating text that the user needs to read with no way to see the rest (add a tooltip — a label that pops up on hover — or an expand control, and keep the full text in the DOM for screen readers).

Self-check

1. Which value clips without making the element scrollable? → `clip`.
2. You need an ellipsis on a single-line product title. Which three declarations? → `white-space: nowrap; overflow: hidden; text-overflow: ellipsis`.
3. Why can `overflow: scroll` look worse than `auto` on Windows? → It always shows scrollbars, even when nothing overflows.

7.8 `z-index` and stacking contexts

What it is

Boxes can overlap (negative margins, positioning, transforms, grid items in the same cell). The **stacking order** decides which one is drawn in front — the imaginary **z-axis** points out of the screen towards you. `z-index` sets an element's position on that axis, but only **within its stacking context**.

A **stacking context** is a group of elements that is painted as one unit. Analogy: each stacking context is a sheet of paper with drawings on it. You can reorder drawings *on* a sheet with `z-index`, and you can reorder whole sheets, but a drawing can never slip between two other sheets — no matter how big its number is.

How it works

1. `z-index` applies to **positioned** elements (`position` not `static`) and to **flex and grid items**. On a static block it does nothing.
2. Elements with a higher `z-index` in the **same** stacking context paint in front; equal values paint in DOM order (later in front).
3. An element with `z-index` set to a number (even `0`) on a positioned or flex/grid item **creates a new stacking context** for its descendants.
4. Its descendants' z-indexes are only compared with each other; to the outside world the whole group has its parent's z-index.

Inside one stacking context, the painting order is (back to front):

1. The context root's background and borders
2. Descendants with **negative** `z-index`
3. In-flow, non-positioned block boxes
4. Floats
5. Inline content (text, inline boxes)
6. Positioned descendants with `z-index: auto` or `0`, in DOM order
7. Positioned descendants with **positive** `z-index`, lowest first

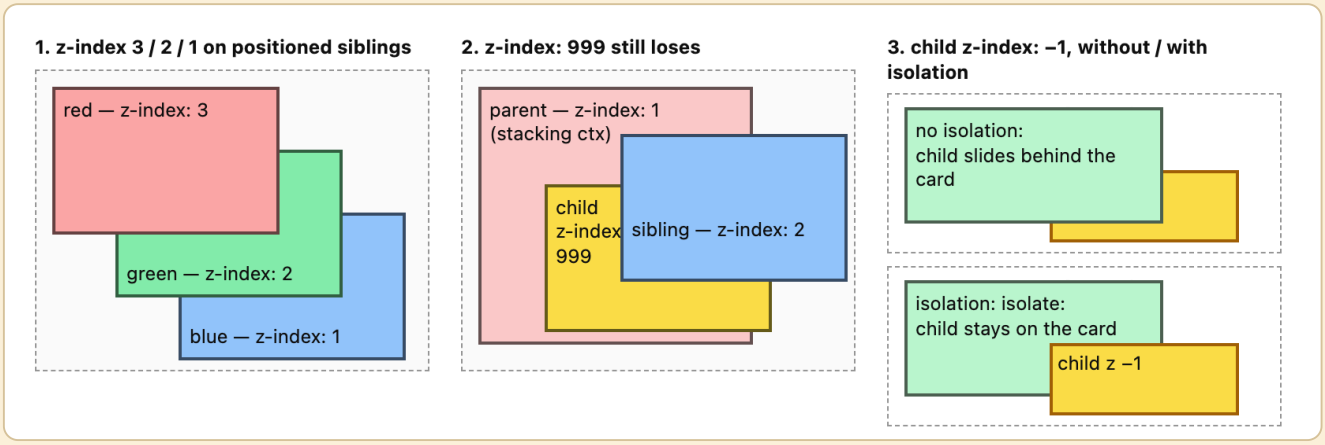


Figure 7.6 — `code/06_stacking.html`.

```
/* demo 2 — the classic trap */
.parent  { position: absolute; z-index: 1; }     /* creates a stacking context */
.child   { position: absolute; z-index: 999; }   /* 999 only counts INSIDE .parent */
.sibling { position: absolute; z-index: 2; }     /* 2 > 1, so it covers .parent and everything in it */

/* demo 3 — isolation: isolate makes a stacking context without z-index */
.card.iso { isolation: isolate; }                /* the child's z-index: -1 now stays above the card's background */
```

`code/06_stacking.mjs` asks the browser `document.elementFromPoint(x, y)` — "which element would receive a click here?" — at points where the boxes overlap:

```
demo 1  where red, green and blue overlap -> r1
demo 2  where child (999) and sibling (2) overlap -> b2
demo 3  inside the yellow child, no isolation -> card
demo 3  inside the yellow child, isolation: isolate -> u2
```

What creates a stacking context

- The root element (`<html>`)
- `position: relative`/`absolute` **with** a `z-index` other than `auto`
- `position: fixed` or `sticky` (always)
- A flex or grid item with `z-index` other than `auto`
- `opacity` less than 1
- `transform`, `scale`, `rotate`, `translate`, `filter`, `backdrop-filter`, `perspective`, `clip-path`, `mask` (any value other than `none`)
- `mix-blend-mode` other than `normal`
- `isolation: isolate`
- `will-change` naming any property above
- `contain: layout`, `paint`, `strict` or `content`; `container-type: size` or `inline-size`
- Elements in the **top layer** — an open modal `<dialog>`, a shown popover (a small floating panel created with the `popover` attribute), a fullscreen element — are painted above *everything*, whatever the z-indexes

Why it matters

"I set `z-index: 99999` and the dropdown is still behind the header" is almost always a stacking-context problem: an ancestor of the dropdown (perhaps with `transform` for an animation, or `opacity: .99`) created a context whose own level is below the header's.

How to debug and fix it

1. Walk up from the element that is "behind" and look for any ancestor in the list above (DevTools → Layers panel in Chrome/Edge, or the 3D view in Edge, visualises contexts).
2. Compare the z-indexes of the **two ancestors that are siblings in the same context**, not the elements themselves.
3. Fix by raising the ancestor's z-index, removing the property that created the unwanted context, or moving the overlay elsewhere in the DOM (or into the top layer with `<dialog>` / the `popover` attribute).
4. Prevent it: keep a small **z-index scale** as custom properties (CSS variables, Chapter 6) (`--z-dropdown: 100; --z-sticky: 200; --z-modal: 300;`) and use `isolation: isolate` on components so their internal z-indexes cannot leak out.

In the interview

30-second answer: "`z-index` only orders elements within the same stacking context, and only works on positioned or flex/grid items. Things like a z-index on a positioned element, opacity below 1, transforms, filters and `isolation: isolate` create a new context, and everything inside is flattened into it — so a child with `z-index: 999` can't escape a parent with `z-index: 1` that sits below a sibling with `z-index: 2`. To debug I find the ancestors that are siblings in a common context and compare those."

- *Does `z-index` work on a static element?* → No (except flex and grid items).
- *What's the difference between `z-index: auto` and `0`?* → Both paint at the same level, but `0` creates a stacking context and `auto` doesn't.

Self-check

1. Name three properties besides `z-index` that create a stacking context. → `opacity < 1`, `transform`, `filter` (also `isolation`, `will-change`, `mix-blend-mode`…).
2. Parent A `z-index: 5` contains child `z-index: -1`. Can the child appear behind A's background? → No; A is a stacking context, so its own background is always at the very back of that context.
3. Why does an open `<dialog>` appear above a `z-index: 10000` header? → Modal dialogs are in the top layer, above all stacking contexts.

7.9 Floats (legacy)

What it is

`float: left` or `float: right` takes a box out of normal flow, pushes it to that side of its container, and lets **inline content (text) wrap around it**. It was designed for magazine-style images inside articles. From roughly 2005 to 2015 floats were also abused to build whole page layouts (columns, grids), with a pile of hacks to go with it. Flexbox and grid replaced that; you still need floats to wrap text around a shape, and to read older code.

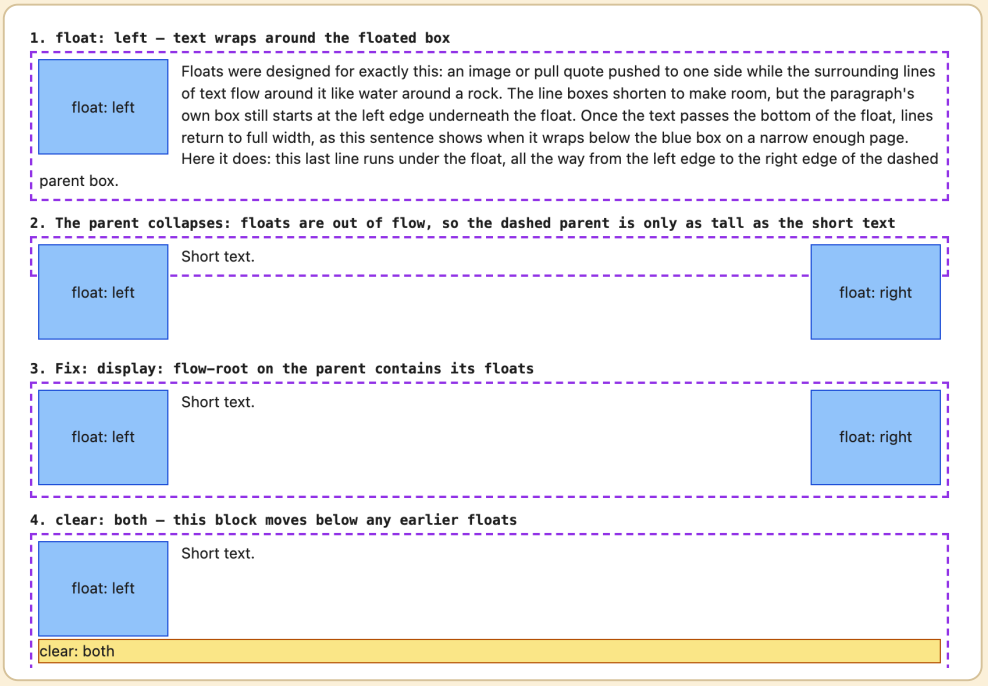


Figure 7.7 — `code/07_floats.html`.

How it works

1. The float is moved as far left (or right) as it can go within its containing block, at the vertical position where it would have been.
2. Following block boxes act as if the float isn't there — but their **line boxes shorten** to avoid it, so text flows around it.
3. The parent's height ignores floats — it **collapses** if everything inside is floated (demo 2).
4. **`clear: left | right | both`** on a later block forces it to start below the earlier floats (demo 4).
5. A parent that establishes a **BFC** — best done with **`display: flow-root`** — grows to contain its floats (demo 3).

Measured parent heights (each dashed parent has `padding: 6px` and a 2px border; the floats are 80px tall with a 4px bottom margin):

```
w1  display: block      parent height 129.0625px
w2  display: block      parent height 34.84375px
w3  display: flow-root  parent height 100px
w4  display: flow-root  parent height 118.84375px
```

Output of `code/07_floats.mjs`. `w2` is only as tall as one line of text plus its padding and border; `w3`, with identical content, contains the floats: 80 + 4 (float margin) + 2×6 (padding) + 2×2 (border) = **100px**.

The clearfix (read it, don't write it)

Before `flow-root`, the standard way to contain floats was this pseudo-element hack; you will still see it in legacy code and Bootstrap 3–era CSS (Bootstrap is a popular CSS framework; version 3 dates from 2013):

```
.clearfix::after { content: ""; display: table; clear: both; }
```

When to use floats today

Need	Use
Wrap paragraph text around an image or pull quote (a quotation repeated in large type)	`float` (optionally `shape-outside: circle(50%)` to wrap around a shape)
Columns, rows of cards, nav bars, sidebars	Flexbox or grid
Push one item to the right in a row	Flexbox `margin-left: auto` (Chapter 8)

Note that `float` is ignored on flex and grid items — another reason floats and modern layout don't mix.

Self-check

1. Why does a parent with only floated children have zero height? → Floats are out of flow, so they don't contribute to the parent's auto height.
2. Modern one-liner to contain floats? → `display: flow-root` on the parent.
3. What does `clear: both` do? → Moves the element below any preceding left or right floats.

Chapter 7 TL;DR

- **Box model**: content → padding → border → margin. Margin is transparent and outside; background covers content + padding (and under the border).
- `box-sizing: border-box` makes `width` include padding and border; put it on `*, *::before, *::after` in every project. Content-box: 200 + 2×20 + 2×6 = 252.
- **Percent padding/margin** (even vertical) use the containing block's **width**. `width: auto` ≠ `100%`.
- **Margin collapsing**: vertical margins in block flow merge to the largest (positive + most negative if mixed); happens between siblings, parent/first-last child, and empty blocks; never in flex/grid, for floats/absolute, or across padding/border/new BFC.
- `display`: outer (block/inline) + inner (flow/flow-root/flex/grid). Inline ignores width/height and vertical margins; inline-block keeps them. `none` removes the box; `contents` removes only the element's own box (check accessibility).
- **Normal flow**: blocks stack in the block direction, inlines wrap into line boxes. A **BFC** (`display: flow-root`) contains floats and stops margin collapse through its edges.
- **Position**: `relative` = shifted, space kept; `absolute` = out of flow, against nearest positioned ancestor's padding box; `fixed` = viewport (unless an ancestor has transform/filter); `sticky` = in flow until its threshold in the nearest scroll container, never leaves its parent.
- **Overflow**: visible / hidden (script-scrollable) / clip (not scrollable, no BFC) / scroll (always bars) / auto (bars when needed). Ellipsis needs `nowrap + hidden + text-overflow`.
- **Stacking**: `z-index` works on positioned and flex/grid items, only within its **stacking context**; contexts come from z-index on positioned elements, opacity < 1, transforms, filters, `isolation: isolate`, fixed/sticky and more. The top layer beats everything.
- **Floats**: for wrapping text around images; contain with `flow-root`, escape with `clear`; don't build layouts with them.

Next: Chapter 8 — Flexbox.

Chapter 8 — Flexbox

Normal flow (the default layout you get with no layout CSS, Chapter 7) stacks blocks and wraps text, and that's all it does. For years, putting three boxes side by side, making them equal height, or centring something vertically meant floats (boxes pushed to one side with text wrapping around them), tables or negative-margin hacks (workarounds that pull boxes into place with margins below zero). **Flexbox** (the CSS Flexible Box Layout module, a W3C Candidate Recommendation — a near-final stage of a web standard — supported in every browser for about a decade) solves the one-dimensional case (one direction at a time): lay out a set of items in a **row or a column**, share out the spare space, and align them. It is the tool for navbars (the row of navigation links at the top of a site), toolbars, button groups, cards, form rows and centring. For two-dimensional layouts (rows *and* columns at once) use Grid (Chapter 9).

What you'll learn

#	Topic	You will be able to…
8.1	Container and items	turn on flexbox and know which properties go on which element
8.2	The two axes, direction and order	reason in "main axis / cross axis" instead of left/right/top/bottom
8.3	`flex-grow`, `flex-shrink`, `flex-basis`	calculate item sizes by hand — verified against Chrome to the pixel
8.4	The `flex` shorthand and minimum sizes	choose between `flex: 1`, `auto`, `none`; fix the `min-width: auto` overflow
8.5	Alignment	use `justify-content`, `align-items`, `align-self`, `align-content` and auto margins
8.6	Wrapping and `gap`	build rows that wrap, and know when wrapping flex should be grid instead
8.7	Common patterns	build a navbar, centring, equal-height cards, sticky footer (a footer that stays at the bottom of short pages), media object (image beside text), input group (a text box joined to a button)
8.8	Pitfalls, debugging and interview prep	recognise the classic flexbox bugs and explain flexbox in an interview

Every example is a page in `code/` with its screenshot in `img/`. Flex containers are drawn with a **dashed purple border**, items with a **solid blue outline**, and item widths printed in **red** are measured by the page itself after layout. The sizing maths is implemented step by step in `code/flex_algorithm.mjs` and checked against real headless Chrome (Chrome driven by a script, with no window) by `code/03_flex_measure.mjs`.

8.1 Flex container and flex items

What it is

Writing `display: flex` on an element makes it a **flex container**. Its **direct children** become **flex items** and are laid out by the flexbox rules instead of normal flow. Grandchildren are not affected — each level decides for itself.

```
<nav class="toolbar">          <!-- flex container -->
  <a href="/">Home</a>         <!-- flex item -->
  <a href="/docs">Docs</a>     <!-- flex item -->
  <span>v2.1 <b>beta</b></span> <!-- flex item; the <b> inside is NOT an item -->
</nav>
```

```
.toolbar { display: flex; }          /* block-level container (takes a full line) */
.badge   { display: inline-flex; }   /* inline-level container (sits in text) */
```

Even a bare run of text directly inside the container becomes an **anonymous flex item** (an item the browser creates for you, with no element of its own, so you cannot target it with CSS).

Which properties go where

On the container	On the items
`display: flex | inline-flex`	`flex-grow`, `flex-shrink`, `flex-basis` (or shorthand `flex`)
`flex-direction`	`align-self`
`flex-wrap` (both via `flex-flow`)	`order`
`justify-content`	`margin` (including `auto`)
`align-items`	`min-width` / `min-height` (matters a lot, see 8.4)
`align-content`	
`gap`, `row-gap`, `column-gap`	

What changes for the children

- Items no longer care about `display: block`/`inline`: every item is "blockified" (a `<span>` item accepts width and height).
- `float` and `clear` are ignored on items; `vertical-align` has no effect.
- Margins of adjacent items **do not collapse** (Chapter 7, §7.3).
- An item with `z-index` creates a stacking context (a group of boxes layered together as one unit) even without `position` (§7.8).
- By default items sit in a row, are as wide as their content, may shrink but not grow, and **stretch** to the container's height.

Self-check

1. `display: flex` on `<ul>` — are the `<a>` elements inside the `<li>`s flex items? → No, only the `<li>`s (direct children).
2. Which property turns a flex container inline? → `display: inline-flex` (or `display: inline flex`).

8.2 The two axes, `flex-direction` and `order`

What it is

Flexbox never talks about "horizontal" and "vertical". It uses two **axes**:

- The **main axis** — the direction items are placed one after another. Its start and end are **main-start** and **main-end**.
- The **cross axis** — perpendicular (at a right angle) to it. **cross-start** / **cross-end**.

`flex-direction` sets the main axis:

Value	Main axis (English, left-to-right text)	Cross axis
`row` (default)	left → right (follows the text direction; right → left in Arabic or Hebrew)	top → bottom
`row-reverse`	right → left	top → bottom
`column`	top → bottom	left → right
`column-reverse`	bottom → top	left → right

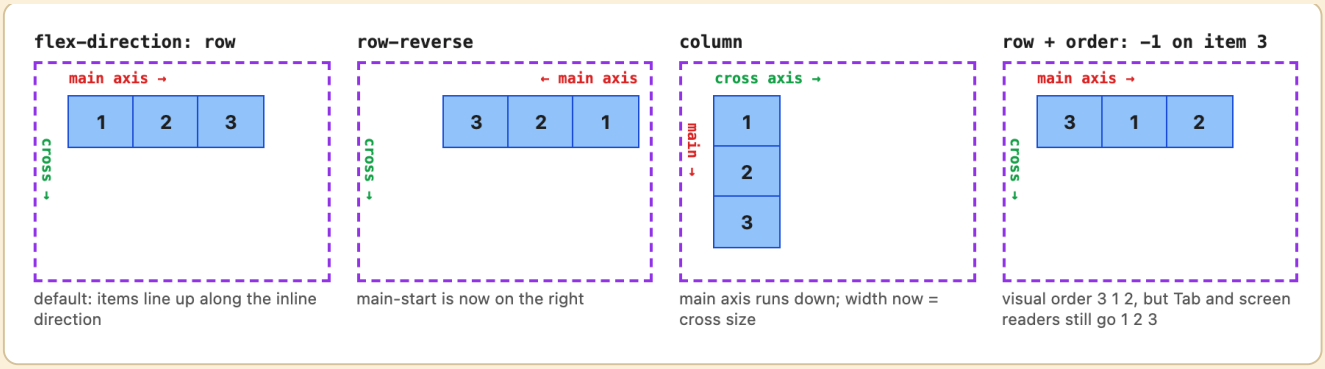


Figure 8.1 — `code/01_axes.html`. The red arrow is the main axis, the green one the cross axis; the labels are drawn with `::before` / `::after`.

Why it matters

Every alignment property is defined per axis: **`justify-content` works on the main axis; `align-items`, `align-self` and `align-content` work on the cross axis.** Switch `flex-direction` from `row` to `column` and `justify-content: center` now centres *vertically*. If you remember only one thing about flexbox, remember this.

`order`

`order: <integer>` (default `0`) changes the **visual** position of an item: items are placed by ascending `order`, ties in source order. In the fourth demo, `order: -1` moves item 3 to the front.

When not to use it (accessibility)

`order`, `row-reverse` and `column-reverse` change only what is *drawn*. The **Tab order (the order the Tab key moves through links and controls) and the screen-reader reading order still follow the DOM** (the element order in the HTML), so a keyboard user sees focus (the highlight on the element receiving keyboard input) jump around. WCAG 2.2 (the Web Content Accessibility Guidelines) success criteria 1.3.2 (Meaningful Sequence) and 2.4.3 (Focus Order) cover this (Chapter 3). Rules of thumb:

- Change the HTML order when the order means something; use `order` only for purely visual tweaks (e.g. an image before or after its text in alternating rows).
- The new CSS `reading-flow` property lets focus and reading order follow the visual flex/grid order, but as of 2026 it ships only in Chromium browsers (those built on Google's open-source engine: Chrome/Edge 137+) — not Baseline (the label for features that work in all major browsers), so don't rely on it.

Self-check

1. With `flex-direction: column`, which property centres items horizontally? → `align-items: center` (horizontal is now the cross axis).
2. Does `row-reverse` change the Tab order? → No.
3. Items have `order` values 2, −1, 0, 0. Which is drawn first? → The one with −1, then the two 0s in source order, then 2.

8.3 `flex-grow`, `flex-shrink` and `flex-basis`

This is the part that makes flexbox "flexible" — and the part most people guess at. Let's calculate it properly.

What it is

Each item has three sizing properties, always read **along the main axis** (width in a row, height in a column):

Property	Meaning	Default
`flex-basis`	The item's starting size before free space is shared: a length (`200px`, `30%`), `0`, `content`, or `auto` (= use `width`/`height` if set, else the content size)	`auto`
`flex-grow`	How many *shares* of **positive free space** the item takes	`0` (don't grow)
`flex-shrink`	How strongly the item gives up space when there is **negative free space** (overflow), weighted by its basis	`1` (may shrink)

Free space = container's inner main size − the sum of the items' flex base sizes (and margins, borders, padding) − gaps. Positive → the items can grow. Negative → they must shrink.

Analogy: a restaurant bill. `flex-basis` is what each person ordered. If the group has budget left over (positive free space), it is split by `flex-grow` "shares". If the bill is over budget (negative free space), each person pays back in proportion to `flex-shrink × what they ordered` — people who ordered more give back more.

How it works — the algorithm, simplified

The CSS spec calls this "resolving the flexible lengths" (Flexbox Level 1, §9.7). For one line of items:

1. **Base sizes.** Work out each item's flex base size from `flex-basis`, then clamp it (force it inside the allowed range) by its `min-*`/`max-*` to get its *hypothetical* size (the size it would have before any growing or shrinking).
2. **Grow or shrink?** Sum the hypothetical sizes, gaps, margins, padding and borders. If less than the container, use `flex-grow`; otherwise use `flex-shrink`.
3. **Freeze inflexible items** — those whose factor is 0 — at their hypothetical size ("frozen" means the size is final and no longer changes in later rounds).
4. **Distribute** the remaining free space among the unfrozen items:
   - growing: `size = basis + free × grow / Σgrow`
   - shrinking: `size = basis − overflow × (shrink × basis) / Σ(shrink × basis)`
   - special case: if the unfrozen factors add up to **less than 1**, only that fraction of the free space is handed out.
5. **Clamp** each result to its `min-*`/`max-*`. If anything was clamped, **freeze** those items at the clamped size and go back to step 4 with the space that is left.
6. Stop when every item is frozen.

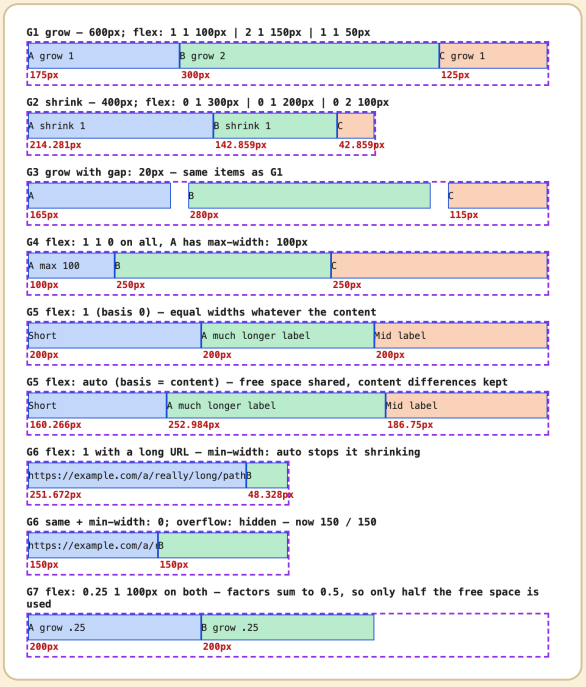


Figure 8.2 — `code/02_grow_shrink.html`. Items have no padding or border, so each item's width is exactly its flex size. A small script at the bottom of the page reads each width with `getBoundingClientRect()` and writes it underneath — the labels are absolutely positioned children of the container, so they take no part in the layout they report on.

Worked example 1 — growing (G1)

Container 600px. Items: `flex: 1 1 100px`, `flex: 2 1 150px`, `flex: 1 1 50px`.

1. Base sizes 100 + 150 + 50 = 300. Free space = 600 − 300 = **+300** → grow.
2. Total grow shares = 1 + 2 + 1 = 4, so one share = 300 / 4 = 75px.
3. A = 100 + 1×75 = **175**; B = 150 + 2×75 = **300**; C = 50 + 1×75 = **125**. Total 600. ✓

Note that B ends up **300**, not "twice as wide as A" — `flex-grow` divides the *free space*, not the final widths. Only when all bases are 0 (next examples) do grow factors become width ratios.

Worked example 2 — shrinking (G2)

Container 400px. Items: `flex: 0 1 300px`, `flex: 0 1 200px`, `flex: 0 2 100px`.

1. Bases 300 + 200 + 100 = 600. Free space = 400 − 600 = **−200** → shrink.
2. Scaled shrink factors (shrink × basis): A = 1×300 = 300, B = 1×200 = 200, C = 2×100 = 200. Sum = **700**.
3. A loses 200 × 300/700 = 85.714 → **214.286**. B loses 200 × 200/700 = 57.143 → **142.857**. C loses 200 × 200/700 = 57.143 → **42.857**.

C has *twice* the shrink factor of B yet loses the *same* amount, because its basis is half as big. Shrinking is weighted by size so small items don't get crushed to zero first.

Worked example 3 — gaps (G3)

Same items as G1 with `gap: 20px`: two gaps take 40px first. Free space = 600 − 300 − 40 = 260 → one share = 65 → **165 / 280 / 115**.

Worked example 4 — a max-width freezes an item (G4)

Container 600px, all items `flex: 1 1 0`, A has `max-width: 100px`.

1. Round 1: free space 600, each gets 200. A violates its max → clamped to 100 and **frozen**.
2. Round 2: free space 600 − 100 = 500 shared by B and C → **250** each.

Result **100 / 250 / 250** — not 100/200/200 with 100px left over. The loop in step 5 is why.

Worked example 5 — factors below 1 (G7)

Container 600px, two items `flex: 0.25 1 100px`. Free space = 400. The factors add up to 0.5, so only half the free space (200) is handed out: each item gets 100 more → **200 / 200**, leaving 200px empty. (With `flex-grow: 1` each they would be 300 / 300.)

Verified — the maths vs. Chrome

`code/02_flex_math.mjs` runs the algorithm with logging (key part of `flex_algorithm.mjs`):

```
if (growing) {
  const sum = open.reduce((s, i) => s + i.grow, 0);
  for (const i of open) i.target = i.basis + free * i.grow / sum;
} else {
  const sum = open.reduce((s, i) => s + i.shrink * i.basis, 0);          // scaled shrink factors
  for (const i of open) i.target = i.basis + free * (i.shrink * i.basis) / sum;   // free < 0 here
}
// ...then clamp to min/max, freeze violators, repeat
```

Output (G2 and G4):

```
== G2 shrink
  space used by hypothetical sizes = 300 + 200 + 100 = 600
  container = 400  ->  overflow, use flex-shrink
  round 1: free space = -200 shared by A, B, C
    A: 300 − 200 × (1×300)/700 = 214.286
    B: 200 − 200 × (1×200)/700 = 142.857
    C: 100 − 200 × (2×100)/700 = 42.857
  result: A=214.286  B=142.857  C=42.857

== G4 flex 1 1 0, A max 100
  space used by hypothetical sizes = 0 + 0 + 0 = 0
  container = 600  ->  free space, use flex-grow
  round 1: free space = 600 shared by A, B, C
    A: 0 + 600 × 1/3 = 200
    B: 0 + 600 × 1/3 = 200
    C: 0 + 600 × 1/3 = 200
    A violates its max -> clamped to 100
  round 2: free space = 500 shared by B, C
    B: 0 + 500 × 1/2 = 250
    C: 0 + 500 × 1/2 = 250
  result: A=100  B=250  C=250
```

Then `code/03_flex_measure.mjs` opens `02_grow_shrink.html` in headless Chrome, reads every item's `getBoundingClientRect().width`, and compares it with the algorithm. For G5 and G6 it first measures each item's content size (min-content — the narrowest it can be without overflowing, roughly its longest word — and max-content — its width with no line breaks at all) with a hidden clone (a copy of the element), because those cases depend on text width:

```
case   predicted (spec algorithm)          measured in Chrome                diff
g1     175 / 300 / 125                     175 / 300 / 125                   0
g2     214.286 / 142.857 / 42.857          214.281 / 142.859 / 42.859        0.005
g3     165 / 280 / 115                     165 / 280 / 115                   0
g4     100 / 250 / 250                     100 / 250 / 250                   0
g5a    200 / 200 / 200                     200 / 200 / 200                   0
g5b    160.266 / 252.984 / 186.75          160.266 / 252.984 / 186.75        0
g6a    251.672 / 48.328                    251.672 / 48.328                  0
g6b    150 / 150                           150 / 150                         0
g7     200 / 200                           200 / 200                         0

content sizes used above (min-content / max-content):
  g5b 33.125 / 33.125  |  125.844 / 125.844  |  59.609 / 59.609
  g6a 251.672 / 251.672  |  6.625 / 6.625
```

Every case matches. The tiny G2 difference is not an error in the maths: Chrome stores layout positions in **1/64-pixel units**, so 214.2857… is stored as 13714/64 = 214.28125, and the rounding leftovers are pushed onto the neighbours so the three still add up to exactly 400.

Common mistakes

- Expecting `flex-grow: 2` to make an item twice as wide as a `flex-grow: 1` sibling. Only true when their bases are equal (typically 0).
- Forgetting that padding and border count toward the base: an item with `flex: 1` (basis 0) and `padding: 0 40px` still ends up 80px wider than an unpadded sibling.
- Giving everything `flex-shrink: 0` and then wondering why the row overflows on phones.

Self-check

1. Container 500px, items `flex: 1 1 100px` and `flex: 3 1 100px`. Widths? → free 300; shares 4 → 75 each → **175 and 325**.
2. Container 300px, items `flex: 0 1 200px` and `flex: 0 1 200px`. Widths? → overflow 100, equal scaled factors → **150 and 150**.
3. Container 300px, items `flex: 0 1 200px` and `flex: 0 3 200px`. Widths? → scaled 200 and 600 (sum 800); losses 25 and 75 → **175 and 125**.
4. Why did G4 give 250 to B and C instead of 200? → A was frozen at its max after round 1 and the leftover was redistributed.

8.4 The `flex` shorthand and minimum sizes

The shorthand

Always prefer `flex` over the three longhands (the individual properties a shorthand stands for): it sets them together and has sensible defaults for the parts you omit (omitted `flex-shrink` → 1, omitted `flex-basis` → **0%**, not `auto`).

You write	Means (grow shrink basis)	Behaviour	Typical use
(nothing) / `flex: initial`	`0 1 auto`	Content-sized, may shrink, won't grow	Default
`flex: 1`	`1 1 0%`	Ignore content size, share **all** space equally	Equal columns, "fill the rest"
`flex: 2`	`2 1 0%`	Twice the share of a `flex: 1` sibling	Proportional columns
`flex: auto`	`1 1 auto`	Start from content size, share the **leftover** equally	Nav tabs that grow but keep content differences
`flex: none`	`0 0 auto`	Rigid: never grows or shrinks	Icons, avatars (small user profile pictures), buttons
`flex: 200px`	`1 1 200px`	Start at 200px, grow/shrink from there	Wrapping card rows
`flex: 0 0 200px`	exactly that	Fixed 200px	Sidebars

G5 in Figure 8.2 shows the `flex: 1` vs `flex: auto` difference with the same three labels: `flex: 1` gives 200/200/200 regardless of text; `flex: auto` starts from the text widths (33.1, 125.8, 59.6) and adds an equal 127.1px to each, giving 160.3 / 253.0 / 186.8.

`min-width: auto` — the hidden minimum

Flex items have `min-width: auto` (in a column, `min-height: auto`), and for flex items `auto` means **"never shrink below my min-content size"** — the width of the longest word, URL or unshrinkable child. This protects text from being crushed, but it is also the cause of the most common flexbox bug: *"my flex item overflows its container even though it has `flex: 1`"*.

G6 shows it: two items with `flex: 1` in a 300px container should be 150/150, but the first contains an unbreakable URL whose min-content width is 251.672px. The browser refuses to go below that, so the second item gets only 48.328px and the text looks broken. The fix is one line:

```
.item { flex: 1; min-width: 0; }           /* allow shrinking below content size */
/* plus a way to deal with the overflowing content: */
.item { overflow: hidden; text-overflow: ellipsis; white-space: nowrap; }  /* or overflow-wrap: anywhere */
```

Typical places this bites: a code block or table inside a flex column, a long file name in a list row, a chart inside a flex child, nested flex layouts where an inner item contains a wide `<pre>`. Making the item a scroll container (a box that clips and can scroll its content: `overflow: hidden`, `auto` or `scroll`) also removes the automatic minimum (`overflow: clip` does not — it isn't a scroll container).

Self-check

1. What is the basis in `flex: 1`? → `0%`.
2. Two items with `flex: auto` and different text lengths — equal widths? → No; each gets its content width plus an equal share of the leftover.
3. A `flex: 1` sidebar containing a 900px-wide `<pre>` pushes the page wider than the screen. Fix? → `min-width: 0` on the flex item (and `overflow: auto` on the `<pre>`).

8.5 Alignment

What it is

Once item sizes are known, there may still be **free space** — along the main axis (items don't fill the line) and on the cross axis (items shorter than the line). Alignment properties decide where that space goes. They come from the CSS Box Alignment module (the part of the CSS spec that defines these alignment properties), which Grid shares.

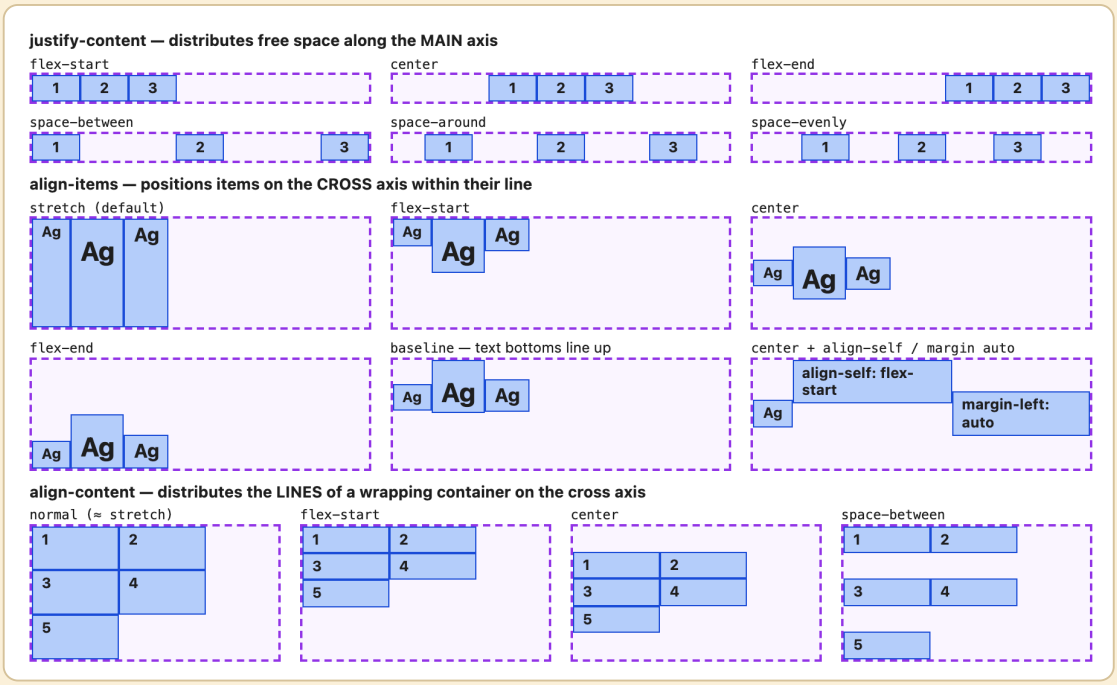


Figure 8.3 — `code/04_alignment.html`.

`justify-content` — main axis, on the container

Value	Where the free space goes
`flex-start` (effectively the default, `normal`)	All at the end; items packed at main-start
`flex-end`	All at the start
`center`	Half on each side
`space-between`	Between items only; first and last touch the edges
`space-around`	Equal space around each item, so edges get **half** a gap
`space-evenly`	Equal gaps everywhere, edges included

`start` / `end` also work (they follow the writing direction — left-to-right or right-to-left, depending on the language — rather than the flex direction, which matters only with `row-reverse` / `column-reverse`). `justify-content` does nothing if any item has `flex-grow` > 0 and there's no max-size limit — there is no free space left to distribute.

`align-items` and `align-self` — cross axis, per line

Value	Effect on each item
`stretch` (default — `normal` behaves like it)	Items fill the line's cross size — **only if** their cross size (`height` in a row) is `auto`
`flex-start` / `start`	Top of the line (in a row)
`flex-end` / `end`	Bottom
`center`	Middle
`baseline`	Items shifted so the **first line of text** in each shares one baseline (the invisible line letters sit on) — perfect for a label next to a big number or mixed font sizes

`align-items` sets the default for all items; `align-self` on one item overrides it. There is **no** `justify-self` in flexbox — the main axis is shared, so you can't align one item alone on it. Use an **auto margin** instead.

Auto margins

In flexbox, `margin: auto` absorbs free space *before* `justify-content` and in either axis:

- `margin-left: auto` (or `margin-inline-start: auto`) on an item pushes it — and everything after it — to the end. The classic "logo left, links left, Sign in far right".
- `margin-top: auto` on the last item in a column pushes it to the bottom (a card's button).
- `margin: auto` on the only item centres it both ways.

`align-content` — distributing the lines

With `flex-wrap: wrap` there may be several **flex lines** (rows of items, when items wrap onto more than one row); `align-content` distributes them on the cross axis like `justify-content` does for items (`flex-start`, `center`, `space-between`, `space-around`, `space-evenly`, `stretch`). The default `normal` behaves as `stretch`: lines grow to fill the container. It has **no effect on a single-line (`nowrap`) container**.

Common mistakes

- Trying `align-items: center` to centre items horizontally in a row (that's `justify-content`).
- Setting a `height` on items and wondering why `stretch` stopped working.
- Using `align-content` on a non-wrapping container.
- `justify-content: center` on a row that overflows: content spills off *both* sides and the left part can't be scrolled to. `justify-content: safe center` falls back to start when overflowing (Baseline since 2024 — Chrome 115, Firefox 63, Safari 17.6), or use `margin: auto` on the items, which never pushes content past the start edge.

Self-check

1. Three items, `justify-content: space-between` — is there space before the first? → No.
2. How do you push only the last button of a toolbar to the right? → `margin-left: auto` on it.
3. Which property aligns the lines of a wrapping flex container? → `align-content`.

8.6 Wrapping and `gap`

`flex-wrap`

Value	Behaviour
`nowrap` (default)	One line. Items shrink (if allowed) and then **overflow**
`wrap`	New lines are added when the next item doesn't fit; lines stack along the cross axis
`wrap-reverse`	Same, but new lines go *above* (cross-start and cross-end swap)

`flex-flow: row wrap` is the shorthand for `flex-direction` + `flex-wrap`.

Each line is sized **independently**: the grow/shrink maths of 8.3 runs once per line, on the items in that line. Items are placed into a line using their *hypothetical* sizes (basis clamped by min/max) — as many as fit, then a new line.

`gap`

`gap: <row-gap> <column-gap>` (or `gap: 12px` for both) puts fixed space **between** items and between lines — never on the outer edges. It is not a margin, never collapses, and is subtracted before free space is shared (G3). `gap` in flexbox has been Baseline (working in all major browsers) since 2021; before that people used margins on items plus a negative margin on the container.

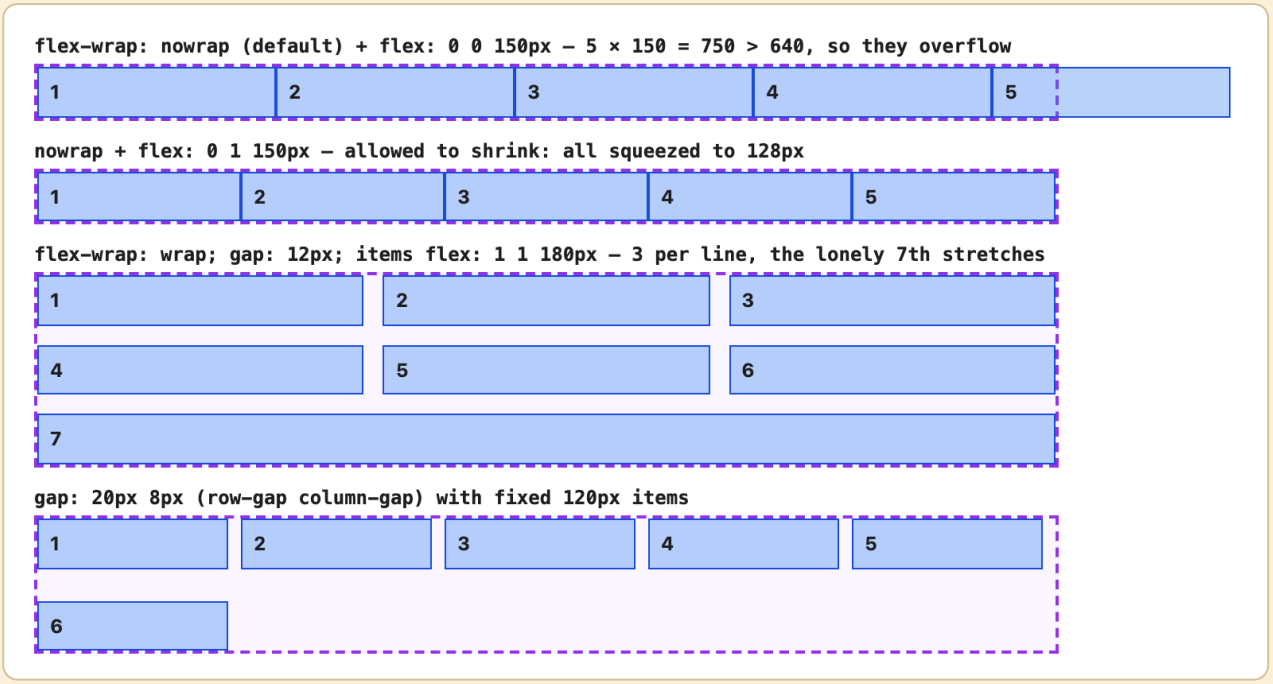


Figure 8.4 — `code/05_wrap_gap.html`.

`code/05_wrap_gap.mjs` groups the cards by their top edge to count items per line, at two container widths:

```
container 640px (flex: 1 1 180px, gap 12px):
  line 1: 3 item(s), widths 205.33, 205.34, 205.33
  line 2: 3 item(s), widths 205.33, 205.34, 205.33
  line 3: 1 item(s), widths 640
container 400px (flex: 1 1 180px, gap 12px):
  line 1: 2 item(s), widths 194, 194
  line 2: 2 item(s), widths 194, 194
  line 3: 2 item(s), widths 194, 194
  line 4: 1 item(s), widths 400
nowrap + flex: 0 1 150px in 640px: 128, 128, 128, 128, 128
```

Check the first line by hand: 3 × 180 + 2 × 12 = 564 ≤ 640, but 4 × 180 + 3 × 12 = 756 > 640, so 3 fit. Free space 640 − 564 = 76 → +25.33 each → 205.33. The last card is alone on its line, so it takes all 640px. The shrink row: overflow 750 − 640 = 110, equal scaled factors → each loses 22 → 128.

The "last row" problem — and when to switch to grid

A wrapping flex row is **content-out** (sizes are driven by the content): each line decides its own sizes, so items on a short last line grow wider than the ones above (the 640px card). That is great for tags or buttons of different lengths, and wrong for a uniform card grid. For equal columns that wrap, use Grid's `repeat(auto-fill, minmax(180px, 1fr))` (Chapter 9), which is **layout-in**: tracks (the rows and columns of a grid) are defined first and items drop into them. (A flex workaround is `flex-grow: 0` with a fixed basis, at the cost of an uneven right edge.)

Self-check

1. Container 500px, items `flex: 1 1 150px`, `gap: 10px`. Items per line? → 3 (3×150 + 2×10 = 470 ≤ 500; 4 would need 630).
2. Does `gap` add space before the first item? → No, only between items and lines.

3. Why does the last card grow wider than the rest? → Each flex line distributes its own free space; a lone item gets all of it.

8.7 Common patterns

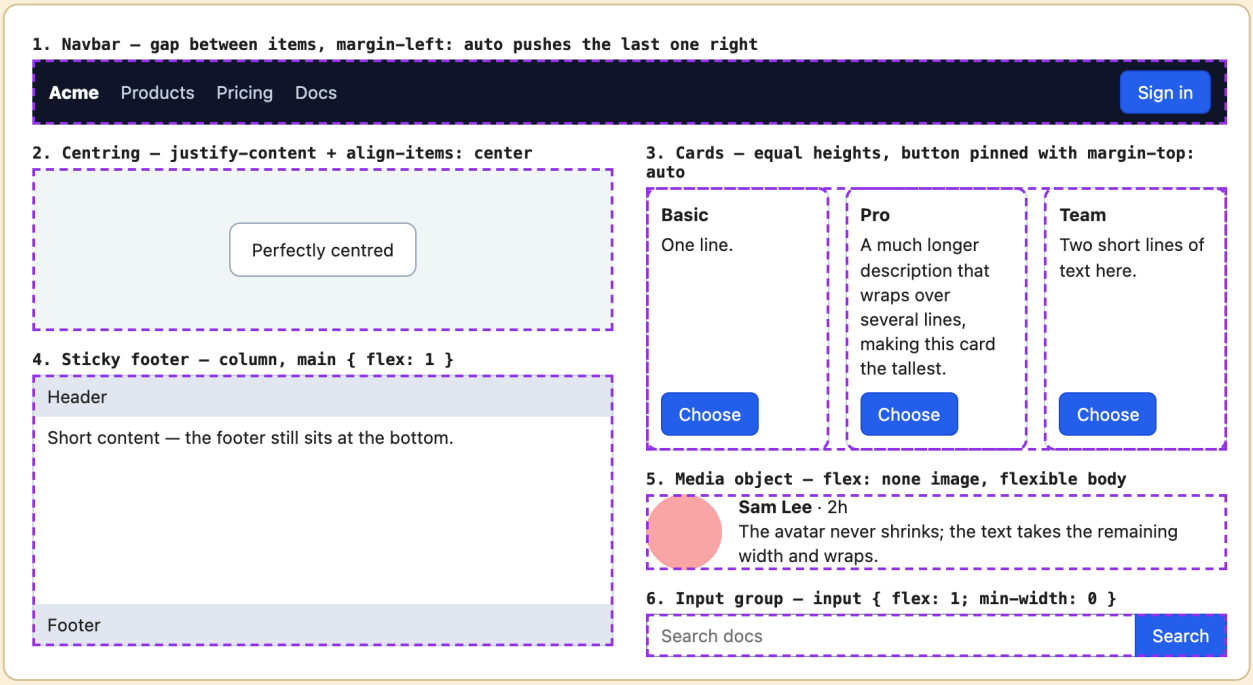


Figure 8.5 — `code/06_patterns.html`. Each flex container has a dashed purple outline.

```css
/* 1. Navbar */
.nav       { display: flex; align-items: center; gap: 16px; }
.nav .push { margin-left: auto; }                    /* everything from here on goes right */

/* 2. Centre anything, both axes */
.center { display: flex; justify-content: center; align-items: center; }
/* (grid alternative: display: grid; place-items: center) */

/* 3. Equal-height cards with buttons aligned at the bottom */
.cards       { display: flex; gap: 12px; }            /* align-items: stretch -> equal heights */
.card        { flex: 1; display: flex; flex-direction: column; }
.card button { margin-top: auto; align-self: flex-start; }

/* 4. Sticky footer: footer at the bottom even when content is short */
body      { display: flex; flex-direction: column; min-height: 100dvh; }   /* demo uses 200px */
body main { flex: 1; }

/* 5. Media object: fixed image, flexible text */
.media         { display: flex; gap: 12px; align-items: flex-start; }
.media .avatar { flex: none; width: 56px; height: 56px; }
.media .body   { flex: 1; min-width: 0; }

/* 6. Input group */
.group       { display: flex; }
.group input { flex: 1; min-width: 0; }
```

Measured by `code/06_patterns.mjs`:

```
navbar : gap nav-right-edge minus button-right-edge = 12px (the 12px padding)
centre : space left/right 144.65625 / 144.65625, top/bottom 39.90625 / 39.90625
cards  : heights 193.5, 193.5, 193.5; button bottoms 308.6875, 308.6875, 308.6875
footer : page bottom 464.1875, footer bottom 463.1875 (only the 1px border below it)
media  : avatar width 56px (flex: none, never shrinks)
input  : input 359.875px + button 68.125px = form 428px
```

Notes on each:

- **Navbar**: use a real `<nav>` with a list of links in production (Chapter 3); the `<ul>` becomes the flex container. Add `flex-wrap: wrap` or a menu button for narrow screens (Chapter 10).
- **Centring**: works for unknown sizes on both axes — the thing that took hacks before flexbox. If the centred child can be taller than the container, prefer `margin: auto` on the child so it can't overflow off the top.
- **Cards**: equal heights come free from `align-items: stretch`; the column + `margin-top: auto` keeps buttons in one line. If you also need the *titles and paragraphs* aligned across cards, that is a job for `subgrid` (letting a nested grid reuse its parent's rows) (Chapter 9).
- **Sticky footer**: `100dvh` is the *dynamic* viewport height (the viewport is the visible browser area), which accounts for mobile browser toolbars; `100vh` can be too tall on phones (Chapter 10).
- **Media object**: `flex: none` keeps the avatar from being squashed; `min-width: 0` lets long names truncate.
- **Input group**: `<input>` has a built-in default width (from its `size` attribute), so without `min-width: 0` it can refuse to shrink on small screens.

Real-world examples

GitHub's repository header, most site navbars, Bootstrap 5's `.d-flex` / `.justify-content-between` utilities and Tailwind's `flex`, `items-center`, `justify-between`, `gap-4` classes (Bootstrap and Tailwind are popular CSS frameworks; their *utility classes* each apply one CSS property) all map one-to-one to the properties in this chapter.

8.8 Pitfalls, debugging, and interview prep

Pitfalls at a glance

Symptom	Cause	Fix
Item with `flex: 1` overflows / text doesn't truncate	`min-width: auto` = min-content	`min-width: 0` (and `overflow` / `text-overflow` on the content)
Image in a flex row is squashed	Images shrink like any item	`flex: none` or `flex-shrink: 0` on it
Image in a row becomes tall and distorted or overflows	`align-items: stretch` stretches images to the line height (a 100×50 image in a 300px-tall row became 600×300 in Chrome)	`align-self: flex-start` (or `center`) on the image
`align-items: stretch` has no effect	Item has an explicit `height`	Remove it or use `min-height`
`height: 100%` on an item's child doesn't work	Parent's height is not definite (not fixed in advance)	Make the item a flex container too, or use `flex: 1` down the chain
`justify-content` does nothing	An item has `flex-grow`, so no free space remains	Remove the grow, or use `max-width`
Last row of wrapped cards is wider	Per-line sizing	Use grid
Tab order doesn't match what users see	`order` / `*-reverse`	Change the HTML instead
Space appears between items that you didn't add	`justify-content: space-*` or leftover `margin`	Inspect in DevTools

Debugging with DevTools

Chrome, Edge and Firefox show a **flex badge** next to flex containers in the Elements panel of DevTools (the built-in developer tools, opened with F12). Clicking it overlays the container, items, free space and gaps; the **Flexbox editor** (icon next to `display: flex` in the Styles pane) lets you click through `justify-content` / `align-items` values. Firefox's Layout panel also explains *why* an item has its size: its base size, how much it grew or shrank, and whether a min/max clamp kicked in.

In the interview

> **30-second answer:** "Flexbox is one-dimensional layout. `display: flex` makes direct children flex items, placed along a main axis set by `flex-direction`; the cross axis is perpendicular. `justify-content` distributes space on the main axis, `align-items` and `align-self` align on the cross axis, and `align-content` distributes wrapped lines. Item sizes come from `flex-basis`, then positive free space is shared by `flex-grow` ratios and overflow is taken back in proportion to `flex-shrink` times the basis, with min/max clamps. `flex: 1` means `1 1 0%`, so items share all the space equally. The classic bug is `min-width: auto`, which stops items shrinking below their content — fix with `min-width: 0`."

Follow-ups:

- *What's the difference between `flex: 1` and `flex: auto`?* → Basis 0% vs basis auto: `flex: 1` ignores content size and splits everything; `flex: auto` keeps content differences and splits only the leftover (G5).
- *How is shrinking calculated?* → Overflow × (shrink × basis) / Σ(shrink × basis) — weighted by basis so small items don't vanish (G2: 214.29 / 142.86 / 42.86).
- *Flexbox or grid for a page layout?* → Grid for two-dimensional page structure and uniform card grids; flexbox inside components for rows of things whose size depends on content. They are often nested.
- *How do you centre a div?* → `display: flex; justify-content: center; align-items: center` on the parent, or `display: grid; place-items: center`.

Self-check

1. An item in a row has `height: 50px`; the container is 200px tall with `align-items: stretch`. How tall is the item? → 50px — stretch only applies to `auto` cross sizes.
2. Which one-liner shrinks a flex item below its longest word? → `min-width: 0`.
3. What is `flex: none` in longhand? → `0 0 auto`.

Chapter 8 TL;DR

- `display: flex` → direct children become **flex items**; no margin collapsing, floats ignored, items blockified.
- **Main axis** = `flex-direction` (`row` default). `justify-content` → main axis. `align-items` / `align-self` / `align-content` → cross axis. Swap direction and the meanings swap with it.
- `order` and `*-reverse` change only visuals, not Tab or reading order (`reading-flow` is Chromium-only, not Baseline).
- **Sizing**: start from `flex-basis`; positive free space split by `flex-grow` shares (`basis + free × grow/Σgrow`); overflow taken back by `flex-shrink × basis` weights; clamp to min/max, freeze, repeat. Verified: 175/300/125 (grow), 214.29/142.86/42.86 (shrink), 100/250/250 (max-width freeze). Chrome stores sizes in 1/64 px.
- **Shorthand**: `flex: 1` = `1 1 0%`, `auto` = `1 1 auto`, `none` = `0 0 auto`, default `0 1 auto`, `flex: 200px` = `1 1 200px`.
- `min-width: auto` = min-content for flex items → overflowing `flex: 1` items; fix with `min-width: 0`.
- **Auto margins** absorb free space: `margin-left: auto` pushes right, `margin-top: auto` pushes down, `margin: auto` centres.
- **Wrap + gap**: lines are sized independently; `gap` only between items; wrapped card grids with an even last row → use grid.
- **Patterns**: navbar, centring, equal-height cards with bottom-aligned buttons, sticky footer (`min-height: 100dvh` + `main { flex: 1 }`), media object, input group.

Next: Chapter 9 — CSS Grid.

Chapter 9 — CSS Grid

Flexbox (Chapter 8) arranges items along **one** axis and lets their content decide the sizes. **CSS Grid** is the other half of modern layout: you define **rows and columns at the same time**, then place items into the cells — or let the browser place them for you. It is the right tool for page structure, card galleries, dashboards (screens of charts and key numbers), forms with aligned labels, and any design that a designer drew on a grid. Grid has worked in every major browser since 2017 (Baseline widely available — Baseline is the label for features that work in all major browsers); `subgrid`, added later, is Baseline since September 2023.

What you'll learn

#	Topic	You will be able to…
9.1	Grid vocabulary	name lines, tracks, cells, areas and gaps, and turn a container into a grid
9.2	Defining tracks	size tracks with px, %, `fr`, `auto`, `min-content`, `max-content`, `fit-content()`, `minmax()` — and do the `fr` maths by hand
9.3	`repeat()`, `auto-fill` vs `auto-fit`	build a responsive grid (one that adapts to the screen width) with no media queries (CSS rules that apply only at certain screen sizes, Chapter 10) and predict its column count
9.4	Placing items	use line numbers, `span`, negative lines, named lines, `grid-template-areas` and overlap
9.5	The implicit grid and auto-placement	control extra rows/columns, `grid-auto-flow` and `dense`
9.6	Alignment	align items in cells and the whole grid in its container
9.7	Subgrid	line up content across sibling cards
9.8	Grid vs Flexbox	choose the right tool (and combine them)
9.9	Patterns	holy grail layout (the classic header + three columns + footer page), responsive card grid, overlays (one element laid over another)
9.10	Debugging, pitfalls and interview prep	find and fix the common grid bugs

As in Chapters 7 and 8, every example is a page in `code/` with a screenshot in `img/`: grid containers have a **dashed purple border**, items a **solid blue outline**, **red badges** mark numbered grid lines and **red numbers** are track sizes the page read back from the browser. The `.mjs` scripts read `getComputedStyle(grid).gridTemplateColumns` (`getComputedStyle` is the JavaScript function that returns an element's final styles), which returns the **resolved** track sizes (the final numbers after the browser has done the maths) in pixels — the browser's own answer — and compare it with the hand calculation.

9.1 Grid vocabulary

What it is

`display: grid` (or `inline-grid`) makes an element a **grid container**; its direct children become **grid items**. The container defines an invisible grid:

Term	Meaning
Grid line	A dividing line between tracks. Numbered from **1** at the start edge; also numbered backwards from **−1** at the end edge. A grid with 3 columns has 4 column lines.
Track	The space between two adjacent lines: a **column** or a **row**.
Cell	The intersection of one row and one column — the smallest unit.
Grid area	Any rectangle of cells bounded by four lines (can be named).
Gutter / gap	Space between tracks, set with `gap`, `row-gap`, `column-gap`. Gaps exist only *between* tracks, never at the edges.
Explicit grid	The tracks you declared with `grid-template-*`.
Implicit grid	Extra tracks the browser creates when items are placed outside the explicit grid, or when there are more items than cells.

Analogy: a spreadsheet. The column and row dividers are the lines, a column is a track, a cell is a cell, and a merged range of cells is a grid area. Unlike a spreadsheet, you size the columns with maths (`fr`, `minmax`) and one item may span several cells.

Grid behaviours to know up front

- As with flex items, grid items are blockified (treated as block boxes, so even a `<span>` accepts width and height), `float` is ignored, margins don't collapse, and `z-index` works without `position` (Chapter 7).
- Grid is **layout-in**: tracks are defined by the container and items fall into them. Flexbox is **content-out**: the items' sizes drive the layout. That difference drives the choice in 9.8.

Self-check

1. How many column lines does `grid-template-columns: 1fr 1fr 1fr 1fr` create? → 5.
2. Which line is `-1`? → The last line (the end edge of the explicit grid).

9.2 Defining tracks

What it is

`grid-template-columns` and `grid-template-rows` list the size of each track, separated by spaces. Values you can use per track:

Value	Meaning
`200px`, `10rem`, `30%`	Fixed size (percent of the container's size)
`fr`	A **fraction of the leftover space** after fixed tracks and gaps are taken out
`auto`	As a maximum: content size, but stretches to fill leftover space when there are no `fr` tracks (with `align/justify-content: normal`). As a minimum: the item's automatic minimum size (usually its min-content size, so content is not crushed)
`min-content`	As narrow as the content can get (longest word)
`max-content`	As wide as the content wants, no wrapping
`fit-content(150px)`	`max-content`, but capped at 150px (and at least `min-content`)
`minmax(min, max)`	Any size between a min and a max, e.g. `minmax(200px, 1fr)`

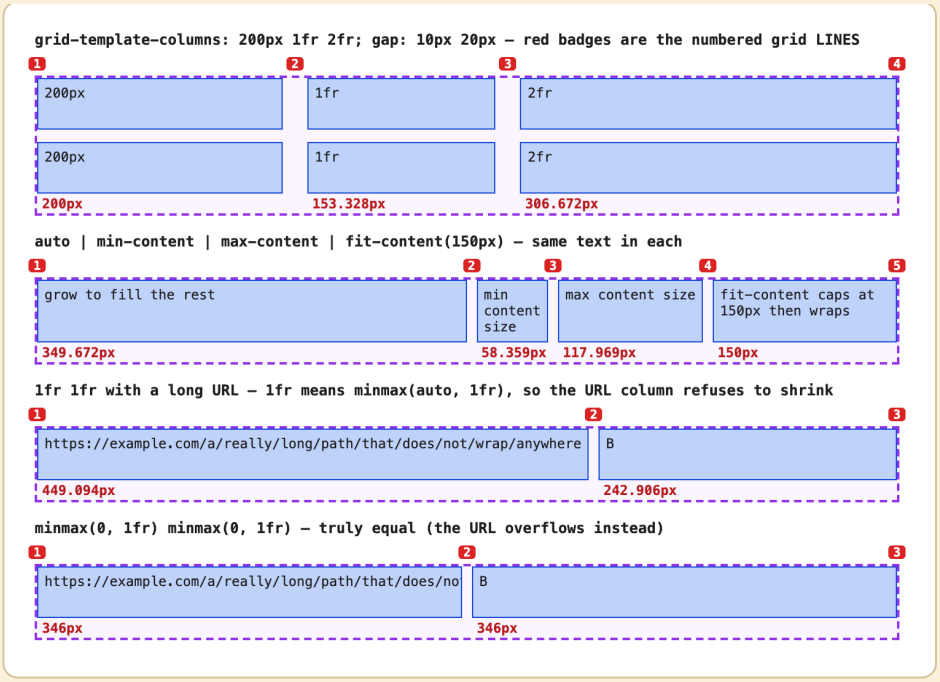


Figure 9.1 — `code/01_tracks.html`. A script in the page reads `getComputedStyle(grid).gridTemplateColumns` and draws the line numbers and sizes; they are absolutely positioned, so they don't change the layout.

How `fr` is calculated

1. Start with the container's inner size (700px here).
2. Subtract every **fixed** track and every gap. For `200px 1fr 2fr` with `column-gap: 20px`: 700 − 200 − 2 × 20 = **460px** of free space.
3. Add up the `fr` values: 1 + 2 = 3, so **1fr = 460 / 3 = 153.33px**, 2fr = 306.67px.
4. (A track whose share would be smaller than its minimum — see below — is fixed at that minimum and the rest is re-split, like flexbox's freeze loop in §8.3.)

`code/01_tracks.mjs` prints the declared value next to what Chrome resolved, then the hand calculation:

```
const rule = [...document.styleSheets[0].cssRules].find(r => r.selectorText === '#' + g.id);
return `${g.id}  ${rule.style.gridTemplateColumns} -> ${getComputedStyle(g).gridTemplateColumns}`;
```

Output:

```
container content width: 700px
t1  200px 1fr 2fr                                   -> 200px 153.328px 306.672px
t2  auto min-content max-content fit-content(150px)  -> 349.672px 58.3594px 117.969px 150px
t3  1fr 1fr                                         -> 449.094px 242.906px
t4  minmax(0px, 1fr) minmax(0px, 1fr)               -> 346px 346px

by hand, t1: 1fr = 460 / 3 = 153.333px, 2fr = 306.667px (Chrome stores 1/64 px: 153.328125, 306.671875)
```

The tiny differences are Chrome's 1/64-pixel layout units, exactly as in the flexbox chapter.

`fr` vs `%`

`%` tracks are a share of the *whole* container and ignore gaps, so `50% 50%` plus a gap overflows. `fr` shares only what's left after gaps: `1fr 1fr` never overflows because of gaps.

The `1fr` minimum trap (t3 vs t4)

`1fr` is shorthand for `minmax(auto, 1fr)`. The `auto` minimum means "never narrower than the item's min-content size". So in t3 a long unbreakable URL forces its column to 449px and the other gets the remaining 242.9px — the columns are not equal. It is the grid twin of flexbox's `min-width: auto` problem (§8.4).

Fix: `minmax(0, 1fr)` (t4) lets the track shrink below its content — the columns become exactly 346px each, and you handle the overflowing content with `overflow`, `overflow-wrap: anywhere` or `min-width: 0` on the item. Rule of thumb: for columns that **must** be equal (a dashboard, a code-and-preview split), write `repeat(3, minmax(0, 1fr))`.

Common mistakes

- Using `%` tracks together with `gap` and getting a horizontal scrollbar (the page becomes wider than the screen and scrolls sideways).
- Expecting `1fr 1fr` to be equal whatever the content (see above).
- Forgetting that rows default to `auto` — a row only gets taller than its content if you give it a size or the container a height with `fr` rows.

Self-check

1. Container 1000px, `grid-template-columns: 250px 1fr 1fr 1fr; column-gap: 10px`. Size of 1fr? → (1000 − 250 − 30) / 3 = **240px**.
2. What is `1fr` short for? → `minmax(auto, 1fr)`.
3. Which track value never grows beyond its content or a cap? → `fit-content(cap)`.

9.3 `repeat()`, `auto-fill` and `auto-fit`

What it is

`repeat(count, tracks)` saves typing: `repeat(4, 1fr)` = `1fr 1fr 1fr 1fr`; `repeat(2, 100px 1fr)` = `100px 1fr 100px 1fr`.

Instead of a number, the count can be `auto-fill` or `auto-fit`: "as many tracks as fit". Combined with `minmax()` this gives a responsive grid with **no media queries** — often called the RAM pattern (**R**epeat, **A**uto, **M**inmax):

```
.gallery { display: grid; gap: 10px;
           grid-template-columns: repeat(auto-fill, minmax(120px, 1fr)); }
```

How it works

1. The browser computes how many tracks of the *minimum* size fit: **n = floor((width + gap) / (min + gap))**.
2. It creates n tracks, then the `1fr` maximum stretches them to share the leftover space.
3. Items are auto-placed into them; extra items wrap onto new rows.
4. The difference appears only when there are **fewer items than tracks**:
   - `auto-fill` keeps the empty tracks — items stay at their "one-column" width and space is reserved on the right.
   - `auto-fit` **collapses** empty tracks to 0 (and the gaps next to them), so the items stretch to fill the row.

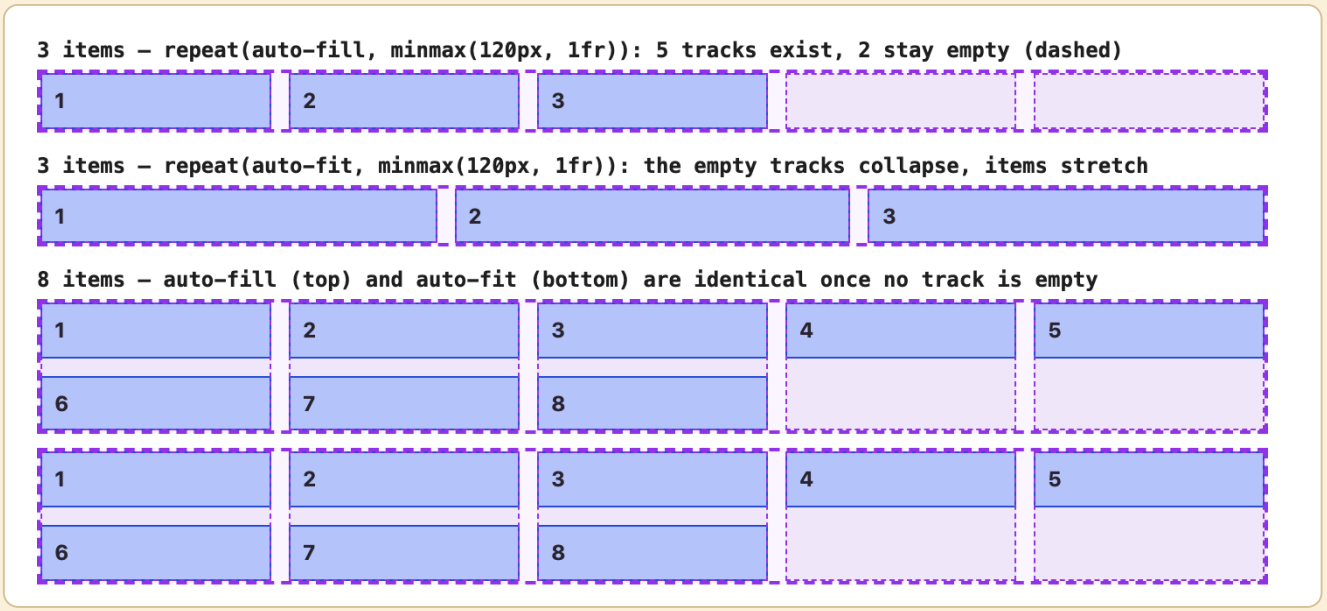


Figure 9.2 — `code/02_auto_fill_fit.html`. Every track, including empty ones, is outlined in dashed purple from the resolved track list.

Output of `code/02_auto_fill_fit.mjs`:

```
container 700px, gap 10px, minmax(120px, 1fr):
  fill3  132px 132px 132px 132px 132px
  fit3   226.656px 226.672px 226.672px 0px 0px
  fill8  132px 132px 132px 132px 132px
  fit8   132px 132px 132px 132px 132px
container 400px, gap 10px, minmax(120px, 1fr):
  fill3  126.656px 126.672px 126.656px
  fit3   126.656px 126.672px 126.656px
  fill8  126.656px 126.672px 126.656px
  fit8   126.656px 126.672px 126.656px
by hand, 700px: n = floor(710 / 130) = 5, track = (700 - 40) / 5 = 132.000px
by hand, 400px: n = floor(410 / 130) = 3, track = (400 - 20) / 3 = 126.667px
by hand, auto-fit with 3 items at 700px: empty tracks and their gaps collapse -> (700 - 20) / 3 = 226.667px
```

Note `fit3`: the two collapsed tracks show up as `0px` in the resolved list.

Which one to use

Want	Use
Cards keep a consistent size even when there are only one or two	`auto-fill`
A few items should stretch to fill the row	`auto-fit`

Guarding the minimum on narrow screens

If the container gets narrower than the minimum (a 120px track on a 100px-wide phone column), the single track overflows. Write the minimum as `min(120px, 100%)` so it can never exceed the container: `repeat(auto-fit, minmax(min(160px, 100%), 1fr))` — used in 9.9.

Self-check

1. Container 1000px, gap 20px, `repeat(auto-fill, minmax(200px, 1fr))`. How many columns and how wide? → floor(1020 / 220) = 4 columns of (1000 − 60) / 4 = **235px**.
2. Same, 2 items, `auto-fit`. Width of each? → Two tracks remain, (1000 − 20) / 2 = **490px**.
3. Can `auto-fill` be combined with a plain `1fr` (`repeat(auto-fill, 1fr)`)? → No — auto-repeat needs a definite size (one known before layout, not dependent on leftover space) (a fixed length or a `minmax` with a fixed minimum) to know how many fit.

9.4 Placing items

By default each item goes into the next free cell (9.5). To put an item somewhere specific you name its **start and end lines** on each axis.

Line-based placement

Property	Example	Meaning
`grid-column-start` / `grid-column-end`	`1` / `3`	Start at column line 1, end at line 3 (2 columns wide)
`grid-column` (shorthand)	`1 / 3`	Same
`span n`	`grid-column: span 2`	2 tracks wide, starting wherever auto-placement puts it
Negative lines	`grid-column: 1 / -1`	From the first to the **last** line: full width
`grid-row`	`1 / span 2`	Start at row line 1, 2 rows tall
`grid-area` (4 values)	`3 / 2 / 4 / 5`	**row-start / column-start / row-end / column-end** (anticlockwise from the top, unlike `margin`)

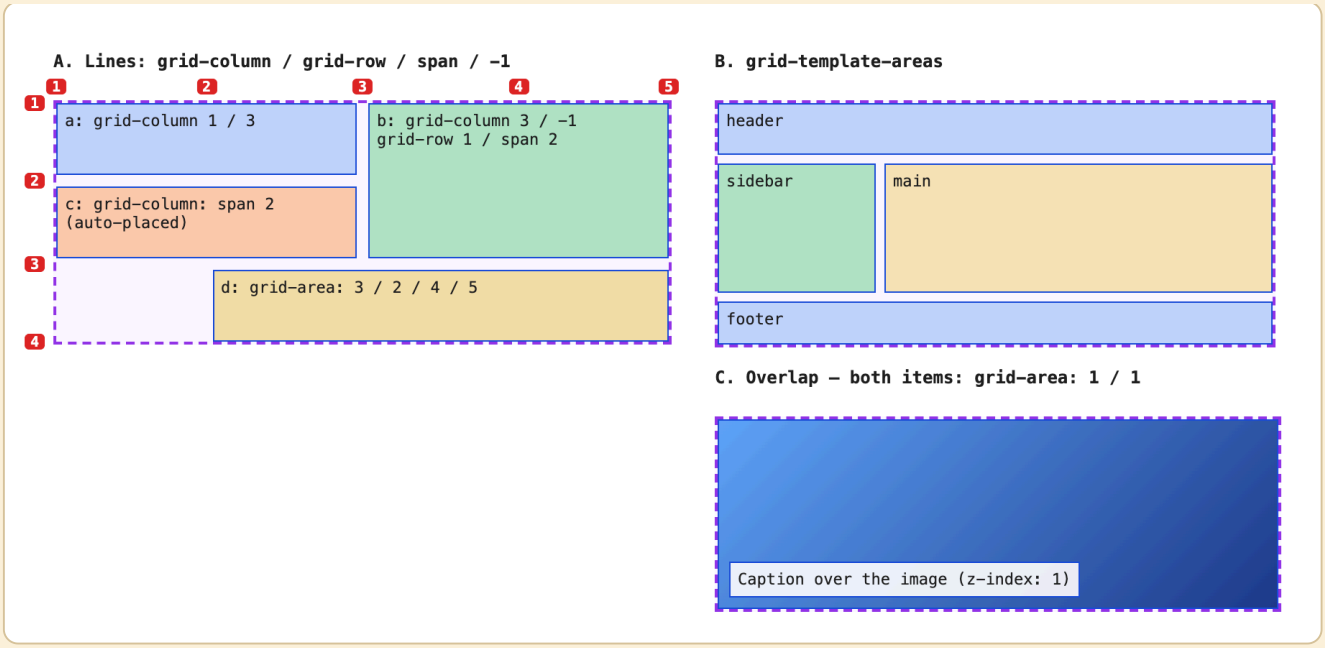


Figure 9.3 — `code/03_placement.html`.

```
#lines    { grid-template-columns: repeat(4, 1fr); grid-template-rows: repeat(3, 50px); gap: 8px; }
#lines .a { grid-column: 1 / 3; }
#lines .b { grid-column: 3 / -1; grid-row: 1 / span 2; }
#lines .c { grid-column: span 2; }           /* only a span: auto-placed into the first free spot */
#lines .d { grid-area: 3 / 2 / 4 / 5; }
```

`code/03_placement.mjs` maps each item's rectangle back to line numbers:

```
a: columns 1 -> 3, rows 1 -> 2
b: columns 3 -> 5, rows 1 -> 3
c: columns 1 -> 3, rows 2 -> 3
d: columns 2 -> 5, rows 3 -> 4
```

`-1` resolved to line 5 (the last line of a 4-column grid). Note: negative numbers count from the end of the **explicit** grid only, so `1 / -1` doesn't reach into implicit columns.

Named lines

You can name lines in square brackets and place by name — useful in big layouts where numbers change:

```
.layout { grid-template-columns: [full-start] 1fr [content-start] minmax(0, 60rem) [content-end] 1fr [full-end]; }
.layout > *      { grid-column: content; }   /* "content" = content-start / content-end */
.layout > .bleed { grid-column: full; }      /* a full-width image or banner */
```

This "full-bleed" layout keeps text in a readable centre column while chosen elements run edge to edge.

`grid-template-areas`

The most readable way to build page layouts: draw the layout as ASCII art, one string per row, then assign items to areas by name.

```
#areas { grid-template-columns: 110px 1fr;
         grid-template-rows: 36px 90px 30px;
         grid-template-areas:
           "header  header"
           "sidebar main"
           "footer  footer"; }
.h { grid-area: header; } .s { grid-area: sidebar; } .m { grid-area: main; } .f { grid-area: footer; }
```

Rules:

- Every string must have the same number of cells.
- A name repeated in adjacent cells makes one area; each area must be a **rectangle** (no L-shapes).
- A `.` (or several dots, `...`) marks an empty cell.
- Named areas automatically create named lines `header-start`/`header-end`, so `grid-column: header` also works.

- Rearranging the layout at a breakpoint (a screen width at which a media query switches the layout) is just rewriting the strings (9.9).

Overlap

Two items may occupy the same cells. Later items paint on top; `z-index` changes that (it works on grid items without `position`). Demo C puts an image and a caption in `grid-area: 1 / 1` and aligns the caption to the bottom-left with `align-self: end; justify-self: start` — a hero banner (the large image-and-headline block at the top of a page) with no `position: absolute`, so the container still grows with its content.

Accessibility note

As with flexbox `order`, visual placement doesn't change DOM order (the order of elements in the HTML): keyboard and screen-reader (software that reads the page aloud) users follow the source (§8.2). Keep source order logical; don't use placement to reorder meaningful content.

Self-check

1. Make an item span the full width regardless of column count. → `grid-column: 1 / -1`.
2. What does `grid-area: 2 / 1 / 4 / 3` mean? → Rows 2 to 4 (2 rows), columns 1 to 3 (2 columns).
3. Is this area template valid: `"a a" "a b"`? → No — area `a` would be L-shaped.

9.5 The implicit grid and auto-placement

What it is

When there are more items than explicit cells, or an item is placed at a line beyond the explicit grid, the browser adds **`implicit tracks`**. Their sizes come from `grid-auto-rows` and `grid-auto-columns` (default `auto`).

`Auto-placement` fills cells in order. `grid-auto-flow` sets how:

Value	Behaviour
`row` (default)	Fill each row left to right, then add rows
`column`	Fill each column top to bottom, then add columns
`row dense` / `column dense`	Also go **`back`** to fill earlier holes with later items that fit

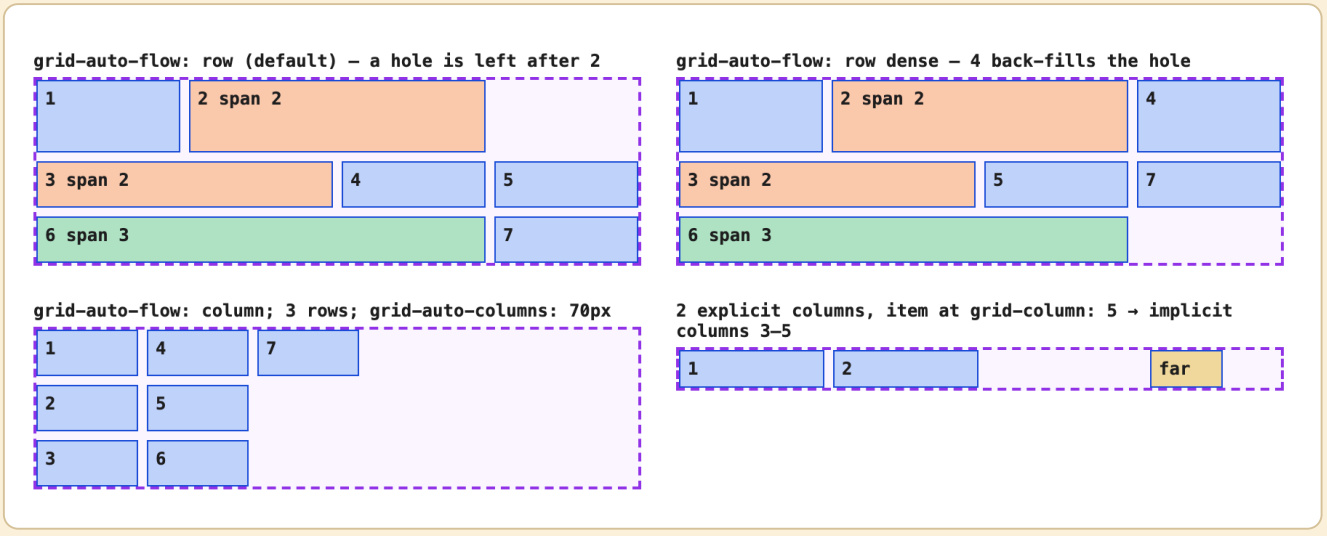


Figure 9.4 — `code/04_implicit.html`. Orange items span 2 columns, green spans 3.

```
.flow   { grid-template-columns: repeat(4, 1fr); grid-template-rows: 50px; grid-auto-rows: 32px; }
.dense  { grid-auto-flow: row dense; }
.by-col { grid-template-rows: repeat(3, 32px); grid-auto-flow: column; grid-auto-columns: 70px; }
.outside      { grid-template-columns: repeat(2, 100px); grid-auto-columns: 50px; }
.outside .far { grid-column: 5; }
```

Output of `code/04_implicit.mjs` (`r2c3` = row 2, column 3):

```
row      columns: 99px 99px 99px 99px                        rows: 50px 32px 32px
         placement: 1@r1c1 2@r1c2 3@r2c1 4@r2c3 5@r2c4 6@r3c1 7@r3c4
dense    columns: 99px 99px 99px 99px                        rows: 50px 32px 32px
         placement: 1@r1c1 2@r1c2 3@r2c1 4@r1c4 5@r2c3 6@r3c1 7@r2c4
bycol    columns: 70px 70px 70px                             rows: 32px 32px 32px
         placement: 1@r1c1 2@r2c1 3@r3c1 4@r1c2 5@r2c2 6@r3c2 7@r1c3
outside  columns: 100px 100px 50px 50px 50px                 rows: 26px
         placement: 1@r1c1 2@r1c2 far@r1c5
```

Read it like this:

- **`row`**: item 3 needs two columns; only column 4 is free in row 1, so it moves to row 2, leaving **`r1c4` empty**. Only row 1 is explicit (50px); rows 2–3 are implicit (32px from `grid-auto-rows`).
- **`dense`**: item 4 (1 column) goes back and fills r1c4. The items are now drawn **out of source order**.
- **`bycol`**: three explicit rows, columns created as needed at 70px.
- **`outside`**: placing an item at column line 5 creates implicit columns 3, 4 and 5 at 50px each.

Useful settings

- `grid-auto-rows: minmax(100px, auto)` — rows at least 100px, but grow with content. The standard for card grids.
- `dense` is fine for image galleries where order doesn't matter; avoid it where reading or tab order does — the visual order no longer matches the DOM.

Self-check

1. `grid-template-columns: repeat(3, 1fr)` and 7 items, no rows declared. How many rows, and how are they sized? → 3 implicit rows, sized by `grid-auto-rows` (default `auto` = content).
2. What does `dense` risk? → Visual order differing from DOM/focus order.

9.6 Alignment

Grid uses the same Box Alignment properties (the shared CSS alignment module, §8.5) as flexbox, but here **both axes have a `justify-*` and an `align-*`**, because items sit in two-dimensional cells:

- **inline axis** (row direction, left→right in English) → `justify-*`
- **block axis** (column direction, top→bottom) → `align-*`

Property	On	Aligns	Default
`justify-items` / `align-items`	container	every item **inside its cell**	`stretch` (unless the item has a set size or an aspect ratio — a fixed width-to-height shape — e.g. images → `start`)
`justify-self` / `align-self`	item	one item inside its cell	inherits from `*-items`
`justify-content` / `align-content`	container	the **whole grid of tracks** inside the container, when the tracks are smaller than it	`normal` (tracks packed at the start; `auto` tracks stretch to fill)
`place-items`, `place-self`, `place-content`	—	shorthands: `align` value first, then `justify`	—

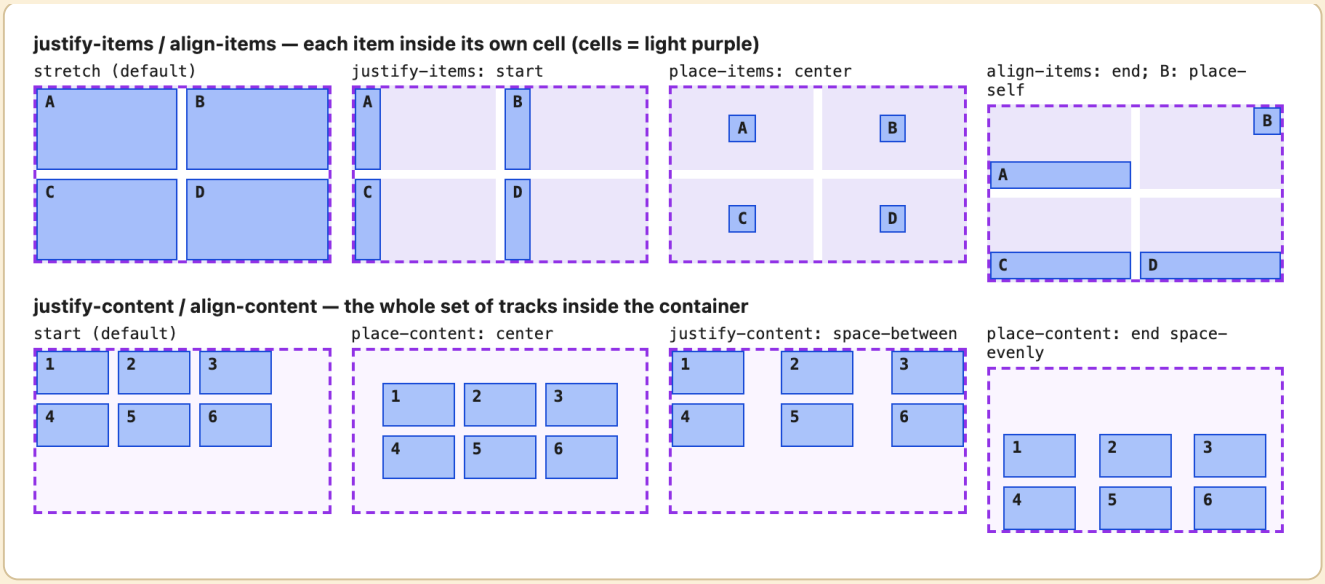


Figure 9.5 — `code/05_alignment.html`. The light-purple rectangles are the cells (painted with background gradients), so you can see each item's position inside its cell.

Key points:

- `place-items: center` on a grid is the shortest centring recipe in CSS: `display: grid; place-items: center`.
- **`*-content` only does something when there is free space** — tracks with fixed sizes smaller than the container. With `fr` tracks there's no free space and it has no effect.
- `space-between` / `space-around` / `space-evenly` in `*-content` **widen the effective gaps**; items spanning several tracks get bigger too.
- Auto margins work in grid as well: `margin-left: auto` pushes an item to the end of its cell.

Self-check

1. Centre one item inside its cell without affecting others. → `place-self: center` on that item.
2. Why does `justify-content: center` do nothing on `grid-template-columns: 1fr 1fr`? → `fr` tracks use up all the space; nothing is left to distribute.
3. What order do values in `place-items: end start` follow? → `align-items: end`, `justify-items: start`.

9.7 Subgrid

The problem it solves

A grid item that is itself a grid has its *own* independent tracks. Three pricing cards side by side each lay out title, text and button, but card 2's longer title pushes its text lower — the rows don't line up across cards. Before subgrid, fixes involved fixed heights or JavaScript.

What it is

`grid-template-rows: subgrid` (or `grid-template-columns: subgrid`) on a grid item that spans several tracks of its parent tells it: **don't create your own tracks on this axis — reuse the parent's tracks I'm sitting on**. The children of the card then take part in the parent's row sizing, so the tallest title in *any* card sets the title row height for *all* cards. Subgrid is defined in CSS Grid Level 2 (the second version of the Grid specification) and has been **Baseline since September 2023** (Firefox 71, Safari 16, Chrome/Edge 117); by 2026 it is "widely available".

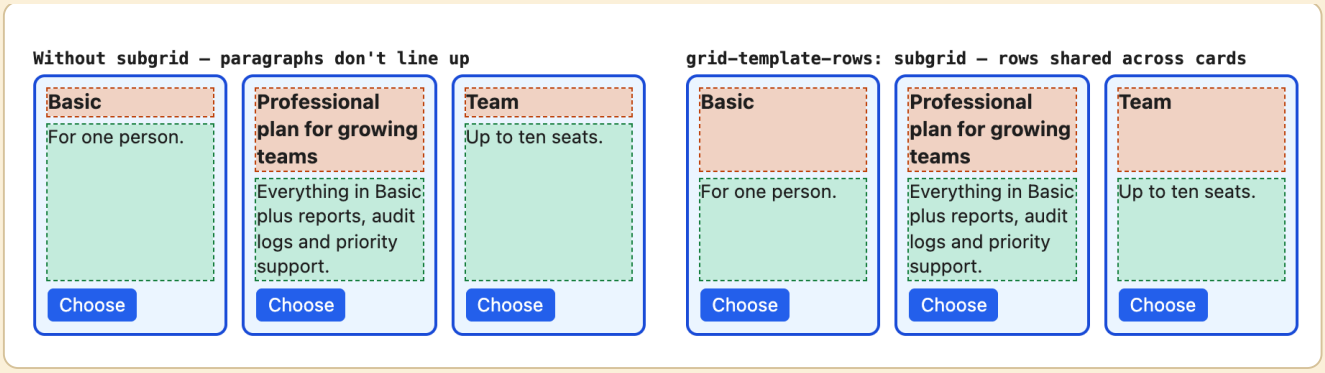


Figure 9.6 — `code/06_subgrid.html`. Orange dashed = title, green dashed = paragraph.

```
.sub      { display: grid; grid-template-columns: repeat(3, 1fr);
            grid-template-rows: auto auto auto; row-gap: 10px; }  /* the parent owns 3 rows */
.sub .card { display: grid; grid-row: span 3;                      /* each card covers all 3 */
            grid-template-rows: subgrid; row-gap: 6px; }           /* ...and reuses them */
```

Measured by `code/06_subgrid.mjs` (top edge of each paragraph and link):

```
plain  <p> tops: 76.9, 114.7, 76.9    link tops: 190.9, 190.9, 190.9
sub    <p> tops: 114.7, 114.7, 114.7  link tops: 190.9, 190.9, 190.9
```

Without subgrid the links happen to align (each card's middle row is `1fr` and the cards stretch to equal height), but the paragraphs don't. With subgrid every row is shared.

Details

- The subgrid item must **span** the parent tracks it wants to share (`grid-row: span 3`).
- A subgrid inherits the parent's gaps but may override them (`row-gap: 6px` above).
- You can subgrid one axis and define your own tracks on the other.
- Line names from the parent are available inside the subgrid.
- Typical uses: card rows, forms where labels and inputs in separate `<fieldset>`s (HTML groups of related form controls, Chapter 5) must align, a list whose items have several aligned columns (a `<ul>` of rows with `grid-template-columns: subgrid`).

Self-check

1. What must a subgrid item do besides `grid-template-rows: subgrid`? → Span the parent's rows it wants to use.
2. Can a subgrid have a different gap? → Yes, it can override the inherited gap.

9.8 Grid vs Flexbox

Question	Flexbox	Grid
Dimensions	One — a row **or** a column	Two — rows **and** columns
Who decides sizes?	**Content-out**: items' content and flex factors	**Layout-in**: the container's track definitions
Items in different rows line up in columns?	No — each flex line is independent	Yes — that's the point
Wrapping	Each line sizes itself; a lonely last item stretches	Items drop into the same columns; last row keeps the column width
Overlapping items	Needs negative margins or positioning	Place two items in the same cell
Gaps	`gap`	`gap`
Reordering	`order`, `*-reverse`	Line/area placement, `order`, `dense`
Typical jobs	Navbars, toolbars, button groups, tags, media objects, centring one thing, "push the last item right"	Page layouts, galleries, card grids, dashboards, form layouts, anything designed on a grid
Aligning content across siblings	Not possible	`subgrid`

Rules of thumb:

1. **Is the layout driven by the content (a row of things of different sizes)?** → Flexbox.
2. **Do things need to line up in both directions, or do you know the track sizes up front?** → Grid.
3. **Use both**: grid for the page and the card gallery, flexbox inside each card's footer to space its buttons. They nest freely.

In the interview

30-second answer: "Flexbox is one-dimensional and content-out: items sit in a row or column and share space based on their content and flex factors. Grid is two-dimensional and layout-in: I define rows and columns and place items into them, so things align across rows. I use grid for page structure and card grids — for example `repeat(auto-fill, minmax(200px, 1fr))` for a responsive gallery — and flexbox for components like navbars and button groups. They're often nested."

- *Can you make a responsive grid without media queries?* → Yes: `repeat(auto-fit | auto-fill, minmax(min(200px, 100%), 1fr))`.
- *Difference between auto-fill and auto-fit?* → Only visible when items don't fill a row: auto-fill keeps empty tracks, auto-fit collapses them so items stretch.
- *Why are my `1fr` columns unequal?* → `1fr` has an `auto` minimum (content size); use `minmax(0, 1fr)`.

9.9 Patterns

Holy grail layout (header, nav, main, aside, footer)

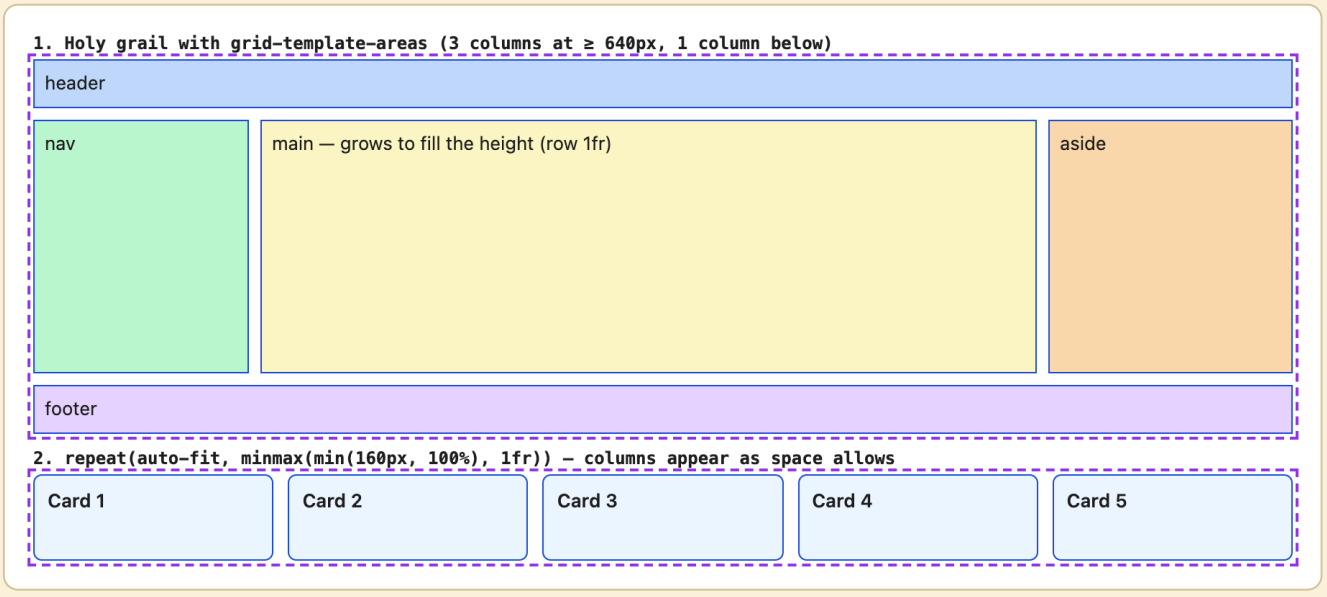
```
/* mobile first: one column, in source order */
.page { display: grid; gap: 8px; min-height: 100dvh;          /* demo uses 260px */
        grid-template-areas: "header" "nav" "main" "aside" "footer";
        grid-template-rows: auto auto 1fr auto auto; }
.page > header { grid-area: header; }  .page > nav    { grid-area: nav; }
.page > main   { grid-area: main; }    .page > aside  { grid-area: aside; }
.page > footer { grid-area: footer; }

/* three columns when there's room */
@media (min-width: 640px) {
  .page { grid-template-columns: 150px 1fr 170px;
          grid-template-rows: auto 1fr auto;
          grid-template-areas:
            "header header header"
            "nav    main   aside"
            "footer footer footer"; }
}
```

The `1fr` row makes `main` take all spare height, so the footer sits at the bottom of the screen (the sticky footer of §8.7, done with grid). Media queries are covered in Chapter 10.

Responsive card grid (RAM)

```
.cards { display: grid; gap: 10px;
         grid-template-columns: repeat(auto-fit, minmax(min(160px, 100%), 1fr)); }
```



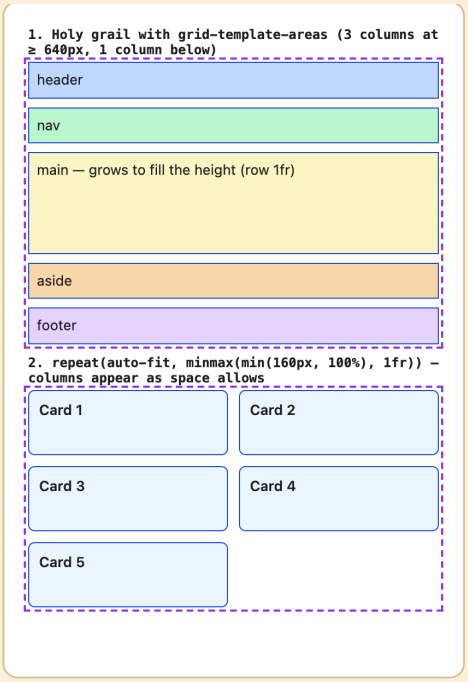


Figure 9.7 — `code/07_patterns.html` screenshotted at 900px and 400px wide.

`code/07_patterns.mjs` loads the page at both widths:

```
viewport 900px
  holy grail: 150px 540px 170px
  cards     : 167.188px 167.203px 167.203px 167.203px 167.188px
viewport 400px
  holy grail: 376px
  cards     : 183px 183px
```

At 900px the container is 876px: 876 − 150 − 170 − 2 × 8 = 540 for `main`; the cards fit floor(886 / 170) = 5 columns. At 400px the media query no longer matches (one 376px column) and only 2 card columns fit. Card 5 keeps the column width on its own row — the thing flex-wrap couldn't do (§8.6).

Other patterns worth knowing

Pattern	Grid recipe
Centre anything	`display: grid; place-items: center`
Overlay / hero caption	Image and caption both `grid-area: 1 / 1`; align the caption with `align-self` / `justify-self`
Full-bleed article	Named lines `[full-start] 1fr [content-start] minmax(0, 60rem) [content-end] 1fr [full-end]` (9.4)
Form with aligned labels	`grid-template-columns: max-content 1fr` on the form, one label + input per row
Sidebar that collapses	`grid-template-columns: fit-content(250px) minmax(0, 1fr)`
12-column design system grid (the column grid a team's shared design rules are built on)	`repeat(12, minmax(0, 1fr))` + `grid-column: span 4` on items

Masonry: not yet

Pinterest-style "masonry" (items packed into columns of uneven height) is being standardised in CSS Grid Level 3 under the name `display: grid-lanes`. As of 2026 it has shipped in Safari (26.4) and is behind flags (hidden experimental settings users must switch on) in Chrome and Firefox — not Baseline. Use CSS multi-column (`columns: 3`, newspaper-style columns) or a JavaScript library, and wrap any experiment in `@supports (display: grid-lanes)` (a feature query: the rules inside apply only if the browser supports that feature).

9.10 Debugging, pitfalls and interview prep

Debugging

All major browsers show a **grid badge** beside grid containers in the Elements/Inspector panel of DevTools (the built-in developer tools, F12). Toggling it overlays line numbers (positive and negative), track sizes, gaps and area names — the same information our red badges draw. Firefox's grid inspector pioneered this and is still the most detailed.

Pitfalls at a glance

Symptom	Cause	Fix
`1fr` columns are unequal / grid overflows on mobile	`1fr` = `minmax(auto, 1fr)`; long words, `<pre>`, images or tables set the minimum	`minmax(0, 1fr)`; `min-width: 0` or `overflow` on the item
Horizontal scrollbar with `%` columns	`%` tracks ignore gaps	Use `fr`
Single auto-fill column overflows narrow screens	Minimum larger than the container	`minmax(min(200px, 100%), 1fr)`
Item lands in an unexpected extra row/column	Placed beyond the explicit grid → implicit tracks; or a typo in an area name (unknown names create implicit lines)	Check line numbers in DevTools; spell area names consistently
`grid-template-areas` ignored	Strings have different cell counts or a non-rectangular area — the whole declaration is invalid	Fix the ASCII art
An image doesn't fill its cell	Items with an aspect ratio (images, video) treat the default alignment as `start`, not `stretch` — checked in Chrome: a 100×50 image in a 200×300 cell stays 100×50	`width: 100%; height: 100%; object-fit: cover` (scale the image to fill the box, cropping the overflow; Chapter 11)
`justify-content` does nothing	`fr` tracks leave no free space	Use fixed tracks, or align items instead
Tab order jumps around	`dense`, placement or `order` moved items visually	Keep DOM order = visual order

In the interview

30-second answer: "Grid is two-dimensional layout. On the container I define tracks with `grid-template-columns` and `-rows` using lengths, `fr` for fractions of leftover space, `minmax`, and `repeat` — `repeat(auto-fill, minmax(200px, 1fr))` gives a responsive grid with no media queries. Items are auto-placed into cells, or placed by line numbers, spans or named `grid-template-areas`. Extra items create implicit tracks sized by `grid-auto-rows`. Alignment uses `justify-*` for the inline axis and `align-*` for the block axis, and `subgrid` lets nested grids share the parent's tracks so cards line up."

Follow-ups:

- *How is `fr` calculated?* → Container size minus fixed tracks and gaps, divided by the total of the `fr` values; tracks whose share is below their minimum are fixed and the rest re-split. 700 − 200 − 40 = 460 → 1fr = 153.33.
- *auto-fill vs auto-fit?* → Same until there are too few items; then auto-fill keeps empty tracks, auto-fit collapses them.
- *How would you build a layout that reorders at a breakpoint?* → Named areas, rewritten in a media query — while keeping DOM order sensible for keyboard and screen-reader users.
- *What is subgrid for?* → Aligning the insides of sibling components (card titles, form labels) to shared tracks.

Self-check

1. `grid-template-columns: repeat(3, 1fr)` overflows on mobile because of a wide table. Two fixes? → `repeat(3, minmax(0, 1fr))` and let the table scroll (`overflow-x: auto` on a wrapper); or fewer columns at small widths.
2. Which property sizes rows that you did not declare? → `grid-auto-rows`.
3. Shortest way to centre a child in both axes? → `display: grid; place-items: center` on the parent.

Chapter 9 TL;DR

- **Grid** = two-dimensional, layout-in. `display: grid` → direct children are items. Vocabulary: lines (1…n+1, and −1 backwards), tracks, cells, areas, gaps; explicit vs implicit grid.
- **Tracks**: px/%/`fr`/`auto`/`min-content`/`max-content`/`fit-content()`/`minmax()`. **`fr` = share of space left after fixed tracks and gaps** (700 − 200 − 40 = 460 → 153.33 / 306.67, verified). `1fr` = `minmax(auto, 1fr)` → use `minmax(0, 1fr)` for truly equal columns.
- **RAM**: `repeat(auto-fill|auto-fit, minmax(min(X, 100%), 1fr))`; n = floor((W + gap) / (X + gap)). auto-fill keeps empty tracks, auto-fit collapses them.
- **Placement**: `grid-column: 1 / 3`, `span 2`, `1 / -1` (full width), `grid-area: row-start / col-start / row-end / col-end`, named lines, and `grid-template-areas` ASCII art (rectangles only, `.` = empty). Items can overlap in one cell.
- **Implicit grid**: `grid-auto-rows/columns` size extra tracks; `grid-auto-flow: row | column | dense` (dense breaks visual/DOM order).
- **Alignment**: `justify-*` = inline axis, `align-*` = block axis; `*-items`/`*-self` inside cells, `*-content` for the whole grid when tracks leave free space; `place-*` shorthands; `place-items: center` centres.
- **Subgrid** (Baseline since Sept 2023): `grid-template-rows: subgrid` on a spanning item reuses the parent's tracks so card internals align.
- **Grid vs Flexbox**: grid for page structure and uniform grids; flexbox for content-sized rows inside components; nest them.
- **Masonry** (`display: grid-lanes`) is Safari-only in 2026 — not Baseline.

Next: Chapter 10 — Responsive Design.

Chapter 10 — Responsive Design

People open the same page on a 360px-wide phone, a tablet turned sideways, a 1440px laptop and a 2560px monitor — and some of them have set a larger default font, zoomed to 200%, switched their system to dark mode (light text on a dark background) or asked it to reduce motion (fewer animations, which can cause dizziness or nausea for some people). **Responsive design** is the practice of building *one* page that works well in all of those situations, instead of a separate "mobile site". Chapters 7–9 gave you the layout tools (flow, flexbox, grid); this chapter is about making those layouts **adapt**.

What you'll learn

#	Topic	You will be able to…
10.1	What responsive design is	name its three ingredients and explain mobile-first (design for phones first, then add for bigger screens)
10.2	The viewport meta tag and viewport units	explain why phones need it, and choose between `vh`, `svh`, `lvh`, `dvh`
10.3	Media queries	write queries with types, features, range syntax and logic, and use them from JavaScript
10.4	Mobile-first in practice	build a page that goes from 1 to 3 columns with two breakpoints (screen widths at which the layout changes)
10.5	User-preference queries	support dark mode, higher contrast and reduced motion
10.6	Container queries	make a component adapt to the space *it* is given, not the screen
10.7	Fluid typography with `clamp()`	derive a fluid font-size formula and keep it accessible
10.8	Responsive images recap	pick the right image with `srcset` / `sizes` / `<picture>` and `image-set()`
10.9	Intrinsic layouts	build layouts that adapt with no breakpoints at all
10.10	Testing responsive design	test with DevTools (the browser's built-in developer tools), real devices, zoom — and automate an overflow check

Every example is a page in `code/`. The screenshots in `img/` were taken in headless Chrome (Chrome driven by a script, with no window) by `code/make_figures.mjs`, and the `NN_*.mjs` scripts print what the browser actually computed (which queries match, resolved grid columns, font sizes). They use the helper `code/measure.mjs` — the same idea as in Chapters 7–9, extended with **viewport width**, **phone emulation** (making desktop Chrome behave like a phone) and **OS-preference emulation** (pretending the operating system has settings such as dark mode or high contrast). Many screenshots are "contact sheets": the *same* page at several widths, side by side, with the width in red above each one.

10.1 What responsive design is

What it is

A **responsive** page rearranges and resizes itself to fit the **viewport** (the visible area of the browser window) and the user's settings. The term was coined by Ethan Marcotte in 2010, with three ingredients:

Ingredient	Meaning	Today's tools
Fluid grids	Widths in relative units (%, `fr`, `vw`) instead of fixed pixels	Flexbox, Grid, `min()` / `max()` / `clamp()`
Flexible media	Images and videos shrink with their container	`max-width: 100%; height: auto`, `srcset`, `aspect-ratio`
Media queries	CSS rules that apply only under conditions (e.g. "at least 640px wide")	`@media`, and now `@container`

Analogy: water takes the shape of whatever glass you pour it into. A fixed-width page is an ice cube — it fits only one glass.

Why it matters

- Most web traffic worldwide is from phones; a page that needs sideways scrolling or pinch-zoom (spreading two fingers to enlarge) on a phone loses readers.
- Search engines index the mobile version of a page first ("mobile-first indexing" at Google), so a broken mobile layout hurts discovery (Chapter 14).
- Accessibility law and WCAG 2.2 (the Web Content Accessibility Guidelines) require content to **reflow** (rewrap into a narrow column without sideways scrolling) at 320 CSS pixels wide and survive 200% text zoom (§10.10, Chapter 3).

Mobile-first vs desktop-first

	Mobile-first	Desktop-first
Base CSS (no query)	The phone layout	The desktop layout
Queries use	`min-width` / `width >=` — *add* features as space grows	`max-width` / `width <` — *undo* features as space shrinks
Small devices download/parse (read and process)	only the rules they need, plus harmless overrides	desktop rules, then overrides to undo them
Design pressure	Forces you to decide what matters most first	Tends to cram a big design into a small screen

Mobile-first is the common default today because the single-column phone layout is the simplest — normal flow already stacks things — and every larger layout is an *enhancement*. That is **progressive enhancement**: start with something that works everywhere, then add.

Responsive vs adaptive

An **adaptive** site serves (or switches between) a few fixed-width layouts; a **responsive** one is fluid *between* breakpoints. Modern practice is responsive, often with fewer breakpoints thanks to intrinsic layouts (10.9) and container queries (10.6).

Self-check

1. Name the three ingredients of responsive design. → Fluid grids, flexible media, media queries.
2. In mobile-first CSS, which kind of width query do you mostly write? → `min-width` (or `width >= …`).

10.2 The viewport meta tag and viewport units

What it is

```
<meta name="viewport" content="width=device-width, initial-scale=1">
```

This line in `<head>` (Chapter 2) tells a mobile browser: "lay the page out at the device's width in CSS pixels, at 100% zoom".

The problem it solves

When the iPhone launched (2007), almost every site was designed for ~1000px-wide desktops. So mobile browsers **pretend** to be wide: without the tag they lay the page out in a **layout viewport** of about **980px**, then zoom out so it fits the screen. Old sites looked like tiny thumbnails you could pinch-zoom — but a responsive page would never see its "phone" media queries fire, because the browser claims it is 980px wide.

A **CSS pixel** is not a hardware pixel. A phone with a **device pixel ratio** (DPR) of 3 draws each CSS pixel with 3×3 physical pixels; `width=device-width` means "as many CSS pixels as the screen is wide", e.g. 390 on a typical modern phone.

Proof

`code/01_no_viewport.html` and `code/01_viewport.html` are the same page except for the tag. `code/01_viewport.mjs` opens both on an emulated 400px phone (Puppeteer — a Node.js library that controls Chrome — has an `isMobile: true` option that makes Chrome honour the meta tag like a real phone):

```
01_no_viewport.html  layout 980px, visible 980px, scale 0.408, font-size 16px
01_viewport.html     layout 400px, visible 400px, scale 1, font-size 16px
```

Without the tag, the page is laid out at 980px and scaled by 400 / 980 ≈ 0.408 — so the "16px" text is really about **6.5px** on screen.

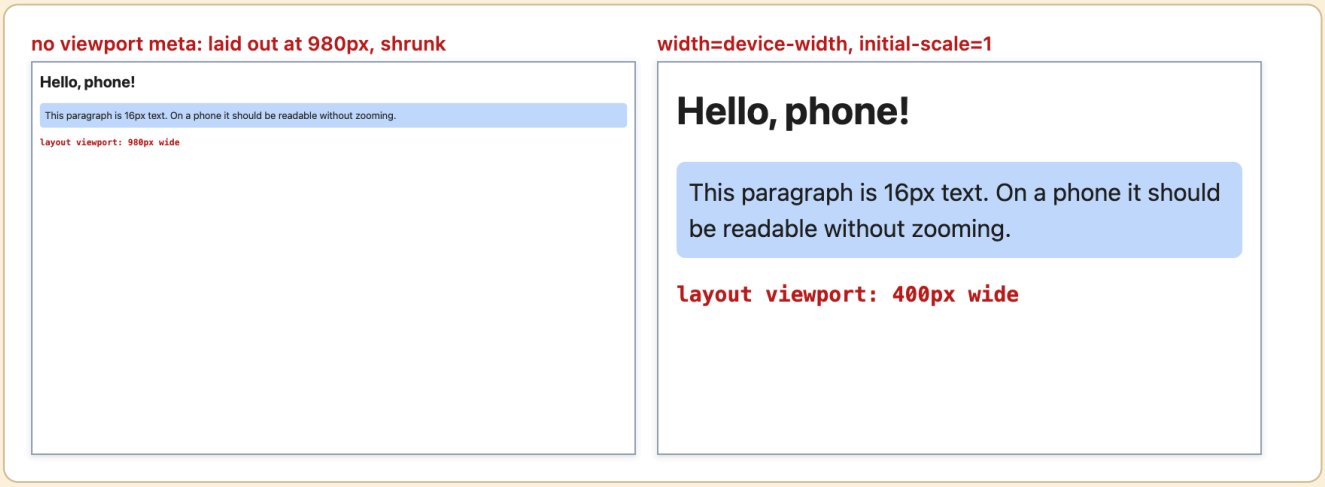


Figure 10.1 — The same page on a 400px phone, without and with the viewport meta tag.

The values

Key	Typical value	Notes
`width`	`device-width`	Layout viewport = screen width in CSS px. A number (`width=600`) is possible but rarely right.
`initial-scale`	`1`	Start at 100% zoom. Also stops iOS from changing zoom on rotation.
`maximum-scale`, `user-scalable=no`	**don't**	Blocking zoom fails WCAG 1.4.4 (Resize text). iOS Safari has ignored `user-scalable=no` since iOS 10 for this reason.
`viewport-fit`	`cover`	For notched phones (screens with a camera cut-out); then use `env(safe-area-inset-*)` padding (values the browser supplies for the unsafe edges).

Viewport units

`vw` and `vh` are 1% of the viewport's width and height (Chapter 6 introduced the units). On phones the viewport height **changes** as the address bar slides in and out, so CSS Values 4 added three variants (Baseline since December 2022):

Unit	1% of…	Behaviour on a phone
`svh` / `svw` (small)	the viewport with browser toolbars **shown**	never too tall; may leave a gap when toolbars hide
`lvh` / `lvw` (large)	the viewport with toolbars **hidden**	may be hidden behind toolbars at first
`dvh` / `dvw` (dynamic)	the viewport **right now**	follows the toolbars; can cause relayout (the browser recalculating the layout) while scrolling
`vh` / `vw` (classic)	in practice the **large** viewport on mobile	why `height: 100vh` "overflows" on phones

Rules of thumb: `min-height: 100svh` or `100dvh` for full-screen heroes (the large banner section at the top of a page) and the sticky footer of §8.7; `vw` for fluid type (10.7), but never alone — it ignores zoom settings. On desktop all four heights are equal. Also available: `vi`/`vb` (inline/block direction) and `vmin`/`vmax`.

Common mistakes

- Forgetting the tag: every phone breakpoint silently fails.
- Disabling zoom "to feel like an app" — an accessibility failure.
- `width: 100vw` on a page with a vertical scrollbar: on desktop `100vw` *includes* the scrollbar, so the element is ~15px too wide and causes sideways scrolling. Use `width: 100%`.

Self-check

1. Without the tag, how wide does a phone lay the page out? → About 980 CSS px (then zooms out).
2. Which unit should a full-height mobile hero use to avoid being covered by the address bar? → `svh` (always fits) or `dvh` (tracks the toolbar).
3. Why avoid `100vw` for full-width elements? → It includes the scrollbar width on desktop and causes horizontal overflow.

10.3 Media queries

What it is

A **media query** is a condition about the device or environment. Rules inside `@media (condition) { … }` apply only while the condition is true, and the browser re-evaluates it live (e.g. when you rotate a phone or resize a window).

```
@media screen and (width >= 40em) { .cards { grid-template-columns: repeat(2, 1fr); } }
```

Media queries don't add specificity (the score that decides which competing CSS rule wins); inside them normal cascade (CSS's conflict-resolution) rules apply (Chapter 6) — which is why mobile-first queries are written **after** the base rules they override.

Where you can use them

Place	Example
CSS at-rule (a CSS statement starting with `@`)	`@media (width >= 40em) { … }`
`<link>`	`<link rel="stylesheet" href="print.css" media="print">` — still downloaded, but not render-blocking (it doesn't delay the first paint) when it doesn't match
`@import`	`@import url(wide.css) (width >= 64em);`
`<source>` in `<picture>`	`<source media="(width >= 800px)" srcset="wide.jpg">` (Chapter 4)
JavaScript	`matchMedia('(width >= 64em)')`

Anatomy

1. **Media type** (optional): `all` (default), `screen`, `print`. Other types (`tv`, `handheld`…) are deprecated (officially retired; don't use them).
2. **Media features** in parentheses: `(width >= 40em)`, `(orientation: landscape)`, `(hover: hover)`.
3. **Logic**: `and`; a comma means **or**; `not` negates a whole query (or a condition in Level 4 syntax: `not (hover: hover)`); `only` is a legacy guard for very old browsers — harmless, no longer needed.

Media features you'll actually use

Feature	Values / example	Use
`width` (+ `min-` / `max-`)	`(width >= 40em)`	Layout breakpoints (the viewport width)
`height`	`(height < 500px)`	Short landscape phones: shrink sticky headers
`orientation`	`portrait` | `landscape`	Rarely needed — width usually says more
`aspect-ratio`	`(aspect-ratio >= 16/9)`	Full-screen media layouts
`hover` / `any-hover`	`hover` | `none`	Only show hover-reveal UI when the *primary* (or *any*) pointer (mouse, finger, pen) can hover
`pointer` / `any-pointer`	`fine` | `coarse` | `none`	Larger tap targets (the area you can touch to activate something) for touch (`coarse`)
`resolution`	`(resolution >= 2x)`	High-DPR (device pixel ratio, explained in §10.2) assets (prefer `srcset` / `image-set()`)
`prefers-color-scheme`	`light` | `dark`	Dark mode (10.5)
`prefers-reduced-motion`	`reduce` | `no-preference`	Tone down animation (10.5, Chapter 12)
`prefers-contrast`	`more` | `less` | `custom` | `no-preference`	Stronger borders/colours
`forced-colors`	`active` | `none`	Windows High Contrast / contrast themes
`print` (type)	`@media print`	Hide nav, show link URLs, black on white

Range syntax (Media Queries Level 4)

The old style uses `min-` / `max-` prefixes; the new **range syntax** uses comparison operators. It is Baseline since March 2023 and *widely available* since September 2025 (Chrome 104, Firefox 102, Safari 16.4).

Old	Range syntax
`(min-width: 640px)`	`(width >= 640px)`
`(max-width: 639.98px)`	`(width < 640px)`
`(min-width: 640px) and (max-width: 1023.98px)`	`(640px <= width < 1024px)`

Why the `.98`? With `max-width: 639px` and `min-width: 640px`, a window 639.5px wide (possible with zoom or fractional DPR) matches **neither**. Range syntax's `<` removes that gap.

`em` or `px` for breakpoints?

In media queries, `em` and `rem` are always relative to the **browser's default font size** (normally 16px), *not* your `html { font-size }`. So `40em` = 640px at defaults, but if a user sets 20px as their default, `40em` becomes 800px and your layout switches to one column earlier — exactly what a user with large text needs. Both units are fine; `em` ties breakpoints to text size.

Choosing breakpoints

- Let the **content** decide: widen the window until the layout looks wrong, and add a breakpoint *there*. Device-based lists go stale.
- For reference, popular frameworks (ready-made CSS libraries) use: Tailwind `sm 640 · md 768 · lg 1024 · xl 1280 · 2xl 1536`, Bootstrap 5 `576 · 768 · 992 · 1200 · 1400`.
- Fewer is better: two or three layout breakpoints, plus intrinsic techniques (10.9) and container queries (10.6) for components.

Media queries in JavaScript

`window.matchMedia(query)` returns a `MediaQueryList` (an object describing that query) with `.matches` and a `change` event (a notification your code can listen for) — useful for behaviour that CSS can't express (swap a

component, pause a carousel — a rotating slideshow of items). Don't listen to `resize` and compare `innerWidth` yourself; `change` fires only when the answer flips.

```
const desktop = matchMedia('(width >= 64em)');
desktop.addEventListener('change', e => console.log(e.matches ? 'desktop' : 'not desktop'));
```

The last line of 10.4's output shows this firing: resizing 400 → 900 → 1100 → 1300 → 700 fired exactly **two** events. (`addListener()` is the deprecated older form.)

Common mistakes

- Desktop-first `max-width` queries mixed with mobile-first `min-width` ones — overlapping rules and gaps. Pick one direction.
- Putting a media query **before** the base rule it should override: equal specificity, so the later base rule wins.
- Using `(hover: hover)` alone to guess "desktop": laptops with touchscreens and phones with mice exist. Design for both.
- Breakpoints for specific devices ("iPhone 15") instead of the content.

Self-check

1. Write "between 600px and 900px, excluding 900" in range syntax. → `@media (600px <= width < 900px)`.
2. Does `em` in a media query follow `html { font-size: 62.5% }`? → No — it uses the browser's initial font size.
3. How does JS find out when a breakpoint is crossed? → `matchMedia(q).addEventListener('change', …)`.

10.4 Mobile-first in practice

The page

`code/02_mobile_first.html` is a small travel page. The base styles (no query) are the phone layout; two `min-width`-style queries add columns. The red badge at the top shows which rule set is active (it is a `::after` pseudo-element — a CSS-generated box — whose `content` each query overrides).

```
/* Base = phone: single column; normal flow + grid gap does the stacking */
.layout { display: grid; gap: 14px; padding: 14px; }
.cards  { display: grid; gap: 10px; }

@media (width >= 40em) {                      /* 640px: tablets, small laptops */
  header { display: flex; justify-content: space-between; align-items: center; }
  .cards { grid-template-columns: repeat(2, 1fr); }
}
@media (width >= 64em) {                      /* 1024px: sidebar appears */
  .layout { grid-template-columns: 1fr 240px; max-width: 1200px; margin-inline: auto; }
  .cards  { grid-template-columns: repeat(3, 1fr); }
}
```

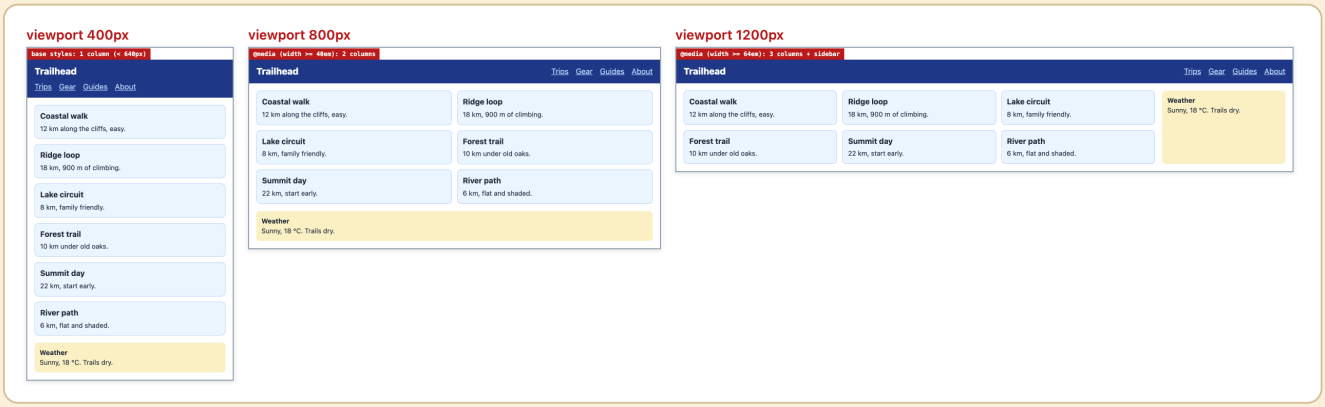


Figure 10.2 — One page, three viewport widths. Nothing in the HTML changes; only which rules apply.

Output of `code/02_media_queries.mjs` — which queries match and what the browser resolved the card grid to:

```
 400px  (width >= 40em) no   (width >= 64em) no   cards: 372px
 800px  (width >= 40em) yes  (width >= 64em) no   cards: 381px 381px
1200px  (width >= 40em) yes  (width >= 64em) yes  cards: 299.328px 299.328px 299.344px
matchMedia events: [ 'change -> desktop at 1100px', 'change -> not desktop at 700px' ]
```

Check by hand at 1200px: `.layout` is `max-width: 1200px` minus 2 × 14px padding = 1172px; minus the 240px sidebar and a 14px gap = 918px for `.cards`; minus two 10px gaps = 898px; ÷ 3 = **299.33px** per card column. ✓

Why it works

- **Normal flow** is already mobile-first: block elements stack. The base CSS mostly styles, it barely lays out.
- Each query **adds** a layout; nothing has to be "undone".
- The **DOM order** (header, cards, aside — the order in the HTML) is the reading order at every width, so keyboard and screen-reader (software that reads pages aloud) users get the same sequence (Chapter 3). Reordering visually with `order` or grid placement at a breakpoint breaks that — avoid it for meaningful content.

Responsive navigation (common pattern)

At phone width a long nav usually becomes a **menu button** that shows/hides the list. Accessible recipe: a real `<button aria-expanded="false" aria-controls="menu">` (ARIA attributes that tell screen readers whether the menu is open and which element it controls) that toggles the list (a few lines of JS), or the Popover API (`popover` attribute, Baseline since January 2025) for a no-JS disclosure (show/hide widget). The flex-wrap nav in the demo is the simplest option when there are only a few links.

Responsive tables

Wide data tables can't reflow into one column without losing their meaning. Wrap them in a scroll container (a box that scrolls its own content): `<div class="table-wrap" tabindex="0" role="region" aria-label="Prices">` with `.table-wrap { overflow-x: auto; }` — WCAG allows two-dimensional scrolling for data tables (Chapter 5).

Self-check

1. At 800px, which of the two queries match? → Only `(width >= 40em)`.
2. Why write the queries after the base rules? → Same specificity, so the later rule wins the cascade.

10.5 User-preference media queries

What it is

Operating systems let people choose a dark theme, reduce motion, or increase contrast. The **user-preference media features** (Media Queries Level 5) expose those choices to CSS. All of the following are Baseline:

Feature	Baseline	Typical use
`prefers-color-scheme: dark`	widely available (2020)	Dark theme
`prefers-reduced-motion: reduce`	widely available (2020)	Remove or soften animation — full treatment in Chapter 12
`prefers-contrast: more`	widely available (2022)	Solid borders, stronger colours
`forced-colors: active`	widely available (2022)	Windows contrast themes replace your colours; restore borders on custom controls, use system colours like `CanvasText`
`prefers-reduced-transparency`	**not** Baseline (Chromium only)	Remove frosted-glass effects
`prefers-reduced-data`	not supported anywhere yet	—

Dark mode — the essentials

1. **Declare support**: `<meta name="color-scheme" content="light dark">` in the head and/or `:root { color-scheme: light dark; }`. The browser then themes its **own** UI (user interface) — default page background, form controls, scrollbars — and avoids a white flash in dark mode. (`color-scheme`: Baseline 2022.)
2. **Colour tokens** (named colour values such as `--bg`) as custom properties (CSS variables, §6.9), light values first.
3. **Override the tokens** inside `@media (prefers-color-scheme: dark)`. Components only ever use the tokens.
4. For one-off values, `light-dark(lightValue, darkValue)` picks based on the element's **used colour scheme** (light or dark, as finally decided by `color-scheme` and the user's setting) — no media query. It needs `color-scheme: light dark` to be set, and it is Baseline since May 2024 (newly available).

```
:root { color-scheme: light dark;
        --bg: #ffffff; --text: #1f2937; --card: #eff6ff; --accent: #1d4ed8; --border: #bfdbfe; }
@media (prefers-color-scheme: dark) {
  :root { --bg: #0f172a; --text: #e2e8f0; --card: #1e293b; --accent: #93c5fd; --border: #334155; }
}
@media (prefers-contrast: more) { :root { --border: currentColor; } }

body  { background: var(--bg); color: var(--text); }
.note { color: light-dark(#92400e, #fcd34d); }
```

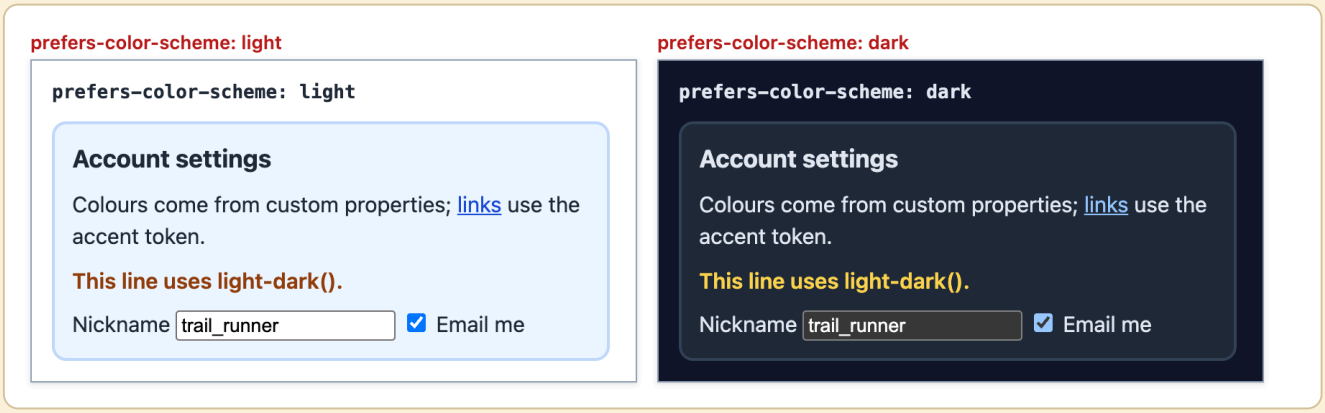


Figure 10.3 — `code/03_preferences.html` under emulated `prefers-color-scheme: light` and `dark`. Notice the text input and checkbox: nobody styled them — `color-scheme` made the browser draw its dark versions. The dark screenshot alone can be reproduced with `node tools_web_shot.mjs chapters/10-responsive-design/code/03_preferences.html out.png 420 --dark`.

`code/03_preferences.mjs` reads the computed colours under three emulated settings:

```
light (default)        bg rgb(255, 255, 255)  text rgb(31, 41, 55)  card border rgb(191, 219, 254)  note rgb(146, 64, 14)
<input> bg rgb(255, 255, 255)
dark                   bg rgb(15, 23, 42)  text rgb(226, 232, 240)  card border rgb(51, 65, 85)  note rgb(252, 211, 77)
<input> bg rgb(59, 59, 59)
light + more contrast  bg rgb(255, 255, 255)  text rgb(31, 41, 55)  card border rgb(31, 41, 55)  note rgb(146, 64, 14)
<input> bg rgb(255, 255, 255)
```

With more contrast, the card border became the text colour (`currentColor`). The input background `rgb(59, 59, 59)` in dark mode is Chrome's built-in dark control colour.

A practical note from making these figures: headless Chrome follows the **computer's** theme when nothing is emulated. On a Mac in dark mode, a page with `color-scheme: light dark` screenshots dark unless you emulate `light` explicitly — `measure.mjs` does exactly that.

Letting users override

Many sites add a theme toggle that sets `data-theme="dark"` on `<html>` and stores the choice. Then tokens are defined for `[data-theme="dark"]` too, and the media query applies only when no choice was made (`:root:not([data-theme="light"])`). That architecture — plus `color-scheme` per theme — is covered in Chapter 13.

Designing a dark theme well

- Don't invert: use dark greys/navies (pure `#000` with pure white text causes halation — a glowing, blurry halo around letters — for many readers), lighten accent colours, and re-check contrast ≥ 4.5:1 (Chapter 11).

- Shadows barely show on dark backgrounds — express elevation (how "raised" a surface looks) with slightly lighter surfaces.
- Images: dim very bright photos slightly (`filter: brightness(.9)`) or supply dark versions with `<picture> <source media="(prefers-color-scheme: dark)" …>`.

Reduced motion in one rule

```
@media (prefers-reduced-motion: reduce) {
  *, *::before, *::after { animation-duration: 0.01ms !important; animation-iteration-count: 1 !important;
                           transition-duration: 0.01ms !important; scroll-behavior: auto !important; }
}
```

This blunt reset is common; §12.9 explains a gentler "motion only when allowed" pattern and what WCAG requires.

Self-check

1. What does `color-scheme: light dark` change even if you write no other dark CSS? → The browser's defaults: canvas background, text colour, form controls, scrollbars.
2. When does `light-dark()` pick its second value? → When the element's used colour scheme is dark (requires `color-scheme` to allow dark).
3. What should a site do when `forced-colors: active`? → Let the system colours win; make sure custom controls still have visible borders/focus indicators (e.g. with system colours like `ButtonText`), and don't rely on background images for meaning.

10.6 Container queries

What it is

A **container query** (`@container`) styles an element based on the size of an **ancestor container** (an element somewhere above it in the HTML tree) instead of the viewport. It is Baseline since February 2023 and **widely available** since August 2025 (Chrome 105, Firefox 110, Safari 16).

The problem it solves

Components are reused in different places: a product card sits in a wide main column, a narrow sidebar and a two-up grid — all on the same screen. A media query only knows the *viewport* width, so it can't tell those three cards apart. You used to add modifier classes (extra classes that switch a component's variant, e.g. `.card--compact`) by hand. With container queries the card adapts to the space **it is given**, wherever it is placed.

Analogy: media queries ask "how big is the room?"; container queries ask "how big is the box I'm sitting in?"

How it works

1. Mark an ancestor as a **size container**: `container-type: inline-size` (width only — the usual choice) or `size` (both axes, but then the container can't get its height from its content, so you must give it one). The shorthand `container: name / type` also sets an optional **name**.
2. Write rules inside `@container [name] (condition) { … }`. They apply to **descendants** of the matching container.
3. The browser finds, for each element, the **nearest ancestor** container (with that name, if given) and evaluates the condition against **its** content-box size.
4. A container can't style *itself* with its own query — that would be circular (its size would depend on styles that depend on its size). This is why the demo puts the container on a wrapper (`.card-slot`) and styles the `.product` inside.

```
.card-slot { container: card / inline-size; }       /* name "card", measured inline */

.product    { display: grid; gap: 8px; }             /* base: picture above text */
.product h3 { font-size: clamp(15px, 3cqi + 8px, 24px); }

@container card (width >= 400px) { .product { grid-template-columns: 160px 1fr; } }
@container card (width >= 600px) { .product { grid-template-columns: 240px 1fr; padding: 16px; } }
```

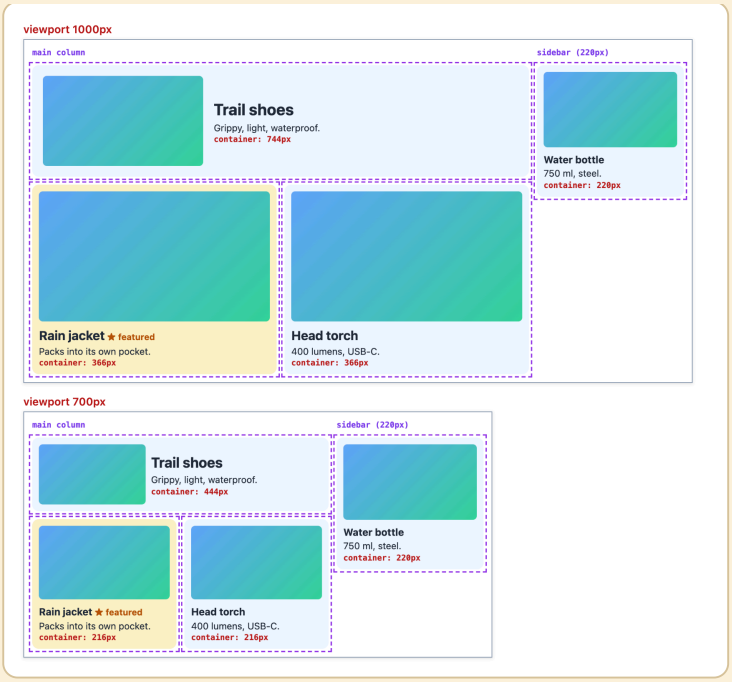


Figure 10.4 — `code/04_container_queries.html`. Dashed purple = the containers; each card prints its container width in red. Same component, same viewport, three different layouts.

Output of `code/04_container_queries.mjs`:

```
viewport 1000px
  Trail shoes   container 744px  columns 240px 456px    h3 24px      ::after none
  Rain jacket   container 366px  columns 346px          h3 18.98px   ::after " ★ featured"
  Head torch    container 366px  columns 346px          h3 18.98px   ::after none
  Water bottle  container 220px  columns 200px          h3 15px      ::after none
viewport 700px
  Trail shoes   container 444px  columns 160px 256px    h3 21.32px   ::after none
  Rain jacket   container 216px  columns 196px          h3 15px      ::after " ★ featured"
  Head torch    container 216px  columns 196px          h3 15px      ::after none
  Water bottle  container 220px  columns 200px          h3 15px      ::after none
```

The sidebar card (220px container) looks identical at both viewport widths, while the "Trail shoes" card switches from the ≥600px layout to the ≥400px layout. A media query could not do this.

Container query units

Inside a container you can size things relative to it:

Unit	1% of the query container's…
`cqw` / `cqh`	width / height
`cqi` / `cqb`	inline size / block size (writing-mode aware — prefer these)
`cqmin` / `cqmax`	smaller / larger of the two

If no suitable container exists, they fall back to the **small viewport** units (`svw` / `svh` …). Check the heading sizes: `clamp(15px, 3cqi + 8px, 24px)` in a 366px container = 3 × 3.66 + 8 = **18.98px** ✓; in a 220px container = 14.6px → clamped up to **15px** ✓; in 744px = 30.3px → capped at **24px** ✓.

Style queries

`@container style(--featured: yes)` tests a **custom property's value** on the container instead of its size. Every element is a style container by default, so no `container-type` is needed. In the demo, `.featured { --featured: yes; }` turns the "Rain jacket" card yellow with a star — a design-token (named design value) switch without a modifier class on the component. Status: **Baseline since May 2026** (newly available: Chrome 111, Safari 18, Firefox 151), and only for **custom properties**; querying ordinary properties (e.g. `style(display: flex)`) is not supported. Treat it as an enhancement for a while.

Other container queries (not Baseline)

`scroll-state()` queries — "is this sticky header stuck?", "is this carousel item snapped?" — are Chromium-only (only browsers built on Google's engine, Chrome 133+). Leave them out of anything that must work everywhere.

Container queries vs media queries

Use a **media query** for…	Use a **container query** for…
Page-level layout (how many columns the page has)	Components that appear in different-sized slots
User preferences (`prefers-*`), print, pointer type	Adapting a card/widget's internal layout
Anything about the device	Sizing text/spacing relative to the component (`cqi`)

They work together: media queries shape the page, container queries let each piece adapt to the space the page gave it.

Common mistakes

- Putting `container-type` on the element you want to restyle — it can only restyle its **descendants**.
- `container-type: size` on an element without an explicit height: it collapses to 0 height, because size containment (the browser treating the box as if it were empty when sizing it) ignores its content.
- Forgetting that `inline-size` containment means the container's **width** can't depend on its children — a `container-type: inline-size` element inside a shrink-to-fit parent (one whose width comes from its content: a flex item with `flex: none`, an inline-block, a float) may collapse. Give it a width from outside (grid track, block flow, `flex: 1`).
- Naming: an unnamed `@container (width > 400px)` matches the **nearest** size container of any name, which might not be the one you meant. Name containers in design systems (a team's shared library of components).

Fallback for older browsers

```
@supports not (container-type: inline-size) { .product { /* a sensible single layout */ } }
```

`@supports (feature: value)` is a **feature query**: its rules apply only if the browser understands that declaration. Container queries are widely available now, so this is only needed for old devices.

In the interview

30-second answer: "Media queries respond to the viewport; container queries respond to the size of an ancestor that I've declared a container with `container-type: inline-size`. That makes components portable: the same card can be stacked in a sidebar and side-by-side in the main column on the same screen. The container can't query itself, so I put it on a wrapper, and I use `cqi` units for sizes relative to it. Size queries are Baseline since 2023; style queries on custom properties became Baseline in 2026."

Follow-ups:

- *Why can't the container use `container-type: size` by default?* → Size containment means its size can't depend on its content; for height that would usually collapse it to 0. `inline-size` only fixes the width dependency.
- *What happens to `cqi` if there's no container?* → It falls back to small-viewport units (`svi`).
- *Would you drop media queries?* → No: page layout and user preferences still belong to media queries.

Self-check

1. Where does `container-type` go: on the card or its wrapper? → On the wrapper (an ancestor); the card's styles come from queries about the wrapper.
2. A container is 500px. What is `10cqi`? → 50px.
3. What does `@container style(--theme: dark)` test? → Whether the nearest container's computed `--theme` custom property equals `dark`.

10.7 Fluid typography with `clamp()`

What it is

Fluid typography makes font sizes (and often spacing) change **smoothly** with the viewport between a minimum and a maximum, instead of jumping at breakpoints. The tool is `clamp(MIN, PREFERRED, MAX)` — Baseline since 2020 (now widely available), along with `min()` and `max()`:

- `clamp(MIN, VAL, MAX)` = `max(MIN, min(VAL, MAX))`: use `VAL`, but never below `MIN` or above `MAX`.

How to derive the formula

Goal: body text **16px at a 320px viewport** growing to **24px at a 1200px viewport**.

1. **Slope** = change in size ÷ change in viewport = (24 − 16) / (1200 − 320) = 8 / 880 = 0.00909. Since `1vw` = 1% of the viewport, that is **0.909vw**.
2. **Intercept** (the size at a 0px viewport) = 16 − 0.00909 × 320 = 13.09px = **0.818rem**.
3. Preferred value = `0.818rem + 0.909vw`, clamped to `1rem … 1.5rem`:

```
:root {
  --step-0: clamp(1rem, 0.818rem + 0.909vw, 1.5rem);      /* 16px → 24px  */
  --step-3: clamp(1.75rem, 1.296rem + 2.273vw, 3rem);     /* 28px → 48px  */
  --space:  clamp(0.75rem, 0.341rem + 2.045vw, 1.875rem); /* 12px → 30px  */
}
p  { font-size: var(--step-0); }
h1 { font-size: var(--step-3); }
```

`code/05_fluid_type.mjs` compares Chrome's computed sizes with the straight-line formula:

```
slope = 0.909vw, intercept = 13.091px = 0.818rem
  280px  p 16.00px (by hand 16.00)   h1 28.00px
  320px  p 16.00px (by hand 16.00)   h1 28.01px
  600px  p 18.54px (by hand 18.55)   h1 34.37px
  900px  p 21.27px (by hand 21.27)   h1 41.19px
 1200px  p 24.00px (by hand 24.00)   h1 48.00px
 1600px  p 24.00px (by hand 24.00)   h1 48.00px
default font 20px, viewport 1000px, rem + vw (ours)  -> 25.45px
default font 20px, viewport 1000px, vw only (bad)    -> 16px
```

The 0.01px differences come from rounding the CSS coefficients to three decimals.

Figure 10.5 — `code/05_fluid_type.html` at three widths. Each page prints its own computed sizes.

Why the `rem` part is essential (accessibility)

The last two output lines emulate a user who changed the browser's **default font size** to 20px. The `rem + vw` formula grows to 25.45px (0.818 × 20 + 9.09). A pure `vw` size stays at **16px** — it ignores the user's setting, and it also doesn't grow with **page zoom** in the way users expect. WCAG 2.2 SC 1.4.4 (Resize text) requires text to scale to 200%; keep a `rem` term in the preferred value, keep `MIN` and `MAX` in `rem`, and keep the ratio MAX/MIN at or below about 2.5 so that zooming still enlarges text meaningfully.

Type scales

Designers usually pick a **modular scale** (a set of font sizes related by one multiplier) — each step = previous × ratio (e.g. 1.2 "minor third" on small screens, 1.333 "perfect fourth" on large). Fluid type interpolates (fills in the in-between values along a straight line for) **each step** between its small-screen and large-screen size, which is what `--step-0 … --step-5` token sets from generators such as Utopia (a free online fluid-type calculator) do. The same interpolation works for spacing (`--space`), making padding and gaps breathe with the screen.

Common mistakes

- `font-size: 4vw` alone — unreadable on phones, huge on monitors, ignores user settings.
- Mixing units in the wrong places: `clamp(16px, …, 24px)` ignores the user's default for the limits; prefer `rem`.
- Forgetting `line-height`: large headings need a smaller unitless line-height (a plain number such as 1.2, meaning 1.2 × the font size) (1.1–1.2), body text ~1.5 (Chapter 11).

Self-check

1. Derive the preferred value for 18px at 400px → 22px at 1400px. → Slope 4/1000 = 0.4vw; intercept 18 − 1.6 = 16.4px = 1.025rem → `clamp(1.125rem, 1.025rem + 0.4vw, 1.375rem)`.
2. Why not `clamp(1rem, 2vw, 1.5rem)`? → It works, but the middle term has no `rem` part, so between the limits it ignores the user's font-size setting.

10.8 Responsive images recap (and `image-set()`)

Chapter 4 §4.3 covers responsive images in depth. The short version:

Need	Tool
Image never overflows its container	`img { max-width: 100%; height: auto; }` + `width` / `height` attributes (reserves space → no layout shift, i.e. content doesn't jump when the image arrives)
Same picture, different file sizes; browser picks by layout width and DPR	`<img srcset="a-400.jpg 400w, a-800.jpg 800w" sizes="(width >= 64em) 33vw, 100vw">`
Same size on screen, sharper on high-DPR screens	`srcset="logo.png 1x, logo@2x.png 2x"`
Different crop at different widths (**art direction**)	`<picture><source media="(width >= 800px)" srcset="wide.jpg"><img src="square.jpg" alt="…"></picture>`
Newer format with fallback	`<picture><source type="image/avif" srcset="a.avif"><img src="a.jpg" alt="…"></picture>`
Keep a box's shape before content loads	`aspect-ratio: 16 / 9`

`image-set()` for CSS background images

`srcset` only works on `<img>`. For **background images**, CSS has `image-set()` — Baseline since September 2023, widely available since March 2026 (older Safari needed `-webkit-image-set()`):

```
.hero {
  background-image: url("media/hills-400.jpg");                 /* fallback */
  background-image: image-set("media/hills-400.jpg" 1x,
                              "media/hills-800.jpg" 2x);
  background-size: cover;
}
```

Candidates can also be chosen by format, e.g. `image-set("a.avif" type("image/avif"), "a.jpg" type("image/jpeg"))`. `code/06_image_set.mjs` loads `code/06_image_set.html` at three pixel ratios and logs the network:

```
devicePixelRatio 1: downloaded hills-400.jpg
devicePixelRatio 2: downloaded hills-800.jpg
devicePixelRatio 3: downloaded hills-800.jpg
```

Only **one** file is downloaded per screen — the whole point. A background image is decoration: if the picture carries meaning, use an `<img>` with `alt` (the demo adds `role="img"` and `aria-label` only because it is a demo of a decorative technique).

10.9 Intrinsic layouts — responsive without breakpoints

What it is

Intrinsic design (a term from Jen Simmons, a CSS designer and advocate) means letting the layout algorithms (the browser's built-in sizing rules) respond to the available space themselves, using `min()`, `max()`, `clamp()`, `flex-wrap`, `auto-fit` and `minmax()`. Fewer breakpoints means fewer "in-between" widths where things look odd, and components that work in any container.

Three patterns

`code/07_intrinsic.html` — no `@media` anywhere:

```
/* 1. Readable centred column: at most 60ch, and never touching the screen edges */
.wrapper { width: min(100% - 2rem, 60ch); margin-inline: auto; }

/* 2. Sidebar that wraps when the content would get narrower than 60% */
.with-sidebar          { display: flex; flex-wrap: wrap; gap: 10px; }
.with-sidebar > .side  { flex: 1 1 150px; }                    /* wants ~150px */
.with-sidebar > .content { flex: 999 1 0; min-inline-size: 60%; } /* takes the rest */

/* 3. Card grid: as many ≥140px columns as fit (Chapter 9 §9.3) */
.grid { display: grid; gap: 10px; grid-template-columns: repeat(auto-fit, minmax(min(140px, 100%), 1fr)); }
```

How pattern 2 works: the content's huge `flex-grow` (999) means it takes almost all free space, so the sidebar stays near its 150px basis. When the line can't hold `150px + gap + 60%`, flex-wrap moves the sidebar to its own line, where it grows to full width. That is a "breakpoint" chosen by the content, not the viewport — and it works inside any container.

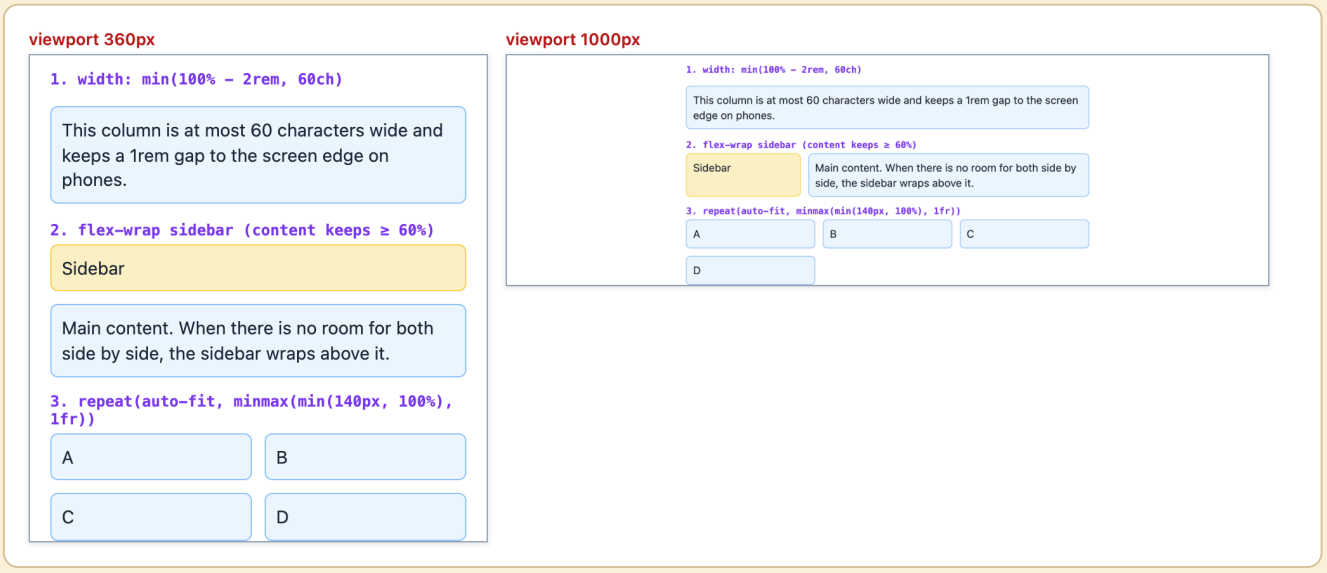


Figure 10.6 — The same page at 360px and 1000px.

Output of `code/07_intrinsic.mjs`:

```
 360px  wrapper 328.0px | sidebar wrapped above content | grid 2 column(s)
 480px  wrapper 448.0px | sidebar beside content (150.3px + 287.7px) | grid 3 column(s)
1000px  wrapper 529.1px | sidebar beside content (150.3px + 368.8px) | grid 3 column(s)
```

By hand at 480px: wrapper = min(480 − 32, 60ch ≈ 529px) = 448px. The line needs 150 + 10 + 0.6 × 448 = 428.8px ≤ 448, so the sidebar stays beside; free space 448 − 10 − 150 = 288px is shared 1 : 999, so the sidebar gets 150 + 0.288 = 150.3px ✓. At 360px the wrapper is 328px and the line would need 150 + 10 + 196.8 = 356.8px > 328, so it wraps ✓.

Other intrinsic tools

- `flex-wrap: wrap` + `flex: 1 1 12rem` on items — a row of buttons that becomes a column when needed (Chapter 8).
- `max-inline-size: 65ch` on text — the **measure** (line length) that reads best (Chapter 11).
- `aspect-ratio` for media placeholders; `object-fit: cover` to crop images into them (Chapter 11).

When not to use it

When the design genuinely needs a different structure (e.g. a sidebar that becomes a bottom tab bar), a media query states that intent more clearly than a clever flex trick.

10.10 Testing responsive design

What to test

Check	How	Pass criterion
No sideways scrolling	Resize from 320px to 1920px+	`scrollWidth ≤ clientWidth` at every width (except data tables, maps, etc.)
Reflow (WCAG 2.2 SC 1.4.10, AA)	View at 320 CSS px wide (= 1280px at 400% zoom)	All content and functions available with vertical scrolling only
Resize text (SC 1.4.4, AA)	Browser zoom 200%; also raise the default font size	No clipped or overlapping text, nothing lost
Text spacing (SC 1.4.12, AA)	Apply line-height 1.5, paragraph spacing 2em, letter 0.12em, word 0.16em	Nothing clipped — avoid fixed heights on text boxes
Target size (SC 2.5.8, AA, new in 2.2)	Inspect tap targets	At least 24 × 24 CSS px (or enough spacing)
Orientation (SC 1.3.4)	Rotate the device	Works in both, unless essential
Touch vs mouse	Real phone; `(hover: none)`	Nothing is reachable *only* on hover
Preferences	Emulate dark, reduced motion, forced colours	Readable, no lost UI

Tools

- **DevTools device mode**: Chrome/Edge (Ctrl/Cmd+Shift+M), Firefox Responsive Design Mode (Ctrl/Cmd+Shift+M), Safari Responsive Design Mode (Develop menu). Drag the width handle slowly — you'll spot the widths where things break. Chrome's Rendering panel emulates `prefers-color-scheme`, `prefers-reduced-motion`, `forced-colors` and more; Elements shows a `container` badge and lets you inspect container queries.
- **Real devices**: emulators (software imitations of a device) don't reproduce touch precision, the dynamic toolbar (`dvh`), the on-screen keyboard or real CPU (processor) speed. Test at least one Android phone and one iPhone; remote debugging (inspecting a page running on a phone from your computer — Chrome `chrome://inspect`, Safari Web Inspector) attaches DevTools to them.
- **Automated screenshots and checks** in CI (continuous integration — automatic checks run on every code change) with Puppeteer or Playwright (browser-automation libraries), like this chapter's scripts.

An automated overflow test

`code/08_overflow_check.mjs` opens pages at 320/768/1280px and reports horizontal overflow and the elements sticking out:

```
const check = page => page.evaluate(() => {
  const vw = document.documentElement.clientWidth;
  const culprits = [...document.body.querySelectorAll('*')]
    .filter(el => el.getBoundingClientRect().right > vw + 0.5 || /* … content overflowing its box … */)
    .map(el => el.tagName.toLowerCase() + (el.className ? '.' + el.className : ''));
  return { overflow: document.documentElement.scrollWidth > vw, scrollWidth: document.documentElement.scrollWidth,
culprits };
});
```

```
02_mobile_first.html   320px  ok
02_mobile_first.html   768px  ok
02_mobile_first.html  1280px  ok
08_broken.html         320px  OVERFLOW (page is 575px wide): div.promo, img, p
08_broken.html         768px  ok
08_broken.html        1280px  ok
08_fixed.html          320px  ok
08_fixed.html          768px  ok
08_fixed.html         1280px  ok
```

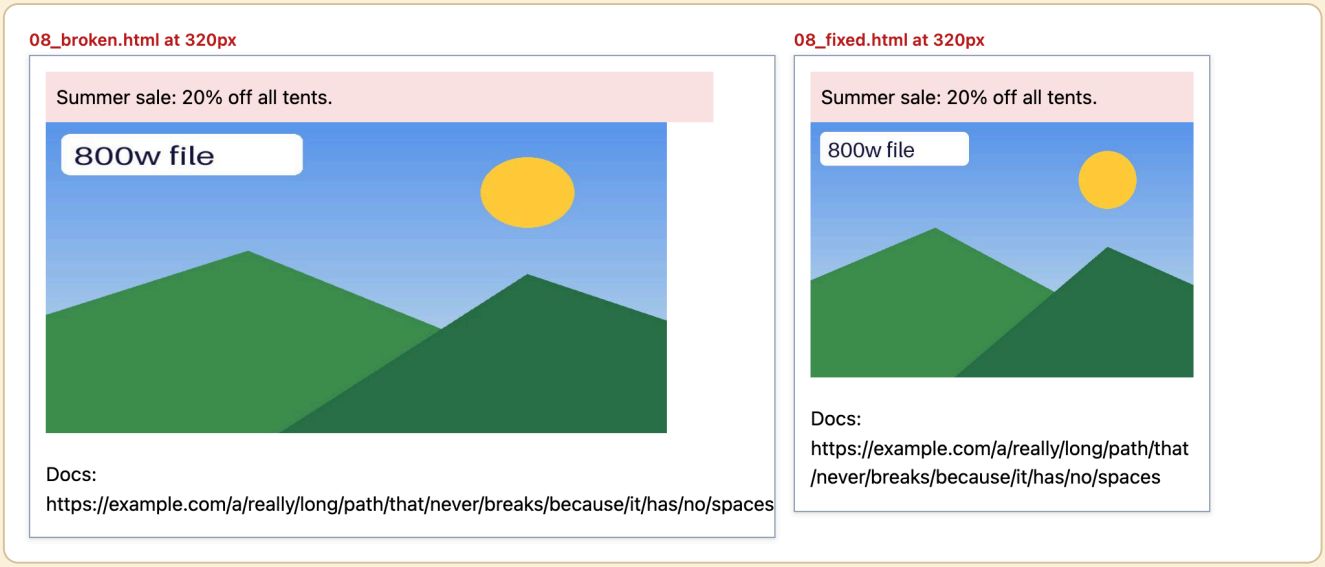


Figure 10.7 — The three most common causes of sideways scrolling on phones, and their one-line fixes.

Bug in `08_broken.html`	Fix in `08_fixed.html`
`.promo { width: 500px }`	`max-width: 500px` — fixed widths are the #1 cause
`<img width="480">` with no CSS	`img { max-width: 100%; height: auto; }`
Long URL with no break opportunities	`overflow-wrap: anywhere` (or `break-word`) on text containers

Other frequent culprits: `100vw` widths (10.2), grid columns with `1fr` that can't shrink below a long word (`minmax(0, 1fr)`, §9.10), flex items with `min-width: auto` (Chapter 8), `<pre>` blocks and wide tables (wrap them in `overflow-x: auto`).

In the interview

30-second answer: "I build mobile-first: the base CSS is the single-column phone layout and `min-width` media queries add columns where the content needs them. The viewport meta tag makes phones lay out at device width. I use intrinsic techniques — flex-wrap, `repeat(auto-fit, minmax())`, `clamp()` for fluid type with a `rem` term — so I need few breakpoints, and container queries for components that live in different-sized slots. I respect user preferences: `prefers-color-scheme` with `color-scheme`, `prefers-reduced-motion`. Images use `srcset`/`sizes`. I test in device mode, on real phones, at 320px and 200% zoom for WCAG reflow."

Follow-ups:

- *Why does `100vh` misbehave on mobile?* → It equals the large viewport (toolbars hidden), so with toolbars visible the bottom is cut off. Use `svh` or `dvh`.
- *How do you pick breakpoints?* → Where the content breaks, not device widths; prefer em so they follow the user's font size.
- *Media vs container queries?* → Viewport vs ancestor size; page layout vs component layout.
- *How do you avoid horizontal scrolling?* → No fixed widths (use `max-width`), flexible media, `overflow-wrap`, `minmax(0,1fr)`, and an automated check at 320px.

Self-check

1. What width does WCAG's Reflow criterion test? → 320 CSS px (equivalent to 1280px at 400% zoom).

2. Name three causes of horizontal overflow. → Fixed widths, images without `max-width`, long unbreakable strings (also `100vw`, rigid grid tracks, wide tables).
3. What can't DevTools device mode emulate well? → Real touch, dynamic toolbars/`dvh`, on-screen keyboard, real performance.

Chapter 10 TL;DR

- **Responsive** = fluid layout + flexible media + conditional CSS. Build **mobile-first**: base CSS is the phone layout; `(width >= …)` queries add columns.
- `<meta name="viewport" content="width=device-width, initial-scale=1">` — without it phones lay out at ~980px and shrink (measured scale 0.408 on a 400px phone). Never disable zoom.
- **Viewport units**: `svh` (toolbars shown), `lvh` (hidden), `dvh` (live); `vh` ≈ `lvh` on phones; `100vw` includes the scrollbar.
- **Media queries**: type + features + `and` / `,` / `not`. Range syntax `(640px <= width < 1024px)` is widely available and removes boundary gaps. `em` in queries = browser default font size. JS: `matchMedia(q)` + `change` event.
- **Preferences**: `prefers-color-scheme` + `color-scheme: light dark` (themes form controls too) + token overrides; `light-dark()` (Baseline 2024); `prefers-contrast`, `forced-colors`, `prefers-reduced-motion`.
- **Container queries** (widely available since Aug 2025): `container: name / inline-size` on a wrapper, `@container name (width >= 400px)` for descendants, `cqi` units. Style queries on custom properties: Baseline 2026. Scroll-state queries: Chromium only.
- **Fluid type**: `clamp(MIN, a·rem + b·vw, MAX)`; slope = Δsize/Δviewport, intercept = size − slope × viewport; keep a `rem` term so user settings and zoom still work (vw-only stayed 16px when the default was 20px).
- **Images**: `max-width: 100%`, `srcset`/`sizes`, `<picture>`; `image-set()` for backgrounds (downloads one file per DPR).
- **Intrinsic layouts**: `min(100% - 2rem, 60ch)`, flex-wrap sidebar, `repeat(auto-fit, minmax(min(X,100%),1fr))` — breakpoints chosen by content.
- **Test**: 320px reflow, 200% zoom, text spacing, 24px targets, real devices, dark/reduced motion, automated overflow checks.

Next: Chapter 11 — Typography, Colour and Visual Effects.

Chapter 11 — Typography, Colour and Visual Effects

Layout decides *where* boxes go; this chapter is about how they *look*. Most of a web page is text, so typography — the choice of fonts, sizes, line spacing and line length — does more for readability than any other styling. Then come colour and contrast (which decide whether some readers can read at all), and the decorative toolkit: backgrounds and gradients (smooth transitions between colours), borders and rounded corners, shadows, filters (photo-style effects such as blur), blend modes (rules for mixing an element's colours with what is behind it) and image fitting. Each has a performance or accessibility side that is easy to miss.

What you'll learn

#	Topic	You will be able to…
11.1	Fonts	write font stacks (ordered lists of fonts to try), load web fonts (font files downloaded with the page) with `@font-face`, and use a variable font
11.2	Loading fonts well	choose a `font-display` value, preload (fetch early), subset (strip unused characters) and self-host (serve from your own server) fonts
11.3	Text properties	set size, line-height, spacing, decoration, wrapping, truncation (cutting text off with "…") and numerals
11.4	Colour and contrast	compute a WCAG (Web Content Accessibility Guidelines) contrast ratio and derive palettes with `color-mix()` and relative colours
11.5	Backgrounds and gradients	layer images and linear/radial/conic gradients, and size/position them
11.6	Borders, radius and outlines	draw any border, rounded or elliptical corner, and a proper outline
11.7	Shadows	build elevation (the look of a surface floating above the page) with `box-shadow`, use `text-shadow` and `drop-shadow()`
11.8	Filters, blend modes, clipping (cutting an element to a shape) and masks (fading parts of it out)	apply `filter`, `backdrop-filter`, `mix-blend-mode`, `clip-path` and `mask`
11.9	`object-fit` and `object-position`	fit images and video into any box without distortion

As in Chapters 7–10, every example is a page in `code/`, its screenshot in `img/` is made by `code/make_figures.mjs`, and the `NN_*.mjs` scripts use `code/measure.mjs` (a copy of Chapter 10's helper) to read back what Chrome computed — font load status, contrast ratios, even individual **pixel colours** from a screenshot, to check the maths of gradients and blend modes. The one web font used, **Inter**, is in `code/media/` with its SIL Open Font License (a free licence that allows using and redistributing the font).

11.1 Fonts

What it is

A **font family** is a designed set of letter shapes (**glyphs**), usually in several **weights** (thin → black) and **styles** (normal, italic). CSS picks one with `font-family` and the related properties:

```
body {
  font-family: "Inter", system-ui, -apple-system, "Segoe UI", Roboto, sans-serif;
  font-size: 1rem;           /* 16px by default */
  font-weight: 400;          /* 100–900; normal = 400, bold = 700 */
  font-style: normal;        /* italic | oblique */
  line-height: 1.5;
}
/* shorthand: [style] [variant] [weight] [stretch] size[/line-height] family  (size and family required) */
h1 { font: italic 700 2rem/1.2 Georgia, serif; }
```

Font stacks and generic families

`font-family` is a **fallback list**. For every character the browser tries each family in order and uses the first one that is available *and* has that glyph. Always end with a **generic family**, which every browser maps to something installed:

Generic	Looks like	Typical mapping
`serif`	strokes with small "feet"	Times New Roman
`sans-serif`	no feet	Helvetica / Arial
`monospace`	every character equally wide	Menlo, Consolas, Courier
`system-ui`	the operating system's UI font	San Francisco (Apple), Segoe UI (Windows), Roboto (Android)
`cursive`, `fantasy`	script / decorative	varies wildly — avoid for body text
`ui-serif`, `ui-sans-serif`, `ui-monospace`, `ui-rounded`	the OS's UI variants	only some platforms (Apple mainly)

A **system font stack** (`system-ui, …, sans-serif`) costs zero downloads and looks native (like the operating system's own apps) — GitHub and many design systems (a team's shared set of styles and components) use one. Family names with spaces should be quoted; generic names must *not* be quoted (`"serif"` means a font literally named "serif").

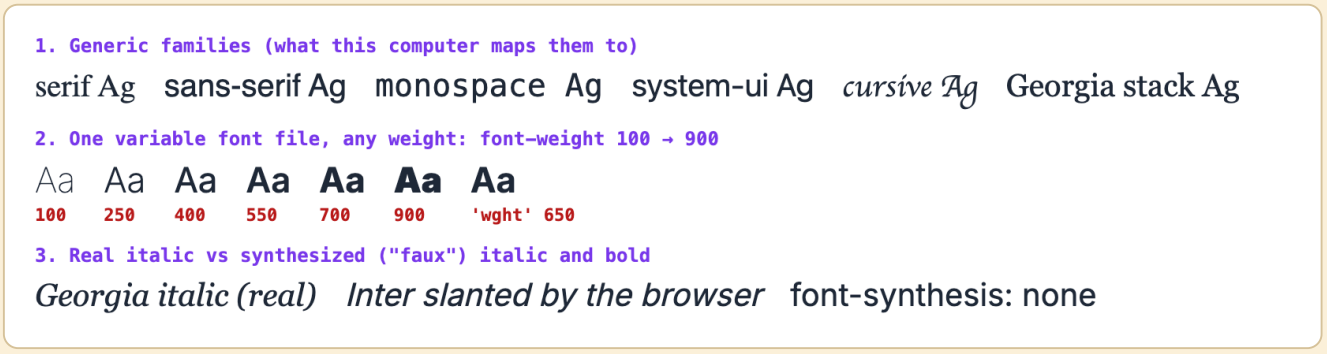


Figure 11.1 — `code/01_fonts.html`: generic families on this Mac, one variable font file at seven weights, and real vs synthesized italic.

`@font-face`: using your own font

```css
@font-face {
  font-family: "Inter Var";                                  /* the name YOU choose */
  src: url("media/inter-latin-wght-normal.woff2") format("woff2");
  font-weight: 100 900;                                      /* this file covers a range */
  font-style: normal;
  font-display: swap;                                        /* 11.2 */
}
.inter { font-family: "Inter Var", system-ui, sans-serif; }
```

How it works:

1. `@font-face` only *declares* a font; nothing downloads yet.
2. When the browser lays out text that needs that family (with a matching weight/style), it **downloads the file** — lazily.
3. For each weight/style request, the browser picks the `@font-face` rule whose descriptors (the property-like settings inside `@font-face`: `font-weight`, `font-style`, `font-stretch`) match best. With static fonts (ordinary fonts with one fixed weight and style per file) you write one rule per file (Regular 400, Bold 700, Italic…) all sharing one `font-family` name.
4. If no face matches a requested bold or italic, the browser **synthesizes** one: it smears the glyphs for bold and slants them for italic. `font-synthesis: none` turns this off (see the last sample in Figure 11.1 — it stays upright instead of faking italic).

Font formats

Format	Use
WOFF2	The only one you need today: compressed with Brotli (a modern compression method), supported by every current browser
WOFF	Older compressed format; only for very old browsers
TTF / OTF	Desktop formats; larger; avoid on the web
EOT, SVG fonts	Obsolete

Variable fonts

A **variable font** stores one or more **axes** of variation in a single file, so every weight between 100 and 900 (not just the nine standard steps) comes from one download. Registered axes and the CSS properties that drive them:

Axis tag	Meaning	Prefer this property
`wght`	weight	`font-weight: 550`
`wdth`	width	`font-stretch: 85%`
`ital`	italic on/off	`font-style: italic`
`slnt`	slant angle	`font-style: oblique 10deg`
`opsz`	optical size (letter shapes tuned for small vs large text)	`font-optical-sizing: auto` (default)

`font-variation-settings: "wght" 650, "GRAD" 50` sets axes directly — needed for **custom axes** (uppercase tags such as `GRAD`, grade) but it overrides the higher-level properties and isn't inherited in a composable way (a child setting one axis wipes out the axes its parent set), so use the normal properties where one exists. Variable fonts are Baseline widely available (supported in all major browsers; since 2018).

`code/01_fonts.mjs` confirms the one file serves the whole range and really changes the outlines:

```
Inter Var weight=100 900 status=loaded display=swap
document.fonts.check("700 22px Inter Var"): true
"Hamburgefonstiv" at 40px, width by weight -> 100: 307.5px, 400: 325.2px, 700: 340.5px, 900: 353.5px
font-synthesis on the last sample: none
```

("Hamburgefonstiv" is a classic type-testing word that contains most letter shapes.) Heavier weights are wider — these are genuinely different glyphs, not a smeared regular.

Trade-offs

Option	+	−
System font stack	0 bytes, instant, native feel	Looks different on each OS; less brand identity
One variable web font	Any weight from one file (~50 KB here for Latin)	One larger file; must manage loading (11.2)
Several static web fonts	Each file small	Every extra weight/style is another request

Common mistakes

- No generic family at the end — if the web font fails, you get the browser default (often Times).
- Asking for `font-weight: 600` when only 400 and 700 files exist — the browser picks 700 (the nearest heavier face); `font-weight: bold` on a family with no bold face → faux bold (fake bold, made by the browser thickening the regular letters).
- Loading five weights "just in case". Check which weights the design actually uses.
- Quoting generic names: `font-family: "sans-serif"` is a (missing) font called sans-serif.

Self-check

1. Why end a stack with `sans-serif`? → It always resolves to an installed font.
2. What does `font-weight: 100 900` inside `@font-face` mean? → This one file supports every weight in that range (a variable font).
3. What is faux bold and how do you prevent it? → Browser-synthesized bold when no bold face exists; provide the face or set `font-synthesis: none`.

11.2 Loading fonts well: `font-display`, preload, subsetting

The problem it solves

A web font is a separate download that usually starts **late** — only after HTML and CSS are parsed and the browser knows the text needs it. Meanwhile the browser must decide what to show. Two historical failure modes:

- **FOIT** (Flash Of Invisible Text): text is hidden until the font arrives. On a slow network, readers stare at blank space.
- **FOUT** (Flash Of Unstyled Text): text shows in a fallback font, then **swaps** — the reflow (re-calculation of layout) may shift content (hurting **CLS**, Cumulative Layout Shift, a Core Web Vital — one of Google's headline page-experience metrics — Chapter 14).

`font-display`

The `font-display` descriptor chooses the trade-off by setting two periods (CSS Fonts Level 4; Baseline widely available):

1. **Block period** — text is drawn invisibly while the font loads.
2. **Swap period** — text is drawn in the fallback; if the font arrives, it swaps in.
3. **Failure period** — too late: keep the fallback for this page.

Value	Block period	Swap period	Use for
`auto`	browser decides (usually like `block`)		—
`block`	short (spec suggests 3 s)	infinite	icon fonts (fonts whose characters are pictures), where fallback glyphs would be wrong
`swap`	extremely short (≈0–100 ms)	infinite	brand/heading fonts that must appear
`fallback`	extremely short	short (≈3 s)	body text: use the font only if it comes quickly
`optional`	extremely short (≈100 ms)	**none**	body text where stability matters most; the font is used from cache (the browser's store of already-downloaded files) on later pages

`code/02_font_display.html` declares the same font three times with `block`, `swap` and `optional` (a Georgia fallback so you can *see* which is in use). `code/02_font_display.mjs` intercepts the network (catches each request before it goes out) and delays every font download by 1.5 seconds (timings are machine-dependent):

```
t≈301ms  F-block=loading  F-swap=loading  F-optional=loading
t≈3503ms  F-block=loaded  F-swap=loaded  F-optional=loaded
  .block     drawn with: Menlo, Inter (web font)
  .swap      drawn with: Menlo, Inter (web font)
  .optional  drawn with: Georgia, Menlo
```

(Menlo is the monospace font of the red `<code>` labels.) The last line is the interesting one: the `optional` font **did** finish downloading, but its window had closed, so the paragraph stays in Georgia for this page view.

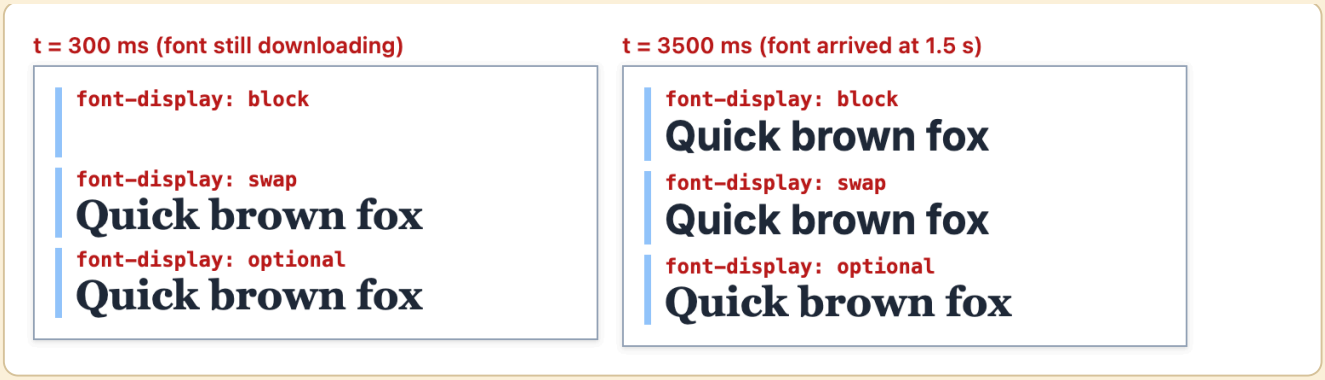


Figure 11.2 — FOIT vs FOUT vs "no swap", captured by delaying the font download.

Making fonts arrive sooner

Technique	How	Why
Preload the critical font	`<link rel="preload" href="/fonts/inter.woff2" as="font" type="font/woff2" crossorigin>`	Starts the download while HTML is parsed instead of after CSS. Font requests are always CORS-mode (CORS, Cross-Origin Resource Sharing, is the set of rules for loading resources across sites), so `crossorigin` is required even for same-origin files — without it the preload is wasted. Preload only 1–2 fonts.
Self-host	Serve the files from your own domain	No extra DNS/TLS connection (no extra address lookup and secure handshake with another server; see System Design Fundamentals); no third-party (another company's server) privacy question (in January 2022 the Munich Regional Court — LG München I, case 3 O 17493/20 — held that loading Google-hosted fonts sent visitors' IP addresses (the numeric network address of their device) to Google without consent, breaching the GDPR (the EU's data-protection law), and awarded the visitor €100)
Subset	Keep only the characters you need; declare `unicode-range: U+0000-00FF, …`	The Inter file here is only the Latin subset (48 KB). Browsers download a `unicode-range` face only if the page uses one of those characters.
Match the fallback's metrics (its letter sizes and line spacing)	`@font-face { font-family: "Inter Fallback"; src: local("Arial"); size-adjust: 107%; }`	The swap then barely moves text, reducing CLS. `size-adjust` works in all current browsers (Safari since 17); `ascent-override` / `descent-override` are not yet supported in Safari.
Fewer files	One variable font instead of 4–6 static ones	Fewer requests
Cache	Long `Cache-Control` max-age (an HTTP header saying how long the browser may reuse the file) with hashed file names (names containing a fingerprint of the content, so a new version gets a new name)	Later pages are instant

From JavaScript

The **CSS Font Loading API** (`document.fonts`; an API is a set of functions a program offers to other code) exposes each face (`status: "unloaded" | "loading" | "loaded" | "error"`), `document.fonts.load('700 1em Inter')` to start a load, and `document.fonts.ready` (a promise — a JavaScript object representing a result that arrives later) — which the chapter scripts await before measuring.

In the interview

30-second answer: "Web fonts load late, so text either goes invisible (FOIT) or shows a fallback then swaps (FOUT) and can shift layout. I choose `font-display`: `swap` for brand text, `optional` or `fallback` for

> body text, `block` only for icon fonts. I self-host WOFF2, subset with `unicode-range`, preload the one or two critical files with `crossorigin`, use a variable font instead of many weights, and tune the fallback with `size-adjust` to reduce CLS."

Follow-ups:

- *Why does font preload need `crossorigin`?* → Fonts are fetched in CORS mode; a preload without it doesn't match the real request, so the font downloads twice.
- *When is `optional` better than `swap`?* → When layout stability matters more than showing the brand font on the first view; the font is cached and used on the next page.

Self-check

1. What does a user see at 300 ms with `font-display: block` on a slow network? → Invisible text (up to about 3 s).
2. Which value never swaps after the first ~100 ms? → `optional`.
3. Name two ways to reduce font-related layout shift. → `font-display: optional`; metric-matched fallback with `size-adjust`; preloading.

11.3 Text properties

The essentials

Property	Example	Notes
`font-size`	`1.125rem`	Use `rem` for text so user settings work (Chapter 6); fluid sizes with `clamp()` (§10.7)
`line-height`	`1.5`	**Unitless** (see below). Body 1.4–1.6, headings 1.1–1.25
`letter-spacing`	`0.08em`	Small positive values for ALL-CAPS labels; slightly negative for large headings
`word-spacing`	`0.1em`	Rarely needed
`text-align`	`start` | `end` | `center` | `justify`	`start`/`end` follow writing direction; avoid `justify` without hyphenation (rivers of space: gaps lining up down the paragraph)
`text-transform`	`uppercase` | `capitalize`	Keep real capitals in the HTML if they matter; screen readers (software that reads pages aloud) read the source text
`text-indent`	`2em`	First-line indent (books)
`text-decoration`	`underline wavy red`	Longhands: `-line`, `-style` (solid/double/dotted/dashed/wavy), `-color`, `-thickness`
`text-underline-offset`	`0.2em`	Moves the underline away from descenders (the tails of g, p, y)
`white-space`	`nowrap` | `pre` | `pre-wrap`	Controls collapsing of spaces and wrapping
`overflow-wrap`	`anywhere` | `break-word`	Break a long word/URL only if it would overflow (§10.10)
`word-break`	`break-all`	Break between any letters — harsh; mainly for CJK (Chinese, Japanese, Korean) or code
`hyphens`	`auto`	Dictionary hyphenation; needs a `lang` attribute; Baseline since 2023
`text-wrap`	`balance` | `pretty`	Better line breaking (below)
`font-variant-numeric`	`tabular-nums`	Equal-width digits for tables and prices
`max-inline-size`	`65ch`	The **measure** — line length

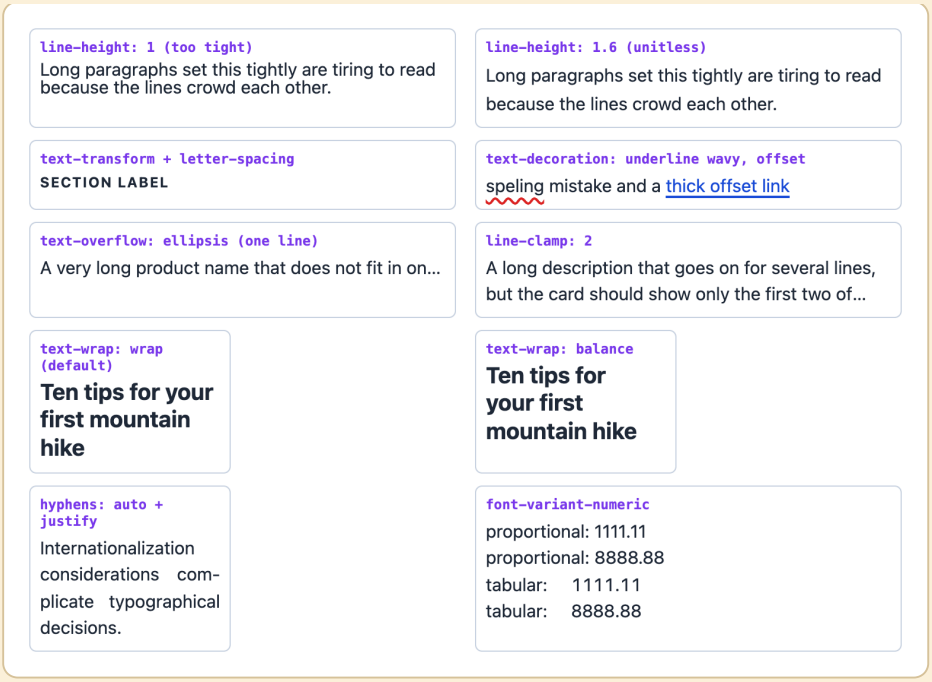


Figure 11.3 — `code/03_text.html`.

Why `line-height` should be unitless

`code/03_text.mjs` puts a 32px child inside a 16px parent with `line-height: 1.5` and `1.5em`:

```
line-height: 1.5   -> child (32px) line box 48px
line-height: 1.5em -> child (32px) computed 24px, line box 24px  (text taller than its line!)
proportional-nums width of "1" 6.73px, of "8" 9.36px
tabular-nums      width of "1" 9.22px, of "8" 9.22px
.caps letter-spacing computed: 0.96px (0.08em of 12px)
.balance text-wrap: balance
```

A unitless value is inherited as a **factor** and re-multiplied by each element's own font size (32 × 1.5 = 48px). A length (`em`, `px`, `%`) is computed **once** on the parent (16 × 1.5 = 24px) and that fixed pixel value is inherited — so bigger text overlaps its neighbours. The same output proves `tabular-nums`: every digit becomes 9.22px wide, so columns of numbers line up.

Line length (the measure)

Comfortable reading needs roughly **45–75 characters per line** (a classic typographic rule of thumb). `max-inline-size: 65ch` (the `ch` unit ≈ the width of "0") keeps paragraphs in that range at any screen size.

Better line breaking: `text-wrap`

- `text-wrap: balance` makes all lines of a short block about equally long — no single orphaned word (a lone word left on the last line) under a long headline. Browsers only balance short blocks (a handful of lines) because it is expensive. **Baseline since May 2024.**
- `text-wrap: pretty` uses a slower algorithm for paragraphs that mainly avoids a very short last line. Supported in Chrome and Safari 26 but not Firefox — **not Baseline**; harmless to add as an enhancement.

Truncation

```css
/* One line with "…" — all three are required */
.title { white-space: nowrap; overflow: hidden; text-overflow: ellipsis; }

/* N lines with "…" — the prefixed syntax is the one that works in every browser */
.desc { display: -webkit-box; -webkit-box-orient: vertical; -webkit-line-clamp: 2; overflow: hidden; }
```

The unprefixed `line-clamp` property is specified but not yet implemented by browsers; the `-webkit-` (vendor-prefixed — originally one engine's own extension) version above is interoperable (works the same in every browser). Truncation hides content from sighted users only — make sure the full text is reachable (a details page, or a `title`/tooltip is **not** enough for touch and keyboard users).

A common trap, visible while building Figure 11.3: in a `1fr` grid column, a `nowrap` line makes the **column** grow to fit the text, so no ellipsis appears. The fix is `minmax(0, 1fr)` (§9.10) or `min-width: 0` on the flex/grid item.

Common mistakes

- `line-height` in `px` on `body` — every heading inherits the same fixed spacing.
- Light grey, thin, small text — fails contrast (11.4) and is hard to read.
- Centre-aligning long paragraphs — ragged left edges are hard to scan.
- All-caps paragraphs; `text-transform: uppercase` on long text.
- Removing link underlines in body text: colour alone must not be the only way to identify a link (WCAG 1.4.1) unless the colour contrast with the surrounding text is at least 3:1 *and* there's another cue on hover/focus.

Self-check

1. Parent 20px, `line-height: 1.2em`; child 40px. Child's line-height? → 24px (computed on the parent and inherited) — too tight. With `1.2`: 48px.
2. Three properties needed for a one-line ellipsis? → `white-space: nowrap`, `overflow: hidden`, `text-overflow: ellipsis`.
3. What does `font-variant-numeric: tabular-nums` do? → Makes all digits the same width so numbers align in columns.

11.4 Colour and contrast

Chapter 6 §6.8 introduced colour notations (hex, `rgb()`, `hsl()`, `oklch()`, alpha, `currentColor`). This section is about choosing colours people can read, and deriving palettes from one brand colour.

Contrast ratio — what it is

WCAG defines **contrast ratio** between two colours as `(L1 + 0.05) / (L2 + 0.05)`, where L1 is the **relative luminance** (how bright a colour looks, from 0 for black to 1 for white) of the lighter colour and L2 of the darker. It runs from **1:1** (same colour) to **21:1** (black on white).

How it is computed

`code/contrast.mjs`:

```
// 1. sRGB channel 0-255 -> 0-1 -> linear light (undo the gamma curve)
export const lin = c => { c /= 255; return c <= 0.04045 ? c / 12.92 : ((c + 0.055) / 1.055) ** 2.4; };
// 2. relative luminance: green counts most, blue least (how human eyes weigh them)
export const luminance = ([r, g, b]) => 0.2126 * lin(r) + 0.7152 * lin(g) + 0.0722 * lin(b);
// 3. ratio of the lighter to the darker, each + 0.05 (for screen flare)
export const ratio = (a, b) => { const [hi, lo] = [luminance(a), luminance(b)].sort((x, y) => y - x); return (hi + 0.05) / (lo + 0.05); };
```

(The printed WCAG 2.0 formula uses 0.03928 as the threshold; newer text corrects it to 0.04045 — for 8-bit colours both give identical results.)

The thresholds (WCAG 2.2)

Criterion	Level	Minimum ratio
1.4.3 Contrast (normal text)	AA	**4.5:1**
1.4.3 Large text (≥ 24px, or ≥ 18.66px bold)	AA	**3:1**
1.4.6 Enhanced contrast	AAA (the highest, strictest WCAG level)	7:1 (large: 4.5:1)
1.4.11 Non-text contrast (icons, input borders, focus rings, chart lines)	AA	**3:1** against adjacent colours

Logos and purely decorative or disabled elements are exempt. See Chapter 3 §3.9 for the rest of WCAG.

Output of `code/04_contrast.mjs` — first by the formula, then using colours the browser computed from `code/04_contrast.html` (the page runs the same formula and prints the result on each swatch):

```
#000000 on #ffffff: 21.00:1  AAA
#767676 on #ffffff: 4.54:1  AA
#777777 on #ffffff: 4.48:1  large/UI only
#1d4ed8 on #ffffff: 6.70:1  AA
#ffffff on #3b82f6: 3.68:1  large/UI only
#e2e8f0 on #0f172a: 14.48:1  AAA
#fcd34d on #ffffff: 1.44:1  FAIL
#fff on var(--brand)                        -> rgb(255, 255, 255) on oklch(0.55 0.2 260)  = 5.02:1 AA
#fff on color-mix(in oklch, brand 70%, black) -> rgb(255, 255, 255) on oklch(0.385 0.14 260)  = 10.04:1 AAA
brand on oklch(from brand 95% 0.03 h)       -> oklch(0.55 0.2 260) on oklch(0.95 0.03 260)  = 4.32:1 large/UI only
#000 on oklch(from brand l c calc(h + 180))  -> rgb(0, 0, 0) on oklch(0.55 0.2 80)  = 4.25:1 large/UI only
```

(Four of the browser rows, identical to the formula rows above them, are omitted.) Things worth noticing:

- **`#767676`** is the lightest grey that passes AA on white; one step lighter (#777777) fails.
- White on a typical "blue-500" button (#3b82f6) is only 3.68:1 — fine for large bold text, **not** for normal-size labels. Many design systems use a darker shade for button backgrounds for this reason.
- Yellow text on white is nearly invisible (1.44:1).
- The last row: the complementary colour (the one opposite on the colour wheel) has the **same** OKLCH lightness (55%) as the brand, yet black on it gives 4.25:1 while white on the brand gives 5.02:1. OKLCH lightness is perceptual but it is *not* the WCAG luminance; always check the actual pair.

Figure 11.4 — Contrast computed inside the page from the browser's own colours.

Deriving colours: `color-mix()` and relative colour syntax

Instead of hard-coding every shade, derive them from one token (a named design value stored in a custom property):

```
:root { --brand: oklch(55% 0.2 260); }
.darker     { background: color-mix(in oklch, var(--brand) 70%, black); }   /* 70% brand, 30% black */
.tint       { background: oklch(from var(--brand) 95% 0.03 h); }           /* keep hue, set L and C */
.complement { background: oklch(from var(--brand) l c calc(h + 180)); }    /* rotate the hue */
.faded      { background: rgb(from var(--brand) r g b / 50%); }            /* same colour, half opacity */
```

- `color-mix(in <space>, A p%, B)` — Baseline since 2023. Mixing in `oklch` or `oklab` gives more even results than `srgb`.
- **Relative colour syntax** `<fn>(from <colour> …)` exposes the source's channels as keywords (`l c h`, `r g b`, `h s l`, `alpha`) that you can keep, replace or compute with `calc()`. Baseline since September 2024 (newly available).

Other colour tools

- `currentColor` — the element's `color` value; great for icons and borders that follow text colour.
- `accent-color: #1d4ed8` — tints checkboxes, radios, range sliders and progress bars. Not yet Baseline (Chrome for Android lacks it), but it degrades gracefully (browsers without it simply show the default colour).

- `color-scheme` and `light-dark()` — §10.5.

Accessibility beyond ratios

- **Never use colour alone** to carry meaning (WCAG 1.4.1): add an icon, text or pattern to "red = error", and underline links in text.
- About 1 in 12 men has some form of colour-vision deficiency (colour blindness); red/green pairs are the classic problem. DevTools (Rendering → Emulate vision deficiencies) simulates them.
- **APCA** (Accessible Perceptual Contrast Algorithm), a newer perceptual contrast method, is being explored for the future WCAG 3; it is not a requirement today. WCAG 2.2's ratio is what audits and laws reference.

In the interview

30-second answer: "WCAG contrast is (L1 + 0.05)/(L2 + 0.05) using relative luminance; AA needs 4.5:1 for body text, 3:1 for large text and for UI components like input borders and focus rings. I check real pairs with DevTools or a script — lightness in HSL or even OKLCH doesn't guarantee it — and I never rely on colour alone. For palettes I define a brand token in OKLCH and derive shades with `color-mix()` or relative colour syntax."

Self-check

1. Minimum AA ratio for a 14px grey caption? → 4.5:1.
2. For a 3px input border against white? → 3:1 (non-text contrast).
3. What does `oklch(from var(--c) l c calc(h + 180))` produce? → The same lightness and chroma with the hue rotated 180° (the complement).

11.5 Backgrounds and gradients

The background properties

Property	Values	Notes
`background-color`	any colour	Painted under all image layers; always set one as a fallback under images
`background-image`	`url()`, gradients, `image-set()`	Comma-separated **layers**; the **first is on top**
`background-repeat`	`repeat` | `no-repeat` | `repeat-x` | `space` | `round`	Default `repeat`
`background-position`	`center`, `right 10px bottom 10px`, `30% 70%`	Where the image sits
`background-size`	`auto` | `cover` | `contain` | `200px 100px`	`cover` fills and crops; `contain` fits entirely
`background-origin`	`padding-box` | `border-box` | `content-box`	Box that positions the image
`background-clip`	`border-box` | `padding-box` | `content-box` | `text`	Box where painting stops
`background-attachment`	`scroll` | `fixed` | `local`	`fixed` is ignored on iOS and is not Baseline; it also costs repaint performance (the browser must redraw on every scroll)
`background`	shorthand	`url(a.jpg) center / cover no-repeat, #e0f2fe` — `size` goes after `position` with a `/`

Gradients

A **gradient** is an image generated by the browser — no download, crisp at any size, animatable (able to be smoothly animated) only indirectly.

Function	Shape	Example
`linear-gradient()`	along a line	`linear-gradient(135deg, #f59e0b, #ef4444 50%, #7c3aed)`
`radial-gradient()`	circles/ellipses from a point	`radial-gradient(circle at 30% 30%, #fff 0 8%, #60a5fa 40%, #1e3a8a)`
`conic-gradient()`	around a point (like a clock hand)	`conic-gradient(#3b82f6 0 40%, #22c55e 0 70%, #f59e0b 0)` — a pie chart
`repeating-*-gradient()`	the pattern repeats	`repeating-linear-gradient(45deg, #fde68a 0 10px, #f59e0b 10px 20px)` — stripes

Colour stops are `colour [position] [position]`. Two stops at the same position make a **hard edge** (no blending), and a position of `0` after a larger one means "same as the previous stop" — that's how the conic pie chart gets sharp slices. Direction: `to right`, `to top left`, or an angle (`0deg` = up, `90deg` = right).

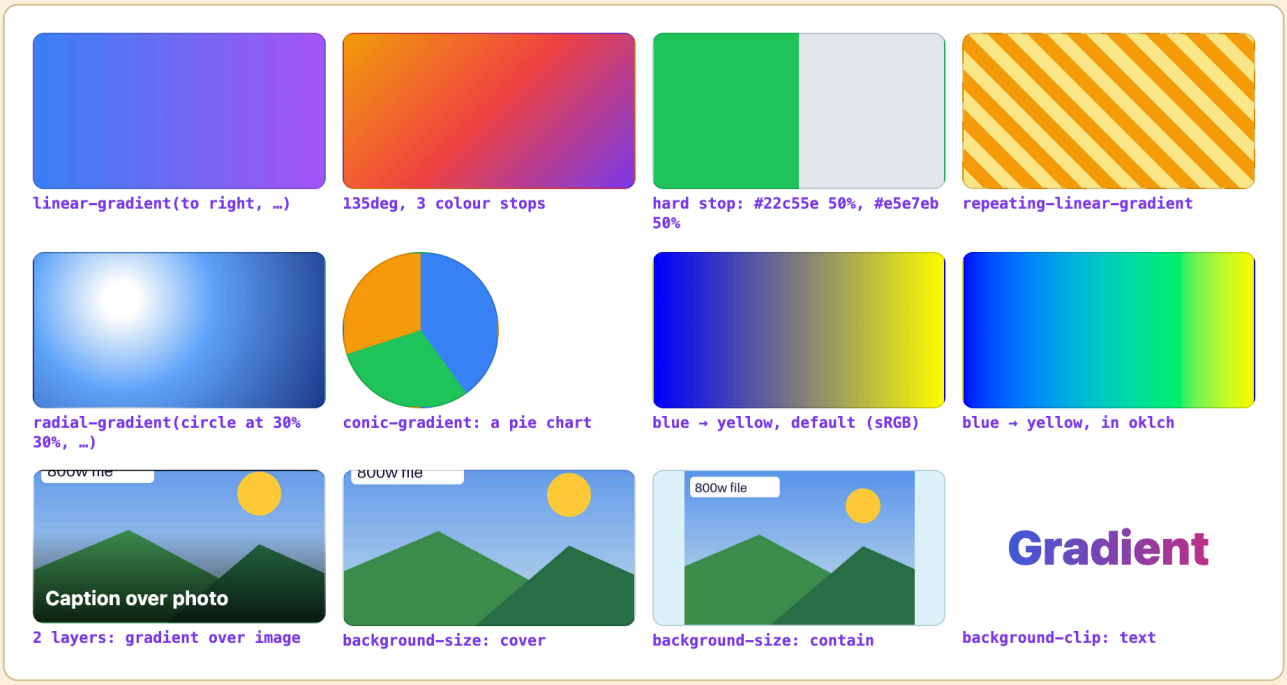


Figure 11.5 — `code/05_backgrounds.html`.

Interpolation colour space

By default gradients blend in **sRGB** (the standard colour space of ordinary screens), which makes complementary colours pass through a muddy grey. Since June 2024 (Baseline) you can pick the space: `linear-gradient(in oklch to right, blue, yellow)`. `code/05_backgrounds.mjs` screenshots the page and reads the pixel in the middle:

```
.srgb   at  50%: rgb(128, 128, 127)
.oklch  at  50%: rgb(0, 208, 188)
.hard   at  49%: rgb(34, 197, 94)
.hard   at  51%: rgb(229, 231, 235)
```

The sRGB midpoint of blue (0, 0, 255) and yellow (255, 255, 0) is exactly grey (128, 128, 127); in OKLCH the hue travels around the colour wheel instead, through a vivid teal. The two `.hard` samples, 2% apart, are the two pure stop colours — no blend.

Multiple backgrounds — a real use

Text over photos needs contrast. Put a gradient layer **above** the image:

```css
.hero {
  background:
    linear-gradient(to top, rgb(0 0 0 / .75), transparent 60%),   /* layer 1: on top */
    url("hero.jpg") center / cover no-repeat;                     /* layer 2: below */
  color: #fff;
}
```

Gradient text

```
.gradient-text { background: linear-gradient(90deg, #2563eb, #db2777);
                 -webkit-background-clip: text; background-clip: text; color: transparent; }
```

Write both lines: web-features (the open dataset behind Baseline labels) still lists unprefixed `background-clip: text` as not Baseline. Check contrast at the gradient's lightest point, and keep a solid `color` fallback in forced-colours mode (an operating-system high-contrast setting that replaces page colours with the user's own).

Common mistakes

- Putting the gradient layer **after** the image — it ends up underneath and invisible.
- No `background-color` under a photo: if the image fails, white text sits on white.
- Meaningful images as CSS backgrounds — no `alt`, not announced; use `<img>` (Chapter 4).
- Huge background photos on mobile — use `image-set()` (§10.8).

Self-check

1. In `background: url(a.png), url(b.png)`, which is on top? → `a.png` (first layer).
2. How do you draw a 50/50 hard split? → `linear-gradient(to right, green 50%, grey 50%)`.
3. Why does blue→yellow look grey in the middle by default? → Interpolation in sRGB averages the channels; use `in oklch`.

11.6 Borders, radius and outlines

Borders

`border: <width> <style> <colour>` on all sides, or per side (`border-left`), or per aspect (`border-width: 1px 2px`). **The style is required** — its initial value is `none` (Chapter 7). Styles: `solid`, `dashed`, `dotted`, `double`, `groove`, `ridge`, `inset`, `outset`, `hidden`, `none`. Borders take up layout space (they're part of the box model). `border-image` can slice an image into a border — rarely used.

`border-radius`

Rounds the corners of the **border box** (and clips the background with it):

Value	Result
`8px`	all four corners
`24px 0 24px 0`	top-left, top-right, bottom-right, bottom-left (clockwise)
`50%` on a square	a circle (`aspect-ratio: 1` keeps it square)
`999px`	a **pill**: a very large radius is reduced to fit, so the ends are exact semicircles at any height
`50% / 30%`	**elliptical** corners: horizontal radii / vertical radii

Percentages are relative to the box's own width (horizontal radius) and height (vertical radius). When adjacent radii would overlap, the browser scales all radii down proportionally. (The `corner-shape` property for squircles (smooth square–circle hybrids) and bevels (cut-off corners) is Chromium-only — only in browsers built on Google's engine — in 2026.)

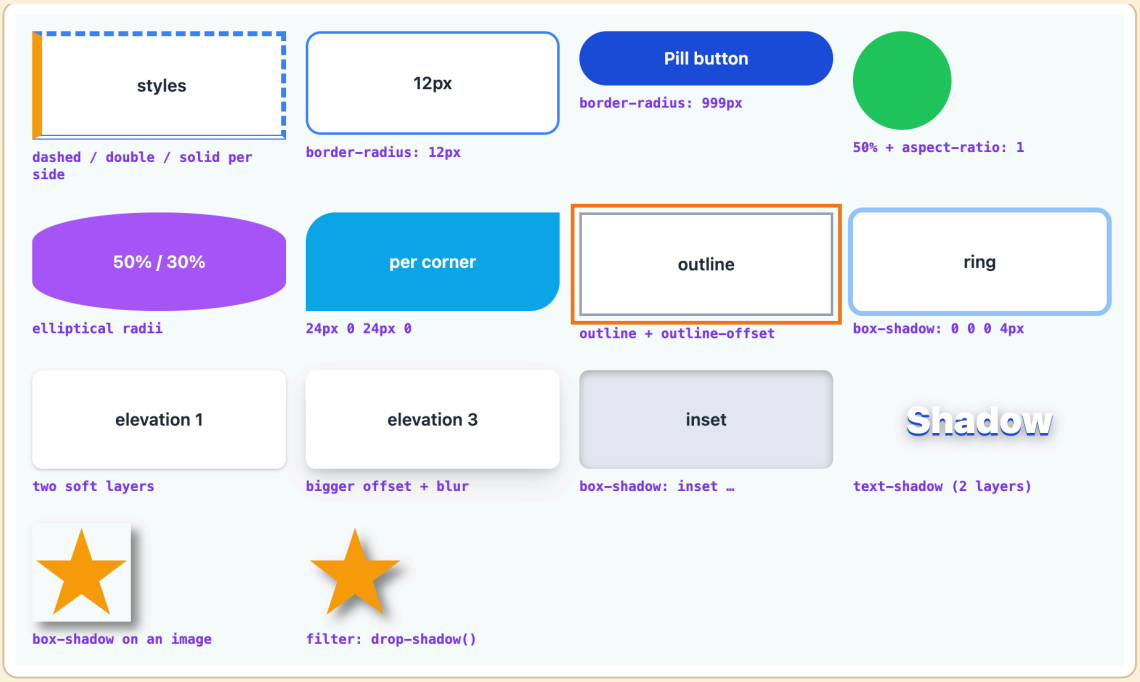


Figure 11.6 — `code/06_borders_shadows.html`.

Outline vs border vs ring

	`border`	`outline`	`box-shadow: 0 0 0 4px` ("ring")
Takes layout space	**yes**	no	no
Per-side control	yes	no	no
Follows `border-radius`	yes	yes (all current browsers since 2023)	yes
Offset	—	`outline-offset` (can be negative)	via extra layers
Shown in forced-colours mode	yes	**yes**	**no** (shadows are removed)
Main use	structure, dividers	**focus indicators**	decorative rings

`code/06_borders_shadows.mjs` confirms that only borders change the layout size (every box has `height: 80px`):

```
.styles   layout box 206x88  border-left 8px  outline none       shadow none
.outline  layout box 206x84  border-left 2px  outline 3px +4px   shadow none
.ring     layout box 206x80  border-left 0px  outline none       shadow rgb(147, 197, 253) 0px 0px 0px 4px
.elev3    layout box 206x80  border-left 0px  outline none       shadow rgba(0, 0, 0, 0.1) 0px 4px 6px -1px, rgba(0,
.ellipse  border-top-left-radius: 50% 30%
```

This is why focus styles use `outline`: `:focus-visible { outline: 3px solid #2563eb; outline-offset: 2px; }` doesn't shift the layout, and still shows in Windows contrast themes. Never remove the outline without a replacement (Chapter 3 §3.7). A popular pattern is a transparent outline plus a box-shadow ring, so forced-colours mode still gets the outline.

Self-check

1. Why doesn't `border: 2px #333` show? → No style; the default `none` wins.
2. How do you make a pill button at any height? → `border-radius: 999px` (or any value ≥ half the height).
3. Which focus indicator survives forced-colours mode? → `outline` (not `box-shadow`).

11.7 Shadows

`box-shadow`

```
box-shadow: [inset] <offset-x> <offset-y> [<blur>] [<spread>] <colour>, … ;
```

Part	Effect
offset-x / offset-y	Moves the shadow right/down (negative = left/up) — implies the light direction
blur	Softness; 0 = hard edge
spread	Grows (positive) or shrinks (negative) the shadow before blurring
`inset`	Draws inside the box (pressed/sunken look)
multiple, comma-separated	First one on top; layer a tight dark shadow and a large soft one for realistic depth

Elevation systems (Material Design — Google's design system — and most component libraries) express how "high" a surface floats with a set of increasingly large, soft, offset shadows:

```
.elev1 { box-shadow: 0 1px 2px rgb(0 0 0 / .12), 0 1px 3px rgb(0 0 0 / .08); }
.elev3 { box-shadow: 0 4px 6px -1px rgb(0 0 0 / .1), 0 10px 20px -5px rgb(0 0 0 / .15); }
```

The negative spread keeps the large soft shadow from peeking out at the sides. Shadows don't affect layout and are painted outside the box (so `overflow: hidden` on a **parent** clips them).

`text-shadow`

`text-shadow: <x> <y> [<blur>] <colour>, …` — no spread, no inset. Use for legibility on busy images (a soft dark halo) or for display effects; keep body text shadow-free.

`filter: drop-shadow()`

`box-shadow` follows the **box** (a rectangle, or its rounded corners). `filter: drop-shadow(x y blur colour)` follows the element's actual **alpha shape** (the outline of its non-transparent pixels) — the star in Figure 11.6 gets a star-shaped shadow. It has no spread or inset, and costs more to render.

Performance

Large blurred shadows are expensive to paint; **animating** `box-shadow` repaints every frame. To animate elevation, put the bigger shadow on a pseudo-element (a CSS-generated box such as `::after`) and animate its `opacity` instead (Chapter 12).

Self-check

1. What does a negative spread do? → Shrinks the shadow before blur, so it doesn't extend beyond the box's sides.
2. Shadow for a transparent PNG logo that follows its outline? → `filter: drop-shadow(…)`.

11.8 Filters, blend modes, clipping and masks

Figure 11.7 — `code/07_filters_blend.html`.

`filter`

Applies image-processing functions to an element **and everything inside it**, in the order written:

Function	Example	Effect
`blur()`	`blur(3px)`	Gaussian blur (a smooth, even blur)
`brightness()`	`brightness(1.4)`	< 1 darker, > 1 brighter
`contrast()`	`contrast(1.8)`	
`grayscale()`	`grayscale(1)`	0–1 (or %)
`hue-rotate()`	`hue-rotate(120deg)`	Rotates hues
`invert()`	`invert(1)`	
`opacity()`	`opacity(.5)`	Like the `opacity` property
`saturate()`	`saturate(3)`	
`sepia()`	`sepia(1)`	
`drop-shadow()`	11.7	
`url(#svgFilter)`		Any SVG filter

Baseline widely available since 2016. Uses: greyscale logos that gain colour on hover, dimming images in dark mode, disabled states.

`backdrop-filter`

Filters whatever is **behind** the element (the element must be at least partly transparent to see it) — the "frosted glass" of macOS and iOS panels: `background: rgb(255 255 255 / .3); backdrop-filter: blur(6px) saturate(1.4);`. Baseline since September 2024 (Safari 18 dropped the `-webkit-` prefix requirement). Check text contrast against the *worst* background that can appear behind it, and provide a more opaque fallback for `prefers-reduced-transparency` where supported.

Blend modes

- `mix-blend-mode` — how the element's pixels combine with the content **behind** it.

- `background-blend-mode` — how an element's **own background layers** (images, gradients, colour) combine with each other; used for duotone images (pictures recoloured in two tones).
- `isolation: isolate` — creates a new stacking context (a group of elements layered as one unit) so blending stays inside a group (§7.8).

Main modes, per colour channel with values 0–1 (a = top, b = backdrop): `multiply` = a·b (always darker; white disappears), `screen` = 1 − (1−a)(1−b) (always lighter; black disappears), `overlay`, `darken`/`lighten`, `difference` = |a − b|, `luminosity`/`color`/`hue`/`saturation` (mix HSL-like components). `code/07_filters_blend.mjs` blends #ef4444 over a sky blue and compares the screenshot pixel with the formula:

```
multiply   by hand rgb(94, 40, 63)  browser rgb(94, 40, 63)
screen     by hand rgb(245, 178, 240)  browser rgb(245, 178, 240)
difference by hand rgb(139, 82, 167)  browser rgb(139, 82, 167)
```

Clipping and masking

- `clip-path` hides everything outside a shape: `circle(40% at 50% 50%)`, `ellipse()`, `inset(10px round 8px)`, `polygon(0 0, 100% 0, 100% 75%, 0 100%)`, `path("…")`, or an SVG `url(#clip)`. The shape is hard-edged; clicks outside it pass through. Baseline since 2021 (animatable between shapes with the same number of points).
- `mask-image` uses an image's **alpha** (or luminance) as a soft mask: `mask-image: linear-gradient(to right, #000 40%, transparent)` fades an element out. Unprefixed masks are Baseline since December 2023.

Side effects to remember

A `filter`, `backdrop-filter`, `mix-blend-mode`, `clip-path`, `mask` or `opacity < 1` **creates a stacking context** (§7.8); a non-`none` `filter` or `backdrop-filter` also becomes the **containing block for** `position: fixed` **descendants** — a "fixed" header inside a filtered parent scrolls with it. Filters and backdrop filters are GPU (graphics chip) work; large blurred areas on low-end phones can drop frames (skip screen updates, so animation stutters).

Self-check

1. Which blend mode makes white disappear? → `multiply`.
2. What's the difference between `filter` and `backdrop-filter`? → `filter` changes the element itself; `backdrop-filter` changes what shows through it.
3. Why might `position: fixed` stop working inside a card? → An ancestor has `filter`/`transform`/`backdrop-filter`, which becomes the containing block.

11.9 `object-fit` and `object-position`

What it is

For **replaced elements** (elements whose content comes from an external file or source, not from CSS) — `<img>`, `<video>`, `<canvas>`, `<iframe>` content — `object-fit` decides how the content fills the element's box when their aspect ratios differ. It's the `background-size` of images. Baseline widely available (2020).

Value	Scale factor	Result
`fill` (default)	separate x and y	Stretched to the box — **distorted**
`contain`	min(box/content) per axis	Whole image visible, **letterboxed** (empty bars fill the leftover space)
`cover`	max(box/content) per axis	Box completely filled, overflow **cropped**
`none`	1	Natural size, centred, cropped
`scale-down`	the smaller of `none` and `contain`	Never enlarges

`object-position` (default `50% 50%`) chooses which part stays visible when cropping or where the letterboxed image sits — like `background-position`.

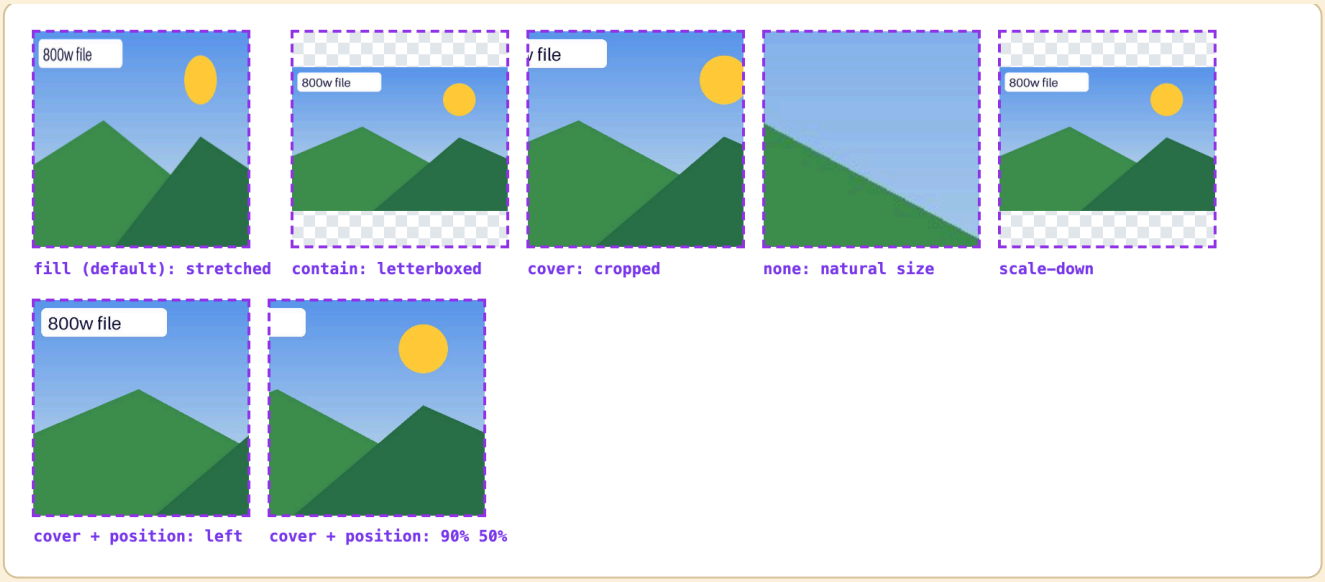


Figure 11.8 — `code/08_object_fit.html`. Dashed purple = the element's box; the checkerboard shows any empty space.

Output of `code/08_object_fit.mjs`, with the maths done by hand from the natural size the browser reports:

```
fill         box 150×150  natural 800×533  object-fit fill       object-position 50% 50%
...
cover-right  box 150×150  natural 800×533  object-fit cover      object-position 90% 50%

contain: scale = min(150/800, 150/533) = 0.1875 -> drawn 150×99.94 (bars above and below)
cover:   scale = max(150/800, 150/533) = 0.2814 -> drawn 225.14×150, 75.14px cropped horizontally
object-position 90% 50% on cover: image x offset = (150 - 225.14) × 0.9 = -67.63px
scale-down: the smaller of none (scale 1) and contain (0.1875) -> same as contain here
```

A percentage position means "align that percentage point of the image with the same percentage point of the box", so the offset is (box − image) × percentage — the same rule as `background-position`.

Real-world use

- **Card thumbnails / avatars** of unpredictable shape: `width: 100%; aspect-ratio: 4 / 3; object-fit: cover;` (and `border-radius: 50%` for avatars).
- **Product photos** that must not be cropped: `object-fit: contain` with a neutral background.
- **Hero video**: `<video autoplay muted loop playsinline>` with `object-fit: cover` to fill a full-width banner.
- **Grid cells**: images inside grid cells don't stretch by default (§9.10); `width: 100%; height: 100%; object-fit: cover` fills them.
- When cropping could cut off the subject (faces), set `object-position` per image, e.g. with an inline `style="object-position: 30% 20%"` from a CMS (content management system — the app editors use to manage site content) focal point (the spot an editor marked as most important).

Common mistakes

- Setting only `width` on an `<img>` and expecting `object-fit` to do anything — the box must have both dimensions (via `height` or `aspect-ratio`) that differ from the image's ratio.
- Using `object-fit` on a `<div>` — it only applies to replaced elements; use `background-size` there.
- Cropping important content (text in images) with `cover`.

Self-check

1. An 800×400 image in a 200×200 box with `cover`: drawn size? → scale = max(0.25, 0.5) = 0.5 → 400×200, 200px cropped horizontally.
2. Same with `contain`? → scale 0.25 → 200×100, 50px bars above and below.
3. `object-position: 0% 50%` keeps which part? → The left edge (vertically centred).

Chapter 11 TL;DR

- **Fonts**: stacks end with a generic family; `system-ui` stack = 0 bytes. `@font-face` + **WOFF2**; one **variable font** (`font-weight: 100 900`) replaces many files — heavier weights really are different glyphs (307.5 → 353.5px wide). Missing faces are **synthesized** unless `font-synthesis: none`.
- **Loading**: `font-display` — `block` (invisible ≤3 s), `swap` (fallback then swap), `fallback` (short swap window), `optional` (≈100 ms, then never — verified: loaded but not used). Preload 1–2 critical fonts **with** `crossorigin`, self-host, subset with `unicode-range`, match fallback metrics with `size-adjust`.
- **Text**: `rem` sizes; **unitless** `line-height` (a length is computed once and inherited — 24px for 32px text); measure 45–75ch; `text-wrap: balance` (Baseline 2024), `pretty` (not Baseline); ellipsis = nowrap + hidden + `text-overflow`; multi-line = `-webkit-line-clamp`; `tabular-nums` for numbers; `hyphens: auto` needs `lang`.
- **Contrast**: (L1+0.05)/(L2+0.05); AA 4.5:1 text, 3:1 large text and UI parts; AAA 7:1. #767676 is the lightest AA grey on white. Never colour alone. Derive shades with `color-mix(in oklch, …)` and relative colours `oklch(from var(--c) l c h)` (Baseline 2024) — and still check each pair.
- **Backgrounds**: layers, first on top; `cover`/`contain`; gradients linear/radial/conic/repeating, hard stops, `in oklch` avoids grey midpoints (measured grey (128,128,127) vs teal).
- **Borders**: style required; `border-radius` %, `999px` pills, `a / b` ellipses. Focus = `outline` (no layout, survives forced colours); rings = `box-shadow: 0 0 0 Npx`.
- **Shadows**: `[inset] x y blur spread colour`, layered for elevation; `text-shadow`; `drop-shadow()` follows alpha. Don't animate shadows directly.
- **Effects**: `filter` (itself), `backdrop-filter` (behind; Baseline 2024), `mix-blend-mode` (multiply = a·b, screen, difference — pixel-verified), `background-blend-mode`, `isolation`, `clip-path`, `mask-image`. They create stacking contexts; filters also capture `position: fixed`.
- **`object-fit`**: fill / contain (min scale) / cover (max scale) / none / scale-down + `object-position` — for img/video in fixed-ratio boxes.

Next: Chapter 12 — Transitions, Transforms and Animation.

Chapter 12 — Transitions, Transforms and Animation

Motion, used well, explains what just happened: a menu slides out of the button that opened it, a card grows into the detail view, a spinner says "working". Used badly it distracts, makes some people physically ill, and makes pages janky (**jank** is visible stutter: the browser misses frames, so motion jumps instead of flowing). CSS gives you three tools — **transforms** (move, rotate, scale an element without touching layout), **transitions** (animate a change from one state to another) and **keyframe animations** (a timeline of states) — plus newer ones for entering/leaving elements, whole-page **view transitions** and **scroll-driven** animation. This chapter explains each, shows how the browser computes every in-between frame, and ends with performance and **accessibility** (making pages usable by people with disabilities, including those who use keyboards, screen readers or motion settings) rules.

What you'll learn

#	Topic	You will be able to…
12.1	Transitions	animate state changes and know which properties can be animated
12.2	Timing functions	read and write `ease`, `cubic-bezier()`, `steps()` and `linear()` — and compute them by hand
12.3	2D transforms	translate, rotate, scale, skew; `transform-origin`; order of functions; individual properties
12.4	3D transforms	use `perspective`, `preserve-3d` and `backface-visibility` for a card flip
12.5	Keyframe animations	write `@keyframes` and every `animation-*` property, especially `fill-mode`
12.6	The Web Animations API	control animations from JavaScript (`element.animate()`, `getAnimations()`)
12.7	Performance	explain why `transform`/`opacity` are cheap and `left`/`width` are not — with measurements
12.8	Enter and exit animations	animate from `display: none` with `@starting-style` and `allow-discrete`
12.9	Reduced motion and accessibility	honour `prefers-reduced-motion` and the WCAG motion rules
12.10	View transitions	morph between two states of the DOM (the page's live tree of elements) (and between pages)
12.11	Scroll-driven animations	link an animation to scroll position — and know it isn't Baseline

How the figures and numbers were made. Every example is a page in `code/`. Animations are hard to show on paper, so `code/make_figures.mjs` makes **filmstrips**: it pauses every animation on the page with the Web Animations API (an **API**, application programming interface, is a set of functions one program offers to another; this one lets JavaScript control animations) (CSS transitions and animations are exposed as `CSSTransition` / `CSSAnimation` objects in `document.getAnimations()`), sets their `currentTime` to an exact moment, and screenshots — so each frame shows exactly t = 0, 200, 400 … ms, with no timing luck. The same trick, in `code/anim.mjs`, lets the `NN_*.mjs` scripts (`.mjs` = a JavaScript module file, run here with Node.js, which drives a **headless** Chrome — a real browser with no visible window — through the Puppeteer library) read **computed values mid-animation** (the final, resolved value the browser actually uses for a property) and compare them with the maths. Screenshots and measurements use `code/measure.mjs`, a copy of Chapter 10's helper.

```
// code/anim.mjs — freeze every animation at the same moment
export const seek = (page, ms) => page.evaluate(ms => {
  for (const a of document.getAnimations()) { a.pause(); a.currentTime = ms; }
  return new Promise(r => requestAnimationFrame(() => requestAnimationFrame(r)));   // let the frame render
}, ms);
```

12.1 Transitions

What it is

A **transition** tells the browser: "when this property's value changes, don't jump — move smoothly from the old value to the new one over this time". Something else changes the value (a `:hover` or `:focus-visible` rule — pseudo-classes that match while the pointer is over the element or it has keyboard focus — a class added by JavaScript, a media query, i.e. an `@media` rule that applies only under conditions like screen width); the transition only animates the change.

```
.button {
  background: #2563eb;
  transition: background-color 200ms ease-out, translate 150ms ease-out;
}
.button:hover { background: #1d4ed8; translate: 0 -2px; }
```

How it works

1. The browser computes the element's style before and after the change.
2. For each property listed in `transition-property` whose value changed **and can be interpolated** (the browser can compute sensible in-between values, as with 10px → 20px), it creates a transition from the old to the new value.
3. On each **frame** (one redraw of the screen, typically 60 per second) it computes *progress* = timing-function(elapsed / duration) and sets the property to old + (new − old) × progress.
4. If the value changes again mid-way (mouse leaves early), the running transition is replaced by one starting from the **current** value. Reversing a half-finished transition is also **shortened** proportionally, so un-hovering after 50ms doesn't take the full duration.

The properties

Longhand	Example	Meaning
`transition-property`	`opacity, translate` | `all`	What to animate (`all` = every animatable property — convenient but can animate things you didn't intend)
`transition-duration`	`200ms`	How long (default `0s` = no transition)
`transition-timing-function`	`ease-out`	Speed curve (12.2); default `ease`
`transition-delay`	`100ms`	Wait before starting (negative = start part-way through)
`transition-behavior`	`allow-discrete`	Also transition **discrete** properties like `display` (12.8)
`transition`	`opacity 200ms ease-out 50ms`	Shorthand; first time = duration, second = delay; comma-separate several

What can be animated

- **Interpolable** values: lengths, percentages, numbers, colours, transforms, shadows, gradients' colour stops (not the whole gradient in most browsers), `grid-template-columns` track sizes (Baseline since 2022), `clip-path` shapes with the same number of points…
- **Discrete** properties (`display`, `visibility`, `font-family`, `position`…) can't be in-betweened. Normally they just flip; with `transition-behavior: allow-discrete` they flip at 50% of the time — except `display`/`content-visibility`, which stay visible for the whole transition (12.8).
- `auto` is not interpolable: `height: 0 → auto` jumps. Chromium's (Chromium is the open-source browser engine behind Chrome, Edge and Opera) new `interpolate-size: allow-keywords` / `calc-size()` fix this but are **not Baseline**; portable alternatives are animating `grid-template-rows: 0fr → 1fr` on a wrapper, or measuring the height in JS (JavaScript).

> **Key term — Baseline:** a label used by MDN and web.dev for web features that work in the current versions of all the major browsers (Chrome, Edge, Firefox and Safari, desktop and mobile). *Newly available* means the last of those browsers has just shipped it; *widely available* means it has been everywhere for 30

> months, so most users' browsers have it. "Not Baseline" means at least one major browser is missing it, so you need a fallback.

Events

These are **events** — named notifications the browser sends that JavaScript can listen for with `addEventListener`. `transitionrun`, `transitionstart` (after the delay), `transitionend` (once **per property**), `transitioncancel` (interrupted). Filter by `event.propertyName` so a `transitionend` of `background-color` doesn't close your menu early.

Common mistakes

- Putting `transition` only in the `:hover` rule — then there is a transition *into* hover but the change back is instant. Put it on the base rule.
- `transition: all 300ms` on elements whose layout properties change — animates widths and paddings you never meant to (and expensively, 12.7).
- Expecting a transition when the element is **added** to the page or goes from `display: none` — there is no "before" style, so nothing animates. That's what `@starting-style` solves (12.8).
- Durations too long: UI (user interface) feedback feels right around **100–300ms**; larger movements up to ~500ms.

Self-check

1. Where should `transition` be declared for hover-in and hover-out to animate? → On the base (non-hover) rule.
2. Why doesn't `height: 0 → auto` animate? → `auto` isn't an interpolable value (use the grid `0fr → 1fr` trick, JS, or `interpolate-size` where supported).
3. How many `transitionend` events does `transition: opacity .2s, translate .2s` fire? → Two, one per property.

12.2 Timing functions (easing)

What it is

A **timing function** (or *easing*) maps **time progress** (0 → 1) to **output progress** (usually 0 → 1). Linear motion looks mechanical; real objects accelerate and decelerate. CSS offers four kinds:

Function	Shape	Typical use
`linear`	straight line	spinners, progress, colour cycling
`ease` (default)	quick start, long gentle stop	general purpose
`ease-in`	slow start, fast end	things **leaving** the screen
`ease-out`	fast start, slow end	things **entering** or responding to input (feels snappy)
`ease-in-out`	slow–fast–slow	moving between two on-screen positions
`cubic-bezier(x1, y1, x2, y2)`	any smooth curve; y may go outside 0–1 for overshoot	brand motion, "back" effects
`steps(n, jump-start | jump-end | jump-none | jump-both)`	n discrete jumps	sprite sheets (one image holding every frame), typewriter, ticking clocks
`linear(0, 0.36 30%, 1 45%, …)`	straight lines through points you list	bounces and springs approximated with many points (Baseline since December 2023)

The keywords are shortcuts: `ease` = `cubic-bezier(.25, .1, .25, 1)`, `ease-in` = `(.42, 0, 1, 1)`, `ease-out` = `(0, 0, .58, 1)`, `ease-in-out` = `(.42, 0, .58, 1)`; `step-start` = `steps(1, jump-start)`, `step-end` = `steps(1, jump-end)`.

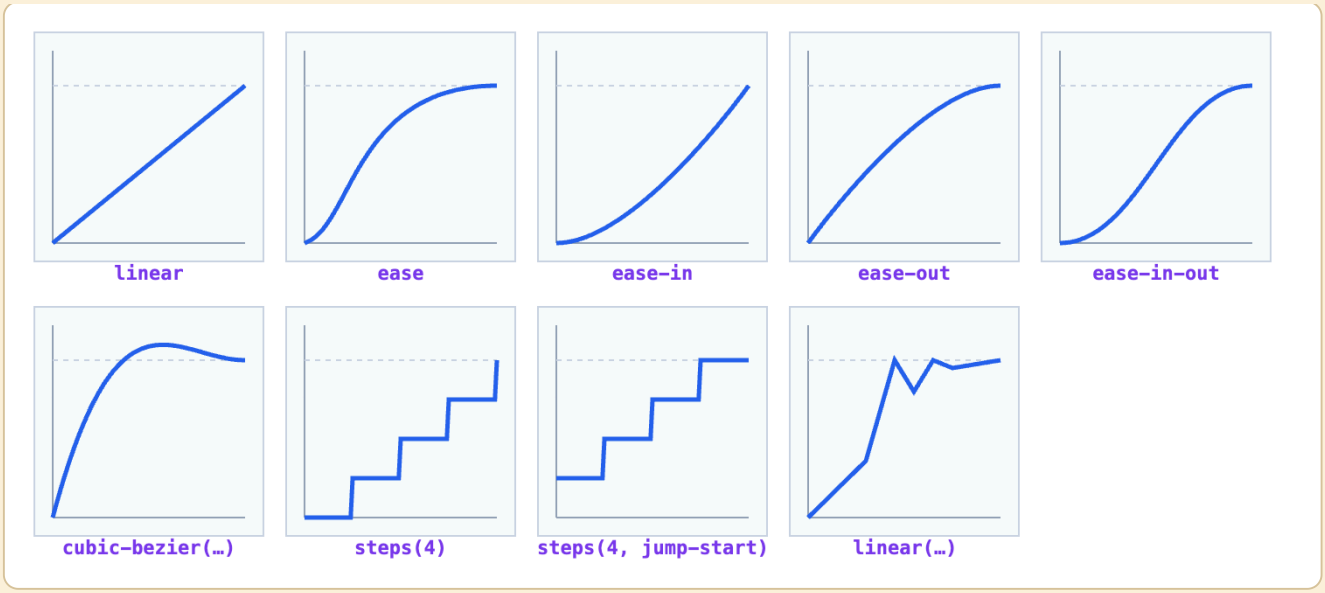


Figure 12.1 — `code/02_curves.html` draws each curve by animating a probe element with that easing, pausing it at 101 moments, and reading the computed value back — so these are Chrome's curves, not drawings of them. The dashed line is 100%.

How a cubic Bézier is computed

A `cubic-bezier(x1, y1, x2, y2)` is a curve from (0, 0) to (1, 1) with two control points. Both coordinates are functions of a curve parameter *s*: `x(s) = 3·x1·s·(1−s)² + 3·x2·s²·(1−s) + s³` and the same for `y` with `y1`, `y2`. To evaluate the easing at time *t*, the browser must **solve x(s) = t** for *s* (x must stay within 0–1 so there is one answer), then return y(s). `code/01_timing.mjs` does exactly that with **bisection** (guess the middle of the range, keep whichever half contains the answer, repeat until close enough) and compares with the browser.

The proof: a race

`code/01_timing.html` transitions eight balls by 160px over 1000ms, each with a different timing function:

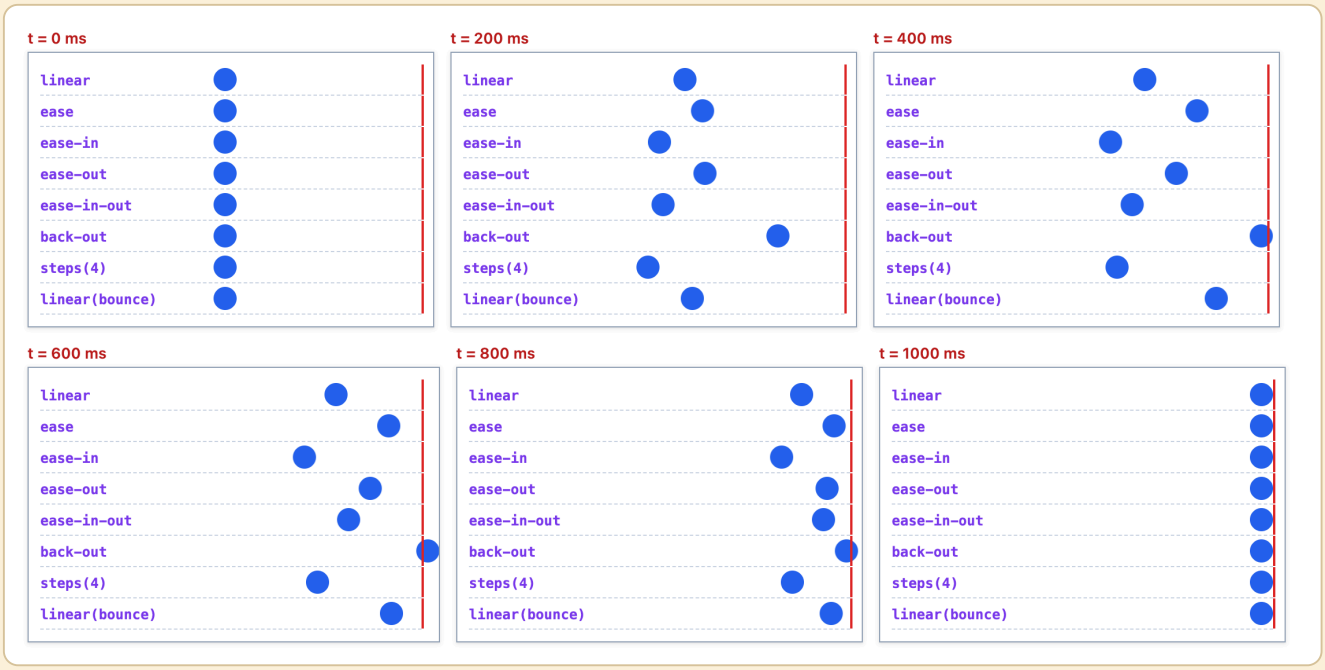


Figure 12.2 — Filmstrip made by pausing all eight transitions and seeking them to each time.

```
progress (0 = start, 1 = end) at t =        0ms   200ms   400ms   600ms   800ms  1000ms
linear          browser       0.000   0.200   0.400   0.600   0.800   1.000
                our maths     0.000   0.200   0.400   0.600   0.800   1.000
ease            browser       0.000   0.295   0.683   0.885   0.976   1.000
                our maths     0.000   0.295   0.683   0.885   0.976   1.000
ease-in         browser       0.000   0.062   0.215   0.429   0.692   1.000
                our maths     0.000   0.062   0.215   0.429   0.692   1.000
ease-out        browser       0.000   0.308   0.571   0.785   0.938   1.000
                our maths     0.000   0.308   0.571   0.785   0.938   1.000
ease-in-out     browser       0.000   0.082   0.332   0.668   0.918   1.000
                our maths     0.000   0.082   0.332   0.668   0.918   1.000
back-out        browser       0.000   0.703   1.030   1.097   1.042   1.000
                our maths     0.000   0.703   1.030   1.097   1.042   1.000
steps(4)        browser       0.000   0.000   0.250   0.500   0.750   1.000
                our maths     0.000   0.000   0.250   0.500   0.750   1.000
linear(bounce)  browser       0.000   0.240   0.787   0.900   0.960   1.000
                our maths     0.000   0.240   0.787   0.900   0.960   1.000
```

Our straight-from-the-formula numbers match Chrome to three decimals. Things to read off the table:

- `ease` has already covered 68% of the distance at 40% of the time — that's why it feels quick.
- `back-out` = `cubic-bezier(.34, 1.56, .64, 1)`: y1 = 1.56 > 1, so progress goes past 1 (109.7% at 600ms) and settles back — an overshoot.
- `steps(4)` uses `jump-end` by default: the value changes at the *end* of each quarter, so at 200ms it's still 0.
- `linear(0, 0.36 30%, 1 45%, 0.8 55%, …)`: between listed points progress is interpolated linearly; at 600ms (between 55% → 0.8 and 65% → 1) it's 0.9.

Choosing

- Entering / responding to the user: `ease-out` (or a custom `cubic-bezier(.2, .8, .2, 1)`-style curve).
- Leaving: `ease-in`, and shorter than the entrance.
- On-screen moves: `ease-in-out`.
- Continuous rotation: `linear`.
- Design systems (a team's shared set of reusable styles and components) define a few named easing/duration **tokens** (design decisions stored as named variables) (`--ease-out`, `--dur-fast: 150ms`) and use only those.

Self-check

1. Which keyword makes an element start fast and land gently? → `ease-out`.
2. Why can `y1` be 1.56 but `x1` must be within 0–1? → x is time, which must move forward and map to a single progress value; y is output, which may overshoot.
3. `steps(2)` over 1s: value at 0.6s? → 0.5 (jump-end: jumps at 0.5s and 1s).

12.3 2D transforms

What it is

The `transform` property changes where an element is **painted** (drawn as pixels) — moved, rotated, scaled, skewed — **without changing its layout box** (the rectangle of space the element occupies when the page is arranged). Neighbours don't move; the space it occupied in the flow stays reserved. That's what makes transforms ideal for animation.

Function	Example	Effect
`translate(x, y)` / `translateX()` / `translateY()`	`translate(40px, 20px)`	Move; `%` is relative to the element's **own** size — `translate(-50%, -50%)` centres an element on a point
`rotate(angle)`	`rotate(30deg)`, `0.25turn`	Clockwise rotation
`scale(sx, sy)`	`scale(1.4, 0.6)`	Resize (1 = unchanged; negative flips)
`skew(ax, ay)` / `skewX()`	`skewX(-20deg)`	Slant
`matrix(a, b, c, d, e, f)`	—	The general form everything above reduces to (six numbers encoding the whole transform)

`transform-origin` (default `50% 50%` — the centre) is the fixed point for rotate, scale and skew.

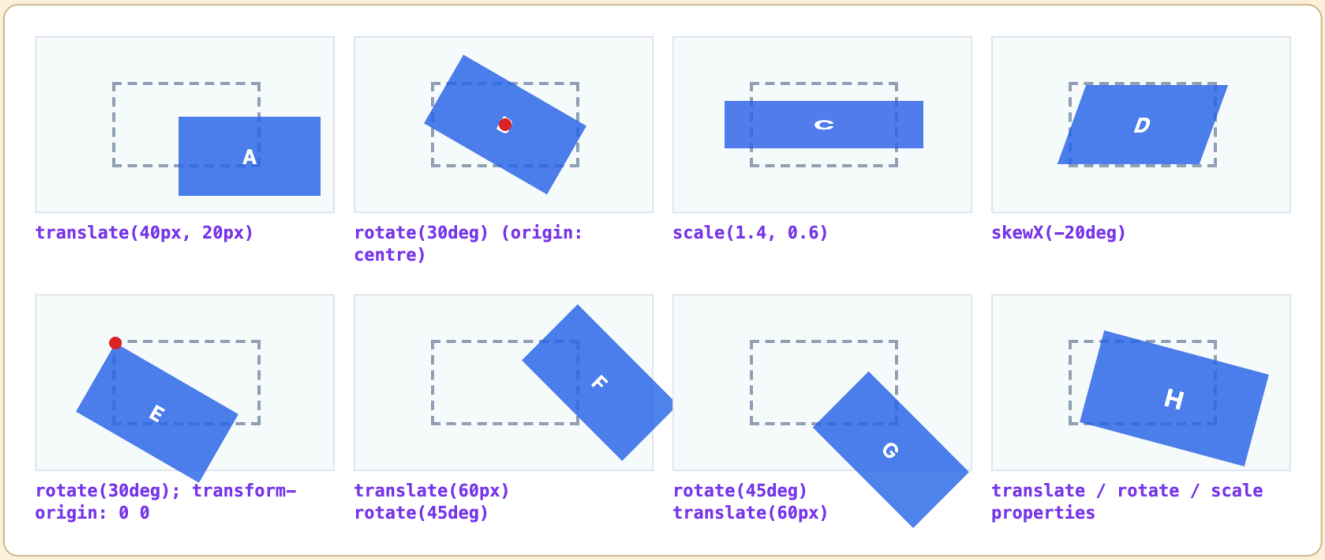


Figure 12.3 — `code/03_transforms.html`. Red dots mark the transform origin.

Order matters

`transform: translate(60px, 0) rotate(45deg)` and `rotate(45deg) translate(60px, 0)` differ (F and G above). Two equivalent ways to think about it:

- Read **right to left** as operations on the element: F = first rotate, then move right 60px.
- Or read **left to right** as moving the **coordinate system**: G = rotate the axes 45°, then move 60px along the *rotated* x-axis — diagonally.

`code/03_transforms.mjs` prints the layout box (unchanged for all of them), the painted **bounding box** (the smallest upright rectangle that encloses the drawn shape) and the resolved matrix:

```
A t1: layout 50,30 90x50 | painted bbox 90,50 90x50 | transform matrix(1, 0, 0, 1, 40, 20)
B t2: layout 50,30 90x50 | painted bbox 43.5,10.8 102.9x88.3 | transform matrix(0.866025, 0.5, -0.5, 0.866025, 0, 0)
C t3: layout 50,30 90x50 | painted bbox 32,40 126x30 | transform matrix(1.4, 0, 0, 0.6, 0, 0)
D t4: layout 50,30 90x50 | painted bbox 40.9,30 108.2x50 | transform matrix(1, 0, -0.36397, 1, 0, 0)
E t5: layout 50,30 90x50 | painted bbox 25,30 102.9x88.3 | transform matrix(0.866025, 0.5, -0.5, 0.866025, 0, 0)
F t6: layout 50,30 90x50 | painted bbox 105.5,5.5 99x99 | transform matrix(0.707107, 0.707107, -0.707107, 0.707107, 60, 0)
G t7: layout 50,30 90x50 | painted bbox 87.9,47.9 99x99 | transform matrix(0.707107, 0.707107, -0.707107, 0.707107,
42.4264, 42.4264)
H t8: layout 50,30 90x50 | painted bbox 55.1,22 119.8x85.9 | transform none
H individual properties: translate 20px 10px, rotate 15deg, scale 1.2 (transform stays none)
```

Check B by hand (cos and sin are the trigonometry functions that give the horizontal and vertical share of a length tilted by an angle): a 90×50 box rotated 30° has a bounding box 90·cos30° + 50·sin30° = 77.9 + 25 = **102.9** wide and 90·sin30° + 50·cos30° = 45 + 43.3 = **88.3** tall ✓. In a `matrix(a, b, c, d, e, f)`, (a, b, c, d) = (cos θ, sin θ, −sin θ, cos θ) for a rotation, and (e, f) is the translation: G's is 60·cos45° = **42.43** in both x and y — the 60px move along the rotated axis ✓. B and E share a matrix; only `transform-origin` differs.

Individual transform properties

`translate`, `rotate` and `scale` are also **separate properties** (Baseline since 2022, widely available). They are applied in a fixed order — translate, then rotate, then scale, then `transform` — so the order-of-functions surprise can't happen, and each can be transitioned or animated **independently** (a hover `scale` doesn't clobber an animation's `translate`). They don't show up in `transform`, as H's `transform none` shows. This chapter's pages use them wherever possible.

Side effects

- A transform other than `none` creates a **stacking context** (a self-contained group whose children are layered by `z-index` only among themselves) and becomes the **containing block** (the box that positioned descendants measure their offsets from) for `position: fixed` and `absolute` descendants (§7.6, §7.8).
- Transforms don't apply to non-replaced **inline** boxes (a plain `<span>`; "non-replaced" means its content is the page's own text, unlike an `<img>` whose content comes from an external file); make it `inline-block`.
- Scaling text up can look blurry mid-animation on some GPUs (graphics processing units — the chip specialised in drawing pixels); prefer animating to a crisp final state.
- `getBoundingClientRect()` (a JavaScript method that reports an element's on-screen rectangle) returns the **painted** box (transform included); `offsetWidth`/`offsetLeft` the layout box.

Self-check

1. Does `translate: 0 -4px` on hover move the elements below? → No — transforms don't affect layout.
2. How do you centre an absolutely positioned element of unknown size? → `top: 50%; left: 50%; translate: -50% -50%;` (or grid/flex centring).
3. `rotate(90deg) translateX(100px)` moves the element which way? → Down (the x-axis now points down).

12.4 3D transforms

What it is

3D transforms add a **z-axis** pointing toward the viewer: `translateZ()`, `rotateX()`, `rotateY()`, `rotate3d()`, `scale3d()`, `matrix3d()`, and the `rotate: y 60deg` form of the individual property. To look 3D they need **perspective**.

Property	On	Meaning
`perspective: 600px`	the **parent** (scene)	Distance from the viewer to the z = 0 plane; smaller = stronger distortion. All children share one vanishing point (the spot where receding lines meet).
`perspective()` function	the element's own `transform`	Per-element perspective (each has its own vanishing point)
`perspective-origin`	the parent	Where the viewer's eye is (default centre)
`transform-style: preserve-3d`	a parent whose children should stay in 3D	Otherwise children are flattened into its plane
`backface-visibility: hidden`	a face	Hide the element when its back faces the viewer

Figure 12.4 — `code/04_3d.html`.

The flip card, step by step

```
.scene { perspective: 600px; }                                  /* 1. give the scene depth */
.card  { position: relative; transform-style: preserve-3d;       /* 2. keep both faces in 3D */
         transition: rotate 600ms ease-in-out; }
.face  { position: absolute; inset: 0; backface-visibility: hidden; }   /* 3. hide a face pointing away */
.back  { rotate: y 180deg; }                                     /* 4. the back starts facing away */
.scene:hover .card, .card.flipped { rotate: y 180deg; }          /* 5. flip the whole card */
```

Past 90° the front face points away (hidden) and the back face — pre-rotated 180° — now points at the viewer. In a real component, flip on a **button** press (toggle `.flipped`), not only on hover, so keyboard and touch users can use it.

Things that flatten 3D

Some properties force an element to render its children into a flat image, breaking `preserve-3d`: `overflow` other than `visible`, `opacity < 1`, `filter`, `clip-path`, `mask`, `mix-blend-mode`, `isolation: isolate`. If your cube collapses, look for one of these on the `preserve-3d` element.

Self-check

1. Perspective on the parent vs the `perspective()` function? → Parent: one shared vanishing point for all children; function: each element gets its own.
2. Why add `backface-visibility: hidden` to both faces? → So each face disappears when turned away, letting the other show.

12.5 Keyframe animations

What it is

A **keyframe animation** runs through a named sequence of states, independent of any state change — it can start on load, loop forever, alternate and be delayed.

```
@keyframes bounce {
  0%, 100% { translate: 0 -12px; animation-timing-function: ease-in; }   /* falling: speed up   */
  50%      { translate: 0 12px;  animation-timing-function: ease-out; }  /* rising: slow down   */
}
.ball { animation: bounce 1000ms infinite; }

@keyframes spin { to { rotate: 1turn; } }        /* missing "from" = the element's own current value */
.spinner { animation: spin 1s linear infinite; }
```

The `animation-*` properties

Longhand	Values	Notes
`animation-name`	`bounce`	The `@keyframes` name
`animation-duration`	`1s`	One iteration
`animation-timing-function`	`ease` …	Applies **between each pair of keyframes**, not across the whole animation; can be overridden per keyframe (as in `bounce`)
`animation-delay`	`500ms`, `-250ms`	A **negative** delay starts part-way through — handy for staggering many spinners
`animation-iteration-count`	`3`, `infinite`, `0.5`	
`animation-direction`	`normal` | `reverse` | `alternate` | `alternate-reverse`	`alternate` plays every other iteration backwards
`animation-fill-mode`	`none` | `forwards` | `backwards` | `both`	What applies **before** (during the delay) and **after** the animation
`animation-play-state`	`running` | `paused`	Pause/resume from CSS
`animation-composition`	`replace` | `add` | `accumulate`	How it combines with the underlying value (Baseline 2023)
`animation-timeline`	`auto` | `scroll()` | `view()`	Drive it by scroll instead of time (12.11)
`animation`	`bounce 1s ease-in-out 200ms infinite alternate both`	Shorthand; first time = duration, second = delay

Keyframe rules: `from` = `0%`, `to` = `100%`; a missing 0% or 100% uses the element's normal value; the same percentage can be listed together (`0%, 100%`); `!important` inside keyframes is ignored. While running, animation values beat normal declarations but lose to `!important` ones (§6.5).

The proof: values mid-animation

`code/05_keyframes.html` runs a bounce, a spinner, an alternating slide and four copies of a colour animation (`red → green`, 500ms, **`after a 500ms delay`**) with different fill modes. `code/05_keyframes.mjs` seeks them all to chosen times:

```
t=   0ms bounce 0px -12px spin 0deg     slide 0px       | fill none=grey (own style), forwards=grey (own style),
backwards=red, both=red
t= 250ms bounce 0px -4.43144px spin 90deg    slide 20.6659px | fill none=grey (own style), forwards=grey (own style),
backwards=red, both=red
t= 499ms bounce 0px 11.9179px spin 179.64deg slide 79.7241px | fill none=grey (own style), forwards=grey (own style),
backwards=red, both=red
t= 750ms bounce 0px -4.43144px spin 270deg   slide 139.334px | fill none=#796538, forwards=#796538, backwards=#796538,
both=#796538
t=1000ms bounce 0px -12px spin 0deg     slide 160px     | fill none=grey (own style), forwards=green, backwards=grey (own
style), both=green
t=1500ms bounce 0px 12px  spin 180deg   slide 80px      | fill none=grey (own style), forwards=green, backwards=grey (own
style), both=green
events on .fm.both (restarted): animationstart after ~500ms, animationend after ~1000ms
```

Reading it:

- **Bounce at 250ms**: the 0% → 50% segment lasts 500ms with `ease-in`; 250ms is half of it; ease-in(0.5) = 0.315; −12 + 24 × 0.315 = **−4.43px** ✓. At 750ms the 50% → 100% segment uses `ease-out`: 12 − 24 × 0.685 = **−4.43px** ✓ — per-keyframe easing in action.
- **Slide** (`ease-in-out`, alternate): 250ms → ease-in-out(0.25) = 0.129 × 160 = **20.67px** ✓; at 1500ms the second iteration runs backwards, half-way → **80px** ✓.
- **fill-mode** — the most misunderstood property:

Mode	During the delay (0–500ms)	After the end (≥ 1000ms)
`none`	element's own style (grey)	own style (grey)
`forwards`	own style	**last keyframe** (green)
`backwards`	**first keyframe** (red)	own style
`both`	first keyframe	last keyframe

- At 750ms, all four show **#796538**, the halfway mix of #dc2626 and #16a34a: (220 + 22)/2 = 121 = 0x79, (38 + 163)/2 ≈ 101 = 0x65, (38 + 74)/2 = 56 = 0x38. Legacy `rgb`/hex colours are interpolated in sRGB (the standard colour space of the web). **Hex** is short for hexadecimal, base-16 counting with digits 0–9 and a–f, where `0x79` means 7 × 16 + 9 = 121.

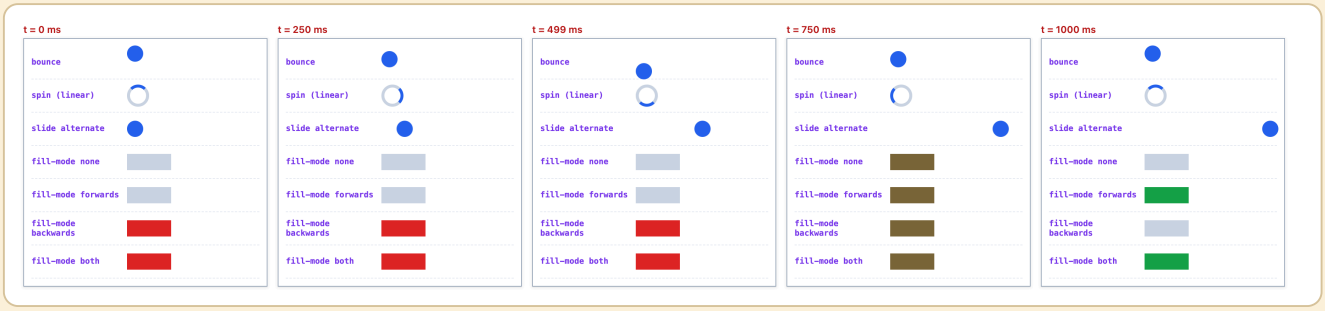


Figure 12.5 — Filmstrip of `code/05_keyframes.html`.

Events

`animationstart` (after the delay), `animationiteration` (each loop), `animationend`, `animationcancel`. The last line of the output shows `animationstart` after the 500ms delay and `animationend` at 1000ms.

Transition or animation?

Use a `transition` when…	Use a `keyframe animation` when…
Animating between two states triggered by a change (hover, class, open/closed)	It must run by itself (on load, looping, attention-seeking)
Only a start and an end	You need intermediate steps
Should reverse naturally when the state flips back	You need iteration, direction, delay choreography

Self-check

1. An element fades in after a 1s delay but is visible during the delay. Fix? → `animation-fill-mode: backwards` (or `both`) so the 0% keyframe applies during the delay.
2. `animation: pulse 2s 500ms` — which is the delay? → `500ms` (the second time value).
3. Does `animation-timing-function` span the whole animation? → No, each keyframe-to-keyframe segment.

12.6 The Web Animations API (WAAPI)

What it is

The **Web Animations API** is the JavaScript interface to the same animation engine that runs CSS transitions and animations (Baseline widely available since 2020–2023). `element.animate(keyframes, options)` creates an animation; every animation, CSS or JS, is an `Animation` object you can pause, seek, speed up, reverse and await (a **Promise** is a JavaScript object standing for a result that arrives later; `await` pauses the code until it does).

```
const anim = box.animate(
  [{ translate: '0 0', opacity: 1 }, { translate: '200px 0', opacity: 0.5 }],   // keyframes
  { duration: 400, easing: 'ease-out', fill: 'forwards' });                     // options
anim.pause(); anim.currentTime = 200;        // scrub
anim.playbackRate = 2; anim.play();          // double speed
await anim.finished;                         // a Promise
anim.reverse();
document.getAnimations();                    // every running animation, CSS ones included
```

Output of `code/11_waapi.mjs` (page: `code/11_waapi.html`):

```
created: playState=running, listed by document.getAnimations(): true
paused at 200ms: translate=136.929px, opacity=0.658
finished: currentTime=400ms (the remaining 200ms took ~100ms of real time at 2x), playState=finished
fill: forwards keeps the end: translate=200px
after reverse(): translate=none  (back at the start; fill: forwards doesn't hold the START value)
```

ease-out(0.5) = 0.685 → 200 × 0.685 = 136.9px and opacity 1 − 0.5 × 0.685 = 0.658 ✓.

When to use it

- Values known only at runtime (animate from a clicked element's position — the FLIP technique in 12.7).
- Sequencing with promises (`await a.finished; b.play()`), scrubbing with a slider, syncing with media.
- Testing and screenshots — exactly how this chapter's figures were made.

Prefer CSS for simple, **declarative** (you describe the result, not the steps) states; WAAPI over hand-written `requestAnimationFrame` loops (`requestAnimationFrame` runs your function just before the next frame is drawn), because WAAPI animations of `transform`/`opacity` can run on the compositor (the browser part that assembles finished layers into the final image, 12.7) and are cancellable and inspectable in DevTools' (the developer tools built into the browser, opened with F12) Animations panel.

`anim.commitStyles()` writes the current animated values into the element's `style` attribute — useful before `cancel()` if you want to keep the end state without `fill: forwards` (fills keep an animation alive forever).

12.7 Performance: animate `transform` and `opacity`

How the browser renders a frame

Recall the pipeline from Chapter 1: **style → layout → paint → composite**. Each animation frame redoes the pipeline from the first stage the changed property affects:

> **Key term — the rendering pipeline: style** works out which CSS rules apply to each element; **layout** computes every box's size and position; **paint** draws the boxes' pixels (text, colours, shadows) into layers; **composite** stacks those layers into the final screen image, often on the GPU. Most of this runs on the **main thread** — the single line of work that also runs your JavaScript — so expensive layout competes with your code, while compositing can happen on a separate **compositor thread** (a thread is an independent sequence of work the computer runs alongside others).

Property changed	Needs	Examples	Cost
Geometry	style → **layout** → paint → composite	`width`, `height`, `top`/`left`, `margin`, `padding`, `font-size`	Highest: the element *and possibly everything around it* is re-laid out
Appearance	style → **paint** → composite	`color`, `background`, `box-shadow`, `border-radius`, `outline`	Medium: pixels re-drawn
Compositor-only	(style) → **composite**	`transform` (and `translate`/`rotate`/`scale`), `opacity`, often `filter`	Lowest: the GPU moves/fades an already painted layer; the main thread can even be busy

At 60 frames per second (fps) a frame has about **16.7ms**; on a 120 Hz screen (one that refreshes 120 times a second), 8.3ms. Layout of a big page can take several ms by itself, so layout animations drop frames ("jank").

Measured

`code/06_performance.mjs` runs four 1-second animations of the same box in `code/06_performance.html` and reads Chrome's internal counters (`page.metrics()`; a "style recalc" is one run of the style stage):

```
move-left       layouts  60   style recalcs  61   layout time 1.8ms
grow-width      layouts  60   style recalcs  61   layout time 1.8ms
move-transform  layouts   1   style recalcs   5   layout time 0.0ms
fade            layouts   1   style recalcs   5   layout time 0.0ms
```

Animating `left` or `width` ran layout on **every frame** (about 60 in one second); `transform` and `opacity` needed **one** layout in total — the compositor animated them without the main thread. (Counts and times are machine-dependent — the counts vary by ±1 between runs. Layout time is tiny here because the page is tiny; on a real page each of those 60 layouts could cost milliseconds.)

Techniques

- **Move with `translate`, not `top` / `left`; resize with `scale`, not `width` / `height`; fade with `opacity`.**
- **Expensive paints**: animate the `opacity` of a pseudo-element that already has the big shadow, instead of animating `box-shadow`.
- **FLIP** (First, Last, Invert, Play) for layout changes: measure the element's **F**irst position, apply the layout change, measure the **L**ast position, **I**nvert with a transform so it *looks* like it hasn't moved, then **P**lay a transform animation back to none. The layout happens once; the motion is a cheap transform. View transitions (12.10) automate this idea.
- `will-change: transform` hints that an element will animate, so the browser can promote it to its own **layer** (a separately painted image the compositor can move or fade without repainting) ahead of time. Use it sparingly and temporarily (add before, remove after): every layer costs GPU memory, and `will-change` also creates a stacking context.
- `contain: layout paint` (promises the browser that the element's insides don't affect the outside) / `content-visibility` (lets the browser skip rendering off-screen content) limit how far layout and paint spread (Chapter 14).
- **Measure**: DevTools Performance panel (look for purple Layout and green Paint blocks per frame), "Paint flashing" and "Layer borders" in the Rendering panel, and the Animations panel to slow animations down.

In the interview

30-second answer: "Each frame the browser does style, layout, paint and composite. Animating layout properties like `left` or `width` re-runs layout and paint every frame — I measured about 60 layouts per second for a `left` animation. `transform` and `opacity` can be handled by the compositor on the GPU, so I animate those — translate instead of top/left, scale instead of width — and use FLIP when a layout change must look animated. `will-change` sparingly, and I check the Performance panel for long frames."

Follow-ups:

- *Is `filter` compositor-only?* → Often in Chromium, but a blur can be expensive; test.
- *Why can a transform animation keep running when JavaScript is busy?* → It runs on the compositor thread, independent of the main thread.
- *Downside of `will-change` everywhere?* → Too many layers → memory pressure, can make things slower; also creates stacking contexts.

Self-check

1. Which properties are generally compositor-only? → `transform` (incl. `translate` / `rotate` / `scale`) and `opacity`.
2. How do you animate a card's expansion from a grid into a detail view cheaply? → FLIP (or a view transition).
3. What does 16.7ms represent? → The time budget per frame at 60 fps.

12.8 Enter and exit animations: `@starting-style` and `allow-discrete`

The problem it solves

Dialogs, popovers (small panels that float above the page, opened with the `popover` attribute), toasts (brief pop-up notifications that disappear on their own) and menus usually go from `display: none` to visible. That breaks transitions twice:

1. **Entering**: an element that wasn't rendered has no "before" style, so there's nothing to transition *from* — it just appears.
2. **Exiting**: `display` flips to `none` immediately, so the fade-out never gets to be seen.

The tools

- `@starting-style { … }` defines the style to transition **from** when an element is first rendered (or goes from `display: none` to shown). Baseline since August 2024.
- `transition-behavior: allow-discrete` (or the keyword in the shorthand) lets discrete properties take part in a transition. For `display`, the browser keeps the element displayed for the whole exit transition and flips to `none` at the end. `transition-behavior` is Baseline since August 2024, but actually transitioning `display` is supported in Chrome and Safari, not yet Firefox, and the `overlay` property (keeps a closing popover/dialog in the **top layer** — a special layer the browser draws above everything else on the page) is Chromium-only. The good news: where unsupported, the element simply closes without the exit fade — safe **progressive enhancement** (build something that works everywhere, then add extras that only capable browsers use).

```
#a { opacity: 0; translate: 0 -16px;                          /* closed look = exit end state */
     transition: opacity 300ms ease-out, translate 300ms ease-out,
                 display 300ms allow-discrete, overlay 300ms allow-discrete; }
#a:popover-open { opacity: 1; translate: 0 0; }               /* open look */
@starting-style { #a:popover-open { opacity: 0; translate: 0 -16px; } }   /* enter starts here */
```

`code/07_starting_style.html` has popover **A** (as above) and **B** (the same transition without `@starting-style` or `allow-discrete`). `code/07_starting_style.mjs` opens and closes both, freezing any transitions at 100ms:

```
opening, 100ms: A: display block opacity 0.49  transitions running: opacity+translate   |   B: display block opacity 1.00
transitions running: none
opened        : A: display block opacity 1.00  transitions running: none   |   B: display block opacity 1.00  transitions
running: none
closing, 100ms: A: display block opacity 0.51  transitions running: display+opacity+overlay+translate   |   B: display
none  opacity 0.00  transitions running: none
closed        : A: display none  opacity 0.00  transitions running: none   |   B: display none  opacity 0.00  transitions
running: none
```

B never animates: no transition runs on open (no starting style) and it is `display: none` immediately on close. A fades in from the `@starting-style` values, and while closing stays `display: block` with a running `display` transition until the fade ends.

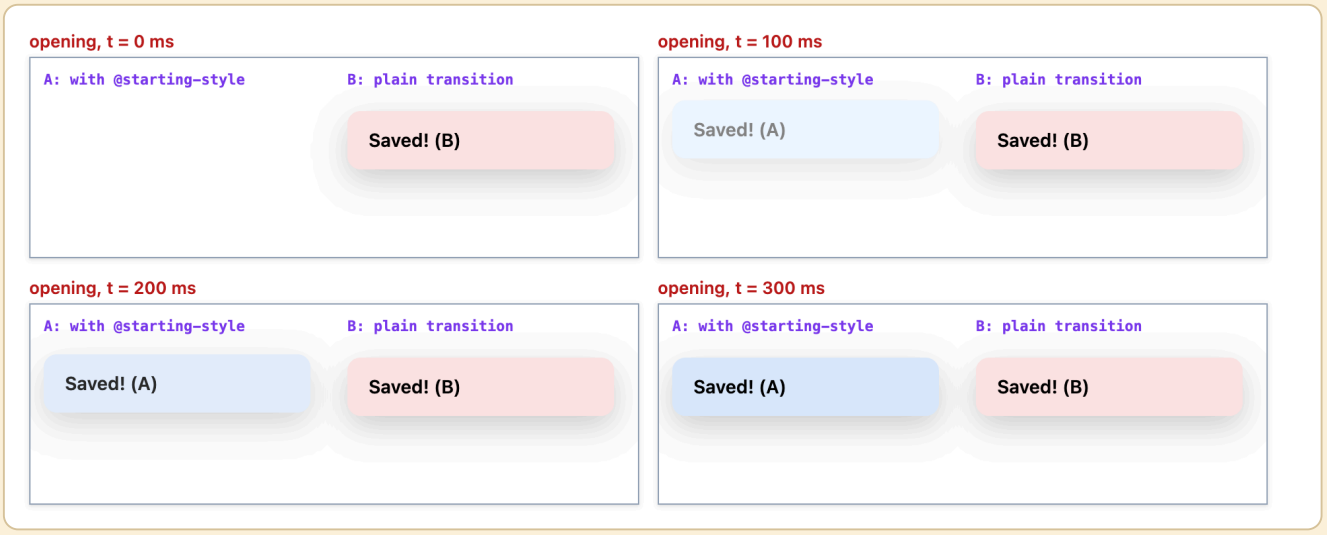


Figure 12.6 — A (blue) animates in; B (red) just appears.

Where you'll use it

`<dialog>` (`[open]` and `::backdrop`), `[popover]` (`:popover-open`), elements added with JavaScript (toasts, list items), `<details>` content, and anything toggled with `display` or the `hidden` attribute. For `<dialog>` also transition `::backdrop` (the full-screen layer drawn behind a modal dialog) the same way.

Self-check

1. Why doesn't a plain transition play when an element goes from `display: none` to `block`? → There's no previous rendered style to transition from; `@starting-style` provides one.
2. What does `allow-discrete` do for `display` on exit? → Keeps the element displayed until the transition ends, then flips to `none`.

12.9 Reduced motion and accessibility

Why it matters

For people with **vestibular disorders** (inner-ear balance problems), large movement on screen — parallax, zooming, spinning, sliding panels — can trigger dizziness, nausea and headaches. Others find motion distracting (ADHD — attention-deficit/hyperactivity disorder — and cognitive disabilities). Every major OS (operating system) has a setting: macOS/iOS "Reduce motion", Windows "Animation effects" off, Android "Remove animations". Browsers expose it as `prefers-reduced-motion: reduce` (Baseline widely available).

The WCAG rules

WCAG (Web Content Accessibility Guidelines) is the international accessibility standard. Each rule (a "success criterion") has a level: **A** is the minimum, **AA** the usual legal and contractual target, **AAA** the strictest.

Criterion	Level	Requirement
2.2.2 Pause, Stop, Hide	A	Moving, blinking or scrolling content that starts automatically, lasts **more than 5 seconds** and is shown with other content must be pausable/stoppable (carousels — rotating slide panels — marquees, animated backgrounds)
2.3.1 Three Flashes or Below Threshold	A	Nothing flashes more than **3 times per second** (seizure risk)
2.3.3 Animation from Interactions	AAA	Motion triggered by interaction can be disabled unless essential

Two patterns

`code/08_reduced_motion.html`:

```
/* 1. Motion only when allowed: big movement is ADDED for users with no preference */
@keyframes slide-in { from { translate: -60px 0; opacity: 0; } }
@media (prefers-reduced-motion: no-preference) {
  .card { animation: slide-in 600ms ease-out both; }
  html  { scroll-behavior: smooth; }
}

/* 2. Replace, don't remove: a loading spinner must still say "busy" */
@keyframes pulse { 50% { opacity: .35; } }
.spinner { animation: spin 800ms linear infinite; }
@media (prefers-reduced-motion: reduce) { .spinner { animation: pulse 2s ease-in-out infinite; } }
```

`code/08_reduced_motion.mjs` emulates both settings:

```
prefers-reduced-motion: no-preference -> card animation: slide-in spinner: spin 0.8s   scroll-behavior: smooth   running animations: 2
prefers-reduced-motion: reduce        -> card animation: none     spinner: pulse 2s   scroll-behavior: auto   running animations: 1
```

Reduced motion ≠ no motion. Fades, colour changes and small scale changes are usually fine; it's large translation, zoom, rotation, **parallax** (layers moving at different speeds as you scroll, faking depth) and **scroll-jacking** (a page overriding the normal speed or direction of scrolling) that cause harm. The blunt global reset in §10.5 (durations → 0.01ms) is a reasonable safety net — `0.01ms` rather than `0` so

`animationend` / `transitionend` still fire for code that waits for them — but designing each animation's reduced version is better.

In JavaScript: `matchMedia('(prefers-reduced-motion: reduce)').matches` (`matchMedia` tests a media query from script) before starting WAAPI animations, smooth `scrollIntoView`, or auto-playing video.

Other rules of thumb

- Autoplaying carousels need a visible pause button (2.2.2) — and usually shouldn't autoplay at all.
- Don't make essential information available only through motion.
- Keep focus behaviour intact during animations: don't move focus into an element that's still invisible.

Self-check

1. What does `@media (prefers-reduced-motion: no-preference)` achieve? → Motion is opt-in: only users who haven't asked for less get it.
2. Why `0.01ms` instead of `0` in the global reset? → So animation/transition end events still fire.
3. A hero background video loops for 30s. Which criterion applies? → 2.2.2 Pause, Stop, Hide — provide a pause control.

12.10 View transitions

What it is

A **view transition** animates between two **states of the page** — before and after a DOM update, or (cross-document) between two pages. You don't write the in-between frames: the browser takes a **snapshot** (a picture) of the old state, lets you update the DOM, snapshots the new state, and animates from one to the other — cross-fading the page and **morphing** any element you've given a `view-transition-name`. It is FLIP (12.7) done for you.

Support (verified September 2026)

Feature	Status
Same-document: `document.startViewTransition()`, `view-transition-name`, the `::view-transition-*` pseudo-elements	**Baseline newly available since October 2025** (Chrome 111, Safari 18, Firefox 144)
`view-transition-class` (style many groups with one selector)	Baseline since October 2025
View transition **types** (`startViewTransition({ update, types: ['forward'] })`, `:active-view-transition-type()`)	Baseline since January 2026
Cross-document (MPA — multi-page application, a site where each link loads a new HTML page) view transitions: `@view-transition { navigation: auto; }`	**Not Baseline**: Chrome 126+, Safari 18.2+, not Firefox yet

How it works (same-document)

1. You call `document.startViewTransition(updateCallback)`.
2. The browser captures the **old** state: a screenshot of the page (`root`) and one of each element with a `view-transition-name` (names must be unique on the page).
3. It runs your callback (the function you passed in; it may be **async**, meaning it can wait for things like network requests before finishing) to change the DOM, then captures the **new** state.
4. It builds a pseudo-element tree over the page:

```
::view-transition
└─ ::view-transition-group(item-2)        ← animates size and position old → new
   └─ ::view-transition-image-pair(item-2)
      ├─ ::view-transition-old(item-2)     ← old snapshot, fades out
      └─ ::view-transition-new(item-2)     ← new live view, fades in
```

5. It animates the tree with ordinary CSS animations (default 250ms `ease`), then removes it. The page underneath already shows the new state.

```
const update = () => items.classList.toggle('list');
toggle.addEventListener('click', () => {
  if (!document.startViewTransition) return update();     // no support: just update
  document.startViewTransition(update);
});
```

```
.item:nth-child(1) { view-transition-name: item-1; }       /* one unique name per morphing element */
.item { view-transition-class: item; }
::view-transition-group(*.item) { animation-duration: 500ms; animation-timing-function: ease-in-out; }
::view-transition-old(*.item), ::view-transition-new(*.item) { height: 100%; }   /* see below */
@media (prefers-reduced-motion: reduce) {
  ::view-transition-group(*), ::view-transition-old(*), ::view-transition-new(*) { animation: none !important; }
}
```

Figure 12.7 — `code/09_view_transition.html` switching from grid to list, frozen at five moments via `document.getAnimations()`.

`code/09_view_transition.mjs` starts the transition, waits for `transition.ready` (the pseudo-elements exist), pauses everything and inspects it:

```
20 animations while the transition runs:
  ::view-transition-group(item-1) -ua-view-transition-group-anim-item-1 500ms
  ::view-transition-group(root) -ua-view-transition-group-anim-root 250ms
  ::view-transition-new(item-1) -ua-mix-blend-mode-plus-lighter 500ms
  ::view-transition-new(item-1) -ua-view-transition-fade-in 500ms
  ::view-transition-new(root) -ua-mix-blend-mode-plus-lighter 250ms
  ::view-transition-new(root) -ua-view-transition-fade-in 250ms
  ::view-transition-old(item-1) -ua-mix-blend-mode-plus-lighter 500ms
  ::view-transition-old(item-1) -ua-view-transition-fade-out 500ms
  ::view-transition-old(root) -ua-mix-blend-mode-plus-lighter 250ms
  ::view-transition-old(root) -ua-view-transition-fade-out 250ms
  … and the same for item-2 and item-3
t=  0ms ::view-transition-group(item-2): width 100.0px, height 70.0px, transform matrix(1, 0, 0, 1, 120, 54)
t=250ms ::view-transition-group(item-2): width 215.0px, height 53.0px, transform matrix(1, 0, 0, 1, 65, 75)
t=500ms ::view-transition-group(item-2): width 330.0px, height 36.0px, transform matrix(1, 0, 0, 1, 10, 96)
```

The `-ua-…` names are Chrome's built-in keyframes. The group of "Two" morphs from a 100×70 box at (120, 54) to a 330×36 row at (10, 96); at the half-way point (`ease-in-out`(0.5) = 0.5) every value is exactly the average — 215 = (100 + 330)/2 ✓. The `root` group keeps the default 250ms because our rule only targeted `*.item`.

Aspect-ratio tip: by default the old and new snapshots keep their own aspect ratios at the group's width; when a tall box turns into a short row they overflow and look smeared mid-way. `height: 100%` on `::view-transition-old/new` makes both stretch to the group, which is what Figure 12.7 uses.

Cross-document view transitions

For multi-page sites, both pages opt in with `@view-transition { navigation: auto; }` and share `view-transition-name`s; a **same-origin** navigation (to a page with the same scheme, host and port, e.g. both on `https://example.com`) then animates without any JavaScript. Chrome and Safari only in 2026 — browsers without support just navigate normally, so it is safe to add.

Pitfalls

- Duplicate `view-transition-name`s on the page abort the transition.
- While a transition runs, the page underneath doesn't receive clicks (the pseudo-element overlay is on top) — keep transitions short.
- Snapshots are images: text inside a morphing element stretches during the morph; cross-fade handles most cases.
- Always keep the no-support path (`if (!document.startViewTransition) return update();`).

Self-check

1. What does `view-transition-name` do? → Captures that element separately so it morphs from its old to its new size/position instead of being part of the page cross-fade.
2. Which pseudo-element would you style to change the morph duration? → `::view-transition-group(name)` (or `(*.class)`).
3. Are cross-document view transitions Baseline? → No (Chrome and Safari only in 2026).

12.11 Scroll-driven animations

What it is

A **scroll-driven animation** replaces the time-based timeline with a **scroll timeline**: the animation's progress follows how far a container has scrolled (`scroll()`) or how far an element has travelled through the **viewport** (the visible area of the browser window) (`view()`). Reading-progress bars, reveal-on-scroll effects and parallax without JavaScript scroll listeners (functions that run on every scroll event) — and they can run off the main thread.

Support: not Baseline. Chrome 115+ and Safari 26+ support it; Firefox has it only behind a flag (a hidden experimental setting users must switch on) (September 2026). Always wrap it in `@supports (animation-timeline: scroll())` (a **feature query**: the rules inside apply only if the browser understands that declaration) and make sure the page works without it.

```
/* Reading progress bar */
@keyframes grow { from { scale: 0 1; } to { scale: 1 1; } }
.progress { transform-origin: 0 50%; animation: grow linear both; animation-timeline: scroll(root block); }

/* Reveal an element as it enters the viewport */
@keyframes fade-up { from { opacity: 0; translate: 0 40px; } to { opacity: 1; translate: 0 0; } }
.reveal { animation: fade-up linear both; animation-timeline: view(); animation-range: entry 0% entry 100%; }
```

Piece	Meaning
`scroll(<scroller> <axis>)`	Anonymous scroll timeline: `nearest` (default), `root` or `self`; axis `block` (default), `inline`, `x`, `y`
`view(<axis> <inset>)`	Timeline of the element's own visibility in its scrollport (the visible window of a scrolling box)
`animation-range`	Which part of the timeline maps to 0–100%: `entry`, `exit`, `contain`, `cover` + percentages
`scroll-timeline-name` / `view-timeline-name`	Named timelines, usable by other elements (with `timeline-scope`)
Duration	Ignored/`auto` — scroll distance replaces time; use `linear` easing so progress tracks scroll 1:1

`code/10_scroll_driven.mjs` scrolls `code/10_scroll_driven.html` (a 400px-tall viewport) and reads the progress bar and the second reveal box:

```
CSS.supports("animation-timeline: scroll()") = true
scrollY   0px (  0% of 540px): bar scale 0 1       #r2 top  562px (height 48) opacity 0.00 translate 0px 40px
scrollY 135px ( 25% of 540px): bar scale 0.25 1    #r2 top  427px (height 48) opacity 0.00 translate 0px 40px
scrollY 186px ( 34% of 540px): bar scale 0.344444 1 #r2 top  376px (height 48) opacity 0.50 translate 0px 20.1042px
scrollY 270px ( 50% of 540px): bar scale 0.5 1     #r2 top  292px (height 48) opacity 1.00 translate 0px
scrollY 540px (100% of 540px): bar scale 1         #r2 top   22px (height 48) opacity 1.00 translate 0px
```

The bar's scale equals the scroll fraction exactly (186 / 540 = 0.344). The box's `entry` range runs from "top edge touches the viewport bottom" (top = 400px) to "fully inside" (top = 400 − 48 = 352px). At top = 376px it is half-way through → opacity ≈ 0.50 and translate ≈ 20px ✓.

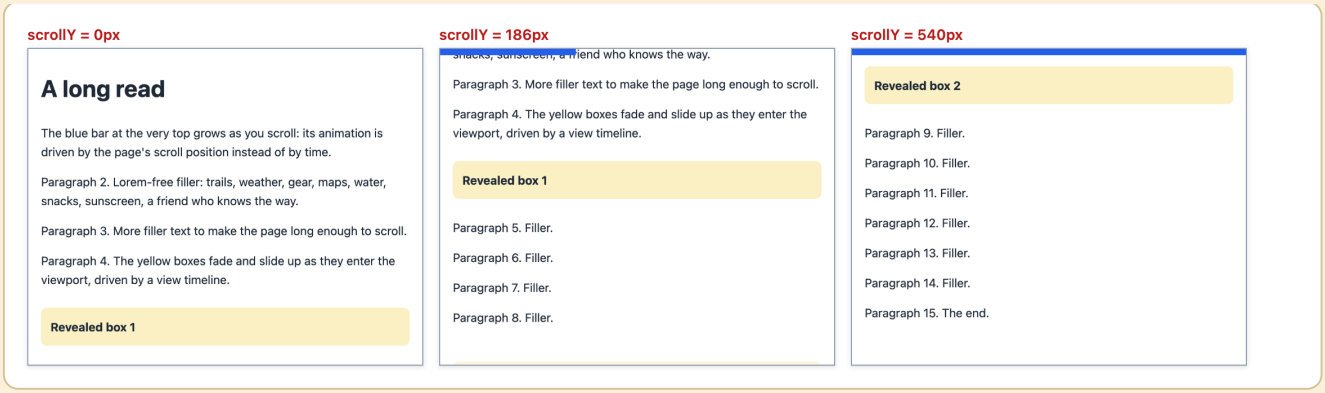


Figure 12.8 — The progress bar at three scroll positions.

Cautions

- Respect reduced motion: big parallax or slide-ins inside `@media (prefers-reduced-motion: no-preference)` (the demo disables the reveal under `reduce`).
- Don't hide content until it scrolls into view if it might never animate (unsupported browser, print, "find in page"): the `@supports` wrapper in the demo keeps content visible by default.
- Scroll-jacking (taking over the scroll speed) is still a bad idea; scroll-driven animations should *follow* scrolling, not replace it.

Chapter 12 TL;DR

- **Transitions** animate a *change*: `transition: prop duration easing delay`; on the base rule; interpolable values only (not `auto`); `transitionend` per property; reversal is shortened.
- **Timing functions**: `ease` (default), `ease-in` (leaving), `ease-out` (entering), `ease-in-out`, `cubic-bezier()` (x ∈ 0–1, y may overshoot), `steps(n)` (jump-end by default), `linear(points)` (Baseline 2023). Our bisection solver matched Chrome to 3 decimals for every curve.
- **2D transforms** move the *painted* box, not the layout box (layout stayed 50,30 90×50 for all eight). Order matters (read right to left); `transform-origin`; `%` translate = own size; individual `translate`/`rotate`/`scale` (Baseline 2022) are applied in a fixed order and animate independently. Transforms create stacking contexts and containing blocks.
- **3D**: `perspective` on the parent, `transform-style: preserve-3d`, `backface-visibility: hidden` → flip cards; `overflow`, `opacity < 1`, `filter` flatten 3D.
- **Keyframes**: `@keyframes` + `animation: name duration easing delay count direction fill-mode`; easing applies per segment; **fill-mode** = what shows during the delay (backwards) and after the end (forwards); negative delays start mid-way.
- **WAAPI**: `el.animate()`, `pause()`, `currentTime`, `playbackRate`, `reverse()`, `await anim.finished`, `document.getAnimations()` (includes CSS animations — how this chapter's filmstrips were frozen).
- **Performance**: animate `transform` and `opacity` (≈1 layout per animation) — not `left`/`width` (≈60 layouts per second, measured). FLIP for layout changes; `will-change` sparingly.
- **Enter/exit**: `@starting-style` (Baseline 2024) for the "from" style; `transition-behavior: allow-discrete` + `display`/`overlay` keep an element visible while it animates out (display transitions: Chrome/Safari; overlay: Chromium).
- **Reduced motion**: `@media (prefers-reduced-motion: no-preference)` to opt motion in; replace rather than remove; WCAG 2.2.2 (pause > 5s auto motion), 2.3.1 (≤ 3 flashes/s), 2.3.3 (AAA).
- **View transitions**: `document.startViewTransition(update)` + `view-transition-name` → snapshot, update, morph via `::view-transition-group/old/new`; same-document Baseline since Oct 2025; cross-document Chrome/Safari only.
- **Scroll-driven animations**: `animation-timeline: scroll()` / `view()` + `animation-range`; **not Baseline** (no Firefox) → wrap in `@supports`, keep content visible without it.

Next: Chapter 13 — Modern CSS and Architecture.

Chapter 13 — Modern CSS and Architecture

Until about 2022, writing CSS for a big site meant reaching for tools: Sass (a language that adds extra features to CSS and is compiled — translated by a program — into plain CSS) for nesting and variables, strict naming conventions so styles didn't collide, and JavaScript for anything like "style the parent when a child is checked". Browsers have since caught up. Nesting, the `:has()` "parent" selector, cascade layers, `@scope`, logical properties and `light-dark()` are all built in, and together they change how you organise stylesheets (the files of CSS rules for a site). This chapter covers each new feature, then the bigger question every team faces: **how do you structure CSS so that it stays maintainable as a project grows?** We compare the four main answers (BEM naming, utility-first with Tailwind CSS, CSS Modules and CSS-in-JS), look at Sass and resets, and finish with a complete theming system.

What you'll learn

#	Topic	You will be able to…
13.1	Native nesting	nest rules without Sass and know exactly what `&` means (and where it differs from Sass)
13.2	`:is()` and `:where()`	shorten selector lists, use forgiving parsing, and control specificity with them
13.3	`:has()`	style an element based on its children, siblings or state, without JavaScript
13.4	Cascade layers	order whole groups of CSS (`reset`, `vendor`, `components`, `utilities`) and tame third-party CSS
13.5	`@scope`	limit styles to a subtree, with a lower boundary ("donut scope") and proximity
13.6	Logical properties	write direction-independent CSS that works for right-to-left and vertical text
13.7	Why CSS needs an architecture; BEM	name the problems and apply the BEM convention
13.8	Utility-first and Tailwind CSS v4	explain utility CSS and read what a real Tailwind build produces
13.9	CSS Modules	explain build-time scoping and read the generated class map
13.10	CSS-in-JS	explain how runtime CSS-in-JS works, its costs, and the zero-runtime alternatives
13.11	Choosing an approach	compare the four approaches and pick one for a given project
13.12	Preprocessors (Sass) and PostCSS	know what Sass adds and which parts native CSS now covers
13.13	Resets and normalize	explain what a reset does, line by line, and why it belongs in a low layer
13.14	Theming and dark mode	build light/dark/brand themes from design tokens and `light-dark()` with a no-flash toggle

How the numbers and figures were made. Every example is a page in `code/`. Each `NN_*.mjs` script (a JavaScript module run with Node.js) opens its page in real **headless** Chrome 154 (a full browser running without a visible window, controlled by a script) (through `code/measure.mjs`, a copy of Chapter 10's helper) and prints what the browser computed — `getComputedStyle()` (the JavaScript call that returns an element's final, resolved CSS values), bounding boxes (the rectangle an element occupies on screen), and the CSSOM (the browser's object model of the stylesheet). The Tailwind example is a real Tailwind CSS v4.3 build, and the CSS Modules example is compiled by Lightning CSS (a fast CSS compiler, 13.12), the same compiler many **bundlers** use (a bundler is a build tool that gathers a project's many source files and outputs a few optimised files for the browser). `code/make_figures.mjs` takes the screenshots. Support data comes from the **web-features** dataset (the source of MDN's — Mozilla's web developer documentation — and caniuse's — a site of browser-support tables — "Baseline" badges), checked in September 2026.

> **Baseline, briefly** (Chapter 10 has more). A feature is **Baseline "newly available"** once it works in the current Chrome, Edge, Firefox and Safari (desktop and mobile), and **"widely available"** 30 months after that. Newly available features are safe for most audiences; for older devices, check your own analytics

> (the statistics your site collects about its visitors, including their browsers) or provide a fallback (an alternative that still works when the feature is missing).

13.1 Native CSS nesting

What it is

Nesting lets you write a rule inside another rule, so the inner selector is relative to the outer one. It's like writing a street address as "Paris → Rue de Rivoli → No. 10" instead of repeating "Paris" on every line. Sass has offered this since 2006; browsers now do it natively: **Baseline newly available since December 2023** (Chrome 120, Firefox 117, Safari 17.2) and **widely available since June 2026**.

Why it matters

Without nesting you repeat the parent selector for every child, hover state and media query, and related rules drift apart in the file. Nesting keeps one component's styles in one block. Because it's native (built into the browser), there's no **build step** (a program that transforms your source files before they are served) and DevTools (the browser's built-in developer tools) shows the nested rule as you wrote it.

How it works

1. Inside a style rule, you can write declarations **and** further rules.
2. A nested selector with no `&` is treated as a **descendant** of the parent: `.title {}` inside `.card {}` means `.card .title`.
3. `&` (the **nesting selector**) stands for the parent's matched elements. Its position decides the relationship: `&:hover` (same element, no space), `& > p` (child), `.dark-page &` (the parent is *inside* `.dark-page`).
4. A nested selector may start with a **combinator** (the symbol that relates two selectors: a space for descendant, `>` for child, `+` for next sibling, `~` for any later sibling): `> p` means `& > p`.
5. Conditional **at-rules** (CSS statements that start with `@`; these ones apply their contents only under a condition) (`@media`, `@supports`, `@container`, `@layer`, `@scope`) can be nested too; their declarations apply to the parent selector.
6. `&` behaves like `:is(<parent selector list>)`. That has one important consequence for specificity (see below).

> **Key term — specificity:** the score the browser uses to decide which of two conflicting rules wins. It is written as three numbers (A,B,C): A counts ID selectors (`#main`), B counts classes, attributes and pseudo-classes (`.card`, `[type]`, `:hover`), C counts element names and pseudo-elements (`p`, `::before`). Compare A first, then B, then C — so (1,0,0) beats (0,3,0). If the scores tie, the rule that comes later in the file wins.

`code/01_nesting.html`:

```
.card {
  padding: 16px;
  border: 2px solid #cbd5e1;

  .title { font-size: 20px; margin: 0; }          /* = .card .title (descendant)        */
  > p { margin: 4px 0 0; color: #334155; }         /* = .card > p (relative selector)    */
  &:hover { border-color: #2563eb; }               /* = .card:hover (no space)           */
  &.featured { border-color: #f59e0b; }            /* = .card.featured (same element)    */
  .dark-page & { background: #0f172a; }            /* = .dark-page .card (& at the end)  */
  @media (width >= 600px) { padding: 24px; }       /* nested at-rule: applies to .card   */
  &__body { color: red; }                          /* NOT Sass: no string joining        */
}

/* & behaves like :is() -> the parent list's HIGHEST specificity is used */
#main, .sidebar {
  .link { color: crimson; }                        /* = :is(#main, .sidebar) .link  (1,1,0) */
}
.sidebar .link.plain { color: green; }             /* (0,3,0) -- still loses!               */
```

`code/01_nesting.mjs` walks the CSSOM (the browser's parsed stylesheet) and reads computed styles at two **viewport** (the visible area of the browser window) widths.

Output:

```
CSSOM tree (selectorText of each rule):
body
.card
  & .title
  & > p
  &:hover
  &.featured
  .dark-page &
  @media (width >= 600px)
    CSSNestedDeclarations
#main, .sidebar
  & .link
.sidebar .link.plain

Was the "&__body" rule kept? false

viewport 500px: .card padding 16px, border rgb(245, 158, 11), background rgb(15, 23, 42)
  .title font-size 20px   .card__body color rgb(0, 0, 0) (not red)   .sidebar .link color rgb(220, 20, 60)

viewport 700px: .card padding 24px, border rgb(245, 158, 11), background rgb(15, 23, 42)
  .title font-size 20px   .card__body color rgb(0, 0, 0) (not red)   .sidebar .link color rgb(220, 20, 60)
```

What the output proves:

- The browser stores `.title` as `& .title` and `> p` as `& > p`: an implicit `&` plus a space (descendant) is added in front.
- The nested `@media` holds a `CSSNestedDeclarations` object — the bare `padding: 24px` is treated as if it were inside `& { … }`. Padding is 16px at 500px and 24px at 700px.
- **`&__body` was thrown away.** In Sass, `&__body` glues text together to make `.card__body`. Native CSS works on selectors, not strings, so this never produces `.card__body`; Chrome discards the rule, and the element stays black. This is the #1 surprise when moving BEM-style (BEM is a class-naming convention, 13.7) Sass to native nesting.
- The sidebar link is **crimson**, although `.sidebar .link.plain` (0,3,0) looks more specific. The nested rule is really `:is(#main, .sidebar) .link`, and `:is()` takes the specificity of its most specific argument — `#main` — giving (1,1,0), even though the element matched via `.sidebar`.

Compiling nesting away for old browsers

If you must support browsers older than late 2023, a build tool can **lower** (flatten) nesting. `code/01_nesting_lower.mjs` asks Lightning CSS to target Chrome 100:

Output:

```
.card {
  padding: 16px;
}

.card .title {
  font-size: 20px;
}
/* ... .card:hover, .dark-page .card ... */

@media (min-width: 600px) {
  .card {
    padding: 24px;
  }
}

:is(#main, .sidebar) .link {
  color: #dc143c;
}
```

The compiler writes `:is(#main, .sidebar) .link` — the same meaning the browser gives native nesting (Sass would instead write `#main .link, .sidebar .link`, two selectors with *different* specificities). It also rewrote the range media query (an `@media` condition written with comparison operators like `>=`) `(width >= 600px)` into the older `min-width` form.

Common mistakes

- Porting Sass `&__title` or `&-large` and wondering why nothing applies.
- **Nesting too deep.** `.page .main .card .body .title a` in nested form is just as fragile as when written flat. Keep nesting to 2–3 levels: states (`&:hover`), children you own, and media/container queries.
- Forgetting that a nested selector list inherits the `:is()` specificity of the *whole* parent list.

In the interview

> **30-second answer:** "CSS now supports nesting natively — Baseline since late 2023. A nested selector is relative to its parent; `&` represents the parent and behaves like `:is()` of the parent list, so it takes the highest specificity in that list. Unlike Sass it doesn't concatenate strings, so `&__title` doesn't work. I use it for states, owned children and nested media queries, and keep it shallow."

- *Q: Is `.a { .b {} }` a child or a descendant selector?* A: Descendant (`.a .b`); write `> .b` for a child.
- *Q: What does `.x & {}` inside `.card` mean?* A: A `.card` that is inside `.x`.

Self-check

1. What does `& + &` mean inside `.item {}`? → An `.item` immediately following another `.item`.
2. Why might a nested rule under `#nav, .menu` beat a rule you think is more specific? → `&` = `:is(#nav, .menu)`, so an ID's weight counts even for `.menu` matches.
3. Does native nesting need a build tool? → No; only to support pre-2023 browsers (lowering).

13.2 `:is()` and `:where()`

What it is

Both take a **selector list** and match any element that matches one of the selectors in it — a "one of these" shortcut. They differ only in **specificity**: `:is()` counts as its **most specific argument**; `:where()` always counts as **zero**. Both are Baseline widely available (since 2021). Chapter 6 introduced the specificity rules; here we use them as architecture tools.

Why it matters

1. **Shorter selectors.** `:is(article, section, aside) :is(h1, h2, h3)` replaces nine comma-separated selectors.
2. **Forgiving parsing.** In a normal selector list, one invalid or unknown selector makes the browser drop the **whole rule**. Inside `:is()`/`:where()`, bad entries are just ignored.
3. **Specificity control.** `:where()` lets you write defaults (resets, library styles) that anything can override; `:is()` lets you raise specificity when you need to.

Proof

`code/02_is_where.html`:

```
:is(article, section, aside) :is(h1, h2, h3) { color: rebeccapurple; }

:is(.promo, :unknown-thing) { font-weight: 700; }           /* forgiving   */
.promo, :unknown-thing { text-decoration: underline; }      /* NOT forgiving */

:where(.content) a  { color: gray; }       /* (0,0,1)                      */
a                   { color: teal; }       /* (0,0,1), later -> wins       */
:is(#nav, .content) b { color: gray; }     /* (1,0,1)                      */
.content b.hot      { color: teal; }       /* (0,2,1) -> loses             */
```

`code/02_is_where.mjs` reads the computed styles and, for comparison, scores the selectors with Chapter 6's specificity calculator:

Output:

```
h2 inside article color      rgb(102, 51, 153) (rebeccapurple = rgb(102, 51, 153))
.promo font-weight           700   <- forgiving :is() kept the rule
.promo text-decoration-line  none   <- plain list with one bad selector was dropped
:where(.content) a vs a      rgb(0, 128, 128) (teal = rgb(0, 128, 128))
:is(#nav,.content) b vs .content b.hot rgb(128, 128, 128) (gray = rgb(128, 128, 128))

Rules the browser kept: body  |  :is(article, section, aside) :is(h1, h2, h3)  |  :is(.promo)  |  :where(.content) a  |  a
|  :is(#nav, .content) b  |  .content b.hot

Specificity (A,B,C):
  :is(article, section, aside) :is(h1, h2, h3)   (0,0,2)
  :where(.content) a                             (0,0,1)
  a                                              (0,0,1)
  :is(#nav, .content) b                          (1,0,1)
  .content b.hot                                 (0,2,1)
```

Notice what Chrome kept: `:is(.promo)` — it silently removed `:unknown-thing` — while `.promo, :unknown-thing` is **missing** from the list of kept rules. And `:is(#nav, .content) b` wins with an ID's weight even though there is no `#nav` on the page.

Where each is used

Use	Pick	Example
Reset / library defaults that must be easy to override	`:where()`	`:where(ul, ol)[role="list"] { list-style: none }`
Shortening a long list without changing priorities much	`:is()`	`:is(h1,h2,h3):hover`
Using a new pseudo-class safely next to old ones	either (forgiving)	`:is(:focus-visible, .focus)`
Deliberately raising a selector's weight	`:is()` with an ID or class	`.btn:is(#_, .btn)` (a hack — prefer layers, 13.4)

Common mistakes

- Using `:is()` in a library's defaults: an argument like `#app` makes every selector inside it hard to override. Use `:where()`.
- Expecting pseudo-elements (parts of an element that CSS can style as if they were separate, like `::before` and `::after`) inside: `:is(::before)` never matches — pseudo-elements aren't allowed in these lists.

Self-check

1. Specificity of `:where(#a) .b`? → (0,1,0).
2. Specificity of `:is(p, .x, #y)`? → (1,0,0), whichever argument matched.
3. Why is `:is()` safer than a plain list when using a new pseudo-class? → An unsupported selector inside it is ignored instead of dropping the entire rule.

13.3 `:has()` — the relational ("parent") selector

What it is

`A:has(B)` matches an element **A** if selector **B**, evaluated *relative to A*, matches something. With a descendant (`:has(img)`), it's the long-awaited **parent selector**; with `+` or `~` (`h3:has(+ .subtitle)`) it's a **previous-sibling** selector (siblings are elements that share the same parent). Baseline newly available since **December 2023** (Firefox 121 was last), widely available since June 2026. Chapter 6 showed the basic syntax.

The problem it solves

CSS selectors used to look only **down and forward** in the document: you could style a child based on its parent, never the reverse. "Give the card a grid layout if it contains an image", "highlight the label whose radio is checked" or "lock page scrolling while a dialog is open" all needed JavaScript that added classes. `:has()` does them in CSS, and they stay in sync automatically.

How it works

1. The argument is a **relative selector list**: each item implicitly starts at the `:has()` element (default combinator = descendant).
2. Specificity = the most specific argument, like `:is()`.
3. The browser re-evaluates it whenever the DOM (Document Object Model — the browser's live tree of the page's elements, which scripts can change) or state changes (checking a box, typing into an input, opening a dialog).
4. Restrictions: `:has()` can't be nested inside another `:has()`, and pseudo-elements can't be arguments.

`code/03_has.html` shows six patterns:

```
.card:has(img)                { display: grid; grid-template-columns: 64px 1fr; }  /* parent       */
.field:has(input:invalid)     { border-left-color: #dc2626; background: #fef2f2; } /* form state   */
h3:has(+ .subtitle)           { margin-bottom: 0; color: #7c3aed; }                /* prev sibling */
ul:has(> li:nth-child(4))     { columns: 2; }                                      /* quantity     */
label.opt:has(input:checked)  { background: #dbeafe; font-weight: 600; }           /* checked      */
html:has(dialog[open])        { overflow: hidden; }                                /* page-level   */
```

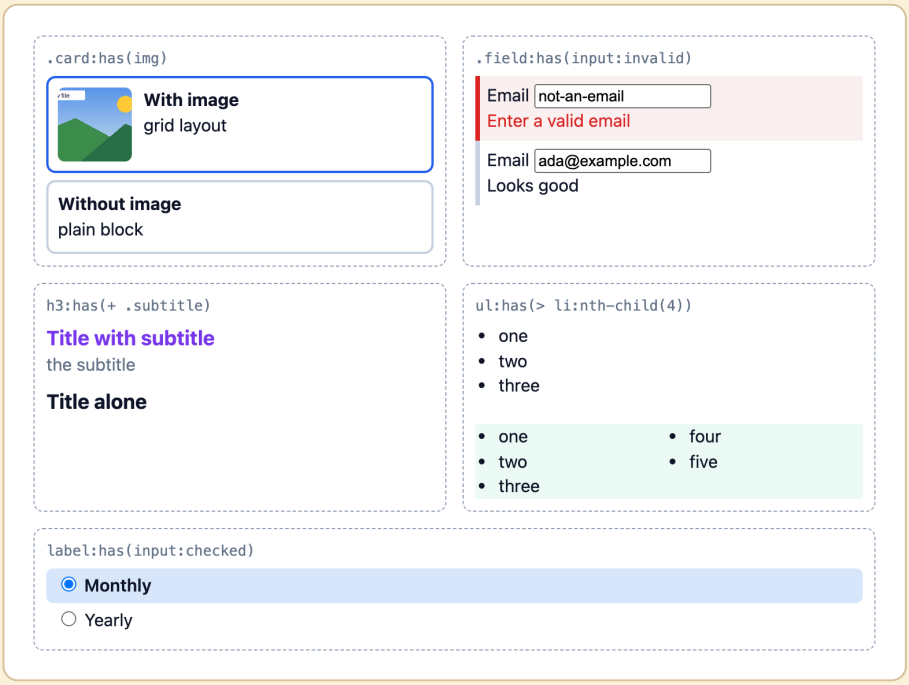


Figure 13.1 — Each box's styling comes from a `:has()` rule; no JavaScript adds classes.

`code/03_has.mjs` reads the computed styles, then changes state the way a user would (click "Yearly", fix the email, add a fourth list item, open the dialog) and reads them again.

Output:

```
.card with img display=grid, without img display=block
invalid field border=rgb(220, 38, 38), valid field border=rgb(203, 213, 225)
h3 followed by .subtitle margin-bottom=0px, lone h3 margin-bottom=8px
3-item list columns=auto, 5-item list columns=2
Monthly (checked) weight=600, Yearly weight=400
<html> overflow with dialog closed: visible

after: click "Yearly", fix the email, add a 4th <li>, showModal()
Monthly weight=400, Yearly weight=600
first field border=rgb(203, 213, 225)
first list columns=2
<html> overflow with dialog open: hidden

a:has(:has(b))  -> SyntaxError
p:has(::before)  -> SyntaxError
li:has(+ li)  -> valid
```

Every style followed the state change instantly, and the two forbidden forms are rejected by the selector parser (the part of the browser that reads selector text and turns it into a structure it can match).

Trade-offs and performance

`:has()` asks the browser to look "up" the tree when something changes, which is more work than ordinary selectors. Engines optimise the common cases, and in normal use it is fine. Two habits keep it cheap:

- **Anchor it** to a specific element: `.card:has(img)` rather than `:has(img)` alone or `body:has(.x .y .z)` with long descendant chains.
- Avoid `*:has(...)` and very broad subjects on huge, constantly changing DOMs.

When not to use it

- For *behaviour* (disabling a button, moving focus) — that's JavaScript's job; `:has()` only changes appearance.
- To style a form field with an error **the moment the page loads**: `:invalid` matches before the user has typed. Prefer `:user-invalid` (matches only after interaction) — `.field:has(:user-invalid)` (Chapter 5).

Real-world examples

Selected table rows (`tr:has(input:checked)`), error-highlighting form groups, quantity queries (styles that change with how many children an element has), modal (a dialog that blocks the rest of the page until closed) scroll locking, and dimming siblings of a hovered item: `.list:has(.item:hover) .item:not(:hover) { opacity: .6 }`.

In the interview

> **30-second answer:** "`:has()` matches an element if the relative selector inside it matches — so `.card:has(img)` styles cards that contain an image. It works as a parent selector and, with `+` / `~`, a previous-sibling selector, and it reacts to state like `:checked` or `:invalid`. It's Baseline since late 2023. I anchor it to specific elements for performance and never use it for behaviour."

- *Q: How do you style a label whose checkbox is checked, with the input inside the label?* A: `label:has(input:checked)`.
- *Q: Specificity of `.card:has(> img, #hero)`?* A: (1,1,0): `.card` + the most specific argument `#hero`.

Self-check

1. Select an `h2` immediately followed by a `p`. → `h2:has(+ p)`.
2. Select a `ul` with exactly 3 items. → `ul:has(> li:nth-child(3)):not(:has(> li:nth-child(4)))`.
3. Can you write `:has(:has(x))`? → No, it's a syntax error.

13.4 Cascade layers (`@layer`)

What it is

A **cascade layer** is a named bucket of rules. You declare the order of the buckets once, and a rule in a later layer beats a rule in an earlier layer **no matter how specific either selector is**. Think of layers as floors of a building: anything on the 5th floor is above anything on the 2nd, however tall the furniture on the 2nd floor. Baseline since March 2022 and widely available since September 2024. Chapter 6 placed layers in the cascade; this section is about using them to structure a project.

The problem it solves

Specificity wars. A third-party library (code written by someone outside your team) ships `#app .btn.btn-primary`; to override it you write something even more specific, or `!important` (a flag on a declaration that makes it beat normal declarations), and the next person escalates again. Layers let you say, once, "vendor CSS is below my components, and utilities are above both".

How it works

1. `@layer reset, vendor, base, components, utilities;` declares the order: **first = weakest**. Put this statement first, so the order doesn't depend on which file loads first.
2. Add rules with `@layer name { … }` (as many blocks as you like; they merge), or import a whole file into a layer: `@import url(x.css) layer(vendor);`.

3. When two declarations conflict, the cascade (the browser's algorithm for choosing a winner) compares **origin & importance → context → style attribute → layer → specificity → order**. Origin is who wrote the CSS (browser defaults, the user, or the page author); context is whether the rule comes from inside a shadow DOM (13.5); the style attribute is CSS written directly on an element (`style="…"`). The layer is compared **before** specificity.
4. **Unlayered** styles form an implicit final layer: they beat every named layer.
5. `!important` **reverses the order**: an `!important` declaration in an *earlier* layer beats one in a later layer, and layered `!important` beats unlayered `!important`. This lets a reset lock something down.
6. `revert-layer` as a value means "ignore this layer's value; use what the earlier layers produce".
7. Layers nest: `@layer components { @layer dialog { … } }` creates `components.dialog`.

`code/04_layers.html` (with `code/04_vendor.css` standing in for a library):

```
@layer reset, vendor, base, components, utilities;
@import url("04_vendor.css") layer(vendor);     /* #app .btn.btn-primary { background: #6b7280; ... } */

@layer components { .btn { background: #2563eb; border-radius: 8px; }  .card a { color: #7c3aed; } }
@layer utilities  { .text-red { color: #dc2626; }  .no-radius { border-radius: 0 !important; } }
@layer reset      { .keep-radius { border-radius: 999px !important; } }
p { color: #065f46; }                                        /* unlayered */
@layer utilities  { .plain-link { color: revert-layer; } }
@layer components { @layer dialog { .dlg { color: #b45309; } } }
```

`code/04_layers.mjs` lists the layers from the CSSOM and reads who won each fight:

Output:

```
statement: reset, vendor, base, components, utilities
import:    04_vendor.css layer(vendor)
block:     reset
block:     base
block:     components
block:     utilities
block:     components.dialog

#app .btn.btn-primary (vendor, 1,2,0) vs .btn (components, 0,1,0):
   background rgb(37, 99, 235) (#2563eb = rgb(37, 99, 235)), radius 8px, padding-left 20px (vendor's, nobody overrode it)
.card a (components)                    color rgb(124, 58, 237)
.text-red (utilities) vs .card a        color rgb(220, 38, 38)
.plain-link { color: revert-layer }     color rgb(124, 58, 237)  <- back to the components value
unlayered p vs base body                color rgb(6, 95, 70)
!important: utilities 0 vs reset 999px  radius 999px  <- earlier layer wins for !important
nested layer components.dialog          color rgb(180, 83, 9)
```

Read it line by line: a one-class selector in `components` beat an ID selector in `vendor`; the vendor's padding survived because nothing higher set padding (layers settle *conflicts*, they don't delete rules); `revert-layer` fell back to the components colour; and for `!important`, the **reset** layer won.

A layer order for real projects

```
@layer reset, tokens, vendor, base, layout, components, utilities, overrides;
```

Layer	Holds	Why here
`reset`	the reset (13.13)	weakest: everything may override it
`tokens`	custom properties (CSS variables like `--brand`, 13.14)	declarations only, rarely conflicts
`vendor`	third-party CSS via `@import … layer(vendor)`	your own base styles should beat it
`base`	element defaults (`body`, `a`, `h1`)	
`layout`	page grids, containers	
`components`	cards, buttons, dialogs	
`utilities`	single-purpose classes (`.visually-hidden` — hidden on screen but still read by screen readers — `.text-center`)	must beat components
`overrides`	temporary fixes	explicit, easy to find and delete

Tailwind CSS v4 itself emits `@layer theme, base, components, utilities;` (13.8), so its utilities slot neatly into this scheme.

Common mistakes

- **Leaving your own CSS unlayered while the framework (a ready-made library that shapes how you build, such as Tailwind) is layered**: unlayered beats all layers, which may be what you want — or may make your stray rules impossible to override with utilities.
- **Order by accident**: the first time a layer name appears fixes its position. Without the up-front statement, a file loaded early can create layers in the wrong order.
- Expecting `!important` in a later layer to win — it's the reverse.
- `@import … layer()` must come before all other rules except `@charset` (declares the file's text encoding) and `@layer` statements, or the import is ignored.

In the interview

> **30-second answer:** "Cascade layers let me order groups of rules explicitly. The cascade compares layers before specificity, so a simple class in a later layer beats an ID selector in an earlier one. I declare the order once — reset, vendor, base, components, utilities — and import third-party CSS into a low layer. Unlayered styles beat all layers, and `!important` inverts the order."

- *Q: How do you override a library's high-specificity CSS without `!important`?* A: Import it into a low layer: `@import url(lib.css) layer(vendor);` and put your rules in a later layer or unlayered.
- *Q: What does `revert-layer` do?* A: Rolls the property back to the value from the previous layers (or earlier origins if none).

Self-check

1. Layer order `a, b`. `#x p` in `a`, `p` in `b`. Which wins? → `p` in `b`.
2. `color: red !important` in `a` vs `color: blue !important` in `b`? → Red (earlier layer wins for important).
3. Unlayered `p { color: green }` vs `@layer b { #x p { color: blue } }`? → Green.

13.5 `@scope`

What it is

`@scope (<root>) to (<limit>) { … }` applies rules only to elements **inside** a scope root and, optionally, **not inside** a lower boundary. The part between root and limit is often called a **donut scope**: the root is the ring, the limit is the hole. Inside, `:scope` means the root element itself.

Status (September 2026): Baseline newly available since March 2026. Chromium shipped `@scope` first (Chrome 118, 2023); web-features counts it as complete from Firefox 146 and Safari 26.4, the versions that finished

the cross-browser set. Treat it as safe for current browsers; older browsers simply ignore the whole `@scope` block, so give unscoped fallbacks for anything essential.

The problem it solves

- **Leaking into content**: a `.card img { border: … }` rule also hits images in user-generated content (text and images posted by site users, such as comments or rich-text articles) inside the card. BEM avoids this with long class names on every element; `@scope … to (.content)` does it with a boundary.
- **Proximity**: with nested themes (`.light` inside `.dark` inside `.light`), `.dark a` and `.light a` have the same specificity, so whichever comes later in the file wins — wrong for the innermost box. With `@scope`, when specificity is equal, the rule whose root is **nearest** (fewest steps up the tree) wins.

How it works

1. Match the scope root(s) (`.card`).
2. Match the limit(s) (`.content`) inside each root; everything at or below a limit is excluded.
3. Apply the rules to elements between. Selectors inside are matched relative to the root; bare selectors like `img` act as descendants of `:scope`.
4. Specificity: the root selector adds **nothing** (unlike nesting); `:scope` adds (0,1,0) like a pseudo-class.
5. **Scoping proximity** is a new cascade step *after* specificity and *before* order of appearance.
6. `@scope { … }` **without a prelude** (the part between `@scope` and `{`, which names the root and limit) inside a `<style>` element scopes to that `<style>`'s parent element — handy for component-local styles in HTML.

`code/05_scope.html`:

```
@scope (.card) to (.content) {
  :scope { background: #f8fafc; }          /* the .card itself                */
  img    { border: 3px solid #2563eb; }    /* card's images, not content's    */
  p      { font-weight: 600; color: #1d4ed8; }
}
@scope (.dark)  { a { color: #fbbf24; } :scope { background: #1e293b; color: #e2e8f0; } }
@scope (.light) { a { color: #1d4ed8; } :scope { background: #fff; color: #0f172a; } }
```

```
<div id="inline">
  <style> @scope { :scope { outline: 2px dashed #16a34a; } b { color: #16a34a; } } </style>
  <b>styled by an inline, prelude-less @scope</b>
</div>
<b>same &lt;b&gt;, outside that div</b>
```

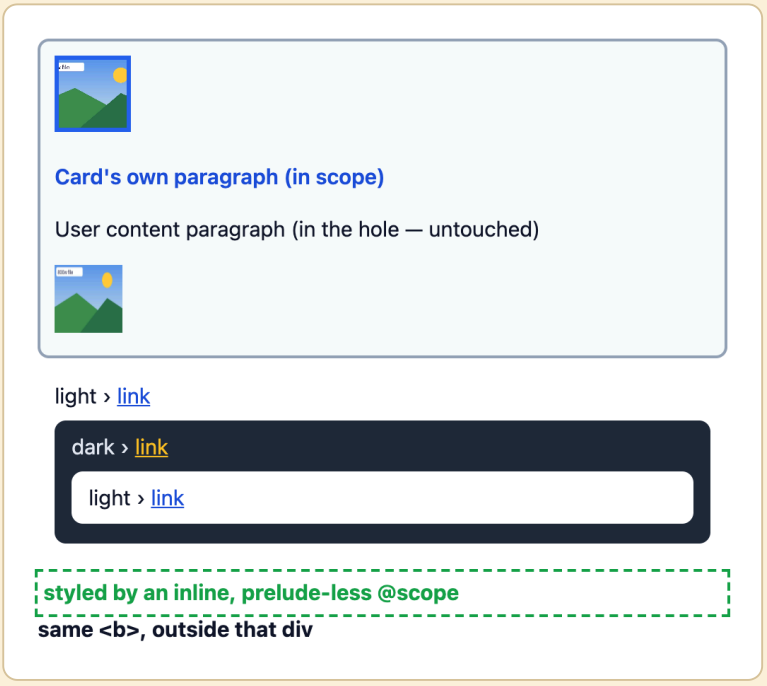


Figure 13.2 — Donut scope (top), proximity with nested themes (middle), prelude-less inline scope (bottom).

`code/05_scope.mjs`:

Output:

```
CSSScopeRule supported: true
start=.card end=.content
start=.dark end=null
start=.light end=null

.card background rgb(248, 250, 252)
card's own img border 3px   img inside .content border 0px
card's own p  rgb(29, 78, 216) 600
p in .content rgb(15, 23, 42) 400

links: light rgb(29, 78, 216)   dark rgb(251, 191, 36)   light-inside-dark rgb(29, 78, 216)
(amber = rgb(251, 191, 36), blue = rgb(29, 78, 216))

inline @scope: <b> inside rgb(22, 163, 74), <b> outside rgb(15, 23, 42)
```

The hole works (the content's `img` and `p` are untouched), and the innermost **light** link is blue although the `.light` rule is *not* the last one in the file for every box — the nearest root decided. Without `@scope`, `.dark a` and `.light a` would tie on specificity and the later `.light a` would also colour the *dark* box's link blue.

`@scope` vs the alternatives

Need	Tool
Keep a component's styles from matching outside it	`@scope (.card)`, BEM names, CSS Modules
Stop styles at a nested region (slots, user content)	`@scope … to (…)` — only this does it natively
Nearest theme wins	`@scope` proximity (or custom properties, which inherit from the nearest ancestor anyway — 13.14)
Full isolation both ways (page CSS can't get in)	Shadow DOM (a hidden, separately styled DOM tree attached to an element) (web components — custom HTML elements you define with JavaScript)

Common mistakes

- Expecting `@scope` to block **inheritance** (some properties, such as `color` and `font`, pass their value from a parent element down to its children automatically): inherited properties (colour, font) still flow into the "hole"; only *selector matching* stops.
- Thinking the root adds specificity (`@scope (#main)` adds nothing to the rules inside).

Self-check

1. What does `:scope` match inside `@scope (.card)`? → The `.card` element itself.
2. Two rules with equal specificity from `@scope (.a)` and `@scope (.b)`; the element is closer to `.b`. Which wins? → The `.b` rule (proximity), regardless of source order.
3. Is `@scope` Baseline? → Yes, newly available since March 2026.

13.6 Logical properties

What it is

Logical properties describe boxes in terms of the **flow of text** instead of physical sides. The **inline** axis is the direction text runs in a line (left→right in English, right→left in Arabic, top→bottom in vertical Japanese). The **block** axis is the direction lines/paragraphs stack. `start`/`end` are the beginning and end of each axis. So `margin-inline-start` is the margin **before the text starts** — the left in English, the right in Arabic. Baseline widely available (since 2021).

Why it matters

A site translated into Arabic, Hebrew, Persian or Urdu is displayed right-to-left (`dir="rtl"`; RTL for short, the opposite of LTR, left-to-right). With physical properties (`padding-left`, `border-left`, `left: 0`) every side needs flipping in a separate RTL stylesheet. With logical properties **one stylesheet works for both**, and also for vertical writing modes (the `writing-mode` property, used for text that runs top to bottom).

The mapping

Physical (horizontal LTR)	Logical
`width` / `height`	`inline-size` / `block-size` (also `min-` / `max-`)
`margin-left` / `margin-right`	`margin-inline-start` / `margin-inline-end`
`margin-top` / `margin-bottom`	`margin-block-start` / `margin-block-end`
`margin-left` + `margin-right`	`margin-inline: a b` (shorthand)
`padding-*`, `border-*`	`padding-inline`, `padding-block-start`, `border-inline-start`, …
`top` / `right` / `bottom` / `left`	`inset-block-start` / `inset-inline-end` / `inset-block-end` / `inset-inline-start`
`border-top-left-radius`	`border-start-start-radius` (block-start, inline-start)
`text-align: left` / `right`	`text-align: start` / `end`
`float: left`	`float: inline-start`
`overflow-x` / `overflow-y`	`overflow-inline` / `overflow-block` (newer — check support for your targets before relying on them)

Units have logical versions too: `vi` / `vb` (viewport inline/block) and `cqi` / `cqb` for containers (elements marked as containers for container queries, which let children adapt to the container's size rather than the window's) (Chapter 10).

Proof

`code/06_logical.html` uses one rule set for three containers — `dir="ltr"`, `dir="rtl"` and `writing-mode: vertical-rl`:

```
.note {
  inline-size: 220px;
  padding-inline-start: 36px;  padding-inline-end: 48px;  padding-block: 10px;
  border-inline-start: 6px solid #2563eb;
  margin-block-end: 12px;
  text-align: start;
}
.note .badge { position: absolute; inset-inline-end: 6px; inset-block-start: 6px; }
```

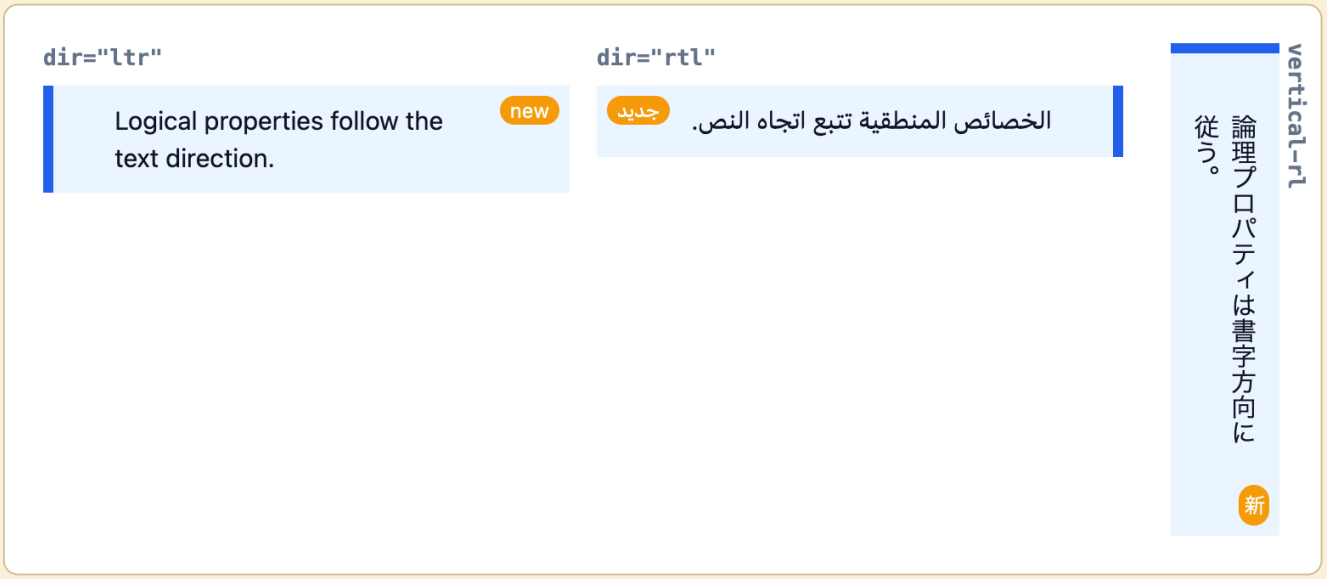


Figure 13.3 — Same CSS, three writing directions. The accent bar is always at the "start" of the text.

`code/06_logical.mjs` measures which **physical** side received each value:

Output:

```
box  size(px)   36px padding  48px padding  6px border  12px margin  badge corner
ltr  310x64     Left          Right         Left        Bottom       right/top
rtl  310x42     Right         Left          Right       Bottom       left/top
vrl  64x290     Top           Bottom        Top         Left         right/bottom
```

In vertical-rl, lines run top→bottom (so the inline axis is vertical) and stack right→left (the block axis), so inline-start = **top**, inline-end = **bottom** and block-end = **left**. The sizes follow the same logic: in LTR and RTL the box is 310px **wide** (220 `inline-size` + 36 + 48 padding + 6 border), while the vertical box (given `inline-size: 200px`) is 290px **tall** (200 + 36 + 48 + 6). Every value moved with the text direction; no rule mentions left, right, top or bottom.

When not to use it

- Things that are genuinely physical: a shadow cast downwards, a fixed "back to top" button in the bottom-right, an arrow that points right because the image does. Keep physical properties there.
- Don't mix `margin-left` and `margin-inline-start` on the same element; they are the same side in LTR and fight.

Common mistakes

- Using `dir="rtl"` on `<html>` but hard-coded `left`/`right` in components.
- Icons that imply direction (→, "next") must be mirrored in RTL: `:dir(rtl) .icon-next { scale: -1 1; }` (`:dir()` is Baseline since 2023).

Self-check

1. Logical equivalent of `padding-left: 1rem; padding-right: 1rem`? → `padding-inline: 1rem`.
2. In RTL, which physical side does `border-inline-end` draw? → Left.
3. What is `block-size` in horizontal writing? → Height.

13.7 Why CSS needs an architecture — and BEM

The problems

CSS was designed for documents, and three of its core ideas hurt at application scale:

Problem	What happens	Symptom
Global namespace (one shared pool of names)	Every selector can match anything on any page	Changing `.title` breaks a page you've never seen
Specificity & order dependence	Winners depend on selector weight and file order	Escalation: `!important`, `#id` hacks
No dead-code detection (dead code = code nothing uses any more)	You can't tell if a rule is still used	Stylesheets only ever grow ("append-only CSS")
Implicit coupling	Styles depend on DOM structure (`.nav ul li a`)	Moving markup breaks styles

Every CSS architecture is a different answer to these. There are two families: **conventions** (rules humans follow, e.g. BEM, OOCSS, SMACSS, ITCSS, CUBE CSS) and **tooling** (build steps that enforce it: CSS Modules, CSS-in-JS, utility generators). Native features (layers, `@scope`, nesting) now help both.

BEM — Block, Element, Modifier

BEM (from Yandex, a Russian search company, ~2009) is a naming convention that gives every styled thing a unique, flat class:

- **Block** — a standalone component: `.card`
- **Element** — a part that only makes sense inside its block: `.card__title` (double underscore)
- **Modifier** — a variant or state: `.card--featured`, `.card__button--disabled` (double hyphen)

`code/07_bem.html`:

```css
.card { width: 170px; padding: 14px; border: 1px solid #cbd5e1; border-radius: 12px; }
.card__title  { margin: 0; font-size: 16px; }
.card__price  { margin: 4px 0 12px; color: #475569; }
.card__button { width: 100%; background: #2563eb; color: #fff; /* … */ }
.card--featured { border: 2px solid #f59e0b; background: #fffbeb; }
.card--featured .card__button { background: #d97706; }
```

```html
<article class="card card--featured">
  <h3 class="card__title">Rain jacket</h3>
  <p class="card__price">$129</p>
  <button class="card__button">Add to cart</button>
</article>
```

Why it works: nearly every selector is a single class → specificity (0,1,0) everywhere, so order and weight stop mattering; names say where things belong; no dependence on tag names or nesting depth. **Rules:** no element-of-element names (`card__body__title` → just `card__title`); modifiers never appear alone (`class="card--featured"` without `card` is a bug); avoid styling by tag inside blocks.

Other conventions in one line each

- **OOCSS** (Object-Oriented CSS, Nicole Sullivan, 2008): separate *structure* from *skin*, and *container* from *content* — reusable "objects" like `.media`.
- **SMACSS**: sort rules into Base, Layout, Module, State (`.is-open`), Theme.
- **ITCSS** (Inverted Triangle CSS): order files from generic/low-specificity to explicit/high-specificity (settings → tools → generic → elements → objects → components → utilities) — cascade layers (13.4) are now the native way to express this triangle.
- **CUBE CSS**: Composition, Utility, Block, Exception — lean on the cascade for most styling, add blocks only where needed, exceptions via data attributes.

Trade-offs

BEM needs discipline and code review (teammates reading each change before it is merged) (nothing enforces the names), class names get long, and there's still one global namespace — two teams can both invent `.card`. That last problem is what tooling (13.9, 13.10) and `@scope` solve.

Self-check

1. Name for a "large" variant of a `.btn`'s `icon` element? → `.btn__icon--large`.
2. Why does BEM avoid `.card h3`? → It couples styles to markup and raises specificity; use `.card__title`.

13.8 Utility-first CSS and Tailwind CSS v4

What it is

Utility-first CSS provides many tiny single-purpose classes — `flex`, `p-4`, `text-slate-600`, `rounded-lg` — and you compose them **in the HTML** instead of writing new CSS for each component. **Tailwind CSS** is the dominant utility framework; **v4** (released January 2025; 4.3.3 used here) moved configuration into CSS and builds on modern features — cascade layers, `@property` (registers a custom property with a type so the browser can animate and validate it), `color-mix()` (blends two colours) and custom properties for every **design token** (a named design decision — a brand colour, a spacing step, a radius — stored as a variable so it is defined once and reused everywhere).

Why teams choose it

- **CSS stops growing**: a new component reuses existing utilities, so the stylesheet size tracks the number of *distinct* utilities, not the number of components.
- **No naming** and no switching between files; deleting HTML deletes its styles (no dead CSS).
- **Constraints**: utilities come from a design scale (spacing multiples, a colour palette), which keeps UIs consistent.
- **Generated on demand**: the build scans your source files and emits only the classes you actually used.

How a v4 build works

1. Your CSS entry file imports Tailwind and optionally defines design tokens in an `@theme` block.
2. The build scans source files for strings that look like class names (it doesn't run your code — class names must appear **complete** in the source).
3. For each recognised class it emits a rule into `@layer utilities`, plus the theme variables those rules use, the Preflight reset (Tailwind's own reset, 13.13) in `@layer base`, and `@property` registrations some utilities need.

`code/08_tailwind.css` and part of `code/08_tailwind.html`:

```
@import "tailwindcss" source(none);    /* don't auto-scan the folder ...        */
@source "./08_tailwind.html";           /* ... scan exactly this file             */

@theme {                                /* tokens -> CSS variables AND utilities */
  --color-brand: #2563eb;               /* enables bg-brand, text-brand, ...     */
  --color-brand-dark: #1d4ed8;
}
```

```
<article class="w-[170px] rounded-xl border border-slate-300 bg-white p-3.5">
  <h3 class="text-base font-bold">Trail shoes</h3>
  <p class="mt-1 mb-3 text-slate-600">$89</p>
  <button class="w-full rounded-lg bg-brand p-2 text-white hover:bg-brand-dark">Add to cart</button>
</article>
```

(By default v4 detects sources automatically, skipping `.gitignore`d files — files listed in Git's ignore list, so not part of the project's version history; `source(none)` + `@source` just makes this example **deterministic**, giving the same output on every run.) `code/08_tailwind_build.mjs` runs the real CLI (command-line interface — the program you run from a terminal) and inspects the output:

Output:

```
classes used in the HTML: 28
utility rules generated:   28
output: 7623 bytes (6513 minified)
top-level at-rules: layer properties | layer theme, base, components, utilities | layer theme  | layer base  | layer
utilities  | property --tw-border-style | property --tw-leading ...
theme variables emitted: --color-amber-50 --color-amber-500 --color-amber-600 --color-amber-700 --color-slate-300 --color-
slate-600 --color-slate-900 --color-white --color-brand --color-brand-dark

  .p-3\.5 {
    padding: calc(var(--spacing) * 3.5);
  }
  .bg-brand {
    background-color: var(--color-brand);
  }
  .w-\[170px\] {
    width: 170px;
  }
    .hover\:bg-brand-dark:hover {
      background-color: var(--color-brand-dark);
    }
```

What to notice:

- **Only what you use**: 28 classes in → 28 rules out, and only the 10 colour variables those classes reference (Tailwind's palette has hundreds). The whole stylesheet, including the reset, is about 6.5 KB before compression ("minified" in the output means whitespace and comments removed; compression such as gzip or Brotli then shrinks the file further for download).
- **Layers**: `@layer theme, base, components, utilities;` — utilities are the strongest layer, so a utility always beats a component class written in `components`, with no specificity games. Your own unlayered CSS beats all of them (13.4).
- **Spacing is a formula**: `p-3.5` = `calc(var(--spacing) * 3.5)` with `--spacing: 0.25rem` → 14px. Change one variable to rescale the whole UI.
- **Arbitrary values** in square brackets (`w-[170px]`) escape the scale when needed.
- **Variants** (prefixes that apply a utility only in a certain state or screen size): `hover:` wraps the rule in `@media (hover: hover)` (in the full output) so touch devices don't get "sticky hover" (a hover style that stays stuck after a tap); `md:` becomes `@media (width >= 48rem)`.

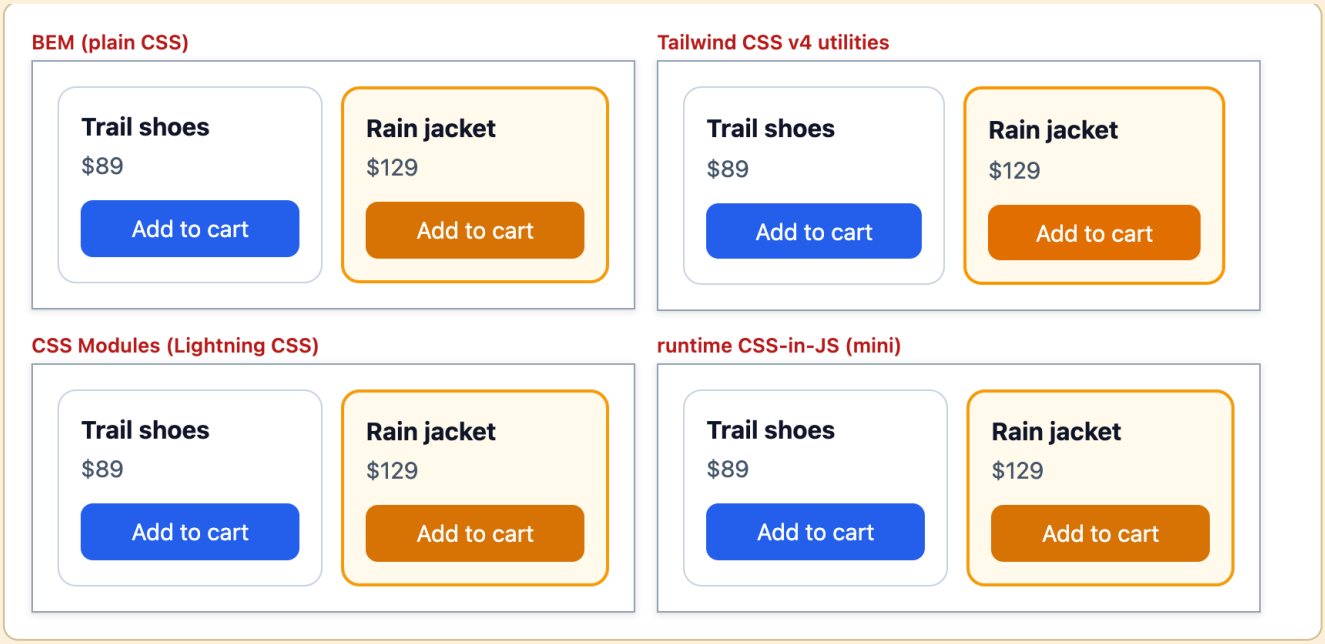


Figure 13.4 — The same two cards built four ways (13.7–13.10). The pixels are the same; what differs is where styles live and how they're scoped.

Trade-offs

Pro	Con
Tiny, bounded CSS; no dead code	Long `class` attributes; HTML harder to scan
No naming, no specificity fights	Repetition across markup unless you extract **components** (React/Vue — JavaScript UI libraries — components, or partials: reusable template snippets)
Design constraints built in	Learning the vocabulary; team must accept styling in markup
Easy to delete / refactor (restructure without changing behaviour) locally	Class names must be static strings (`bg-${color}-500` built at runtime — while the page runs — won't be found)

Common mistakes

- **Building class names dynamically** (`'text-' + color`): the scanner can't see them, so no CSS is generated. Map to full names: `{ red: 'text-red-600', blue: 'text-blue-600' }[color]`.
- Recreating BEM with `@apply` (a Tailwind directive that copies utilities into your own CSS rule) everywhere — you lose the benefits of both. Use it sparingly.
- Forgetting accessibility: utilities don't add `:focus-visible` styles for you (`focus-visible:ring-2` does).

In the interview

> **30-second answer:** "Utility-first CSS composes small single-purpose classes in the markup. Tailwind scans the source files and generates only the utilities used, into cascade layers, so CSS size stays bounded and there's no dead code or specificity war. The costs are verbose markup and a learning curve; you control repetition by extracting components, and class names must appear in full in the source."

Self-check

1. Why does `className={'bg-' + c + '-500'}` fail? → The class never appears in full in the source, so the build doesn't generate it.
2. Where does v4 keep configuration? → In CSS, e.g. `@theme { --color-brand: … }`.
3. Which layer do Tailwind utilities live in? → `utilities`, the last of `theme, base, components, utilities`.

13.9 CSS Modules

What it is

A **CSS Module** is an ordinary CSS file (named `*.module.css` by convention) whose class names are **local by default**: the build step swaps every class for a generated, collision-free name and gives JavaScript an object that maps your short names to the generated ones. You write `.title`; the page gets `.title_14He7q`; your component writes `className={styles.title}`. Supported out of the box by Vite, Next.js, webpack's css-loader, Parcel and others (popular build tools and frameworks for JavaScript apps).

How it works

1. You import the file from a component: `import styles from './Card.module.css'`.
2. At build time the compiler parses the CSS and renames every class and `@keyframes` name using a pattern (file name, class name, hash — a short code computed from the file's contents, so different files get different codes).
3. It emits the renamed CSS into the bundle (the combined output file) and returns the **name map** as the module's default export (the main value other files receive when they `import` it).
4. `composes: other` adds another class's generated name to the export, so the element gets both classes.
5. `:global(...)` opts a selector out of renaming.

`code/09_card.module.css`:

```
.card   { width: 170px; padding: 14px; border: 1px solid #cbd5e1; border-radius: 12px; background: #fff; }
.title  { margin: 0; font-size: 16px; }
.button { width: 100%; background: #2563eb; /* … */ }
.featured { composes: card; border: 2px solid #f59e0b; background: #fffbeb; }
.featured .button { background: #d97706; }
:global(body) { font: 15px/1.4 system-ui, sans-serif; margin: 16px; display: flex; gap: 12px; }
```

`code/09_css_modules.mjs` compiles it with Lightning CSS and renders the component HTML with the generated names (Figure 13.4, third panel):

Output:

```
import styles from "./card.module.css"  ->  styles = {
  button: 'button_14He7q',
  card: 'card_14He7q',
  featured: 'featured_14He7q card_14He7q',
  price: 'price_14He7q',
  title: 'title_14He7q'
}

Generated CSS (first rules):
.card_14He7q {
  background: #fff;
  border: 1px solid #cbd5e1;
  border-radius: 12px;
  width: 170px;
  padding: 14px;
}

.title_14He7q {
  margin: 0;
  font-size: 16px;
}

wrote 09_css_modules.out.css and 09_css_modules.html
```

The same `.title` in another module file gets a different hash, so the two can never collide. `styles.featured` contains **two** class names because of `composes`.

Trade-offs

Pro	Con
Real CSS (all features, DevTools, caching — the browser keeps a downloaded copy for reuse — as a static file)	Needs a build tool and JS imports
Zero runtime cost	Styling by props (the inputs a component receives) means toggling classes (`clsx(styles.btn, big && styles.big)`)
Scoping enforced by the tool, not discipline	Generated names are unreadable in production (the live site real users see) (fine in dev with a readable pattern)
Unused module files can be detected (no import → dead)	Unused classes *within* a file aren't removed automatically

> **Don't confuse with "CSS module scripts"**: `import sheet from './x.css' with { type: 'css' }` gives you a `CSSStyleSheet` object (for shadow DOM). That's a different, newer browser feature — **not Baseline** (no Safari as of September 2026).

Self-check

1. What does `import styles from './a.module.css'` give you? → An object mapping local class names to generated unique names.
2. How do you write a global rule inside a module? → `:global(.selector) { … }`.

13.10 CSS-in-JS

What it is

CSS-in-JS means writing styles in JavaScript, next to the component, often computed from props. Two kinds exist:

- **Runtime** libraries (styled-components, Emotion): in the browser, while rendering, they **serialise** the style object (turn it into a string), hash it into a class name, and inject a rule into a `<style>` element.
- **Zero-runtime / compile-time** libraries (vanilla-extract, Linaria, Panda CSS, StyleX): you write styles in JS/TS (TypeScript — JavaScript with type annotations), but a build step extracts them to static CSS files, like CSS Modules.

How runtime CSS-in-JS works — built in 25 lines

`code/10_mini_css_in_js.js` is a toy version of the mechanism (not a real library):

```
function css(styles) {
  const key = JSON.stringify(styles);
  if (cache.has(key)) return cache.get(key);          // same styles -> same class, inject once
  const cls = 'css-' + hash(key);                     // name derived from content
  sheet.insertRule(`.${cls}{${toCss(styles)}}`);      // + nested '&:hover' rules
  cache.set(key, cls);
  return cls;
}
// a component: styles depend on props
el.className = css({ border: featured ? '2px solid #f59e0b' : '1px solid #cbd5e1', /* … */ });
```

`code/10_css_in_js.mjs` renders two cards in jsdom (a JavaScript imitation of a browser's DOM that runs inside Node.js) (the fourth panel of Figure 13.4 is the same code in Chrome, `code/10_css_in_js.html`):

Output:

```
card 1 classes: css-1i6k3fq css-1qt5iag css-1d6s3yc css-11at6p7
card 2 classes: css-kegznd css-1qt5iag css-1d6s3yc css-n2x86t

rules injected at runtime (8):
  .css-1i6k3fq { width: 170px; padding: 14px; border-radius: 12px; border: 1px solid rgb(203, 213, 225); backgro
  .css-1qt5iag { margin: 0px; font-size: 16px; }
  ...
  .css-kegznd { width: 170px; padding: 14px; border-radius: 12px; border: 2px solid rgb(245, 158, 11); backgroun
  .css-n2x86t { width: 100%; padding: 8px; border: 0px; border-radius: 8px; background: rgb(217, 119, 6); color
  .css-n2x86t:hover { filter: brightness(0.9); }

progress bar rendered at 0..100%: 101 extra rules
with width: var(--pct) instead: 1 class (css-1gyb3r) + style="--pct: 42%"
```

What it shows:

- Identical styles share a class (`css-1qt5iag` for both titles) — the cache (a lookup table of results already computed) deduplicates.
- Different props produce **different classes** (the two card backgrounds and buttons).
- **The cost**: a style that depends on a continuously changing value (a progress bar) creates a new rule for every value — 101 rules — and the work of serialising, hashing and inserting happens **during rendering, on the main thread**. The fix every CSS-in-JS guide recommends is exactly the native one: one static rule with a custom property (`width: var(--pct)`) set inline.

Trade-offs

Pro	Con
Styles colocated with components; dynamic styles from props are natural	Runtime cost: JS bundle + style work during render (hurts INP/LCP, Chapter 14)
Scoped automatically; dead styles die with the component	Server rendering needs extra setup to collect styles; a poor fit for **React Server Components** (runtime libraries have traditionally needed client components)
TypeScript types for style props (zero-runtime libs)	CSS can't be cached separately from JS (runtime libs)

Key terms — INP, LCP, server rendering, React Server Components: **LCP** (Largest Contentful Paint) and **INP** (Interaction to Next Paint) are two of Google's Core Web Vitals: how fast the main content appears, and how quickly the page reacts to clicks and key presses. **Server rendering** (SSR) means the server builds the page's HTML before sending it, instead of JavaScript building it in the browser. **React Server Components** (RSC) are React components that run only on the server and send no JavaScript to the browser; *client components* are the ones that also run in the browser.

The ecosystem has moved towards zero-runtime tools, CSS Modules and Tailwind. In March 2025 the styled-components maintainer announced **maintenance mode** (bug fixes only, no major new features) and said he would not recommend it for new projects, citing its poor fit with React Server Components. The library is still maintained, though: v6.4 (April 2026) added RSC-friendly theming through CSS custom properties and faster re-renders — so existing projects are safe, but the general advice for new work stands.

Self-check

1. Why can runtime CSS-in-JS hurt responsiveness? → Styles are serialised/hashed/inserted on the main thread during render, and inserting rules triggers style recalculation.
2. What is "zero-runtime" CSS-in-JS? → Styles authored in JS but extracted to static CSS at build time.

13.11 Choosing an approach

	BEM (plain CSS)	Utility-first (Tailwind)	CSS Modules	Runtime CSS-in-JS	Zero-runtime CSS-in-JS
Scoping	Convention	N/A (no custom selectors)	Build tool (renamed classes)	Generated classes	Build tool
Build step	Optional	Required	Required	No (runtime)	Required
Runtime cost	None	None	None	**Yes**	None
Dynamic styles	Classes / custom props	Classes / custom props	Classes / custom props	Props → styles	Variants / custom props
Dead-code removal	Manual	Automatic	Per file	Automatic	Automatic
CSS size growth	Grows with features	Plateaus	Grows with features	Grows with features	Grows / atomic (one class per declaration) (StyleX) plateaus
Works without a JS framework	Yes	Yes	Needs a bundler	No	Needs a bundler
Good fit	Content sites, CMS (content management system, e.g. WordPress) themes, small teams	Product UIs, design systems with fixed scale	React/Vue apps wanting "just CSS"	Legacy apps already using it	TS-heavy apps wanting typed styles

Practical defaults in 2026:

1. **A content site / no framework** → plain CSS with layers, nesting, a BEM-like naming scheme or `@scope`, and tokens.
2. **A React/Vue/Svelte (another component framework) app** → Tailwind *or* CSS Modules (Svelte and Vue also scope `<style>` blocks per component natively) + design tokens as custom properties.
3. **Whatever you choose**: keep a reset and tokens in low layers, make dynamic values custom properties, and don't mix three approaches in one codebase (all the source code of a project).

In the interview

30-second answer: "CSS's global namespace and specificity cause most maintenance pain. BEM solves it by convention with flat single-class names; CSS Modules by renaming classes at build time; utility-first by generating single-purpose classes so you rarely write new CSS; CSS-in-JS by generating classes from JS. I'd pick CSS Modules or Tailwind for a component app, plain CSS with layers and BEM or `@scope` for a content site, and avoid runtime CSS-in-JS in new projects because of its render-time cost and poor fit with server components."

- *Q: How would you migrate a big legacy stylesheet?* A: Wrap it in `@layer legacy` (low), write new components in a higher layer with the chosen approach, delete legacy rules as pages are migrated.

13.12 Preprocessors (Sass) and PostCSS

What it is

A **preprocessor** is a language that compiles to CSS. **Sass** (with its SCSS syntax, a superset of CSS — every valid CSS file is also valid SCSS) is by far the most used; Less and Stylus are older alternatives. **PostCSS** is different: a tool that parses CSS into a tree and runs plugins (small add-on programs) on it (Autoprefixer adds **vendor prefixes** — browser-specific spellings such as `-webkit-` that older browsers needed; others lower new syntax). **Lightning CSS** is a fast all-in-one CSS compiler (lowering, prefixing, minifying — stripping spaces and comments to shrink the file — CSS Modules) written in Rust (a fast systems programming language) — used in 13.1 and 13.9.

What Sass adds

`code/11_sass_example.scss` — **(Sass — not run: Sass isn't installed in this book's toolchain — the set of tools used to build and test it)**:

```
@use "sass:color";
@use "sass:math";
$brand: #2563eb;                        // compile-time variable
$sizes: (sm: 12px, md: 16px, lg: 20px);

@mixin focus-ring($c: $brand) { outline: 2px solid $c; outline-offset: 2px; }
%button-base { border: 0; font: inherit; }   // placeholder for @extend

.card {
  padding: math.div(24px, 2);          // -> 12px
  &__title { font-size: 20px; }        // -> .card__title (string joining)
  &--featured { border-color: color.adjust($brand, $lightness: -10%); }
}
.button {
  @extend %button-base;
  &:focus-visible { @include focus-ring; }
}
@each $name, $size in $sizes { .text-#{$name} { font-size: $size; } }   // generates 3 classes
```

Modern Sass notes: use the module system (`@use` / `@forward`, which loads each file once and keeps its names private unless shared) — `@import` is **deprecated** (still works but is scheduled for removal; don't use it in new code) since Dart Sass 1.80 (October 2024) and will be removed in Dart Sass 3.0; global functions like `darken()` are deprecated in favour of `color.adjust()` / `color.scale()`; **Dart Sass** is the only maintained implementation (LibSass and node-sass are deprecated).

What native CSS now covers

`code/11_native.html` rewrites the Sass ideas in plain CSS, and `code/11_native.mjs` checks Chrome's results. (`oklch(L C H)` writes a colour as lightness, chroma — how vivid — and hue angle, in a colour space designed so equal number steps look like equal visual steps.)

```
:root { --brand: #2563eb; --radius: 8px; }
.card {
  padding: calc(24px / 2);
  .card-title { font-size: 20px; }
  &.is-featured { border-color: oklch(from var(--brand) calc(l - 0.1) c h); }  /* relative colour */
}
.button {
  background: var(--brand);
  &:hover { background: color-mix(in oklch, var(--brand), black 15%); }
}
.theme-green { --brand: #16a34a; }     /* runtime re-theming */
```

Output:

```
padding: calc(24px / 2)                 -> 12px
border-color: oklch(from --brand l-0.1) -> oklch(0.44613 0.215194 262.885)
.button background (var(--brand))        -> rgb(37, 99, 235)
same rule under .theme-green             -> rgb(22, 163, 74)
hover: color-mix(in oklch, brand, black 15%) -> oklch(0.464211 0.182915 262.885)
```

Sass feature	Native CSS today	Status (Sept 2026)
`$variables`	Custom properties `--x` — **live**, inherit, can change per element and at runtime (the green button above)	Widely available
Nesting	Native nesting (13.1) — no `&__` joining	Widely available
`math.div`, arithmetic	`calc()`, `min()`, `max()`, `clamp()`, `round()`, `mod()`	`calc` etc. widely; `round()/mod()/rem()` newly available (2024)
`darken()` / `color.adjust`	`color-mix()` (widely) and relative colour syntax (derive a new colour from an existing one) `oklch(from …)` (newly available, Sept 2024)	✓
Partials / `@use`	`@import` (blocks parallel loading — the browser can't fetch the imported file until it has read the first — avoid in production) or bundler concatenation (joining files into one); `@import … layer()`	✓
`@mixin`	CSS `@mixin` / `@apply` spec is still being designed	No browser
`@function`	Custom functions `@function --name()`	**Chromium only** (139+)
`@if` / `if()`	`if()` function	**Chromium only** (137+)
`@each` / loops generating classes	None — generate at build time (or use utilities)	—
`@extend`	None (and rarely a good idea: it rewrites selectors unpredictably)	—

Verdict. For new projects, native CSS covers most daily Sass use; keep Sass if you rely on mixins, loops or an existing Sass design system. Whichever you use, a PostCSS/Lightning CSS step for prefixes and lowering is still common.

Self-check

1. Why can't a Sass `$brand` be changed for one section at runtime? → It's replaced by its value at compile time; only custom properties exist in the browser.
2. Which Sass rule is deprecated and replaced by `@use`? → `@import`.

13.13 Resets and normalize

What it is

Every browser applies a **user-agent stylesheet** (its default CSS): 8px body margin, margins on headings and paragraphs, bullets and 40px padding on lists, form controls with their own small font. A **reset** removes or neutralises these defaults so you start from a predictable base. Three generations:

1. **Hard resets** (Eric Meyer's reset, 2007/2011): zero out almost everything, including list styles and heading sizes — you must restyle all of it.
2. **normalize.css** (2012): keep useful defaults but make browsers **consistent** with each other and fix bugs.
3. **Modern minimal resets** (Andy Bell, Josh Comeau, and the resets built into frameworks such as Tailwind's Preflight): a dozen rules that fix today's real annoyances. Browsers are now consistent enough that normalizing matters less.

A modern reset, line by line

`code/12_reset.css` — in the lowest layer so anything can override it:

```css
@layer reset {
  *, *::before, *::after { box-sizing: border-box; }          /* width includes padding + border */
  * { margin: 0; }                                             /* add spacing on purpose */
  html { -webkit-text-size-adjust: none; text-size-adjust: none; }  /* no iOS text inflation */
  body { min-height: 100dvh; line-height: 1.5; -webkit-font-smoothing: antialiased; }
  img, picture, video, canvas, svg { display: block; max-width: 100%; }
  input, button, textarea, select { font: inherit; }          /* controls don't inherit fonts by default */
  p, h1, h2, h3, h4, h5, h6 { overflow-wrap: break-word; }    /* long URLs can't overflow */
  h1, h2, h3, h4, h5, h6 { text-wrap: balance; }
  p { text-wrap: pretty; }
  :where(ul, ol)[role="list"] { list-style: none; padding: 0; }
  @media (prefers-reduced-motion: reduce) {
    *, *::before, *::after { animation-duration: 0.01ms !important; transition-duration: 0.01ms !important; scroll-
behavior: auto !important; }
  }
}
```

`code/12_reset.mjs` loads `code/12_reset.html` at 400px wide, measures, adds the reset, and measures again:

Output:

```
property                              browser default          with reset
body margin                           8px                      0px
h1 margin-block                       21.44px 21.44px          0px 0px
h1 font-size                          32px                     32px
#box width (200px + padding + border) 250px                    200px
img display / rendered width          inline / 800px           block / 400px
button font                           13.3333px Arial          16px Times
ul list-style / padding-left          disc / 40px              none / 0px
p scrollWidth (overflow?)             606px of 384px           400px of 400px
```

Each row is one fix: margins gone; `box-sizing` makes a `width: 200px` box actually 200px (Chapter 7); the 800px image no longer overflows a 400px screen; the button now uses the page font (here the default Times, since this demo sets no font — normally your `body` font); the list with `role="list"` lost its bullets; and the long URL wraps instead of overflowing (606px of content in a 384px box before). Heading sizes are untouched — this reset only removes *surprises*.

Why some lines are written that way:

- `role="list"`: Safari's VoiceOver (Apple's built-in **screen reader** — software that reads the page aloud for blind and low-vision users) stops announcing a list as a list when `list-style: none` is applied; adding `role="list"` restores it, and the reset only strips bullets from lists that carry it.
- `:where()` keeps that rule at low specificity (13.2); the **layer** keeps *all* of it below your styles (13.4).
- `text-wrap: pretty` is not Baseline (no Firefox as of September 2026) — harmless where unsupported. `text-size-adjust` is also non-standard/partial, hence both spellings (the `-webkit-` vendor-prefixed one and the standard one).
- **Never** put `outline: none` / `*:focus { outline: 0 }` in a reset — it removes the keyboard focus indicator (the outline that shows which element keyboard input will go to) (Chapter 3).

Self-check

1. Why wrap a reset in `@layer reset`? → So every later style overrides it without specificity fights.
2. Why does the reset set `font: inherit` on buttons? → Form controls don't inherit font properties by default; they'd use the system's small UI font.

13.14 Theming and dark mode with custom properties

What it is

A **theme** is a set of values (colours, radii, fonts, spacing) applied consistently across components. The modern way to build one is with **design tokens** stored as **custom properties**, organised in three tiers, plus `color-scheme` (tells the browser which light/dark modes the page supports) and `light-dark()` (picks one of two values depending on the active mode) for dark mode. Chapter 10 introduced `prefers-color-scheme` and the basic dark-mode steps; Chapter 6 explained custom properties. This section puts them together into an architecture.

The three tiers of tokens

Tier	Example	Who uses it
Primitive (reference) tokens	`--blue-600: #2563eb`	Only the semantic tokens
Semantic (system) tokens	`--color-accent`, `--color-surface`, `--color-text`	Components
Component tokens (optional)	`--card-bg: var(--color-surface)`	One component; lets a single instance be tweaked

Components never mention a raw colour. To add dark mode or a new brand you change **only semantic tokens**.

How it works

1. `color-scheme: light dark` on `:root` (the document's root element, `<html>`) tells the browser the page supports both. The browser then themes its own UI (form controls, scrollbars, default background) and **`light-dark(a, b)` picks `a` or `b` based on the element's used colour scheme**. `light-dark()` is Baseline newly available since May 2024 (colours only; image values are newer still).
2. Each semantic token gets a light and dark value with `light-dark()` — no duplicated `@media` block.
3. A user choice **overrides** the OS preference by pinning `color-scheme`: `:root[data-theme="dark"] { color-scheme: dark }`. Because `light-dark()` follows `color-scheme`, every token switches automatically.
4. A tiny **blocking script in `<head>`** (one the browser must run before it continues rendering) reads the saved choice from `localStorage` (a small key–value store the browser keeps per site, which survives reloads) and sets `data-theme` (a custom `data-*` attribute) *before the first paint* (the first moment anything is drawn on screen), so there's no flash of the wrong theme (a small, justified exception to "don't block rendering", Chapter 14).
5. Other axes (brand, density, high contrast — a mode with stronger colour differences for low-vision users) are more attribute-scoped token overrides: `[data-brand="green"] { --color-accent: … }`. Custom properties inherit, so an override on any element re-themes its whole subtree.

`code/13_theming.html` (key parts):

```
<meta name="color-scheme" content="light dark">
<script>
  try { const t = localStorage.getItem('theme');
        if (t === 'light' || t === 'dark') document.documentElement.dataset.theme = t; } catch {}
</script>
```

```
@layer tokens, base, components;
@layer tokens {
  :root {
    --blue-600: #2563eb;  --blue-300: #93c5fd;  --slate-900: #0f172a; /* … primitives */
    color-scheme: light dark;
    --color-bg:      light-dark(var(--slate-50), var(--slate-900));
    --color-surface: light-dark(#fff, var(--slate-800));
    --color-text:    light-dark(var(--slate-900), var(--slate-200));
    --color-accent:  light-dark(var(--blue-600), var(--blue-300));
  }
  :root[data-theme="light"] { color-scheme: light; }
  :root[data-theme="dark"]  { color-scheme: dark; }
  [data-brand="green"] { --color-accent: light-dark(#15803d, #86efac); }
}
@layer components {
  .card a { color: var(--color-accent); }
  .btn { background: var(--color-accent); color: var(--color-surface); }
}
```

```
select.addEventListener('change', () => {
  if (select.value === 'system') delete root.dataset.theme; else root.dataset.theme = select.value;
  try { select.value === 'system' ? localStorage.removeItem('theme') : localStorage.setItem('theme', select.value); }
catch {}
});
```

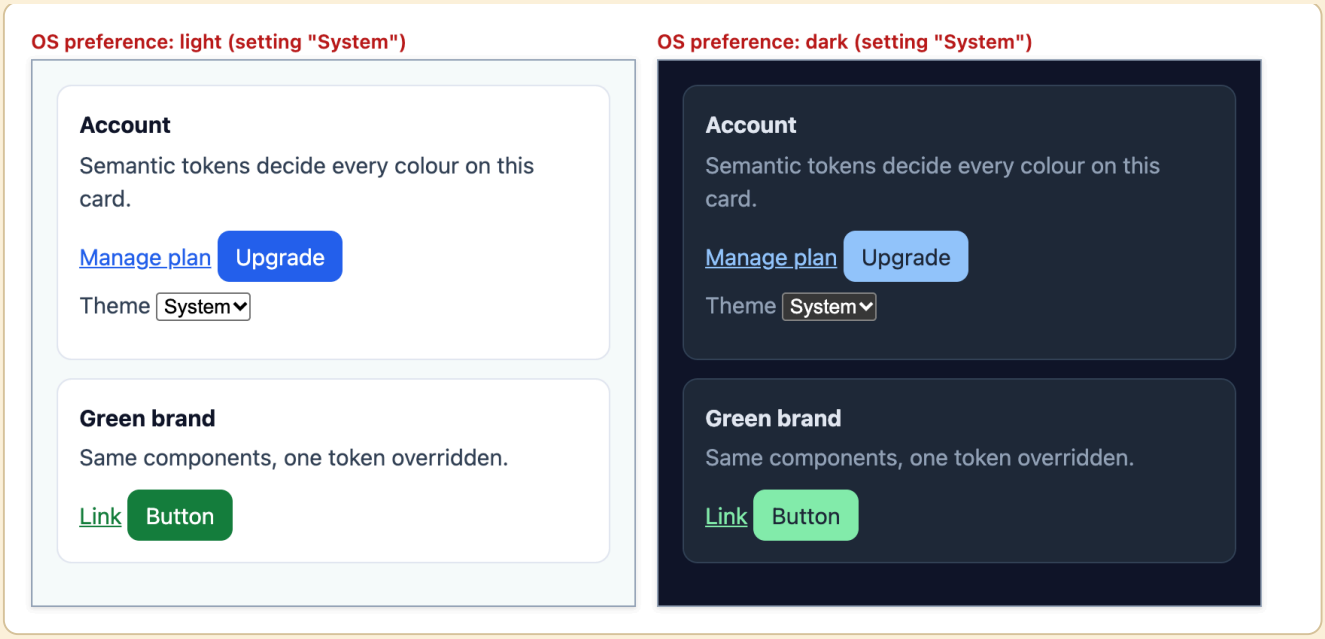


Figure 13.5 — The same page with the setting on "System", under an emulated light and dark OS preference. The select box and page background are themed by `color-scheme` without any CSS for them.

`code/13_theming.mjs` tests every combination of OS preference × page setting, then reloads to test persistence:

Output:

```
OS pref  setting   page bg              text                accent (link)       green-brand link
light    system    rgb(248, 250, 252)   rgb(15, 23, 42)     rgb(37, 99, 235)    rgb(21, 128, 61)    saved=null
light    light     rgb(248, 250, 252)   rgb(15, 23, 42)     rgb(37, 99, 235)    rgb(21, 128, 61)    saved=light
light    dark      rgb(15, 23, 42)      rgb(226, 232, 240)  rgb(147, 197, 253)  rgb(134, 239, 172)  saved=dark
dark     system    rgb(15, 23, 42)      rgb(226, 232, 240)  rgb(147, 197, 253)  rgb(134, 239, 172)  saved=null
dark     light     rgb(248, 250, 252)   rgb(15, 23, 42)     rgb(37, 99, 235)    rgb(21, 128, 61)    saved=light
dark     dark      rgb(15, 23, 42)      rgb(226, 232, 240)  rgb(147, 197, 253)  rgb(134, 239, 172)  saved=dark

after reload: data-theme=dark  body bg=rgb(15, 23, 42)  <select> shows "dark"
```

All six combinations are right: "system" follows the OS, an explicit choice wins over the OS, and the brand override keeps its own light/dark pair. After a reload the saved "dark" was applied by the head script before the page rendered.

Going further

- **Typed tokens with `@property`** (Baseline newly available since July 2024): registering `@property --accent { syntax: '<color>'; inherits: true; initial-value: #2563eb; }` lets the browser *interpolate* the token (animate theme changes) and rejects invalid values.
- **Fallback for browsers without `light-dark()`**: define tokens in `:root` for light and override them inside `@media (prefers-color-scheme: dark)` and `[data-theme=dark]` — the Chapter 10 pattern.
- **Contrast**: check every token pair (text on bg, text on surface, accent on surface) at ≥ 4.5:1 (the contrast ratio — how much lighter one colour is than the other, from 1:1 for identical up to 21:1 for black on white — that WCAG requires for normal text) in *both* themes (Chapter 11).
- **`contrast-color()`** (returns black or white for readable text on a given background) became Baseline in April 2026 — useful for user-chosen brand colours.

Common mistakes

- Using `light-dark()` without `color-scheme` set → it always picks the light value.
- Components using primitive tokens (`var(--blue-600)`) directly — then dark mode needs component changes.
- Setting the theme in a `useEffect` (a React hook that runs after the component has rendered)/`DOMContentLoaded` (the event fired once the HTML has been parsed) handler → the page paints light, then flashes dark.
- Storing the theme only in a cookie (a small piece of data the browser stores and sends to the server with each request) read by JavaScript after load — same flash.

In the interview

> **30-second answer:** "I define primitive tokens, then semantic tokens like `--color-surface` that components use. For dark mode I set `color-scheme: light dark` and give each semantic token a `light-dark()` pair; a user override just pins `color-scheme` on the root via a `data-theme` attribute, saved in localStorage and applied by a tiny inline script in the head to avoid a flash. Extra themes are scoped token overrides, which work because custom properties inherit."

- *Q: Why does a `data-theme` override work without duplicating every token?* A: `light-dark()` follows the element's `color-scheme`, so changing that one property flips all tokens.
- *Q: How do you avoid the dark-mode flash?* A: Apply the saved theme synchronously in `<head>` before first paint (or render it server-side from a cookie).

Self-check

1. What does `light-dark(#fff, #000)` return when `color-scheme` is `light dark` and the OS is dark? → `#000`.
2. Which tier of token should a button reference? → Semantic (`--color-accent`).
3. Why prefer `data-theme` on `<html>` over a class on `<body>`? → Tokens are defined on `:root`, and the head script can set it before `<body>` exists.

Chapter 13 TL;DR

- **Nesting** (Baseline 2023): nested selectors are relative to the parent; `&` = parent, like `:is()` (highest specificity of the parent list); no `&__` string joining (Chrome dropped that rule); nested `@media` works; Lightning CSS can lower it for old browsers.
- **`:is()` / `:where()`**: "any of"; forgiving lists (unknown selectors ignored); `:is()` = max argument specificity, `:where()` = 0 → use `:where()` for resets and library defaults.
- **`:has()`** (Baseline 2023): parent / previous-sibling / state selector (`.card:has(img)`, `h3:has(+ .subtitle)`, `label:has(:checked)`, `html:has(dialog[open])`); no nesting of `:has()`; anchor it for performance.
- **`@layer`**: declare the order first; later layers win regardless of specificity (a class in `components` beat `#app .btn.btn-primary` in `vendor`); unlayered beats layers; `!important` flips; `revert-layer`; `@import … layer()`.
- **`@scope`** (Baseline March 2026): `@scope (.card) to (.content)` donut scope; `:scope`; proximity breaks specificity ties (nested themes); prelude-less `@scope` in inline `<style>`.
- **Logical properties**: `inline`/`block` + `start`/`end`; one stylesheet for LTR, RTL and vertical text (measured: every value moved with the text direction).
- **Architecture**: global namespace + specificity + dead code are the problems. **BEM** (`block__element--modifier`, flat single classes), **Tailwind v4** (generated utilities in `@layer utilities`, only what's used — 28 classes → 28 rules, ~6.5 KB), **CSS Modules** (build renames `.title` → `.title_14He7q`, `composes`), **CSS-in-JS** (runtime: hash + inject during render, 101 rules for 101 progress values; prefer zero-runtime or custom properties).
- **Sass**: variables, mixins, loops, `&__` joining; `@use` not `@import`; native CSS now covers variables, nesting, colour maths (`color-mix()`, relative colours) — not mixins/loops (`@function`/`if()` Chromium-only).
- **Reset**: small modern reset in `@layer reset` (box-sizing, margins, media `max-width`, `font: inherit`, `overflow-wrap`); never remove focus outlines.
- **Theming**: primitive → semantic → component tokens; `color-scheme` + `light-dark()` (Baseline 2024); override with `[data-theme]` pinning `color-scheme`; persist in localStorage and apply in `<head>` before first paint.

Next: Chapter 14 — Performance, SEO, Interview Prep and Cheat Sheets.

Chapter 14 — Performance, SEO, Interview Prep and Cheat Sheets

A page can be semantic, accessible and beautifully laid out and still fail its users if it takes four seconds to show the main image, jumps around while loading, or freezes for half a second after every click. It can also fail its *owners* if search engines can't understand it (and so don't show it in results). This last chapter covers both: **web performance** (the critical rendering path, render-blocking resources and Google's Core Web Vitals, measured on real pages in **headless** Chrome — a real browser run by a script, with no visible window) and **SEO basics** (the HTML that helps search engines). It then turns the whole book into interview and revision material: 47 interview questions with answers linked back to the chapters, one-page cheat sheets, and a glossary of more than 200 terms.

What you'll learn

#	Topic	You will be able to…
14.1	The critical rendering path	say what must happen before the first pixel and which resources are "critical"
14.2	Render-blocking resources	explain why CSS and synchronous scripts delay rendering, and fix it (measured: FCP 848 → 40 ms)
14.3	Core Web Vitals	define LCP, INP and CLS, their thresholds, and measure them with `PerformanceObserver`
14.4	Fixing each metric	apply a checklist for LCP, INP and CLS, plus the other performance basics
14.5	SEO basics	write the title, meta, canonical, structured data (JSON-LD) and semantic HTML search engines need
14.6	Interview questions	answer 47 common HTML/CSS interview questions
14.7	Cheat sheets	look up selectors, specificity, box model, flexbox, grid, units and responsive patterns
14.8	Glossary	find any term from the book and the chapter that explains it

How the numbers were made. Performance only means something on a slow network, so `code/serve.mjs` builds a **fake network** without starting a server (the program that normally answers the browser's requests for files): headless Chrome requests `http://site.test/<file>`, Puppeteer (a Node.js library for controlling Chrome from a script) intercepts every request and answers from the `code/` folder after an artificial delay (for example "the stylesheet takes 800 ms"). Absolute timings are machine-dependent, but they repeat to within a few milliseconds from run to run. The metrics are read with the browser's own performance APIs (an **API** is a set of functions a program can call; `PerformanceObserver` is the one that delivers timing records), the same APIs the `web-vitals` library and **real-user monitoring** (RUM: collecting timings from actual visitors' browsers) tools use. Thresholds were checked on web.dev (Google's web-development guidance site) and SEO rules on Google Search Central (Google's official documentation for site owners) in September 2026.

14.1 The critical rendering path

What it is

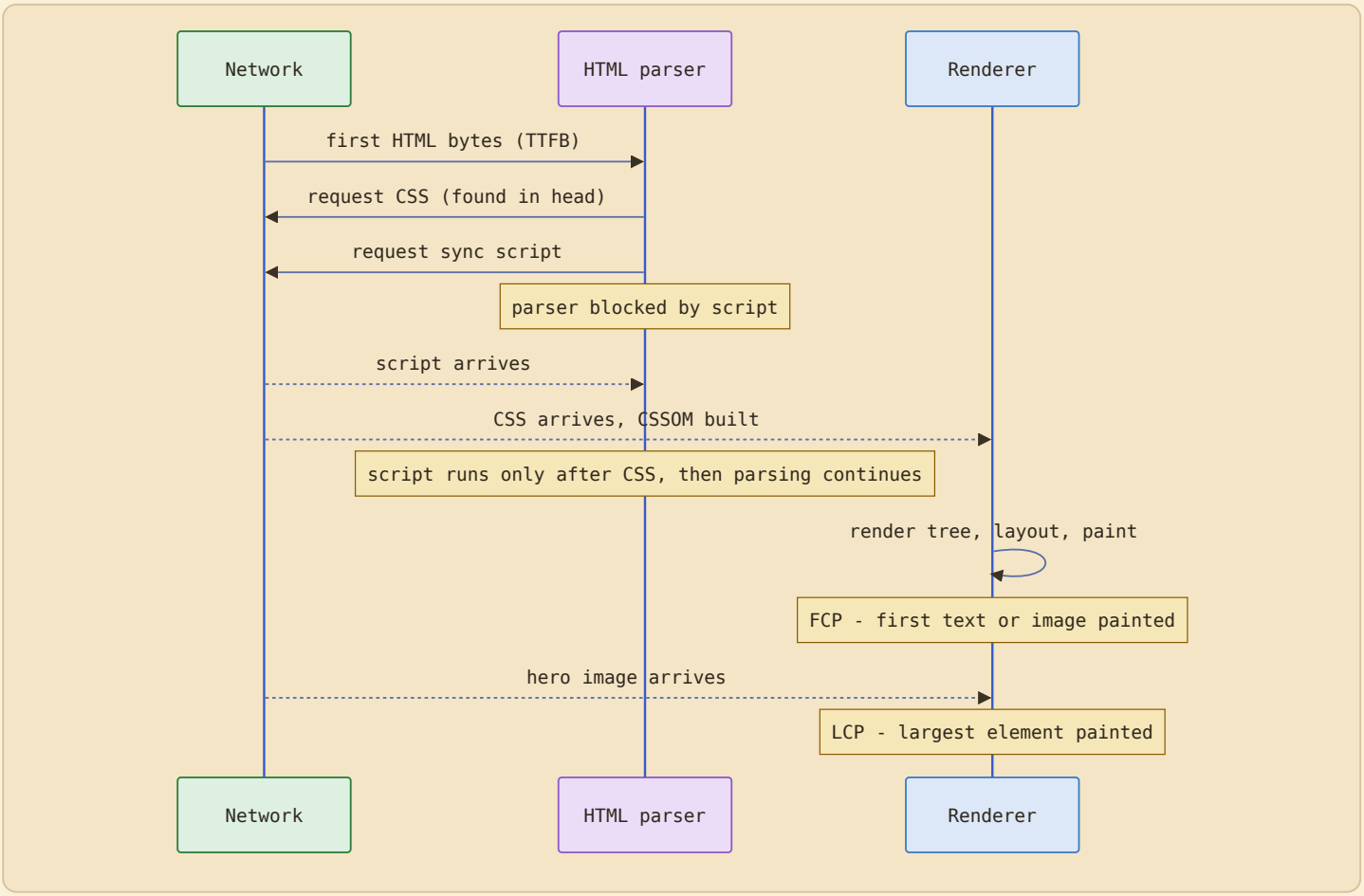
The **critical rendering path** (CRP) is the minimum sequence of work the browser must finish before it can draw the first pixels of a page: download and parse (read the text and build a data structure from it) the HTML into the **DOM** (Document Object Model — the tree of elements), download and parse the CSS into the **CSSOM** (CSS Object Model — the browser's structured copy of all the style rules), combine them into the **render tree** (only the elements that will be shown, each with its final styles), compute **layout** (each box's size and position), **paint** (draw the pixels) and **composite** (stack the painted layers into the final image). Chapter 1 walked through each step; here we care about **what makes it slow**.

A **resource** is any file the page needs (stylesheet, script, image, font). A resource is **critical** if the first render has to wait for it. By default:

- **CSS is render-blocking.** The browser won't paint until every stylesheet in `<head>` has loaded and been parsed; otherwise you'd see unstyled content that then jumps (a **FOUC**, flash of unstyled content).

- **Classic `<script>` without `async`/`defer` is parser-blocking** (`async` and `defer` are attributes that let a script download in the background; 14.2). The HTML parser (the part of the browser that reads the HTML and builds the DOM) stops, downloads the script, *waits for any pending stylesheets* (the script might read styles), runs it, and only then continues. Nothing below the script is in the DOM yet.
- **Images, fonts, `async`/`defer` scripts are not render-blocking** — but they can still delay the metrics users feel (the main image, text appearing).

How it works — the timeline and its metrics



Metric	Measures	Good (75th percentile — the value that 75% of real visits beat or match)
TTFB (Time To First Byte)	Server + network until the first HTML byte	≤ 0.8 s
FCP (First Contentful Paint)	First text/image painted	≤ 1.8 s
LCP (Largest Contentful Paint)	Main content painted (Core Web Vital, 14.3)	≤ 2.5 s
INP, **CLS**	Responsiveness and visual stability (Core Web Vitals)	≤ 200 ms, ≤ 0.1

The three levers

1. **Fewer critical resources** — inline (paste directly into the HTML instead of linking a separate file) small critical CSS, `defer` scripts, don't load what the first screen doesn't need.
2. **Fewer critical bytes** — minify (strip spaces, comments and long names) and compress (gzip/Brotli/zstd — compression formats the server applies and the browser undoes), remove unused CSS/JS.
3. **Shorter critical path length** (round trips — each time the browser must send a request and wait for the answer before it can ask for the next file) — make critical resources discoverable in the HTML, preconnect to other **origins** (an origin is a scheme + host + port, such as `https://cdn.example.com`), avoid chains (CSS that `@import`s CSS, JS that injects the main image).

Self-check

1. Why is CSS render-blocking? → Painting before styles arrive would show unstyled content and then reflow it (redo layout, moving everything).
2. Why does a synchronous script also wait for CSS? → It might read computed styles, so the browser must apply pending stylesheets first.
3. What's the difference between render-blocking and parser-blocking? → Render-blocking delays painting; parser-blocking stops the HTML parser from building the rest of the DOM.

14.2 Render-blocking resources

The experiment

The same small article page with three different `<head>`s. The stylesheet takes **800 ms** to arrive and the script **600 ms** (`code/01_render_blocking.mjs`).

`code/01_blocking.html` — the classic mistake:

```
<link rel="stylesheet" href="01_styles.css">   <!-- render-blocking -->
<script src="01_app.js"></script>              <!-- parser-blocking -->
```

`code/01_defer.html` — only the script fixed:

```
<link rel="stylesheet" href="01_styles.css">
<script src="01_app.js" defer></script>
```

`code/01_optimized.html` — critical CSS inlined, the rest loaded without blocking:

```
<style>/* only what the first screen needs */ body{…} header{…} main{…}</style>
<link rel="preload" href="01_styles.css" as="style" onload="this.onload = null; this.rel = 'stylesheet'">
<noscript><link rel="stylesheet" href="01_styles.css"></noscript>
<script src="01_app.js" defer></script>
```

Output:

```
01_blocking.html   FCP  848 ms   DOMContentLoaded  833 ms   load  834 ms   app.js ran at 833 ms
                   01_app.js blocking (done at 632 ms)  |  01_styles.css blocking (done at 833 ms)
01_defer.html      FCP  844 ms   DOMContentLoaded  833 ms   load  837 ms   app.js ran at 833 ms
                   01_app.js non-blocking (done at 631 ms)  |  01_styles.css blocking (done at 832 ms)
01_optimized.html  FCP   40 ms   DOMContentLoaded  631 ms   load  631 ms   app.js ran at 631 ms
                   01_app.js non-blocking (done at 630 ms)  |  01_styles.css non-blocking (done at 830 ms)
```

(`blocking`/`non-blocking` is Chrome's own classification, read from `PerformanceResourceTiming.renderBlockingStatus`; expect the millisecond values to differ slightly on your machine.)

What the numbers teach:

1. **Blocking**: nothing appears for ~850 ms. The script finished downloading at 632 ms but **ran at 833 ms** — it waited for the stylesheet.
2. **Only `defer`**: FCP didn't improve at all, because the stylesheet was the bottleneck (the slowest step, which limits everything behind it). Fixing the wrong resource buys nothing — always find the *critical* one first. (`defer` still helps: the parser was never stopped.)
3. **Optimized**: first paint at **40 ms**, ~20× sooner. The full stylesheet still arrived at 830 ms and applied itself; the page was usable long before. Note also that `load` fired at 631 ms without waiting for the preloaded stylesheet. (`DOMContentLoaded` is the event fired when the HTML has been fully parsed and deferred scripts have run; `load` fires when the page and its blocking resources have finished loading.)

The toolbox

Resource	Default	Better
Stylesheet	Render-blocking	Keep small; inline critical CSS (≈ first screen, a few KB); load the rest non-blocking; `media="print"` or a non-matching `media` makes a stylesheet non-blocking; split per page
Classic script in `<head>`	Parser-blocking	`defer` (runs in order after parsing, before `DOMContentLoaded`) or `async` (runs as soon as downloaded, any order — for independent scripts like analytics, i.e. visitor-tracking); `type="module"` (a JavaScript module script) is deferred by default
Web font	Text invisible until loaded (up to ~3 s)	`font-display: swap` or `optional`, `<link rel="preload" as="font" type="font/woff2" crossorigin>`, subset (keep only the characters you use), self-host (serve the font from your own site) (Chapter 11)
Main image	Discovered when the parser reaches it	In the HTML, `fetchpriority="high"`, never `loading="lazy"` above the fold (the part of the page visible without scrolling) (Chapter 4)
Other origins (CDN, fonts)	DNS + TCP + TLS on first use	`<link rel="preconnect" href="https://cdn.example">` for the 1–3 most important
`@import` in CSS	Serial chain: CSS → discover → fetch CSS	`<link>` both files, or bundle (combine into one file at build time)

Key terms — CDN, DNS, TCP, TLS: a **CDN** (content delivery network) is a set of servers around the world that keep copies of your files close to users. Before the first request to a new host, the browser must do a **DNS** lookup (turn the host name into an IP address), open a **TCP** connection (a reliable channel between the two computers) and do a **TLS** handshake (agree on encryption for HTTPS) — each costs at least one round trip, which is what `preconnect` gets out of the way early. See System Design Fundamentals for these in depth.

```
<!-- defer vs async -->
<script src="app.js" defer></script>       <!-- download in parallel, run after parsing, in order -->
<script src="analytics.js" async></script>  <!-- download in parallel, run ASAP, any order -->
<script type="module" src="main.js"></script>  <!-- deferred by default -->
```

Related HTML features: `rel="preload"` (Baseline widely available) fetches a resource you know you'll need early; `rel="modulepreload"` does the same for ES modules (JavaScript files that use `import`/`export`); `fetchpriority` (Baseline since October 2024) nudges priority; the `blocking="render"` attribute (make something *deliberately* render-blocking) is **not Baseline** (no Firefox); `<script type="speculationrules">` for prerendering (loading and rendering a page in the background before the user clicks) the next page is **Chromium-only** (Chromium is the open-source engine behind Chrome and Edge). HTTP **103 Early Hints** lets a server send `Link: rel=preload` headers (extra lines of information sent before a response's body) before the HTML is ready.

Common mistakes

- Preloading everything → preloads compete with truly critical requests. Preload only late-discovered critical resources (a font referenced from CSS, a hero image set in CSS or JS).
- Forgetting `crossorigin` on font preloads → the font is downloaded twice.
- Moving scripts to the end of `<body>` *and* adding `defer` — fine, but `defer` in `<head>` is better: the download starts earlier.
- Inlining **all** CSS: it can't be cached (kept by the browser and reused without downloading again) across pages and bloats every HTML response.

In the interview

30-second answer: "CSS in the head is render-blocking and classic scripts are parser-blocking; a script also waits for pending CSS. So I keep critical CSS small or inline it, load the rest without blocking,

> `defer` scripts (or `async` for independent ones), make the LCP image discoverable in the HTML with `fetchpriority=high`, preconnect to key origins, and avoid request chains like CSS `@import`."

- *Q: `async` vs `defer`?* A: Both download in parallel; `defer` runs after parsing, in document order; `async` runs as soon as it arrives, in any order, possibly mid-parse.
- *Q: Where should `<script>` go?* A: In `<head>` with `defer` (or `type="module"`), so the download starts early without blocking the parser.

Self-check

1. Does `defer` fix a slow page whose stylesheet is the bottleneck? → No — measured: FCP 848 → 844 ms.
2. What does `onload="this.rel='stylesheet'"` on a preload do? → Turns the preloaded CSS into an applied stylesheet once it has loaded, without ever blocking render.
3. Which attribute makes `<link rel="stylesheet">` non-blocking? → A `media` that doesn't match (e.g. `print`); it's still downloaded, at low priority.

14.3 Core Web Vitals: LCP, INP, CLS

What it is

Core Web Vitals are three metrics chosen by Google to capture the user experience of loading, interactivity and visual stability. They are measured on **real visits** ("field data", as opposed to "lab data" from test runs), and a page passes when the **75th percentile** of visits (mobile and desktop separately) is "good".

Metric	Question it answers	Good	Needs improvement	Poor
LCP — Largest Contentful Paint	When did the main content appear?	≤ 2.5 s	2.5–4.0 s	> 4.0 s
INP — Interaction to Next Paint	How quickly does the page respond to clicks, taps and key presses?	≤ 200 ms	200–500 ms	> 500 ms
CLS — Cumulative Layout Shift	How much does content jump around unexpectedly?	≤ 0.1	0.1–0.25	> 0.25

INP **replaced First Input Delay (FID)** as a Core Web Vital in **March 2024**. FID only measured the *delay before the first* interaction's handler started; INP measures the *whole* time until the next frame is painted, for (almost) *all* interactions.

How each is computed

LCP

1. Candidates are: `<img>`, `<image>` inside SVG (Scalable Vector Graphics, an image format made of shapes), `<video>` (poster — the still image shown before playback — or first frame), elements with a CSS `background-image: url()`, and block-level elements containing text.
2. Each time a larger candidate is painted, the browser emits a new `largest-contentful-paint` entry. The **last** entry before the user first interacts (click, tap, key press, scroll) is the LCP.
3. Its time breaks into four parts: **TTFB** → **resource load delay** (until the image request starts) → **resource load duration** → **element render delay**.

INP

1. For each discrete **interaction** (a click/tap = pointerdown + pointerup + click; a key press = keydown + keyup — these are the browser's event names; hover and scroll don't count), the **latency** (time from the input to the visible response) = **input delay** (the main thread — the single line of work that runs JavaScript, style and layout — is busy before **handlers**, the functions you attached to the event, can run) + **processing time** (your handlers) + **presentation delay** (style, layout, paint until the next frame).
2. The page's INP is the **worst** interaction latency — or, on pages with many interactions, a high percentile (one outlier — an unusually extreme value — is ignored per 50 interactions).
3. The Event Timing API reports it, rounded to 8 ms; entries shorter than `durationThreshold` (default 104 ms, minimum 16 ms) are not reported.

CLS

1. Every time a *visible* element changes position between frames without the user causing it, the browser records a **layout shift** with a score = **impact fraction** (share of the viewport affected) × **distance fraction** (how far it moved, relative to the viewport).
2. Shifts within 500 ms of user input are excluded (`hadRecentInput`) — expected movement.
3. Shifts are grouped into **session windows** (gaps < 1 s, window ≤ 5 s); CLS = the **largest window's sum**.

The experiment

Two versions of a product-guide page on a simulated phone (412×800) and slow network (CSS 300 ms, JS 800 ms, hero image 1800 ms). `code/02_bad.html` makes the classic mistakes; `code/02_fixed.html` fixes them:

	02_bad	02_fixed
Script	Sync in `<head>`	`defer`
Hero image	Injected by JS after `DOMContentLoaded`, `loading="lazy"`, no `width` / `height`	In the HTML, `width="800" height="533"`, `fetchpriority="high"`
Promo banner (appears after 1 s)	Inserted above the content into an empty `<div>`	Inserted into a slot with `min-height` reserved
Click handler	300 ms of synchronous (runs start to finish without pausing) work, then updates the UI	Updates the UI first, yields (pauses its own work so the browser can paint and handle other input), then does the 300 ms in 50 ms chunks

The key part of the fix for INP:

```
const yieldToMain = () => globalThis.scheduler?.yield ? scheduler.yield() : new Promise(r => setTimeout(r, 0));

button.addEventListener('click', async () => {
  status.textContent = 'Saved ✓';           // 1. visible feedback first
  await yieldToMain();                        // 2. let the browser paint
  for (let i = 0; i < 6; i++) {               // 3. the same work, in small chunks
    busyFor(50);
    await yieldToMain();
  }
});
```

`code/02_vitals.mjs` installs three `PerformanceObserver`s before any page script runs, loads each page, waits, clicks the button like a user, and computes the metrics exactly as defined above:

```
new PerformanceObserver(l => l.getEntries().forEach(e => v.lcp.push({ t: e.startTime, el: e.element?.tagName })))
  .observe({ type: 'largest-contentful-paint', buffered: true });
new PerformanceObserver(l => l.getEntries().forEach(e => v.shifts.push({ t: e.startTime, value: e.value, input:
e.hadRecentInput })))
  .observe({ type: 'layout-shift', buffered: true });
new PerformanceObserver(l => l.getEntries().forEach(e => e.interactionId && v.events.push({ name: e.name, id:
e.interactionId, d: e.duration })))
  .observe({ type: 'event', durationThreshold: 16, buffered: true });
// CLS = largest session window; INP = worst interaction (max duration per interactionId)
```

Output:

```
02_bad.html
  LCP 2648 ms  (element <img>, candidates: <p> @844, <img> @2648)  -> needs improvement
  CLS 0.195  (shifts: 0.069 @1833ms, 0.126 @2638ms)  -> needs improvement
  INP 304 ms  (slow events in the click: pointerdown, pointerup, click)  -> needs improvement
02_fixed.html
  LCP 1844 ms  (element <img>, candidates: <p> @340, <img> @1844)  -> good
  CLS 0.000  (shifts: none)  -> good
  INP < 16 ms  (slow events in the click: none)  -> good
```

(Timings are machine-dependent and vary by a few ms between runs; the fixed page's INP is sometimes reported as 16–24 ms.)

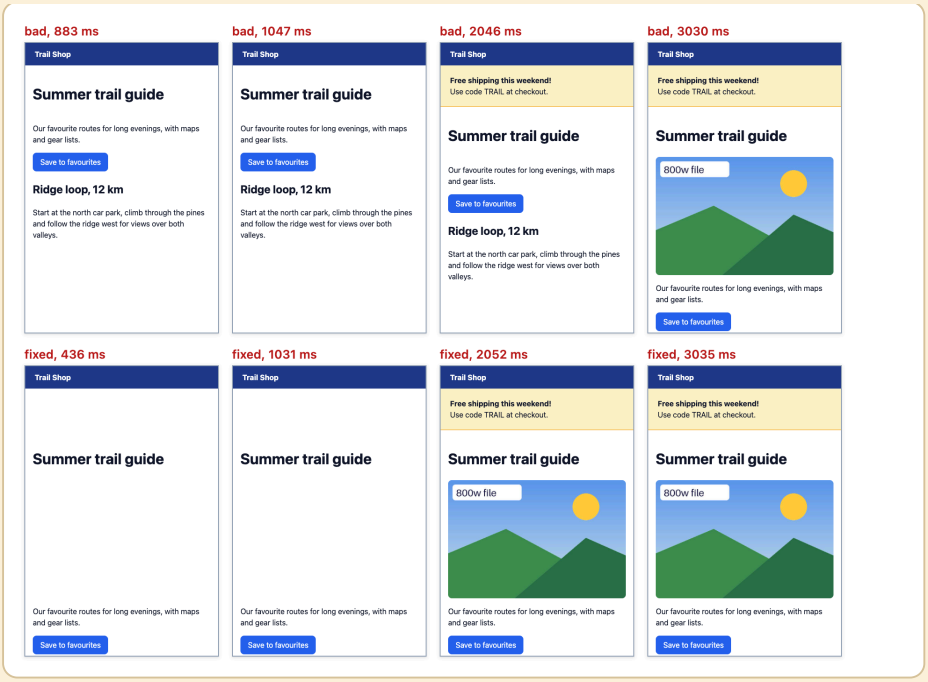


Figure 14.1 — `code/make_figures.mjs`: screenshots requested at ~0.4, 1, 2 and 3 s; labels show when Chrome delivered each frame (the bad page can't be captured before its first paint at ~880 ms).

Reading the results:

- **LCP 2648 → 1844 ms.** In the bad page the image request couldn't start until the blocking script (800 ms) had run and injected it: a **resource load delay** of ~850 ms before an 1800 ms download. In the fixed page the preload scanner (the browser's look-ahead parser) finds the `<img>` in the HTML at once, so LCP ≈ the download time. The remaining 1.8 s is **resource load duration** — the next fix is a smaller image (`srcset` — a list of image sizes for the browser to choose from — and AVIF/WebP, modern compressed image formats — Chapter 4).
- **CLS 0.195 → 0.** Two shifts 0.8 s apart fell in one session window (0.069 + 0.126). Reserving space — `width`/`height` attributes give the image an `aspect-ratio`, `min-height` holds the banner's slot — removes both.
- **INP 304 ms → under 16 ms.** All three events of the click (pointerdown, pointerup, click) belong to one interaction; the 300 ms loop delayed the next paint for all of them. After the fix the feedback paints immediately; the same total work still runs, but in chunks between frames.

> **Two measurement gotchas.** (1) An early version of the fixed page still shifted by 15.7 px because its `height` attribute said 500 for an 800×**533** image — wrong dimensions are almost as bad as none. (2) Headless Chrome reported no Event Timing entries for a button outside its real (smaller) window, so `code/serve.mjs` launches it with `--window-size=1200,1400`. Lab tooling (tests run in a controlled setup rather than on real visitors) has quirks — cross-check with field data.

Field data vs lab data

	Field (real users, "RUM")	Lab (synthetic)
Sources	Chrome UX Report (CrUX, 28-day rolling, public), Search Console's (Google's free dashboard for site owners) Core Web Vitals report, your own RUM with the `web-vitals` library	Lighthouse (an auditing tool built into Chrome DevTools), PageSpeed Insights' lab section, WebPageTest, DevTools Performance panel, scripts like this chapter's
Strength	What users actually experience; what Google uses	Repeatable, debuggable, before release
INP	Measured	Needs a scripted interaction; Lighthouse's navigation mode reports **Total Blocking Time (TBT)** as a proxy (a stand-in measure)

Browser support of the APIs (web-features, September 2026): the LCP and Event Timing APIs are **Baseline newly available since December 2025** (Safari 26.2 added them). The **Layout Instability API** behind CLS is still

Chromium-only, so CLS field data comes from Chrome users. The metrics apply to every browser's users; only measurement coverage differs.

In the interview

> **30-second answer:** "Core Web Vitals are LCP for loading — good is ≤ 2.5 s; INP for responsiveness — ≤ 200 ms; and CLS for visual stability — ≤ 0.1, all at the 75th percentile of real visits. INP replaced FID in March 2024. I measure field data with CrUX or the web-vitals library and debug in the lab with Lighthouse and DevTools. Typical fixes: make the LCP image discoverable and high-priority, break up long tasks and yield to the main thread, and reserve space for images, ads and late content."

- *Q: Why can't Lighthouse show INP on a page load?* A: INP needs real interactions; lab tools use TBT as a proxy or require a scripted user flow.
- *Q: Is a layout shift after a click counted?* A: Not if it happens within 500 ms of the input (`hadRecentInput`).
- *Q: Does a big font-swap count as CLS?* A: Yes, if text reflows and moves other content; match fallback metrics (`size-adjust`) or use `font-display: optional`.

Self-check

1. Thresholds for "good"? → LCP ≤ 2.5 s, INP ≤ 200 ms, CLS ≤ 0.1, at the 75th percentile.
2. When does the browser stop reporting new LCP candidates? → At the first user input (click, tap, key press, scroll).
3. Why did the fixed page's image not count as a shift when it loaded? → Its box was already reserved by `width`/`height`, so nothing moved.

14.4 Fixing each metric (and the other basics)

LCP checklist

1. **Find the LCP element** (DevTools Performance panel or `largest-contentful-paint` entries) and its four sub-parts.
2. **TTFB**: caching (CDN, HTTP cache headers — instructions telling browsers and CDNs how long they may reuse a response), avoid redirects (responses that send the browser to another URL, costing an extra round trip), server-side rendering (the server builds the full HTML) or static generation (HTML built once at deploy time) for content pages.
3. **Load delay**: put the image in the HTML (not CSS/JS); `fetchpriority="high"`; never lazy-load it; preload if it must be referenced from CSS.
4. **Load duration**: right-sized responsive images (`srcset`/`sizes`), modern formats (AVIF/WebP), a CDN.
5. **Render delay**: remove render-blocking CSS/JS (14.2); avoid hiding content until JS runs (A/B testing snippets — scripts that show different versions to different visitors — and client-only rendering, where JavaScript builds the page in the browser).

INP checklist

1. **Input delay**: break up **long tasks** (> 50 ms of uninterrupted main-thread work) during load; defer third-party scripts (code from other companies, such as ads, chat widgets and analytics); don't run heavy work on a timer that collides with input.
2. **Processing**: do only the visible update in the handler; `await scheduler.yield()` (or `setTimeout`) before the rest; debounce input handlers (wait until the user pauses before reacting, instead of on every keystroke); move heavy work to a Web Worker (a background thread that runs JavaScript off the main thread).
3. **Presentation**: keep the DOM small; avoid layout thrashing (reading layout after writing styles in a loop); use `content-visibility: auto` for long off-screen sections (Baseline since September 2025); prefer CSS animations on `transform`/`opacity` (Chapter 12).

CLS checklist

1. Always set **`width` and `height`** on images and videos (or CSS `aspect-ratio`) — correct values.
2. **Reserve space** for ads, embeds (content from another site, such as a video player or map), banners and late-loading components (`min-height`, skeletons — grey placeholder shapes shown while content loads), or

put them in an overlay (`position: fixed`) that doesn't push content.
3. Never insert content **above** existing content unless the user asked for it.
4. Fonts: `font-display: optional`, or tune the fallback with `size-adjust`/`ascent-override` so the swap doesn't reflow text.
5. Animate with `transform`, not `top`/`height` (transforms don't cause layout shifts).
6. Keep pages eligible for the **back/forward cache** (bfcache) — instant, shift-free back navigations (avoid `unload` handlers and `Cache-Control: no-store` — a header forbidding any caching — where possible).

Other performance basics

Area	Practice
Transfer	Compression (Brotli/gzip/zstd), HTTP/2 or HTTP/3 (newer protocol versions that send many files over one connection), long cache lifetimes for fingerprinted files (a content hash in the file name, so a changed file gets a new name) (`app.3f9a1c.css`, `Cache-Control: max-age=31536000, immutable`) — Chapter 1
Images	Formats and responsive images (Chapter 4); `loading="lazy"` below the fold
CSS	Remove unused CSS; avoid huge selectors on huge DOMs; `contain`/`content-visibility` for independent regions
JavaScript	Ship less; code-split (break the bundle into pieces loaded on demand) per route (per page/URL of the app); `defer`; audit third parties
Budgets	Set performance budgets (e.g. LCP < 2.5 s on a mid-range phone, JS < 200 KB compressed) and check them in CI (continuous integration — automated checks that run on every code change) with Lighthouse CI

Self-check

1. Name the four LCP sub-parts. → TTFB, resource load delay, resource load duration, element render delay.
2. What's a long task? → Main-thread work over 50 ms; it delays input handling.
3. Two ways to stop an image causing CLS? → `width`/`height` attributes or CSS `aspect-ratio` (with correct values).

14.5 SEO basics

What it is

Search engine optimization (SEO) is making pages easy for search engines to **crawl** (fetch), **render** (Google runs JavaScript in an up-to-date Chromium), **index** (store and understand) and **rank**, and making the result attractive to click. Content quality and relevance matter most; HTML can't make a thin page (one with little useful content) rank. But broken HTML *can* keep a good page out of the index. This section covers the HTML side, following Google Search Central.

The essentials, in the order a crawler meets them

Item	HTML	Notes
Crawl permission	`robots.txt` (a plain-text file of rules for crawlers — the bots search engines use to fetch pages) at the site root	Blocks **crawling**, not indexing. A blocked URL can still be indexed from links; to keep a page out, allow crawling and use `noindex`
Index permission	`<meta name="robots" content="noindex">` (or `X-Robots-Tag` header)	The classic accident: a staging (private test copy of the site) `noindex` shipped to production (the live site)
Title	`<title>Specific page title | Brand</title>`	Shown as the result's headline (Google may rewrite it). Unique per page; no hard length limit, but long titles are truncated by width — roughly 50–60 characters display fully
Description	`<meta name="description" content="…">`	Often used for the snippet (the short text under a search result's title); not a ranking signal (a factor that affects position in results); unique, specific; truncated around 150–160 characters
Canonical	`<link rel="canonical" href="https://…">`	Tells Google the preferred URL among duplicates (tracking parameters — like `?utm_source=` — `http`/`https`, `www`). A strong **hint**, not a directive (an order the search engine must obey). Absolute URLs recommended
Language	`<html lang="en">`; `<link rel="alternate" hreflang="de" href="…">`	`hreflang` links translated versions (include a self-reference and optionally `x-default`)
Mobile	`<meta name="viewport" content="width=device-width, initial-scale=1">`	Google indexes the **mobile** version of pages (mobile-first indexing: the phone version is what gets judged)
Keywords meta	`<meta name="keywords">`	**Ignored** by Google Search
Social previews	`og:title`, `og:image`, … (Open Graph)	For link previews on social apps; not a Google ranking signal
Sitemap	`sitemap.xml` (an XML — tag-based data format — list of the site's URLs), listed in robots.txt / Search Console	Helps discovery of new or deep pages

Semantic HTML is SEO

Search engines read structure the same way assistive technology (software and devices disabled people use to access computers, such as screen readers) does (Chapter 3):

- **Headings** describe the outline; one clear `<h1>` is a helpful convention (Google says multiple `<h1>`s are not a problem), and levels shouldn't skip.
- **Links** must be real `<a href>` elements (Google follows `href`, not `onclick` — a JavaScript click handler) with **descriptive text** — "Ridge loop route and map", not "click here".
- **Images**: `alt` text describes the image for image search and accessibility; use descriptive file names.
- **Landmarks** (page regions like `<header>`, `<nav>`, `<main>` that tools can jump between) **and** `<article>`/`<time>` help identify the main content and dates.
- Content should be in the HTML or reliably rendered; content that needs a click to load may never be seen.

Structured data (JSON-LD)

Structured data describes the page's content with the **schema.org** vocabulary (a shared list of types like Article, Product and Person, and their properties) in a machine-readable form, so search engines can show **rich results** (breadcrumbs — the "Home › Guides › Summer" trail — product price and rating, recipe cards, article dates). Google supports JSON-LD (JSON for Linked Data; **JSON** is a text format of `{ "key": value }` objects and lists), Microdata and RDFa (two older formats written as HTML attributes), and **recommends JSON-LD**: a `<script type="application/ld+json">` block, separate from the visible markup.

```
<script type="application/ld+json">
{
  "@context": "https://schema.org",
  "@type": "Article",
  "headline": "Summer Trail Guide: 5 Evening Hikes Near Denver",
  "image": ["https://www.trailshop.example/img/hills-1200.jpg"],
  "datePublished": "2026-06-01T08:00:00-06:00",
  "dateModified": "2026-06-15T09:30:00-06:00",
  "author": [{ "@type": "Person", "name": "Maya Ortiz", "url": "https://www.trailshop.example/team/maya" }]
}
</script>
```

Rules: the data must describe what's **visible** on the page; include each rich-result type's **required** properties (Article has none required; the ones above are Google's recommended set); valid JSON (no trailing commas — a comma after the last item — and double quotes). Structured data makes a page *eligible* for rich results; it **doesn't guarantee** them and isn't a direct ranking boost. Google also restricts or retires rich-result types over time (FAQ and HowTo results were cut back in 2023), so check the current Search Gallery. Test with Google's **Rich Results Test** and the **Schema Markup Validator**.

The experiment

`code/03_seo_audit.mjs` is a small on-page checker built on jsdom (a JavaScript imitation of a browser DOM that runs in Node.js; it parses HTML like a crawler's first pass). It audits `code/03_seo_bad.html` — `div` soup (markup made of meaningless `<div>`s instead of semantic elements) with an accidental `noindex`, a relative canonical, two `<h1>`s, no `alt`, "Click here" and a JSON-LD block with a trailing comma — and `code/03_seo_good.html`:

Output:

```
03_seo_bad.html
  FAIL  doctype present (standards mode)
  FAIL  <html lang> = null
  FAIL  <title> 4 chars: "Home"
  FAIL  meta description missing
  FAIL  meta viewport (mobile-friendly)
  FAIL  robots: "noindex"
  FAIL  canonical: /guide (absolute URL recommended)
  NOTE  meta keywords present — ignored by Google, harmless but useless
  FAIL  one <h1> (found 2) — a clarity convention, not a Google rule
  FAIL  heading levels 1→1→4 (skips a level)
  FAIL  <main> landmark
  FAIL  images without alt: 1
  FAIL  vague link text: "Click here"
  FAIL  JSON-LD does not parse: Expected double-quoted property name in JSON at position 95 (line 2 column 95)
  score: 0/13

03_seo_good.html
  PASS  doctype present (standards mode)
  PASS  <html lang> = "en"
  PASS  <title> 60 chars: "Summer Trail Guide: 5 Evening Hikes Near Denver | Trail Shop"
  PASS  meta description 136 chars
  PASS  meta viewport (mobile-friendly)
  PASS  robots: "(none = index, follow)"
  PASS  canonical: https://www.trailshop.example/guides/summer-trails (absolute URL recommended)
  ...   (headings, <main>, alt text and link text all pass)
  PASS  JSON-LD Article: recommended properties present
  PASS  JSON-LD BreadcrumbList: recommended properties present
  score: 14/14
```

The single most damaging line in the bad page is `noindex`: every other fix is wasted while it's there. The trailing comma in JSON-LD is the second classic: the whole block is ignored silently.

Performance and SEO

Core Web Vitals are part of Google's page-experience signals (ranking factors about how pleasant a page is to use): good vitals help, but relevance dominates — a fast page with poor content won't outrank a slower, better answer. Treat performance as a user-experience goal, and SEO as a side benefit.

Common mistakes

- Client-only rendering of content pages with empty initial HTML (rendering may be delayed; other crawlers and social previews may not run JS at all). Prefer server rendering or static generation.
- Blocking CSS/JS in `robots.txt` → Google can't render the page properly.
- Same `<title>`/description on every page; canonical pointing to the home page from every page.
- Links built with `<span onclick>` or `<a>` without `href`.

In the interview

30-second answer: "For SEO the HTML needs a unique, descriptive title and meta description, a canonical URL, `lang` and viewport, no accidental `noindex`, semantic structure — headings, landmarks, real links with descriptive text, alt text — and structured data in JSON-LD using schema.org to become eligible for rich results. Content should be server-rendered or reliably renderable, and good Core Web Vitals help as part of page experience."

Self-check

1. Does `robots.txt` `Disallow` keep a page out of Google's index? → No, it prevents crawling; use `noindex` on a crawlable page.
2. Is `rel="canonical"` a command? → No, a strong hint Google usually follows.
3. Which structured-data format does Google recommend? → JSON-LD.

14.6 Interview questions

Forty-seven questions that come up in front-end (the part of a website that runs in the browser) interviews, with short answers and links to the full explanation. Cover the answer, say yours out loud, then check.

HTML and accessibility

1. **What does `<!doctype html>` do?** → Puts the browser in standards mode (without it: quirks mode with legacy — old, kept-for-compatibility — layout rules). §2.2
2. **Why is `<meta charset="utf-8">` first in `<head>`?** → It must be within the first 1024 bytes so the parser decodes the rest correctly. §2.3
3. **Block vs inline elements?** → Block boxes start on a new line and take the available width; inline boxes flow within text. It's CSS `display`, not the element "type". §7.4
4. **`<section>` vs `<article>` vs `<div>`?** → `<article>` is self-contained (could be syndicated — republished elsewhere on its own, like in a feed); `<section>` is a thematic group with a heading; `<div>` has no meaning. §3.2
5. **What are landmarks and why do they matter?** → `header`, `nav`, `main`, `aside`, `footer` (and `form`/`section` with a name) map to ARIA (Accessible Rich Internet Applications — attributes that tell assistive technology what an element is and its state) landmark roles that screen-reader users jump between. §3.2
6. **What is the first rule of ARIA?** → Don't use ARIA if a native element does the job; no ARIA is better than bad ARIA. §3.6
7. **How do you write good alt text?** → Describe the image's purpose in context; `alt=""` for decorative images; never omit `alt`. §3.5
8. **`<button>` vs `<a>`?** → Links navigate (have `href`); buttons perform actions. Both are keyboard-accessible natively; a `div` with `onclick` is not. §3.7
9. **How do you label a form field accessibly?** → `<label for="id">` (or wrap the input); group radios with `fieldset`/`legend`; link errors with `aria-describedby`. §3.8, §5.7
10. **GET vs POST forms?** → GET puts fields in the URL (bookmarkable, for searches); POST sends them in the body — the data part of the HTTP request, after the headers — (for changes, sensitive data). §5.2
11. **How do `srcset` and `sizes` work?** → `srcset` lists candidate files with widths; `sizes` tells the browser how wide the image will display; it picks the smallest adequate file for the screen's density. §4.3
12. **When do you use `<picture>`?** → Art direction (different crops per breakpoint — a screen width at which the layout changes) or format fallbacks (AVIF → WebP → JPEG). §4.3
13. **What does WCAG AA require for text contrast?** → 4.5:1 for normal text, 3:1 for large text and UI components. §3.9

CSS fundamentals

14. **How is specificity calculated?** → (IDs, classes/attributes/pseudo-classes, types/pseudo-elements), compared left to right; inline styles beat selectors; `!important` changes the origin ordering. §6.4
15. **What decides which declaration wins?** → Origin and importance, then context, then inline style, then cascade layer, then specificity, then (for `@scope` rules) scope proximity, then order of appearance. §6.5
16. **Pseudo-class vs pseudo-element?** → `:hover` selects an element in a state; `::before` creates/selects a part of an element. §6.3
17. **Which properties inherit?** → Mostly text properties (colour, font, line-height); box properties (margin, border, width) don't. `inherit`, `initial`, `unset`, `revert` control it. §6.6
18. **`em` vs `rem`?** → `em` is relative to the element's own font size (the parent's for `font-size`); `rem` to the root's. Use `rem` for predictable scaling. §6.7
19. **What are custom properties and how do they differ from Sass variables?** → Live, inherited values that can change per element and at runtime; Sass variables disappear at compile time. §6.9, §13.12
20. **`:is()` vs `:where()`?** → Same matching; `:is()` takes its most specific argument's specificity, `:where()` is zero. §13.2
21. **What does `:has()` enable?** → Selecting an element by what it contains or what follows it — a parent/previous-sibling selector. §13.3
22. **How do cascade layers help?** → They order groups of rules before specificity is considered, ending specificity wars with third-party CSS. §13.4

Box model and layout

23. **Explain the box model and `box-sizing`.** → Content + padding + border + margin; `content-box` sizes the content only, `border-box` includes padding and border. §7.1, §7.2
24. **What is margin collapsing?** → Adjacent vertical margins of blocks in normal flow combine into the larger one; not in flex/grid containers or across a BFC (block formatting context, see Q27) boundary. §7.3
25. **Explain `position` values.** → `static` (normal), `relative` (offset, keeps its space), `absolute` (out of flow, relative to the nearest positioned ancestor), `fixed` (viewport), `sticky` (relative until a threshold, then sticks within its container). §7.6
26. **Why doesn't my `z-index` work?** → It only applies to positioned (or flex/grid) items, and it's compared within the same stacking context; a parent with `opacity < 1`, `transform` etc. creates a new one. §7.8
27. **What is a block formatting context (BFC)?** → An independent layout region: contains floats, prevents margin collapse with children; created by `display: flow-root`, flex/grid items, `overflow` other than `visible`/`clip`, etc. §7.5
28. **What does `flex: 1` mean?** → `flex: 1 1 0%`: grow equally from a zero basis, allowed to shrink. §8.4
29. **Why does a flex item with long text overflow instead of shrinking?** → Its automatic minimum size is `min-width: auto` (its content); set `min-width: 0` (making it a scroll container with `overflow: hidden`/`auto` also works; `overflow: clip` does not). §8.4
30. **How do you center something horizontally and vertically?** → `display: grid; place-items: center` (or flex with `justify-content` and `align-items: center`). §8.7
31. **Flexbox or Grid?** → Flexbox for one-dimensional content-driven layout (rows of items); Grid for two-dimensional, layout-driven structure (rows and columns together). §9.8
32. **`auto-fill` vs `auto-fit`?** → Both create as many tracks as fit; `auto-fit` collapses empty ones so items stretch. §9.3
33. **What is subgrid for?** → Letting nested items align to the parent grid's tracks (e.g. card titles aligned across cards). §9.7

Responsive, visual and motion

34. **What does the viewport meta tag do?** → Tells mobile browsers to lay out at the device width instead of a ~980px virtual desktop. §10.2
35. **Mobile-first — what and why?** → Base styles for small screens, `min-width` queries add complexity; simpler CSS and faster phones. §10.4
36. **Media queries vs container queries?** → Media queries respond to the viewport; container queries to the size of a component's container — reusable components. §10.6
37. **How does `clamp()` give fluid typography?** → `clamp(min, preferred, max)` with a viewport-relative preferred value (plus `rem` so zoom still works). §10.7
38. **How do you make a web font load without invisible text?** → `font-display: swap` (or `optional`), preload the WOFF2, subset it. §11.2

39. **Which properties are cheap to animate and why?** → `transform` and `opacity`: the compositor can animate them without layout or paint. §12.7
40. **How do you respect users who get motion sickness?** → Wrap non-essential motion in `@media (prefers-reduced-motion: no-preference)` or reduce it under `reduce`. §12.9

Architecture, performance and SEO

41. **BEM vs CSS Modules vs Tailwind?** → Convention (flat unique class names) vs build-time renamed classes vs generated single-purpose utilities; all fight the global namespace. §13.11
42. **How do you implement dark mode with a user toggle?** → Semantic tokens with `light-dark()`, `color-scheme: light dark`, a `data-theme` override that pins `color-scheme`, saved in localStorage and applied before first paint. §13.14
43. **What happens when you type a URL and press Enter?** → DNS, TCP/TLS, HTTP request/response, parsing, DOM/CSSOM, render tree, layout, paint, composite. §1.6
44. **What are render-blocking resources?** → Stylesheets (block paint) and synchronous scripts (block parsing); fix with critical CSS, `defer`/`async`. §14.2
45. **What are the Core Web Vitals and their thresholds?** → LCP ≤ 2.5 s, INP ≤ 200 ms, CLS ≤ 0.1 at p75 (the 75th percentile). §14.3
46. **How do you prevent layout shift from images?** → `width`/`height` attributes (correct ratio) or `aspect-ratio`. §14.4
47. **What HTML matters for SEO?** → Title, description, canonical, lang, viewport, no stray `noindex`, headings, real links, alt, JSON-LD. §14.5

14.7 Cheat sheets

Selectors

Selector	Matches	Specificity
`*`	every element	(0,0,0)
`p`	type	(0,0,1)
`.card`	class	(0,1,0)
`#main`	id	(1,0,0)
`[type="email"]`, `[href^="https"]`, `[href$=".pdf"]`, `[class*="icon"]`, `[lang|="en"]`	attribute: equals / starts / ends / contains / language prefix	(0,1,0)
`A B`	descendant	sum
`A > B`	child	sum
`A + B`	next sibling	sum
`A ~ B`	any later sibling	sum
`:hover :focus :focus-visible :focus-within :active :visited`	user action state	(0,1,0) each
`:checked :disabled :required :invalid :user-invalid :placeholder-shown`	form state	(0,1,0)
`:first-child :last-child :nth-child(2n+1) :nth-of-type(3) :only-child :empty`	structure	(0,1,0)
`:nth-child(2 of .item)`	nth among matches	(0,1,0) + argument
`:not(X) :is(X) :has(X)`	logic	most specific argument
`:where(X)`	logic	(0,0,0)
`::before ::after ::marker ::placeholder ::selection ::first-line ::backdrop`	parts	(0,0,1)
`&` (nesting)	parent rule's selector	as `:is(parent)`

Details: §6.2, §6.3, §13.1–13.3.

Specificity and the cascade

- Score = (A = IDs, B = classes + attributes + pseudo-classes, C = types + pseudo-elements); compare A, then B, then C. `*` and combinators add nothing.
- The cascade, strongest criterion first (UA = user agent, i.e. the browser's own default styles): **origin & `!important`** (transition > `!important` UA > `!important` user > `!important` author > animation > author > user > UA) → **context** (shadow DOM) → **inline `style`** → **layers** (later wins; unlayered last; reversed for `!important`) → **specificity** → **scope proximity** (`@scope`) → **order of appearance**.
- Keywords: `inherit` (parent's value), `initial` (spec default — the value the CSS specification defines), `unset` (inherit if inherited property, else initial), `revert` (browser/user value), `revert-layer` (previous layers' value).

Worked examples: `#nav .item a` = (1,1,1); `ul li:nth-child(2)` = (0,1,2); `a:not(.btn, #x)` = (1,0,1); `:where(#x) .y` = (0,1,0). §6.4

Box model

```
margin  →  border  →  padding  →  content
content-box:  width = content          total = width + padding + border (+ margin)
border-box:   width = content + padding + border
```

- Vertical margins between blocks in normal flow **collapse** (largest wins); not in flex/grid, not through padding/border, not inside a BFC. §7.3
- Percentage padding/margin are relative to the **containing block's inline size** — its width in horizontal text, even for top/bottom.
- `display`: `block`, `inline`, `inline-block`, `flex`, `inline-flex`, `grid`, `flow-root` (new BFC), `contents` (box removed, children stay — careful with accessibility), `none` (removed from layout and accessibility tree).
- Positioning: `relative` → offset; `absolute` → nearest positioned ancestor; `fixed` → viewport (or a transformed ancestor); `sticky` → scroll threshold. `inset: 0` = `top/right/bottom/left: 0`.

Flexbox

Container	Values (default first)
`display`	`flex` | `inline-flex`
`flex-direction`	`row` | `row-reverse` | `column` | `column-reverse`
`flex-wrap`	`nowrap` | `wrap` | `wrap-reverse`
`flex-flow`	shorthand: direction + wrap
`justify-content` (main axis)	`flex-start` / `normal` | `center` | `flex-end` | `space-between` | `space-around` | `space-evenly`
`align-items` (cross axis, per line)	`stretch` / `normal` | `flex-start` | `center` | `flex-end` | `baseline`
`align-content` (lines, when wrapping)	`normal` | `flex-start` | `center` | `space-between` | `stretch` …
`gap`, `row-gap`, `column-gap`	spacing between items

Item	Values
`flex-grow`	`0` — share of free space
`flex-shrink`	`1` — share of overflow to absorb (weighted by basis)
`flex-basis`	`auto` — starting size before grow/shrink
`flex`	`initial` = `0 1 auto`; `auto` = `1 1 auto`; `none` = `0 0 auto`; `1` = `1 1 0%`
`align-self`	overrides `align-items` for one item
`order`	`0` — visual order only (keyboard/reading order unchanged)
`min-width: 0`	lets items shrink below their content (`overflow: clip` doesn't; `hidden` / `auto` do)

Chapter 8

Grid

Container	Example
`grid-template-columns` / `-rows`	`200px 1fr 2fr`, `repeat(3, 1fr)`, `repeat(auto-fit, minmax(16rem, 1fr))`
`grid-template-areas`	`"head head" "side main" "foot foot"`
`grid-auto-rows` / `-columns`	size of implicit tracks: `minmax(100px, auto)`
`grid-auto-flow`	`row` | `column` | `dense`
`gap`	`1rem 2rem` (row column)
`justify-items` / `align-items` / `place-items`	align items inside their cells
`justify-content` / `align-content`	align the whole grid inside the container

Item	Example
`grid-column` / `grid-row`	`1 / 3`, `2 / span 2`, `1 / -1` (full width)
`grid-area`	`main` (named area) or `1 / 1 / 3 / 3`
`justify-self` / `align-self` / `place-self`	per-item alignment
`grid-template-columns: subgrid`	reuse the parent's tracks

Track sizes: `px`/`rem`, `%`, `fr` (share of free space), `auto`, `min-content`, `max-content`, `fit-content(300px)`, `minmax(min, max)`. Chapter 9

Units

Unit	Relative to	Typical use
`px`	CSS pixel (1/96 in; scales with zoom and device pixel ratio)	borders, small fixed details
`rem`	root font size (16px default)	font sizes, spacing
`em`	own font size (parent's for `font-size`)	padding that scales with text
`%`	containing block (varies by property)	fluid widths
`ch` / `ex` / `lh`	width of "0" / x-height / line height	`max-width: 65ch` for readable lines
`vw` / `vh`	1% of viewport width / height	full-screen sections
`svh` / `lvh` / `dvh`	small / large / dynamic viewport height (mobile toolbars)	`min-height: 100dvh`
`vi` / `vb`	viewport inline / block size	writing-mode-aware
`cqi` / `cqw` / `cqb`	container query inline / width / block	component-relative type
`fr`	free space in a grid	grid tracks
`deg` `turn` `ms` `s`	angles, time	transforms, transitions

§6.7, §10.2

Responsive patterns

```
<meta name="viewport" content="width=device-width, initial-scale=1">
```

```
/* mobile-first breakpoints (range syntax) */
.layout { display: grid; gap: 1rem; }
@media (width >= 48rem) { .layout { grid-template-columns: 16rem 1fr; } }

/* intrinsic grid, no breakpoints */
.cards { display: grid; grid-template-columns: repeat(auto-fit, minmax(min(16rem, 100%), 1fr)); gap: 1rem; }

/* component responds to its container */
.card-wrap { container-type: inline-size; }
@container (inline-size >= 30rem) { .card { display: flex; } }

/* fluid type and space */
h1 { font-size: clamp(1.75rem, 1.2rem + 2.5vw, 3rem); }

/* user preferences */
@media (prefers-color-scheme: dark) { /* or use light-dark() tokens */ }
@media (prefers-reduced-motion: reduce) { * { animation: none; } }

/* images */
img { max-width: 100%; height: auto; }
```

```
<img src="a-800.jpg" srcset="a-400.jpg 400w, a-800.jpg 800w, a-1600.jpg 1600w"
     sizes="(width >= 48rem) 50vw, 100vw" width="800" height="533" alt="…">
```

Chapter 10

HTML head starter

```
<!doctype html>
<html lang="en">
<head>
  <meta charset="utf-8">
  <meta name="viewport" content="width=device-width, initial-scale=1">
  <title>Page title | Site</title>
  <meta name="description" content="What this page offers, in one or two sentences.">
  <link rel="canonical" href="https://example.com/page">
  <meta name="color-scheme" content="light dark">
  <link rel="preload" href="/fonts/inter.woff2" as="font" type="font/woff2" crossorigin>
  <link rel="stylesheet" href="/css/site.3f9a1c.css">
  <script src="/js/app.8b2e0d.js" defer></script>
  <link rel="icon" href="/favicon.svg" type="image/svg+xml">
</head>
```

14.8 Glossary

Alphabetical. Each term links to the chapter (and usually the section) that explains it.

Term	Meaning	Where
`!important`	Flag that moves a declaration to a higher-priority "important" origin level	§6.5
`@container`	Container query rule	§10.6
`@font-face`	Declares a web font and its files	§11.1
`@import`	Loads another stylesheet (can add a layer)	§6.1
`@keyframes`	Named animation timeline	§12.5
`@layer`	Cascade layer	§13.4
`@media`	Media query rule	§10.3
`@property`	Registers a typed custom property	§13.14
`@scope`	Limits rules to a subtree, optionally with a lower bound	§13.5
`@starting-style`	Style to transition *from* when an element first renders	§12.8
`@supports`	Feature query	§6.1
Absolute positioning	Out of flow, placed relative to the nearest positioned ancestor	§7.6
Absolute URL	URL with scheme and host	§1.2
Accessibility tree	The browser's tree of roles, names and states for assistive tech	§3.4
Accessible name	What a screen reader announces for an element	§3.4
`alt` text	Text alternative for an image	§3.5
ARIA	Attributes that add roles/states/properties for assistive technology	§3.6
`aria-describedby`	Links an element to its description (e.g. an error)	§5.7
`aspect-ratio`	Preferred width/height ratio of a box; `<img width height>` sets it automatically	§4.1
`async` (script)	Download in parallel, run as soon as available	§14.2
Attribute	Name/value pair on an element	§2.1
Attribute selector	`[name="value"]` and variants	§6.2
`autocomplete`	Hints the browser what data a field wants	§5.6
AVIF	Modern image format with strong compression	§4.2
Baseline	web-features status: works in all major browsers (newly / widely available)	§13 intro
BEM	Block__Element--Modifier naming convention	§13.7
bfcache	Back/forward cache: instant back navigations	§14.4
Block axis	Direction in which blocks/lines stack	§13.6
Block formatting context (BFC)	Independent layout region for block boxes	§7.5
Block-level element	Starts on a new line and fills the width	§7.4
`border-box`	`box-sizing` in which width includes padding and border	§7.2
Box model	Content, padding, border, margin	§7.1
Breakpoint	Width at which a layout changes	§10.3
Canonical URL	Preferred URL among duplicates (`rel="canonical"`)	§14.5
Canvas	Scriptable bitmap drawing surface	§4.9
Cascade	Algorithm that picks the winning declaration	§6.5
Cascade layer	Named, ordered group of rules	§13.4

Term	Meaning	Where
Character reference	`&`, `©` — escapes in HTML	§2.9
`clamp()`	Value clamped between a min and max	§10.7
CLS	Cumulative Layout Shift — visual stability metric	§14.3
`color-mix()`	Mixes two colours in a colour space	§11.4
`color-scheme`	Declares supported light/dark schemes	§10.5
Combinator	Relationship between selectors (space, `>`, `+`, `~`)	§6.2
Composite	Final step: combining layers into the screen image	§1.7
Compositor-only properties	`transform`, `opacity` — animate without layout/paint	§12.7
Computed value	Value after cascade and inheritance, before layout	§6.10
Constraint Validation API	JS API behind built-in form validation	§5.5
Container query	Styles based on a container's size	§10.6
Containing block	Box that percentages and positions are resolved against	§7.6
`content-visibility`	Skips rendering off-screen content	§14.4
Core Web Vitals	LCP, INP and CLS	§14.3
Critical CSS	CSS needed for the first screen, often inlined	§14.2
Critical rendering path	Work required before the first render	§14.1
CrUX	Chrome UX Report — public field data	§14.3
CSS Modules	Build-time locally scoped class names	§13.9
CSS-in-JS	Styles authored in JavaScript	§13.10
CSSOM	CSS Object Model — parsed stylesheets	§1.7
`cubic-bezier()`	Custom easing curve	§12.2
Custom property	`--name` value that inherits and can change at runtime	§6.9
`defer` (script)	Download in parallel, run in order after parsing	§14.2
Design token	Named design value (colour, space) stored as a variable	§13.14
Device pixel ratio	Physical pixels per CSS pixel	§4.3
DevTools	Browser developer tools	§1.8
`<dialog>`	Native modal/non-modal dialog element (focus handling built in)	§3.7
`display`	Outer/inner display type of a box	§7.4
DNS	Resolves domain names to IP addresses	§1.3
Doctype	`<!doctype html>` — standards mode	§2.2
DOM	Document Object Model — tree of the parsed HTML	§1.7
Donut scope	`@scope (root) to (limit)` excluding an inner region	§13.5
`dvh`	Dynamic viewport height unit	§10.2
Easing	Speed curve of an animation (timing function)	§12.2
Element	Start tag, content, end tag	§2.1
`em`	Unit relative to font size	§6.7
Event Timing API	Reports input event latencies (INP)	§14.3
FCP	First Contentful Paint	§14.1

Term	Meaning	Where
`fetchpriority`	Hint to raise/lower a resource's priority	§4.4
Field data	Metrics from real users	§14.3
`<fieldset>` / `<legend>`	Group of related controls and its caption	§5.4
`<figure>` / `<figcaption>`	Self-contained content with caption	§4.5
`animation-fill-mode`	Whether keyframe styles apply before the animation starts / after it ends	§12.5
`flex` shorthand	grow, shrink, basis	§8.4
`flex-basis`	Item's starting main size	§8.3
Flex container / item	Element with `display: flex` / its children	§8.1
Float	Legacy layout: box shifted left/right, text wraps	§7.9
FLIP	First-Last-Invert-Play animation technique	§12.7
Focus indicator	Visible outline showing keyboard focus	§3.7
`:focus-visible`	Focus that should show an indicator (keyboard)	§3.7
`font-display`	How text renders while a web font loads	§11.2
FOUC	Flash of unstyled content	§14.1
`fr`	Fraction of free space in a grid	§9.2
`gap`	Space between flex/grid items	§8.6
Gradient	`linear-/radial-/conic-gradient()` images	§11.5
Grid area	Named or line-bounded region	§9.4
Grid line / track / cell	Dividing lines / rows-columns / single slot	§9.1
`:has()`	Relational ("parent") pseudo-class	§13.3
Heading levels	`h1`–`h6` outline	§3.3
`hreflang`	Links language versions of a page	§14.5
HSL / OKLCH	Colour notations (hue-based, perceptual)	§6.8
HTTP	Request/response protocol of the web	§1.5
HTTP/2, HTTP/3	Multiplexed HTTP versions (HTTP/3 over QUIC)	§1.4
`<iframe>`	Embedded browsing context	§4.7
Implicit grid	Tracks created automatically for extra items	§9.5
INP	Interaction to Next Paint — responsiveness metric	§14.3
Inheritance	Children taking a parent's computed value	§6.6
Inline axis	Direction text runs within a line	§13.6
Inline element	Flows within a line of text	§7.4
Input types	`email`, `number`, `date`, …	§5.3
Intrinsic layout	Responsive without breakpoints (`auto-fit`, `min()`)	§10.9
`:is()`	Matches any selector in a list (max specificity)	§13.2
JSON-LD	JSON format for structured data	§14.5
Keyboard trap	Focus that can't leave a component (WCAG failure)	§3.7
Lab data	Metrics from synthetic tests	§14.3
Landmark	Region role (`banner`, `navigation`, `main`, …)	§3.2

Term	Meaning	Where
Layout (reflow)	Computing box sizes and positions	§1.7
Layout shift	Unexpected movement of visible content	§14.3
Lazy loading	`loading="lazy"`: fetch when near the viewport	§4.4
LCP	Largest Contentful Paint — loading metric	§14.3
`light-dark()`	Picks a light or dark value by colour scheme	§13.14
Lighthouse	Lab auditing tool (performance, a11y, SEO)	§14.3
Link (`<a>`)	Hyperlink; `href`, `target`, `rel`	§2.8
Lists	`ul`, `ol`, `dl`	§2.7
Logical properties	Flow-relative properties (`margin-inline-start`)	§13.6
Long task	Main-thread task over 50 ms	§14.4
Margin collapsing	Adjacent vertical margins combine	§7.3
Media query	Condition on the device/viewport/preferences	§10.3
Meta description	Page summary for search snippets	§14.5
`min-content` / `max-content`	Intrinsic sizes	§9.2
`minmax()`	Track size range	§9.2
Mobile-first	Base styles for small screens, enhance upward	§10.4
Nesting (CSS)	Rules inside rules; `&`	§13.1
`noindex`	Robots directive keeping a page out of the index	§14.5
normalize.css	Stylesheet making browser defaults consistent	§13.13
Normal flow	Default layout of block and inline boxes	§7.5
`object-fit`	How replaced content fills its box	§11.9
Open Graph	`og:` meta tags for social previews	§14.5
`order`	Visual order of flex/grid items	§8.2
Origin (cascade)	User-agent, user or author stylesheet	§6.5
Outline	Line outside the border that doesn't take space	§11.6
`overflow`	What happens to content that doesn't fit	§7.7
Paint	Filling in pixels for boxes	§1.7
`PerformanceObserver`	API to receive performance entries	§14.3
`<picture>`	Art direction and format fallbacks	§4.3
`place-items`	Shorthand for `align-items` + `justify-items`	§9.6
Positioned element	`position` other than `static`	§7.6
PostCSS	Plugin-based CSS transformer	§13.12
`preconnect`	Early connection to another origin	§14.2
`prefers-color-scheme`	Media feature for light/dark preference	§10.5
`prefers-reduced-motion`	Media feature for reduced motion	§12.9
`preload`	Fetch a known-critical resource early	§14.2
Preload scanner	Look-ahead parser that finds resources early	§14.3
Preprocessor	Language compiled to CSS (Sass, Less)	§13.12

Term	Meaning	Where
Pseudo-class	`:state` selector (`:hover`, `:checked`)	§6.3
Pseudo-element	Selects or creates a part of an element (`::before`, `::marker`)	§6.3
Quirks mode	Legacy rendering mode without a doctype	§2.2
`rel`	Link relationship (`noopener`, `canonical`, `preload`)	§2.8
Relative colour syntax	Derive a colour from another's channels: `oklch(from var(--c) l c calc(h + 180))`	§11.4
`rem`	Unit relative to root font size	§6.7
Render tree	Styled boxes that will be displayed	§1.7
Render-blocking	Delays the first paint (stylesheets)	§14.2
`repeat()`	Repeats track definitions	§9.3
Reset (CSS)	Rules neutralising browser defaults	§13.13
Responsive design	Layouts that adapt to any screen and setting	§10.1
`revert-layer`	Roll back to earlier layers' value	§13.4
Rich results	Enhanced search results from structured data	§14.5
`robots.txt`	Crawl rules for bots	§14.5
Role	ARIA/semantic purpose of an element	§3.6
Sass	The most used CSS preprocessor	§13.12
`sandbox`	Restricts an iframe's capabilities	§4.7
schema.org	Vocabulary for structured data	§14.5
Screen reader	Software that speaks or brailles the UI	§3.1
`<search>`	Landmark element for search/filter forms (widely available since April 2026)	§3.2
Scroll-driven animation	Animation progress linked to scroll	§12.11
Selector	Pattern choosing elements to style	§6.2
Semantic HTML	Elements chosen for meaning	§3.1
SEO	Search engine optimization	§14.5
Session window (CLS)	Group of layout shifts < 1 s apart, ≤ 5 s	§14.3
Shorthand property	Sets several longhands at once (`margin`, `flex`, `inset`)	§7.1
Sitemap	XML list of URLs for crawlers	§14.5
`sizes`	Displayed image width per condition	§4.3
Specificity	Weight of a selector (A,B,C)	§6.4
`srcset`	Candidate image files	§4.3
Stacking context	Group painted together along z	§7.8
Sticky positioning	Relative until a scroll threshold, then fixed within its container	§7.6
Structured data	Machine-readable page description	§14.5
Style query	`@container style(--x: y)` — query a container's custom property (Baseline since May 2026)	§10.6
Subgrid	Nested grid reusing parent tracks	§9.7
SVG	Vector image format written in XML	§4.8
Table semantics	`caption`, `thead`, `th scope`	§5.1

Term	Meaning	Where
Tailwind CSS	Utility-first CSS framework	§13.8
TBT	Total Blocking Time — lab proxy for responsiveness	§14.3
Text-level semantics	`em`, `strong`, `code`, `abbr`, `time`, …	§2.6
TLS	Encryption layer of HTTPS	§1.4
`<track>`	Captions/subtitles for media	§4.6
Transform	Moves/rotates/scales the painted box	§12.3
Transition	Animates a property change	§12.1
TTFB	Time To First Byte	§14.1
URL	Address: scheme, host, path, query, fragment	§1.2
`:user-invalid`	Invalid after the user interacted	§5.5
User-agent stylesheet	The browser's default CSS	§13.13
Utility-first CSS	Composing single-purpose classes	§13.8
Validation (HTML)	Checking markup against the spec	§2.10
View transition	Animated morph between DOM states	§12.10
Viewport	Visible area of the page	§10.2
Viewport meta tag	Sets the layout viewport on mobile	§10.2
WAAPI	Web Animations API	§12.6
WCAG	Web Content Accessibility Guidelines (2.2)	§3.9
Web font	Font downloaded with the page (`@font-face`, WOFF2)	§11.1
WebP	Image format, typically smaller than JPEG/PNG at similar quality	§4.2
`:where()`	Like `:is()` with zero specificity	§13.2
`will-change`	Hint that a property will animate	§12.7
Writing mode	Horizontal or vertical text flow	§13.6
`z-index`	Stacking order of positioned boxes	§7.8

Chapter 14 TL;DR

- **Critical rendering path**: HTML → DOM, CSS → CSSOM, render tree, layout, paint, composite. CSS is **render-blocking**; classic scripts are **parser-blocking** and also wait for pending CSS.
- **Render-blocking fixes** (measured, CSS 800 ms / JS 600 ms): all-blocking FCP 848 ms; `defer` only → 844 ms (CSS was the bottleneck); inline critical CSS + non-blocking stylesheet + `defer` → **40 ms**.
- **Core Web Vitals** (p75 of real visits): **LCP ≤ 2.5 s**, **INP ≤ 200 ms** (replaced FID in March 2024), **CLS ≤ 0.1**. LCP = last largest paint before input; INP = worst interaction's input delay + processing + presentation; CLS = largest session window of unexpected shifts.
- **Measured**: bad page LCP 2648 ms / CLS 0.195 / INP 304 ms → fixed page 1844 ms / 0 / < 16 ms, by putting the hero `<img>` in the HTML with correct `width`/`height` and `fetchpriority="high"`, reserving the banner's space, deferring the script, and yielding in the click handler.
- **Fix lists**: LCP (TTFB, discovery, size, render delay); INP (no long tasks, yield, small DOM); CLS (dimensions, reserved space, nothing inserted above, font metrics).
- **SEO**: title, meta description, canonical (hint, absolute), `lang`, viewport, no accidental `noindex`, `robots.txt` blocks crawling not indexing, semantic headings/links/alt, **JSON-LD** schema.org structured data (eligibility, not a guarantee), keywords meta ignored.
- **Revision**: 47 interview questions (§14.6), cheat sheets for selectors, specificity, box model, flexbox, grid, units and responsive patterns (§14.7), and a 212-term glossary (§14.8).

That's the end of the HTML & CSS Handbook. Next steps: the JavaScript Handbook for behaviour and the DOM (start with its Chapter 1, or jump to Chapter 13 — The DOM and Events if you already know the language), then the React Handbook for component-based UIs. Back to the front page of this book.